ওয়েব ডেভেলপমেন্ট প্রজেক্ট · কোড ২৮৫৬৬

IMS ক্লাস কিট
শিক্ষকের সম্পূর্ণ ওভারভিউ

১০ পর্ব · ৩৭ ক্লাস · ৫টি গভীরতা ডেক · ১৭টি PDF · ৭৪৮ স্লাইড

প্রতিটি সিদ্ধান্তের প্রমাণ আসল কোডবেসে

একটি উদাহরণ, শূন্য থেকে প্রোডাকশন পর্যন্ত

কোর্স

এক বাক্যে এই কোর্স

একটি বাস্তব স্কুল-ম্যানেজমেন্ট সিস্টেমকে দুভাগে ভেঙে, প্রথম লাইন থেকে ডিপ্লয় পর্যন্ত

"সফটওয়্যার লেখা মানে সিদ্ধান্ত নেওয়া — আর প্রতিটি সিদ্ধান্তের কারণ বলতে পারা।"

১

চিন্তার ক্রম

সমস্যা → প্রয়োজন → মডেল → পরীক্ষা। কোডবারবার না লিখে আগে **কী বানাচ্ছি** ঠিক করা।

২

সিদ্ধান্তের ভাষা

"এভাবে করেছি" নয় — "এটা বেছেছি, কারণ এটা বাদ দিলাম"। বিকল্প আর বাতিলের কারণ।

৩

প্রমাণ

প্রতিটি দাবির পাশে ফাইল, লাইন, টেস্টের নাম বা মাপা সংখ্যা — অনুমান নয়।

৪

সততা

যা করা হয়নি, যা ভাঙা — সেটাও পাঠের অংশ। ফাঁক লুকালে শেখা অসম্পূর্ণ।

শিক্ষকের নোট: প্রথম দিনেই এই চারটি কথা বলুন। পুরো ৩৭ ক্লাসে এই চারটি কাঠামো বারবার ফিরে আসবে — শিক্ষার্থী যেন জানে কোথায় দাঁড়িয়ে আছে।

উদাহরণ

IMS আসলে কী

একটি মাল্টি-টেন্যান্ট স্কুল ম্যানেজমেন্ট সিস্টেম — Django ৫ + PostgreSQL

যা আছে	সংখ্যা	অর্থ
Django অ্যাপ	**১১**	এক অ্যাপ = এক দায়িত্ব
মডেল / কলাম	**৩১ / ৩০১**	ডেটাবেসের আকৃতি
URL রুট (অ্যাপ)	**৮৫**	+ `config`-এ ১২
টেমপ্লেট	**৭৫**	পর্দা
টেস্ট	**১০৩**	ভরসার ভিত্তি
মাইগ্রেশন	**১২**	স্কিমার ইতিহাস
কোড / টেস্ট	**৫,৪৮০ / ১,৩৫৩** লাইন	আনুপাত ৪ : ১

মাল্টি-টেন্যান্ট মানে: একই ডেটাবেসে অনেক স্কুল; প্রতিটি সারিতে `institution` — আর প্রতিটি কুয়েরিতে সেটি ফিল্টার করা **বাধ্যতামূলক**। এই একটি নিয়ম থেকেই নিরাপত্তার পুরো অধ্যায় (D3) দাঁড়ায়।

কেন এই প্রজেক্ট: ছোট খেলনা নয়, কিন্তু এক কোডবেসেই auth, permission, রিপোর্ট, বিলিং, ব্যাকআপ — কোর্সের ৩৭টি বিষয়ের প্রতিটির আসল ঠিকানা এখানে আছে।

৫,৪৮০ লাইন কোড — একটা সেমিস্টারে পড়ার মাপে, কারণ সবই একই ছাঁচে লেখা।

শিক্ষকের নোট: সংখ্যাগুলো মুখস্থ করানোর জন্য নয় — দেখানোর জন্য যে "৩১টি মডেল" মানে ৩১টি সিদ্ধান্ত। প্রথম ক্লাসেই বলুন: আমরা এই ৩১টির পেছনের কারণগুলো পড়ব।

মানচিত্র

১০টি পর্ব — কোন পর্বে কোন প্রশ্নের উত্তর

সিলেবাসের ক্রম অটুট; প্রতিটি পর্ব শেষ হলে একটি PDF

পর্ব	নাম	ক্লাস	স্লাইড	যে প্রশ্নের উত্তর মেলে
১	কনসেপ্ট	৪	৫৮	আমরা আসলে কী বানাচ্ছি, আর কার জন্য?
২	প্ল্যান	৬	১০৬	কাজ ভাগ করব কীভাবে, কে কী করবে?
৩	ডিজাইন	৫	৮৩	ডেটা কীভাবে সাজাব, পর্দা কেমন হবে?
৪	কোডিং	৮	১৩০	আসল কোড — মডেল থেকে রিপোর্ট
৫	টেস্টিং	৪	৭০	কীভাবে জানব এটি ঠিক আছে?
৬	ডিপ্লয়	৩	৫৬	নিজের ল্যাপটপ থেকে সার্ভারে
৭	মেইনটেন্যান্স	২	৩৪	ছয় মাস পরেও চলবে তো?
৮	ডকুমেন্ট	৩	৪৪	নতুন মানুষ কীভাবে বুঝবে?
৯	ভবিষ্যৎ	১	১৬	এর পরে কী, কী বাদ দিলাম?
১০	উপস্থাপন	১	১৬	কীভাবে ১০ মিনিটে দেখাব?

মোট: ৩৭ ক্লাস · ৬১৩ স্লাইড (মূল কিট) + ৫টি গভীরতা ডেক ১২০ স্লাইড + সূচিপত্র ১৫ = **৭৪৮**

শিক্ষকের নোট: এই টেবিলটি কোর্সের প্রথম দিনেই দেখান, আর প্রতিটি পর্বের শুরুতে আঙুল দিয়ে দেখান "আমরা এখানে"। শিক্ষার্থী দিক হারায় না।

গভীরতা

পাঁচটি ডিপ ডেক — যেখানে থামলে ক্ষতি হয়

মূল কিটে ধারণা, ডিপ ডেকে মাপা প্রমাণ ও আসল সংখ্যা

ডেক	বিষয়	স্লাইড	ভেতরে যা আছে
D1	ডেটাবেস ডিজাইন	৩০	৩০১ ফিল্ড · ৭০ FK · ১৩ unique_together · CASCADE vs SET_NULL vs PROTECT
D2	স্ট্যাক ও সংযোগ	২৮	রিকোয়েস্ট থেকে DB পর্যন্ত প্রতিটি স্তর · middleware · ক্যাশ
D3	নিরাপত্তা	২২	৩৬ `PermissionDenied` · ০ raw SQL · ০ `|safe`
D4	লাইফসাইকেল ও অপস	২০	ব্যাকআপ · রিলিজ · ক্ষমতা হিসাব · যা লিখে কখনো চালানো হয়নি
D5	ডিজাইন সিস্টেম	২০	১৫ টোকেন · ৫৯ ক্লাস · কনট্রাস্ট মাপা · ওয়াইট-লেবেল

কোথায় ব্যবহার করবেন: মূল ক্লাসের সময় সংক্ষেপে ধারণা, তারপর ডিপ ডেকের নির্দিষ্ট স্লাইড ধরে গভীরে। ডিপ ডেক স্বাধীনভাবে পড়া যায় — সময় কম থাকলে সেখান থেকেই শুরু করা যায়।

শিক্ষকের নোট: ডিপ ডেকগুলো "বাড়তি" নয় — এগুলো কোর্সের সবচেয়ে কঠিন প্রশ্নগুলোর উত্তর যেখানে করা হয়েছে। বহিরাগত প্রশিক্ষকের ক্লাসে বা সেমিনারে এগুলোই সবচেয়ে বেশি কাজে লাগে।

ছন্দ

প্রতিটি ক্লাসের ছয় ধাপ

শিক্ষক যা বলবেন, শিক্ষার্থী যা করবে — একই ছাঁচ প্রতিবার

১

লক্ষ্য

এই ক্লাস শেষে **একটি** প্রশ্নের উত্তর দিতে পারবে। একটির বেশি নয়।

২

পটভূমি

আসল সমস্যা — কোন স্কুল, কোন ব্যবহারকারী, কী আটকে ছিল।

৩

সিদ্ধান্ত

দুই-তিনটি বিকল্প, কেন একটি বেছে নেওয়া হলো, বাকিগুলো কেন বাদ।

৪

প্রমাণ

ফাইল, লাইন, টেস্টের নাম, মাপা সংখ্যা — বোর্ডে দেখানো হয়।

৫

ফাঁদ

"সবাই এখানে আটকায়" — আসল বাগ থেকে নেওয়া, বানানো নয়।

৬

নিজে করা

১৫–২০ মিনিটের কাজ, নিজের মেশিনে, একই প্রজেক্টে।

সময়ের হিসাব: ধাপ ১–৫ মিলিয়ে ২৫–৩০ মিনিট, ধাপ ৬-এ ১৫–২০ মিনিট। একটি ক্লাস = ৪৫–৫০ মিনিট; দুই ক্লাস একসাথে করলেও ছন্দ একই থাকে।

শিক্ষকের নোট: ছয় ধাপ লিখে বোর্ডের পাশে রেখে দিন। শিক্ষার্থী একসময় নিজেই বুঝবে "এখন সিদ্ধান্তের ধাপ চলছে" — এটাই কোর্সের সবচেয়ে বড় অভ্যাসগত লাভ।

চিহ্ন

স্লাইডের রঙ কী বলে

একই চিহ্ন ৭৪৮ স্লাইডে — চিনতে শেখা একবারই

সবুজ — যা কাজ করে / মূল নিয়ম
যেটা পরীক্ষিত, যেটা এই সিস্টেমে সত্যি কার্যকর।

হলুদ — সতর্কতা / যেখানে ভাঙে
ঝুঁকি, সীমা, ভবিষ্যতে সমস্যা হওয়ার সম্ভাবনা।

লাল — সৎ ফাঁক
যা করা হয়নি, যা নেই, যা ভুল মাপা হয়েছে।

অংশ	কী দেয়
`.tnote` (নিচে হলুদ)	শিক্ষকের বলার কথা — ছয় ধাপের কোন ধাপে কী বলবেন
উপরের ব্যাজ	স্লাইডের বিভাগ (কোড, নিরাপত্তা, ফর্ম...)
নিচের সাব-টাইটেল	স্লাইডের মূল দাবি, এক লাইনে

কাগজে: `.tnote` স্লাইডের একদম নিচে হলুদ ফিতায় থাকে — প্রিন্ট করলে সেটিও আসে। শিক্ষকের কপি তাই সম্পূর্ণ, আলাদা স্ক্রিপ্ট লাগে না।

শিক্ষকের নোট: প্রথম ক্লাসে এই রঙের নিয়মটি একবার বলুন। তারপর শিক্ষার্থীরা নিজেরাই লাল বাক্স লেখায় লম্বা হয় — কারণ ওটাই যেখানে বাস্তবতা আর প্রত্যাশার ফাঁক।

প্রমাণ

সংখ্যাগুলো কোথা থেকে

প্রতিটি সংখ্যা গোনা বা মাপা — অনুমান নয়

দাবি	কীভাবে মাপা
৩১ মডেল · ৩০১ ফিল্ড	মডেল ফাইলগুলো পার্স করে ফিল্ড ও সম্পর্ক গোনা
৮৫ রুট · ৭৫ টেমপ্লেট	`urls.py` ও `*.html` ফাইল গোনা
১০৩ টেস্ট	টেস্ট রানার — সবগুলো সত্যিই চলে
১,০০৭ `ims-*` ব্যবহার	`grep -rho` — ৫৪টি আলাদা ক্লাস
কনট্রাস্ট ৩.৭৫	WCAG সূত্রে হাতে হিসাব, দুই রঙের অনুপাত
৮ KB / ৫৩৪ KB	ফাইলের আসল আকার (বাইট)

শিক্ষার্থীদের জন্য নিয়ম: নিজের রিপোর্টে যেকোনো সংখ্যা লিখলে — সেটি কীভাবে গোনা হলো, সেটাও লিখতে হবে। না পারলে সংখ্যাটি বাদ।

যা মাপা হয়নি: লোড টাইম, কতজন ব্যবহারকারী একসাথে, ডেটাবেস কুয়েরির সময় — কোনো লোড টেস্ট চালানো হয়নি, তাই এগুলো কোর্সে দাবি করা হয় না।

শিক্ষকের নোট: এই স্লাইডটি সবচেয়ে বেশি কাজে দেয় শিক্ষার্থীদের নিজের রিপোর্ট লেখার সময়। "আমার অ্যাপ দ্রুত" লেখা যায় না; "প্রথম পেজ ৪২০ ms, Lighthouse" লেখা যায় — শুধু যদি মাপা হয়।

প্রস্তুতি

ক্লাসের আগে যা চাই

শিক্ষক ও শিক্ষার্থী — দুই দিকের চেকলিস্ট

শিক্ষকের জন্য	বিস্তারিত
PDF প্রিন্ট	A4 ল্যান্ডস্কেপ, প্রতি পৃষ্ঠা ১ স্লাইড
ল্যাপটপে প্রজেক্ট	`bash run.sh` — চালু আছে কি না আগেই পরীক্ষা
আইকন ফোল্ডার	`ims/` এডিটর খোলা, `ims.css` ও `base.html` ট্যাব খোলা
ডেমো লগইন	`@ims.local` · `DemoPass!2026`

ভুলে যাবেন না: প্রথম ব্যবহারে `bash run.sh --reset` — নইলে পুরনো ডেটা দেখে সংখ্যা মিলবে না।

শিক্ষার্থীর জন্য	বিস্তারিত
Python ৩.১১+ ও PostgreSQL	একবার ইনস্টল, তারপর `run.sh` সব করে
নিজের ফোল্ডার	`~/Projects/"Web Development Project"/ims`
টার্মিনাল + এডিটর	একই সময়ে দুটো — টার্মিনালে চালাবে, এডিটরে দেখবে
পুরো কোর্স জুড়ে	একই প্রজেক্ট কপি — প্রতিটি ক্লাসে সেটাই বদলাবে

নিয়ম: কোড কপি-পেস্ট নয় — ক্লাসে যা টাইপ হচ্ছে, নিজে টাইপ করবে। হাতের অভ্যাসটাই মূল।

শিক্ষকের নোট: প্রথম দিনের শুরুতে ৫ মিনিট হাতে-কলমে সবাইকে `run.sh` চালানো দেখান। ইনস্টলজনিত সমস্যা ক্লাসের ভেতরেই সমাধান করলে ভালো, বাইরে ফেলে রাখলে দ্বিতীয় সপ্তাহে কেউ প্রজেক্ট চালু করতে পারে না।

ফাইল

১৭টি PDF — কোনটা কে পাবে

মূল কিট ১১টি + গভীরতা ৫টি + সূচিপত্র ১টি

ফাইল	স্লাইড
part1_concept.pdf	৫৮
part2_plan.pdf	১০৬
part3_design.pdf	৮৩
part4_code.pdf	১৩০
part5_test.pdf	৭০
part6_deploy.pdf	৫৬

ফাইল	স্লাইড
part7_maintain.pdf	৩৪
part8_docs.pdf	৪৪
part9_future.pdf	১৬
part10_demo.pdf	১৬
deep/d1…d5	১২০
00_overview.pdf	১৬

মোট স্লাইড: ৭৪৮ · **ক্লাস:** ৩৭ · **আকার:** সব মিলিয়ে ~২৩ MB · **ক্রমানুসারে** পড়াই সবচেয়ে ভালো

শিক্ষকের নোট: ফাইলগুলো এক ফোল্ডারে রাখুন, নাম না বদলান — কোর্সের ক্রম নামেই আছে। শিক্ষার্থীদের কেবল অংশ ১–১০ দিন; ডিপ ডেক রাখুন নিজের কাছে বা রেফারেন্স হিসেবে পরে।

নিয়ম

চারটি নিয়ম — পুরো সেমিস্টার

শিক্ষার্থীদের প্রথম দিনেই বলে দেওয়া উচিত

১

আগে প্রশ্ন, পরে কোড

কোড লেখার আগে এক বাক্যে বলতে হবে **কী সমস্যা সমাধান করছি**। না পারলে কোড লেখা শুরু নয়।

২

প্রতিটি দাবির প্রমাণ

"কাজ করে" নয় — **কোন টেস্ট**, **কোন লাইন**, **কত মাপা**। প্রমাণ নেই মানে দাবি নেই।

৩

একবারে একটি পরিবর্তন

এক কমিটে একটি কাজ। না হলে ভাঙলে বোঝা যায় না **কোনটা ভেঙেছে**।

৪

না-জানা স্বীকার করা

"জানি না" বলা ভুল নয় — **ভুল জানা** বলার চেয়ে ভালো। ফাঁক লিখে রাখলে সেটাই পরের কাজ।

কেন এই চারটি: এই সিস্টেমের নিজের কোডেই চারটি দৃশ্যমান — ১০৩ টেস্ট (নিয়ম ২), ২০ কমিট (নিয়ম ৩), আর D4-এর "যা লিখে কখনো চালানো হয়নি" তালিকা (নিয়ম ৪)। নিয়ম শুধু বলা হয় না, দেখানো হয়।

শিক্ষকের নোট: নিয়ম ৪ সবচেয়ে কঠিন, কারণ শিক্ষার্থীরা ভাবে না-জানা মানে নম্বর কম। স্পষ্ট করে বলুন: কোর্সে "জানি না, যাচাই করব" — এটাই সবচেয়ে ভালো উত্তর।

ভুল

পাঁচটি ভুল যা বারবার হয়

এই সিস্টেমের আসল অভিজ্ঞতা থেকে, বানানো গল্প নয়

ভুল	কী হয়	প্রথম লক্ষণ
টেস্ট পরে লিখব	পরে আর লেখা হয় না — ১০৩ টেস্টই সীমা, তার বেশি যায়নি	ভাঙা কোড রিলিজে যায়
স্কুল ফিল্টার ভুলে যাওয়া	এক স্কুল আরেক স্কুলের ডেটা দেখে — মাল্টি-টেন্যান্টে সবচেয়ে বিপজ্জনক	টেস্টে ধরা পড়ে, প্রোডাকশনে দেরি
রঙ সরাসরি হার্ডকোড	ব্র্যান্ড রঙ বদলালে অর্ধেক বাটন পুরনো রঙে থাকে	স্কুল বলবে "লোগো বদলেছে, বাটন বদলায়নি"
ব্যাকআপ লিখে চালানো হয়নি	দরকারের দিনে স্ক্রিপ্ট কাজ করে না — কেউ জানে না	প্রথম আসল সংকটে ধরা পড়ে
ডকুমেন্টেশন বাদ	নতুন মানুষ ৫৯ ক্লাস মুখস্থ করতে পারে না	প্রশ্নের উত্তর শুধু একজনের মাথায়

আর সবচেয়ে বড় ভুল: সমস্যা না বুঝেই কোড শুরু করা। এই কোর্সের প্রথম ১০ ক্লাস (পর্ব ১–২) শুধু এই একটি ভুল এড়াতে লেখা।

শিক্ষকের নোট: প্রতিটি ভুলের পাশে বলুন কোন ক্লাসে সেটি বিস্তারিত আলোচনা হয়। শিক্ষার্থীরা তখন ক্লাসটির জন্য অপেক্ষা করে — আর মনে রাখে।

সময়

১৫ সপ্তাহে কীভাবে ভাগ করবেন

সপ্তাহে ২–৩ ক্লাস ধরে; সময় কম থাকলে ক্রম ভাঙবেন না

সপ্তাহ	পর্ব / ডেক	ক্লাস	সপ্তাহের শেষে শিক্ষার্থী যা পারে
১–২	পর্ব ১ কনসেপ্ট	৪	সমস্যা ও ব্যবহারকারী চিহ্নিত করা
৩–৫	পর্ব ২ প্ল্যান · D1	৬	কাজ ভাগ + স্কিমার প্রথম খসড়া
৬–৮	পর্ব ৩ ডিজাইন · D1	৫	মডেল ও পর্দার নকশা
৯–১১	পর্ব ৪ কোডিং · D2 · D5	৮	চলমান অ্যাপ, প্রথম পর্দা
১২	পর্ব ৫ টেস্টিং · D3	৪	নিজের লেখা টেস্ট, নিরাপত্তার ধারণা
১৩	পর্ব ৬ ডিপ্লয় · D4	৩	সার্ভারে চালু + ব্যাকআপ
১৪	পর্ব ৭–৯	৬	রক্ষণাবেক্ষণ, ডকুমেন্ট, ভবিষ্যৎ
১৫	পর্ব ১০ উপস্থাপন	১	১০ মিনিটে ডেমো

সময় কমলে কী বাদ দেবেন: পর্ব ৮–৯ ও কিছু ডিপ স্লাইড — কিন্তু **পর্ব ৫ (টেস্টিং)** কখনো নয়। বাদ দিলে শিক্ষার্থী "চলে গেছে" আর "ঠিক আছে" — এই পার্থক্যটাই শিখবে না।

শিক্ষকের নোট: এই সূচি বাধ্যতামূলক নয় — কিন্তু ক্রম বাধ্যতামূলক। পর্ব ৫-এর আগে পর্ব ৬ শুরু করলে ডিপ্লয় শেখানো যাবে, কিন্তু কেন টেস্ট দরকার তা বোঝানো যাবে না।

প্রস্তুতি

প্রথমবার পড়াতে গেলে যা জানা দরকার

শিক্ষকের প্রস্তুতির ক্রম — কোনটা আগে পড়বেন

ধাপ	কী করবেন	সময়
১. প্রজেক্ট চালান	`bash run.sh --reset` → সব পর্দা নিজে ঘুরে দেখুন	১–২ ঘণ্টা
২. স্লাইড পড়ুন	যে পর্ব পড়াবেন, সেটি প্রিন্ট করে নোট নিন	২–৩ ঘণ্টা
৩. ডিপ ডেক দেখুন	সংশ্লিষ্ট ডেকের সংখ্যাগুলো মিলিয়ে নিন	১ ঘণ্টা
৪. ফাইল খুলে রাখুন	যে ফাইল উল্লেখ আছে, সব এডিটরে ট্যাব করে রাখুন	৩০ মিনিট
৫. ভাঙার চেষ্টা করুন	ইচ্ছা করে ভুল লগইন, খালি ফর্ম, অন্য স্কুলের আইডি	৩০ মিনিট

সবচেয়ে দরকারি ধাপ ৫: স্লাইডে যে ফাঁদগুলো লেখা আছে, সেগুলো নিজে একবার ঘটিয়ে দেখুন। মুখে বলা আর পর্দায় দেখানো — এক নয়; যা নিজে ভেঙেছেন, সেটাই ক্লাসে নিশ্চিতভাবে বলা যায়।

শিক্ষকের নোট: প্রস্তুতিতে সবচেয়ে বেশি সময় লাগে ধাপ ১ — আর সেটাই সবচেয়ে বেশি ফল দেয়। পর্দা না দেখে স্লাইড পড়ালে শিক্ষার্থীর প্রথম প্রশ্নেই আটকে যাবেন।

সূচি

শিক্ষার্থীর প্রশ্ন কোথায় উত্তর পাবে

সাধারণ প্রশ্ন → কোন PDF, কোন অংশ

প্রশ্ন	কোথায়	কীভাবে
"কেন এই ডেটাবেস, অন্য রকম হলে কী হতো?"	D1 · স্লাইড ৪–১২	সম্পর্কের ধরন ও CASCADE/SET_NULL/PROTECT তুলনা
"অনুরোধ আসার পর আসলে কী ঘটে?"	D2 · স্লাইড ৩–১০	middleware থেকে ভিউ থেকে DB
"এক স্কুল আরেক স্কুলের ডেটা দেখে না কেন?"	D3 · স্লাইড ৫–১৪	৩৬ `PermissionDenied` ও ফিল্টার নিয়ম
"সার্ভার বন্ধ হলে কী হবে?"	D4 · স্লাইড ১১–১৬	ব্যাকআপ, রিস্টোর, সংকটের রুটিন
"পর্দাগুলো একই রকম দেখায় কেন?"	D5 · স্লাইড ৩–৭	১৫ টোকেন, ৫৯ ক্লাস, তিন স্তর
"নিজের প্রজেক্টে কোথা থেকে শুরু করব?"	পর্ব ১ · স্লাইড ১–১০	সমস্যা থেকে প্রয়োজন

শিক্ষকের জন্য: এই টেবিলটি ক্লাসে হাতে হাতে দিলে শিক্ষার্থীরা নিজেই ডিপ ডেকে ঢুকবে — পুরো কোর্সের সবচেয়ে সহজ লাভ।

শিক্ষকের নোট: ডিপ ডেকের স্লাইড নম্বরগুলো এই টেবিলে স্বনির্ধারিত দিকনির্দেশ — মূল কিটের স্লাইডের সাথে মিলিয়ে নিন। প্রশ্ন এলে "ডিপ ডেকের অমুক অংশ" বলে দিন; শিক্ষার্থী নিজে খুঁজে নেবে।

শেষ কথা

শুরু করার আগে একটি কথা

এই কিট একটি নির্দিষ্ট প্রজেক্টের কথা বলে — কিন্তু শেখায় সাধারণ নিয়ম

৩৭ ক্লাস পরে শিক্ষার্থী IMS মুখস্থ করবে না — সিদ্ধান্ত নেওয়ার ছাঁচ মনে রাখবে।

যা সরাসরি অন্যত্র লাগবে: মাল্টি-টেন্যান্ট ফিল্টার · পারমিশনের স্তর · টেস্টের নামকরণ · মাইগ্রেশনের শৃঙ্খলা · ব্যাকআপের অভ্যাস · ডকুমেন্টেশনের জায়গা।

যা কপি করা যাবে না: স্কুলের নিয়ম, রিপোর্টের ফরম্যাট, বিলিংয়ের হার — এগুলো এই প্রতিষ্ঠানের সিদ্ধান্ত, সর্বজনীন নয়।

প্রথমেই যা ঠিক করবেন: নিজের প্রজেক্টে একটি ফাঁক বেছে নিন, লিখে ফেলুন, তারপর কোড। লিখতে না পারলে সেটি এখনো বোঝা হয়নি।

শিক্ষকের নোট: ক্লাস শুরু করার আগে এই স্লাইডটি একবার দেখিয়ে বলুন — "আমরা মুখস্থ করছি না, ছাঁচ শিখছি।" এতে শিক্ষার্থীর প্রত্যাশা ঠিক থাকে, আর প্রশ্নও বদলে যায়।

পর্ব ১ · কনসেপ্ট · ক্লাস ১ / ৩৭

শূন্য থেকে শুরু: কী বানাচ্ছি আর কার জন্য

সিলেবাস: প্র্যাকটিক্যাল ১ · টপিক ১.১ ও ১.২

প্রজেক্ট চিহ্নিত করা · ব্যবহারকারী চেনা · স্কোপ বোঝা

IMS — Institute Management System

শুরু করুন প্রশ্ন দিয়ে

আপনার স্কুলে আজকের উপস্থিতি কোথায় লেখা হচ্ছে?

হাত তুলে উত্তর নিন — ২–৩ জন

খাতায়? রেজিস্টারে? এক্সেলে?

আর মাসিক রিপোর্ট বানাতে কত সময় লাগে?

এই তিনটে উত্তর থেকেই আজকের প্রজেক্টের জন্ম — আর তিনটেই আসলে **একই সমস্যার তিনটি মুখ**।

শিক্ষকের নোট: উত্তর যাই আসুক, থামবেন না। কেউ "এক্সেলে করি" বললে জিজ্ঞেস করুন — "দুই শিক্ষক একই ফাইলে একসাথে লিখলে কী হয়?"

ধাপ ১

সমস্যাটা কী

খাতা-ভিত্তিক ব্যবস্থার তিনটি বাস্তব ক্ষতি

📕

তথ্য হারায়

খাতা ভিজলে বা হারালে এক বছরের উপস্থিতি ও নম্বর একসাথে মুছে যায়। কোনো কপি নেই।

⏳

রিপোর্টে সময়

"গত মাসে কোন ক্লাসের উপস্থিতি সবচেয়ে খারাপ?" — এই এক প্রশ্নের উত্তর দিতে কয়েক ঘণ্টা হিসাব।

🕵️

দায় থাকে না

কে নম্বর বদলাল, কখন বদলাল — কোনো রেকর্ড নেই। ভুল হলে দায় কেউ নেয় না।

সফটওয়্যারের কাজ এক লাইনে: **তথ্য একবার লেখা হবে, অনেকবার কাজে লাগবে।**

শিক্ষকের নোট: বলুন — "তিনটি সমস্যার একটাও প্রযুক্তির নয়। তিনটিই তথ্য হারানো, সময় নষ্ট, আর জবাবদিহি। ভালো সফটওয়্যার শুরু হয় সমস্যা দেখে, প্রযুক্তি দেখে নয়।"

সিদ্ধান্ত ১

এক স্কুলের সফটওয়্যার নয় — একটি প্ল্যাটফর্ম

প্রজেক্টের আকার ঠিক করে দেওয়া প্রথম সিদ্ধান্ত

এক স্কুলের জন্য বানালে

২০টি স্কুলে দিতে হলে ২০টি কপি →
২০টি ডেটাবেস · ২০টি আপডেট · ২০টি ব্যাকআপ · ২০টি আলাদা বাগ

একজন ডেভেলপার সামলাতে পারবে না। প্রথম গ্রাহক এলেই থেমে যাবে।

এক প্ল্যাটফর্ম বানালে

একই সিস্টেমে অনেক প্রতিষ্ঠান →
একটি ডেটাবেস · একটি আপডেট · একটি ব্যাকআপ

প্রতিষ্ঠান নিজেই সাইনআপ করে, ১৪ দিন ট্রায়াল নেয়, তারপর সাবস্ক্রিপশন।

এই এক সিদ্ধান্তই ঠিক করে দিল ডেটাবেসের নকশা, লগইন, অনুমতি, বিলিং — সব। প্রজেক্টের প্রথম দিনে এটা ভাবা সবচেয়ে জরুরি কাজ।

শিক্ষকের নোট: বোর্ডে লিখুন — "এক স্কুল = কপি। অনেক স্কুল = প্ল্যাটফর্ম।" তারপর জিজ্ঞেস করুন: "আমাদের প্রজেক্ট কোনটা?"

ধারণা

একটি পণ্য, তিনটি স্তর

প্রতিটির ব্যবহারকারী আলাদা — তাই উন্নয়নেও আলাদা

পাবলিক সাইট

দাম দেখানো → সাইনআপ → ১৪ দিনের ফ্রি ট্রায়াল → লগইন

লগইনের আগে

কাজের সফটওয়্যার

উপস্থিতি · পাঠ · হোমওয়ার্ক · পরীক্ষা · রিপোর্ট কার্ড

লগইনের পরে

অপারেটর প্যানেল — /platform/

গ্রাহক প্রতিষ্ঠান · প্ল্যান · ট্রায়াল · ইনভয়েস

শুধু আপনার

শিক্ষকের নোট: তিনটে পর্দা লাইভ দেখান — প্রথমে pricing পেজ, তারপর লগইন করে ড্যাশবোর্ড, শেষে `/platform/`। বলুন: "একই কোড, তিনটে আলাদা পণ্য।"

ধারণা

পাঁচটি ভূমিকা

প্রতিটি ভূমিকা শুধু নিজের অংশটুকু দেখে — পর্দায় এবং সার্ভারে

অ্যা প্রতিষ্ঠান অ্যাডমিন

✓ ক্লাস-বিষয় তৈরি, শিক্ষক যোগ, ছাত্র ভর্তি, ফল প্রকাশ, ব্র্যান্ডিং, সাবস্ক্রিপশন নবায়ন

× অন্য প্রতিষ্ঠানের তথ্য

কো কোঅর্ডিনেটর

✓ একটি বিভাগ বা একটি ক্লাসের তদারকি — উপস্থিতি ও নম্বরের অবস্থা

× নিজের স্কোপের বাইরে যাওয়া, অ্যাডমিনের কাজ

শি শিক্ষক

✓ উপস্থিতি নেওয়া, পাঠ পরিকল্পনা, হোমওয়ার্ক দেওয়া, নম্বর তোলা

× ফল প্রকাশ, বিলিং, অন্য ক্লাসের নম্বর

ছা শিক্ষার্থী

✓ হোমওয়ার্ক জমা, নিজের ফল ও উপস্থিতি দেখা

× অন্য কারো ফল, নম্বর বদলানো

প্ল্যা প্ল্যাটফর্ম অ্যাডমিন — শুধু আপনার

✓ সব গ্রাহক প্রতিষ্ঠান, প্ল্যান, ট্রায়াল, ইনভয়েস — পুরো ব্যবসা

× কোনো প্রতিষ্ঠানের ভেতরের পড়াশোনার তথ্য

খেয়াল করুন — প্রতিটি ভূমিকার নিচে একটা লাল × আছে। **সেই লাল অংশটাই পরে আমরা কোডে আটকাব।**

শিক্ষকের নোট: পাঁচজনের দায়িত্ব পড়তে বলুন, তারপর জিজ্ঞেস করুন — "কে সবচেয়ে কম দেখবে?" উত্তর: শিক্ষার্থী। "কে সবচেয়ে বেশি?" উত্তর: অ্যাডমিন — কিন্তু সে-ও অন্য প্রতিষ্ঠান দেখে না।

কেন্দ্রীয় ধারণা

এক ডেটাবেস, আলাদা ঘর

প্রতিটি প্রতিষ্ঠান একটি Workspace — এর বাইরের কোনো আইডি দিলে ৪০৪

Workspace A — Sunrise Model School

- শিক্ষার্থী · উপস্থিতি · নম্বর
- ক্লাস · বিষয় · পরীক্ষা
- সাবস্ক্রিপশন · ইনভয়েস
- ব্র্যান্ডিং · লোগো

শিক্ষক, শিক্ষার্থী, কোঅর্ডিনেটর — সবাই এখানে

B-এর আইডি
A-তে খুঁজলে
৪০৪

Workspace B — Springfield Coaching

- শিক্ষার্থী · উপস্থিতি · নম্বর
- ক্লাস · বিষয় · পরীক্ষা
- সাবস্ক্রিপশন · ইনভয়েস
- ব্র্যান্ডিং · লোগো

সম্পূর্ণ আলাদা — একটাও সারি ভাগ করা নয়

"পাওয়া যায়নি" — "নিষেধ" নয়

শিক্ষকের নোট: ৪০৪ আর ৪০৩-এর পার্থক্যটা জিজ্ঞেস করুন। উত্তর: ৪০৩ বললে বোঝা যায় জিনিসটা আছে, শুধু অনুমতি নেই — সেটাও তথ্য ফাঁস। এই একটা পার্থক্যই এই প্রজেক্টের নিরাপত্তার ভিত্তি।

পার্থক্য

"ব্যবহারকারী" আর "গ্রাহক" এক নয়

এই পার্থক্য না বুঝলে বিলিং মডেল উল্টো হয়ে যাবে

	ব্যবহারকারী (user)	গ্রাহক (customer)
কে	শিক্ষক, শিক্ষার্থী, কোঅর্ডিনেটর, অ্যাডমিন	প্রতিষ্ঠান (স্কুল/কোচিং)
সংখ্যা	এক প্রতিষ্ঠানে ৫–৮০০ জন	এক প্রতিষ্ঠান = এক গ্রাহক
টাকা দেয়?	**না**	**হ্যাঁ** — সাবস্ক্রিপশন
অ্যাকাউন্ট বন্ধ হলে	নিজের লগইন হারায়, প্রতিষ্ঠান টিকে থাকে	প্রতিষ্ঠানের সেবা বন্ধ হয়
কোডে কোথায় বাঁধা	`User` + `Membership`	`Workspace` + `Subscription`

এই প্রজেক্টে একবার ভুল হয়েছিল: সাবস্ক্রিপশন বাঁধা ছিল ব্যক্তির সদস্যপদে। ফলে শিক্ষক প্রতিষ্ঠান ছাড়লে সাবস্ক্রিপশনও চলে যেত। ঠিক করা হয়েছে `Workspace`-এ বাঁধে।

শিক্ষকের নোট: ক্লাসে জিজ্ঞেস করুন — "তোমার প্রজেক্টে টাকা কে দেবে? ব্যবহারকারী, না প্রতিষ্ঠান?" উত্তরটা নিজের প্রজেক্টের ডেটা মডেলে লিখে রাখতে বলুন।

লাইভ

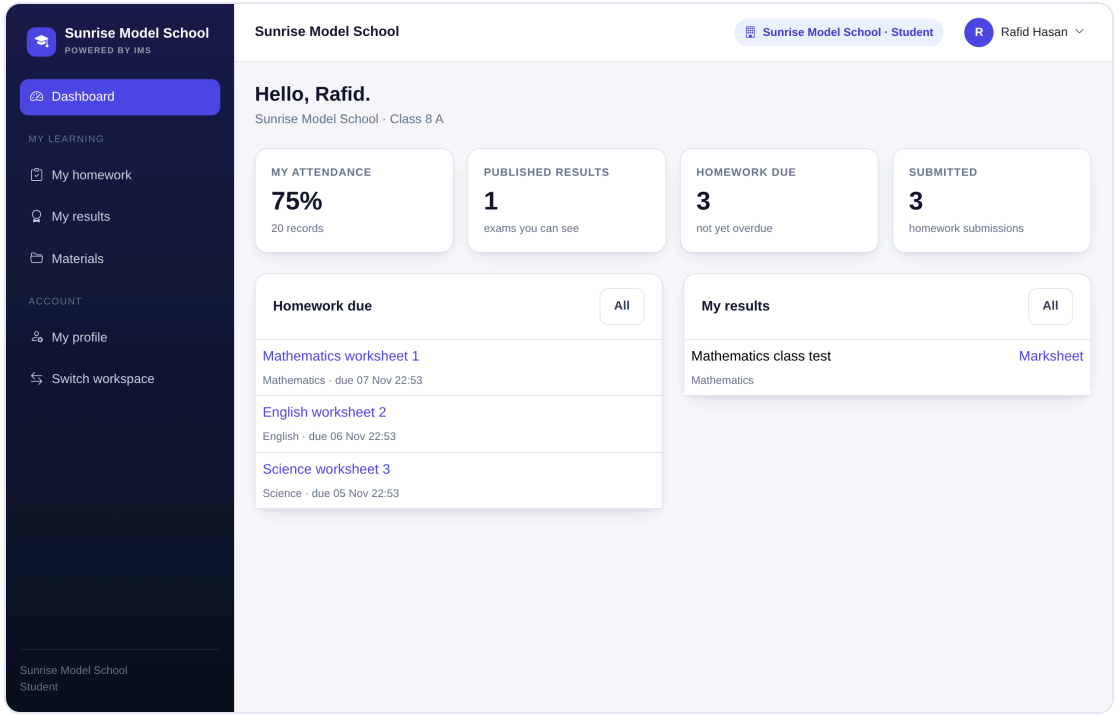
চারটি লগইন, চারটি ভিন্ন পর্দা

একই সিস্টেম — শুধু ভূমিকা বদলান, মেনু বদলে যায়

লগইন	ভূমিকা	মেনু সংখ্যা
`admin@ims.local`	প্রতিষ্ঠান অ্যাডমিন	সব
`coordinator@ims.local`	কোঅর্ডিনেটর	তদারকি
`teacher@ims.local`	শিক্ষক	৫টি
`student@ims.local`	শিক্ষার্থী	৩টি

```
cd ~/Projects/"Web Development Project"/ims
bash run.sh
→ http://127.0.0.1:8000/

সবার পাসওয়ার্ড:  DemoPass!2026
```



শিক্ষার্থীর পর্দা — মাত্র তিনটি মেনু

ক্লাসে করণীয়: অ্যাডমিন হয়ে মেনু **গুনুন** ও খাতায় লিখুন। তারপর শিক্ষার্থী হয়ে ঢুকে আবার গুনুন।

শিক্ষকের নোট: ছাত্ররাই বলবে কী কী হারিয়ে গেছে। এই মুহূর্তটাই রোলভিত্তিক ইন্টারফেস শেখানোর সবচেয়ে ভালো সুযোগ।

ক্লাসের সেরা মুহূর্ত

মেনুতে নেই, তবু ঢোকা যায়?

শিক্ষক হয়ে হাতে এই ঠিকানা লিখুন

```
/w/sunrise-model-school/billing/invoices/
```

ভুল ধারণা

"মেনুতে লিংক নেই, তাই কেউ ঢুকতে পারবে না।" — এটা নিরাপত্তা নয়, শুধু লুকানো।

আসল নিয়ম

মেনু লুকানো = **সুবিধা**। আটকাতে হয় **সার্ভারে** — তাই ৪০৩ পর্দা আসে।

নিয়ম: **একই তালিকা** ব্যবহার হয় মেনু আঁকতে এবং অনুমতি যাচাই করতে — `apps/core/navigation.py`।

শিক্ষকের নোট: "এই একটু জায়গাতেই বেশিরভাগ ছাত্রদের প্রজেক্টে ফাঁক থাকে — মেনু লুকিয়ে দেয়, কিন্তু URL দিয়ে ঢোকা আটকায় না। পরীক্ষায় এটা জিজ্ঞেস করা হবে।"

প্রমাণ

"কাজ করছে" বলার আগে স্ক্রিপ্টে দেখান

৫৫ পেজ × ৪ ভূমিকা = ২২০টি স্বয়ংক্রিয় যাচাই

```
python smoke_test.py --base http://127.0.0.1:8000

  ORG_ADMIN     55/55 as expected
  COORDINATOR   55/55 as expected
  TEACHER       55/55 as expected
  STUDENT       55/55 as expected

PASS: 0 mismatched expectations
```

১০৩

স্বয়ংক্রিয় টেস্ট — `apps/*/tests.py`

২২০

ভূমিকা × পেজ যাচাই — `smoke_test.py`

১৯

Git commit — প্রতিটি সিদ্ধান্তের প্রমাণ

শিক্ষকের নোট: বলুন — "কেউ ভুল করে শিক্ষার্থীর জন্য বিলিং পেজ খুলে দিলে, এই স্ক্রিপ্ট পরের বার চালানোর সময়েই ধরে ফেলবে। আপনার না দেখেও।"

আলোচনা

ক্লাসে জিজ্ঞেস করুন

এই চারটি প্রশ্নের উত্তর নিজের প্রজেক্টে খুঁজতে হবে

- তোমার সিস্টেমে **কে কে** ব্যবহার করবে — তিনটি ভূমিকা লিখে ফেলো।
- প্রতিটি ভূমিকা **কোন জিনিসটা দেখতে পাবে না**? (এটাই আসল ডিজাইন)
- "গ্রাহক" আর "ব্যবহারকারী" কি তোমার প্রজেক্টে **একই মানুষ**?
- তোমার সিস্টেম **কী করবে না** — একটা উত্তর এখনই বলো।

শিক্ষকের নোট: শেষ প্রশ্নটা কঠিন — ২০ সেকেন্ড সময় দিন। যে উত্তর দিতে পারে না, তার স্কোপ এখনো ঠিক হয়নি। এটাই পরের ক্লাসের প্রবেশদ্বার।

সারাংশ

আজকের পাঁচটি শিক্ষা

ক্লাস শেষে বোর্ডে এগুলোই লিখে রাখুন

#	শিক্ষা	প্রমাণ
১	সমস্যা আগে, প্রযুক্তি পরে	তিনটি ক্ষতি — একটাও প্রযুক্তিগত নয়
২	ভূমিকা ঠিক মানে পর্দা ঠিক	৪টি লগইন, ৪টি ভিন্ন মেনু
৩	"দেখায় না" মানে "আটকায় না" নয়	৪০৩ পর্দা — সার্ভারে যাচাই
৪	ব্যবহারকারী ≠ গ্রাহক	`User` বনাম `Workspace`
৫	দাবির প্রমাণ থাকতে হয়	`smoke_test.py` — ২২০ যাচাই

শিক্ষকের নোট: ক্লাসের শেষ দুই মিনিট: প্রত্যেকে একটা বাক্যে বলবে — "আজকের কোন শিক্ষাটা আমার প্রজেক্টে সবচেয়ে দরকার?"

১

পরের ক্লাস · ক্লাস ২

ধারণা অনেক আছে, কোনটা বাছব?

১২টি প্রজেক্ট · ৫টি মাপকাঠি · একটি স্কোর টেবিল

পর্ব ১ · কনসেপ্ট · ক্লাস ২ / ৩৭

কোন প্রজেক্ট, আর কোনটা বাদ দেব

সিলেবাস: প্র্যাকটিক্যাল ১ · টপিক ১.৩ (প্রজেক্ট নির্বাচন ও স্কোপ)

স্কোর কার্ড · স্কোপ ফ্রিজ · ফিচার ক্যাটালগ

IMS — ৫৭ ফিচার · ৬ ভূমিকা · ৩১ টেবিল

শুরু করুন প্রশ্ন দিয়ে

বারোটা আইডিয়া, একটা সেমিস্টার — কোনটা নেবেন?

হাত তুলে বলুন: আপনার পছন্দের প্রজেক্টটা কী, আর কেন?

সবাই বলে "যেটা ভালো লাগে"।
কিন্তু ভালো লাগা দিয়ে প্রজেক্ট শেষ হয় না।

সেমিস্টারের শেষে দুই ধরনের প্রজেক্ট দেখা যায় — **একটা ডেমো দিতে পারে**, আরেকটা "প্রায় হয়ে গেছে" বলে চলে যায়।
পার্থক্যটা কোডিংয়ে তৈরি হয় না। তৈরি হয় **প্রথম সপ্তাহের সিদ্ধান্তে**।

শিক্ষকের নোট: কেউ "আমি ই-কমার্স বানাব" বললে থামিয়ে জিজ্ঞেস করুন — "কার ই-কমার্স? ক্রেতা কে, বিক্রেতা কে, টাকা কে নেবে?" উত্তর যত দেরিতে আসে, প্রজেক্টটা তত বেশি ঝুঁকিতে।

ধাপ ১

পাঁচটি মাপকাঠি — ১০ নম্বরের স্কোর কার্ড

প্রতিটি মাপকাঠিতে ০ / ১ / ২ — মোট সর্বোচ্চ ১০

👤

১. ব্যবহারকারী আছে?

২ = চেনা মানুষ প্রতিদিন ব্যবহার করবে
১ = কল্পনা করতে হচ্ছে
০ = "সবাই ব্যবহার করবে"

🧩

২. ডেটার জটিলতা

২ = সম্পর্ক আছে, ভূমিকা আছে
১ = শুধু তালিকা
০ = একটি টেবিলই যথেষ্ট

🔍

৩. প্রমাণ দেখানো যাবে?

২ = স্ক্রিনশট/টেস্ট দিয়ে দেখানো যায়
১ = আংশিক
০ = শুধু মুখে বলতে হবে

⏱️

৪. সময়ে শেষ হবে?

২ = ৮ সপ্তাহে MVP
১ = টানাটানি
০ = অসম্ভব

🔥

৫. নিজের আগ্রহ

২ = রাতে ঘুম আসবে না
১ = ঠিক আছে
০ = বাধ্য হয়ে

নিয়ম: **৩ নম্বর মাপকাঠিতে ০ পেলে বাকি স্কোর যতই হোক — প্রজেক্ট বাদ।** যে কাজের প্রমাণ দেখানো যায় না, সেটা কোর্সে "কাজ" হিসেবে গণ্য হয় না।

শিক্ষকের নোট: ৫ নম্বর (আগ্রহ) কে ছোট করে দেখবেন না — এটি একমাত্র মাপকাঠি যা ১২ সপ্তাহের ক্লান্তি সামলায়। কিন্তু এটি একা কখনোই যথেষ্ট নয়।

ধাপ ২

১২টি প্রজেক্টের স্কোর

ক্লাসের কাজ: নিজের পছন্দের প্রজেক্টটি এই টেবিলে বসিয়ে স্কোর দিন

#	প্রজেক্ট	ব্যবহারকারী	ডেটা	প্রমাণ	সময়	আগ্রহ	মোট /১০	সিদ্ধান্ত
১	**স্কুল/কলেজ ম্যানেজমেন্ট (IMS)**	২	২	২	১	২	**৯**	✓ নেওয়া হলো
২	রেস্টুরেন্ট POS ও বিলিং	২	২	১	২	২	**৯**	◐ টাই — পরে দেখুন
৩	হাসপাতাল অ্যাপয়েন্টমেন্ট	২	২	১	০	২	৭	○ বাদ
৪	গার্মেন্টস অ্যাটেনডেন্স	২	২	২	০	১	৭	○ বাদ
৫	লাইব্রেরি বই ইস্যু	১	১	২	২	১	৭	○ বাদ
৬	টিউশন বুকিং	২	১	১	১	১	৬	○ বাদ
৭	কৃষি পরামর্শ অ্যাপ	১	১	০	২	২	৬	✕ প্রমাণ ০
৮	রক্তদান ম্যাচিং	১	১	১	২	১	৬	○ বাদ
৯	ই-কমার্স (কাপড়)	২	১	১	০	১	৫	✕ সময় ০
১০	ফ্রিল্যান্স মার্কেটপ্লেস	১	২	১	০	১	৫	✕ সময় ০
১১	সোশ্যাল মিডিয়া ক্লোন	১	২	০	০	২	৫	✕ প্রমাণ ০
১২	ট্রাভেল বুকিং	১	২	১	০	০	৪	✕ সময় ০

খেয়াল করুন: **দুই নম্বরে ৯ আর এক নম্বরে ৯ — টাই।** স্কোর টেবিল একা সিদ্ধান্ত দেয় না, শুধু বাজে আইডিয়া ছেঁটে দেয়।

শিক্ষকের নোট: "সময় ০" মানে অসম্ভব নয় — মানে ওই প্রজেক্টের MVP এখনো কেউ সংজ্ঞায়িত করেনি। কেউ প্রতিবাদ করলে বলুন: "তাহলে বলো, প্রথম সপ্তাহে ঠিক কী কাজ করবে?"

ধাপ ৩

৯টি কেন বাদ পড়ল — তিনটি কারণে

বাদ দেওয়াও একটা সিদ্ধান্ত, লিখে রাখতে হয়

প্রমাণ দেওয়া যাবে না

কৃষি পরামর্শ, সোশ্যাল ক্লোন — ব্যবহারকারী পাওয়া যায় না, তাই স্ক্রিনশটের পেছনে গল্প ছাড়া কিছু নেই।

৩টি প্রজেক্ট

MVP সংজ্ঞায়িত হয়নি

ই-কমার্স, মার্কেটপ্লেস, ট্রাভেল — "পুরোটা" করতে গেলে ২ বছরের কাজ, "কোনটা বাদ" বলা যায়নি।

৪টি প্রজেক্ট

ডেটা খুব সরল

লাইব্রেরি, রক্তদান — শেখানোর জন্য ভালো, কিন্তু সম্পর্ক (relationship) শেখানোর মতো গভীরতা নেই।

২টি প্রজেক্ট

নিয়ম: **প্রতিটি বাদ দেওয়া আইডিয়ার পাশে এক লাইনে কারণ লিখুন।** তিন মাস পর "আমার আরেকটা আইডিয়া ছিল" বলে প্রজেক্ট বদলানো শুরু হয় ঠিক এই এক লাইন না লেখার কারণে।

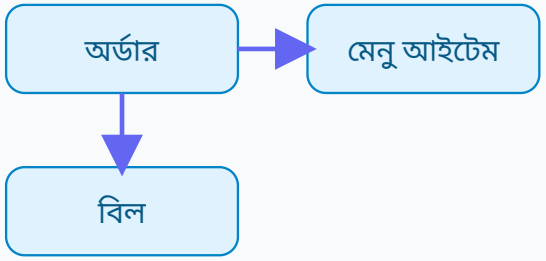
শিক্ষকের নোট: এই স্লাইডে বলা "৯টি" = ১২ − ৩ (IMS + রেস্টুরেন্ট POS + যেটা আপনি টাই-ব্রেকে রাখবেন)। শ্রেণিকক্ষে সংখ্যা বদলাতে পারেন, কারণটা বদলাবেন না।

ধাপ ৪

টাই ভাঙল কীভাবে — স্কুলের ডেটার আকৃতি

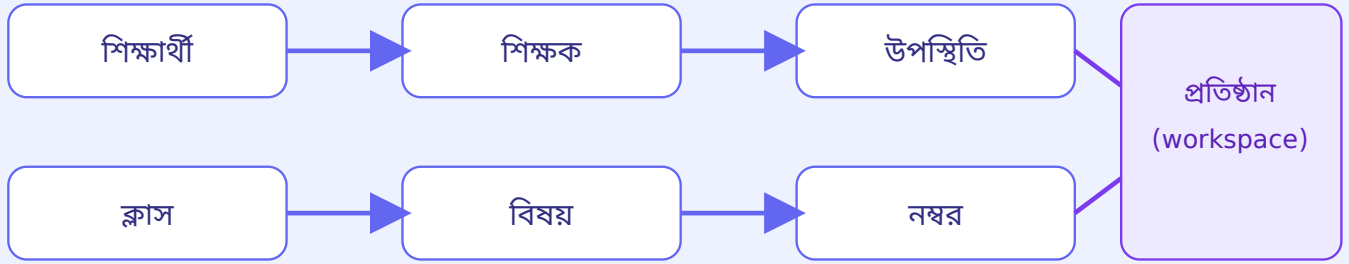
৯ বনাম ৯: রেস্টুরেন্ট আর স্কুলের মধ্যে তফাত কোথায়

রেস্টুরেন্ট POS



এক দিকের স্রোত · ভূমিকা ২টি

IMS — স্কুল



একই শিক্ষার্থী বহু ক্লাসে · বহু শিক্ষক · ভূমিকা ৫টি · সবকিছু প্রতিষ্ঠানভিত্তিক

POS-এ সম্পর্ক একমুখী — অর্ডার → বিল। স্কুলে সম্পর্ক **জালের মতো**: এক শিক্ষার্থী বহু ক্লাসে, এক ক্লাসে বহু শিক্ষক, এক শিক্ষক বহু প্রতিষ্ঠানে। এই জালটাই আপনাকে শেখাবে **foreign key, many-to-many, এবং tenant isolation** — তিনটে সবচেয়ে দামি ধারণা।

শিক্ষকের নোট: ছাত্রদের বলুন — "যে প্রজেক্টে টেবিলের মধ্যে তির ছোড়া যায়, সেটা নাও। যে প্রজেক্টে একটাই টেবিল — সেটা টিউটোরিয়াল, প্রজেক্ট নয়।"

ধাপ ৫

"স্কোপ ফ্রিজ" — এক পাতায় তিনটি বাক্স

আজকের কাজ শেষ হলে এই তিনটি বাক্স ভরে ফেলুন, তারপর বরফে ঢালুন

🧊 MVP — ফ্রিজে

কোর্স শেষে **অবশ্যই** চলবে। কম নয়, বেশি নয়।
IMS-এ: লগইন · প্রতিষ্ঠান · ছাত্র · উপস্থিতি · নম্বর · ড্যাশবোর্ড

🕐 v1.1 — পরে

দরকার, কিন্তু ফ্রিজের বাইরে। তারিখ লিখে রাখুন।
IMS-এ: আমন্ত্রণ · ব্যাকআপ · রিপোর্ট এক্সপোর্ট · নোটিফিকেশন

🚫 কখনো নয়

সচেতনভাবে বাদ। কারণ লিখে রাখুন, নইলে তিন মাস পর ফিরে আসবে।
IMS-এ: SMS গেটওয়ে · AI উপস্থিতি · মোবাইল অ্যাপ

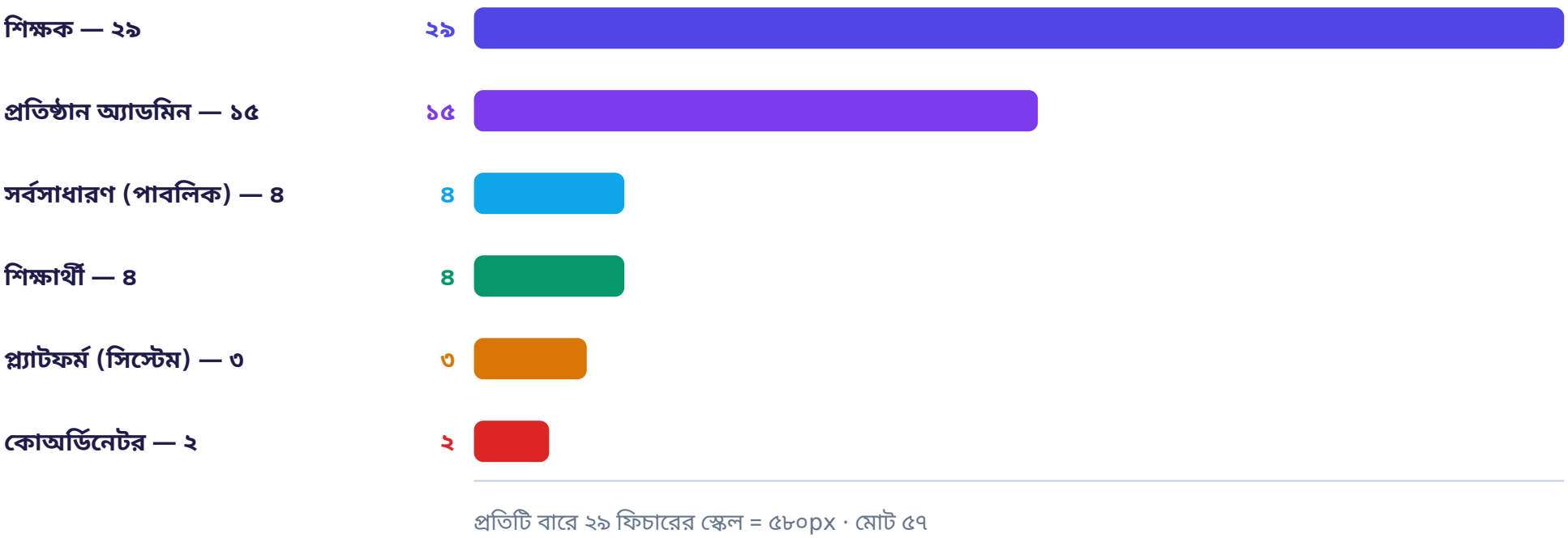
ফ্রিজের নিয়ম একটাই: **নতুন ফিচার যোগ করতে হলে অন্য একটা ফিচার বাদ দিতে হবে।** স্কোপ বাড়ে না — শুধু বদলায়।

শিক্ষকের নোট: "কখনো নয়" বাক্সটা সবচেয়ে জরুরি আর সবচেয়ে বেশি বাদ পড়ে। বোর্ডে লিখে রাখুন — সেমিস্টারের মাঝে কেউ ওই ফিচার চাইলে এই লেখাটাই আপনার উত্তর।

ধাপ ৬

৫৭টি ফিচার, ৬টি ভূমিকা — ফ্রিজ করা তালিকা

প্রজেক্টের আসল ডকুমেন্ট: `docs/catalogue_data.py` → F-01 … F-57



> প্রথমেই ভাববেন না "কী বানাব" — ভাবুন **"কে ব্যবহার করবে, আর সে কী করতে পারবে"**। ভূমিকা আগে, পর্দা পরে।

> **শিক্ষকের নোট:** জিজ্ঞেস করুন — "শিক্ষকের ২৯, শিক্ষার্থীর ৪। এটা কি অন্যায়?" উত্তর: না — শিক্ষক হচ্ছেন সিস্টেমের *উৎপাদক*, শিক্ষার্থী *ভোক্তা*। যে লেখে তার কাজ বেশি, যে পড়ে তার কাজ কম।

ধাপ ৭

একটি ফিচার দেখতে কেমন — F-31, হুবহু প্রজেক্ট থেকে

এই ছয়টি লাইন না লিখলে ফিচারটা "ধারণা", "স্পেসিফিকেশন" নয়

```
F-31  উপস্থিতি নেওয়া
ভূমিকা : TEACHER
রুট     : /w/<id>/attendance/take/
উদ্দেশ্য: এক দিনের জন্য পুরো ক্লাসকে
          একবারে উপস্থিত/অনুপস্থিত/
          দেরি/ছুটি চিহ্নিত করা
ক্রিয়া  : ক্লাস, তারিখ, বিষয় বাছুন
          bulk বোতামে সবাইকে চিহ্নিত করুন
          নোট যোগ করুন → সেভ
ডেটা    : AttendanceRecord unique(
          student, class, subject, date)
          লকড টার্মে তারিখ froze
অবস্থা  : আজ নেওয়া হয়ে গেছে → edit mode
          ভবিষ্যৎ তারিখ বন্ধ
          ছুটির দিনে সতর্কবার্তা
          কোনো ছাত্র নেই
```

ছয়টি লাইন = ছয়টি প্রশ্নের উত্তর

ভূমিকা — কে ঢুকতে পারবে?
রুট — URL কী? (তাই permission যাচাই করা যায়)
উদ্দেশ্য — এক বাক্যে কাজটা
ক্রিয়া — পর্দায় ঠিক কী কী বোতাম
ডেটা — কোন টেবিল, কোন unique নিয়ম
অবস্থা — ভুল হলে কী দেখাবে

"অবস্থা" লাইনটা বাদ দিলেই ৮০% বাগ জন্মায় — খালি টেবিল, ভবিষ্যতের তারিখ, ডবল সাবমিট।

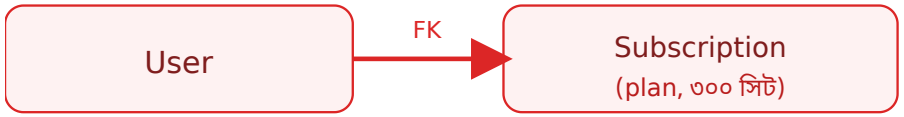
শিক্ষকের নোট: বোর্ডে নিজের প্রজেক্টের একটা ফিচার এই ছয় লাইনে লিখিয়ে নিন। যে ছাত্র "ডেটা" লাইনটা লিখতে পারবে না, তার স্কোপ এখনো ফ্রিজ হয়নি — ক্লাস ১৪-এ তার ডেটাবেস ডিজাইন ভাঙবে।

ধাপ ৮

ফ্রিজ ভাঙার আসল ঘটনা — সাবস্ক্রিপশন ভুল জায়গায় বাঁধা ছিল

কমার্শিয়াল লেয়ার দেরিতে যোগ হয় → টেবিলের সম্পর্ক উল্টে যায়

ভুল (প্রথম লেখা)



শিক্ষক ২ স্কুলে পড়ান → কার প্ল্যান? ব্যক্তির নামে বিল, প্রতিষ্ঠানের নয়।

ঠিক (commit 8a25724)



বিল প্রতিষ্ঠানের নামে · সিট প্রতিষ্ঠানের
শিক্ষক শুধু সদস্য

যা শিখতে হবে: স্কোপ ফ্রিজ শুধু "কী বানাব" ঠিক করে না — **কে কার অধীন** সেটাও ঠিক করে। এই একটি সিদ্ধান্ত ভুল হলে টেবিল, ফর্ম, পারমিশন, রিপোর্ট — সব একসাথে বদলাতে হয়।

শিক্ষকের নোট: প্রশ্ন করুন — "এক শিক্ষক দুই প্রতিষ্ঠানে পড়ান। তার এক প্রতিষ্ঠানের সাবস্ক্রিপশন শেষ। সে কি অন্য প্রতিষ্ঠানে ঢুকতে পারবেন?" সঠিক উত্তর: হ্যাঁ — কারণ বিল ব্যক্তির নয়, প্রতিষ্ঠানের।

ধাপ ৯

একটা ফিচারের দাম কত — বাস্তব হিসাব

IMS-এর ১১টি অ্যাপ থেকে গোনা সংখ্যা

🗄️

৩১

ডেটাবেস টেবিল · ২৩২টি ফিল্ড · ১২টি মাইগ্রেশন

🖼️

৭৫

টেমপ্লেট — অর্থাৎ প্রতি ফিচারে ~১.৭টি পর্দা

🔗

১১

urls.py · এক ফিচার = অন্তত ১টি রুট + ১টি ভিউ

🧪

১০৩

টেস্ট — প্রতি ফিচারে গড়ে ১.৮টি স্বয়ংক্রিয় পরীক্ষা

ফিচারের ধরন	যা লাগে	আনুমানিক সময়
তালিকা পর্দা (list)	মডেল + ভিউ + ১ টেমপ্লেট + ১ টেস্ট	আধা দিন
ফর্ম পর্দা (create/edit)	ফর্ম + ভ্যালিডেশন + ৪টি error অবস্থা	১ দিন
রিপোর্ট / হিসাব	কোয়েরি + অ্যাগ্রিগেট + এক্সপোর্ট	২ দিন
নতুন ডেটা মডেল	মাইগ্রেশন + অ্যাডমিন + পুরনো ডেটার যত্ন	৩ দিন

শিক্ষকের নোট: এই টেবিলটা মুখস্থ করাতে হবে না — ব্যবহার করাতে হবে। ছাত্র "আরেকটা ফিচার যোগ করব" বললে বলুন: "ঠিক আছে, কোন ধরনের? কত দিন? কোনটা বাদ দেবে?"

ধাপ ১০

স্কোপ সত্যিই কতটা শেষ — সংখ্যাটা সৎভাবে বলুন

৫৭টি ফ্রিজ করা ফিচার · প্রমাণ হিসেবে স্ক্রিনশট গোনা হয়েছে

৪৩

স্ক্রিনশট প্রমাণসহ তৈরি

`docs/build_status.py`

১১

কোড আছে, ছবি নেই

প্রতিটির নিজের কারণ লেখা — F-06 · F-32 · F-53 · F-35 · F-15 · F-14 · F-23 · F-10 · F-03 · F-04 · F-07

৩

সত্যিই বাকি

F-11 আমন্ত্রণ · F-12 আমন্ত্রণ গ্রহণ · F-57 ব্যাকআপ কনসোল

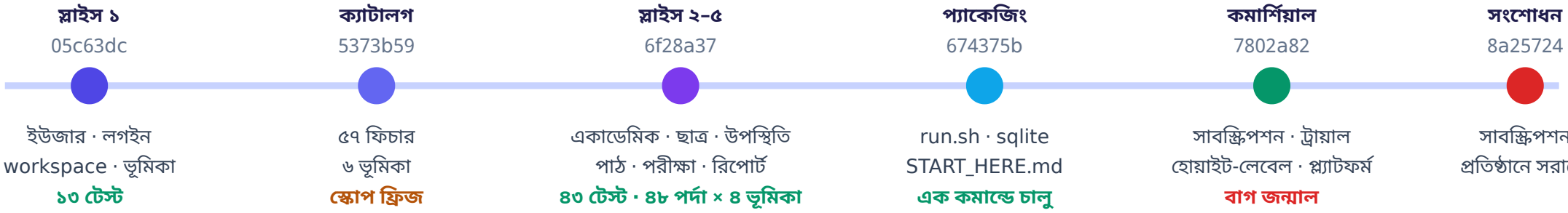
অবস্থা	কী বলবেন ক্লাসে / ডেমোতে	কী করবেন
৪৩ — স্ক্রিনশটসহ	"চালু আছে, স্ক্রিনশট দেখুন"	ডেমোতে দেখান
১১ — ছবি নেই	"কোড আছে ও টেস্টে যাচাই হয়েছে, শুধু পর্দার ছবি নেই"	টেস্টের নাম দেখান
৩ — বাকি	"পরিকল্পিত, হয়নি — কারণ এই"	v1.1 তালিকায় রাখুন

শিক্ষকের নোট: এটাই কোর্সের সবচেয়ে সৎ স্লাইড। "সব হয়ে গেছে" বললে এক প্রশ্নেই ধরা পড়বে। "৪৩টার স্ক্রিনশট আছে, ১১টা টেস্টে যাচাই হয়েছে কিন্তু ছবি নেই, ৩টা সত্যিই বাকি" — এই উত্তর আপনাকে বিশ্বাসযোগ্য করে।

ধাপ ১১

বড় কাজকে স্লাইসে ভাগ করা — আসল কমিটগুলোর ক্রম

প্রতিটি স্লাইস শেষে সিস্টেম চলে, টেস্ট সবুজ



নিয়ম: প্রতিটি স্লাইস এমন একটা সিস্টেম রেখে যায় যা চলে, এবং প্রতিটি স্লাইসের শেষে টেস্ট সবুজ থাকে।
এক সপ্তাহে "সব কিছু" বানাতে গিয়ে কিছুই চলে না — এই ফাঁদেই পড়ে ৮০% প্রজেক্ট।

শিক্ষকের নোট: ভার্সিটি দেখিয়ে বলুন — ক্যাটালগ (স্লাইস ১.৫) স্লাইস ২–৫ এর *আগে* লেখা হয়েছিল। প্ল্যান আগে, কোড পরে। এখানেই "স্কোপ ফ্রিজ" শব্দটার মানে বাস্তবে দেখা যায়।

ধাপ ১২

"আরও একটা ছোট ফিচার" — এই বাক্যের দাম

ফিচার ক্রিপ: দেরিতে যোগ হওয়া ফিচার যখন পুরনো ডিজাইন ভাঙে

🧨 যা হয়

কমার্শিয়াল লেয়ার যোগ হলো **স্লাইস ২–৫ শেষে**। তখন ৩১ টেবিল দাঁড়িয়ে গেছে। সাবস্ক্রিপশন ব্যক্তির নামে বাঁধায় প্রতিষ্ঠানের ধারণাটাই ভুল হয়ে গেল।

ঠিক করতে হয়েছে: ১টি মাইগ্রেশন · বিলিং ভিউ · প্ল্যাটফর্ম কনসোল · ৪টি টেস্ট।

✅ যা করা উচিত ছিল

প্রথম দিনেই একটা লাইনে ঠিক করা: **"টাকার একক = প্রতিষ্ঠান, মানুষ নয়।"** এক মিনিটের সিদ্ধান্ত, তিন দিনের কাজ বাঁচাত।

ফিচার ক্রিপের তিনটি চেহারা

১. **"শুধু একটা বোতাম"** — পুরনো ফর্মে নতুন চেকবক্স → ভ্যালিডেশন ভাঙে।
২. **"শুধু একটা রিপোর্ট"** — নতুন কোয়েরি → নতুন ইনডেক্স → নতুন টেবিল।
৩. **"শুধু একটা ভূমিকা"** — ৫ ভূমিকা থেকে ৬ → **সব** পারমিশন আবার যাচাই।

প্রতিরক্ষা: নতুন ফিচার এলে **প্রথমে লিখুন কোন ফিচার বাদ যাচ্ছে**, তারপর কোড লিখুন।

শিক্ষকের নোট: এখানে বাস্তবের সবচেয়ে অনুভূত অংশটা বলুন — "একটা ফিচার যোগ করার সবচেয়ে বড় খরচ কোড লেখা নয়, সব পুরনো জিনিস আবার যাচাই করা।"

আলোচনা

চারটি প্রশ্ন — জোড়ায় জোড়ায় ৫ মিনিট

উত্তর বোর্ডে লিখুন, আজকের শেষে ফিরে আসব

১. আপনার প্রজেক্টের স্কোর কার্ডে "প্রমাণ দেখানো যাবে?" ঘরে কত দিলেন? ঠিক কী স্ক্রিনশট দিয়ে প্রমাণ করবেন, সেটা কি আজই বলতে পারবেন?

২. আপনার MVP-তে কতটি ফিচার? ১৪ দিনের কাজ ফেলে দিলে কোনটা সবচেয়ে আগে বাদ যাবে?

৩. আপনার সিস্টেমে "টাকার একক" কী — ব্যক্তি, প্রতিষ্ঠান, নাকি অর্ডার? এক লাইনে লিখুন।

৪. যে ফিচারটা আপনি সবচেয়ে ভালোবাসেন — সেটার প্রমাণ কি আজ দেখাতে পারবেন? না পারলে সেটা ভালোবাসা, ফিচার নয়।

শিক্ষকের নোট: ৪ নম্বর উত্তরটা সবচেয়ে যন্ত্রণার আর সবচেয়ে দরকারি। যারা "পারব না" বলবে, তাদের কাছ থেকেই সেরা প্রজেক্টগুলো আসে — কারণ তারা সত্যি বলছে।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
ভালো লাগা দিয়ে প্রজেক্ট বাছা যায় না — ৫ মাপকাঠির স্কোর কার্ড লাগে	১২টি আইডিয়ার টেবিলে দুইটি ৯ পেয়েছিল; টাই ভাঙল ডেটার জাল
স্কোপ ফ্রিজ = তিনটি বাক্স (MVP / পরে / কখনো নয়)	৫৭ ফিচারের তালিকা `catalogue_data.py`-তে ফ্রিজ করা
ফিচার লিখতে হয় ছয় লাইনে — ভূমিকা, রুট, উদ্দেশ্য, ক্রিয়া, ডেটা, অবস্থা	F-31 হুবহু প্রজেক্ট ফাইল থেকে
সৎ সংখ্যা বলাই শক্তি	৪৩ স্ক্রিনশটসহ + ১১ টেস্টে যাচাই + ৩ বাকি = ৫৭
দেরিতে যোগ হওয়া ফিচার পুরনো ডিজাইন ভাঙে	কমার্শিয়াল লেয়ার (7802a82) → সাবস্ক্রিপশন সংশোধন (8a25724)

শিক্ষকের নোট: ছাত্রদের বলুন এই পাঁচ লাইন তাদের প্রজেক্ট ডায়েরির প্রথম পাতা হিসেবে লিখে রাখতে — সেমিস্টার শেষে "কী শিখলাম" প্রশ্নের উত্তর এখানেই তৈরি হয়ে যাবে।

০৩

পরের ক্লাস — "এটা কি সত্যিই সম্ভব?"

ফিজিবিলিটি স্টাডি: প্রযুক্তি · পরিচালনা · অর্থ · সময় — আর ভাঙার বিন্দুর হিসাব

পর্ব ১ · কনসেপ্ট · ক্লাস ৩ / ৩৭

এটা কি সত্যিই সম্ভব?

সিলেবাস: প্র্যাকটিক্যাল ২ · টপিক ২.১ (ফিজিবিলিটি স্টাডি)

চারটি দিক: প্রযুক্তি · পরিচালনা · অর্থ · সময়

৯,৩২২ লাইন Python · ২ দিন · ভাঙার বিন্দু ২ গ্রাহক

শুরু করুন প্রশ্ন দিয়ে

ফিজিবিলিটি মানে "পারব কি না" — এটা ভুল ধারণা

হাত তুলুন: আপনার প্রজেক্ট "সম্ভব" — কীসের ভিত্তিতে বলছেন?

"পারব" একটা অনুভূতি।
ফিজিবিলিটি একটা হিসাব — চারটি আলাদা খাতায়।

চারটির যেকোনো একটিতে লাল বাতি জ্বললে প্রজেক্টটা মরে — বাকি তিনটি সবুজ হলেও। আজ প্রতিটি খাতায় **আসল সংখ্যা** বসাব, নিজের অনুমান নয়।

শিক্ষকের নোট: কেউ "সবই পারব" বললে জিজ্ঞেস করুন — "মাসে সার্ভার খরচ কত পড়বে? কে দেবে?" উত্তর না এলে বলুন: "তাহলে এটা এখনো ফিজিবিলিটি নয়, ইচ্ছা।"

ধাপ ১

চারটি খাতা — প্রতিটির নিজস্ব প্রশ্ন

প্রতিটি খাতায় একটি প্রশ্ন, একটি প্রমাণ, একটি লাল বাতির শর্ত

⚙️ প্রযুক্তি

প্রশ্ন: এই কাজটা করার মতো উপকরণ আছে?

প্রমাণ: **Django 6.1.2 · Python 3.13 · ৬টি প্যাকেজ**

লাল বাতি: যে কাজটা লাইব্রেরি ছাড়া করতে হবে

🔧 পরিচালনা

প্রশ্ন: চালু রাখবে কে, কীভাবে?

প্রমাণ: **deploy_check.sh · ৩টি settings · ২ worker**

লাল বাতি: "শুধু আমার ল্যাপটপে চলে"

💰 অর্থ

প্রশ্ন: খরচ কত, আয় কত, কোথায় ভাঙবে?

প্রমাণ: **৳২,১০০/মাস খরচ · ভাঙার বিন্দু ২ গ্রাহক**

লাল বাতি: খরচের হিসাবই নেই

⏳ সময়

প্রশ্ন: কার কত সময় লাগবে?

প্রমাণ: **২ কর্মদিবস · ২০টি কমিট · ১০৩ টেস্ট**

লাল বাতি: শেষ ২ সপ্তাহে সব পেন্ডিং

আজকের ক্লাসে চারটি খাতার প্রতিটিতে **আসল সংখ্যা** বসাব — প্রজেক্টের ফাইল থেকে তোলা।

শিক্ষকের নোট: বোর্ডে চারটি বাক্স আঁকুন, প্রতি বাক্সে একটি প্রশ্ন লিখুন। ছাত্রদের বলুন নিজের প্রজেক্টের জন্য একই চারটি বাক্স খাতায় আঁকতে — ক্লাস ৪-এর কনসেপ্ট পেপারে এটাই যাবে।

খাতা ১

প্রযুক্তি — আটটি নাম, ছয়টি আসলে ব্যবহৃত

প্রজেক্টের `requirements.txt` হুবহু — তারিখ ও ভার্সনসহ

উপকরণ	ভার্সন	কী কাজে	না থাকলে কী হতো
Python	৩.১৩	ভাষা — সব কোড এর ওপর দাঁড়িয়ে	—
Django	৬.১.২	লগইন, পারমিশন, ORM, ফর্ম, অ্যাডমিন	নিজে লিখতে হতো ~২,০০০ লাইন
psycopg	৩.৩.৬	PostgreSQL-এর সাথে কথা বলা	sqlite-এ ৮০০ সিট আটকাবে না
python-dotenv	১.২.৪	গোপন তথ্য `.env` ফাইলে রাখা	পাসওয়ার্ড কোডে ঢুকে যেত
Pillow	১২.৩.০	লোগো/প্রতিষ্ঠানের ব্র্যান্ডিং ছবি	লোগো আপলোড ভাঙত
openpyxl	৩.১.৫	Excel (.xlsx) এক্সপোর্ট	CSV-তে বাংলা ভাঙত
reportlab	৫.১.০	মার্কশিট PDF	ব্রাউজারের প্রিন্ট ভরসা করতে হতো
gunicorn	শুধু prod	আসল সার্ভারে চালানোর জন্য	dev server দিয়ে prod নয়

নিয়ম: **প্রতিটি প্যাকেজের জন্য এক লাইনে লিখতে হবে "না থাকলে কী হতো"**। যে প্যাকেজের উত্তর দিতে পারেন না — সেটা বাদ দিন।

শিক্ষকের নোট: ভার্সন পিন করা কেন — জিজ্ঞেস করুন। উত্তর: "আজ যা চলছে, ৬ মাস পরে আবার ঠিক তাই চলবে।" পিন না করলে যেকোনো একটা আপডেট সব ভেঙে দেয়।

খাতা ১

চারটি বিকল্প, একটা বাছাই — কেন Django

প্রযুক্তি বাছাই = স্বাদ নয়, প্রয়োজন

বিকল্প	লগইন ও ভূমিকা	ডেটাবেস	ডেটা জটিলতা	ডাকুমেন্টেশন	মোট /৮	সিদ্ধান্ত
Django	বিল্ট-ইন (২)	ORM + মাইগ্রেশন (২)	৩১ টেবিল সহজে (২)	অসাধারণ (২)	**৮**	✓ **বাছা হলো**
Flask	নিজে লিখতে হবে (০)	SQLAlchemy যোগ (১)	সম্ভব, কিন্তু কাঠামো নেই (১)	ভালো (২)	৪	○ বাদ
Node + Express	নিজে (০)	Prisma/Sequelize (১)	সম্ভব (২)	ভালো (২)	৫	○ ভাষা বদলাতে হতো
PHP (Laravel)	আছে (২)	Eloquent (২)	সম্ভব (২)	ভালো (২)	৮	◐ সমান, কিন্তু কোর্স Python
No-code (Airtable ইত্যাদি)	আছে (২)	সীমিত (১)	সীমিত — কোড নেই (০)	সহজ (২)	৫	✕ শেখার উদ্দেশ্যই ভাঙে

শিক্ষকের সিদ্ধান্তের কারণ: কোর্স Python-ভিত্তিক, আর Django-তে **পারমিশন আর ভূমিকা** নিজে বানাতে হয় না — শিখতে হয়।

সতর্কতা: Laravel-ও ৮ পেয়েছে। "সেরা প্রযুক্তি" বলে কিছু নেই — যেটা **শেখানো হচ্ছে** আর যেটা **আপনি জানেন**, সেটাই সেরা।

শিক্ষকের নোট: ছাত্র "রিঅ্যাক্ট + Node" বললে বাধা দেবেন না, কিন্তু প্রশ্ন করুন: "লগইন, ভূমিকা, অডিট লগ — এই তিনটে কে লিখবে? কত দিনে?"

খাতা ১ · প্রমাণ

স্কেলের হিসাব — ৮০০ সিট মানে কত লাইন ডেটা

এখানেই sqlite আর PostgreSQL-এর পার্থক্য তৈরি হয়

👥

৩০০ শিক্ষার্থী

শিক্ষকদের আটেনড্যান্স দৈনিক একবার

→ বছরে ৬০,০০০ সারি

📚

দিনে ৬ পিরিয়ড

প্রতি পিরিয়ডে আলাদা হাজিরা

→ বছরে ৩,৬০,০০০ সারি

🏫

২০ প্রতিষ্ঠান

একই সার্ভারে ২০টি স্কুল

→ বছরে ৭২ লাখ সারি

প্রশ্ন	sqlite	PostgreSQL
একসাথে কয়জন লিখতে পারে?	প্রায় একজন (লেখার সময় লক)	অনেকজন, সারি-পর্যায়ে লক
৭২ লাখ সারিতে রিপোর্ট	ধীরে, ইনডেক্স সীমিত	ইনডেক্স + partial index + EXPLAIN
নিরাপত্তা / ব্যবহারকারী আলাদা করা	ফাইল অনুমতি	রোল, গ্রান্ট, স্কিমা
ব্যাকআপ	ফাইল কপি	pg_dump, PITR

প্রজেক্টে তাই **তিনটি settings**: `dev` (sqlite, দ্রুত শুরু) · `sqlite` (এক-ক্লিকে ডেমো) · `prod` (PostgreSQL + gunicorn)।

শিক্ষকের নোট: সংখ্যাগুলো বোর্ডে ধীরে ধীরে গুণ করুন — ৩০০ × ২০০ = ৬০,০০০। ছাত্ররা তখনই বুঝবে "আমার ছোট ডেটা" বলে কিছু নেই; সেমিস্টার শেষে ডেটা বড় হয়।

খাতা ২

পরিচালনা — "আমার ল্যাপটপে চলে" যথেষ্ট নয়

প্রমাণ: `docs/deploy_check.sh` — প্রোডাকশনের মহড়া

```
$ bash docs/deploy_check.sh
-> collectstatic
-> gunicorn on :8010 (DEBUG=False)
   --workers 2
-> checks
  login page            200
  static css            200
  unknown page (404)    404
  workspace page anon   302
-> security headers
  X-Content-Type-Options: nosniff
  X-Frame-Options: DENY
  Referrer-Policy: same-origin
-> done (server stopped)
```

চারটি সংখ্যা যাচাই হলো

200 — পর্দা খুলছে

200 — CSS লোড হচ্ছে (না হলে সাদা পাতা)

404 — ভুল ঠিকানায় ভদ্র উত্তর

302 — অচেনা লোক প্রতিষ্ঠানের ভেতরে ঢুকতে পারছে না

প্রোডাকশনে `DEBUG=True` থাকলে ভুল হলেই সোর্স কোড, পাসওয়ার্ড, টেবিলের নাম — সব browser-এ দেখা যায়।

শিক্ষকের নোট: ৩০২ কেন ৪০৪ নয় — এটাই আজকের সবচেয়ে দামি প্রশ্ন। উত্তর: "ঢুকতে দিই না" আর "কিছু নেই" আলাদা কথা। ৩০২ মানে "আগে লগইন করুন"।

খাতা ৩

অর্থ — মাসিক খরচের সৎ হিসাব

বাংলাদেশের বাজারদরে, ২০২৬ · সংখ্যাগুলো অনুমান, পদ্ধতিটা আসল

খরচের খাত	মাসিক	ব্যাখ্যা
VPS (২ GB RAM, ২ vCPU)	৳১,৮০০	৳২২,০০০/বছর — Django + PostgreSQL একসাথে
ডোমেইন + SSL	৳১০০	SSL বিনামূল্যে (Let's Encrypt), ডোমেইন ৳১,২০০/বছর
ব্যাকআপ স্টোরেজ	৳২০০	প্রতিদিনের dump, ৩০ দিন রাখা
ইমেইল (SMTP)	৳০	ফ্রি টিয়ার দিয়ে শুরু
মোট নির্দিষ্ট খরচ	**৳২,১০০**	গ্রাহক শূন্য হলেও কাটবে
পেমেন্ট গেটওয়ে ফি	২.৫%	প্রতি লেনদেনে — নির্দিষ্ট খরচ নয়
সাপোর্ট (প্রতি গ্রাহক)	৳২৫০	মাসে ২–৩টি ফোন/মেসেজের সমতুল্য

যে খরচটা সবাই ভুলে যায়: **সাপোর্ট**। সফটওয়্যার বিক্রি একবার, ফোন আসে প্রতি সপ্তাহে। এই লাইনটা না লিখলে হিসাব সবসময় মিথ্যা দেখায়।

শিক্ষকের নোট: নিজের শহরের আসল দাম জিজ্ঞেস করুন — কেউ জানলে বোর্ডে লিখুন। সংখ্যা বদলালে সমস্যা নেই, পদ্ধতি বদলালে সমস্যা।

খাতা ৩ · হিসাব

ভাঙার বিন্দু — কতজন গ্রাহকে খরচ উঠে যায়

প্রতি গ্রাহক থেকে হাতে কত থাকে, তারপর ভাগ

Starter — ৳১,৫০০/মাস

আয় ১,৫০০ | −৩৭ গেটওয়ে ২.৫% | −২৫০ সাপোর্ট | **হাতে থাকে ১,২১৩** → ২ জনে খরচ উঠে যায়

Growth — ৳৩,৫০০/মাস

আয় ৩,৫০০ | −৮৮ | −৪০০ সাপোর্ট | **হাতে থাকে ৩,০১২** → ১ জনেই খরচ উঠে যায়

নির্দিষ্ট খরচ ৳২,১০০/মাস ÷ প্রতি গ্রাহকের নিট আয় = ভাঙার বিন্দু

Starter: ২,১০০ ÷ ১,২১৩ = ১.৭ → ২ জন · Growth: ২,১০০ ÷ ৩,০১২ = ০.৭ → ১ জন

> এই একটি হিসাবই বলে দেয়: প্রজেক্টটা **টিকতে ২ জন গ্রাহক লাগবে**। "লাখ লাখ ব্যবহারকারী" নয় — মাত্র দুইজন।

শিক্ষকের নোট: ছাত্রদের নিজের প্রজেক্টে এই ভাগটা করাতে হবে। যাদের উত্তর "কোনো খরচ নেই" আসবে, তাদের বলুন — ডোমেইন, হোস্টিং, ব্যাকআপ, নিজের সময়: অন্তত একটি আছে।

খাতা ৩ · প্রমাণ

দামগুলো কোথা থেকে — তিনটি প্ল্যান, আসল কোড থেকে

`apps/billing/management/commands/seed_plans.py`

প্ল্যান	দাম	শিক্ষার্থী সীমা	স্টাফ সীমা	কার জন্য
Starter	৳১,৫০০/মাস	৩০০	২৫	এক ক্যাম্পাস, ছোট স্কুল
Growth	৳৩,৫০০/মাস	৮০০	সীমা নেই	বড় স্কুল, অনেক শিক্ষক
Campus	৳৬,৫০০/মাস	সীমা নেই	সীমা নেই	মাল্টি-ব্রাঞ্চ + নিজের ডোমেইন

সীমাটা আসে কোথা থেকে?

সীমা কোডে `max_students` ফিল্ডে থাকে — `null` মানে সীমা নেই। তাই নতুন প্ল্যান যোগ করতে **কোড বদলাতে হয় না**, শুধু ডেটা।

কেন ৩০০, ৮০০, ∞?

প্রতিটি সীমা একটা **সিদ্ধান্তের বেড়া**: ৩০০-এর নিচে একজন অ্যাডমিন হাতে সামলাতে পারে; ৮০০ ছাড়ালে আলাদা কোঅর্ডিনেটর দরকার হয়।

ভুলটা কী ছিল

সাবস্ক্রিপশন **ব্যক্তির** নামে বাঁধা ছিল। প্রতিষ্ঠানের সিট কে ব্যবহার করছে — সেই প্রশ্নের উত্তরই ছিল না।

শিক্ষকের নোট: প্রশ্ন করুন — "৩০০ সিটের স্কুল ৩০১-তম ছাত্র ভর্তি করলে কী হবে?" সঠিক উত্তর: সিস্টেম আটকাবে, আর সেই আটকানোটা কোম্পানির আয় — এটা নিষ্ঠুর শোনালেও এটাই ব্যবসার নিয়ম, তাই সীমাটা কোডে নয়, প্ল্যানে থাকে।

খাতা ৪ · প্রমাণ

সময় — আসল টাইমলাইন, অনুমান নয়

২০টি কমিট, ২ কর্মদিবস · `git log` থেকে

দিন ১ — ২৯ অক্টোবর, সকাল

০৬:২১ স্লাইস ১ · ০৬:২৪ CSRF ফিক্স · ০৬:২৮ স্ক্রিনশট
০৬:৩৬ ৫৭ ফিচারের ক্যাটালগ · ০৬:৫২ স্লাইস ২–৫
০৭:০৮ প্যাকেজিং · ০৮:১০ UI সংশোধন

৯ কমিট

৬ দিন

দিন ২ — ৪ নভেম্বর, দুপুর

১৩:২৩ প্রোডাকশন হার্ডেনিং · ১৩:৩৯ কমার্শিয়াল লেয়ার
১৪:০৪ বিলিং বাগ ফিক্স · ১৫:০২ workspace ফিক্স
১৫:৪০–১৭:১২ রিনেম ও ডকুমেন্টেশন

১১ কমিট

৯,৩২২
লাইন Python
(মাইগ্রেশন বাদ দিয়ে)

৩,২৬৫
লাইন HTML (৭৫ টেমপ্লেট)
স্ক্রিনশট প্রমাণসহ

×৪ – ×৬
ছাত্রের জন্য গুণক
৪–৬ সপ্তাহের সমান কাজ

শিক্ষকের নোট: এই "×৪–×৬" গুণকটাই সবচেয়ে দরকারি কথা। অভিজ্ঞ ডেভেলপারের ২ দিন = ছাত্রের ৮–১২ দিন, আর তার সাথে পড়া, পরীক্ষা, অন্য বিষয়। তাই ১২ সপ্তাহকে ফাঁকা ধরে নয়, ৬ সপ্তাহ ধরে প্ল্যান করতে হয়।

খাতা ৪ · পদ্ধতি

সময়ের হিসাব করার তিনটি ধাপ

যে হিসাব "আশা" দিয়ে হয়, সেটা হিসাব নয়

#	ধাপ	কীভাবে	IMS-এ প্রয়োগ
১	**ভাগ করো**	প্রতিটি ফিচারকে ক্ষুদ্রতম "চলমান" টুকরোয় ভাঙো	৫৭ ফিচার → ৫টি স্লাইস, প্রতিটি নিজে নিজে চলে
২	**গুণ করো**	সবচেয়ে ধীর দিনের গতিতে হিসাব করো, সেরা দিনে নয়	স্লাইস ২–৫: ৪ ঘণ্টার কাজ, ধরা হয়েছিল ১ দিন
৩	**বাদ দাও ৪০%**	যা বাদ দিলেও সিস্টেম চলে — সেটা MVP নয়	আমন্ত্রণ, ব্যাকআপ, PDF এক্সপোর্ট → v1.1

ভুল হিসাব: "৫৭ ফিচার ÷ ১২ সপ্তাহ = সপ্তাহে ৫টি" — কাগজে সুন্দর, বাস্তবে অসম্ভব, কারণ ফিচারগুলো সমান নয়।

ঠিক হিসাব: স্লাইসে ভাগ করো, প্রতিটির জন্য **প্রমাণ** ঠিক করো (টেস্ট সবুজ + স্ক্রিনশট), তারপর যোগ করো।

শিক্ষকের নোট: "৪০% বাদ দাও" নিয়মটা ছাত্ররা প্রথমে মানতে চায় না। বলুন — "যে ফিচারটা বাদ দিলে তুমি কষ্ট পাচ্ছ, সেটাই সবচেয়ে ভালো প্রার্থী। কারণ তুমি এটা ভালোবাসছ, ব্যবহারকারী চায়নি।"

সিদ্ধান্ত

তিনটি বাতি — কখন এগোবেন, কখন থামবেন

ফিজিবিলিটির ফলাফল তিন রঙের বেশি হয় না

🟢

এগিয়ে যান

চার খাতার চারটিতেই সংখ্যা আছে, প্রমাণ আছে, আর MVP এক সপ্তাহে দাঁড়ায়।

IMS: খাতা ৪টিই সবুজ

🟡

ছোট করে এগোন

একটি খাতায় সংখ্যা নেই বা সন্দেহ আছে — স্কোপ কাটুন, তারপর এগোন।

উদাহরণ: পেমেন্ট গেটওয়ে বাদ, ম্যানুয়াল বিলিং

🔴

থামুন

দুই বা তার বেশি খাতায় সংখ্যা নেই — অর্থাৎ এখনো প্রজেক্টটা সংজ্ঞায়িতই হয়নি।

চিহ্ন: "ডেটা পরে ভাবব"

সবচেয়ে বিপজ্জনক অবস্থা **হলুদ** — কারণ তখন সবাই এগোতে চায়, আর কেউ স্কোপ কাটতে চায় না।

শিক্ষকের নোট: হলুদ অবস্থায় স্কোপ কাটার সিদ্ধান্তটা একা নেবেন না — ছাত্রকে নিজে বলতে দিন "কোনটা বাদ দেব"। নিজের মুখে বলা কাটছাঁট সে আর ফিরিয়ে আনবে না।

আলোচনা

চারটি প্রশ্ন — নিজের প্রজেক্টের খাতা মিলিয়ে

প্রতিটির উত্তর একটি সংখ্যা দিয়ে শেষ করতে হবে

১. আপনার প্রজেক্টে বছরে কত সারি ডেটা জমবে? সংখ্যাটা লিখুন — sqlite কি পারবে?

২. মাসিক নির্দিষ্ট খরচ কত? প্রতি গ্রাহকের নিট আয় কত? ভাঙার বিন্দু কতজন?

৩. আপনার সিস্টেম "অন্য কারও সার্ভারে" চালু করতে কী কী লাগবে — তিনটি জিনিস বলুন।

৪. কোন খাতায় আপনার হাতের কাছে *কোনো সংখ্যাই* নেই? ওই খাতাটাই আপনার প্রজেক্টের আসল ঝুঁকি।

শিক্ষকের নোট: ৪ নম্বরের উত্তর দিয়ে ক্লাস ৪ শুরু করবেন — ওখান থেকেই ঝুঁকির তালিকা বের হয়, আর ঝুঁকির তালিকা থেকেই কনসেপ্ট পেপারের ৬টি অংশ।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
ফিজিবিলিটি = চারটি খাতার হিসাব, অনুভূতি নয়	প্রযুক্তি · পরিচালনা · অর্থ · সময় — প্রতিটির একটি সংখ্যা
প্রযুক্তি বাছাই প্রয়োজন দিয়ে হয়, স্বাদ দিয়ে নয়	Django ৮/৮ · Flask ৪/৮ · Laravel ৮/৮ কিন্তু কোর্স Python
ছোট ডেটাও এক বছরে বড় হয়	৩০০ শিক্ষার্থী × ৬ পিরিয়ড × ২০০ দিন = ৩,৬০,০০০ সারি
ভাঙার বিন্দু মাত্র দুইজন গ্রাহক	৳২,১০০ নির্দিষ্ট খরচ ÷ ৳১,২১৩ নিট = ১.৭ → ২ জন
অভিজ্ঞ ডেভেলপারের সময় ×৪–×৬	২০ কমিট · ২ কর্মদিবস · ৯,৩২২ লাইন Python

শিক্ষকের নোট: পাঁচটি লাইনেই একটি সংখ্যা আছে — এটাই এই ক্লাসের শর্ত। ছাত্ররা নিজের প্রজেক্টের চারটি সংখ্যা (ডেটা সারি · মাসিক খরচ · ভাঙার বিন্দু · ব্যবধানের গুণক) খাতায় লিখে রাখবে; ক্লাস ৪-এ সেগুলোই কনসেপ্ট পেপারে যাবে।

০৪

পরের ক্লাস — "কী কী ভুল হতে পারে?"

ঝুঁকির তালিকা, আগাম সংকেত, আর কনসেপ্ট পেপারের ছয়টি অংশ

পর্ব ১ · কনসেপ্ট · ক্লাস ৪ / ৩৭

কী কী ভুল হতে পারে?

সিলেবাস: প্র্যাকটিক্যাল ২ · টপিক ২.২–২.৩

ঝুঁকি রেজিস্টার · আগাম সংকেত · কনসেপ্ট পেপারের ছয় অংশ

৬টি ঝুঁকি — প্রতিটি সত্যিই ঘটেছিল

ধাপ ১

ছয়টি ঝুঁকি — প্রতিটির আগাম সংকেত সহ

ঝুঁকি লিখতে হয় তিন কলামে: কী ভুল হতে পারে · আগে থেকে কী দেখে বুঝবেন · কী করবেন

#	ঝুঁকি	আগাম সংকেত	প্রতিকার (প্রজেক্টে যা করা হয়েছে)
১	**মূল ধারণা ভুল বোঝা** কে কার অধীন	"একই প্রশ্নটা দুইবার আসছে"	এক লাইনে লিখে রাখা: **টাকার একক = প্রতিষ্ঠান** (`8a25724`)
২	**পরিবেশ আলাদা** ছাত্রের মেশিনে চলে না	"আমার ল্যাপটপে চলে না"	`run.sh` পাইথন ভার্সন চেক (৩.১২+) + `sqlite` settings (`2cc4e2d`)
৩	**দেরিতে নামকরণ** পরে নাম বদলানো	কোড লেখার *পরে* নাম নিয়ে আলোচনা	নাম `settings`-এ সরানো: `PRODUCT_NAME` (`bfd058e`)
৪	**ডেমো পরিবেশ** লগইন হয়ে আবার লগআউট	প্রিভিউ উইন্ডোতে কুকি থাকে না	কুকি-গার্ড ব্যানার + ভদ্র CSRF পাতা (`e1ead8a`)
৫	**স্কোপ ফুলে যাওয়া**	প্রতিদিন একটা করে নতুন ফিচার	তিন-বাক্স ফ্রিজ + সৎ হিসাব ৪৩/৫৭
৬	**প্রমাণ না রাখা**	"স্ক্রিনশট পরে তুলব"	পরিকল্পনা আর প্রমাণ **আলাদা ফাইল**: `catalogue_data.py` বনাম `build_status.py`

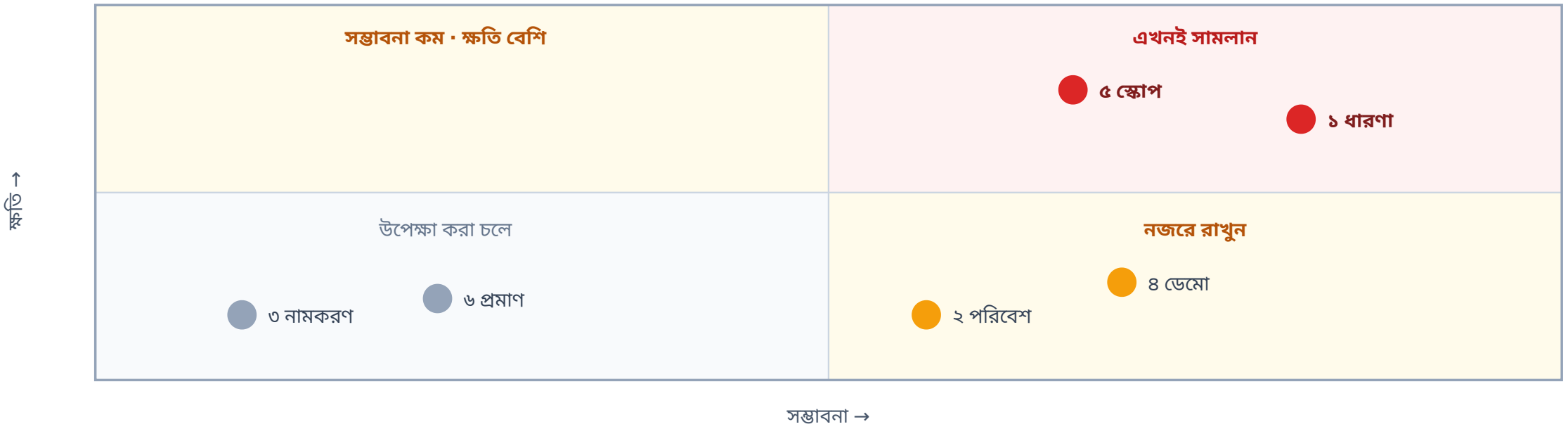
আগাম সংকেত কলামটাই আসল — ঝুঁকি এলে প্রতিকার কাজে লাগে, কিন্তু **সংকেত দেখলে** ঝুঁকিটাই এড়ানো যায়।

শিক্ষকের নোট: এই ছয়টি বানানো ঝুঁকি নয় — প্রতিটির পাশে কমিট নম্বর দেওয়া আছে, অর্থাৎ প্রতিটি সত্যিই ঘটেছিল। ছাত্রদের বলুন নিজের প্রজেক্টে ঠিক এই ছয়টি দিয়ে শুরু করতে; তারপর নিজের মতো দুটি যোগ করতে।

ধাপ ২

কোন ঝুঁকিতে আগে হাত দেবেন — ম্যাপ

সম্ভাবনা × ক্ষতি — উপরে-ডানে থাকলে আগে সামলাতে হবে



সংখ্যাগুলো আগের স্লাইডের ঝুঁকির ক্রম — ১ = ধারণা ভুল বোঝা, ৬ = প্রমাণ না রাখা

১ ও ৫ সবচেয়ে বিপজ্জনক — কারণ এরা ক্ষতি করে **সবচেয়ে দেরিতে** ধরা পড়ে: ধারণা ভুল হলে ৩১ টেবিল ভাঙে, স্কোপ ফুললে শেষ সপ্তাহে।

শিক্ষকের নোট: ছাত্রদের নিজের দুটি ঝুঁকি এই ম্যাপে বসাতে বলুন। সবাই যেটা "উপেক্ষা করা চলে" ঘরে ফেলে, সেটাই সাধারণত পরীক্ষার আগের রাতে ফেটে পড়ে — জিজ্ঞেস করুন কেন ওটা ওখানে ফেলেছে।

ধাপ ৩ · প্রমাণ

যে ঝুঁকিগুলো সত্যিই ঘটেছিল

চারটি ঘটনা, চারটি কমিট — আর প্রতিটির দাম

🍪 ডেমোতে কুকি থাকে না

প্রিভিউ উইন্ডো কুকি রাখতে পারে না → লগইন করে ঢুকলেই আবার লগইন পাতা।

দাম: ১টি আলাদা কমিট (`e1ead8a`) + ভদ্র এরর পাতা

🧾 বিল ভুল জায়গায়

সাবস্ক্রিপশন ব্যক্তির নামে — প্রতিষ্ঠানের সিট কে ব্যবহার করছে, তার উত্তরই নেই।

দাম: ১ মাইগ্রেশন + বিলিং ভিউ + ৪ টেস্ট (`8a25724`)

🏷️ দেরিতে নাম বদল

TMS → IMS। কোডে নাম ছড়িয়ে ছিল; বদলাতে ৩টি কমিট লাগল, তারপর আবার স্ক্রিনশট।

দাম: ৩ কমিট শুধু নামে (`bfd058e` , `7c27e1b` , `86660f3`)

🐍 ছাত্রের পাইথন পুরনো

"চলে না" — কারণ ৩.১০, প্রজেক্ট চায় ৩.১২+। ছয়টি ভিন্ন এরর, একটাই মূল কারণ।

দাম: `run.sh` -এ ভার্সন চেক + স্পষ্ট বার্তা (`2cc4e2d`)

চারটির কোনোটাই "কোডিং ভুল" নয়। চারটিই **সিদ্ধান্ত বা পরিবেশের** ঝুঁকি — তাই এদের ঠিক করতে কোড নয়, **সিদ্ধান্ত** ঠিক করতে হয়েছে।

শিক্ষকের নোট: এখানেই বলুন — "বাগ মেটানো মানে কোড লেখা নয়, আগের সিদ্ধান্ত ভাঙা।" চারটি উদাহরণেই দেখুন: এক লাইন বদলাতে গিয়ে পাঁচ জায়গা বদলাতে হয়েছে।

ধাপ ৪

এখন সব এক পাতায় — কনসেপ্ট পেপারের ছয়টি অংশ

আগের তিন ক্লাসের সব কাজ এই ছয়টি ঘরে ঢোকে

১. সমস্যা ও প্রেক্ষাপট

খাতা কেন যথেষ্ট নয় — তিনটি ক্ষতি

এক লাইনে সমস্যা-বিবৃতি

ক্লাস ১ · স্লাইড ৩

২. ব্যবহারকারী ও ভূমিকা

৫ ভূমিকা · কে কী দেখতে পায় না

ব্যবহারকারী ≠ গ্রাহক

ক্লাস ১ · স্লাইড ৬–৮

৩. স্কোপ

MVP / পরে / কখনো নয়

৫৭ ফিচার, ছয়-লাইনের স্পেক

ক্লাস ২ · স্লাইড ২১–২৩

৪. ফিজিবিলিটি

চারটি খাতা, চারটি সংখ্যা

ভাঙার বিন্দু · সময়ের গুণক

ক্লাস ৩

৫. ঝুঁকি

৬টি ঝুঁকি + আগাম সংকেত

ম্যাপে কোথায় দাঁড়িয়ে

ক্লাস ৪ · এই স্লাইডের আগের তিনটি

৬. ডেলিভারি পরিকল্পনা

৫টি স্লাইস · প্রতিটির প্রমাণ

কোন সপ্তাহে কী সবুজ হবে

ক্লাস ২ · স্লাইড ২৭

নিয়ম: **ছয়টি অংশ, সর্বোচ্চ দুই পাতা।** যেটা দুই পাতায় ধরছে না — সেটা এখনো ভাবা হয়নি, শুধু লেখা হয়েছে।

শিক্ষকের নোট: এই ছয়টি ঘর বোর্ডে আঁকুন, প্রতিটির নিচে ক্লাস নম্বর লিখুন। ছাত্ররা বুঝবে — পেপারটা নতুন কিছু নয়, গত তিন ক্লাসের কাজের সাজানো রূপ।

অংশ ১–২

সমস্যা আর মানুষ — দুই পাতা, কোনও ভূমিকা নয়

লেখার নিয়ম: প্রতিটি বাক্যে একটা সংখ্যা বা একটা নাম থাকতে হবে

১. সমস্যা ও প্রেক্ষাপট

একটি মাধ্যমিক স্কুলে **৩০০ শিক্ষার্থী**, **২৫ শিক্ষক**। উপস্থিতি ও নম্বর খাতায় লেখা হয়। ফলে — মাসিক রিপোর্টে **কয়েক ঘণ্টা** লাগে, এক বছর পেছনের তথ্য **খোঁজা যায় না**, আর কে নম্বর বদলাল তার **কোনো রেকর্ড থাকে না**।

এক লাইনে: তথ্য একবার লেখা হবে, অনেকবার কাজে লাগবে — আর প্রতিটি বদলের দায় থাকবে।

ভুল লেখা: "এটি একটি আধুনিক, ব্যবহারবান্ধব, স্কেলেবল সিস্টেম।" — এতে কোনও সংখ্যা নেই, কোনও নাম নেই, তাই কিছুই বলা হয়নি।

২. ব্যবহারকারী ও ভূমিকা

প্রশাসক	সব প্রতিষ্ঠান, প্ল্যান	ছাত্রের নম্বর দেখে না
অ্যাডমিন	নিজের প্রতিষ্ঠান	অন্য প্রতিষ্ঠান দেখে না
কোঅর্ডিনেটর	নিজের বিভাগ	বিল দেখে না
শিক্ষক	নিজের ক্লাস	অন্য ক্লাসের নম্বর না
শিক্ষার্থী	শুধু নিজের	অন্য কারও কিছু না

গ্রাহক ≠ ব্যবহারকারী: টাকা দেয় প্রতিষ্ঠান, ব্যবহার করে ২৫ জন শিক্ষক।

শিক্ষকের নোট: "ভুল লেখা" বাক্সটা মুখস্থ করান — "আধুনিক, ব্যবহারবান্ধব, স্কেলেবল" এই তিন শব্দ পেলেই লাল পতাকা। কোনও শিক্ষক এভাবে নিজের প্রজেক্টের পরিচয় লিখলে সেটা পেপারের প্রথম বাক্য হতে পারে না।

অংশ ৩–৪

স্কোপ আর সংখ্যা — দুই পাতা

স্কোপে তালিকা, ফিজিবিলিটিতে সংখ্যা — দুইটাই লাগবে

৩. স্কোপ

MVP (৫টি স্লাইস): লগইন ও ভূমিকা · প্রতিষ্ঠান · শিক্ষার্থী ও ভর্তি · উপস্থিতি · নম্বর ও রিপোর্ট

v1.1: আমন্ত্রণ · ব্যাকআপ · পূর্ণ এক্সপোর্ট · নোটিফিকেশন

কখনো নয়: SMS গেটওয়ে · মোবাইল অ্যাপ · AI উপস্থিতি

মোট: ৫৭ ফিচার · ৬ ভূমিকা · ৩১ টেবিল — প্রতিটির ID (F-01 … F-57)

৪. ফিজিবিলিটি — চারটি সংখ্যা

প্রযুক্তি	Django 6.1.2 / Python 3.13
ডেটা	৩০০ × ৬ × ২০০ = ৩,৬০,০০০ সারি/বছর
খরচ	৳২,১০০/মাস → ভাঙার বিন্দু ২ গ্রাহক
সময়	৫ স্লাইস · অভিজ্ঞ ২ দিন · ছাত্র ×৪–×৬

সিদ্ধান্ত: 🟢 এগিয়ে যাওয়া — চার খাতাতেই সংখ্যা আছে, প্রতিটির প্রমাণ আছে।

স্কোপে **সংখ্যা** থাকলে সেটা মাপা যায়; "বড় সিস্টেম" লেখা থাকলে কিছুই মাপা যায় না।

শিক্ষকের নোট: ফিজিবিলিটির ঘরে "সিদ্ধান্ত" লাইনটা বাদ দেবেন না — সংখ্যা থাকলেও সিদ্ধান্ত না লিখলে পেপারটা পড়ে কেউ জানবে না আপনি এগোচ্ছেন কি থামছেন।

অংশ ৫–৬

ঝুঁকি আর পরিকল্পনা — শেষ দুই পাতা

ঝুঁকিতে সংকেত, পরিকল্পনায় তারিখ ও প্রমাণ

৫. ঝুঁকি (৬টি)

১ ধারণা ভুল	"একই প্রশ্ন দুইবার"
২ পরিবেশ	"আমার মেশিনে চলে না"
৩ দেরিতে নাম	কোডের পরে নাম-আলোচনা
৪ ডেমো পরিবেশ	লগইন হয়ে আবার লগআউট
৫ স্কোপ ফোলা	প্রতিদিন নতুন ফিচার
৬ প্রমাণ নেই	"পরে স্ক্রিনশট তুলব"

৬. ডেলিভারি পরিকল্পনা

সপ্তাহ	কী হবে	প্রমাণ
১–২	লগইন · প্রতিষ্ঠান · ভূমিকা	১৩ টেস্ট
৩	৫৭ ফিচারের তালিকা	ক্যাটালগ ফাইল
৪–৭	ছাত্র · উপস্থিতি · নম্বর · রিপোর্ট	৪৩ টেস্ট · ৪৮ পর্দা × ৪ ভূমিকা
৮	প্যাকেজিং · এক কমান্ডে চালু	`run.sh`
৯–১০	প্রোডাকশন মহড়া	`deploy_check.sh`
১১–১২	ডকুমেন্ট · ডেমো	ম্যানুয়াল · স্ক্রিনশট

পরিকল্পনার প্রতিটি সারিতে **"প্রমাণ"** কলাম থাকতেই হবে। যে সারিতে প্রমাণ লেখা যায় না, সেটা কাজ নয় — ইচ্ছা।

শিক্ষকের নোট: এই টেবিলটাই ছাত্রদের নিজের প্রজেক্টে পূরণ করতে দেবেন। সবচেয়ে কঠিন প্রশ্ন: "৮ম সপ্তাহে তোমার হাতে কী প্রমাণ থাকবে?" উত্তর না এলে বাকি পরিকল্পনাও বদলাতে হবে।

ফলাফল

কনসেপ্ট পেপার — এক পাতায় পুরো প্রজেক্ট

এটাই সেই কাগজ যা নিয়ে কেউ প্রশ্ন করলে ৩০ সেকেন্ডে উত্তর দেওয়া যায়

IMS — Institute Management System

কনসেপ্ট পেপার v1.0 · ১ পাতা · ২০২৬

সমস্যা
খাতায় উপস্থিতি ও নম্বর → রিপোর্টে ঘণ্টা, পেছনের তথ্য নেই, দায় নেই।

ব্যবহারকারী
৫ ভূমিকা: প্রশাসক · অ্যাডমিন · কোঅর্ডিনেটর · শিক্ষক · শিক্ষার্থী। গ্রাহক = প্রতিষ্ঠান।

স্কোপ
৫৭ ফিচার · MVP ৫ স্লাইস · v1.1 তে ৪টি · কখনো নয়: SMS, অ্যাপ, AI।

ফিজিবিলিটি
Django 6.1.2 · ৩.৬ লাখ সারি/বছর · ৳২,১০০/মাস · ভাঙার বিন্দু ২ গ্রাহক।

ঝুঁকি
৬টি — ধারণা ভুল, পরিবেশ, দেরিতে নাম, ডেমো কুকি, স্কোপ, প্রমাণ।

পরিকল্পনা
১২ সপ্তাহ · ৫ স্লাইস · প্রতিটির শেষে টেস্ট সবুজ + স্ক্রিনশট।

আজকের অবস্থা: ৫৪/৫৭ ফিচার তৈরি (৪৩টি স্ক্রিনশটসহ) · ১০৩ টেস্ট · ১৯ কমিট · ৩১ টেবিল · ৭৫ টেমপ্লেট

পরীক্ষা: এই পাতাটা **একজন অপরিচিত মানুষকে** দিন — সে কি ৩০ সেকেন্ডে বলতে পারবে কী বানাচ্ছেন, কার জন্য, কত সময়ে?

শিক্ষকের নোট: ক্লাসে সত্যিই এই পরীক্ষাটা করুন — এক ছাত্রের পেপার আরেক ছাত্রকে পড়তে দিন, ৩০ সেকেন্ড গুনুন। পারলে পেপার ভালো, না পারলে পেপার সুন্দর।

ক্লাসের কাজ

নিজের পেপার লেখা — পাঁচটি ধাপ, ২৫ মিনিট

আজ শেষ না হলেও চলবে; কিন্তু প্রতিটি ঘরে অন্তত একটা সংখ্যা থাকতে হবে

১. ছয়টি ঘর আঁকুন

কাগজে ছয়টি বাক্স — আগে ভরবেন না, শুধু আঁকুন। খালি ঘর দেখলে মাথা কাজ করে।

২. সমস্যা এক লাইনে

কে, কতজন, কী হারাচ্ছে — তিনটে তথ্য একটি বাক্যে।

৩. ভূমিকা তালিকা

প্রতিটি ভূমিকার পাশে লিখুন **সে কী দেখতে পাবে না** — এটাই পারমিশনের নকশা।

৪. স্কোপ তিন বাক্স

MVP-তে সর্বোচ্চ ৫টি, "পরে"-তে ৩টি, "কখনো নয়"-তে ২টি।

৫. ঝুঁকি ৬ + সংকেত

আগের স্লাইডের ছয়টি থেকে শুরু করুন, তারপর নিজের দুটি যোগ করুন।

আলোচনার প্রশ্ন (হাতে তুলুন): আপনার ছয়টি ঘরের কোনটায় এখনো **একটিও সংখ্যা নেই**? ওই ঘরটাই আপনার প্রজেক্টে সবচেয়ে দেরিতে ফেটে পড়বে।

শিক্ষকের নোট: ২৫ মিনিট শেষে ২–৩ জনের পেপার বোর্ডে টাঙিয়ে একসাথে দেখুন। যার খালি ঘর সবচেয়ে বেশি, তার পেপারটাই সবচেয়ে বেশি শেখাবে — সেটাই বেছে নিন।

০৫

পর্ব ১ শেষ — পর্ব ২: পরিকল্পনা

ক্লাস ৫ থেকে: প্রয়োজন চিহ্নিত করা, ব্যবহারকারীর গল্প, আর গ্রহণযোগ্যতার শর্ত

পর্ব ২ · ইঞ্জিনিয়ারিং প্ল্যান · ক্লাস ৫ / ৩৭

কী চান — সেটা জিজ্ঞেস করবেন না

সিলেবাস: প্র্যাকটিক্যাল ৩ · টপিক ৩.১ (রিকোয়্যারমেন্ট এলিসিটেশন)

ইন্টারভিউ · প্রশ্ন-সিঁড়ি · কাঁচা কথা থেকে ফিচার

৫ ভূমিকা · ১৫ সেশন · ৫৭ ফিচার

শুরু করুন প্রশ্ন দিয়ে

"আপনার কী কী ফিচার দরকার?" — এই প্রশ্নটাই ভুল

হাত তুলুন: আপনার প্রজেক্টে ব্যবহারকারীকে কী জিজ্ঞেস করবেন বলে ভেবেছেন?

মানুষ নিজের কাজ জানে।
সফটওয়্যারের সমাধান সে জানে না — সেটা আপনার কাজ।

"ফিচার কী চান" জিজ্ঞেস করলে উত্তর আসে **নকল সমাধান** — "আমার আগের সফটওয়্যারে যেটা ছিল", "একটা এক্সেল এক্সপোর্ট হলেই হয়"।
তাই প্রশ্নটা ঘোরাতে হয় কাজের দিকে: **"গত সোমবার কী করলেন, কোথায় আটকালেন?"**

শিক্ষকের নোট: কেউ "হ্যাঁ আগের সফটওয়্যারটাই চাই" বললে জিজ্ঞেস করুন — "আগেরটাতে সবচেয়ে বিরক্তিকর কী ছিল?" এই এক প্রশ্নেই আসল প্রয়োজন বেরিয়ে আসে, কারণ মানুষ অভিযোগ বলতে ভালোবাসে।

ধাপ ১

ইন্টারভিউয়ের তিনটি নিয়ম

১৫ মিনিটের ইন্টারভিউ — এই তিনটি না মানলে ফল শূন্য

১. গতকালের কথা বলান

"সাধারণত কী করেন" নয় — **"গত সোমবার সকালে ঠিক কী করলেন"**। নির্দিষ্ট দিন মনে করলে বানানো উত্তর কম আসে।

২. সমাধান নয়, কারণ

"কেন" তিনবার জিজ্ঞেস করুন। প্রথম "কেন"-এর উত্তর সাধারণত অজুহাত, তৃতীয়টায় আসল কারণ।

৩. যা নেই সেটাও জিজ্ঞেস করুন

"কোন তথ্য আপনি **দেখতে পান না**?" — পারমিশনের নকশা আসে এই প্রশ্ন থেকেই, ফিচার-তালিকা থেকে নয়।

ইন্টারভিউয়ের লক্ষ্য **তালিকা নয়** — লক্ষ্য হলো **একটা বাক্য**, যা দিয়ে আপনি তালিকাটা নিজে বানাতে পারবেন।

শিক্ষকের নোট: ২ নম্বর নিয়মটা ক্লাসে অভিনয় করে দেখান — একজন "শিক্ষক", আপনি তিনবার "কেন?" জিজ্ঞেস করছেন। তৃতীয় উত্তরেই সাধারণত "কারণ ভুল হলে দায় আমার ঘাড়ে পড়ে" — এটাই আসল প্রয়োজন, আর সেটা থেকে জন্মায় অডিট লগ।

ধাপ ২

প্রশ্ন-সিঁড়ি — পাঁচটি ধাপ, একই দিকে

প্রতিটি ধাপ আগেরটার উপর দাঁড়ায়; সিঁড়ি লাফ দিলে উত্তরও ফাঁকা আসে

১. কাজ
"গতকাল কী করলেন?"

২. বাধা
"কোথায় আটকালেন?"
"কত সময় লাগল?"

৩. কারণ
"কেন এটা সমস্যা?"
"কার ক্ষতি হয়?"
"কী হলে ঠিক হতো?"

৪. ব্যতিক্রম
"কখন নিয়ম ভাঙে?"
"ছুটির দিন?"
"দুই শিক্ষক একসাথে?"
"কেউ ভুল করলে?"

৫. শেষবার
"শেষ কখন ঘটেছে?"
"তখন কী করলেন?"
"কে টের পেল?"
"তা হলে করলেন কী?"

→ ফিচার নয়, প্রয়োজন বেরোয়

৪ নম্বর ধাপটাই সবচেয়ে বেশি বাদ পড়ে — অথচ ব্যতিক্রম থেকেই আসে "আজ নেওয়া হয়ে গেছে → এডিট মোড", "ভবিষ্যৎ তারিখ বন্ধ", "ছুটির দিনে সতর্কবার্তা"।

শিক্ষকের নোট: এই পাঁচটি ধাপ বোর্ডে লিখে দিন। ক্লাসের কাজে প্রতিটি জোড়া অন্তত সিঁড়ির ৪ ও ৫ নম্বর ধাপ পর্যন্ত যাবে — না গেলে ইন্টারভিউ অর্ধেক হয়েছে।

ধাপ ৩

পাঁচ ভূমিকা, চারটি আলাদা প্রশ্ন

একই প্রশ্ন সবাইকে করলে তিনজনের উত্তর নষ্ট হয়

ভূমিকা	প্রতিদিনের কাজ (জানতে হবে)	সবচেয়ে বড় ভয় (জানতে হবে)	যা দেখতে পায় না (জানতে হবে)	কতজনকে
শিক্ষক	হাজিরা · সিলেবাস · হোমওয়ার্ক · নম্বর	"ভুল হলে দায় আমার"	অন্য ক্লাসের নম্বর	৩–৫ জন
প্রতিষ্ঠান অ্যাডমিন	কাঠামো · শিক্ষক · ভর্তি · গ্রেডিং	"কেউ নিয়ম না মানলে?"	—	১–২ জন
কোঅর্ডিনেটর	নিজের বিভাগের দেখাশোনা	"কে হাজিরা নেয়নি জানি না"	বিল · অডিট লগ · সেটিংস	১–২ জন
শিক্ষার্থী	হোমওয়ার্ক দেখা · ফলাফল	"ফলাফল কখন আসবে"	অন্য কারও কিছু	২–৩ জন
প্রশাসক (প্ল্যাটফর্ম)	প্রতিষ্ঠান চালু · প্ল্যান	"ভুল প্রতিষ্ঠানে ডেটা"	ছাত্রের নম্বর	১ জন

মোট **৮–১৩টি ইন্টারভিউ**। এর কমে গেলে আপনি একজনের মতকে পুরো প্রতিষ্ঠানের মত ভাবছেন — এই ভুলটার নাম **নমুনা-পক্ষপাত**।

শিক্ষকের নোট: "যা দেখতে পায় না" কলামটাই এই টেবিলের সম্পদ। ছাত্ররা সাধারণত শুধু কাজ আর ভয় জিজ্ঞেস করে; তৃতীয় কলাম বাদ দিলে প্রতিটি পেজের পারমিশন পরে ঠিক করতে হবে — অর্থাৎ ক্লাস ৯-এ আবার সব করতে হবে।

ধাপ ৩ · স্ক্রিপ্ট

১৫ মিনিটের ইন্টারভিউ — সময়ের ভাগ

সময় না ভাগ করলে ১৫ মিনিটে শুধু ভূমিকা বলেই শেষ হয়ে যাবে

০–২ মিনিট
ভূমিকা
"গোপন রাখব"

২–৯ মিনিট · ৭ মিনিট
সিঁড়ির ১–৩ ধাপ: কাজ · বাধা · কারণ
গতকালের একটা দিন ধরে হাঁটুন

৯–১৩ মিনিট · ৪ মিনিট
ধাপ ৪–৫: ব্যতিক্রম · শেষবার
এখানেই আসল ফিচার বেরোয়

১৩–১৫ মিনিট
"যা দেখতে পান না?"
+ ধন্যবাদ · পরের প্রশ্ন

ভাগটা মন দিয়ে দেখুন: অর্ধেকের বেশি সময় যায় প্রথম তিন ধাপে, কিন্তু ফিচারের ৭০% আসে শেষ দুই ধাপ থেকে।
তাই ব্যতিক্রমের সময় কখনো কাটবেন না — ওখানেই ব্যবহারকারী নিজের সিস্টেম বানিয়ে ফেলেছেন।

শেষ ২ মিনিটে সবসময় একই প্রশ্ন: "আর কিছু আছে যা আমি জিজ্ঞেস করিনি?" — এখানে সবচেয়ে অপ্রত্যাশিত উত্তরটা আসে।

শিক্ষকের নোট: ক্লাসের কাজে এই ভাগটাই ব্যবহার করান — ১৫ মিনিটের টাইমার দিয়ে, প্রতিটি জোড়া একে অন্যকে ইন্টারভিউ করবে। সময় পার হলে থামিয়ে দিন; এতে সময়ের মূল্য বোঝা যাবে।

উদাহরণ ১

শিক্ষকের ইন্টারভিউ — পাঁচটি কাজ, একটা ক্রম

উত্তর থেকে সরাসরি তৈরি হয়েছে ম্যানুয়ালের অধ্যায় ৩

শিক্ষক বললেন (সারাংশ)

"সকালে ক্লাসে ঢুকে প্রথমে হাজিরা। তারপর সিলেবাস — কোন অধ্যায় পর্যন্ত গেছি মনে থাকে না। দিনে কী পড়ালাম লিখে রাখি খাতায়। হোমওয়ার্ক দিই, কে জমা দিল দেখি। মাস শেষে নম্বর তোলা — ওখানেই সপ্তাহ পার।"

ভয়: "নম্বর ভুল হলে অভিভাবক আমার সাথে কথা বলে।"

যা তৈরি হলো

উপস্থিতি	F-31 · F-32 · F-33 · F-34
সিলেবাস	F-36 · F-37 · F-38
হোমওয়ার্ক	F-39 · F-41 · F-42
পরীক্ষা	F-43 · F-44 · F-46 · F-47
রিপোর্ট	F-50 · F-51 · F-52

৫টি বাক্য → **২১টি ফিচার**। কারণ প্রতিটি দৈনন্দিন কাজের তিন-চারটি ধাপ থাকে।

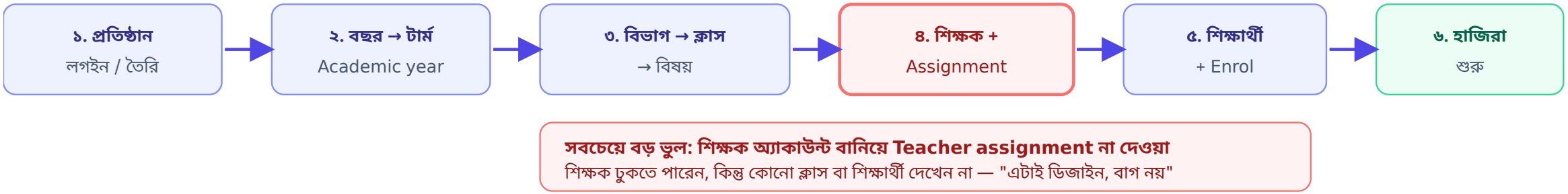
খেয়াল করুন: উত্তরে "সফটওয়্যার" শব্দটাই আসেনি। এসেছে **কাজ আর ভয়** — সেখান থেকেই ফিচার বেরোয়।

শিক্ষকের নোট: "ফলাফল প্রকাশের আগে অভিভাবক জানতে পারে না" — এই একটি ভয় থেকেই F-47 (publish) আর F-48 (শিক্ষার্থী শুধু publish-এর পরে দেখবে) এসেছে। ছাত্রদের বলুন: ভয় থেকে ফিচার বানানো easiest win।

উদাহরণ ২

অ্যাডমিনের ইন্টারভিউ — প্রথম দিনের ক্রম

"নতুন স্কুলে ঢুকে প্রথমে কী করবেন?" — উত্তরটাই সেটআপের নকশা



ইন্টারভিউয়ের সবচেয়ে দামি উত্তর প্রায়ই **"নতুনরা এখানে আটকায়"** — কারণ সেটা একসাথে দুটো জিনিস দেয়: সেটআপের ক্রম, আর একটা খালি-পর্দার ব্যাখ্যা।

শিক্ষকের নোট: এই ছয় ধাপ হুবহু ব্যবহারকারী ম্যানুয়ালের "প্রথম দিনের ক্রম" থেকে। ছাত্রদের বলুন নিজের প্রজেক্টে এমন একটা ছয়-ধাপের ক্রম লিখতে — না লিখলে ডেমোর দিনে নিজেই আটকে যাবে।

উদাহরণ ৩

কোঅর্ডিনেটরের ইন্টারভিউ — "আমি কী, না কে"

প্রশ্নের উত্তর থেকে জন্ম নিল একটা সম্পূর্ণ ভূমিকা

প্রশ্ন: "আপনি কে?"

প্রথম উত্তর: "আমি শিক্ষক, সাথে বিভাগের দেখাশোনা করি।"

দ্বিতীয় প্রশ্ন: "আপনার আর দশজন শিক্ষকের সাথে তফাত কী?"

আসল উত্তর: "আমি দেখি কে হাজিরা নিয়েছে, কে নেয়নি। কিন্তু বিল বা সেটিংসে আমার কিছু করার নেই।"

সিদ্ধান্ত: কোঅর্ডিনেটর পৃথক প্রতিষ্ঠান নয় — **একটা পদ**, যা অ্যাডমিন বসান।

"যা আপনার নেই" — সবচেয়ে দামি তালিকা

যা নেই	কার জন্য	কারণ
Audit log	অ্যাডমিন	পুরো প্রতিষ্ঠানের নিরাপত্তা-লগ
Workspace settings	অ্যাডমিন	কাঠামো বদলানোর অধিকার
Grading	অ্যাডমিন	স্কেল পুরো প্রতিষ্ঠানের
Coordinator scope	অ্যাডমিন	নিজের ক্ষমতা নিজে বাড়ানো যাবে না
Branding · বিল	অ্যাডমিন	পরিচয় ও টাকা প্রতিষ্ঠানের সিদ্ধান্ত

এই পাঁচটি লাইন থেকেই তৈরি হলো **পারমিশন ম্যাট্রিক্স** — শুধু "কে কী পারে" নয়, **"কে কী পারবে না, আর কেন"**।

শিক্ষকের নোট: সম্মানের সূক্ষ্মতা: "Coordinator scope" পাতা কোঅর্ডিনেটরের মেনুতেই নেই — শুধু লুকানো নয়, সার্ভারেও আটকায়। এই দ্বৈত সুরক্ষা ক্লাস ৯ ও পর্ব ৪-এ আবার আসবে।

ধাপ ৪

কাঁচা কথা → ফিচার আইডি: পাঁচটি আসল রূপান্তর

বাঁ দিকে ব্যবহারকারীর বাক্য, ডানে যা তৈরি হলো

ব্যবহারকারী যা বললেন	ফিচার	কীভাবে বদলালো
"সকালে পুরো ক্লাসের হাজিরা লিখতে ২০ মিনিট লাগে, মোবাইল থেকে লিখি"	**F-31**	বাল্ক বোতাম (সবাই উপস্থিত → যাদের অনুপস্থিত শুধু বদলান) + মোবাইল লেআউট
"একই দিনে দুইবার সেভ চাপলে ডুপ্লিকেট হয়ে যায়"	**F-31**	`unique(student, class, subject, date)` + দ্বিতীয়বার হলে **এডিট**
"কে হাজিরা নেয়নি — মাসের শেষে জানতে চাই"	**F-35**	উপস্থিতি **পর্যবেক্ষণ** পর্দা — কোঅর্ডিনেটরের নিজের, শিক্ষকের নয়
"ফলাফল কখন দেবে কিছু জানি না, বারবার দেখতে আসি"	**F-47 · F-48**	**প্রকাশ** একটা আলাদা কাজ; শিক্ষার্থী কেবল প্রকাশের পরেই দেখবে
"ভুল হলে কে বদলাল, জানতে চাই"	**F-54**	সংশোধনে **কারণ** বাধ্যতামূলক → অডিট লগে থেকে যায়

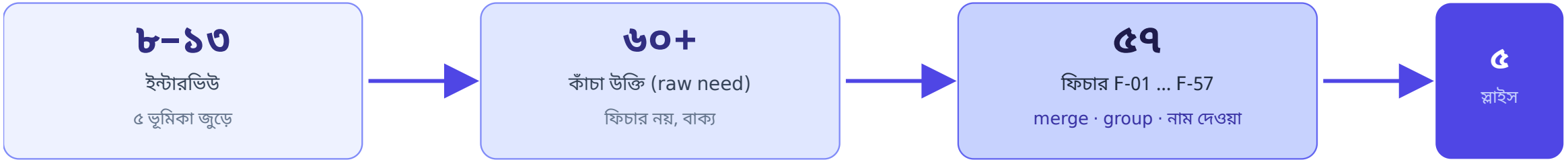
নিয়ম: প্রতিটি ফিচারের পাশে **যে বাক্যটা থেকে এসেছে** সেটা লিখে রাখুন। পরে "এই ফিচারটা কে চেয়েছিল?" প্রশ্নের উত্তর থাকবে।

শিক্ষকের নোট: দ্বিতীয় সারিটা সবচেয়ে শিক্ষণীয় — ব্যবহারকারী "বাগ" বলেছেন, আমরা "ডিজাইন" দিয়েছি (unique + এডিট)। ছাত্রদের জিজ্ঞেস করুন: এখানে বাগটা আসলে কোথায় ছিল? উত্তর: ডেটাবেসে নিয়ম ছিল না, শুধু ফর্মে ছিল।

ধাপ ৫

১৫ সেশন → ৫৭ ফিচার

ইন্টারভিউয়ের সংখ্যা আর ফিচারের সংখ্যা সমান হয় না — কারণ ব্যবহারকারী ভাগে ভাগে ভাবেন



মাঝের ধাপে তিনটি কাজ: একই কথা একত্র করা · ধাপগুলো দলে ভাগ করা · প্রতিটিকে নাম ও আইডি দেওয়া

৬০+ কাঁচা উক্তি থেকে ৫৭ ফিচার — অর্থাৎ প্রায় প্রতিটি উক্তিই একটা ফিচার, কিন্তু তার আগে ২০টি ডুপ্লিকেট বাদ দিতে হয়েছে

> ডুপ্লিকেট বাদ দেওয়াই সবচেয়ে বড় কাজ — পাঁচজন শিক্ষক পাঁচভাবে একই কথা বলেন; সেটা **একটা** ফিচার, পাঁচটা নয়।

শিক্ষকের নোট: বোর্ডে দুটি সংখ্যা লিখুন — কাঁচা উক্তি আর ফিচার। ছাত্ররা দেখবে দুটো প্রায় সমান, কিন্তু ফিচারের তালিকায় প্রতিটি লাইন একবারই আছে।

সতর্কতা

যে পাঁচটি প্রশ্ন করবেন না

বাঁ দিকের প্রশ্নে উত্তর আসে "হ্যাঁ", ডান দিকের প্রশ্নে আসে গল্প

× ভুল প্রশ্ন	✓ ঠিক প্রশ্ন	কারণ
"আপনার কি ড্যাশবোর্ড দরকার?"	"সকালে ঢুকে প্রথমে কী দেখতে চান?"	প্রথমটা সমাধান চাপিয়ে দেয়
"আপনি কি মোবাইলে ব্যবহার করবেন?"	"শেষবার কখন মোবাইল থেকে কাজ করেছেন? কী করছিলেন?"	দ্বিতীয়টায় আসল প্রেক্ষাপট
"আপনি কি অডিট লগ চান?"	"শেষ কখন কেউ নম্বর বদলেছে টের পেয়েছেন?"	প্রয়োজন থাকলে নিজেই বলবেন
"এটা কি ভালো হবে না?"	"এটা হলে আপনার কাজ কমবে না বাড়বে?"	সম্মতির চাপ এড়ানো
"আপনার সমস্যা কী?"	"গতকাল কোন কাজটা সবচেয়ে বেশি সময় নিল?"	"সমস্যা" জিজ্ঞেস করলে সাধারণ অভিযোগ আসে

ভদ্রতা একটা ফাঁদ: মানুষ **না বলতে চায় না**। তাই "দরকার?" প্রশ্নের উত্তর কখনো "না" আসে না — আর সেই "হ্যাঁ"-এর পেছনে কোনো কাজ থাকে না।

শিক্ষকের নোট: ক্লাসে এক জোড়া দিয়ে দুটো প্রশ্নই করিয়ে দেখান — পার্থক্যটা নিজের কানে শুনলে আর ভুলবে না।

ধাপ ৬

নোট রাখার ফরম্যাট — পাঁচ কলাম

ইন্টারভিউ শেষে ১০ মিনিটেই লিখে ফেলুন, নইলে স্মৃতি বানিয়ে বলবে

তারিখ	কে	উক্তি (হুবহু)	দেখা সমস্যা	সম্ভাব্য ফিচার
২৮/১০	শিক্ষক · রফিক	"২০ মিনিট লাগে, মোবাইলে ভুল চাপ পড়ে"	হাজিরা ধীর · মোবাইলে ছোট বোতাম	F-31 বাল্ক + মোবাইল
২৮/১০	শিক্ষক · রফিক	"দুইবার সেভ করলে দুটো রেকর্ড"	ডেটাবেসে নিয়ম নেই	F-31 unique + এডিট
২৮/১০	কোঅর্ডিনেটর · সুমি	"কে নেয়নি জানতে মাস পার হয়"	পর্যবেক্ষণের পর্দা নেই	F-35 oversight
২৯/১০	শিক্ষার্থী · নীলা	"ফল কখন আসবে জানি না"	প্রকাশের অবস্থা দৃশ্যমান নয়	F-47 → F-48
২৯/১০	অ্যাডমিন · হাসান	"কে বদলাল সেটা জানা দরকার"	কোনো রেকর্ড থাকে না	F-54 audit log

হুবহু কেন: আপনার ভাষায় লিখলে পরে নিজেই নিজের অনুমান পড়বেন, ব্যবহারকারীর কথা নয়।

"সম্ভাব্য" শব্দটা জরুরি: সব উক্তি ফিচার হয় না — ইন্টারভিউয়ে সব নয়, পরে ছাঁটবেন।

শিক্ষকের নোট: এই পাঁচ কলামের খাতা ক্লাস ৬-এ টানা হবে — সেখানেই "সম্ভাব্য ফিচার" কলাম থেকে ৫৭ ফাংশনের ক্যাটালগ দাঁড়াবে।

দক্ষতা

উত্তর শুনে যা করতে হয় — তিনটি ছাঁকনি

ইন্টারভিউ শেষ, এখন কাঁচা কথা ছেঁকে ফিচার বানানো

🔁

১. একত্র করা

পাঁচজন শিক্ষকের "হাজিরা ধীর" — **একটাই** ফিচার। প্রতিষ্ঠানভেদে ভিন্ন হলে **সেটিং**, ফিচার নয়।

✂️

২. আলাদা করা

"ফলাফল দেখতে চাই" আসলে তিনটি কাজ: **হিসাব** · **প্রকাশ** · **দেখা**। তাই তিনটি ফিচার, একটা নয়।

🚫

৩. বাদ দেওয়া

"SMS পাঠান" — খরচ আছে, কিন্তু MVP-তে দরকার নেই। **"কখনো নয়"** বাক্সে যাবে, কারণ লিখে রাখতে হবে।

পরীক্ষা: প্রতিটি ফিচারকে **একজন ব্যবহারকারীর এক বাক্যে** বলা যায় কি? না গেলে ফিচারটা তখনো দুটো ফিচার — ভাঙুন।

শিক্ষকের নোট: "আলাদা করা" নিয়মটা ক্লাস ৬-এর ভিত্তি — একটা উক্তি থেকে তিনটি ফিচার বেরোলে তিনটির আইডি আলাদা হবে, আর স্লাইসে তিনটি আলাদা সময়ে তৈরি হবে।

বাস্তব

ইন্টারভিউ যেখানে সত্যিই বাধে — চারটি ঘটনা

প্রজেক্টের ফাইল থেকে, অনুমান নয়

পরিস্থিতি	যা ধরা পড়ল	যা করা হলো
শিক্ষক অ্যাকাউন্ট খুলে ঢুকলেন, কিছু নেই	Assignment ছাড়া ভূমিকা অর্থহীন	সেটআপের ক্রম ম্যানুয়ালে ৬ ধাপে লেখা · খালি পর্দায় ব্যাখ্যা
কোঅর্ডিনেটর নিজের ক্ষমতা বাড়াতে চাইলেন	পদটি প্রতিষ্ঠানভিত্তিক, ব্যক্তিভিত্তিক নয়	"Coordinator scope" পাতা শুধু অ্যাডমিনের, সার্ভারেও আটকায়
শিক্ষক দুই প্রতিষ্ঠানে পড়ান	বিল কার নামে?	সাবস্ক্রিপশন প্রতিষ্ঠানে (`8a25724`), শিক্ষক শুধু সদস্য
এক প্রতিষ্ঠান নিজের ডোমেইন চাইল	ভুল ডোমেইনে অন্য প্রতিষ্ঠানের লোগো যেতে পারে	নিজে সেট করার অধিকার নেই — প্ল্যাটফর্ম টিম করে

চারটিরই শুরু **একটা প্রশ্নে**, একটা ফিচার-অনুরোধে নয়। তাই ইন্টারভিউয়ের ফল ফিচার-তালিকা নয় — **নিয়মের তালিকা**ও।

শিক্ষকের নোট: শেষ সারিটা সবচেয়ে অপ্রত্যাশিত — "কাস্টম ডোমেইন" শুনতে ফিচার মনে হয়, আসলে নিরাপত্তার সিদ্ধান্ত। ছাত্রদের বলুন: ব্যবহারকারীর অনুরোধ শুনে প্রথম প্রশ্ন "এটা কার ক্ষতি করতে পারে?"

আলোচনা

চারটি প্রশ্ন — ইন্টারভিউয়ের আগে

উত্তর দিয়ে আজকের ক্লাস শেষ করুন

১. আপনার প্রজেক্টে সবচেয়ে কম **৩ জন** কে ইন্টারভিউ করবেন? নাম লিখুন — নাম না লিখলে কখনো হবে না।

২. সিঁড়ির ৪ ও ৫ নম্বর ধাপের (ব্যতিক্রম, শেষবার) জন্য আপনি ঠিক কী প্রশ্ন করবেন?

৩. আপনার প্রজেক্টে "যা দেখতে পায় না" — এই কলামে কার জন্য কী লেখা হবে?

৪. আপনি ইতিমধ্যে যে ফিচারটা নিজের ইচ্ছায় ঠিক করেছেন — সেটার পেছনে **কোন ব্যবহারকারীর কোন বাক্য** আছে? না থাকলে সেটা ফিচার নয়।

শিক্ষকের নোট: ৪ নম্বরের উত্তর বেশিরভাগের কাছে খালি থাকবে — এটাই স্বাভাবিক, কারণ এখনো কেউ ইন্টারভিউ করেনি। এটাই ক্লাস ৬-এর প্রবেশপথ: ফিচার তালিকা ইন্টারভিউ থেকে আসে, মাথা থেকে নয়।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
"কী চান" জিজ্ঞেস করলে নকল সমাধান আসে — কাজ ও ভয় জিজ্ঞেস করতে হয়	শিক্ষকের ৫টি দৈনন্দিন কাজ → ২১টি ফিচার, উত্তরে "সফটওয়্যার" শব্দই আসেনি
পাঁচ ধাপের প্রশ্ন-সিঁড়ি — ৪ ও ৫ বাদ দেওয়া যায় না	"দুইবার সেভ" → unique + এডিট মোড (F-31)
"যা দেখতে পায় না" — পারমিশনের আসল উৎস	কোঅর্ডিনেটরের ৫টি "যা নেই" → পারমিশন ম্যাট্রিক্স
উত্তর থেকে আসে নিয়মও, শুধু ফিচার নয়	কাস্টম ডোমেইন = নিরাপত্তার সিদ্ধান্ত, ফিচার নয়
প্রতিটি ফিচারের পাশে তার উক্তি রাখুন	ক্যাটালগে `F-31`-এর পাশে "purpose" লাইন — প্রশ্নের উত্তর সেখানেই

শিক্ষকের নোট: তৃতীয় লাইনটা পর্ব ৩–৪-এ বারবার ফিরে আসবে; ছাত্রদের এখনই মনে করিয়ে দিন যে পারমিশন ডিজাইন করার সময় নতুন কিছু ভাবতে হবে না — শুধু এই কলামটা দেখতে হবে।

০৬

পরের ক্লাস — "৫৭ ফাংশন, এক তালিকা"

কাঁচা উক্তি থেকে ফাংশন আইডি · অ্যাপ × ভূমিকা ম্যাপ · স্কোপ ফ্রিজে সই

পর্ব ২ · ইঞ্জিনিয়ারিং প্ল্যান · ক্লাস ৬ / ৩৭

৫৭ ফাংশন, এক তালিকা

সিলেবাস: প্র্যাকটিক্যাল ৩ · টপিক ৩.২ (ফাংশনাল রিকোয়্যারমেন্ট)
ফাংশন আইডি · অ্যাপ × ভূমিকা ম্যাপ · স্কোপ ফ্রিজে সই

১১টি অ্যাপ · ৫৭টি আইডি · ৬টি ভূমিকা

শুরু করুন প্রশ্ন দিয়ে

আপনার প্রজেক্টে কয়টা ফিচার? — সংখ্যাটা জানেন?

হাত তুলুন: কে বলতে পারবেন — ঠিক কতটা?

"অনেকগুলো আছে" — এটা সংখ্যা নয়।

আর সংখ্যা না জানলে স্কোপও নেই।

আজকের ক্লাসের ফলাফল একটাই জিনিস: **একটা তালিকা** — যার প্রতিটি লাইনের একটা **আইডি**, একটা **উদ্দেশ্য**, একটা **রুট**, আর একটা **মালিক-ভূমিকা**।
তালিকা ছাড়া "প্রজেক্ট ৭০% শেষ" বলার কোনো মানে নেই।

শিক্ষকের নোট: কেউ "আমার ২০টা ফিচার" বললে জিজ্ঞেস করুন — "নাম বলুন।" ১৪-তে থেমে যাবে। এই মুহূর্তটাই ক্লাসের সূচনা: মুখে বলা সংখ্যা সবসময় বেশি, তালিকায় কম।

ধাপ ১

একটি ফিচারের চারটি অংশ — আর আইডির নিয়ম

প্রজেক্টের ক্যাটালগে প্রতিটি ফিচার হুবহু এই গঠনে লেখা

```
F-33  Attendance register (by period)
role  : TEACHER
route : /w/<id>/attendance/register/
why   : একই দিনের সব পিরিয়ড এক পাতায় দেখা,
        কে নিয়েছে আর কে নেয়নি বোঝা
do    : ক্লাস · তারিখ · বিষয় বাছুন
        পিরিয়ড অনুযায়ী তালিকা দেখুন
data  : AttendanceRecord(student, class,
        subject, date) — একটি সারি = এক দিন
state : কোনো রেকর্ড নেই → খালি অবস্থার বার্তা
        লকড টার্ম → সম্পাদনা বন্ধ
```

আইডির চারটি নিয়ম

১. নম্বর **কখনো পুনর্ব্যবহার নয়** — ফিচার বাদ গেলেও আইডি মরে যায়।

২. **কখনো পুনঃক্রম নয়** — F-33 সবসময় রেজিস্টারই থাকবে।

৩. না-তৈরি ফিচারও **আইডি পায়** — নইলে পরিকল্পনা আর প্রমাণ মেলানো যায় না।

৪. আইডি **লেখায় ব্যবহার হয়** — কমিট, টেস্ট, ডকুমেন্ট, স্ক্রিনশটের নামে।

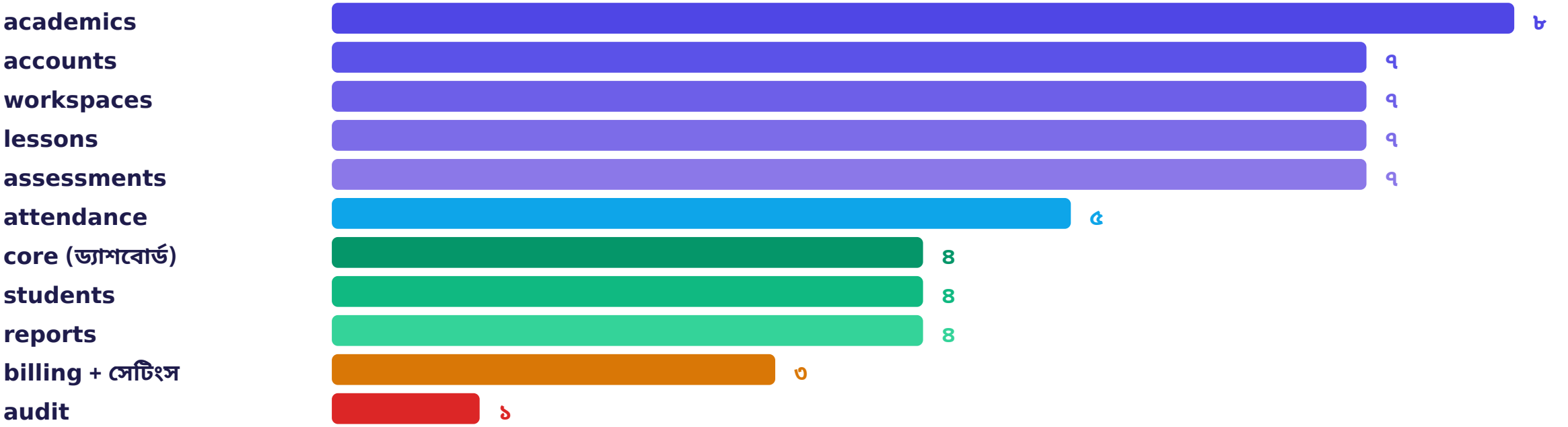
আইডি বদলালে পুরনো কমিট, পুরনো টেস্ট, পুরনো ডকুমেন্ট — সবের সংযোগ ছিঁড়ে যায়।

শিক্ষকের নোট: ৩ নম্বর নিয়মটা প্রথমে অদ্ভুত লাগে — "যেটা বানাব না, তার আবার আইডি কেন?" উত্তর: কারণ স্কোপের প্রশ্নটা "কোনটা বানাব" নয়, "কোনটা বানাব না আর কেন" — বাদ দেওয়া ফিচারও হিসাবের অংশ।

ধাপ ২

১১টি অ্যাপে ৫৭টি ফিচার — কোথায় কত

ব্লক লম্বা মানে ওই অ্যাপে কাজ বেশি — অর্থাৎ ওখানেই বাগও বেশি হবে



academics-এ ৮টি — অথচ এটা সিস্টেমের ভিত্তি। ভিত্তিতে ভুল হলে উপরের সবকিছু ভাঙে, তাই এই অ্যাপের টেস্ট সবচেয়ে বেশি চাই।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন — "কোন অ্যাপটা সবচেয়ে আগে বানাতে হবে?" উত্তর প্রায়ই "যেটা বেশি দেখা যায়"। আসল উত্তর: **যেটার উপর অন্য সব দাঁড়িয়ে** — academics। নইলে পরে সব মডেল বদলাতে হবে।

ধাপ ৩

অ্যাপ × ভূমিকা — কে কোন ঘরে ঢোকে

সংখ্যাগুলো ক্যাটালগ থেকে গোনা; খালি ঘর মানে "এই ভূমিকার জন্য কোনো ফিচার নেই"

অ্যাপ	শিক্ষক	অ্যাডমিন	কোঅর্ডি.	শিক্ষার্থী	প্রশাসক	পাবলিক	মোট
accounts	২	১	—	—	—	৪	**৭**
workspaces	১	৪	—	—	২	—	**৭**
core (ড্যাশবোর্ড)	১	—	১	১	১	—	**৪**
academics	৩	৫	—	—	—	—	**৮**
students	৪	—	—	—	—	—	**৪**
attendance	৪	—	১	—	—	—	**৫**
lessons	৬	—	—	১	—	—	**৭**
assessments	৪	১	—	২	—	—	**৭**
reports	৪	—	—	—	—	—	**৪**
audit	—	১	—	—	—	—	**১**
billing + সেটিংস	—	৩	—	—	—	—	**৩**
মোট	**২৯**	**১৫**	**২**	**৪**	**৩**	**৪**	**৫৭**

এই গ্রিডটাই ক্লাস ৯-এর **পারমিশন ম্যাট্রিক্সের** ভিত্তি — শুধু উল্টো করে পড়তে হবে: "এই ঘরে কে আছে" থেকে "এই ঘরে কে থাকবে না"।

শিক্ষকের নোট: ডান কলামের যোগফল ৫৭ — মিলছে কি না জিজ্ঞেস করুন। না মিললে বোঝা যায় কোনো ফিচার দুই ভূমিকায় গোনা হয়েছে, আর সেটাই পরে দ্বিগুণ কাজ হিসেবে ফিরে আসে।

ধাপ ৪

ফ্রিজে সই — তিনটি বাক্স, চারটি শর্ত

সই মানে "থাকবে না" বলা, শুধু "থাকবে" নয়

🧊 MVP — ৫টি স্লাইস

লগইন · প্রতিষ্ঠান ও ভূমিকা · শিক্ষার্থী ও ভর্তি · উপস্থিতি · নম্বর ও রিপোর্ট

≈ ৪৩টি ফিচার

🕐 v1.1

আমন্ত্রণ · ব্যাকআপ · পূর্ণ এক্সপোর্ট · নোটিফিকেশন

৪–৫টি ফিচার

🚫 কখনো নয়

SMS গেটওয়ে · মোবাইল অ্যাপ · AI উপস্থিতি

কারণ লিখে রাখা

চারটি শর্ত — সইয়ের আগে

১. প্রতিটি ফিচারের **আইডি** আছে

২. প্রতিটির **এক বাক্য উদ্দেশ্য** আছে

৩. প্রতিটির **একটি ভূমিকা** আছে (দুই হলে ভাঙুন)

৪. প্রতিটির **একটি রুট** আছে — না থাকলে সেটা ফিচার নয়, ইচ্ছা

স্কোপ ফ্রিজের আসল পরীক্ষা: "কোনটা বাদ দিলে প্রজেক্ট মরবে না?" — এই প্রশ্নের উত্তর দিতে না পারলে ফ্রিজ হয়নি।

শিক্ষকের নোট: স্লাইডে ৪৩ নম্বরটা ইচ্ছাকৃত — এটা স্ক্রিনশট-প্রমাণসহ ফিচারের সংখ্যা, আর ৫৭ মোট পরিকল্পনা। দুটো সংখ্যা গুলিয়ে ফেলা যাবে না; ক্লাস ১৩-এ এই পার্থক্যটাই ডেমোর সততা।

ধাপ ৫

এক ফিচার, এক মালিক

দুই ভূমিকার ফিচার মানে দুইবার পারমিশন যাচাই — তাই ভাঙতে হয়

প্রলোভন	কেন ভাঙতে হয়	প্রজেক্টে যা করা হলো
"উপস্থিতি শিক্ষক ও কোঅর্ডিনেটর — দুজনেই দেখবে"	দেখার নিয়ম দুজনের আলাদা: শিক্ষক নিজের ক্লাস, কোঅর্ডিনেটর পুরো বিভাগ	**F-33** রেজিস্টার (শিক্ষক) আর **F-35** oversight (কোঅর্ডিনেটর) — আলাদা ফিচার
"ফলাফল শিক্ষক ও শিক্ষার্থী — একই পর্দা"	শিক্ষক হিসাব করেন, শিক্ষার্থী শুধু প্রকাশের পরে দেখে	**F-46** হিসাব · **F-47** প্রকাশ · **F-48** আমার ফলাফল
"সেটিংস সবই এক পাতা"	গ্রেডিং আর ব্র্যান্ডিং সম্পূর্ণ আলাদা সিদ্ধান্ত	**F-45** গ্রেড স্কেল · **F-30** প্রতিষ্ঠান ও ব্র্যান্ডিং · **F-55** উপস্থিতি নীতি
"ড্যাশবোর্ড একটাই, শুধু ডেটা বদলাবে"	চার ভূমিকার প্রশ্নই আলাদা — এক পর্দায় চার প্রশ্নের উত্তর হয় না	**F-13** · **F-16** · **F-29** · **F-56** — চারটি ড্যাশবোর্ড

নিয়ম: **এক ফিচার = এক ভূমিকা = এক রুট = এক পরীক্ষা।** দ্বিতীয় ভূমিকা দরকার হলে সেটা নতুন ফিচার, পুরনোতে শর্ত নয়।

শিক্ষকের নোট: তৃতীয় সারিটা সবচেয়ে বেশি কাজে লাগে — "সব সেটিংস এক জায়গায়" শুনতে সুন্দর, কিন্তু তার মানে একজনের হাতে গ্রেডিং আর ব্র্যান্ডিং দুটোই। ছাত্রদের বলুন: সুবিধা আর অধিকার আলাদা জিনিস।

ধাপ ৬

চার ধরনের ফিচার — দাম আলাদা

তালিকা বানানোর সময়ই ধরন লিখে রাখুন, তখন সময়ের হিসাব নিজে থেকে আসবে

তালিকা (list)

প্রশ্ন: **কী আছে?**
অংশ: ভিউ + ১ টেমপ্লেট + ১ টেস্ট
সময়: **আধা দিন**
F-19 ছাত্র তালিকা

ফর্ম (form)

প্রশ্ন: **কী লিখব?**
অংশ: ফর্ম + ভ্যালিডেশন + ৪টি ভুল-অবস্থা
সময়: **১ দিন**
F-31 হাজিরা

রিপোর্ট (report)

প্রশ্ন: **কত?**
অংশ: কোয়েরি + অ্যাগ্রিগেট + এক্সপোর্ট
সময়: **২ দিন**
F-34 সারাংশ

মডেল (model)

প্রশ্ন: **কোথায় রাখব?**
অংশ: মাইগ্রেশন + অ্যাডমিন + পুরনো ডেটার যত্ন
সময়: **৩ দিন**
F-24 শিক্ষাবর্ষ

তালিকায় ৫৭টি লাইন দেখে "১০ দিনে হয়ে যাবে" ভাববেন না — **প্রতি লাইনের ধরন দেখুন**। নয়টি মডেল মানেই তিন সপ্তাহ।

শিক্ষকের নোট: ক্লাসে একটা ছোট কাজ দিন: ৫৭টি ফিচারকে চার ধরনে ভাগ করতে বলুন (মোটামুটি)। যে দলে "মডেল" বেশি পড়বে, তার প্রজেক্টের সময়সূচি সবচেয়ে বেশি ভাঙবে।

ধাপ ৭

ডুপ্লিকেট ধরা — একই জিনিস দুই নামে

তালিকা বড় হয় না, ফুলে যায় — আর ফোলা তালিকা কেউ পড়ে না

দুটো লাইন যেমন দেখায়	আসল প্রশ্ন	সিদ্ধান্ত
"আমার ক্লাস" · "ক্লাস ও সেকশন"	একই টেবিল, ভিন্ন দর্শক?	**দুটোই রাখা** — F-17 শিক্ষকের দেখা, F-26 অ্যাডমিনের তৈরি
"হাজিরা নেওয়া" · "হাজিরা রেজিস্টার"	একই ডেটা, দুই কাজ?	**দুটোই রাখা** — লেখা আর পড়া আলাদা কাজ (F-31 · F-33)
"ফলাফল" · "মার্কশিট PDF"	একটা অন্যটার রূপ?	**আলাদা** — PDF-এর আলাদা রেন্ডারার ও আলাদা টেস্ট (F-48 · F-49)
"সারাংশ" · "ক্লাস সারাংশ"	একই রিপোর্ট?	**এক করে ফেলা** — F-52 এক জায়গায়, ফিল্টার দিয়ে ভাগ

পরীক্ষা: **একই রুট?** → একটাই ফিচার। **একই টেবিল কিন্তু ভিন্ন কাজ?** → দুটো ফিচার। শুধু নাম দেখে মিলিয়ে ফেলা যাবে না।

শিক্ষকের নোট: চতুর্থ সারিটা "এক করে ফেলা"-র উদাহরণ — সব সারি জোড়া লাগাতে হবে না। কখনো এক করলে ফিচার সরল হয়, কখনো দুই ভূমিকা এক হয়ে যায় আর পারমিশন অস্পষ্ট হয়।

ধাপ ৮

যা ফ্রিজে ঢোকেনি — আর কেন

"কখনো নয়" তালিকাটাই আপনাকে সেমিস্টারের মাঝে বাঁচাবে

ফিচার	বাক্স	কারণ (এক লাইনে)	কখন ফিরবে
সদস্য আমন্ত্রণ	**v1.1**	ইমেইল পাঠানোর সেটআপ দরকার; এখন অ্যাডমিন নিজে অ্যাকাউন্ট খুলে দেন	একই কাজ দুই জায়গায় হলে
ব্যাকআপ কনসোল	**v1.1**	সার্ভারে cron + pg_dump যথেষ্ট; UI দরকার নেই	একাধিক প্রতিষ্ঠান এলে
পূর্ণ এক্সপোর্ট (XLSX)	**v1.1**	CSV প্রথমে যথেষ্ট, বাংলা সমস্যা হলে XLSX	গ্রাহক চাইলে
SMS নোটিফিকেশন	**কখনো নয়**	প্রতি SMS-এ টাকা লাগে; খরচ মডেল বদলে যাবে	প্ল্যানে আলাদা মূল্য যোগ হলে
মোবাইল অ্যাপ	**কখনো নয়**	ওয়েবসাইট মোবাইলে কাজ করে; অ্যাপ একই কাজ দুইবার	অফলাইন দরকার হলে
AI উপস্থিতি (ছবি থেকে)	**কখনো নয়**	গোপনীয়তা, নির্ভুলতা, খরচ — তিনটেই বাইরে	ক্যামেরা ও অনুমোদন থাকলে

শিক্ষকের নোট: "কখন ফিরবে" কলামটা বাদ দেবেন না — এটি ঝুঁকি ব্যবস্থাপনা: ফিচার ফিরে আসবে, কিন্তু শর্ত পূরণ হলে। শর্ত ছাড়া ফেরা মানেই স্কোপ ফাটল।

ধাপ ৯

তালিকা কোথায় থাকে — ক্যাটালগ ফাইলের গঠন

একটি ফাইলে ৫৭টি ফিচার, প্রতিটির ছয়টি কী — `docs/catalogue_data.py`

```
FEATURES = [
  dict(id="F-31", role="TEACHER",
       name="Take attendance",
       route="/w/<id>/attendance/take/",
       status="built",
       layout=SUBMIT,
       purpose="Marks a whole class present/
                absent/late/leave for one date",
       actions=["Pick class, date, subject",
                "Bulk buttons",
                "Add a note", "Save"],
       data="AttendanceRecord unique(student,
             class, subject, date)",
       states="Already taken today, future date
               blocked, holiday warning",
       cls="C099, C100"),
  ...
]

ROLES = { "PUBLIC": "Anonymous visitor",
          "TEACHER": "Individual teacher", ... }
```

ছয়টি কী কেন

id — সংযোগের নাম
role — কোন ভূমিকা
route — কোথায় থাকে
purpose — এক বাক্যে কেন
actions — পর্দায় কী কী
data · states — কোন টেবিল, কোন ভুল-অবস্থা

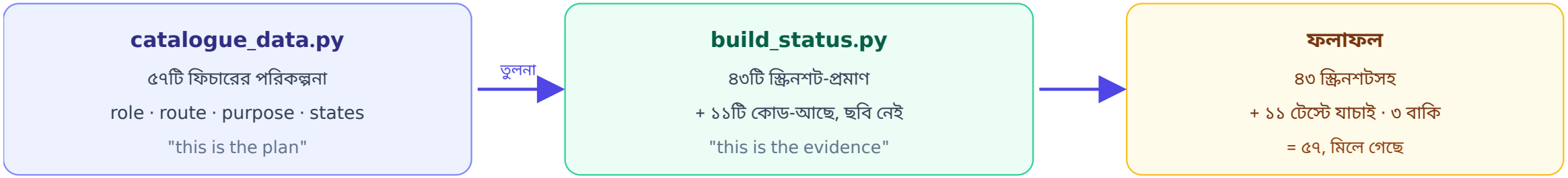
তালিকা **কোডে** রাখার কারণ: তালিকা থেকে ডকুমেন্ট, PDF, স্ক্রিনশট-চেক — সব একই সূত্র থেকে তৈরি হয়।

শিক্ষকের নোট: "Excel-এ তালিকা রাখলেই তো হয়" — এই প্রশ্ন আসবেই। উত্তর: Excel থেকে কোনো কিছু তৈরি করা যায় না; কোড থেকে PDF, চেক, টেস্ট — সবই যায়, আর সেটাই কমিট 5373b59-এ করা হয়েছে।

ধাপ ১০

পরিকল্পনা আর প্রমাণ — দুই ফাইল, ইচ্ছাকৃত

একই ফাইলে রাখলে স্ক্রিনশট বানাতে গিয়ে তালিকা বদলে যেত



ফাইলটির নিজের মন্তব্য: "Regenerating a screenshot list can never corrupt the feature specification."
অর্থাৎ স্ক্রিনশট আবার তুললে পরিকল্পনা বদলাবে না — দুইটা আলাদা ফাইল থাকার এটাই কারণ।

> এটাই ক্লাসের সবচেয়ে ব্যবহারিক পরামর্শ: **পরিকল্পনা আর প্রমাণ কখনো একই ফাইলে রাখবেন না।**

শিক্ষকের নোট: ছাত্রদের বলুন নিজের প্রজেক্টে দুটো ফাইল বানাতে — `plan.md` আর `done.md`। প্রতি সপ্তাহে `done.md` আপডেট হবে, `plan.md` নয়। ক্লাস ১৩-এর ডেমোতে এই দুটো ফাইলই তাদের সততার প্রমাণ।

ধাপ ১১

ফ্রিজ ভাঙার প্রক্রিয়া — চার লাইনের অনুরোধ

ফ্রিজ মানে পরিবর্তন বন্ধ নয়; ফ্রিজ মানে পরিবর্তন লিখিত

পরিবর্তনের অনুরোধ (CR)

১. **কী:** কোন ফিচার, কোন আইডি

২. **কেন:** কোন ব্যবহারকারীর কোন বাক্য

৩. **কী বদলাবে:** কোন ফিচার বাদ যাবে বা পিছাবে

৪. **কে অনুমোদন:** নাম ও তারিখ

নিয়ম: শুধু যোগ করা যায় না — কিছু বদলাতে হবে। স্কোপ বাড়ে না, শুধু অদল-বদল হয়।

আসল উদাহরণ — কমার্শিয়াল লেয়ার

কী: সাবস্ক্রিপশন, ট্রায়াল, হোয়াইট-লেবেল, প্ল্যাটফর্ম কনসোল

কেন: "এক স্কুলের সফটওয়্যার নয়, প্ল্যাটফর্ম" — ক্লাস ১-এর সিদ্ধান্ত

কী বদলালো: সাবস্ক্রিপশন ব্যক্তি থেকে **প্রতিষ্ঠানে** (`8a25724`)

খরচ: ১ মাইগ্রেশন + বিলিং ভিউ + ৪ টেস্ট

শিক্ষকের নোট: এই স্লাইডে সততার প্রশ্ন তুলুন: প্রজেক্টেও স্কোপ বেড়েছিল — কমার্শিয়াল লেয়ারটা ফ্রিজের বাইরে ছিল। বলুন: "আমরাও ভুল করেছি; পার্থক্য একটাই — ভুলটা লিখে রাখা হয়েছে।"

আলোচনা

চারটি প্রশ্ন — নিজের তালিকা মিলিয়ে

প্রতিটি উত্তর একটা সংখ্যা দিয়ে শেষ করুন

১. আপনার প্রজেক্টে এখন কতটি ফিচার? সংখ্যাটা লিখুন — ২০-এর বেশি হলে ভাগ হয়নি, ফুলেছে।

২. কোন ফিচারের **দুটো ভূমিকা** আছে? সেটা ভাঙলে কতটি হবে?

৩. আপনার "কখনো নয়" বাক্সে কয়টি ফিচার, আর প্রতিটির কারণ এক লাইনে লিখতে পারবেন?

৪. আপনার তালিকার কোন লাইনের **রুট নেই**? রুট না থাকলে সেটা ফিচার নয় — ইচ্ছা।

শিক্ষকের নোট: ৪ নম্বরের উত্তর দিয়ে ক্লাস ৭ শুরু হবে — 루트 없는 লাইনগুলোই পরে "এটা তো হয়নি" বলে ধরা পড়ে, আর ডেমোর দিনে সবচেয়ে বিব্রতকর হয়।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
ফিচার মানে আইডি + উদ্দেশ্য + রুট + ভূমিকা	F-33-এর ছয়টি কী হুবহু `catalogue_data.py` থেকে
এক ফিচার = এক ভূমিকা — দ্বিতীয় ভূমিকা মানে নতুন ফিচার	উপস্থিতি ভাঙা হয়েছে: F-33 (শিক্ষক) · F-35 (কোঅর্ডিনেটর)
তালিকার দাম তার ধরনে — list · form · report · model	৯টি মডেল-ধরনের ফিচার = প্রায় ৩ সপ্তাহের কাজ
"কখনো নয়" বাক্স + কারণ — সবচেয়ে দরকারি অংশ	SMS · অ্যাপ · AI — তিনটিরই কারণ ও ফেরার শর্ত লেখা
পরিকল্পনা আর প্রমাণ আলাদা ফাইল	`catalogue_data.py` বনাম `build_status.py` → ৪৩ + ১১ + ৩ = ৫৭

শিক্ষকের নোট: পাঁচটি লাইন একসাথে পড়ে দেখুন — এগুলোই পর্ব ৪-এর প্রতিটি স্লাইসের শিরোনাম হবে। ফিচার আইডি ছাড়া স্লাইস বানানো যায় না, কারণ স্লাইসের সীমা ফিচার দিয়েই টানা হয়।

০৭

পরের ক্লাস — "কেন এই স্ট্যাক"

তিনটি বিকল্পের তুলনা · ADR লেখার নিয়ম · সিদ্ধান্ত লিখে রাখা

পর্ব ২ · ইঞ্জিনিয়ারিং প্ল্যান · ক্লাস ৭ / ৩৭

কেন এই স্ট্যাক — আর কেন নয়

সিলেবাস: প্র্যাকটিক্যাল ৩ · টপিক ৩.৩ (প্রযুক্তি নির্বাচন ও ADR)
তিন বিকল্পের তুলনা · সিদ্ধান্তের নথি · সিদ্ধান্ত কোথায় থাকে

৬টি প্যাকেজ · ৩টি settings · ১টি ADR-নিয়ম

শুরু করুন প্রশ্ন দিয়ে

"কেন Django?" — এই প্রশ্নের উত্তর মুখস্থ নয়, লিখিত হতে হবে

হাত তুলুন: আপনার স্ট্যাক কেন বেছেছেন — এক বাক্যে বলুন

"সবাই ব্যবহার করে" — এটা কারণ নয়।

কারণ হলো: এই কাজে এটার কী সুবিধা, আর কী দাম।

প্রযুক্তি বাছাই দুটি প্রশ্নের উত্তর: **"কী সহজ হবে?"** আর **"কী কঠিন হবে?"**। দুটোর উত্তর লিখে রাখলে সেটাকে বলে **ADR** — Architecture Decision Record।
না লিখলে ছয় মাস পরে আপনি নিজেই জানবেন না কেন এটা বেছেছিলেন।

শিক্ষকের নোট: কেউ "React শিখতে চাই বলে React" বললে খুব ভালো — কিন্তু সেটাও লিখে রাখতে বলুন। ADR-এ "আমি এটা শিখতে চাই" একটি সম্পূর্ণ বৈধ কারণ, যদি সেটা লেখা থাকে।

ধাপ ১

স্ট্যাকের ছয়টি স্তর — প্রতিটির একটা সিদ্ধান্ত

"স্ট্যাক" মানে শুধু ফ্রেমওয়ার্ক নয় — ছয় জায়গায় ছয়টা পছন্দ

১. ভাষা
Python 3.13 — কোর্সের ভাষা

২. ফ্রেমওয়ার্ক
Django 6.1.2 — auth · ORM · forms

৩. ডেটাবেস
PostgreSQL · sqlite (ডেমো)

৪. পর্দা
Django templates · ৭৫টি — SPA নয়

৫. ফাইল আউটপুট
openpyxl · reportlab — Excel · PDF

৬. সার্ভার
gunicorn · ২ worker

নিয়ম: প্রতিটি স্তরের পছন্দের জন্য একটি করে ADR — ছয়টি স্তর, ছয়টি সিদ্ধান্ত, ছয়টি কারণ।
এগুলোর কোনোটাই "ডিফল্ট" নয় — প্রতিটি বেছে নেওয়া, আর প্রতিটির একটা না-বাছাইয়ের বিকল্প ছিল।

শিক্ষকের নোট: ছয়টি বাক্স বোর্ডে আঁকুন। ছাত্ররা সাধারণত ২ আর ৩ নিয়ে ভাবে, ৪ ও ৬ ভুলে যায় — অথচ ডেমোর দিনে সমস্যা হয় ঠিক ওই দুটোতেই।

ধাপ ২

তিনটি বিকল্প — একই কাজ, তিন পথ

স্কোর নয়, **প্রতিদান** — প্রতিটি সারিতে "কী পাব, কী হারাব"

মানদণ্ড	Django (Python)	Node + Express + React	Laravel (PHP)
লগইন ও ভূমিকা	✓ বিল্ট-ইন, ২ ঘণ্টার কাজ	✗ নিজে: JWT, রিফ্রেশ, গার্ড	✓ বিল্ট-ইন
ডেটাবেস বদল	✓ মাইগ্রেশন স্বয়ংক্রিয়	◐ Prisma/Sequelize যোগ	✓ Eloquent মাইগ্রেশন
৩১ টেবিল, ৮টি সম্পর্ক	✓ ORM সহজে সামলায়	◐ সম্ভব, কাঠামো নিজের	✓ সম্ভব
পর্দা বানানো	✓ টেমপ্লেট — ১ ফাইলে ১ পর্দা	✗ বিল্ড স্টেপ, npm, bundler	✓ Blade
এক কমান্ডে চালু	✓ `run.sh`	✗ ২টি সার্ভার + ২টি পোর্ট	◐ সম্ভব
কোর্সের ভাষা	✓ Python	✗ JavaScript	✗ PHP
ডাকুমেন্টেশন	✓ সর্বোত্তম	✓ ভালো	✓ ভালো

খেয়াল করুন: **React-এর কোনো সারিতে "খারাপ" লেখা নেই** — শুধু "এই প্রজেক্টে অতিরিক্ত"। ADR-এ এভাবে সৎ ভাষা লিখতে হয়।

শিক্ষকের নোট: কেউ "React বাদ কেন" জিজ্ঞেস করলে বলুন — "বাদ নয়, দামি।" ৩১ টেবিলের CRUD-এ SPA-র সুবিধা কম, খরচ বেশি। ছাত্রদের নিজের প্রজেক্টে একই টেবিল লিখতে বলুন।

ধাপ ৩

কেন SPA নয় — সিদ্ধান্তটার সৎ হিসাব

৭৫টি টেমপ্লেট · কোনো npm · কোনো বিল্ড স্টেপ নেই

সার্ভার-রেন্ডারড যা দেয়

- ✓ প্রথম পাতাই পূর্ণ HTML — ধীর নেটেও পড়া যায়
- ✓ SEO নিজে থেকেই (পাবলিক সাইট)
- ✓ এক সার্ভার, এক পোর্ট, এক কমান্ড
- ✓ ফর্ম ভ্যালিডেশন সার্ভারে — বাইপাস করা যায় না
- ✓ ৪ ভূমিকার জন্য ৪টি টেমপ্লেট — শর্ত দিয়ে লুকানো নয়

যা হারাতে হয়েছে

- ✗ পূর্ণ-পাতা রিলোড — একটু ধীর অনুভূত হয়
- ✗ অফলাইনে কাজ করে না
- ✗ জটিল ইন্টারঅ্যাকশন (টানা ড্র্যাগ) কষ্টকর
- ✗ BFF/রিয়েল-টাইম পরে যোগ করতে হবে

সিদ্ধান্ত: ৪৩টি ফিচারের **৩৮টিই ফর্ম-ভিত্তিক** — অর্থাৎ SPA-র সুবিধা প্রায় লাগবেই না। তাই সার্ভার-রেন্ডারডই সঠিক পছন্দ।

শিক্ষকের নোট: এখানে সততার একটা উদাহরণ দিন: "যদি আপনার প্রজেক্টে লাইভ চ্যাট বা ক্যানভাস অঙ্কন থাকে, তাহলে এই সিদ্ধান্ত উল্টো হবে।" ADR নির্দিষ্ট প্রজেক্টের জন্য, সব প্রজেক্টের জন্য নয়।

ধাপ ৪

ছয়টি প্যাকেজ, ভার্সন পিন করা — হুবহু ফাইল থেকে

`requirements.txt`-এর নিজের মন্তব্য: "Exact versions used and verified in this build (2026-10-29, Python 3.13)"

```
Django==6.1.2
psycopg[binary]==3.3.6
python-dotenv==1.2.4
Pillow==12.3.0

# Used for exports and the marksheet PDF
openpyxl==3.1.5
reportlab==5.1.0
```

== কেন, >= কেন নয়

>= লিখলে আজ ঠিক চললেও ছয় মাস পরে নতুন ভার্সন আসবে — আর আপনার কোড ভাঙতে পারে।

== লিখলে "আজ যা চলছে, সেটাই চলবে"।

আপডেট তখনই, যখন **ইচ্ছাকৃতভাবে** করবেন — টেস্ট চালিয়ে, এক প্যাকেজ করে।

৬টি প্যাকেজই যথেষ্ট — ৩১ টেবিল, ৭৫ পর্দা, PDF, Excel, PostgreSQL। লাইব্রেরির সংখ্যা কম মানে দুর্বলতা নয়, নিয়ন্ত্রণ।

শিক্ষকের নোট: "pip freeze" করে ফাইলটা বানানোর কথা বলুন — হাতে লেখা নয়। আর commit করার আগে ভার্সনগুলো মিলিয়ে নেওয়া, নইলে ডেমোর দিনে ভিন্ন মেশিনে ভিন্ন ভার্সন চলবে।

ধাপ ৫

ADR কী — এক পাতার সিদ্ধান্ত-নথি

পাঁচটি অংশ, একটির বেশি নয় — বড় হলে সিদ্ধান্তটা এখনো স্পষ্ট নয়

১. প্রসঙ্গ

কী পরিস্থিতিতে
সিদ্ধান্ত নিতে হচ্ছে

৩–৪ লাইন

২. সিদ্ধান্ত

এক বাক্যে
"আমরা X ব্যবহার করব"

আপস নেই

৩. বিকল্প

অন্তত দুটি
+ কেন বাদ

এটাই ADR-এর প্রাণ

৪. পরিণাম

কী লাভ, কী দাম
কী কঠিন হয়ে যাবে

সৎ ভাষায়

৫. তারিখ ও নাম

কবে, কে
কোন ক্লাসে

পরে খুঁজে পাওয়ার জন্য

পাঁচটি ঘর ভরে ফেলুন — এক পাতায়, বাংলায়, নিজের ভাষায়। এক ঘর খালি থাকলে সিদ্ধান্তটা অসম্পূর্ণ।

ADR মূলত ভবিষ্যতের নিজের কাছে লেখা চিঠি — "কেন এটা করেছিলাম" তার উত্তর।

শিক্ষকের নোট: "৪ নম্বর ঘরটা লিখতে গিয়ে থেমে যাবেন — সেটাই আসল কাজ।" যে সিদ্ধান্তের কোনো খারাপ দিক নেই, সেটা সিদ্ধান্ত নয়, ইচ্ছা।

ADR-001

ভরা উদাহরণ — কেন Django

পাঁচটি ঘর, পাঁচটি উত্তর — এক পাতায়

ADR-001 · ফ্রেমওয়ার্ক নির্বাচন

২০২৬-১০-২৯ · ক্লাস C071

প্রসঙ্গ: ৩১ টেবিল, ৬ ভূমিকা, ৫৭ ফিচারের একটি প্রতিষ্ঠান-ব্যবস্থাপনা সিস্টেম; টিম এক জন; সময় ১২ সপ্তাহ। লগইন, পারমিশন ও অডিট — তিনটেই লাগবেই।

সিদ্ধান্ত: Django 6.1.2 (Python 3.13), সার্ভার-রেন্ডারড টেমপ্লেট সহ। কোনো SPA ফ্রেমওয়ার্ক নয়।

বিকল্প ও কেন বাদ: **Flask** — auth/ভূমিকা নিজে লিখতে হতো (+~২০০০ লাইন); **Node + React** — দুই সার্ভার, বিল্ড স্টেপ, আর কোর্স Python-ভিত্তিক; **Laravel** — সমমানের ভালো, কিন্তু ভাষা PHP; **No-code** — কোড শেখার উদ্দেশ্যই ভাঙে।

পরিণাম: লাভ — ২ ঘণ্টায় লগইন+ভূমিকা, স্বয়ংক্রিয় মাইগ্রেশন, admin প্যানেল। দাম — Django-র শেখার বাঁক, আর জটিল ইন্টারঅ্যাকশনে কম স্বাধীনতা।

শিক্ষকের নোট: এই পাতা হুবহু ছাত্রদের খাতায় লিখিয়ে নিন — তারপর নিজের প্রজেক্টের জন্য একই পাঁচ ঘর ভরতে বলুন। ভাষা বদলাবে, কাঠামো একই থাকবে।

ADR-002

যে সিদ্ধান্তটা একবারই নেওয়া যায় — কাস্টম User

কোডের নিজের মন্তব্য: "never start a Django project with the default User model"

ADR-002 · কাস্টম ইউজার মডেল

প্রসঙ্গ: প্রথম মাইগ্রেশনের আগে ঠিক করতে হবে — ইউজারকে ইমেইল দিয়ে লগইন করাব? ফোন? নাম আলাদা রাখব?

সিদ্ধান্ত: ইমেইল-ভিত্তিক কাস্টম `User` মডেল, প্রথম মাইগ্রেশনের **আগে**।

বিকল্প: ডিফল্ট `User` (username দিয়ে) — পরে বদলানো প্রায় অসম্ভব।

পরিণাম: লাভ — ইমেইল লগইন, ভূমিকা আলাদা অ্যাপে। দাম — নিজে ম্যানেজার লিখতে হয়েছে।

কেন এটা "একবারই"?

Django-তে ইউজার মডেল বদলাতে হলে **সব মাইগ্রেশন নতুন করে** করতে হয় — অর্থাৎ ডেটাবেস খালি করা। স্কুলের এক বছরের উপস্থিতি থাকলে সেটা চলে না।

সিদ্ধান্তের চিহ্ন কোডে

```
AUTH_USER_MODEL = "accounts.User"
# MUST be set before the first migration
```

শিক্ষকের নোট: এই ADR-টা ছাত্রদের সবার আগে লিখতে বলুন — এটাই একমাত্র সিদ্ধান্ত যা ভুল হলে পুরো কাজ নতুন করে শুরু করতে হয়।

ADR-003

যে সিদ্ধান্ত ভুল হয়েছিল — টাকার একক

এটাই ADR লেখার সবচেয়ে বড় যুক্তি: ভুলটাও লিখে রাখতে হয়

প্রথম সিদ্ধান্ত (ভুল)

সাবস্ক্রিপশন **ব্যবহারকারীর** নামে বাঁধা। কারণ তখন ধরা হয়েছিল "একজন মানুষই একাউন্ট খোলে।"

ফল: শিক্ষক দুই স্কুলে পড়ালে বিল কার নামে? প্রতিষ্ঠানের সিট কে ব্যবহার করছে — সেই প্রশ্নের উত্তরই নেই।

ADR-003 (সংশোধিত · `8a25724`)

সিদ্ধান্ত: সাবস্ক্রিপশন **Workspace-এ**; মানুষ শুধু সদস্য।
পরিণাম: বিল প্রতিষ্ঠানের, সিট প্রতিষ্ঠানের; শিক্ষক একাধিক প্রতিষ্ঠানে থাকতে পারেন।
দাম: ১টি মাইগ্রেশন + বিলিং ভিউ + ৪টি টেস্ট।

নিয়ম: পুরনো ADR **মুছবেন না** — "superseded by ADR-003b" লিখে রাখুন। ভুলের ইতিহাসই সবচেয়ে দামি নথি।

শিক্ষকের নোট: শিক্ষকেরা জিজ্ঞেস করবেন "ভুল করা ADR কেন রাখব?" উত্তর: কারণ তিন মাস পরে কেউ একই ভুল করতে যাবে। মুছে ফেলা মানে তাকে একই দেয়ালে আবার ধাক্কা খেতে দেওয়া।

ধাপ ৬

সিদ্ধান্ত কোথায় থাকে — প্রজেক্টের আসল প্রমাণ

`docs/`-এ ADR ফোল্ডার নেই; কিন্তু সিদ্ধান্তগুলো আছে ফাইলের মন্তব্যে

ফাইল	যা লেখা আছে (হুবহু)	কোন সিদ্ধান্ত
`config/settings/base.py`	"Nothing secret lives here. Everything that changes between dev, demo and production is read from environment variables."	সেটিংস ভাগ
`config/settings/prod.py`	"Every value here is deliberate. Nothing in this file may be relaxed to make a deployment 'work'."	প্রোডাকশন কঠোরতা
`config/settings/sqlite.py`	"Use this ONLY to try the application quickly… The real stack (PostgreSQL) remains the documented path."	ডেমো ডেটাবেস
`apps/accounts/models.py`	"Custom user model — created BEFORE the first migration… never start a Django project with the default User model."	কাস্টম User
`docs/build_status.py`	"the catalogue is the *plan*, this file is the *evidence*."	পরিকল্পনা/প্রমাণ পৃথক
`apps/accounts/urls.py`	পাসওয়ার্ড রিসেটের চারটি রুট একসাথে	রিসেট ফ্লো

এগুলো ADR নয়, কিন্তু **ADR-এর কাজ করছে** — কারণ প্রতিটি ভবিষ্যতের পাঠককে "কেন" বলছে।

শিক্ষকের নোট: ভালো অভ্যাস: নিজের প্রজেক্টে `docs/adr/` ফোল্ডার বানিয়ে `001-...md` রাখুন। প্রজেক্টে যেটা হয়নি, সেটা ছাত্ররা আরও ভালোভাবে করতে পারে — বলুন এটা একটা উন্নতির সুযোগ।

ধাপ ৭

ADR লেখার ছয়টি নিয়ম

এক পাতা না হলে সিদ্ধান্তটা তখনো স্পষ্ট নয়

১. এক ADR = এক সিদ্ধান্ত

"ডেটাবেস ও টেমপ্লেট" একসাথে লিখবেন না — পরে একটা বদলালে অন্যটার ইতিহাস হারায়।

২. বিকল্প অন্তত দুটি

একটাই বিকল্প মানে সিদ্ধান্ত নয়, অনুমোদন। যে দুটি বিকল্প ভেবেছেন, দুটিই লিখুন।

৩. পরিণামে খারাপ দিকও

"কোনো অসুবিধা নেই" — এই বাক্য থাকলে ADR-টা বিশ্বাসযোগ্য নয়।

৪. তারিখ ও নাম

কে, কবে, কোন ক্লাসে — পরে "এটা কার ধারণা ছিল" প্রশ্নের উত্তর থাকবে।

৫. বদলালে নতুন ADR

পুরনোটা মুছবেন না; নতুনটায় "supersedes ADR-003" লিখুন।

৬. কোড দেখিয়ে লিখুন

"আমরা X ব্যবহার করি" নয় — **কোন ফাইলে, কোন লাইনে** সেটা পাওয়া যায়।

দৈর্ঘ্য: **এক পাতা, সর্বোচ্চ দুই পাতা।** তিন পাতায় গেলে সেটা ADR নয়, ডিজাইন ডকুমেন্ট — আলাদা জিনিস।

শিক্ষকের নোট: ৬ নম্বর নিয়মটা সবচেয়ে বেশি বাদ পড়ে — অথচ এটাই ADR-কে অনুমান থেকে প্রমাণে নিয়ে যায়। ছাত্রদের প্রতিটি ADR-এ অন্তত একটি ফাইলের নাম লিখতে বাধ্য করুন।

ধাপ ৮

প্রতিটি পছন্দের একটা দাম আছে

ADR-এর "পরিণাম" ঘরটা সৎভাবে ভরার জন্য

পছন্দ	যা পাওয়া গেল	যা কিনতে হলো (দাম)
Django 6.1.2	auth, ORM, forms, admin — বিনা লেখায়	মেজর আপডেটে ভাঙার ঝুঁকি; শেখার বাঁক
PostgreSQL	৭২ লাখ সারি, ইনডেক্স, লক	সার্ভারে আলাদা ডেটাবেস সার্ভিস চালাতে হয়
sqlite ডেমো	এক ক্লিকে চালু, কোনো সেটআপ নেই	দুই পথ — ডেমো আর প্রোডাকশনে আচরণ আলাদা হতে পারে
সার্ভার-রেন্ডারড	এক সার্ভার, SEO, সহজ ডিপ্লয়	পূর্ণ-পাতা রিলোড; সমৃদ্ধ ইন্টারঅ্যাকশন কষ্টকর
reportlab + openpyxl	মার্কশিট PDF, Excel এক্সপোর্ট	দুই লাইব্রেরি, দুই ধরনের টেস্ট
gunicorn ২ worker	সহজ, কম মেমরি	এক worker মরলে অর্ধেক ক্ষমতা; পরে স্কেলিং লাগবে

শিক্ষকের নোট: ছাত্রদের বলুন নিজের স্ট্যাকের জন্য এই তিন-কলামের টেবিল বানাতে — বিশেষ করে ডান কলামটা। যে টেবিলে ডান কলাম খালি, সে টেবিলটা মিথ্যা বলছে।

সতর্কতা

বাংলা + PDF-এর ফাঁদ — আমাদের নিজের অভিজ্ঞতা

এই ডেকটাই যেভাবে তৈরি হয়েছে, সেটাই এখানে প্রমাণ

যা কাজ করে না

reportlab বাংলা অক্ষর জোড়া লাগাতে পারে না — "ক্ষ" আলাদা দুই অক্ষর হয়ে যায়, মাত্রা (vowel sign) জায়গায় বসে না। ফল: PDF-এ অক্ষর দেখতে পাওয়া যায়, কিন্তু পড়া যায় না।

যা কাজ করে

HTML + CSS দিয়ে পাতা সাজিয়ে **হেডলেস ব্রাউজার** দিয়ে PDF বানানো — ব্রাউজারের শেপিং ইঞ্জিন বাংলা নিখুঁতভাবে সামলায়।

এই ডেকের আসল পাইপলাইন

১. বাংলা ফন্ট `@font-face` দিয়ে এমবেড
২. ligature বন্ধ — `font-variant-ligatures:none`
৩. প্রতিটি স্লাইড এক পাতা (338.7 × 190.5 mm)
৪. Chromium: `page.pdf(print_background=True)`
৫. প্রতিটি পাতা ছবি করে চোখে দেখা

শিক্ষা: "লাইব্রেরি আছে" মানে "লাইব্রেরি আপনার ভাষা পারে" নয় — সবসময় নিজের ভাষায় এক পাতা বানিয়ে দেখুন।

শিক্ষকের নোট: এই স্লাইডটা প্রজেক্টের নিজের কষ্ট থেকে — এই পুরো কোর্স কিটই এই পাইপলাইনে তৈরি। ছাত্রদের বলুন: "তোমাদের ডেমোতে বাংলা PDF লাগলে এই পাঁচ ধাপ লাগবেই।"

ধাপ ৯

কোন সিদ্ধান্ত বদলানো সবচেয়ে কঠিন

আগে যেগুলো ঠিক করবেন, সেগুলোই এই তালিকার উপরের দিকে

১. ইউজার মডেল	সব মাইগ্রেশন নতুন করে — ডেটা খালি করা ছাড়া উপায় নেই	**প্রায় অসম্ভব**
২. টেন্যান্টের একক	কে মালিক — ব্যক্তি না প্রতিষ্ঠান? টেবিল, পারমিশন, বিল সব বদলায়	**খুব কঠিন**
৩. ডেটাবেস ইঞ্জিন	dump → restore, SQL ভিন্নতা, দুটি সেটিংস রক্ষণাবেক্ষণ	**কঠিন**
৪. পর্দার প্রযুক্তি	টেমপ্লেট → SPA: প্রতিটি ৭৫ পর্দা আবার লেখা	**খরচসাপেক্ষ**
৫. লাইব্রেরি	একটি প্যাকেজ বদলানো — সাধারণত সহজ, টেস্ট থাকলে	**সহজ**

তাই নিয়ম: **উপরের তিনটি সিদ্ধান্ত প্রথম সপ্তাহে, ADR-সহ।** নিচের দুটি যেকোনো সময়।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন — "তোমার কোন সিদ্ধান্ত এখনো নেওয়া হয়নি, যেটা তালিকার ১ বা ২ নম্বরে?" বেশিরভাগের ক্ষেত্রে উত্তর থাকবে "টেন্যান্টের একক" — কারণ তারা এটা নিয়ে ভাবেইনি।

ধাপ ১০

ADR আর কমিট — একই সিদ্ধান্তের দুই চিহ্ন

প্রতিটি বড় সিদ্ধান্তের পাশে একটা কমিট থাকা উচিত, আর উল্টোটাও

কমিট	যা ঘটল	কোন সিদ্ধান্ত	কীভাবে যাচাই করবেন
`05c63dc`	স্লাইস ১: কাস্টম ইউজার, লগইন, workspace, ভূমিকা, ১৩ টেস্ট	**ADR-002** কাস্টম User	`git show 05c63dc --stat`
`674375b`	তিনটি settings, `run.sh`, pinned requirements	**ADR-004** পরিবেশ ভাগ	`ls config/settings/`
`7802a82`	কমার্শিয়াল লেয়ার: সাবস্ক্রিপশন, ট্রায়াল, প্ল্যাটফর্ম কনসোল	**ভুল সিদ্ধান্ত** — টাকার একক ব্যক্তি	ওই কমিটের মাইগ্রেশন দেখুন
`8a25724`	"the subscription belongs to the institution, not to individuals"	**ADR-003b** সংশোধন	`git show 8a25724`
`bfd058e`	TMS → IMS, কিন্তু **settings থেকে চালিত**	**ADR-005** নাম কোডে নয়	`grep PRODUCT_NAME config/settings/base.py`

পরীক্ষা: **ADR-এ লেখা প্রতিটি দাবির পাশে একটা কমিট দেখাতে পারবেন?** না পারলে দাবিটা এখনো প্রমাণ হয়নি — শুধু লেখা হয়েছে।

শিক্ষকের নোট: ক্লাস ২৩-এ গিট নিয়ে পুরো ক্লাস আছে; এখানে শুধু সংযোগটা দেখান। ছাত্রদের বলুন প্রতিটি ADR-এ কমিট হ্যাশ লেখার অভ্যাস করতে — এক লাইনে ভবিষ্যতের অনেক প্রশ্নের উত্তর হয়ে যায়।

ক্লাসের কাজ

একটি ADR লিখুন — ১৫ মিনিট, এক পাতা

নিজের প্রজেক্টের সবচেয়ে গুরুত্বপূর্ণ সিদ্ধান্তটা নিয়ে

১. সিদ্ধান্ত বাছুন

ভাষা · ফ্রেমওয়ার্ক · ডেটাবেস · টেন্যান্টের একক — একটাই বেছে নিন।

২. প্রসঙ্গ তিন লাইনে

কী বানাচ্ছেন, কতজন ব্যবহার করবে, কত সময় আছে।

৩. দুটি বিকল্প লিখুন

যেটা আপনি সত্যিই ভেবেছিলেন — বানানো বিকল্প নয়।

৪. পরিণামে দাম লিখুন

অন্তত একটি খারাপ দিক। না থাকলে আরেকবার ভাবুন।

৫. ফাইলের নাম দিন

`docs/adr/001-framework.md` — তারিখ ও নিজের নাম সহ।

যাচাই: পাশের জনকে দিন — সে কি ৩০ সেকেন্ডে বলতে পারবে **কী সিদ্ধান্ত, কেন, আর কী দাম**? না পারলে ADR-টা এখনো শেষ হয়নি।

শিক্ষকের নোট: ক্লাসের শেষে ২–৩টি ADR পড়ে শোনান। যেটায় "পরিণাম"-এর ঘর সবচেয়ে সৎ, সেটাকেই সবচেয়ে ভালো ADR বলুন — সবচেয়ে বড় সিদ্ধান্তটা নয়।

আলোচনা

চারটি প্রশ্ন — স্ট্যাক নিয়ে

উত্তর লিখে রাখুন, ক্লাস ৮-এ ফিরে আসবে

১. আপনার স্ট্যাকের **তিনটি** পছন্দ লিখুন — প্রতিটির একটি সুবিধা ও একটি দাম।

২. আপনার কোথায় **দুই ভাষা** লাগছে (Python + JavaScript)? সেটা কি সত্যিই দরকার?

৩. আপনার প্রজেক্টে **PDF বা Excel** আউটপুট লাগবে? কোন লাইব্রেরি, আর বাংলা পারবে কি?

৪. আপনার কোন সিদ্ধান্তটা তালিকার **১ বা ২ নম্বরে**, আর সেটা কি এখনো নেওয়া হয়নি?

শিক্ষকের নোট: ৩ নম্বর প্রশ্নটা হালকা মনে হলেও অনেক প্রজেক্টের ডেমো এখানেই আটকায় — বিশেষ করে বাংলা মার্কশিট। যার উত্তর "পরে দেখব", তার ডেমোর শেষ সপ্তাহটা নষ্ট হবে।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
স্ট্যাক = ছয়টি স্তর, ছয়টি সিদ্ধান্ত — শুধু ফ্রেমওয়ার্ক নয়	ভাষা · ফ্রেমওয়ার্ক · ডেটাবেস · পর্দা · ফাইল · সার্ভার
প্রযুক্তি বাছাই হয় প্রতিদান দিয়ে, স্কোর দিয়ে নয়	Django ২ ঘণ্টায় auth · Node-এ দুই সার্ভার · Laravel সমান কিন্তু ভাষা PHP
ভার্সন পিন করা মানে নিয়ন্ত্রণ	`requirements.txt`: ৬টি প্যাকেজ, সব `==` সহ
ADR = ৫ ঘরের এক পাতা, আর খারাপ দিক লেখা বাধ্যতামূলক	তিনটি ভরা ADR — Django · কাস্টম User · টাকার একক
ভুল সিদ্ধান্তও লিখে রাখুন — মুছবেন না	সাবস্ক্রিপশন সংশোধন `8a25724` · ১ মাইগ্রেশন + ৪ টেস্ট

শিক্ষকের নোট: শেষ লাইনটা সবচেয়ে গুরুত্বপূর্ণ নৈতিক শিক্ষা — ডেমোতে "কোন ভুল করেছিলেন?" প্রশ্নের সেরা উত্তর হলো একটা ADR, কারণ সেটা প্রমাণ করে আপনি ভুল ধরতে পারেন।

০৮

পরের ক্লাস — "১১টি অ্যাপ, এক সিস্টেম"

আর্কিটেকচার: অ্যাপ নির্ভরতার গ্রাফ · স্তর ভাগ · কেন এত অ্যাপ

পর্ব ২ · ইঞ্জিনিয়ারিং প্ল্যান · ক্লাস ৮ / ৩৭

১১টি অ্যাপ, এক সিস্টেম

সিলেবাস: প্র্যাকটিক্যাল ৩ · টপিক ৩.৪ (সিস্টেম আর্কিটেকচার)

অ্যাপ নির্ভরতার গ্রাফ · স্তর ভাগ · চক্র ভাঙা

১১ অ্যাপ · ৫,৩৮০ লাইন · ১১ urls.py

শুরু করুন প্রশ্ন দিয়ে

সব কোড এক ফাইলে রাখলে কী হয়?

হাত তুলুন: কেউ কি এক ফাইলে ৫ হাজার লাইন লিখেছেন?

প্রথম সপ্তাহে ঠিকই চলে।

তৃতীয় সপ্তাহে খোঁজা বন্ধ হয়ে যায়।

আর্কিটেকচার মানে সাজানো নয়, **দায়িত্ব ভাগ করা**। অ্যাপ মানে এক ফোল্ডার = এক দায়িত্ব — উপস্থিতি কোথায় থাকে, তার উত্তর যেন একটা শব্দে দেওয়া যায়: **attendance**।
আর সবচেয়ে বড় লাভ: **পরীক্ষা করা যায়** — একটা অ্যাপের টেস্ট বাকিগুলো না জেনেও চলে।

শিক্ষকের নোট: কেউ "আমার প্রজেক্টে ৫টা ফাইল" বললে জিজ্ঞেস করুন — "উপস্থিতির কোড কোন ফাইলে?" উত্তর যদি হয় "views.py-তে সব আছে", তাহলে আজকের ক্লাসটা তার জন্য সবচেয়ে বেশি কাজে লাগবে।

ধাপ ১

একটি অ্যাপের ভেতরে কী থাকে

চারটি বাধ্যতামূলক ফাইল, তিনটি ঐচ্ছিক — প্রতিটির একটাই কাজ

```
apps/attendance/
  models.py      ← ডেটা: কোন টেবিল
  views.py       ← কাজ: কী হবে
  urls.py        ← ঠিকানা: কোন রুট
  tests.py       ← প্রমাণ: কী যাচাই
  forms.py       ← ইনপুট যাচাই
  services.py    ← জটিল নিয়ম
  migrations/    ← ডেটাবেসের ইতিহাস
```

প্রতিটির এক লাইনের দায়িত্ব

models — ডেটার আকৃতি; এখানে কোনো HTTP নেই

views — অনুরোধ নেওয়া, উত্তর দেওয়া

urls — কোন পথ কোন কাজে

tests — "এটা ঠিক আছে" বলার প্রমাণ

forms — ব্যবহারকারীর ইনপুট ছাঁকা

services — যে নিয়ম view-এ আঁটে না

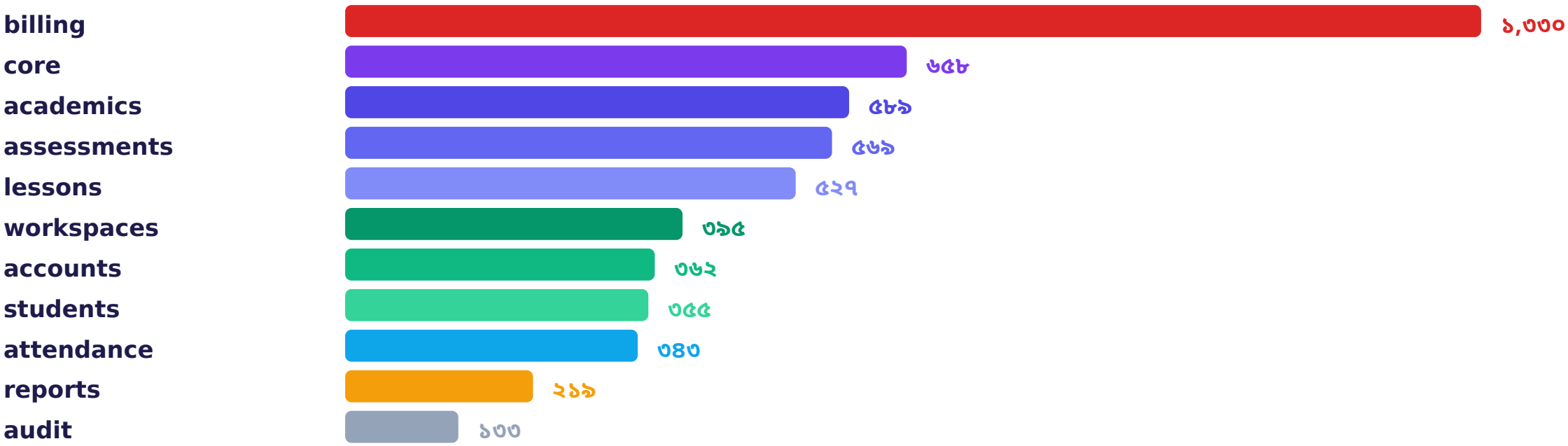
গন্ধ: models.py-তে `HttpResponse` বা views.py-তে ২০০ লাইনের হিসাব — দুটোই দায়িত্ব মিশে যাওয়ার চিহ্ন।

শিক্ষকের নোট: services.py-র উদাহরণ দিন — `apps/billing/services.py`-তে `seat_block_reason` ফাংশন আছে, যেটা view-এ আঁটে না। এই ভাগটা পরে ক্লাস ১৭-এ স্লাইস ১-এ ফিরে আসবে।

ধাপ ২

১১টি অ্যাপের আসল আকার

লাইন গোনা হয়েছে টেস্ট ও মাইগ্রেশন বাদ দিয়ে — `apps/*/`



billing সবচেয়ে বড় — অথচ এটা ব্যবসার অংশ, স্কুলের কাজ নয়। কারণ: সাবস্ক্রিপশন, ট্রায়াল, ইনভয়েস, হোয়াইট-লেবেল, প্ল্যাটফর্ম কনসোল — সব একসাথে।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন — "তোমার সবচেয়ে বড় ফাইল কোনটা?" যদি উত্তর "views.py, ৮০০ লাইন" হয়, তাহলে ওখানেই ভাঙার দরকার। আগে আকার দেখুন, তারপর দায়িত্ব।

ধাপ ২ · গঠন

apps/ ফোল্ডারের আসল গাছ — পাশে টেস্টের সংখ্যা

১১টি অ্যাপ, ১১টি urls.py, ১০৩টি টেস্ট — গোনা সংখ্যা

```
apps/
  accounts/      362     9 tests
  workspaces/    395    10 tests
  core/          658    19 tests
  academics/     589     4 tests
  students/      355     5 tests
  attendance/    343     5 tests
  lessons/       527     4 tests
  assessments/   569    11 tests
  reports/       219     — tests
  audit/         133     — tests
  billing/     1,330    36 tests

config/
  settings/  base · dev · sqlite · prod
  urls.py    ১১টি include
```

এই গাছ থেকে তিনটি কথা

১. প্রতিটি অ্যাপে `urls.py` আছে — অর্থাৎ ঠিকানা অ্যাপের নিজের, `config`-এর নয়।

২. টেস্ট **অ্যাপের ভেতরেই** থাকে — কোড আর প্রমাণ এক জায়গায়।

৩. `settings/` চারটি ফাইলে ভাগ — পরিবেশ বদলালে শুধু ফাইল বদলায়।

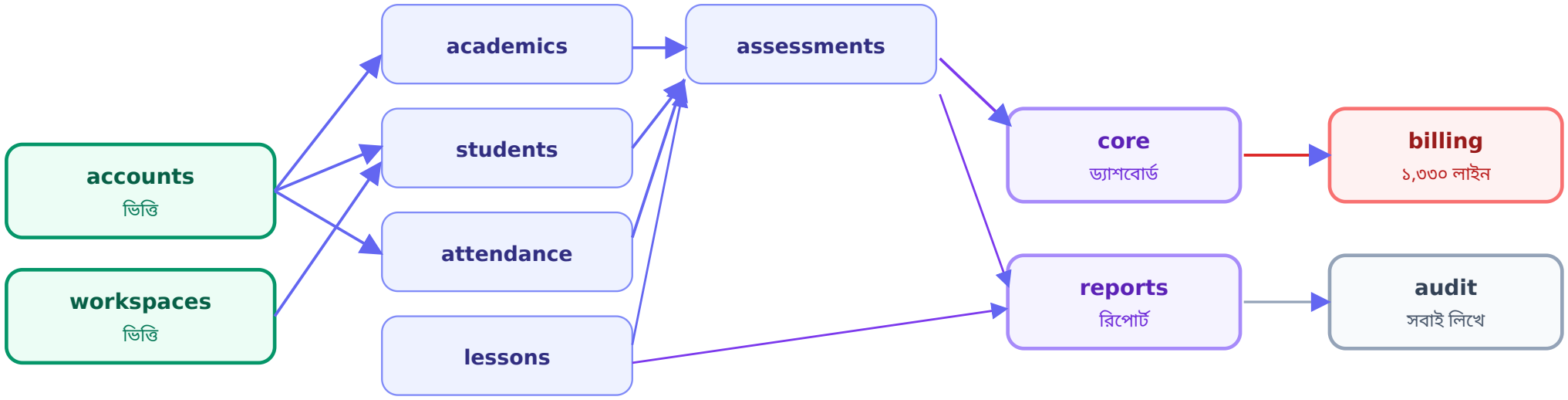
config আর apps আলাদা কেন? `config` হলো বিল্ডিংয়ের বিদ্যুৎ-সংযোগ, `apps` হলো ভেতরের ঘরগুলো। এক জায়গায় গুলিয়ে ফেললে অ্যাপ পুনরায় ব্যবহার করা যায় না।

শিক্ষকের নোট: ক্লাসের আগে টার্মিনালে `find apps -maxdepth 1 -type d` আর `grep -c "def test_" apps/*/tests.py` চালিয়ে আসুন — সংখ্যাগুলো নিজের চোখে দেখলে ছাত্ররা বেশি বিশ্বাস করে।

ধাপ ৩

আসল নির্ভরতার গ্রাফ

তির মানে "কে কাকে import করে" — কোড থেকে যাচাই করা



তিরের দিকটাই আসল: **accounts/workspaces কাউকে import করে না** (প্রায়), আর সবাই ওদের import করে। এটাই ভিত্তি হওয়ার প্রমাণ।

শিক্ষকের নোট: এই গ্রাফটা ড্রয়ারে আঁকার নয়, টার্মিনাল থেকে বের করার — `grep -rn "from apps\." apps/`। ছাত্রদের দেখান কীভাবে কমান্ড দিয়ে নিজের গ্রাফ নিজে বের করবে।

ধাপ ৪

স্তর ভাগ — পাঁচটি স্তর, এক দিকের তির

নিয়ম একটাই: উপরের স্তর নিচের স্তরকে ডাকতে পারে, উল্টোটা নয়

স্তর	অ্যাপ	দায়িত্ব	উপরের স্তরকে কী দেয়
১ · ভিত্তি	accounts · workspaces	কে আপনি, কোন প্রতিষ্ঠানে	ইউজার, সদস্যতা, ভূমিকা
২ · ডোমেইন	academics · students · attendance · lessons · assessments	স্কুলের আসল কাজ	ক্লাস, শিক্ষার্থী, হাজিরা, নম্বর
৩ · সংযোজন	core · reports	একসাথে জোড়া লগিয়ে দেখানো	ড্যাশবোর্ড, সামারি
৪ · ব্যবসা	billing	প্ল্যান, সিট, ইনভয়েস	সীমা ও সাবস্ক্রিপশন
৫ · নিরাপত্তা	audit	কে কী করল, কবে	লগ এন্ট্রি (সবাই লেখে)

বাস্তব সীমা: audit সবাই ব্যবহার করে, কিন্তু কেউ audit-কে ব্যবহার করে না — তাই সে সবার নিচে নেই, **পাশে** আছে। স্তর সবসময় সিঁড়ি নয়, কখনো পাশের দরজা।

শিক্ষকের নোট: ছাত্রদের নিজের প্রজেক্টে এই পাঁচটি স্তর লিখতে বলুন। যার প্রজেক্টে সব কিছু এক স্তরে, তার আর্কিটেকচার এখনো তৈরি হয়নি — শুধু ফোল্ডার আছে।

ধাপ ৫

চক্র (cycle) — আর প্রজেক্টে সেটা কীভাবে ভাঙা হয়েছে

students ↔ billing: দুই দিকের import, তবু সিস্টেম চলে — কীভাবে?

সমস্যাটা কী

students/views.py চায় `seat_block_reason` — সিট ভরা কি না জানতে।
billing/models.py চায় `StudentProfile` — কতজন শিক্ষার্থী আছে গুনতে।

ফাইলের উপরে দুটো import লিখলে Python ঘুরে ঘুরে আটকে যায় — `ImportError`।

সমাধান — ফাংশনের ভেতরে import

```
def add_student(request, slug):
    from apps.billing.services \
        import seat_block_reason
    if seat_block_reason(...):
        return render(request, "seat_full.html")
```

import টা **কল করার সময়** হয়, লোডের সময় নয় — তাই চক্র ভাঙে।

কিন্তু সতর্কতা: **এটা ওষুধ, খাদ্য নয়।** বারবার এমন import দেখলে বুঝবেন দায়িত্ব ভাগ ভুল হয়েছে — সমাধান অ্যাপ সরানো, import লুকানো নয়।

শিক্ষকের নোট: `core/views.py`-তেও একই কৌশল — ফাংশনের ভেতরে মডেল import। ছাত্রদের বলুন: "প্রথমে ভাবো দায়িত্ব কোথায় থাকা উচিত, তারপর দরকার হলে import লুকাও।"

ধাপ ৬

ফাংশনের ভেতরে import — তিনটে বৈধ কারণ

তিনটির বাইরে হলে সেটা লুকোচুরি, ডিজাইন নয়

🔁

১. সত্যিকারের চক্র

দুই অ্যাপ একে অন্যকে দরকার, কিন্তু একটাকে উপরে তুলতেই হবে। তখন **এক দিক** ফাংশনের ভেতরে।

students → billing

🐢

২. ধীর স্টার্টআপ

ভারী লাইব্রেরি (যেমন reportlab) শুধু PDF চাওয়ার সময় দরকার। সার্ভার চালু হওয়ার সময় নয়।

মার্কশিট PDF

🧪

৩. টেস্টে বদলানো

টেস্টের ভেতরে import করলে `mock` বসানো সহজ — কারণ ওই সময়েই নামটা খোঁজা হয়।

১০৩টি টেস্ট

অবৈধ কারণ: "উপরে লিখলে এরর আসছিল" — এররটা এল কেন, সেটাই আসল প্রশ্ন। লুকালে ডিজাইনের সমস্যা থেকে যায়, শুধু দেখতে পান না।

শিক্ষকের নোট: ছাত্রদের বলুন নিজের প্রজেক্টে ফাংশনের ভেতরের import গুনতে — এক-দুটো স্বাভাবিক, দশটা হলে অ্যাপ ভাগ ভুল হয়েছে।

ধাপ ৭

core অ্যাপ — সবকিছুর জোড়া লাগানোর জায়গা

তিনটি ফাইল, তিনটি দায়িত্ব — আর এখানেই সবচেয়ে বড় ফাঁদ

ফাইল	দায়িত্ব	কেন এখানে
`core/navigation.py`	মেনুর একমাত্র তালিকা — কোন ভূমিকা কী দেখবে	একই তালিকা মেনু আর সার্ভারের যাচাইয়ে ব্যবহার হয়, তাই দুই জায়গায় অমিল হওয়া অসম্ভব
`core/permissions.py`	কে কোন প্রতিষ্ঠানে ঢুকতে পারবে	প্রতিটি view-তে আলাদা যাচাই লিখলে একটায় ভুল হলে ফুটো
`core/views.py`	চারটি ড্যাশবোর্ড (F-13 · F-16 · F-29 · F-56)	ড্যাশবোর্ড কোনো এক অ্যাপের নয় — সব অ্যাপের ডেটার সারাংশ

core-এ থাকা উচিত: ক্রস-কাটিং জিনিস — মেনু, পারমিশন, হোম, শেয়ার্ড ট্যাগ।

core-এ থাকা উচিত নয়: উপস্থিতির হিসাব বা নম্বরের নিয়ম — সেগুলো ওই অ্যাপের নিজের কাজ।

শিক্ষকের নোট: `navigation.py`-র কথা মনে করিয়ে দিন — ক্লাস ১-এর "মেনুতে নেই, তবু ঢোকা যায়?" স্লাইডে এটাই প্রমাণ ছিল। এখন তারা দেখছে সেটা কোন অ্যাপে থাকে।

ধাপ ৮

১১টি urls.py — ঠিকানার নিয়ম

সব অ্যাপ একই ছাদে যুক্ত, কিন্তু প্রত্যেকের নিজের পথ

```
# config/urls.py — শুধু জোড়া লাগানো
path("", include("apps.accounts.urls")),
path("", include("apps.workspaces.urls")),
path("", include("apps.core.urls")),
path("", include("apps.academics.urls")),
path("", include("apps.students.urls")),
path("", include("apps.attendance.urls")),
path("", include("apps.lessons.urls")),
path("", include("apps.assessments.urls")),
path("", include("apps.reports.urls")),
path("", include("apps.audit.urls")),
path("", include("apps.billing.urls")),
```

পথের তিনটি পরিবার

পাবলিক — `accounts/login/` · `pricing/`

প্রতিষ্ঠানভিত্তিক — `w/<slug>/students/`

প্ল্যাটফর্ম — `ops/workspaces/` · `workspaces/`

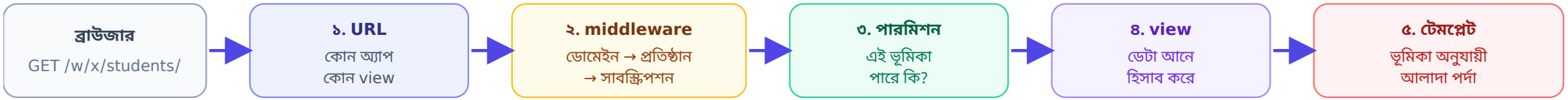
`w/<slug>/` উপসর্গটাই **টেন্যান্টের চিহ্ন** — এই উপসর্গ দেখলেই বোঝা যায় ওই পাতায় প্রতিষ্ঠান যাচাই লাগবে।

শিক্ষকের নোট: জিজ্ঞেস করুন — "কেন সব include-এ prefix খালি?" উত্তর: কারণ প্রতিটি অ্যাপ নিজের পথ নিজে লিখছে; prefix চাপালে URL দুইবার সাজাতে হতো। এটা সচেতন সিদ্ধান্ত, দুর্ঘটনা নয়।

ধাপ ৯ · যাত্রা

একটি অনুরোধের যাত্রা — ৬টি দরজা

ব্রাউজার থেকে পর্দা পর্যন্ত; প্রতিটি দরজায় কেউ না কেউ থামাতে পারে



দরজা ২-এ পরপর তিনটি middleware: CustomDomain → CurrentWorkspace → SubscriptionGate — ক্রম বদলালে নিরাপত্তার ফাঁক তৈরি হয়।

দরজা ৩ ও ৫ মিলে দ্বৈত যাচাই: মেনুতে না দেখানো, আর সার্ভারেও আটকানো — একটার উপর ভরসা করলে ফুটো।

এই ছয়টি দরজা **পর্ব ৪-এর স্লাইস ১**-এ বিস্তারিত আসবে। এখন শুধু মনে রাখুন: **প্রতিটি দরজা একটা অ্যাপের কাজ**।

শিক্ষকের নোট: বোর্ডে এই ছয়টি বাক্স আঁকুন, আর ছাত্রদের জিজ্ঞেস করুন — "ভুল প্রতিষ্ঠানের ছাত্র দেখানো ঠেকানো হয় কোন দরজায়?" উত্তর: দরজা ২ আর ৩, দুই জায়গাতেই।

ধাপ ১০

স্তর ভাগ ঠিক আছে কি না — তিনটি আসল পরীক্ষা

আর্কিটেকচার সুন্দর দেখায় কি না নয়, প্রশ্নটা হলো: টিকে আছে কি না

পরীক্ষা	কীভাবে করবেন	IMS-এ ফল
১. মুছে ফেলার পরীক্ষা	একটা অ্যাপ বাদ দিয়ে সিস্টেম চলে কি?	audit বাদ দিলে চলে (শুধু লগ থাকে না) · accounts বাদ দিলে কিছুই চলে না ✔
২. তিরের দিক পরীক্ষা	নিচের স্তর কি উপরের স্তরকে import করে?	accounts → workspaces (উপরের দিকে এক ধাপ) — একমাত্র ছাড়, কারণ সদস্যতা ছাড়া ইউজার অর্থহীন
৩. নাম বদলের পরীক্ষা	একটা অ্যাপের নাম বদলালে কত জায়গা বদলাতে হয়?	rename TMS → IMS-এ দেখা গেছে: নাম ছড়িয়ে থাকলে ৩টি কমিট লাগে — তাই `PRODUCT_NAME` settings-এ

> **২ নম্বর পরীক্ষায় ছাড় থাকা স্বাভাবিক** — আর্কিটেকচারের নিয়ম ভাঙা যায়, কিন্তু **লিখে রাখতে হয় কেন**। নাহলে পরে কেউ সেটাকে অনুমতি ভেবে আরও ভাঙবে।

শিক্ষকের নোট: "মুছে ফেলার পরীক্ষা" ক্লাসে করে দেখান — একটা অ্যাপের ফোল্ডার সাময়িকভাবে সরিয়ে (git stash দিয়ে) দেখা যায় কী ভাঙে। এটা সবচেয়ে দ্রুত শেখার উপায়।

ধাপ ১১

কখন একটি অ্যাপ ভাঙতে হয় — পাঁচটি সংকেত

অ্যাপ বড় হওয়া দোষ নয়; দায়িত্ব মিশে যাওয়া দোষ

১. দুই কাজ, এক ফাইল

views.py-তে ২০০ লাইনের হিসাব — ওটা `services.py`-র কাজ।

২. দুই দলে ভাগ হয় মেনু

একই অ্যাপের পর্দা যদি দুই সম্পূর্ণ ভিন্ন ভূমিকার হয়, ভাগ করার সময়।

৩. টেস্ট ফাইল ৫০০+ লাইন

টেস্ট ভাগ করা কঠিন মানে কোডও ভাগ করা কঠিন।

৪. একই টেবিল দুই অ্যাপে

দুই জায়গায় একই মডেল — ডেটার মালিক কে, সেটাই অস্পষ্ট।

৫. সাইকেল বাড়ছে

ফাংশনের ভেতরের import প্রতিনিয়ত বাড়ছে — দায়িত্ব ভাগ ভুল।

যখন ভাঙবেন না

শুধু "ফাইল বড়" — এই এক কারণে ভাঙলে ৬টি ফাইলে ৬টি ছোট সমস্যা হবে।

প্রজেক্টে **billing** ১,৩৩০ লাইনে পৌঁছেছে — কারণ কোম্পানির নিজের কাজ আর গ্রাহকের কাজ এক অ্যাপে। এটা আরেকটা ভাঙার প্রার্থী, v2-তে।

শিক্ষকের নোট: শেষ বাক্যটা গুরুত্বপূর্ণ — নিজের প্রজেক্টের সীমাটাও বলুন। ছাত্ররা তখনই বিশ্বাস করে, যখন দেখে আপনি নিজের কাজের দুর্বলতাও বলছেন।

ধাপ ১২

একটা মডেল কোন অ্যাপে থাকবে — সাতটি প্রশ্ন

ডেটাবেস ডিজাইনের আগে দায়িত্ব ঠিক করে নিন

প্রশ্ন	যদি "হ্যাঁ" হয়
এটা কি মানুষ সম্পর্কে (কে আপনি)?	**accounts**
এটা কি প্রতিষ্ঠান/সদস্যতা সম্পর্কে?	**workspaces**
এটা কি কাঠামো (বছর, ক্লাস, বিষয়)?	**academics**
এটা কি একজন শিক্ষার্থীর পরিচয় ও অবস্থা?	**students**
এটা কি দিনের ঘটনা (কে উপস্থিত ছিল)?	**attendance**
এটা কি পড়ানোর বিষয় (সিলেবাস, পাঠ, কাজ)?	**lessons**
এটা কি নম্বর বা ফলাফল?	**assessments**
এটা কি টাকা বা সীমা?	**billing**
হ্যাঁ-র সংখ্যা দুইয়ের বেশি?	**মডেলটা ভাঙতে হবে — দুই অ্যাপের দুই টেবিল**

একই মডেল দুই জায়গায় রাখার প্রলোভন সবচেয়ে বড় ফাঁদ — দুই কপি ডেটা দুই সময়ে বদলায়, তারপর কেউ জানে না কোনটা সত্যি।

শিক্ষকের নোট: শেষ সারিটা সবচেয়ে কাজে লাগে — ছাত্ররা প্রায়ই "Student" মডেলে ক্লাস, ফি, হাজিরা, ফল — সব ঢুকিয়ে ফেলে। এখনই থামান, নইলে ক্লাস ১৪-এ তাদের ERD ভাঙবে।

ধাপ ১৩

আর্কিটেকচার ডায়াগ্রাম — কী দেখাবেন, কী বাদ দেবেন

পর্দায় সব দেখানো যায় না; দেখাতে হয় **সিদ্ধান্ত**টা

দেখাবেন

✓ স্তর ভাগ (ভিত্তি → ডোমেইন → সংযোজন)
✓ এক দিকের তির (কে কাকে ডাকে)
✓ টেন্যান্টের সীমানা — `w/<slug>/`
✓ দুইটি বাইরের জিনিস: ব্রাউজার ও ডেটাবেস
✓ প্রতিটি বাক্সে **লাইনের সংখ্যা** — আকার বোঝায়

বাদ দেবেন

✗ প্রতিটি ফাংশনের নাম
✗ প্রতিটি মডেলের ফিল্ড (ওটা ERD-র কাজ)
✗ লাইব্রেরির ভেতরের গঠন
✗ ফাংশনের ভেতরের import-গুলো
✗ যা কেউ বদলাবে না — যেমন `manage.py`

পরীক্ষা: **এই ডায়াগ্রাম দেখে নতুন কেউ বলতে পারবে — নতুন ফিচারটা কোন বাক্সে যাবে?** না পারলে ডায়াগ্রামটা সাজসজ্জা, নকশা নয়।

শিক্ষকের নোট: ছাত্রদের ডায়াগ্রাম ২ মিনিটে আঁকতে বলুন — সময় কম দিলে তারা আসল সিদ্ধান্তটাই আঁকে, সাজসজ্জা বাদ পড়ে। এটাই সবচেয়ে ভালো পরীক্ষা।

ক্লাসের কাজ

নিজের প্রজেক্টের অ্যাপ ভাগ করুন — ২০ মিনিট

ফলাফল: স্তরের তালিকা + একটি ডায়াগ্রাম

১. দায়িত্বের তালিকা

প্রজেক্টের ১০–১৫টি বড় কাজ লিখুন — ফিচার নয়, **দায়িত্ব**।

২. দলে ভাগ

যেগুলো একসাথে বদলায়, এক অ্যাপে। সাধারণত ৫–৭টি অ্যাপ হয়।

৩. স্তর সাজান

ভিত্তি → ডোমেইন → সংযোজন — এক দিকের তিরে সাজিয়ে নিন।

৪. চক্র খুঁজুন

দুই অ্যাপ একে অন্যকে ডাকছে? দায়িত্ব সরান, import লুকাবেন না।

৫. আঁকুন

প্রতি বাক্সে নাম + কাজের এক লাইন — ২ মিনিটের সীমায়।

যাচাইয়ের প্রশ্ন: আপনার ডায়াগ্রামে **accounts/ভিত্তি** কি কাউকে import করে? যদি করে — তিরটা উল্টো, সেটা ঠিক করুন।

শিক্ষকের নোট: ২০ মিনিট শেষে ২–৩টি ডায়াগ্রাম বোর্ডে টাঙান, একসাথে তিরের দিক মিলিয়ে দেখুন। ভুল তির সবার সামনে ধরুন — এটাই এই কাজটির সবচেয়ে বড় শিক্ষা।

আলোচনা

চারটি প্রশ্ন — আর্কিটেকচার নিয়ে

উত্তর লিখে রাখুন — ক্লাস ১১-এ এগুলোই আবার আসবে

১. আপনার প্রজেক্টে **সবচেয়ে বড় ফাইল** কোনটা, কত লাইন? সে কি এক দায়িত্ব, না তিনটি?

২. আপনার অ্যাপগুলোর মধ্যে **চক্র** আছে? কোন দিকটা ফাংশনের ভেতরে লুকানো?

৩. আপনার **মুছে ফেলার পরীক্ষা** — কোন অ্যাপ বাদ দিলে সিস্টেম এখনো চলবে?

৪. আপনার কোন মডেলটা **দুই অ্যাপের** কাজ করছে? সেটা ভাঙলে কোনটা কোথায় যাবে?

শিক্ষকের নোট: ৪ নম্বরের উত্তর প্রায়ই "Student — ক্লাস, ফি, হাজিরা সব ওখানে" আসবে। বলুন: এটা আজ ঠিক করা সস্তা, ক্লাস ১৪-এর পরে ডেটা ভরা অবস্থায় ঠিক করা অনেক ব্যয়বহুল।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
অ্যাপ = এক দায়িত্ব, ফোল্ডার নয় শুধু	models · views · urls · tests · forms · services — ছয়টি আলাদা কাজ
আকার দেখলে দায়িত্বের ভারসাম্য বোঝা যায়	billing ১,৩৩০ · core ৬৫৮ · academics ৫৮৯ · audit ১৩৩
তির এক দিকে — ভিত্তি কাউকে ডাকে না, সবাই ভিত্তিকে ডাকে	accounts → workspaces (এক ধাপ, লিখিত ছাড়)
চক্র ভাঙা হয় দায়িত্ব সরিয়ে, import লুকিয়ে নয়	students ↔ billing — ফাংশনের ভেতরে import, তিনটি বৈধ কারণ
আর্কিটেকচার যাচাই হয় তিনটি পরীক্ষায়	মুছে ফেলা · তিরের দিক · নাম বদল (TMS→IMS: ৩ কমিট)

শিক্ষকের নোট: তৃতীয় ও চতুর্থ লাইন একসাথে পড়ুন — তির এক দিকের হলে চক্রই জন্মায় না। চক্র দেখা মানে তিরের দিক কোথাও উল্টো হয়েছে — এটাই ক্লাসের মূল কথা।

০৯

পরের ক্লাস — "কে কী পারবে, কে পারবে না"

রোল ম্যাট্রিক্স · ৫×৫ অনুমতি গ্রিড · মেনু ও সার্ভারের দ্বৈত যাচাই

পর্ব ২ · ইঞ্জিনিয়ারিং প্ল্যান · ক্লাস ৯ / ৩৭

কে কী পারবে, কে পারবে না

সিলেবাস: প্র্যাকটিক্যাল ৩ · টপিক ৩.৫ (ভূমিকা ও অনুমতি)
রোল ম্যাট্রিক্স · ৪০৪ বনাম ৪০৩ · এক তালিকা তিন ভোক্তা

২২ মেনু আইটেম · ৫ গ্রুপ · ৪ ভূমিকা

শুরু করুন প্রশ্ন দিয়ে

মেনু থেকে লুকিয়ে দিলেই কি নিরাপত্তা হয়?

প্রথম ক্লাসে এই প্রশ্নটা এসেছিল — আজ উত্তরটা কোডে দেখব

না। মেনু লুকানো, নিরাপত্তা আটকানো।

দুটো আলাদা কাজ — আর দুটোই লাগবে।

কারণ ব্যবহারকারী মেনু দেখে না, **ঠিকানা টাইপ করে**। তাই প্রতিটি অনুরোধে সার্ভারকে নিজে জিজ্ঞেস করতে হয়: "এই মানুষটা এই প্রতিষ্ঠানের সদস্য কি? তার ভূমিকা কি এই কাজ করতে দেয়?"
আজকের ক্লাসে এই দুই প্রশ্নের নকশা — এবং একটি তৃতীয় প্রশ্ন।

শিক্ষকের নোট: কেউ "মেনু লুকালেই তো হয়" বললে বলুন: "ঠিক আছে, তুমি শিক্ষক। এখন ব্রাউজারে `/w/<slug>/audit/` টাইপ করো।" লাইভে দেখালে এই একটা মুহূর্তই পুরো ক্লাসের ভিত্তি হয়ে যাবে।

ধাপ ১

তিনটি প্রশ্ন — প্রতিটি অনুরোধে

`apps/core/permissions.py` -এর নিজের মন্তব্য থেকে

🏢

১. টেন্যান্ট যাচাই

"এই ইউজার কি URL-এ থাকা প্রতিষ্ঠানের **সদস্য**?"

`get_membership(user, workspace)`

না হলে → ৪০৪

🎭

২. ভূমিকা যাচাই

"এই প্রতিষ্ঠানে তার **ভূমিকা** কি এই কাজ করতে দেয়?"

`allowed_roles`

না হলে → ৪০৩

📚

৩. বরাদ্দ যাচাই

"এই শিক্ষক কি **এই ক্লাসে** পড়ান?"

শিক্ষামূলক বস্তুতে তৃতীয় প্রশ্ন

না হলে → ৪০৪ (কিছু নেই)

প্রথম দুই প্রশ্ন **সব** প্রতিষ্ঠান-পাতায়; তৃতীয়টি শুধু যেখানে শিক্ষকের নিজের ক্লাসের কথা — যেমন হাজিরা, নম্বর।

শিক্ষকের নোট: তৃতীয় প্রশ্নটা সবচেয়ে বেশি বাদ পড়ে — ফলে একই প্রতিষ্ঠানের দুই শিক্ষক একে অন্যের ক্লাসের নম্বর বদলাতে পারেন। এটা স্কুলের দৃষ্টিতে গুরুতর ফুটো।

ধাপ ২

NAV ফাইল — ৫ গ্রুপ, ২২ আইটেম, এক তালিকা

`apps/core/navigation.py` — মেনুর একমাত্র সত্য

```
NAV = [
 { "title": None,  "items": [
     _item("Dashboard", …, STAFF+STUDENT) ]},

 { "title": "Teaching", "items": [          # ১০টি
     Students · Classes & subjects
     Attendance · Syllabus · Lesson log
     Homework · Materials · Exams
     Reports · My oversight area ]},

 { "title": "My learning", "items": [        # ৩টি
     My homework · My results · Materials ]},

 { "title": "Administration", "items": [     # ৬টি
     Coordinator scope · Audit log
     Workspace settings · Grading
     Branding · Subscription ]},

 { "title": "Account", "items": [            # ২টি
     My profile · Switch workspace ]},
]
```

প্রতিটি আইটেমে পাঁচটি তথ্য

label — যা দেখা যাবে

icon — কোন চিহ্ন

url_name — কোন রুট (নাম দিয়ে, ঠিকানা দিয়ে নয়)

roles — কারা দেখবে

badge — সংখ্যা দেখানোর জায়গা

url_name কেন? `reverse()` দিয়ে তৈরি হয় — তাই রুট বদলালে মেনু নিজেই বদলে যায়, হাতে ঠিক করতে হয় না।

শিক্ষকের নোট: "Teaching"-এর ১০টি আইটেম বোর্ডে লিখুন — এটাই শিক্ষকের পুরো দিন। তারপর জিজ্ঞেস করুন: এর মধ্যে কোনটা শিক্ষার্থী দেখা উচিত? উত্তর: একটাও না, তার নিজের তিনটি আলাদা গ্রুপে।

ধাপ ৩

ভূমিকা × মেনু গ্রুপ — আসল সংখ্যা

কোড চালিয়ে গোনা: কোন ভূমিকা কতটি আইটেম দেখে

ভূমিকা	Dashboard	Teaching	My learning	Administration	Account	মোট
প্রতিষ্ঠান অ্যাডমিন	১	৯	—	৬	২	**১৮**
কোঅর্ডিনেটর	১	১০	—	—	২	**১৩**
শিক্ষক	১	৯	—	—	২	**১২**
শিক্ষার্থী	১	—	৩	—	২	**৬**
প্ল্যাটফর্ম প্রশাসক	—	—	—	—	—	**০**

কোঅর্ডিনেটর ১৩, শিক্ষক ১২ — পার্থক্য একটাই: "My oversight area"। বাকি সব এক।

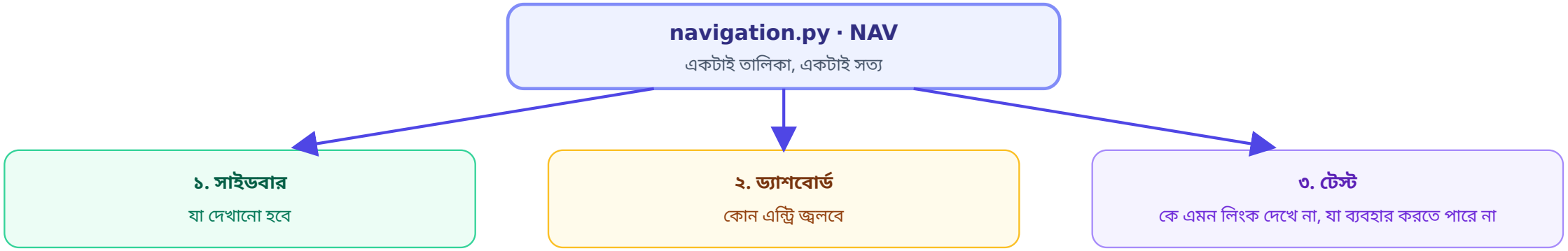
প্ল্যাটফর্ম প্রশাসকের ০ — ইচ্ছাকৃত। তাঁর জগৎ আলাদা: `/ops/`। প্রতিষ্ঠানের মেনুতে তাঁর কিছু নেই, কারণ সেখানে ঢোকার অধিকারও নেই।

শিক্ষকের নোট: "০" ঘরটা নিয়ে আলোচনা করুন — ছাত্ররা ভাবে সুপার-অ্যাডমিন সব পারে। বাস্তবে সে প্রতিষ্ঠানের ভেতরের কোনো ব্যক্তিগত রেকর্ড দেখতে পারে না; পারে শুধু প্রতিষ্ঠান বন্ধ করা বা প্ল্যান বদলানো।

ধাপ ৪

এক তালিকা, তিন ভোক্তা — আর একটি নিরাপদ ব্যর্থতা

ফাইলের মন্তব্যটাই স্লাইডের শিরোনাম: "One definition, three consumers"



নিরাপদ ব্যর্থতা: নতুন পাতা যোগ করে এই ফাইল ভুলে গেলে পাতা মেনুতে আসে না — অর্থাৎ "অদৃশ্য" হয়, "অনিরাপদ" নয়।

কিন্তু সতর্কতা: এই ফাইল শুধু **দেখানো** ঠিক করে। **আটকানো** সবসময় সার্ভারে — দুই জায়গায় একই নিয়ম, কিন্তু দুই দায়িত্ব।

শিক্ষকের নোট: "নিরাপদ ব্যর্থতা" ধারণাটা নতুন — ব্যাখ্যা করুন: ভুল হলে সিস্টেম যেন কম দেখায়, বেশি নয়। বেশি দেখানো মানে ফুটো, কম দেখানো মানে শুধু অসুবিধা।

ধাপ ৫

তিনটি ভূমিকা-দল — কোডের তিনটি ধ্রুবক

প্রতিটি আইটেমে আলাদা করে ভূমিকা লেখা যায়, কিন্তু তিনটি দল বারবার লাগে

⚙️

CONFIG_ROLES

`(ORG_ADMIN,)`

প্রতিষ্ঠান-ব্যাপী কনফিগ: **Branding · Subscription**

কেন শুধু অ্যাডমিন: পরিচয় ও টাকা — দুটোই প্রতিষ্ঠানের সিদ্ধান্ত

🗂️

MANAGER_ROLES

`(ORG_ADMIN,)`

ব্যবস্থাপনার পর্দা: **Coordinator scope · Audit · Settings · Grading**

কোঅর্ডিনেটর এখানে নেই — কারণ নিজের ক্ষমতা নিজে বাড়াতে পারবে না

🎓

STAFF_ROLES

`(ORG_ADMIN, COORDINATOR, TEACHER)`

পড়ানোর কাজ: **Teaching-এর ৯–১০টি আইটেম**

কেন তিনজন: অ্যাডমিনও পড়াতে পারেন, কোঅর্ডিনেটর শিক্ষকও

তিনটি দল = তিনটি **নীতি**। নতুন আইটেম যোগ করার সময় ভাবতে হয় না "কারা দেখবে" — শুধু "কোন দলে"।

শিক্ষকের নোট: জিজ্ঞেস করুন — "কোঅর্ডিনেটরকে MANAGER_ROLES-এ যোগ করলে কী হবে?" উত্তর: সে নিজের স্কোপ বাড়িয়ে ফেলতে পারবে — অর্থাৎ ক্ষমতার সীমা ভেঙে যাবে।

ধাপ ৬

৪০৪ বনাম ৪০৩ — কোনটা কখন

ভুলটা শুধু বার্তা নয়, তথ্য ফাঁসের প্রশ্ন

পরিস্থিতি	কোড	কেন এটাই	ফল
অন্য প্রতিষ্ঠানের সদস্য নন	**৪০৪**	৪০৩ দিলে বোঝা যেত "এই প্রতিষ্ঠান আছে, শুধু আপনি নন" — অর্থাৎ প্রতিষ্ঠানের অস্তিত্ব ফাঁস	"{slug} নেই"
সদস্য, কিন্তু ভূমিকা এই কাজে নয়	**৪০৩**	সে প্রতিষ্ঠানটা জানে, ভেতরে আছে — তাই লুকানোর কিছু নেই, শুধু অনুমতি নেই	"আপনার ভূমিকা এটা করতে দেয় না"
শিক্ষক, কিন্তু ক্লাসটা তার নয়	**৪০৪**	অন্য ক্লাসের অস্তিত্ব জানানোর দরকার নেই	"ক্লাস পাওয়া যায়নি"
লগইনই করা নেই	**৩০২**	লগইন পাতায় পাঠানো — আর ফেরার ঠিকানা সাথে রাখা	`?next=/w/…`

কোডে হুবহু: `raise Http404("Workspace not found.")` — মন্তব্য সহ: *"A platform admin may look at platform-level pages, never at private records of a workspace they do not belong to."*

শিক্ষকের নোট: এই স্লাইডটাই ডেমোতে সবচেয়ে প্রশ্ন তোলে — শিক্ষক লগইন করে অন্য প্রতিষ্ঠানের ঠিকানা টাইপ করলে ৪০৪ পাবেন। লাইভে দেখান।

ধাপ ৭

রিসোর্স ম্যাট্রিক্স — কোন বস্তুতে কে কী

ভূমিকা নয়, **বস্তু** ধরে ভাবুন — এটাই আসল নকশা

বস্তু	প্রশাসক	অ্যাডমিন	কোঅর্ডি.	শিক্ষক	শিক্ষার্থী
প্রতিষ্ঠান	সব দেখে	নিজের	—	—	—
সদস্য	সব	সব	নিজের বিভাগ	—	—
ক্লাস	—	সব	নিজের	নিজের	নিজের
শিক্ষার্থী	—	সব	নিজের বিভাগ	নিজের ক্লাস	শুধু নিজে
উপস্থিতি	—	সব	নিজের + পর্যবেক্ষণ	নিজের ক্লাস	শুধু নিজের
নম্বর	—	গ্রেড স্কেল	—	নিজের ক্লাস	প্রকাশের পরে
বিল	প্ল্যান	সব	—	—	—
অডিট লগ	—	সব	—	—	—

নিয়ম: প্রতিটি ঘরে **"সব"** / **"নিজের"** / **"ধরন"** / **"—"** — চারটি শব্দের বেশি নয়। ঘরে যদি দুই বাক্য লিখতে হয়, নিয়মটা তখনো তৈরি হয়নি।

শিক্ষকের নোট: "নিজের ক্লাস" ঘরটাই যেখানে তৃতীয় প্রশ্ন (assignment) লাগে। ছাত্রদের বলুন এই গ্রিডে নিজের প্রজেক্টের ৫টি বস্তু বসাতে — বেশিরভাগ দেখবে তাদের কোথাও কোথাও "—" লেখার সাহস হচ্ছে না।

ধাপ ৮

ডেকোরেটর — দুই প্রশ্ন এক জায়গায়

প্রতিটি view-তে নিয়ম লিখলে একটায় ভুল হলে ফুটো; তাই নিয়ম এক জায়গায়

```
@workspace_view(Roles.TEACHER,
                Roles.ORG_ADMIN)
def take_attendance(request, slug, …):
    # এখানে পৌঁছানোর মানেই:
    #   ✓ লগইন করা আছে
    #   ✓ প্রতিষ্ঠান আছে ও সক্রিয়
    #   ✓ সদস্য আছেন
    #   ✓ ভূমিকা অনুমতি দেয়
    # …তাই ভেতরে আর যাচাই লিখতে হয় না
    workspace  = request.workspace
    membership = request.membership
```

ডেকোরেটর কী করে (ক্রম অনুযায়ী)

১. লগইন নেই → ৩০২ + `?next=`

২. প্রতিষ্ঠান নেই/বন্ধ → ৪০৪

৩. সদস্য নন → ৪০৪

৪. ভূমিকা মেলে না → ৪০৩

৫. সব ঠিক → `request.workspace` ও `request.membership` বসিয়ে view চালানো

দারুণ কৌশল: ভেতরে `request.workspace` তৈরি হয়ে আসে — ফলে view ভুল করে অন্য প্রতিষ্ঠানের ডেটা আনতে **পারেই না**।

শিক্ষকের নোট: শেষ লাইনটা ভালো করে বলুন — "ভুল করা অসম্ভব বানানো" (make it impossible to get wrong) হলো ভালো আর্কিটেকচারের লক্ষণ। এই ধারণাটা ক্লাস ১৭-এ আবার আসবে।

ধাপ ৯ · প্রমাণ

অনুমতি যাচাই করা হয় টেস্ট দিয়ে — নাম হুবহু

তালিকার তৃতীয় ভোক্তা: "কোন ভূমিকা এমন লিংক দেখে না, যা সে খুলতে পারে না"

টেস্টের নাম (কোডে যেমন আছে)	কী প্রমাণ করে
`test_each_role_is_offered_only_its_own_links`	মেনু ভূমিকা অনুযায়ী সত্যিই ভিন্ন
`test_no_role_sees_a_link_it_cannot_open`	প্রতিটি দেখানো লিংক সত্যিই খোলে
`test_teacher_cannot_open_administration_pages_by_typing_the_url`	ঠিকানা টাইপ করেও লাভ নেই
`test_coordinator_cannot_open_owner_only_pages`	নিজের ক্ষমতা নিজে বাড়ানো যায় না
`test_student_cannot_open_teaching_pages`	শিক্ষার্থীর দরজা বন্ধ
`test_non_member_gets_404_not_403`	প্রতিষ্ঠানের অস্তিত্ব ফাঁস হয় না
`test_cannot_open_another_workspace_by_guessing_its_slug`	slug অনুমান করেও ঢোকা যায় না

লক্ষ্য করুন নামগুলোর ভাষা: **"cannot" আর "gets 404 not 403"** — টেস্টের নামেই নিয়মটা লেখা আছে, তাই ব্যর্থ হলে কী ভাঙল তা সাথে সাথে বোঝা যায়।

শিক্ষকের নোট: ছাত্রদের বলুন নিজের প্রজেক্টে অন্তত এই সাতটির সমতুল্য টেস্ট লিখতে। এগুলো লেখা হয়ে গেলে পারমিশন নিয়ে আর ভয় থাকে না — কারণ প্রতিবার কোড বদলালে টেস্ট বলে দেবে ফুটো হয়েছে কি না।

ধাপ ১০

পাঁচটি সাধারণ ভুল — প্রতিটির পরিণাম

এই পাঁচটি প্রায় সব ছাত্র-প্রজেক্টে পাওয়া যায়

ভুল	কী হয়	ঠিক পথ
শুধু মেনুতে লুকানো	ঠিকানা টাইপ করলেই ঢুকে যায়	view-এ `@workspace_view(...)`
টেমপ্লেটে `if role` দিয়ে লুকানো	HTML-এ শর্ত, সার্ভারে কিছু নেই	শর্ত সার্ভারে, টেমপ্লেটে শুধু সাজসজ্জা
সব জায়গায় ৪০৩	প্রতিষ্ঠানের অস্তিত্ব ফাঁস হয়	অচেনা প্রতিষ্ঠান/ক্লাসে ৪০৪
শিক্ষকের ক্লাস যাচাই না করা	একই প্রতিষ্ঠানের দুই শিক্ষক একে অন্যের নম্বর বদলাতে পারে	তৃতীয় প্রশ্ন: বরাদ্দ যাচাই
`is_superuser` দিয়ে বাইপাস	সুপার-অ্যাডমিন প্রতিষ্ঠানের গোপন রেকর্ড দেখে ফেলে	প্ল্যাটফর্ম প্রশাসকের আলাদা `/ops/` জগৎ

সবগুলোর একটাই কারণ: **"লুকানো" আর "আটকানো" গুলিয়ে ফেলা**। প্রথমটা পর্দার কাজ, দ্বিতীয়টা সার্ভারের।

শিক্ষকের নোট: চতুর্থ ভুলটা ক্লাসে সবচেয়ে বেশি পাওয়া যায় — কারণ ছাত্ররা "একই প্রতিষ্ঠান = সব এক" ভাবে। স্কুলের বাস্তবে শিক্ষক নিজের ক্লাসের বাইরের নম্বর দেখতে পাওয়াটাই বড় সমস্যা।

ক্লাসের কাজ

নিজের অনুমতি গ্রিড বানান — ২০ মিনিট

ফলাফল: একটি টেবিল, যার প্রতিটি ঘরে এক শব্দ

১. বস্তু লিখুন

প্রজেক্টের ৫–৭টি মূল বস্তু: ক্লাস, ছাত্র, অর্ডার, বিল...

২. ভূমিকা লিখুন

সর্বোচ্চ ৫টি — বেশি হলে ভূমিকা ভাগ হয়নি।

৩. ঘর ভরুন

শুধু চার শব্দ: **সব · নিজের · ধরন · —**

৪. দুর্বল ঘর খুঁজুন

যেখানে দুই বাক্য লিখতে হচ্ছে, সেখানে নিয়মটা এখনো স্পষ্ট নয়।

৫. লিখে রাখুন

গ্রিডটা `docs/roles.md` -এ — ক্লাস ১৭-এ এটাই কোড হবে।

যাচাই: আপনার গ্রিডে কি "নিজের" ঘর আছে? না থাকলে তৃতীয় প্রশ্ন (বরাদ্দ যাচাই) নেই — অর্থাৎ আপনার সিস্টেমে সবাই সবাইকে দেখতে পায়।

শিক্ষকের নোট: গ্রিড শেষে ৩ জনকে বোর্ডে ডাকুন। যে গ্রিডে সব ঘরে "সব" লেখা, সেটাই সবচেয়ে বড় শিক্ষার উদাহরণ — ওটা দিয়ে দেখান কী হারাচ্ছে।

আলোচনা

চারটি প্রশ্ন — অনুমতি নিয়ে

উত্তর দিয়ে ক্লাস শেষ করুন

১. আপনার সিস্টেমে **একই প্রতিষ্ঠানের দুই শিক্ষক** একে অন্যের ক্লাসের নম্বর বদলাতে পারবে? কেন?

২. আপনার কোথায় **৪০৪** আর কোথায় **৪০৩** হবে — দুটো উদাহরণ দিন।

৩. আপনার মেনুতে **একই আইটেম** দুই ভূমিকার জন্য ভিন্ন অর্থ বোঝায়? সেটা কি আলাদা করা দরকার?

৪. আপনার সিস্টেমে **সুপার-অ্যাডমিন** আছে? সে কি প্রতিষ্ঠানের ব্যক্তিগত রেকর্ড দেখতে পারে — পারলে কেন?

শিক্ষকের নোট: ৪ নম্বরের উত্তর দিয়ে বোঝা যায় ছাত্র "শক্তি" আর "দায়িত্ব" আলাদা করতে শিখেছে কি না। "আমিই সব, তাই সব পারব" — এটা নকশা নয়, অভ্যাস।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
প্রতিটি অনুরোধে তিনটি প্রশ্ন — টেন্যান্ট · ভূমিকা · বরাদ্দ	`permissions.py`-এর নিজের ডকস্ট্রিং, হুবহু তিনটি প্রশ্ন
মেনু লুকায়, সার্ভার আটকায় — দুটো আলাদা দায়িত্ব	`navigation.py`: "this file only decides what is shown"
এক তালিকা, তিন ভোক্তা + নিরাপদ ব্যর্থতা	সাইডবার · ড্যাশবোর্ড · টেস্ট — ফাইল ভুলে গেলে পাতা অদৃশ্য, অনিরাপদ নয়
অচেনা লোকের জন্য ৪০৪, ভুল ভূমিকার জন্য ৪০৩	`Http404("Workspace not found.")` — অস্তিত্ব ফাঁস না করার সিদ্ধান্ত
ভূমিকা ৪টি, মেনু আইটেম ২২টি, প্রশাসকের ০	অ্যাডমিন ১৮ · কোঅর্ডিনেটর ১৩ · শিক্ষক ১২ · শিক্ষার্থী ৬ · প্রশাসক ০

শিক্ষকের নোট: শেষ লাইনে "প্রশাসকের ০" — এটা নিয়ে শেষ কথা বলুন: "যত কম দেখতে পারে, তত কম ভাঙতে পারে। সর্বশক্তিমান অ্যাডমিন হলো সবচেয়ে বড় ঝুঁকি।"

১০

পরের ক্লাস — "কত দিনে শেষ হবে"

শিডিউল · নির্ভরতার ক্রম · ১২ সপ্তাহের বাজেট ও বাফার

কত দিনে শেষ হবে

সিলেবাস: প্র্যাকটিক্যাল ৩ · টপিক ৩.৬ (শিডিউল ও বাজেট)

নির্ভরতার ক্রম · ১২ সপ্তাহের গ্যান্ট · বাফার

৫ স্লাইস · ১২ সপ্তাহ · ১,৭৫০ লাইন/সপ্তাহ

শুরু করুন প্রশ্ন দিয়ে

"১২ সপ্তাহে হয়ে যাবে" — এই বাক্যের সমস্যা কী?

হাত তুলুন: আপনার প্রজেক্টের শিডিউল কি কাগজে আছে, নাকি মাথায়?

শিডিউল মানে তারিখের তালিকা নয়।

শিডিউল মানে কোন কাজ কোন কাজের পরে।

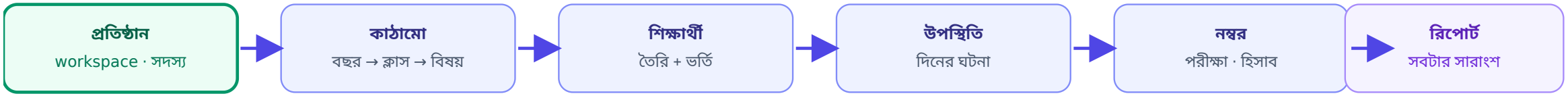
কারণ কাজগুলো **এলোমেলো নয়** — উপস্থিতি নেওয়ার পর্দা বানানো যায় না, যতক্ষণ ক্লাস আর শিক্ষার্থীর টেবিল না দাঁড়ায়। তাই আগে **নির্ভরতার ক্রম**, তারপর তারিখ। উল্টো করলে শিডিউল সুন্দর হয়, কিন্তু কাজ আটকে থাকে।

শিক্ষকের নোট: কেউ "আমার প্রজেক্ট ৪ সপ্তাহে হবে" বললে জিজ্ঞেস করুন: "প্রথম সপ্তাহের শেষে ঠিক কী চলবে?" উত্তর যদি "সব কিছু শুরু করেছি" হয়, তাহলে সপ্তাহ শেষে কিছুই চলবে না।

ধাপ ১

নির্ভরতার ক্রম — কোন কাজ কীসের পরে

তির মানে "আগে এটা, তারপর ওটা" — উল্টো করলে সময় নষ্ট, শুধু নয়, কাজ ভাঙে



উল্টো করার পরিণাম: নম্বরের পর্দা আগে বানালে শিক্ষার্থীর টেবিল পরে বদলাতে হয় → প্রতিটি পর্দা আবার বদলাতে হয়।
উপস্থিতি আগে বানালে ক্লাসের কাঠামো পরে বদলালে সব রেকর্ড নতুন করে মেলাতে হয়।

স্লাইড ১-এর ক্রমটাই আসল: **ভিত্তি → কাঠামো → মানুষ → দৈনন্দিন → ফল → সারাংশ**। তারিখ বসানোর আগে এই ক্রমটা ঠিক করুন।

শিক্ষকের নোট: ব্ল্যাকবোর্ডে এই ছয়টি বাক্স আঁকুন, তারপর ছাত্রদের নিজের প্রজেক্টের কাজগুলোতে বসাতে বলুন। বেশিরভাগ দেখবে তারা "পর্দা বানানো" দিয়ে শুরু করতে চেয়েছিল।

ধাপ ২

১২ সপ্তাহ — আসল গ্যান্ট চার্ট

প্রতিটি সপ্তাহের শেষে সিস্টেম চলে, টেস্ট সবুজ — এটাই শিডিউলের একমাত্র শর্ত



শিক্ষকের নোট: গ্যান্ট চার্টে "স্লাইস" শব্দটা জোর দিন — ফিচারের তালিকা নয়, চলমান সিস্টেমের টুকরো। প্রতিটি স্লাইস শেষে ডেমো দেওয়া যায়, এটাই শিডিউলের আসল নিরাপত্তা।

ধাপ ৩

প্রতিটি স্লাইসে ঠিক কী থাকে

এক স্লাইস = এক সম্পূর্ণ ব্যবহারযোগ্য সিস্টেম, অর্ধেক পর্দা নয়

স্লাইস	কী যোগ হয়	শেষে ব্যবহারকারী কী করতে পারে	প্রমাণ
১	লগইন · প্রতিষ্ঠান · সদস্যতা · ভূমিকা · ৪টি ড্যাশবোর্ড	লগইন করে নিজের প্রতিষ্ঠানে ঢুকে নিজের ভূমিকার ড্যাশবোর্ড দেখতে পারে	১৩ টেস্ট
২	বছর · টার্ম · বিভাগ · ক্লাস · বিষয় · শিক্ষক-বরাদ্দ · শিক্ষার্থী · ভর্তি	প্রতিষ্ঠানের পূর্ণ কাঠামো দাঁড়ায়, শিক্ষার্থী ক্লাসে যুক্ত হয়	৪৮ পর্দা × ৪ ভূমিকা
৩	উপস্থিতি · সিলেবাস · পাঠ-লগ · হোমওয়ার্ক	শিক্ষক প্রতিদিনের কাজ করতে পারে — হাজিরা থেকে হোমওয়ার্ক	শিক্ষকের পূর্ণ দিন
৪	পরীক্ষা · নম্বর · গ্রেড · হিসাব · প্রকাশ · মার্কশিট PDF	নম্বর থেকে ফলাফল পর্যন্ত পুরো পথ — প্রকাশের পরে শিক্ষার্থী দেখে	মার্কশিট PDF
৫	রিপোর্ট · সারাংশ · এক্সপোর্ট · অডিট লগ · পলিসি	প্রতিষ্ঠান চালানোর সিদ্ধান্ত নেওয়ার মতো তথ্য	১০৩ টেস্ট

লক্ষ্য করুন: স্লাইস ১-এ **শিক্ষার্থী নেই**, স্লাইস ২-এ **হাজিরা নেই** — তবু প্রতিটি স্লাইস শেষে সিস্টেম চলে। এটাই "চলমান টুকরো" হওয়ার শর্ত।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন — "তোমার স্লাইস ১-এ কী থাকবে?" উত্তর যদি "৬টি ফিচার" হয়, জিজ্ঞেস করুন "শেষে ব্যবহারকারী ঠিক কী করতে পারবে?" — উত্তর না এলে স্লাইসটা ভাঙতে হবে।

ধাপ ৪

গতির হিসাব — আসল সংখ্যা দিয়ে

৯,৩২২ লাইন কোড ÷ ১২ সপ্তাহ — কিন্তু এই ভাগটা একা মিথ্যা বলে

📄 **৯,৩২২ লাইন**
Python (মাইগ্রেশন বাদ)

🖼️ **৩,২৬৫ লাইন**
HTML (৭৫ টেমপ্লেট)

🧪 **১০৩ টেস্ট**
১১টি অ্যাপে

🗓️ **১২ সপ্তাহ**
× ৫ দিন × ৪ ঘণ্টা = ২৪০ ঘণ্টা

হিসাব	ফল	কেন বিভ্রান্তিকর
মোট লাইন ÷ মোট ঘণ্টা	**প্রতি ঘণ্টায় ~৫২ লাইন**	শেষ সপ্তাহে ৪০০ লাইন, প্রথম সপ্তাহে ২০ — গড় কিছুই বলে না
স্লাইস ÷ সপ্তাহ	**প্রতি ২.৪ সপ্তাহে ১ স্লাইস**	প্রথম স্লাইস ছোট, দ্বিতীয় স্লাইস সবচেয়ে বড়
টেস্ট ÷ সপ্তাহ	**সপ্তাহে ~৯টি**	বাস্তবে সপ্তাহে ০, তারপর একদিনে ৩০

যেটা কাজে দেয়: "গত সপ্তাহে আমি কতগুলো স্লাইস-শর্ত শেষ করেছি" — শেষ **৩ সপ্তাহের** গড় দিয়ে বাকি সময় হিসাব করুন।

শিক্ষকের নোট: ছাত্রদের বলুন প্রতি সপ্তাহে এক লাইনে লিখতে: "এই সপ্তাহে কী কাজ করল, কী আটকে আছে"। তিন সপ্তাহ পরে ওই তিন লাইনই তাদের সবচেয়ে নির্ভুল শিডিউল।

ধাপ ৫

বাফার — কেন ২৫%, আর কোথায় রাখবেন

বাফার "অলসতার জায়গা" নয় — অজানা জিনিসের দাম

বাফার কোথায় রাখা ভুল

✓ শেষে ৩ সপ্তাহ খালি রাখা — "পরে ঠিক করে নেব"

✗ তখন প্রথম ৯ সপ্তাহেই সব দেরি জমে যায়

✗ আর শেষে বাফারটা কাজে লাগে **দেরি ঢাকতে**, নতুন কিছু শিখতে নয়

সঠিক নিয়ম

প্রতিটি স্লাইসের ভেতরেই বাফার — অর্থাৎ স্লাইস ১-এর কাজ ৪ দিনে ধরুন, ৫ দিনের ঘরে; স্লাইস ২-এর কাজ ৮ দিনে ধরুন, ১০ দিনের ঘরে।

দেরি হলে স্লাইসেই ধরা পড়ে, শেষে নয়।

২৫% কোথা থেকে

IMS-এর আসল টাইমলাইনে **৯,৩২২ লাইন কোড ২ কর্মদিবসে** লেখা হয়েছে — কিন্তু এর মধ্যে **৪টি বাগ সংশোধন** আছে (CSRF, বিলিং, workspace, রেনেম)।

অর্থাৎ "লেখার সময়" আর "ঠিক করার সময়" — মিলিয়ে ২৫–৪০%।

ভয়ংকর হিসাব: ৫৭ ফিচার ÷ ১২ সপ্তাহ = সপ্তাহে ৫টি। কাগজে ঠিক, বাস্তবে অসম্ভব — কারণ ফিচারগুলো সমান নয়।

শিক্ষকের নোট: "বাফার মানে অলসতা নয়" — এই বাক্যটা ক্লাসে দুবার বলুন। ছাত্ররা বাফার রাখে না কারণ মনে করে অধ্যাপক কম পরিশ্রম দেখবেন; আসলে বাফারহীন শিডিউলই সবচেয়ে বেশি মিথ্যা।

ধাপ ৬

সমালোচনামূলক পথ — কোন কাজ দেরি হলে সব দেরি

Critical path: সব কাজ সমান নয়, কয়েকটা পুরো প্রজেক্ট টেনে নেয়



সমান্তরালে চলতে পারে (দেরি হলেও ক্ষতি নেই):

- রিপোর্ট এক্সপোর্ট
- অডিট লগ
- ব্র্যান্ডিং / হোয়াইট-লেবেল
- ব্যাকআপ কনসোল

সমালোচনামূলক পথের মোট = ২২ দিন (কাজের দিন) → প্রায় ৫ সপ্তাহ। বাকি কাজ এই পথের ফাঁকে চলে।

নিয়ম: সমালোচনামূলক পথে একজন মানুষ কাজ করবে — দুই জন দিলে দুই জন একে অন্যের কাজে বাধা দেয়।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন — "তোমার সবচেয়ে বেশি দেরি হওয়া কাজ কোনটা, আর তার পরে কোন কাজ আটকে আছে?" উত্তরই তাদের সমালোচনামূলক পথ।

ধাপ ৭

শিডিউলের চারটি ফাঁদ

প্রতিটি ফাঁদের একটা লক্ষণ আছে — শুরুতেই ধরা যায়

ফাঁদ	লক্ষণ	প্রতিকার
১. ছাত্র-ঘণ্টা ধরে নেওয়া	"দিনে ৮ ঘণ্টা কাজ করব" — ৩ দিনও হয়নি	দিনে **৩–৪ ঘণ্টা** ধরে হিসাব করুন; বাকিটা ক্লাস, পড়া, জীবন
২. সব কাজ সমান ধরা	"৫৭ ফিচার ÷ ১২ সপ্তাহ"	ধরন অনুযায়ী ভাগ: list আধা দিন · form ১ দিন · report ২ দিন · model ৩ দিন
৩. পর্দা দিয়ে শুরু	প্রথম সপ্তাহে সুন্দর ড্যাশবোর্ড, ডেটাবেস পরে	নির্ভরতার ক্রম মানুন: ভিত্তি → কাঠামো → মানুষ → ঘটনা → ফল
৪. শেষ সপ্তাহে ডিপ্লয়	১১তম সপ্তাহেও কোথাও চালু হয়নি	**৮ম সপ্তাহেই** ডিপ্লয় করুন — সস্তায়, কুৎসিতভাবে, কিন্তু চালু

চতুর্থ ফাঁদটা সবচেয়ে মারাত্মক: শেষ সপ্তাহে ডিপ্লয় করতে গেলে প্রথমবারই দেখা যায় `DEBUG=False`-এ সাইট ভাঙে — ঠিক করার সময় তখন আর নেই।

শিক্ষকের নোট: প্রজেক্টে এই ফাঁদ এড়ানো হয়েছে — দিন ১-এই `run.sh` আর `sqlite` settings তৈরি। ছাত্রদের বলুন ৮ম সপ্তাহের ডিপ্লয়টা ক্যালেন্ডারে এখনই লিখে ফেলতে।

ধাপ ৮

বাজেট — টাকা আর সময় একসাথে

ক্লাস ৩-এর ফিজিবিলিটি এখানে শিডিউলে মিশে যায়

খাত	পরিমাণ	নোট
সময় (প্রধান খরচ)	**২৪০ ঘণ্টা**	১২ সপ্তাহ × ৫ দিন × ৪ ঘণ্টা
ডেমো সার্ভার	৳০ – ৳১,৮০০/মাস	ডেমোর জন্য ফ্রি টিয়ারও চলে; প্রোডাকশনে VPS
ডোমেইন	৳১,২০০/বছর	ডেমোতে সাবডোমেইনই যথেষ্ট
ইমেইল	৳০	প্রথমে কনসোল ব্যাকএন্ড (ডেমোতে দেখা যায়)
PDF / Excel	৳০	লাইব্রেরি ওপেন সোর্স
মোট নগদ খরচ (ডেমো)	**৳০ – ৳৩,০০০**	আসল বিনিয়োগ সময়, টাকা নয়

এই কারণেই শিক্ষার্থী-প্রজেক্টে বাজেট আলোচনা হয় **সময়** দিয়ে, টাকা দিয়ে নয়। সময়ই একমাত্র খরচ যা ফেরত পাওয়া যায় না।

শিক্ষকের নোট: কেউ "সার্ভারে টাকা লাগবে" বললে বোঝান: ৳১,৮০০ মাসিক মানে সেমিস্টার জুড়ে ৳৭,২০০ — একটি ল্যাপটপের ১%। তাই অর্থ সমস্যা নয়, সময়ই আসল সীমা।

ধাপ ৯ · প্রমাণ

শিডিউলের রিপোর্টিং — সপ্তাহে এক টেবিল

কেউ জিজ্ঞেস করলে ৩০ সেকেন্ডে উত্তর দেওয়ার মতো ফরম্যাট

সপ্তাহ	লক্ষ্য ছিল	যা শেষ হলো	প্রমাণ
৭	উপস্থিতি + সিলেবাস	উপস্থিতি শেষ, সিলেবাস অর্ধেক	`30_attendance_take.png` · ৫ টেস্ট
৮	সিলেবাস + হোমওয়ার্ক	দুটোই শেষ	`36_syllabus_detail.png` · ৪ টেস্ট
৯	পরীক্ষা সেটআপ	লক্ষ্যের বেশি — নম্বর এন্ট্রিও	`43_marks_entry.png` · ১১ টেস্ট
১০	ফলাফল + PDF	বাকি — PDF রেন্ডারার জটিল	`44_marksheet.png` (আংশিক)

দশম সপ্তাহের সারিটাই সবচেয়ে দরকারি — এখানে সৎভাবে **"বাকি"** লেখা আছে। যে রিপোর্টে কখনো "বাকি" থাকে না, সেটা রিপোর্ট নয়, বিজ্ঞাপন।

শিক্ষকের নোট: ছাত্রদের প্রতিটি সারিতে স্ক্রিনশটের নাম বা টেস্টের নাম লিখতে বাধ্য করুন। প্রমাণ কলাম না থাকলে শিডিউল রিপোর্টিং আত্মপ্রশংসায় পরিণত হয়।

ধাপ ১০

দেরি হলে কী করবেন — পাঁচটি ক্রম

দেরি হবে, সেটা ধরে নিন; প্রশ্ন হলো কোনটা আগে ছাড়বেন

১. আগে বাদ দিন

v1.1-এর কাজ

আমন্ত্রণ · ব্যাকআপ · এক্সপোর্ট

২. সরল করুন

PDF বাদ, প্রিন্ট-ভিউ দিন

এক্সেল বাদ, CSV দিন

৩. কম লোক দেখান

কোঅর্ডিনেটর ভূমিকা

v1.1-এ সরান

৪. কখনোই বাদ দেবেন না

লগইন · টেন্যান্ট · ভূমিকা

অডিট · প্রকাশের নিয়ম

৫. দিন শেষে

ডেমো ডেটা তৈরি করুন

খালি সিস্টেম দেখাবেন না

এই ক্রমটাই সঠিক কারণ ১–৩ বাদ দিলেও সিস্টেম নিরাপদ থাকে, আর ৪ বাদ দিলে সিস্টেম ঝুঁকিপূর্ণ হয়ে যায়।

সবচেয়ে খারাপ সিদ্ধান্ত: সময় বাঁচাতে টেন্যান্ট যাচাই বা পারমিশন বাদ দেওয়া — সেটা সময় বাঁচায় না, বিশ্বাস হারায়।

দ্বিতীয় খারাপ: খালি ডেটাবেস নিয়ে ডেমো দেওয়া — "কিছু নেই" আর "কিছু কাজ করে না" দেখতে একই লাগে।

শিক্ষকের নোট: এই পাঁচটি ক্রম বোর্ডে লিখে রাখুন, সেমিস্টারের শেষে কেউ "আরও দুই সপ্তাহ লাগবে" বললে ওই লেখাটাই আপনাকে সাহায্য করবে — কোনটা ছাড়তে হবে তা নিয়ে বিতর্ক কমবে।

ক্লাসের কাজ

নিজের ১২ সপ্তাহের শিডিউল — ২৫ মিনিট

ফলাফল: একটি টেবিল, যার প্রতিটি সপ্তাহে একটা প্রমাণ

১. স্লাইস ভাগ করুন

৪–৫টি স্লাইস, প্রতিটির শেষে সিস্টেম চলে।

২. ক্রম ঠিক করুন

ভিত্তি → কাঠামো → মানুষ → ঘটনা → ফল।

৩. সপ্তাহ বসান

প্রতি স্লাইসে ২৫% বাফার — বেশি নয়, কমও নয়।

৪. প্রমাণ লিখুন

প্রতি সপ্তাহে একটা টেস্টের নাম বা স্ক্রিনশটের নাম।

৫. ৮ম সপ্তাহে ডিপ্লয়

ক্যালেন্ডারে তারিখ লিখুন — এখনই।

যাচাই: আপনার শিডিউলে কি এমন কোনো সপ্তাহ আছে যার শেষে **কিছুই চলে না**? থাকলে ওই সপ্তাহটাই ভুল করে সাজানো হয়েছে — স্লাইস ভাঙুন।

শিক্ষকের নোট: শেষে ২ জনের শিডিউল বোর্ডে দেখুন — যার প্রতি সপ্তাহে প্রমাণ লেখা আছে, তার শিডিউল বাস্তবে টিকবে; যার শুধু কাজের নাম আছে, সে তৃতীয় সপ্তাহেই পিছিয়ে পড়বে।

আলোচনা

চারটি প্রশ্ন — শিডিউল নিয়ে

পর্ব ২ শেষ হচ্ছে — উত্তরগুলো পর্ব ৩-এ কাজে লাগবে

১. আপনার **স্লাইস ১**-এ কী থাকবে, আর শেষে ব্যবহারকারী ঠিক কী করতে পারবে?

২. আপনার **সমালোচনামূলক পথ** কোন কাজগুলো দিয়ে যায়? দেরি হলে কী আটকাবে?

৩. আপনার শিডিউলে **বাফার** কত? সেটা প্রতিটি স্লাইসের ভেতরে, না শুধু শেষে?

৪. দেরি হলে আপনি **প্রথমে** কোনটা ছাড়বেন? যদি উত্তর "পারমিশন যাচাই" হয় — শিডিউলটা নতুন করে ভাবুন।

শিক্ষকের নোট: ৪ নম্বর উত্তরই পর্ব ২-এর শেষ পরীক্ষা — সেটা ঠিক থাকলে ছাত্র বুঝেছে "কী ছাড়া যায়, কী ছাড়া যায় না"। পরের পর্বে এই বোধটাই ডিজাইনের ভিত্তি হবে।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
শিডিউল আগে নির্ভরতা, পরে তারিখ	ভিত্তি → কাঠামো → মানুষ → ঘটনা → ফল → সারাংশ
স্লাইস মানে চলমান সিস্টেম, কাজের তালিকা নয়	৫ স্লাইস · প্রতিটির শেষে ডেমো দেওয়া যায়
গড় হিসাব মিথ্যা বলে — শেষ ৩ সপ্তাহের গতি ধরুন	৯,৩২২ লাইন ÷ ২৪০ ঘণ্টা = ৫২ লাইন/ঘণ্টা — বাস্তবে অসম
বাফার প্রতিটি স্লাইসের ভেতরে, শেষে নয়	২ দিনের কাজ ৫ দিনের ঘরে — দেরি স্লাইসেই ধরা পড়ে
দেরি হলে ছাড়ার ক্রম আগেই ঠিক করা	v1.1 → সরলীকরণ → কম লোক · কখনোই নয়: টেন্যান্ট/ভূমিকা

শিক্ষকের নোট: এই পাঁচটি লাইনে পর্ব ২ শেষ হচ্ছে। বলুন: "পর্ব ৩-এ আমরা এই শিডিউল ধরে ডিজাইন শুরু করব — ERD, wireframe, আর সিস্টেম ডিজাইন।"

পর্ব ৩

ক্লাস ১১ — "ডেটা কীভাবে ভাবে"

ডিজাইন পর্ব: সিস্টেম ডিজাইন · রিকোয়েস্ট ফ্লো · wireframe · DFD · ERD

পর্ব ৩ · ডিজাইন · ক্লাস ১১ / ৩৭

ডেটা কীভাবে ভাবে

সিলেবাস: প্র্যাকটিক্যাল ৪ · টপিক ৪.১ (সিস্টেম ডিজাইন)
রিকোয়েস্ট ফ্লো · ১০ middleware · সেশন ও কুকি

৭ Django · ৩ নিজের · ৬টি স্তর

শুরু করুন প্রশ্ন দিয়ে

একটি পর্দা দেখা যাওয়ার আগে কতটা কাজ হয়?

হাত তুলুন: অনুমান করুন — ২টি ধাপ? ৫টি? ১০টি?

শিক্ষার্থীর ড্যাশবোর্ড খুলতে ১০টি middleware পার হয়, তারপর কুয়েরি, টেমপ্লেট, আর ৭৫টি HTML ফাইলের একটি।

এই যাত্রাটা না জানলে বাগ খুঁজতে গিয়ে ভুল জায়গায় খুঁজবেন। "লগইন হয়ে আবার লগআউট হয়ে যাচ্ছে" — সমস্যাটা view-এ নয়, **কুকিতে**। আজকের ক্লাসে এই যাত্রার প্রতিটি স্টেশন।

শিক্ষকের নোট: সংখ্যাটা জিজ্ঞেস করে হাত তুলিয়ে নিন — কেউ ১০ বলবে না। তারপর বলুন: "আপনি যেখানে বাগ খুঁজছিলেন, সেটা যাত্রার শেষ স্টেশন; সমস্যা ছিল প্রথম স্টেশনে।"

ধাপ ১

সিস্টেম ডিজাইন মানে ছয়টি প্রশ্নের উত্তর

প্রতিটি প্রশ্নের উত্তর একটা ডায়াগ্রামে আঁকা যায় — আর সেটাই ডিজাইন ডকুমেন্ট

প্রশ্ন	উত্তর দেয় কোন আর্টিফ্যাক্ট	IMS-এ কোথায়
কে ব্যবহার করবে?	ভূমিকা ম্যাট্রিক্স	পর্ব ২ ক্লাস ৯
ডেটা কোথায় থাকবে?	ERD ও স্কিমা	ক্লাস ১৪–১৫
ডেটা কীভাবে চলাচল করবে?	DFD Level 0 ও 1	ক্লাস ১৩
একটা অনুরোধ কীভাবে প্রক্রিয়া হবে?	**রিকোয়েস্ট ফ্লো**	**এই ক্লাস**
ব্যবহারকারী কী দেখবে?	wireframe ও লেআউট	ক্লাস ১২
ব্যর্থ হলে কী হবে?	ভুল-অবস্থা ও গার্ড	এই ক্লাস + ক্লাস ১৭

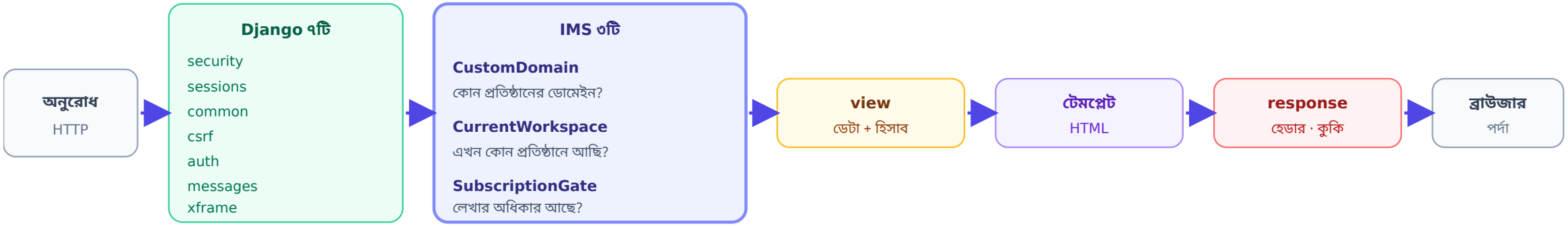
ছয়টির মধ্যে পাঁচটি ডায়াগ্রাম, একটি টেবিল। **কোনোটা লিখিত অনুচ্ছেদ নয়** — কারণ ডিজাইন দেখতে হয়, পড়তে হয় না।

শিক্ষকের নোট: ছাত্রদের বলুন এই ছয়টি প্রশ্নের উত্তর দিয়ে নিজের প্রজেক্টের ডিজাইন ডকুমেন্ট শুরু করতে — ৬টি পাতা, তার বেশি নয়।

ধাপ ২

Middleware কী — কেন এতগুলো

৭টি Django নিজের, ৩টি আমাদের — প্রতিটি একটা নির্দিষ্ট প্রশ্নের উত্তর দেয়



প্রতিটি middleware একই কাজ: অনুরোধ পেল →

হয় পাস করল, নয় থামাল/ঘুরাল। একটাও ডেটা বদলায় না, শুধু সিদ্ধান্ত নেয়।

শিক্ষকের নোট: বোর্ডে একটাই বাক্য লিখুন: "middleware = দরজার প্রহরী"। প্রতিটি প্রহরী একটাই প্রশ্ন করে, আর উত্তর না মিললে ভেতরে ঢুকতে দেয় না বা অন্য দরজায় পাঠায়।

ধাপ ৩

তিনটি প্রহরীর কাজ — এবং ক্রমের গুরুত্ব

`config/settings/base.py` — ক্রম বদলালে নিরাপত্তার ফাঁক

🌐

১. CustomDomain

হোস্টনাম দেখে বোঝে কোন প্রতিষ্ঠানের নিজের ডোমেইন। ভেরিফায়েড ডোমেইন হলে `ALLOWED_HOSTS`-এ যোগ করে।

না থাকলে: হোয়াইট-লেবেল অসম্ভব

🏢

২. CurrentWorkspace

সেশন থেকে বেছে নেয় "এখন কোন প্রতিষ্ঠানে আছি" — ভূমিকা ও ধরন অনুযায়ী সেরাটা। `request.workspace` বসায়।

না থাকলে: প্রতিটি view-তে ভুলে যাওয়ার ঝুঁকি

🔒

৩. SubscriptionGate

মেয়াদ শেষ হলে **শুধু লেখা** আটকায় — পড়া, লগআউট, প্ল্যাটফর্ম কনসোল খোলা থাকে।

না থাকলে: মেয়াদহীন প্রতিষ্ঠান অনির্দিষ্টকাল লেখে

ক্রম ভুলের পরিণাম: `SubscriptionGate` আগে বসালে সে জানে না কোন প্রতিষ্ঠান — কারণ `CurrentWorkspace` তখনো চলেনি। ফলে গেট চুপচাপ পাস করে দেয় → **নিরাপত্তার ফাঁক**, কোনো এরর নেই।

শিক্ষকের নোট: "চুপচাপ ফাঁক" ধারণাটা জোর দিন — সবচেয়ে বিপজ্জনক বাগ সেটাই, যেটা কোনো এরর দেয় না। ছাত্রদের বলুন প্রতি middleware-এ লিখতে: "আমি কার উপর নির্ভর করি?"

ধাপ ৪

সেশন ও কুকি — "লগইন হয়ে আবার লগআউট" বাগের আসল কারণ

প্রজেক্টের একমাত্র কমিট যেটা শুধু ডেমো-পরিবেশের জন্য

যা ঘটেছিল

ডেমো প্রিভিউ একটা **স্যান্ডবক্স iframe**-এ চলে, যেখানে কুকি রাখা যায় না।

ফল: লগইন সফল → সেশন কুকি হারিয়ে যায় → পরের পাতায় আবার লগইন পাতা।

ব্যবহারকারীর কাছে মনে হয় **পাসওয়ার্ড ভুল** — অথচ পাসওয়ার্ড ঠিক।

যা করা হলো (`e1ead8a`)

✓ কুকি-গার্ড ব্যানার — "এই প্রিভিউতে কুকি রাখা যায় না"

✓ ভদ্র CSRF এরর পাতা (৫০০ নয়)

✓ CSRF ব্যর্থ হলে কী করবেন তার নির্দেশ

এখান থেকে শেখার তিনটি কথা

১. সেশন = সার্ভারে ডেটা, কুকি = শুধু চাবি। কুকি না থাকলে সেশন খুঁজে পাওয়া যায় না।

২. পরিবেশ বদলালে **আচরণও** বদলায় — কোড একই থাকলেও।

৩. ব্যর্থতা **বোঝানো** যায়: ভদ্র বার্তা দেওয়া যায়, আর সেটাই ব্যবহারকারীর অভিজ্ঞতা।

সাধারণ নিয়ম: "কাজ করছে না" শুনলে প্রথম প্রশ্ন — **কোন পরিবেশে?** লোকাল, ডেমো, নাকি প্রোডাকশন।

শিক্ষকের নোট: নিজের অভিজ্ঞতা থেকে বলুন — এই বাগটা হয়েছিল বলে কুকি-গার্ড ব্যানার যোগ করতে হয়েছে। ছাত্রদের জিজ্ঞেস করুন: "তোমার প্রজেক্টে কোনো পরিবেশভেদে আচরণ বদলায়?"

ধাপ ৫

কোন প্রতিষ্ঠানে ঢুকব — সিদ্ধান্তের নিয়ম

`resolve_workspace()` — এক 함수, তিনটি সিদ্ধান্ত

```
def resolve_workspace(request):
    # ১. সেশনে সংরক্ষিত পছন্দ আছে?
    ws_id = request.session.get(
        "active_workspace_id")
    if ws_id and valid(ws_id):
        return membership        # পছন্দ মানুন

    # ২. নেই → সবচেয়ে উঁচু ভূমিকা
    membership = qs.order_by(
        ROLE_PRIORITY,       # ORG_ADMIN আগে
        KIND_PRIORITY,       # প্রতিষ্ঠান আগে
        "workspace__name")  # তারপর নাম
    request.session["active_workspace_id"]
        = membership.workspace_id
    return membership
```

তিনটি সিদ্ধান্ত, তিনটি কারণ

১. আগের পছন্দ থাকলে সেটাই — ব্যবহারকারী যেখানে ছিলেন সেখানেই ফিরবেন।

২. না থাকলে **উঁচু ভূমিকা** — যিনি স্কুল চালান, তিনি স্কুলেই ঢুকবেন, ব্যক্তিগত ঘরে নয়।

৩. তারপর **প্রতিষ্ঠান**, তারপর নাম — যাতে ফলাফল নির্দিষ্ট ও পুনরাবৃত্তিযোগ্য হয়।

> কোডের মন্তব্য হুবহু: *"the personal one is a side room, not the front door."*

শিক্ষকের নোট: তৃতীয় নিয়মটা ("তারপর নাম") বিশদ মনে হলেও জরুরি — এটা না থাকলে একই ইউজারের দুই প্রতিষ্ঠানের ডিফল্ট প্রতিবার বদলাতে পারে, আর ব্যবহারকারী ভাববেন "সিস্টেম এলোমেলো"।

ধাপ ৬

বিলিং গেট — মেয়াদ শেষ মানে "সব বন্ধ" নয়

পড়া খোলা, লেখা বন্ধ — আর বার্তাটা কে পাবে তা ভূমিকা ঠিক করে

পরিস্থিতি	গেট কী করে	কারণ
GET / পড়া	**পাস**	তথ্য দেখা বন্ধ করা মানে প্রতিষ্ঠানের ডেটা হারানো নয়, শুধু শাস্তি
POST ও লেখা	**বন্ধ**	মেয়াদ শেষে নতুন ডেটা জমা নেওয়া উচিত নয়
লগআউট · লগইন · স্ট্যাটিক · `/ops/`	**সবসময় পাস**	তালিকা: `ALLOW_PREFIXES` — নাহলে ব্যবহারকারী আটকে যান
অ্যাডমিন নিজে	**নবায়ন পাতায়**	শুধু সে-ই ঠিক করতে পারে → `?blocked=1`
বাকি সবাই	**ড্যাশবোর্ডে ফেরত**	বার্তা: "করুননি কিছু হারায়নি, আপনার অ্যাডমিনকে বলুন"

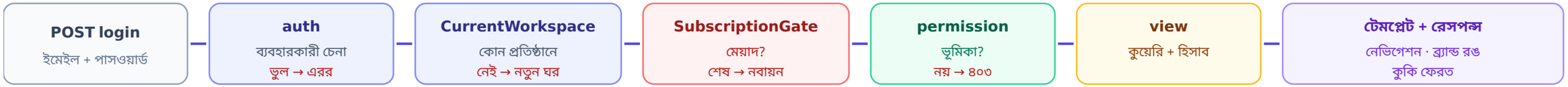
নীতি: মেয়াদ শেষ = **read-only**, delete নয়। ডেটা মুছে ফেলা মানে গ্রাহক ফিরে এলে কিছু দেওয়া নেই — আর সে ফিরবেই না।

শিক্ষকের নোট: প্রশ্ন করুন — "কেন সুপার-অ্যাডমিনের জন্য বাইপাস নেই?" কোডের নিজের উত্তর: যে প্ল্যাটফর্ম-অ্যাডমিন একই সাথে স্কুলের অ্যাডমিন, মেয়াদ শেষে সে-ও যেন লিখতে না পারে।

ধাপ ৭

পুরো যাত্রা — লগইন থেকে ড্যাশবোর্ড

প্রতিটি স্টেশনে কে সিদ্ধান্ত নেয়, আর ব্যর্থ হলে কী হয়



ড্যাশবোর্ড টেমপ্লেটে তিনটি জিনিস মেশে: নেভিগেশন (ভূমিকা অনুযায়ী) · ব্র্যান্ড (প্রতিষ্ঠানের রঙ) · ডেটা (view থেকে)।

রঙটা কোথা থেকে? base.html ফাইলে --ims-brand = প্রতিষ্ঠানের ব্র্যান্ড রঙ — তাই একই টেমপ্লেট, ভিন্ন প্রতিষ্ঠানে ভিন্ন রঙ।

ব্যর্থতার চারটি সম্ভাব্য জায়গা: auth · workspace · subscription · permission — চারটিই view-এর আগে।

তাই "পাতা খুলছে না" শুনলে প্রথমে view-এ নয়, এই চারটি প্রহরীর দিকে তাকান।

শিক্ষকের নোট: এই চারটি ব্যর্থতার জায়গা বোর্ডে আলাদা করে লিখুন। ছাত্ররা বাগ খুঁজতে সবসময় view-এ যায় — অথচ বাস্তবে বেশিরভাগ সমস্যা প্রথম চার স্টেশনে।

ধাপ ৮

রেসপন্সের ভেতরে কী থাকে

স্ট্যাটাস কোড, হেডার, কুকি — তিনটিই ব্যবহারকারীর আচরণ বদলায়

অংশ	IMS-এ উদাহরণ	ভুল হলে কী হয়
স্ট্যাটাস	৪০৪ অচেনা প্রতিষ্ঠান · ৪০৩ ভুল ভূমিকা · ৩০২ লগইনে পাঠানো	সব জায়গায় ৪০৩ দিলে প্রতিষ্ঠানের অস্তিত্ব ফাঁস
সিকিউরিটি হেডার	`X-Frame-Options: DENY` · `nosniff` · `Referrer-Policy`	কেউ আপনার সাইট অন্য সাইটের iframe-এ খুলে প্রতারণা করতে পারে
সেশন কুকি	প্রোডাকশনে `Secure` · `HttpOnly` · `SameSite=Lax`	স্ক্রিপ্ট দিয়ে কুকি পড়া যায় → অ্যাকাউন্ট দখল
ভ্যারি: Host	`patch_vary_headers(response, ["Host"])`	ক্যাশ ভিন্ন ডোমেইনের কনটেন্ট দেখাতে পারে
ব্র্যান্ড রঙ	`--ims-brand` = প্রতিষ্ঠানের রঙ	সব প্রতিষ্ঠানকে একই রকম দেখায় → হোয়াইট-লেবেল অর্থহীন

সবচেয়ে কম আলোচিত: **Vary: Host**। ক্যাশ সার্ভার হোস্ট আলাদা না করলে এক স্কুলের ডোমেইনে অন্য স্কুলের লোগো দেখাতে পারে — কোড ঠিক, ক্যাশ ভুল।

শিক্ষকের নোট: `deploy_check.sh`-এর আউটপুটটা এখানে মনে করিয়ে দিন — চারটি হেডার ওখানেই যাচাই হয়। বলুন: "হেডার লেখা মানে নয়, প্রমাণ করা লাগে।"

ধাপ ৯

স্কোপিং — টেন্যান্টের সীমানা কীভাবে রক্ষা হয়

প্রতিটি কুয়েরিতে প্রতিষ্ঠানের ফিল্টার — ভুলে যাওয়ার সুযোগ নেই

ঝুঁকিপূর্ণ পদ্ধতি

```
StudentProfile.objects.all()
# একটি view-তে filter ভুলে গেলেই
# অন্য প্রতিষ্ঠানের শিক্ষার্থী দেখা যাবে
```

ফুটো হয় না — **ফুটো হতে পারে**। এটাই ভয়ংকর।

নিরাপদ পদ্ধতি — `scoping.py`

```
students_in(request.workspace)
# ফিল্টার ভেতরেই — বাইরে ভুলে যাওয়া
# অসম্ভব
```

ভুল করা **অসম্ভব** বানানো — ভালো আর্কিটেকচারের লক্ষণ।

সোনার নিয়ম: ব্যবহারকারীর ইনপুট থেকে প্রতিষ্ঠান নেবেন না; `request.workspace` থেকে নিন — সেটা middleware যাচাই করে বসিয়েছে।

শিক্ষকের নোট: এই স্লাইডটা পর্ব ৪-এর ভিত্তি। ছাত্রদের বলুন আজই ঠিক করতে: তাদের প্রজেক্টে "এক প্রতিষ্ঠানের ডেটা" আনার একটাই ফাংশন থাকবে, আর সব view সেটাই ব্যবহার করবে।

ধাপ ১০

ডিজাইন ডকুমেন্টে যা আঁকবেন — এক পাতায়

চারটি বাক্স আর ছয়টি তির, তার বেশি নয়

থাকবে

- ✓ ব্রাউজার → প্রহরী → view → টেমপ্লেট
- ✓ প্রতিটি প্রহরীর **এক প্রশ্ন**
- ✓ ব্যর্থ হলে কোথায় যায় (৪০৩/৪০৪/৩০২)
- ✓ প্রতিষ্ঠান কোথা থেকে আসে
- ✓ কে ডেটাবেসে যায়, কে যায় না

থাকবে না

- ✗ প্রতিটি ফাংশনের নাম
- ✗ প্রতিটি টেবিলের ফিল্ড (ওটা ERD)
- ✗ CSS ক্লাস ও রঙের কোড
- ✗ লাইব্রেরির ভেতরের গঠন
- ✗ স্ক্রিনশট (ওটা wireframe-এর ক্লাস)

যাচাই: **এই ডায়াগ্রাম দেখে কেউ বলতে পারবে "পাতা খুলছে না" সমস্যাটা কোন বাক্সে?** না পারলে ডায়াগ্রামে ব্যর্থতার পথ আঁকা হয়নি।

শিক্ষকের নোট: ছাত্রদের বলুন ডায়াগ্রামে ব্যর্থতার তিরগুলো **লাল** রঙে আঁকতে — সফল পথ কালো, ব্যর্থ পথ লাল। এই এক অভ্যাসেই তাদের ডায়াগ্রাম অন্যদের চেয়ে বেশি কাজে লাগবে।

ক্লাসের কাজ

নিজের রিকোয়েস্ট ফ্লো আঁকুন — ২০ মিনিট

চারটি বাক্স, ছয়টি তির, দুই রঙ

১. চারটি বাক্স

ব্রাউজার · প্রহরী (একাধিক হতে পারে) · view · টেমপ্লেট

২. প্রহরীর প্রশ্ন

প্রতিটি বাক্সের নিচে এক লাইনে: "এই প্রহরী কী জিজ্ঞেস করে?"

৩. লাল তির

প্রতিটি প্রহরীর ব্যর্থতার পথ — কোথায় যায়, কী কোড।

৪. প্রতিষ্ঠান কোথা থেকে

URL? সেশন? টোকেন? — একটাই বেছে নিন, আর লিখে রাখুন।

৫. ক্যাশ ও কুকি

কোন ডেটা সেশনে থাকবে, কোনটা প্রতিবার ডেটাবেস থেকে আসবে।

যাচাই: আপনার ডায়াগ্রামে কি **একটি লাল তিরও** নেই? তাহলে আপনি ব্যর্থতার কথা ভাবেননি — আর বাস্তবে ঠিক ওখানেই বাগ হবে।

শিক্ষকের নোট: ২০ মিনিট শেষে যার ডায়াগ্রামে সবচেয়ে বেশি লাল তির, তারটাই বোর্ডে দেখান — বলুন "এটাই সবচেয়ে পরিণত ডিজাইন"।

আলোচনা

চারটি প্রশ্ন — রিকোয়েস্ট ফ্লো নিয়ে

উত্তরগুলো ক্লাস ১৭-এ কোড হিসেবে ফিরবে

১. আপনার সিস্টেমে **কোন প্রহরী কোন ক্রমে** বসবে? প্রতিটির এক প্রশ্ন লিখুন।

২. প্রতিষ্ঠান কোথা থেকে আসবে — URL, সেশন, নাকি দুটোই? দ্বন্দ্ব হলে কোনটা জিতবে?

৩. আপনার কোথায় **"ভুল করা অসম্ভব"** বানানো যায় — কোন কুয়েরিগুলো এক ফাংশনে ঢোকাবেন?

৪. আপনার সিস্টেম যদি কোনো **স্যান্ডবক্স iframe**-এ চলে, কী ভাঙবে? (প্রজেক্টে এটাই ঘটেছিল)

শিক্ষকের নোট: ৪ নম্বর প্রশ্নটা মজার কিন্তু গভীর — উত্তর দিতে গিয়ে ছাত্ররা বুঝবে কুকি-নির্ভর সিস্টেম কোথায় কোথায় ভাঙে। এই বোঝাটাই পরে সেশন বনাম টোকেনের সিদ্ধান্তে কাজে লাগবে।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
এক পর্দার আগে ১০টি প্রহরী — ৭ Django + ৩ নিজের	`MIDDLEWARE`-এর হুবহু তালিকা, দশটি লাইন
প্রহরীর ক্রম নিরাপত্তার অংশ — ভুল ক্রমে চুপচাপ ফাঁক	CustomDomain → CurrentWorkspace → SubscriptionGate
পরিবেশ বদলালে আচরণ বদলায় — কোড একই, তবু	স্যান্ডবক্স iframe-এ কুকি থাকে না → `e1ead8a` কুকি-গার্ড
মেয়াদ শেষ = read-only, মুছে ফেলা নয়	`SAFE_METHODS` খোলা · `ALLOW_PREFIXES` তালিকা · বার্তা "আপনার কাজ হারায়নি"
ভুল করা অসম্ভব বানানো — ভুলে যাওয়ার উপর ভরসা নয়	`scoping.py` — ফিল্টার ফাংশনের ভেতরে, view-এ নয়

শিক্ষকের নোট: শেষ লাইনটা বারবার বলুন — ভালো সিস্টেম চেনা যায় এরকম প্রশ্নে: "এই ভুলটা করা কি সম্ভব?" উত্তর যদি "হ্যাঁ, যদি কেউ ভুলে যায়" হয়, ডিজাইনটা শেষ হয়নি।

১২

পরের ক্লাস — "স্ক্রিন আগে, কোড পরে"

wireframe ও UI: লেআউট · ভূমিকা অনুযায়ী পর্দা · খালি অবস্থা · ব্র্যান্ডিং

পর্ব ৩ · ডিজাইন · ক্লাস ১২ / ৩৭

স্ক্রিন আগে, কোড পরে

সিলেবাস: প্র্যাকটিক্যাল ৪ · টপিক ৪.১ (সিস্টেম ডিজাইন)

wireframe · ৪টি খোলস · ১৫টি টোকেন · খালি অবস্থা · white-label

৭৫ টেমপ্লেট · ৪ খোলস · ৬ partial

শুরু করুন প্রশ্ন দিয়ে

৭৫টি পর্দা লিখতে কতটা HTML লাগে?

হাত তুলুন: ৭৫টি ফাইল মানে ৭৫ বার sidebar লিখতে হয়েছে?

একবার sidebar লিখলে ৪৪টি পর্দায় দেখা যায়।
বাকিটা খোলস (shell) — পর্দা নয়, কাঠামো।

এই ক্লাসের বিষয় UI সাজানো, রং বাছাই নয়। কোন জিনিস **একবার** লিখে সব জায়গায় ব্যবহার করবেন, আর কোনটা পর্দাভেদে বদলাবে — এই সিদ্ধান্তটাই ডিজাইন।

শিক্ষকের নোট: ৭৫ সংখ্যাটা বোর্ডে লিখুন, নিচে ৪। বলুন: "পার্থক্যটা ৭১ নয় — পার্থক্যটা হলো ৪টি কাঠামো, যার ভেতরে ৭৫টি ঘর।"

ধাপ ১

চারটি খোলস — পুরো সিস্টেম এতেই ঢোকে

সবগুলো `base.html` থেকে আসে: শুধু `<head>` আর `<body>` ব্লক

base.html

head + body ব্লক · টোকেন

লগইন ৬ + এরর পাতা ৩

৯ পাতা

app_base.html

sidebar + topbar + content

আসল অ্যাপের সব পর্দা

৪৪ পাতা

platform/base.html

৬টি ট্যাব · নিজের রং

প্ল্যাটফর্ম অ্যাডমিনের ঘর

৮ পাতা

public/base_public.html

লগইনের আগের জগৎ

দাম · ট্রায়াল সাইনআপ

২ পাতা

৬টি partial টুকরো: sidebar · topbar · messages · billing_banner · field · cookie_guard — এগুলো খোলসের ভেতরে বসে।

> হোয়াইট-লেবেল: একই `app_base.html`, কিন্তু ব্র্যান্ড রং ভেরিয়েবল থেকে — তাই ভিন্ন প্রতিষ্ঠানে ভিন্ন রঙ, কোড এক।

শিক্ষকের নোট: চারটি বক্স বোর্ডে আঁকুন, তারপর বলুন: "নতুন পর্দা বানানোর সময় প্রশ্ন একটাই — এটা কোন খোলসে?" উত্তরটা না জানলে কোড দুবার লেখা হয়।

ধাপ ২

খোলসের ভেতরে কী কী ঢোকে

`templates/app_base.html` — মোট ১১ লাইন, আর ৪৪টি পর্দার কাঠামো তৈরি

```
{% extends "base.html" %}
{% block body %}
<div class="ims-shell">
  {% include "partials/sidebar.html" %}
  <div class="ims-main">
    {% include "partials/topbar.html" %}
    <div class="ims-content">
      {% include "partials/messages.html" %}
      {% include "partials/billing_banner.html" %}
      {% block content %}{% endblock %}
    </div>
  </div>
</div>
{% endblock %}
```

অংশ	দায়িত্ব
sidebar	ভূমিকা অনুযায়ী মেনু
topbar	শিরোনাম · workspace পিল · প্রোফাইল
messages	সেভ / এরর বার্তা
billing_banner	ট্রায়াল · read-only · সিট
`block content`	**একমাত্র** যা পর্দা বদলায়

নিয়ম: পর্দা লিখলে **শুধু** `content` **ব্লক** লেখা হয়। কেউ sidebar নিজে আঁকলে সেটা বাগের শুরু — দুবার রক্ষণাবেক্ষণ।

শিক্ষকের নোট: ছাত্রদের বলুন নিজের প্রজেক্টে একই ফাইল বানাতে — ১০ লাইনে। যে প্রজেক্টে প্রতিটি পেজে আলাদা মেনু লেখা, সেখানে নতুন মেনু আইটেম যোগ করতে ৩০টি ফাইল ছুঁতে হয়।

ধাপ ৩

ওয়্যারফ্রেম — বক্স আর মাপ, রং পরে

IMS-এর আসল মাপ: sidebar ২৬৪px · topbar ৫২px · content সর্বোচ্চ ১২৮০px · কার্ড রেডিয়াস ১৪px



মাপ আগে ঠিক করুন, রং পরে। রং আগে ঠিক করলে লেআউট বদলালে সব রং আবার বদলাতে হয়।

শিক্ষকের নোট: কাগজে ৫ মিনিটের কাজ দিন — শুধু বক্স আর সংখ্যা, কোনো রং নয়। যে দল মাপ লেখে না, তাদের ওয়্যারফ্রেম কোডিংয়ের সময় বদলে যাবে।

ধাপ ৪

১৫টি টোকেন — রং কোথাও হার্ডকোড নয়

`static/css/ims.css` — `:root`-এ ১৫টি ভেরিয়েবল, বাকি ১০০ লাইন শুধু সেগুলো ব্যবহার করে

শ্রেণি	টোকেন	কেন লাগে
রং (৯)	`primary` · `primary-dark` · `primary-soft`	ব্র্যান্ড বদলালে এখানেই
	`ink` · `ink-2` · `muted` · `line` · `bg` · `surface`	লেখা ও পৃষ্ঠতলের ধাপ
অবস্থা (৩)	`success` · `warning` · `danger`	একই অর্থে সব জায়গায় একই রং
আকৃতি (২)	`radius` · `radius-sm`	১৪px কার্ড, ১০px বাটন — মিশ্রণ নয়
গভীরতা (১)	`shadow`	দুই স্তরের ছায়া, একটাই ডেফিনিশন

টোকেন থাকলে: ১ লাইনে ব্র্যান্ড রং — `--ims-brand`।

টোকেন না থাকলে: ৭৫টি ফাইলে #4f46e5 খুঁজে বদলাতে হয় — একটা বাদ পড়বেই।

শিক্ষকের নোট: ছাত্রদের ১০টি রঙের নাম জিজ্ঞেস করুন — বেশিরভাগের উত্তর ভিন্ন হবে। বলুন: "আপনার নিজের প্রজেক্টে একই সবুজ কতবার লেখা আছে? grep করে দেখুন।"

ধাপ ৫

ভূমিকা অনুযায়ী পর্দা — এক ভিউ, তিন রূপ

`apps/reports/views.py` — `dashboard()` এক, কিন্তু টেমপ্লেট আলাদা

কে	কোন টেমপ্লেট	পর্দায় কী থাকে
STUDENT	`reports/dashboard_student`	উপস্থিতি % · প্রকাশিত পরীক্ষা · খোলা হোমওয়ার্ক · জমা দেওয়ার সংখ্যা
TEACHER / COORDINATOR / ORG_ADMIN	`reports/dashboard_staff`	৪টি KPI — শিক্ষার্থী · আজকের উপস্থিতি · অপ্রকাশিত পরীক্ষা · পর্যালোচনার অপেক্ষায়
তাঁদের মধ্যে ম্যানেজার	একই টেমপ্লেট	+ সদস্য সংখ্যা · অডিট লগ · উপস্থিতি নীতি
PLATFORM_ADMIN	`core/dashboard_platform`	মোট প্রতিষ্ঠান · ব্যবহারকারী · ইনভয়েস

আসল ফাঁদ: `core/` ফোল্ডারে ৪টি ড্যাশবোর্ড টেমপ্লেট আছে (`dashboard_teacher`, `_student`, `_coordinator`, `_institution`) — কোনো ভিউ সেগুলো রেন্ডার করে না। পুরোনো স্কেলিটন, এখনো ফাইলে পড়ে আছে। **মৃত টেমপ্লেট = বিভ্রান্তি:** নতুন ডেভেলপার ওটাই এডিট করে বাগ খুঁজবে।

শিক্ষকের নোট: এই লাইনটা বলুন: "কোড আছে মানে কোড চলে না।" নিজের প্রজেক্টে শেখার নিয়ম — যে ফাইল কোথাও ব্যবহৃত হয় না, সেটা মুছুন, নাহলে ভবিষ্যতে আপনি নিজেই ভুল ফাইল খুলবেন।

ধাপ ৬

একটি মেনু-তালিকা, তিনটি ব্যবহার

`apps/core/navigation.py` — ৫টি গ্রুপ · ২২টি আইটেম · ভূমিকা-ফিল্টার

```
NAV = [
  {"title": None, "items": [
     _item("Dashboard", "bi-speedometer2",
           "reports:dashboard",
           STAFF_ROLES + (Roles.STUDENT,))]},
  {"title": "Teaching", "items": [
     _item("Students", "bi-people",
           "students:list", STAFF_ROLES),
     _item("Attendance", "bi-calendar-check",
           "attendance:take", STAFF_ROLES), ...]},
  ...
]
def navigation_for(request):
    role = request.membership.role
    # ভূমিকা না মিললে আইটেম বাদ
    if role not in item["roles"]: continue
    url = reverse(item["url_name"], kwargs={"slug": slug})
    ...
```

গ্রুপ	আইটেম	কে
(শিরোনামহীন)	Dashboard	সবাই
Teaching	১০	STAFF
My learning	৩	STUDENT
Administration	৬	ORG_ADMIN
Account	২	সবাই

তিনটি ব্যবহার: **sidebar রেন্ডার** · **ড্যাশবোর্ড হাইলাইট** · **টেস্ট** ("এ ভূমিকা ওই লিংক দেখবে না")।

মেনু **দেখানো** নিয়ন্ত্রণ করে, **অনুমতি** নয়। সার্ভার প্রতিটি অনুরোধ আলাদা করে যাচাই করে — লিংক লুকানো মানে নিরাপত্তা নয় (ক্লাস ৯ দেখুন)।

শিক্ষকের নোট: শেষ লাইনটি জোর দিন — ছাত্ররা মনে করে লিংক লুকালেই কাজ শেষ। বলুন: "URL হাতে টাইপ করলে কী হয়? সেটাই আসল প্রশ্ন।"

ধাপ ৭

একই পর্দা, ভূমিকা অনুযায়ী ছোট ছোট পার্থক্য

পুরো পর্দা বদলানো ব্যয়বহুল — তাই টেমপ্লেটের ভেতরে শর্ত

শর্ত	কী বদলায়	কেন
`{% if nav_role == "ORG_ADMIN" %}`	ট্রায়াল ব্যানারে "প্ল্যান বাছুন" লিংক	শিক্ষক নবায়ন করতে পারেন না → লিংক দিলে ৪০৩
`{% if platform_console %}`	sidebar-এ Platform গ্রুপ	সাধারণ ব্যবহারকারীর কাছে অদৃশ্য
`{% if brand.logo %}`	লোগো, নাহলে আইকন	ব্র্যান্ডিং সবসময় পূর্ণ নয়
`{% if subscription.is_writable %}`	ব্যানারের ধরন	read-only অবস্থায় ভিন্ন বার্তা
`{% if e.is_published %}`	Published / Draft ব্যাজ	অপ্রকাশিত নম্বর শিক্ষার্থীর কাছে নেই

নিয়ম: শর্ত মানে **"না দেখানো"**, কখনো **"না করা"** নয়। প্রতিটি শর্তের সমান্তরাল সার্ভার-চেক থাকতেই হবে।

শিক্ষকের নোট: একটা উদাহরণ দিন: "ব্যানারে নবায়ন লিংক সবাইকে দিলে শিক্ষক ক্লিক করবেন, ৪০৩ পাবেন — আর ভাববেন সিস্টেম নষ্ট। শর্তটা ভদ্রতার জন্য, নিরাপত্তার জন্য নয়।"

ধাপ ৮

খালি অবস্থা — শূন্য মানে ভাঙা নয়

৩৩টি টেমপ্লেটে ৪১টি `ims-empty` · ১৬টি `{% empty %}` ব্লক

কোথায়	কী লেখা
classes	"No classes match this filter."
students	"No students enrolled yet."
attendance	"No attendance recorded this month."
exams	"No marks entered yet."
materials	"No materials uploaded yet."

১. আইকন

বোঝায় এটা তালিকা, এরর নয় — `bi-people` , `bi-award` ।

২. এক বাক্য

কী নেই, আর **কেন** নেই — "this month", "match this filter"।

৩. পরের ধাপ

সেরাটা: "Create an academic year first, then a term." — পথ দেখায়।

সাধারণ ভুল: শূন্য তালিকায় **সাদা স্ক্রিন**। ব্যবহারকারী ভাবেন সাইট আটকে গেছে — আসলে প্রশ্নটা ছিল "এখানে কিছু নেই, এখন কী করব?"

শিক্ষকের নোট: ক্লাসে ৩০ সেকেন্ডের কাজ দিন: নিজের প্রজেক্টের যে তালিকায় এখনো ডেটা নেই, তার জন্য এক বাক্য লিখুন। এটাই সবচেয়ে সস্তা উন্নতি।

ধাপ ৯

ফর্মের এক টুকরো — ৪৩টি ফর্ম, এক প্যাটার্ন

`templates/partials/field.html` — ৬ লাইন, প্রতিটি ফিল্ডের অভিন্ন চেহারা

```
<div class="mb-3">
  <label class="form-label">
    {{ field.label }}
    {% if field.field.required %}
      <span class="text-danger">*</span>
    {% endif %}</label>
  {{ field }}
  {% if field.help_text %}
    <div class="ims-hint">{{ field.help_text }}</div>
  {% endif %}
  {% for error in field.errors %}
    <div class="ims-error">{{ error }}</div>
  {% endfor %}
</div>
```

লেবেল + বাধ্যতামূলক

তারকা চিহ্ন সবসময় একই জায়গায় — চোখে পড়ে।

সহায়ক লেখা

`help_text` ফিল্ডের নিচে, ছোট ও ধূসর — প্লেসহোল্ডারে নয়।

এরর ফিল্ডের ঘরে

উপরের সারাংশে নয় — ব্যবহারকারী ঠিক কোন ঘর ঠিক করবেন দেখতে পান।

একই কারণে লগইন পাতার নিচে `ims-slice-note` বক্সে ডেমো লগইন লেখা থাকে — কাগজে লিখে দেওয়ার দরকার নেই।

শিক্ষকের নোট: বলুন: "একটা প্যাটার্ন বানান, তারপর প্রতিটি পেজে সেটাই include করুন। ২০টি ফর্ম হাতে সাজালে ২০টি ভিন্ন উচ্চতা, ভিন্ন এরর-রং — আর ব্যবহারকারী ভাববেন সাইটটা ভাঙা।"

ধাপ ১০

ব্র্যান্ডিং — এক প্রতিষ্ঠানের রং, সবার কোডে

`BrandingForm`-এ ৭টি ঘর · `base.html`-এ একটাই ভেরিয়েবল

```
:root {
  --ims-brand: {{ brand.color|default:"#4f46e5" }};
}
.btn-ims,
.ims-nav.is-active,
.ims-auth__submit { background: var(--ims-brand); }
.ims-sidebar {
  background: linear-gradient(180deg,
    color-mix(in srgb, var(--ims-brand) 22%, #0b1220),
    #0b1220);
}
a { color: var(--ims-brand); }
```

ঘর	কোথায় দেখা যায়
`display_name`	sidebar · লগইন শিরোনাম
`primary_color`	বাটন · সক্রিয় মেনু · sidebar
`tagline`	লগইনের বড় লেখা
`logo`	sidebar ও লগইন
`login_message`	লগইনের নির্দেশনা
`support_email` · `phone`	সহায়তার তথ্য

সেটিংস **ডেটাবেসে**, কোডে নয় — তাই একই কোডবেসে ১০টি প্রতিষ্ঠান, ১০টি রং।

ফাঁদ: ব্যবহারকারী নিজের পছন্দের রং বাছেন — হালকা হলুদে সাদা লেখা পড়া যায় না। সেটিংসে **অন্ধকার রঙের তালিকা**, আর প্রিভিউ কার্ড রাখতেই হয়।

শিক্ষকের নোট: ব্র্যান্ডিং পাতার "Preview" কার্ডটাই আসল শিক্ষা — সেভ করার আগেই ব্যবহারকারী ফলাফল দেখেন। বলুন: "যেখানে ব্যবহারকারী ভুল করতে পারেন, সেখানে আগেই ফলাফল দেখান।"

ধাপ ১১

বার্তা ও গার্ড — ফর্মের আগেই সতর্কবার্তা

দুই partial: `billing_banner` (৪ অবস্থা) · `cookie_guard` (১৫ লাইন JS)

অবস্থা	রং	ব্যবহারকারীকে যা বলা হয়
TRIAL	নীল	কত দিন বাকি · কোন প্ল্যান · "প্ল্যান বাছুন"
মেয়াদ শেষ (read-only)	কমলা	"আপনার ডেটা নিরাপদ ও এক্সপোর্টযোগ্য — নবায়ন করুন"
SUSPENDED	কমলা	কারণ · "ডেটা রাখা আছে, পড়া যায়"
সিট ৯০%+	কমলা	"৪৫০/৫০০ সিট ব্যবহৃত" · আপগ্রেড
কুকি বন্ধ	হলুদ	"এই ভিউতে লগইন ব্যর্থ হবে — সাধারণ ট্যাবে খুলুন"

বার্তার নিয়ম: কী হয়েছে → কেন → এখন কী করব। তিনটি ছাড়া বার্তা নয়।

গার্ড কেন: cookieless iframe-এ লগইন বারবার ব্যর্থ হয় — ব্যবহারকারী ৫ মিনিট টাইপ করার **আগেই** জানানো হয়।

শিক্ষকের নোট: "আপনার ডেটা নিরাপদ" কথাটা কেন লেখা — জিজ্ঞেস করুন। উত্তর: ভয় পাওয়া ব্যবহারকারী সাপোর্টে ফোন করে; আশ্বস্ত করা ব্যবহারকারী নবায়ন করে।

অনুশীলন

২০ মিনিট — নিজের প্রজেক্টের তিনটি খোলস

কাগজে, তারপর কোডে — এই ক্রমেই

১. খোলস আলাদা করুন (৫ মিনিট)

আপনার সিস্টেমে কত ধরনের পর্দা? (লগইন · অ্যাপ · পাবলিক · অ্যাডমিন) প্রতিটির জন্য **একটি** base ফাইল ঠিক করুন।

২. ওয়্যারফ্রেম আঁকুন (৭ মিনিট)

প্রধান পর্দার বক্স + মাপ (px)। রং নয়, ছবি নয় — শুধু আয়তক্ষেত্র আর সংখ্যা।

৩. তিনটি খালি অবস্থা লিখুন (৫ মিনিট)

যে তালিকা এখনো শূন্য, তার জন্য এক বাক্য — কী নেই, কেন নেই, এখন কী করব।

৪. টোকেন তালিকা (৩ মিনিট)

১০–১৫টি ভেরিয়েবল লিখুন — রং, মাপ, ছায়া। তারপর নিজের CSS-এ grep করুন কতগুলো হার্ডকোড রং আছে।

যাচাই: আপনার প্রজেক্টে একটা নতুন পর্দা যোগ করতে কয়টি ফাইল ছুঁতে হলো? ১ হলে ডিজাইন ঠিক, ৫ হলে খোলস ভাঙা।

শিক্ষকের নোট: শেষ প্রশ্নটাই মূল্যায়নের আসল মাপকাঠি — ফাইল সংখ্যা গুনে হাত তুলতে বলুন। যার উত্তর ৫, তার কাজ পরের সপ্তাহে হালকা।

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

উত্তর এক বাক্যে — ব্যাখ্যা বাড়াবেন না

- শিক্ষকের ড্যাশবোর্ড আর প্রতিষ্ঠান প্রধানের ড্যাশবোর্ড — এক টেমপ্লেটে হবে, নাকি দুটো? কোন তথ্য ভিন্ন?
- খালি অবস্থায় "কোনো ডেটা নেই" লেখা কি যথেষ্ট? ব্যবহারকারী এরপর কী চান?
- ব্র্যান্ড রং ব্যবহারকারী বদলাতে পারলে কোন রং নিষিদ্ধ করবেন, আর কীভাবে?
- মেনুতে লিংক লুকানো কি নিরাপত্তা? এক লাইনে যুক্তি দিন।

শিক্ষকের নোট: চতুর্থ প্রশ্নের উত্তরেই ক্লাসের সারমর্ম — "না, লুকানো মানে শুধু দেখা যায় না; অনুমতি সার্ভারে।" এই উত্তর না এলে ক্লাস ৯-এর অংশ আবার বলুন।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
কাঠামো একবার, পর্দা বহুবার	৪টি খোলসে ৭৫টি টেমপ্লেট · `app_base.html` ১১ লাইন
মাপ আগে, রং পরে	wireframe: ২৬৪ / ৫২ / ১২৮০ / ১৪px · ১৫টি টোকেন
ভূমিকা = ভিন্ন পর্দা, ভিন্ন তথ্য	STUDENT · STAFF · PLATFORM — তিন ড্যাশবোর্ড, এক ভিউ
শূন্য অবস্থাও একটা পর্দা	৩৩ ফাইলে ৪১টি `ims-empty` · "এখন কী করব" লেখা
টেমপ্লেট শর্ত = ভদ্রতা, অনুমতি নয়	nav_role শর্ত + সার্ভার-চেক · লুকানো লিংক ≠ সুরক্ষা
মৃত কোড সরান	৪টি অপ্রয়োজনীয় ড্যাশবোর্ড টেমপ্লেট এখনো পড়ে আছে

শিক্ষকের নোট: শেষ সারিটা নিজের প্রজেক্টেও প্রযোজ্য — বলুন: "আজ রাতে ২০ মিনিট লাগবে, নিজের রিপোতে এমন মৃত টেমপ্লেট খুঁজে মুছে ফেলুন।"

১৩

পরের ক্লাস — "ডেটা কোথায় যায়"

DFD: Level 0 · Level 1 · ডেটা স্টোর · বাহ্যিক সত্তা · প্রসেসের নামকরণ

পর্ব ৩ · ডিজাইন · ক্লাস ১৩ / ৩৭

ডেটা কোথায় যায়

সিলেবাস: প্র্যাকটিক্যাল ৪ · টপিক ৪.২ (DFD)
Level 0 · Level 1 · ৯টি স্টোর · ব্যাচ প্রসেস

৮৫টি রুট · ৩১টি টেবিল · ১১টি প্রসেস

শুরু করুন প্রশ্ন দিয়ে

একটি উপস্থিতি সেভ হলে ডেটা কত জায়গায় যায়?

হাত তুলুন: এক জায়গায়? তিন জায়গায়?

উপস্থিতির একটি সারি লেখা হয় ১টি টেবিলে,
কিন্তু তার ছাপ পড়ে অডিট লগে, রিপোর্টে, ড্যাশবোর্ডের সংখ্যায়।

DFD এই ছাপগুলো ছবিতে আঁকে — কে দেয়, কে নেয়, কোথায় জমা থাকে। কোড লেখার আগে এই ছবি থাকলে "এই ডেটা কোথা থেকে এলো?" প্রশ্নের উত্তর খুঁজতে তিন ঘণ্টা লাগে না।

শিক্ষকের নোট: উত্তর সাধারণত "এক জায়গায়" আসে। তারপর বলুন: "আজ আমরা সেই একটি লেখার পেছনের পাঁচটি পড়া বের করব।"

ধাপ ১

DFD-এর চারটি প্রতীক — এর বেশি লাগে না

চারটি আকৃতি, একটাই নিয়ম: প্রতিটি তীরের গায়ে লেখা থাকবে **কী যাচ্ছে**



নিয়ম: বাহ্যিক সত্তা সিস্টেমের বাইরে · প্রসেস কাজ করে · স্টোর শুধু রাখে, নিজে কিছু করে না · তীরের গায়ে সর্বদা নাম।

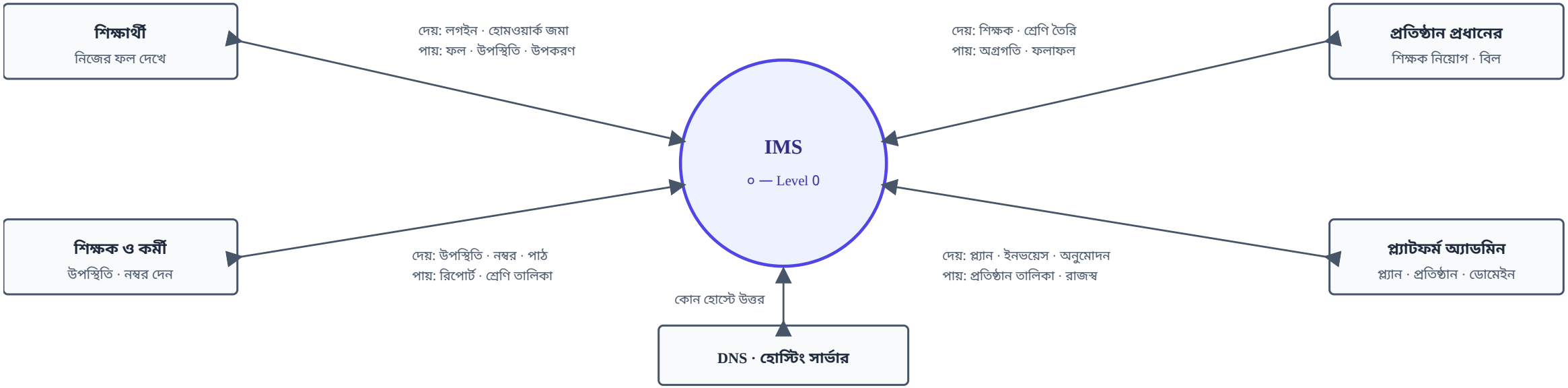
একটি গল্প বলতে চারটি প্রতীকই যথেষ্ট। রং, ছায়া, আইকন — ডায়াগ্রামের মান বাড়ায় না, **নামের স্পষ্টতা** বাড়ায়।

শিক্ষকের নোট: বোর্ডে চারটি আকৃতি আঁকুন, তারপর ছাত্রদের নিজের প্রজেক্টের একটি কাজকে এই চারটিতে ভাগ করতে বলুন। যার তীরে নাম নেই, তার DFD অর্ধেক।

ধাপ ২

Level 0 — পুরো সিস্টেম একটি বৃত্ত

পাঁচটি বাহ্যিক সত্তা · শূন্য ডেটা স্টোর · একটি প্রসেস



Level 0-তে **ডেটা স্টোর দেখানো হয় না** — কারণ বাইরের কেউ স্টোর দেখে না, সে সিস্টেমকে একটি কাজ হিসেবে দেখে।

শিক্ষকের নোট: জিজ্ঞেস করুন: "প্ল্যাটফর্ম অ্যাডমিন কি বাহ্যিক সত্তা?" উত্তর: হ্যাঁ — সে IMS-এর ব্যবহারকারী নয়, IMS-কে চালায়। এই পার্থক্যটাই Level 0-এর মূল শিক্ষা।

ধাপ ৩

Level 1 — ১১টি অ্যাপ মানে ১১টি প্রসেস

কোনো সংখ্যা বানানো নয়: প্রতিটি অ্যাপের `urls.py` গুনে মোট ৮৫টি রুট

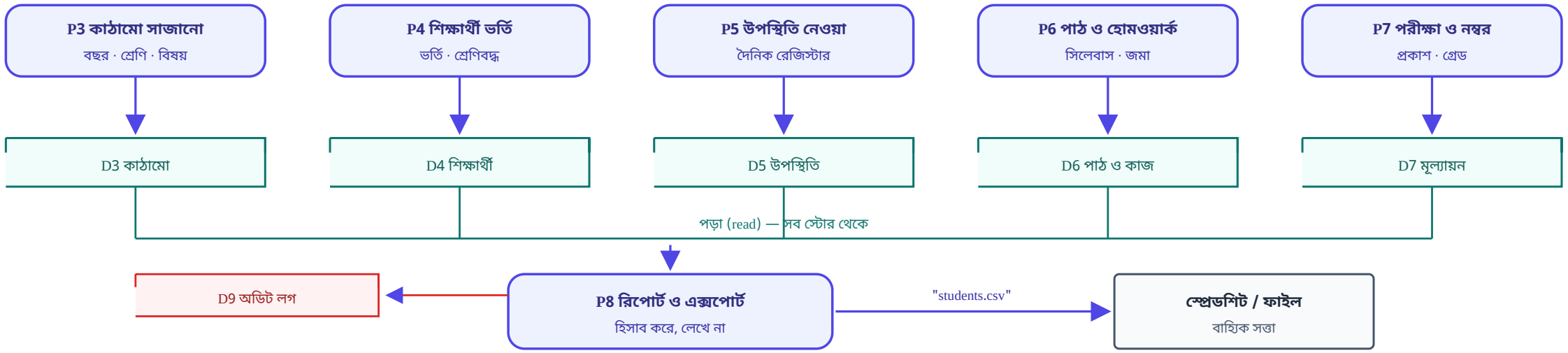
#	প্রসেস (ক্রিয়া)	অ্যাপ	রুট	যে স্টোরে লেখে
P1	লগইন ও সেশন চালানো	accounts	৯	D1 ব্যবহারকারী
P2	প্রতিষ্ঠান নির্ধারণ ও সদস্যতা	workspaces	২	D1 · D2
P3	একাডেমিক কাঠামো সাজানো	academics	১৩	D3 কাঠামো
P4	শিক্ষার্থী ভর্তি ও শ্রেণিবদ্ধ করা	students	৭	D4 শিক্ষার্থী
P5	উপস্থিতি নেওয়া ও সংশোধন	attendance	৪	D5 উপস্থিতি
P6	পাঠ, হোমওয়ার্ক ও উপকরণ দেওয়া	lessons	১৩	D6 পাঠ
P7	পরীক্ষা তৈরি ও নম্বর প্রকাশ	assessments	৯	D7 মূল্যায়ন
P8	রিপোর্ট তৈরি ও ফাইল বের করা	reports	৭	পড়ে D3–D7 · লেখে D9
P9	সাবস্ক্রিপশন ও ইনভয়েস	billing	১৮	D8 বাণিজ্যিক
P10	প্ল্যাটফর্ম প্রশাসন	core · views_platform	১ + ট্যাব	D8 · D2
P11	অডিট লেখা (সবাই ডাকে)	audit	২	D9 অডিট

শিক্ষকের নোট: বলুন: "Level 1 বানাতে নতুন কিছু ভাবতে হয় না — আপনার ফোল্ডার তালিকাই প্রসেসের তালিকা।" এতে DFD লেখা সহজ হয়ে যায়, আর ভুল কমে।

ধাপ ৪

Level 1 ছবিতে — লেখা আর পড়া আলাদা

প্রতিটি প্রসেস নিজের স্টোরে লেখে; রিপোর্ট সবার স্টোর পড়ে — একটিতেও লেখে না



P8 একটি স্টোরেও লেখে না — কিন্তু অডিটের জন্য ডাকে। তাই D9-এ তার লেখা দুটি: "students.csv" আর "attendance.save"।

শিক্ষকের নোট: তীরের দিকটাই এখানে আসল শিক্ষা — একই প্রসেস সরবরাহ করে, আবার পড়ে; কিন্তু কোনো প্রসেস কখনো একই স্টোরে দুই দিকে তীর আঁকে না।

ধাপ ৫

৩১টি টেবিল, কিন্তু ৯টি ডেটা স্টোর

DFD-তে টেবিল গোনা হয় না — **একসাথে রাখা দরকার** এমন জিনিস গোনা হয়

স্টোর	টেবিল	ভেতরে কী
D1 ব্যবহারকারী ও সদস্যতা	২	User · Membership (ভূমিকা, সক্রিয়তা)
D2 প্রতিষ্ঠান ও ব্র্যান্ডিং	২	Workspace · WorkspaceBranding (ডোমেইন, রং)
D3 একাডেমিক কাঠামো	৭	বছর · টার্ম · বিভাগ · শ্রেণি · বিষয় · শিক্ষক-বণ্টন · কোঅর্ডিনেটর স্কোপ
D4 শিক্ষার্থী ও ভর্তি	২	StudentProfile · Enrollment
D5 উপস্থিতি	২	AttendanceRecord · AttendancePolicy
D6 পাঠ ও কাজ	৭	Syllabus · Chapter · Topic · LessonLog · Homework · Submission · Material
D7 মূল্যায়ন	৪	GradeScale · GradeBand · Exam · Mark
D8 বাণিজ্যিক	৪	Plan · Subscription · Invoice · SignupRequest
D9 অডিট	১	AuditLog — ২৫টি ভিন্ন action

ডেটাবেসের বাইরেও দুটি স্টোর: **সেশন স্টোর** (কোন প্রতিষ্ঠানে আছেন) আর **মিডিয়া ফাইল** (উপকরণ, লোগো)। DFD-তে এগুলোও স্টোর — কারণ তারা টিকে থাকে।

শিক্ষকের নোট: ৯ বনাম ৩১ — এই পার্থক্যটা বলুন: "৩১ হলো আলমারির ড্রয়ার, ৯ হলো আলমারি। DFD-তে আলমারি আঁকি, ড্রয়ার নয় — নাহলে ছবিটা পড়া যায় না।"

ধাপ ৬

নামকরণ — DFD-এর সবচেয়ে সস্তা গুণ

ভুল নাম থাকলে ছবিটা ভুল হয়ে যায়, আর কেউ ধরতে পারে না

কী	ভুল	ঠিক
প্রসেস	"শিক্ষকের কাজ" · "Attendance"	"উপস্থিতি রেকর্ড করা" — **ক্রিয়া + বিশেষ্য**
স্টোর	"উপস্থিতি নেওয়া"	"D5 উপস্থিতি" — **বিশেষ্য**, সংখ্যায়িত
সত্তা	"ড্যাশবোর্ড" · "ডেটাবেস"	"শিক্ষক" · "প্ল্যাটফর্ম অ্যাডমিন" — **সিস্টেমের বাইরের মানুষ**
তীর	"ডেটা" · "তথ্য"	"উপস্থিতির সারি" · "নম্বর প্রকাশের অনুরোধ"

খারাপ তীর: "ডেটা" — পড়ে কেউ বলতে পারে না কী যাচ্ছে, তাই ভুল জায়গায় লেখা ধরাও পড়ে না।

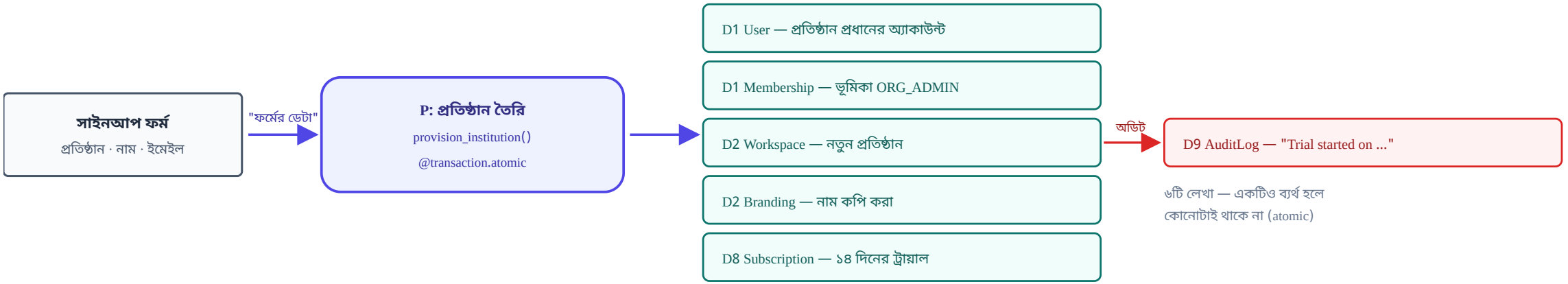
ভালো তীর: "উপস্থিতির সারি (শিক্ষার্থী, তারিখ, অবস্থা)" — নিজেই একটা স্কিমার খসড়া।

শিক্ষকের নোট: শ্রেণিতে একটি নিয়ম চালু করুন: তীরের গায়ে "ডেটা" লেখা পেলে খাতা ফেরত। এক সপ্তাহে অভ্যাস হয়ে যাবে।

ধাপ ৭

একটি সেভ, ছয়টি স্টোরে লেখা

`provision_institution()` — পাবলিক সাইনআপ ফর্ম থেকে প্রতিষ্ঠান তৈরি



DFD-তে একটি প্রসেস একাধিক স্টোরে লিখতে পারে — কিন্তু **সব লেখা একসাথে সফল বা একসাথে ব্যর্থ**। এটাই ডেটা-অখণ্ডতার নিয়ম, আর কোডে সেটি `@transaction.atomic`।

শিক্ষকের নোট: বলুন: "যদি পঞ্চম লেখাটা ফেল করে, প্রথম চারটে থেকে গেলে কী হবে? ব্যবহারকারী তৈরি, প্রতিষ্ঠান নেই — অর্ধেক মানুষ।" এজন্যই transaction।

ধাপ ৮

একটি ডিজাইন-সিদ্ধান্ত: তাৎক্ষণিক, নাকি অনুমোদন?

একই DFD, দুই রকম তীর — পার্থক্য শুধু একটি প্রসেসে

প্রশ্ন	তাৎক্ষণিক (IMS যা করছে)	অনুমোদনের পর
ব্যবহারকারীর অভিজ্ঞতা	ফর্ম শেষ → সোজা ড্যাশবোর্ড	ফর্ম শেষ → "অপেক্ষা করুন" পর্দা
ট্রেইল	`SignupRequest` তৈরি হয়, অবস্থা **PROVISIONED**	তৈরি হয়, অবস্থা **NEW → অনুমোদিত**
স্প্যাম	ঝুঁকি বেশি → পরে **SPAM/REJECTED** মার্ক	ঝুঁকি কম, দেরি বেশি
কে থামাতে পারে	প্ল্যাটফর্ম অ্যাডমিন, পরে (suspend)	প্ল্যাটফর্ম অ্যাডমিন, আগেই

যেটাই বাছুন, **লিখে রাখুন কেন**। IMS বেছে নিয়েছে "আগে ঢোকা, পরে থামানো" — কারণ ট্রায়ালের মূল্য হলো শিক্ষক পাঁচ মিনিটে ভেতরে ঢুকতে পারেন। ঝুঁকির প্রতিকার DFD-তেই দেখা যায়: `SignupRequest` নামের একটি স্টোর আছে যেখানে IP ঠিকানা পর্যন্ত জমা থাকে।

শিক্ষকের নোট: জিজ্ঞেস করুন: "আপনার প্রজেক্টে কোনটা?" তারপর বলুন: "উত্তরটা যেটাই হোক, DFD-তে সেই পথটাই তীর হয়ে আঁকতে হবে — নয়তো ছবি আর কোড আলাদা কথা বলবে।"

ধাপ ৯

প্রসেসের শেষ প্রান্ত: ফাইল

৩টি CSV এক্সপোর্ট — `reports/urls.py`-এ দেখা যায়, বাইরের সত্তা "স্প্রেডশিট"

```
def _csv_response(filename, header, rows):
    resp = HttpResponse(
        content_type="text/csv; charset=utf-8")
    resp["Content-Disposition"] = (
        f'attachment; filename="{filename}"')
    resp.write("\ufeff")   # BOM: Excel-এ বাংলা ঠিক দেখায়
    writer = csv.writer(resp)
    writer.writerow(header)
    for row in rows:
        writer.writerow(row)
    return resp
```

ফাইল	কোন স্টোর পড়ে
`students.csv`	D4 শিক্ষার্থী
`attendance-*.csv`	D5 উপস্থিতি
`results-*.csv`	D7 মূল্যায়ন

নিয়ম: ফাইল বের করা একটি **পড়ার** কাজ — কোনো স্টোরে লেখা হয় না, শুধু অডিটে ছাপ পড়ে।

ছাত্র `results.csv` চাইলে ৪০৩ — রিপোর্টও ভূমিকা মেনে চলে।

শিক্ষকের নোট: BOM লাইনটা দেখিয়ে বলুন: "এই একটি অক্ষর না থাকলে Excel-এ বাংলা অক্ষর ভাঙা দেখায়। DFD-তে এটা দেখা যায় না — কিন্তু ব্যবহারকারীর কাছে এটাই পুরো সিস্টেম।"

ধাপ ১০

ব্যাচ প্রসেস — যেখানে কোনো ব্যবহারকারী নেই

৪টি ম্যানেজমেন্ট কমান্ড: `seed_demo` · `seed_plans` · `billing_report` · `sync_domains`

কমান্ড	কোন স্টোরে	কী কাজ
`seed_plans`	D8 লেখে	ডিফল্ট প্ল্যান তৈরি — খালি ডেটাবেসে সিস্টেম চলে
`seed_demo`	D1–D7 লেখে	ডেমো প্রতিষ্ঠান, শিক্ষক, ছাত্র, উপস্থিতি
`billing_report`	D8 পড়ে	আয়ের সারসংক্ষেপ ছাপে
`sync_domains`	D2 পড়ে	ভেরিফায়েড ডোমেইনের CNAME ও `ALLOWED_HOSTS` লাইন ছাপে

DFD-তে ব্যাচ প্রসেস: যে প্রসেস ব্যবহারকারীর অনুরোধ ছাড়াই চলে, তাকেও আঁকতে হয় — নাহলে "এই রেকর্ড কে বানাল?" প্রশ্নের উত্তর ছবিতে থাকে না।

DNS বাইরের সত্তা: `sync_domains` শুধু **কী করতে হবে** ছাপে — DNS বদলায় না। অ্যাপ নিজে নিজের ডোমেইন বানাতে পারে না।

শিক্ষকের নোট: এখানে সততার পাঠ: "যা করতে পারে না, তা কোডে লেখা নেই। স্কুলকে CNAME বানাতে হবে — আমাদের কাজ লাইনটা ছাপা, ভান করা নয়।"

সতর্কতা

DFD-তে ছয়টি সাধারণ ভুল

পরীক্ষায় নম্বর কাটে, প্রজেক্টে বিভ্রান্তি ছড়ায়

ভুল	কারণ ও সমাধান
স্টোর → স্টোর সরাসরি তীর	টেবিল নিজে কিছু করে না। মাঝখানে প্রসেস বসান
Level 0-তে ডেটা স্টোর	Level 0 হলো বাইরের চোখের ছবি — ভেতরের আলমারি ওখানে নেই
Level 1 ও 2 এক পাতায়	এক পাতা, এক স্তর। নাহলে ছবি পড়া যায় না
প্রসেসের নামে মানুষ ("শিক্ষক যা করেন")	প্রসেস সবসময় **ক্রিয়া**: "উপস্থিতি রেকর্ড করা"
নামহীন তীর	"ডেটা" লেখাও নাম নয় — কী ডেটা, কার থেকে কার
অডিট স্টোর ভুলে যাওয়া	২৫টি action লেখা হয় — D9 বাদ দিলে ছবি অসম্পূর্ণ

সবচেয়ে বড় ভুল: **ছবি একবার আঁকা, তারপর কোড বদলালে ছবি না বদলানো।** DFD কোডের সাথে বাঁচে — বদলালে আপডেট করুন, নয়তো ছবিটা মিথ্যা বলে।

শিক্ষকের নোট: শেষ লাইনটা নিজের ছাত্রজীবনের উদাহরণ দিয়ে বলুন — "প্রথম প্রজেক্টে আমার ডায়াগ্রাম আর কোড দুই রকম হয়ে গিয়েছিল, ভাইভায় সেটাই ধরা পড়ল।"

অনুশীলন

২০ মিনিট — নিজের সিস্টেমের Level 0 আর Level 1

কাগজ就可以了 — হাতের আঁকাই যথেষ্ট

১. বাহ্যিক সত্তা লিখুন (৪ মিনিট)

কে সিস্টেমের বাইরের মানুষ? (৪–৬ জন) প্রত্যেকে কী দেয়, কী নেয় — দুই কলামে।

২. Level 0 আঁকুন (৪ মিনিট)

একটি বৃত্ত, চারপাশে সত্তা, প্রতিটি তীরে নাম। **কোনো স্টোর নয়।**

৩. Level 1 ভাঙুন (৭ মিনিট)

আপনার অ্যাপ/মডিউল = প্রসেস। প্রতিটির পাশে লিখুন কোন স্টোরে **লেখে** আর কোনটা **পড়ে**।

৪. যাচাই (৫ মিনিট)

আপনার মডেল ফাইল খুলুন — DFD-র প্রতিটি স্টোরের টেবিল আছে? টেবিল আছে কিন্তু স্টোর নেই — কোথায়?

যাচাইয়ের প্রশ্ন: DFD দেখে একজন নতুন ডেভেলপার বলতে পারবে "উপস্থিতির ডেটা কোথা থেকে আসে?" — না পারলে ছবি অসম্পূর্ণ।

শিক্ষকের নোট: চতুর্থ ধাপটাই আসল কাজ — ছাত্ররা দেখবে তাদের কোডে এমন টেবিল আছে যা DFD-তে নেই (বা উল্টো)। সেটাই পরের ক্লাসের (ERD) সূচনা।

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

উত্তর এক বাক্যে

- প্ল্যাটফর্ম অ্যাডমিন Level 0-তে বাহ্যিক সত্তা, কিন্তু সে তো আমাদেরই একজন — কেন বাইরে?
- অডিট লগ কি "স্টোর" না "প্রসেস"? এক লাইনে যুক্তি দিন।
- সাইনআপ তাৎক্ষণিক না অনুমোদনসাপেক্ষ — আপনার প্রজেক্টে কোনটা, আর কেন?
- DFD-তে যা দেখানো হয় না, কোডে তা থাকলে কী সমস্যা? উদাহরণ দিন।

শিক্ষকের নোট: চতুর্থ প্রশ্নে উদাহরণ আসবে "ব্যাকগ্রাউন্ড জব" বা "ক্যাশ"। বলুন: "যা ছবিতে নেই, তা কেউ রক্ষণাবেক্ষণও করে না — ছবিটাই সিস্টেমের স্মৃতি।"

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
Level 0 = বাইরের চোখ — এক প্রসেস, স্টোর নেই	৫টি সত্তা · ১টি বৃত্ত · ০ স্টোর
Level 1 = আপনার ফোল্ডার তালিকা	১১ অ্যাপ · ১১ প্রসেস · ৮৫ রুট
স্টোর গোনা হয় দরকারে, টেবিলে নয়	৩১ টেবিল → ৯ স্টোর + সেশন + মিডিয়া
প্রতিটি সেভ একাধিক স্টোর ছুঁতে পারে	`provision_institution()` — ৬টি লেখা, একটি transaction
ফাইল বের করা = পড়া, লেখা নয়	৩টি CSV · BOM সহ · ছাত্রের জন্য ৪০৩
ব্যাচ প্রসেস আর বাইরের সিস্টেমও ছবিতে থাকে	৪টি কমান্ড · DNS CNAME · `ALLOWED_HOSTS`

শিক্ষকের নোট: তৃতীয় সারিটা বোর্ডে লিখুন — "৩১ → ৯"। এই একটি সংখ্যাই DFD আর ERD-এর পার্থক্য বোঝায়।

১৪

পরের ক্লাস — "কোন ডেটা কার সাথে থাকে"

ERD: ৩১টি মডেল · সম্পর্কের ধরন · কী-এর পছন্দ · স্কোপ কলাম কেন প্রতিটি টেবিলে

পর্ব ৩ · ডিজাইন · ক্লাস ১৪ / ৩৭

কোন ডেটা কার সাথে থাকে

সিলেবাস: প্র্যাকটিক্যাল ৪ · টপিক ৪.৩ (ডেটাবেস ডিজাইন)

ERD: ৩১ টেবিল · ৭৫ সম্পর্ক · কার্ডিনালিটি

৭০টি FK · ৪টি 1:1 · ১টি M:N

শুরু করুন প্রশ্ন দিয়ে

একজন শিক্ষার্থী চারটি শ্রেণিতে ছিল — কোন টেবিলে লেখা থাকে?

হাত তুলুন: শিক্ষার্থীর টেবিলে? নাকি আলাদা টেবিলে?

৩১টি টেবিল, ৭৫টি সম্পর্ক। সম্পর্কটা না বুঝে কোড লিখলে শ্রেণি বদলানোর সময় পুরোনো ইতিহাস হারিয়ে যায়।

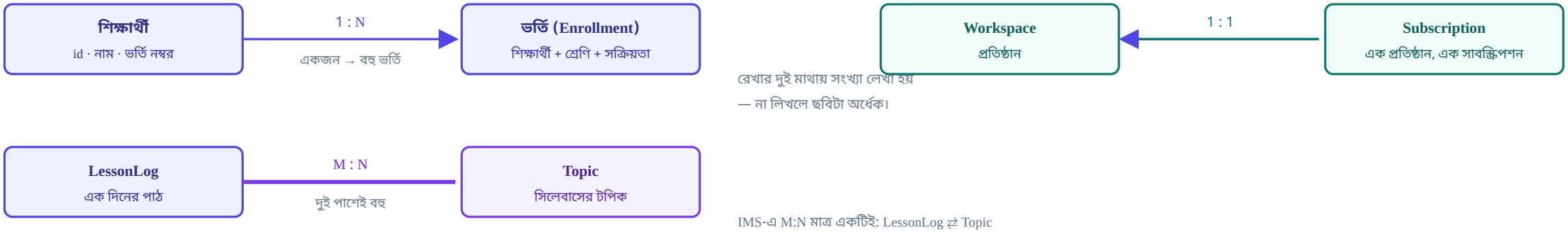
IMS-এ উত্তরটি হলো `Enrollment` — শিক্ষার্থী আর শ্রেণির মাঝখানের টেবিল। আজকের ক্লাসে কেন উত্তরটা শিক্ষার্থীর টেবিলে **হতে পারে না**, সেটাই মূল আলোচনা।

শিক্ষকের নোট: বেশিরভাগের উত্তর আসবে "শিক্ষার্থীর টেবিলে class_id"। বলুন: "তাহলে গত বছরের শ্রেণি কোথায় থাকল? মুছে গেল?" — এই একটি প্রশ্নেই আজকের ক্লাস দাঁড়িয়ে যায়।

ধাপ ১

ERD পড়ার তিনটি চিহ্ন

বাক্স = টেবিল · রেখা = সম্পর্ক · রেখার মাথার লেখা = কতজন



কার্ডিনালিটি পড়ার নিয়ম: **"এক X-এর অধীনে কতজন Y?"** — উত্তর ১ হলে 1:1, বহু হলে 1:N, আর দুই দিকেই বহু হলে M:N।

শিক্ষকের নোট: ছাত্রদের দিয়ে প্রশ্নটাই উচ্চারণ করান: "এক শ্রেণিতে কতজন ছাত্র?" (বহু) "একজন ছাত্র কত শ্রেণিতে?" (বহু, সময়ের সাথে) — তাই মাঝখানে টেবিল।

ধাপ ২

৭৫টি সম্পর্কের হিসাব

`grep "ForeignKey\|OneToOneField\|ManyToManyField" apps/*/models.py` — হাতে গোনা

অ্যাপ	FK	1:1	M:N	মোট
lessons	২১	—	১	**২২**
academics	১৭	—	—	**১৭**
assessments	১০	—	—	**১০**
attendance	৬	১	—	**৭**
billing	৬	১	—	**৭**
students	৫	১	—	**৬**
workspaces	৩	১	—	**৪**
audit	২	—	—	**২**

মোট **৭৫**: ৭০টি FK · ৪টি 1:1 · ১টি M:N।
৩১টি টেবিলের মধ্যে **২৩টি** সরাসরি প্রতিষ্ঠান-স্কোপড।

lessons-এ সবচেয়ে বেশি সম্পর্ক (২২) — কারণ এক পাঠে টপিক, হোমওয়ার্ক, জমা, উপকরণ, শ্রেণি, বিষয়, শিক্ষক — সাতটি জিনিস একসাথে জড়ায়। ডিজাইনে সবচেয়ে কঠিন অ্যাপও এটাই।

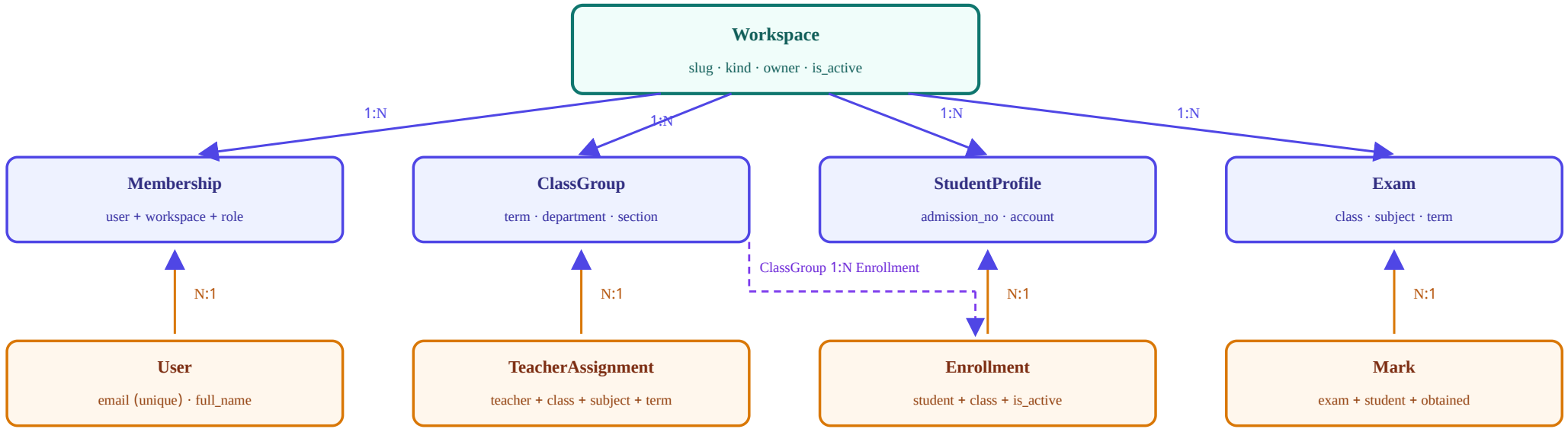
অডিটে মাত্র ২টি সম্পর্ক — তবু প্রতিটি প্রসেস সেখানে লেখে। সম্পর্ক কম মানে গুরুত্ব কম নয়।

শিক্ষকের নোট: বোর্ডে ৭৫ লিখুন, নিচে ২৩। বলুন: "৭৫ সম্পর্কের মধ্যে ২৩টি প্রতিষ্ঠানকে ধরে রাখে — বাকিগুলো ভেতরের গল্প।"

ধাপ ৩

ERD — প্রথম ভাগ: প্রতিষ্ঠান, মানুষ ও ভূমিকা

চারটি টেবিলেই প্রতিষ্ঠানের সীমা তৈরি হয়: Workspace · Membership · ClassGroup · Enrollment



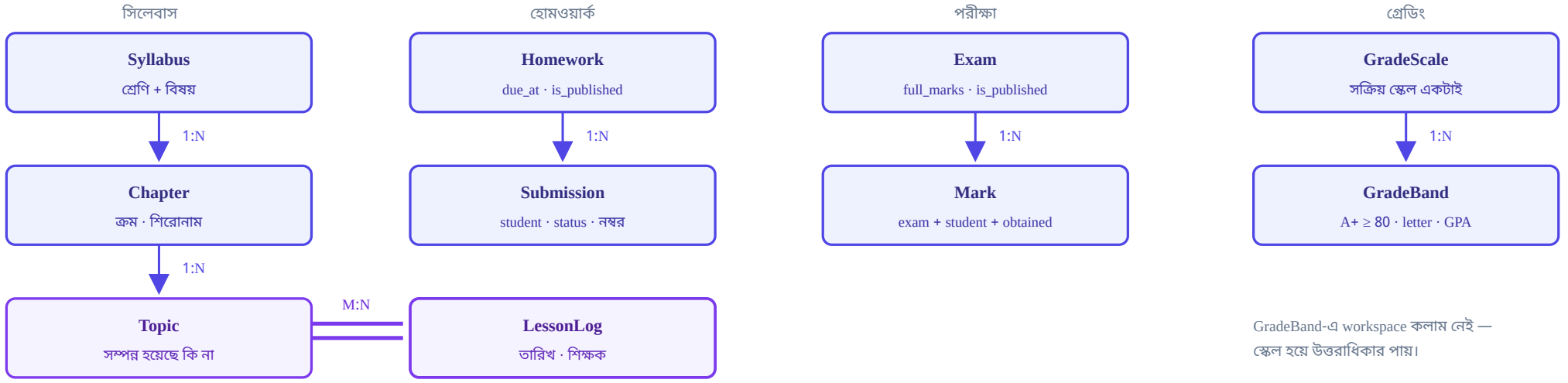
লক্ষ্য করুন: **User** আর **Workspace** সরাসরি জুড়তে হয় না — মাঝখানে Membership, কারণ এক ব্যবহারকারী বহু প্রতিষ্ঠানে ভিন্ন ভূমিকায় থাকতে পারে।

শিক্ষকের নোট: Membership-এর জায়গাটা আঙুল দিয়ে দেখান: "ভূমিকা কোথায় থাকবে? User-এ রাখলে এক ব্যবহারকারীর একটাই ভূমিকা হতো — শিক্ষক আবার অভিভাবক হতে পারতেন না।"

ধাপ ৪

ERD — দ্বিতীয় ভাগ: পাঠ, কাজ ও মূল্যায়ন

তিনটি চেইন: সিলেবাস · হোমওয়ার্ক · পরীক্ষা — আর একটাই M:N



তিনটি চেইনই **1:N** — কারণ পিতৃ-টেবিলের একটি সারির অধীনে বহু সন্তান। M:N একমাত্র সিলেবাসে, কারণ এক পাঠে একাধিক টপিক, আর এক টপিক বহু পাঠে।

শিক্ষকের নোট: ছাত্রদের বলুন প্রতিটি চেইনে "পিতা কে, সন্তান কে" মুখে বলতে — মুখে বলতে না পারলে ERD মুখস্থ, বোঝা নয়।

ধাপ ৫

চারটি 1:1 — কেন আলাদা টেবিল

1:1 মানে "একটির বেশি হতে পারে না" — সেটাই ডিজাইনের যুক্তি

সম্পর্ক	কেন একটির বেশি নয়	আলাদা টেবিল না হলে কী হতো
Workspace → Subscription	এক প্রতিষ্ঠানের একটাই চুক্তি	সাবস্ক্রিপশন ফিল্ড ৭টি প্রতিষ্ঠান-টেবিলে ঢুকে যেত
Workspace → AttendancePolicy	একটি উপস্থিতি নীতি	নীতি ছাড়া প্রতিষ্ঠান চলত, NULL-এ ভরা কলাম
Workspace → WorkspaceBranding	এক সেট ব্র্যান্ডিং	ব্র্যান্ডিং এডিট করতে প্রতিষ্ঠান-টেবিলে হাত দিতে হতো
User → StudentProfile	একজন ব্যবহারকারী এক শিক্ষার্থী	লগইন ছাড়া ছাত্র থাকত না — অথচ অ্যাকাউন্ট ঐচ্ছিক

সাধারণ নিয়ম: 1:1 ব্যবহার করি যখন তথ্য **ঐচ্ছিক**, **ভিন্ন দায়িত্বে**, বা **বড়** (ফাইল/JSON)।

ফাঁদ: 1:1-এ ডেটা দুই টেবিলে ভাগ হলে প্রতিটি কুয়েরিতে join লাগে — অকারণ ভাগ করলে সব ধীর ও জটিল হয়ে যায়।

শিক্ষকের নোট: চতুর্থ সারিটি জোর দিন: "StudentProfile.account ঐচ্ছিক — কারণ ছোট প্রতিষ্ঠানে সব ছাত্রের লগইন থাকে না। ERD-তে ঐচ্ছিকতা দেখা যায় NULL দিয়ে।"

ধাপ ৬

M:N একটাই — কিন্তু যৌগিক টেবিল অনেকগুলো

`LessonLog.topics` Django নিজেই মাঝের টেবিল বানায়; বাকি জায়গায় হাতে বানানো হয়েছে

```
class LessonLog(TimeStamped):
    topics = models.ManyToManyField(
        Topic, blank=True, related_name="lesson_logs")

class AttendanceRecord(TimeStamped):
    class Meta:
        constraints = [
          models.UniqueConstraint(
            fields=["student", "class_group",
                    "subject", "date"],
            name="uniq_attendance_student_class_"
                 "subject_date"),
        ]

class Mark(TimeStamped):
    class Meta:
        constraints = [
          models.UniqueConstraint(
            fields=["exam", "student"],
            name="uniq_mark_exam_student"),
        ]
```

Django-র M:N কখন যথেষ্ট

মাঝের টেবিলে **অতিরিক্ত তথ্য নেই** — শুধু "কোন পাঠে কোন টপিক"।

নিজের টেবিল কখন লাগে

যখন সম্পর্কের নিজের গুণ আছে — কে রেকর্ড করল, কখন, কোন অবস্থা। AttendanceRecord-এ ৮টি নিজের ফিল্ড।

ট্রিক: যৌগিক unique

M:N টেবিলে ডুপ্লিকেট আটকাতে ৪টি কলামের unique — একই দিনে একই ছাত্রের একই ক্লাসে দ্বিতীয় সারি অসম্ভব।

ফাঁদ: M:N ব্যবহার করলে **কে কখন যোগ করল** তথ্যটি হারায়। উপস্থিতি, নম্বর, জমা — এগুলো কখনোই "শুধু সম্পর্ক" নয়, এগুলো **ঘটনা**।

শিক্ষকের নোট: বলুন: "যদি প্রশ্নের উত্তর দিতে হয় 'কে করল, কখন করল' — তাহলে M:N নয়, নিজের টেবিল।" এই এক লাইনেই ছাত্রদের অনেক ভুল বাঁচবে।

ধাপ ৭

কী (Primary Key) — কখন অটো, কখন প্রাকৃতিক

৩১টি টেবিলে অটো `id`; কিন্তু ৬টি জায়গায় নিজস্ব ইউনিক কী

কী	ধরন	কেন এটা
`id` (সব টেবিলে)	অটো ইনক্রিমেন্ট	কখনো বদলায় না, ছোট, ইনডেক্সে হালকা
`User.email`	প্রাকৃতিক · unique	লগইনের পরিচয় — `USERNAME_FIELD`
`Workspace.slug`	প্রাকৃতিক · unique	URL-এ দেখা যায়: `/w/springfield/`
`StudentProfile.admission_no`	যৌগিক unique `(workspace, admission_no)`	প্রতিষ্ঠানভেদে একই নম্বর চলতে পারে
`Invoice.number`	প্রাকৃতিক · unique	হিসাবরক্ষণের নথি — পুনরায় ব্যবহার নিষিদ্ধ
`Plan.code`	প্রাকৃতিক · slug unique	কোডে উল্লেখযোগ্য: `starter`

নিয়ম: **PK অদৃশ্য থাকুক** (id), **মানুষের দেখা নম্বরটা unique কলাম হোক** (admission_no)।

ভুল: `admission_no`-কে PK বানানো — প্রতিষ্ঠান বদলালে বা নম্বর সংশোধন হলে পুরো সম্পর্ক ভাঙে।

শিক্ষকের নোট: জিজ্ঞেস করুন: "ভর্তি নম্বর ভুল ছিল, এখন ঠিক করবেন — PK হলে কী হবে?" উত্তর: সব foreign key ভেঙে যাবে। এজন্যই অটো id।

ধাপ ৮

ইউনিক — ৬টি পুরোনো, ৮টি নতুন

`unique_together` বনাম `UniqueConstraint` — দ্বিতীয়টি শর্ত দিতে পারে

unique_together (৬)	কীসের উপর
AcademicYear	workspace + name
AcademicTerm	workspace + year + name
Department	workspace + name
ClassGroup	workspace + term + name + section
Subject	workspace + name
TeacherAssignment	teacher + class + subject + term

UniqueConstraint (৮)	শর্ত
uniq_active_term_per_workspace	একটাই সক্রিয় টার্ম
uniq_active_grade_scale_per_workspace	একটাই সক্রিয় স্কেল
uniq_active_enrollment_student_class	একটাই সক্রিয় ভর্তি
unique_custom_domain	খালি ডোমেইন ছাড়া
uniq_membership_user_workspace	শর্ত নেই
uniq_mark_exam_student	শর্ত নেই
uniq_attendance_...date	শর্ত নেই
uniq_submission_homework_student	শর্ত নেই

প্রথম ৪টি **শর্তসাপেক্ষ (partial)** — শুধু "সক্রিয়" সারিগুলোর মধ্যে ইউনিক।

পুরোনো বছরগুলোর ইতিহাস রাখতে হবে, তাই `(workspace, year, name)` সাধারণ unique; কিন্তু একসাথে দুইটি **সক্রিয়** টার্ম চলা যাবে না — সেটি partial। বিস্তারিত ক্লাস ১৫-এ।

শিক্ষকের নোট: পার্থক্যটা এক বাক্যে: "সাধারণ unique বলে 'সবসময় একটাই'; partial unique বলে 'একটাই সক্রিয়, বাকিগুলো ইতিহাস'।"

ধাপ ৯

স্কোপ কলাম — ২৩টিতে সরাসরি, ৮টিতে উত্তরাধিকারে

`workspace` কলাম আছে কি না — প্রতিটি কুয়েরির নিরাপত্তা এর উপর নির্ভর করে

২৩টি টেবিলে সরাসরি `workspace`:
কাঠামো · ছাত্র · উপস্থিতি · পাঠ · পরীক্ষা · অডিট · সাবস্ক্রিপশন...

৮টিতে নেই: User · Workspace · Plan · GradeBand · Chapter · Topic · Mark · Submission

টেবিল	স্কোপ কোথা থেকে আসে
Chapter · Topic	`syllabus.workspace` — দুই ধাপ
Mark	`exam.workspace` — এক ধাপ
Submission	`homework.workspace` — এক ধাপ
GradeBand	`scale.workspace` — এক ধাপ
Plan · User	প্ল্যাটফর্ম-স্তরের, স্কোপ নেই
Workspace	নিজেই স্কোপ

ঝুঁকি: `Mark.objects.all()` লিখলে প্রতিষ্ঠান আলাদা হয় না — join করে ফিল্টার করতেই হবে।

তাই IMS-এ `scoping.py` নামের একটাই জায়গা — প্রতিটি তালিকা-ভিউ নিজে ফিল্টার লেখে না। স্কোপ ও কোয়েরি ক্লাস ১৫ ও পর্ব ৪-এ।

শিক্ষকের নোট: বলুন: "কলাম না থাকলে নিরাপত্তা হারায় না — কিন্তু ভুলে যাওয়ার সুযোগ বেড়ে যায়।" উত্তরাধিকার-স্কোপ মানে প্রতিটি কুয়েরিতে ভাবতে হবে।

ধাপ ১০

ERD থেকে কোডে — তীরগুলো কোথায় থাকে

`related_name` ছাড়া উল্টো দিকের কুয়েরি লেখা কঠিন

```
class Enrollment(TimeStamped):
    workspace = models.ForeignKey(
        Workspace, on_delete=models.CASCADE,
        related_name="enrollments")
    student = models.ForeignKey(
        StudentProfile, on_delete=models.CASCADE,
        related_name="enrollments")
    class_group = models.ForeignKey(
        ClassGroup, on_delete=models.CASCADE,
        related_name="enrollments")
    is_active = models.BooleanField(default=True)

# ERD-র 1:N তীর মানে কোডে:
#   এক ছাত্র  →  student.enrollments.all()
#   এক শ্রেণি  →  cls.enrollments.filter(is_active=True)
```

তীর = ForeignKey

সন্তান-টেবিলে পিতার id থাকে। ERD-র "N" দিকের টেবিলে কলাম বসে।

উল্টো দিক = related_name

নাহলে `enrollment_set` — পড়তে কষ্ট, খুঁজতেও।

তিনটি FK = একই টেবিলে

Enrollment-এ workspace, student, class — তিনটাই দরকার, তিনটাই আছে।

শিক্ষকের নোট: প্রশ্ন করুন: "workspace কলাম থাকা সত্ত্বেও student আর class_group কেন দরকার?" উত্তর: এক ছাত্র একবার এক শ্রেণিতে — ternary relationship.

সতর্কতা

ERD-তে পাঁচটি সাধারণ ভুল

প্রতিটির প্রতিকার একটি বাক্যে

ভুল	পরিণতি ও প্রতিকার
১. ইতিহাস মুছে ফেলা — ছাত্রের টেবিলে একটাই `class_id`	শ্রেণি বদলালে গত বছরের রেকর্ড হারায় → Enrollment টেবিল
২. কার্ডিনালিটি না লেখা	ছবি পড়া যায় না — "এক শ্রেণিতে কতজন?" প্রশ্নের উত্তর ছবিতেই থাকা দরকার
৩. ঘটনাকে M:N ভাবা	উপস্থিতি/নম্বর/জমা = ঘটনা, নিজের টেবিল ও নিজের ফিল্ড চায়
৪. যৌগিক unique ভুলে যাওয়া	একই পরীক্ষায় দুইবার নম্বর → `uniq_mark_exam_student`
৫. স্কোপ কলাম ছাড়া টেবিল যোগ করা	নতুন টেবিলে `workspace` না দিলে এক প্রতিষ্ঠানের ডেটা আরেকটিতে দেখা যাবে

পঞ্চম ভুলটাই সবচেয়ে বিপজ্জনক — কারণ এটি এরর দেয় না, **চুপচাপ ভুল ডেটা দেখায়**।

শিক্ষকের নোট: শেষ লাইনটা বলুন ধীরে: "নিরাপত্তার ভুল কখনো চিৎকার করে না।" তারপর ছাত্রদের নিজের প্রজেক্টে নতুন টেবিলে workspace আছে কি না দেখতে বলুন।

অনুশীলন

২২ মিনিট — নিজের সিস্টেমের ERD

IMS-এর কাঠামো নকল নয়, পদ্ধতি কপি করুন

১. টেবিলের তালিকা (৫ মিনিট)

নিজের সব মডেল লিখুন — পাশে কোনটি "ঘটনা" (উপস্থিতি, পেমেন্ট, জমা) আর কোনটি "তথ্য" (ছাত্র, শ্রেণি)।

২. সম্পর্ক আঁকুন (৮ মিনিট)

প্রতিটি রেখায় 1:N / M:N / 1:1 লিখুন। যে রেখায় সংখ্যা লিখতে পারেন না — সেখানে বোঝাপড়া এখনো হয়নি।

৩. পিতৃ-সন্তান যাচাই (৫ মিনিট)

প্রতিটি N-দিকের টেবিলে পিতার FK কলাম আছে? উল্টো দিকের `related_name` লেখা?

৪. ইতিহাস পরীক্ষা (৪ মিনিট)

একটি রেকর্ড বদলালে পুরোনো অবস্থা হারায় কি? হারালে মাঝের টেবিল বা `is_active` দরকার।

যাচাই: আপনার ERD দেখে কেউ বলতে পারবে "এই ছাত্র গত বছর কোন শ্রেণিতে ছিল"? না পারলে উত্তরটা কোথায় থাকবে ঠিক করুন।

শিক্ষকের নোট: চতুর্থ ধাপে ছাত্ররা প্রথমবার বুঝবে কেন IMS-এ `is_active` আছে — মুছে ফেলা মানে ইতিহাস হারানো।

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

উত্তর এক বাক্যে

- শিক্ষার্থীর শ্রেণি কেন শিক্ষার্থী-টেবিলে কলাম হিসেবে থাকতে পারে না?
- Membership না থাকলে একজন শিক্ষক দুই প্রতিষ্ঠানে কাজ করতে পারতেন না — কেন?
- কোন তথ্য 1:1 টেবিলে রাখা উচিত, আর কোনটা মূল টেবিলেই?
- একই পরীক্ষায় একই ছাত্রের দুইটি নম্বর ঢুকে গেলে সমস্যাটা কোথায় ধরা পড়বে — কোডে, নাকি ডেটাবেসে?

শিক্ষকের নোট: চতুর্থ প্রশ্নের সেরা উত্তর: "ডেটাবেসে — কারণ কোড ভুলে যেতে পারে, constraint ভোলে না।" এটাই ক্লাস ১৫-এর সেতু।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
ঘটনা আর তথ্য আলাদা টেবিলে	Enrollment · AttendanceRecord · Mark · Submission
ভূমিকা User-এ নয়, Membership-এ	এক ব্যবহারকারী বহু প্রতিষ্ঠানে, ভিন্ন ভূমিকায়
1:1 কেবল ঐচ্ছিক বা ভিন্ন দায়িত্বের তথ্যে	Subscription · Policy · Branding · StudentProfile.account
M:N-এ সম্পর্কের নিজের গুণ থাকলে নিজের টেবিল	একটাই Django M:N (LessonLog ⇄ Topic)
PK অদৃশ্য, মানুষের নম্বরটি unique কলাম	id + `(workspace, admission_no)`
নতুন টেবিলে স্কোপ কলাম না থাকলে বিপদ	২৩টি সরাসরি · ৮টি উত্তরাধিকারে

শিক্ষকের নোট: প্রথম ও ষষ্ঠ সারি বোর্ডে আলাদা করে লিখুন — এ দুটোই ছাত্রদের প্রজেক্টে সবচেয়ে বেশি ভুল হয়।

১৫

পরের ক্লাস — "স্কিমা মাটিতে নামে"

partial unique index · workspace কলাম কোথায় নেই · cascade নিয়ম · ১২ মাইগ্রেশনের ক্রম

পর্ব ৩ · ডিজাইন · ক্লাস ১৫ / ৩৭ · শেষ ক্লাস

স্কিমা মাটিতে নামে

সিলেবাস: প্র্যাকটিক্যাল ৪ · টপিক ৪.৩ (ডেটাবেস ডিজাইন)
১২ মাইগ্রেশন · partial unique · cascade · ইনডেক্স মানচিত্র

PostgreSQL · ১২ মাইগ্রেশন · ২৩২ ফিল্ড

শুরু করুন প্রশ্ন দিয়ে

ডেটাবেসে ছবি আর ডেটা — কোথায় থাকে?

হাত তুলুন: ডেটাবেসে? ফোল্ডারে? দুই জায়গায়?

স্কিমা মানে শুধু টেবিল নয় — নিয়ম, ইনডেক্স, ফাইল আর ব্যাকআপের ঠিকানা। ভুল হলে ডেটা হারায়, ধীরে চলে, বা চুপচাপ দ্বিগুণ হয়।

আজকের ক্লাসে ৩১টি টেবিলকে **মাটিতে নামানো** হয়: ১২টি মাইগ্রেশন, কোথায় partial unique লাগে, কোথায় ডিলিট আটকাতে হয়, আর কোন কুয়েরির জন্য কোন ইনডেক্স।

শিক্ষকের নোট: উত্তর সাধারণত "ডেটাবেসে" আসে। বলুন: "লোগো ফাইলটা ডেটাবেসে নেই — ডেটাবেসে আছে তার নাম। ফাইল আর নাম আলাদা জায়গায় থাকলে কী হয়, আজ শেষে দেখবেন।"

ধাপ ১

১২টি মাইগ্রেশন — ক্রমটাই ইতিহাস

৬টি অ্যাপে ১টি করে, ৩টি অ্যাপে ২টি করে

মাইগ্রেশন	কী করে
accounts 0001	User (email-ভিত্তিক লগইন)
workspaces 0001	Workspace · Membership
workspaces 0002	**WorkspaceBranding** — পরে যোগ
academics 0001	বছর · টার্ম · বিভাগ · শ্রেণি · বিষয় · বণ্টন
academics 0002	**CoordinatorScope** — পরে যোগ
students 0001	StudentProfile · Enrollment
attendance 0001	AttendanceRecord · Policy
lessons 0001	৭টি টেবিল — সবচেয়ে বড়
assessments 0001	GradeScale · Band · Exam · Mark
audit 0001	AuditLog + ২টি ইনডেক্স
billing 0001	Plan · Subscription · Invoice · SignupRequest
billing 0002	**ডেটা** মাইগ্রেশন — প্ল্যান বপন + grandfather

মাইগ্রেশন ফাইল মুছবেন না, এডিটও করবেন না। নতুন পরিবর্তন = **নতুন নম্বর**।

একই নম্বর দুই জায়গায় থাকলে `migrate` থেমে যায় — ইতিহাস দ্বিধায় পড়ে।

কোডের ইতিহাস git-এ থাকে; ডেটাবেসের ইতিহাস মাইগ্রেশনে। দুটোই রাখতে হয় — একটার উপর ভরসা করলে অন্যটা হারায়।

শিক্ষকের নোট: বোর্ডে ৬+৬=১২ লিখুন। বলুন: "৩টি অ্যাপে ২টি করে মাইগ্রেশন — কারণ ওই তিনটে জিনিস (Branding, Scope, Plans) শুরুর স্কোপে ছিল না; পরে যোগ হয়েছে।"

ধাপ ২

একটি ডেটা মাইগ্রেশন — deploy যেন অর্ধেক না থাকে

`billing/0002_seed_plans_and_grandfather.py` — স্কিমা নয়, **সারি** তৈরি করে

```
def forwards(apps, schema_editor):
    Plan = apps.get_model("billing", "Plan")
    Subscription = apps.get_model("billing", "Subscription")
    Workspace = apps.get_model("workspaces", "Workspace")

    for order, (code, name, ...) in enumerate(PLANS, 1):
        Plan.objects.update_or_create(code=code, defaults={...})

    for ws in Workspace.objects.filter(kind="INSTITUTION"):
        if not Subscription.objects.filter(workspace=ws).exists():
            Subscription.objects.create(
                workspace=ws, plan=campus, status="ACTIVE",
                current_period_end=today + timedelta(days=365),
                notes="Grandfathered at launch...")

class Migration:
    dependencies = [("billing", "0001_initial"),
                    ("workspaces", "0002_workspacebranding")]
```

কেন মাইগ্রেশনে, স্ক্রিপ্টে নয়

প্রতিটি পরিবেশে একই ক্রমে একবারই চলে — কেউ ভুলে যেতে পারে না।

grandfather মানে কী

পুরোনো প্রতিষ্ঠানকে এক বছরের জন্য সক্রিয়, বিনামূল্যে — ব্যবসার সিদ্ধান্ত কোডে লেখা।

নির্ভরতা অন্য অ্যাপে

`workspaces 0002`-এর পরে — কারণ Branding আগে তৈরি হতে হবে।

নিয়ম: ডেটা মাইগ্রেশন **idempotent** বানান (`update_or_create` , `exists()` চেক) — দুইবার চালালেও ক্ষতি নয়।

শিক্ষকের নোট: প্রশ্ন করুন: "সাবস্ক্রিপশন ছাড়া প্রতিষ্ঠান থাকলে কী হয়?" উত্তর: চিরকাল ফ্রি — অর্থাৎ রাজস্ব হারায়। এজন্যই এটা মাইগ্রেশনে বাধ্যতামূলক করা হয়েছে।

ধাপ ৩

partial unique — "একটাই সক্রিয়" নিয়মটা ডেটাবেসে

শর্তসহ unique index · PostgreSQL-এ সত্যিই partial index

```
constraints = [
  models.UniqueConstraint(
    fields=["workspace"],
    condition=Q(is_active=True),
    name="uniq_active_term_per_workspace"),
]

constraints = [
  models.UniqueConstraint(
    fields=["student", "class_group"],
    condition=models.Q(is_active=True),
    name="uniq_active_enrollment_student_class"),
]

constraints = [
  models.UniqueConstraint(
    fields=["custom_domain"],
    condition=~models.Q(custom_domain=""),
    name="unique_custom_domain"),
]
```

নিয়ম	নাহলে কী হতো
এক workspace-এ একটাই সক্রিয় টার্ম	দুই টার্মে উপস্থিতি — ড্যাশবোর্ড দ্বিগুণ
এক ছাত্র এক শ্রেণিতে একবার সক্রিয়	ভর্তি দুইবার → দুইবার হাজিরা
এক ডোমেইন এক প্রতিষ্ঠানের	দুই স্কুল একই ডোমেইনে → কে জিতবে?

partial বলতে বোঝায়: শুধু শর্ত মানা সারিগুলোর মধ্যে ইউনিক। পুরোনো (নিষ্ক্রিয়) সারি যত চান থাকুক।

SQLite-ও partial index বোঝে — তাই `run.sh`-এর ডেমো DB-তেও নিয়মটা কাজ করে।

শিক্ষকের নোট: বলুন: "কোডে `if active: ...` লিখে আটকানো যায়, কিন্তু দুজন একসাথে সেভ করলে দুইটাই ঢুকে যায়। ডেটাবেস আটকালে সেটা অসম্ভব।" — race condition-এর প্রথম পাঠ।

ধাপ ৪

cascade নিয়ম — ডিলিট করলে কী হয়

৭৫ সম্পর্কে তিনটি পছন্দ: CASCADE ৫১ · SET_NULL ১৭ · PROTECT ৬

CASCADE · ৫১

পিতা গেলে সন্তানও যায়

Workspace মুছলে তার শ্রেণি, ছাত্র, উপস্থিতি, নম্বর — সব যায়

টেন্যান্ট মুছে ফেলা = ইচ্ছাকৃত ধ্বংস

SET_NULL · ১৭

পিতা গেলে সন্তান থাকে, রেফারেন্স ফাঁকা

যে শিক্ষক অ্যাকাউন্ট মুছলেন, তার দেওয়া উপস্থিতি থেকে যায়

recorded_by = NULL

PROTECT · ৬

ডিলিটই হতে দেয় না

টার্ম · প্ল্যান · ইনভয়েসের প্রতিষ্ঠান · প্রতিষ্ঠানের মালিক — আটকে দেয়

নথি কখনো অনাথ হয় না

নিয়ম: **অর্থ** থাকলে CASCADE · **ইতিহাস** মূল্যবান হলে SET_NULL · **নথি** হলে PROTECT। একটি শব্দে: "এই সারিটা না থাকলে অন্যটার মানে থাকে কি?"

শিক্ষকের নোট: উদাহরণ দিন: "সাবস্ক্রিপশনে ব্যবহৃত প্ল্যান মুছলে আগের ইনভয়েসের দাম কী হবে? তাই PROTECT — প্ল্যান নিষ্ক্রিয় করুন, মুছবেন না।"

ধাপ ৫

PROTECT-এর ছয়টি জায়গা — কেন এগুলোই

প্রতিটির পেছনে একটি করে ব্যবসার নিয়ম

সম্পর্ক	যে নিয়মটি রক্ষা করে
ClassGroup.term	টার্ম মুছলে শ্রেণি অনাথ হয়ে যেত — উপস্থিতি কার বিরুদ্ধে?
Exam.term	ফলাফল কোন টার্মের, তা না থাকলে রিপোর্ট কার্ড অর্থহীন
Subscription.plan	চালু চুক্তির প্ল্যান মুছে ফেলা যাবে না — নিষ্ক্রিয় করুন
Invoice.workspace	আর্থিক নথি কখনো প্রতিষ্ঠানহীন হয় না
Invoice.plan	পুরোনো ইনভয়েসে দাম ও প্ল্যান অপরিবর্তিত থাকে
Workspace.owner	মালিক অ্যাকাউন্ট মুছলে প্রতিষ্ঠান ঝুলে থাকত

ভুল সমাধান: PROTECT এড়াতে কোডে `try/except` — নিয়মটা তখন অকার্যকর।

ঠিক সমাধান: `is_active=False` — রেকর্ড থাকে, তালিকা থেকে সরে যায়।

শিক্ষকের নোট: "মুছুন" আর "নিষ্ক্রিয় করুন" — দুটো আলাদা কাজ। বলুন: "শিক্ষক চলে গেলেন, তালিকা থেকে সরান — কিন্তু তাঁর দেওয়া নম্বর থাকবে।"

ধাপ ৬

ইনডেক্স মানচিত্র — ১৩টি নিজের, ২টি কলামে

প্রতিটি ইনডেক্স একটি নির্দিষ্ট কুয়েরির জন্য — অকারণ ইনডেক্স লিখন ধীর করে

ইনডেক্স	কোন পর্দা
`[ws, date]` · attendance	আজকের উপস্থিতি
`[class_group, date]` · attendance	শ্রেণির রেজিস্টার
`[ws, class_group, due_at]`	খোলা হোমওয়ার্ক
`[ws, term, is_published]`	অপ্রকাশিত পরীক্ষা
`[ws, term, is_archived]`	চলতি শ্রেণি তালিকা
`[ws, status]` · students	শিক্ষার্থী তালিকা
`[ws, class_group, is_active]`	সক্রিয় ভর্তি
`[ws, role, is_active]`	সদস্য তালিকা
`[user, is_active]`	আমার প্রতিষ্ঠানসমূহ
`[ws, -created_at]` · `[action]`	অডিট লগ
`[ws, user, is_active]`	কোঅর্ডিনেটর স্কোপ
`[kind, is_active]`	প্রতিষ্ঠান তালিকা

প্রায় সব ইনডেক্স `workspace` দিয়ে শুরু (`ws`) — কারণ প্রতিটি কুয়েরিতেই প্রথম শর্ত প্রতিষ্ঠান।

db_index ২টি: `User.email` (লগইন) · `WorkspaceBranding.custom_domain` (প্রতিটি অনুরোধে হোস্ট দেখে)।

ForeignKey কলামে ইনডেক্স Django নিজেই দেয় — সেটি এই ১৩টির হিসাবে নেই।

শিক্ষকের নোট: বলুন: "ইনডেক্স হলো বইয়ের সূচি। প্রতিটি অধ্যায়ে সূচি রাখলে বই ভারী হয় — তাই যেখানে খুঁজবেন, শুধু সেখানে।"

ধাপ ৭

ইনডেক্স ছাড়া কী হয় — একটি সংখ্যা দিয়ে বোঝা

একই কুয়েরি, দুই স্কিমা: বাড়ির কাজ হিসেবে চালান

```
-- কোন ইনডেক্স ব্যবহার হলো?
EXPLAIN ANALYZE
SELECT * FROM attendance_attendancerecord
WHERE workspace_id = 1 AND date = '2026-11-05';

-- ইনডেক্স থাকলে:
--   Index Scan using attendance_workspace_date_idx
--   Planning: 0.2 ms   Execution: 0.4 ms

-- ইনডেক্স ছাড়া (বড় টেবিলে):
--   Seq Scan on attendance_attendancerecord
--   Rows Removed by Filter: 480000
--   Execution: 180 ms
```

১. EXPLAIN পড়ুন

Seq Scan মানে পুরো টেবিল পড়ছে — বড় টেবিলে সেটাই সমস্যা।

২. কলামের ক্রম মানে

`[workspace, date]` — প্রথমে workspace না দিলে ইনডেক্স কাজে লাগে না।

৩. আগে মাপুন, পরে যোগ

অনুমানে ইনডেক্স দিলে লেখা ধীর হয়, লাভ শূন্য।

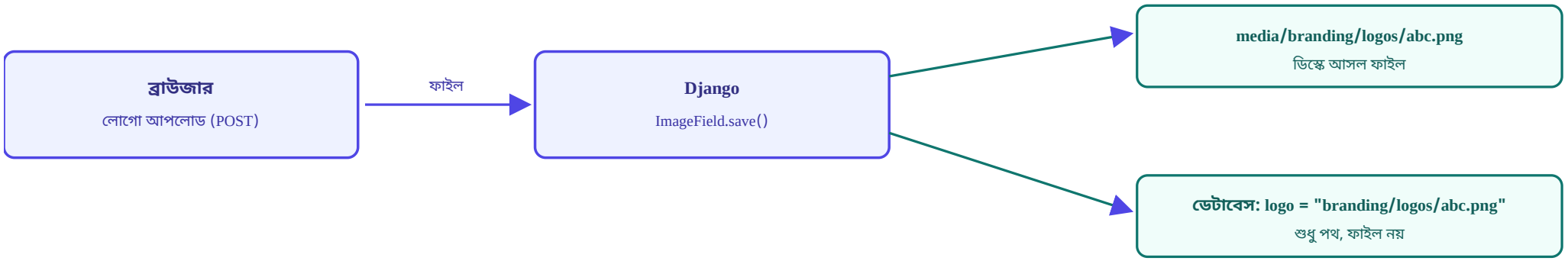
নিয়ম: ইনডেক্স ঠিক করে **পড়া** দ্রুত হয়, কিন্তু প্রতিটি **লেখা** একটু ধীর হয়। তাই প্রতিটি ইনডেক্সের জবাবদিহি থাকা দরকার।

শিক্ষকের নোট: সংখ্যাগুলো উদাহরণ — বলুন স্পষ্ট করে: "এগুলো আপনার ডেটার সংখ্যা নয়, তুলনার জন্য। নিজের টেবিলে EXPLAIN চালিয়ে আসল সংখ্যা দেখুন।"

ধাপ ৮

ফাইল কোথায় থাকে — ডেটাবেসে শুধু নাম

`MEDIA_ROOT` = `BASE_DIR / "media"` · লোগো যায় `branding/logos/`-এ



পরিণতি: ডেটাবেস ব্যাকআপ নিলে ছবি আসে না — media ফোল্ডার আলাদা ব্যাকআপ করতে হয়।

৪টি সেটিংস ফাইল: base (সব) · dev · prod · sqlite (ডেমো, `run.sh`)।

prod-এ ডেটাবেস: PostgreSQL · `CONN_MAX_AGE=60` · media = FileSystemStorage।

শিক্ষকের নোট: প্রশ্ন করুন: "সার্ভার বদলালে ছবি হারাবে কেন?" উত্তর: ডেটাবেস কপি হয়েছে, media ফোল্ডার নয়। এটাই সবচেয়ে সাধারণ ডিপ্লয়-দুর্ঘটনা।

ধাপ ৯

দুই ডেটাবেস, এক স্কিমা

ডেমো SQLite-এ, আসল PostgreSQL-এ — কিন্তু কোড এক, মাইগ্রেশন এক

বিষয়	sqlite (ডেমো)	postgresql (আসল)
কে ব্যবহার করে	`bash run.sh` — নতুন শিক্ষার্থী	প্রতিষ্ঠানের সার্ভার
সেটিংস	`config.settings.sqlite`	`config.settings.prod`
partial unique	কাজ করে (partial index)	কাজ করে
একসাথে লেখা	এক লেখক — সহজ	অনেক লেখক — লক ও transaction দরকার
ব্যাকআপ	একটি ফাইল কপি	`pg_dump` + media ফোল্ডার

শিক্ষার নিয়ম: SQLite দিয়ে শুরু করুন, কিন্তু **নিয়মগুলো PostgreSQL-এর মতোই লিখুন** — constraint, index, transaction। পরে ডেটাবেস বদলালে কোড বদলাতে হবে না।

শিক্ষকের নোট: বলুন: "SQLite খারাপ নয়, ছোট কাজে সেরা। কিন্তু যেখানে একসাথে ৩০ জন শিক্ষক লিখবেন, সেখানে দরকার সার্ভার-ডেটাবেস।"

ধাপ ১০

মাইগ্রেশন-দিনের চেকলিস্ট

স্কিমা বদলানোর আগে-পরে যে পাঁচটি কাজ ভুললে বিপদ

#	কাজ	কেন
১	ব্যাকআপ (`pg_dump` বা ফাইল কপি)	মাইগ্রেশন ফেরানো কষ্টকর, ব্যাকআপ সহজ
২	`makemigrations` আউটপুট পড়ুন	স্বয়ংক্রিয় ফাইলও ভুল হতে পারে — কলাম মুছে ফেলা সহ
৩	নতুন কলাম `null=True` বা default দিন	পুরোনো সারিতে মান ছাড়া কলাম যোগ করা যায় না
৪	ডেটা মাইগ্রেশন আলাদা ফাইলে	স্কিমা আর ডেটা একসাথে গুলিয়ে গেলে ফেরানো কঠিন
৫	মাইগ্রেশনের পরে `103` টেস্ট চালান	স্কিমা বদলালে লুকানো ভাঙন ধরা পড়ে

সবচেয়ে বিপজ্জনক: **কলামের নাম বদলানো** — Django পুরোনো কলাম ফেলে নতুন বানায়, ডেটা রক্ষা পায় না। সমাধান: নতুন কলাম যোগ → ডেটা কপি → পুরোনো কলাম বাদ (তিন ধাপ)।

শিক্ষকের নোট: তৃতীয় সারিটি বোর্ডে লিখুন। বলুন: "প্রোডাকশনে ৫০০০ সারি আছে, নতুন কলাম NOT NULL — migrate মুহূর্তেই থেমে যাবে।"

অনুশীলন

২০ মিনিট — নিজের স্কিমার নিয়ম লিখুন

কোড নয়, সিদ্ধান্ত — তারপর কোড

১. three "একটাই সক্রিয়" নিয়ম খুঁজুন (৫ মিনিট)

আপনার সিস্টেমে কোথায় "একসাথে দুইটি চলতে পারে না"? প্রতিটির জন্য partial unique লিখুন।

২. cascade সিদ্ধান্ত (৫ মিনিট)

প্রতিটি FK-এর পাশে লিখুন: CASCADE / SET_NULL / PROTECT — এবং এক শব্দে কারণ।

৩. ইনডেক্স তালিকা (৬ মিনিট)

আপনার ৫টি সবচেয়ে বড় তালিকা-পর্দা লিখুন; প্রতিটির WHERE-এর কলামগুলো ক্রমে সাজান।

৪. ব্যাকআপ বাক্য (৪ মিনিট)

এক বাক্যে লিখুন: "আমার সিস্টেমের ব্যাকআপ মানে ___।" media ফোল্ডার ভুলে যাবেন না।

যাচাই: আপনার স্কিমায় এমন কোনো নিয়ম আছে কি যা শুধু কোডে লেখা, ডেটাবেসে নয়? থাকলে সেটাই আজকের কাজ।

শিক্ষকের নোট: শেষ প্রশ্নটাই মূল: "কোডে লেখা নিয়ম ভুলে যাওয়া যায়, ডেটাবেসে লেখা নিয়ম ভোলে না।"

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

পর্ব ৩-এর শেষ আলোচনা

- partial unique কেন সাধারণ unique-এর চেয়ে ভালো, যখন ইতিহাস রাখতে হয়?
- শিক্ষক অ্যাকাউন্ট মুছলে তাঁর দেওয়া নম্বর থেকে যাওয়া উচিত — কোন on_delete সেটি করে?
- প্রতিটি ইনডেক্স `workspace` দিয়ে শুরু কেন?
- ডেটা মাইগ্রেশন আর স্কিমা মাইগ্রেশন আলাদা না রাখলে কী কী সমস্যা হয়?

শিক্ষকের নোট: দ্বিতীয় প্রশ্নের উত্তর SET_NULL — এবং এর পরের প্রশ্ন করুন: "তাহলে 'কে দিয়েছিল' জানা যাবে না?" উত্তর: অডিট লগে থাকে। তিনটি স্তর মিলেই সত্য।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
মাইগ্রেশন = ডেটাবেসের git ইতিহাস	১২টি ফাইল · ৩টি অ্যাপে দ্বিতীয় মাইগ্রেশন
নিয়ম ডেটাবেসে রাখুন, কোডে নয়	৪টি partial unique index
ডিলিটের তিন পছন্দ, কারণসহ	CASCADE ৫১ · SET_NULL ১৭ · PROTECT ৬
ইনডেক্স প্রতি কুয়েরির জবাবদিহি	১৩টি Meta index · ২টি db_index · সবই workspace-first
ফাইল ডিস্কে, পথ ডেটাবেসে	`media/` আলাদা ব্যাকআপ লাগে
এক স্কিমা, দুই ডেটাবেস	sqlite (ডেমো) · postgresql (আসল)

শিক্ষকের নোট: পর্ব ৩ শেষ। বলুন: "ডিজাইন শেষ মানে কোড লেখা শুরু নয় — ডিজাইন শেষ মানে এখন কোড লেখার সিদ্ধান্তগুলো আর বদলাবে না।"

৪

পর্ব ৪ — "কোড লেখা শুরু"

ক্লাস ১৬: মডেল লেখা — ৩১টি টেবিল Django-তে · ফিল্ডের ধরন · migration-এর অভ্যাস

পর্ব ৪ · কোডিং · ক্লাস ১৬ / ৩৭

এক ফাইলেই পুরো নিয়ম

সিলেবাস: প্র্যাকটিক্যাল ৫ · টপিক ৫.১ (মডেল ও স্কিমা বাস্তবায়ন)

৩১ মডেল · ২৩২ ফিল্ড · ১৪টি clean() · ৩টি save()

Django model = টেবিল + নিয়ম + আচরণ

শুরু করুন প্রশ্ন দিয়ে

একটি মডেল ফাইলে কি শুধু কলাম থাকে?

হাত তুলুন: শুধু কলাম? না হয় আরও কিছু?

IMS-এর ৩১টি মডেলে ২৩২টি ফিল্ড, ১৪টি clean(), ৩টি save(), ১৪টি @property — সব একই ফাইলে।

কারণ মডেল মানে শুধু টেবিল নয়: **নিয়ম** (কী বৈধ) আর **আচরণ** (এই সারি দিয়ে কী করা যায়) — দুটোই ডেটার সঙ্গেই থাকা উচিত, view-এ ছড়িয়ে নয়।

শিক্ষকের নোট: প্রশ্নটা করুন, তারপর বলুন: "আজ আপনি দেখবেন একই প্রশ্নের উত্তর তিন জায়গায় দেওয়া যায় — ফর্মে, ভিউতে, আর মডেলে। শেষটা কেন সেরা, সেটাই আজকের গল্প।"

ধাপ ১

এক মডেলের তিনটি স্তর

কলাম · নিয়ম · আচরণ — তিনটাই এক ফাইলে, তিনটাই একসাথে যায়

১. কলাম (Fields)

```
name = CharField(max_length=40)
starts_on = DateField()
is_closed = BooleanField()
```

ডেটাবেসে যা দেখবেন

২. নিয়ম (Meta, clean)

```
unique_together = [(workspace, name)]
ordering = ["-starts_on"]
clean(): ends_on > starts_on
```

ভুল ডেটা ঢুকতেই পারে না

৩. আচরণ (Methods)

```
def __str__(self)
@property def is_writable(self)
def save(self, *args, **kwargs)
```

কোডে "লজিক" যেখানে খুঁজে পাবেন

তিনটি স্তর এক ফাইলে থাকলে টেবিলের সাথে তার নিয়ম কখনো বিচ্ছিন্ন হয় না — এটাই Django-র মূল ধারণা।

বিকল্প কি ছিল? নিয়মগুলো ভিউতে `if` দিয়ে লেখা যেত — কিন্তু তখন শেল, অ্যাডমিন, ম্যানেজমেন্ট কমান্ড, ইমপোর্ট — সবাই সেই নিয়ম ভাঙতে পারত।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন: "কোন জায়গা থেকে ডেটা আসতে পারে?" (ফর্ম, অ্যাডমিন, শেল, স্ক্রিপ্ট, টেস্ট) — প্রতিটির জন্য আলাদা করে নিয়ম লিখতে হলে একদিনে একটি ভুল হবেই।

ধাপ ২

একটি পূর্ণ মডেল, লাইন ধরে ধরে

`apps/academics/models.py` — AcademicYear, ১৭ লাইন

```
class AcademicYear(TimeStamped):
    workspace = models.ForeignKey(
        Workspace, on_delete=models.CASCADE,
        related_name="academic_years")
    name = models.CharField(max_length=40,
        help_text="e.g. 2026 or 2026-2027")
    starts_on = models.DateField()
    ends_on = models.DateField()
    is_closed = models.BooleanField(default=False)

    class Meta:
        unique_together = [("workspace", "name")]
        ordering = ["-starts_on"]

    def __str__(self):
        return self.name

    def clean(self):
        if self.starts_on and self.ends_on \
                and self.ends_on <= self.starts_on:
            raise ValidationError({"ends_on":
                "End date must be after the start date."})
```

লাইন	সিদ্ধান্ত
`TimeStamped`	created_at · updated_at বিনামূল্যে
workspace FK	স্কোপ কলাম — ক্লাস ১৪
`help_text`	ফর্মেই ব্যাখ্যা দেখাবে
দুটি DateField	তারিখ, সময় নয় — কারণ দিনই দরকার
`unique_together`	এক প্রতিষ্ঠানে এক নাম
`ordering`	নতুন বছর আগে — `-` মানে উল্টো
`__str__`	ড্রপডাউনে নাম দেখাবে
`clean()`	শেষ তারিখ শুরুর পরে

শিক্ষকের নোট: ধীরে ধীরে লাইন ধরে পড়ুন। প্রতিটি লাইনে জিজ্ঞেস করুন: "এটা না থাকলে কী হতো?" — বিশেষ করে `ordering` আর `help_text`, যা ছাত্ররা প্রায়ই বাদ দেয়।

ধাপ ৩

২৩২টি ফিল্ডের ভাঙা হিসাব

৭৫টি সম্পর্ক + ১৫৭টি সাধারণ কলাম — কোনটা কতবার

ধরন	সংখ্যা	কোথায়
`ForeignKey`	৭০	সম্পর্ক
`CharField`	৬২	নাম · শিরোনাম · কোড
`BooleanField`	২৩	is_active · is_published
`DateField`	১৮	তারিখ (দিন)
`DateTimeField`	১৬	created_at · submitted_at
`PositiveIntegerField`	১২	full_marks · seats
`TextField`	৯	বিবরণ · নোট · ফিচার
`DecimalField`	৬	**টাকা** · শতাংশ
বাকি ৬ ধরন	১৪	Email · Slug · IP · File · Image · SmallInt

১৩৯টি ফিল্ড শুধু CharField আর ForeignKey — কারণ বেশিরভাগ ব্যবসা "নাম + সম্পর্ক" দিয়েই চলে।

টাকা সবসময় Decimal — `FloatField` শূন্যটি জায়গায় ভুল হিসাব দেয়।

লক্ষ্য করুন: `FloatField` **একটিও নেই**। সংখ্যা দরকার হলে Integer, টাকা দরকার হলে Decimal — এই দুইয়ের বাইরে ভাবনার দরকার নেই।

শিক্ষকের নোট: Float-এর ফাঁদটা বোর্ডে দেখান: ০.১ + ০.২ = ০.৩০০০০০০০০০০০০০০০৪। বলুন: "টাকার হিসাবে এই ভুল জমে জমে বড় হয়।"

ধাপ ৪

ফিল্ড বাছাই — পাঁচটি সিদ্ধান্ত

প্রতিটি পছন্দের পেছনে একটি প্রশ্ন

প্রশ্ন	বেছে নিন	IMS-এর উদাহরণ
দিন দরকার, নাকি দিন+সময়?	`DateField` / `DateTimeField`	পরীক্ষার তারিখ = Date · জমা দেওয়ার সময় = DateTime
সংখ্যা, নাকি টাকা?	`PositiveIntegerField` / `DecimalField`	পূর্ণ নম্বর ১০০ · ফি ৳১,৫০০.০০
ছোট নাম, নাকি বড় লেখা?	`CharField(max_length)` / `TextField`	শ্রেণির নাম · হোমওয়ার্কের বিবরণ
নির্বাচিত মানগুলোর বাইরে যাবে?	`choices` (TextChoices)	উপস্থিতি: PRESENT · ABSENT · LATE · EXCUSED
ফাইল?	`FileField` / `ImageField`	উপকরণ · প্রতিষ্ঠানের লোগো

max_length বাড়িয়ে দিন যা একদিন বাড়তে পারে — নাম, ব্যাখ্যা।

max_length ঠিক রাখুন যা কখনো বাড়ে না — হেক্স কালার ৭, মুদ্রা ৩, ফোন ১৬।

শিক্ষকের নোট: শেষ দুটি কার্ডের পার্থক্যটা প্রশ্ন করান: "শ্রেণির নাম ৪০ না ২০০?" উত্তর: বাড়তে পারে, তাই বড় রাখুন — কিন্তু TextField নয়, কারণ তালিকায় দেখাতে হবে।

ধাপ ৫

TimeStamped — ১৯টি মডেলের সাধারণ ভিত্তি

`abstract = True` — নিজের টেবিল হয় না, শুধু কলাম ধার দেয়

```
class TimeStamped(models.Model):
    created_at = models.DateTimeField(
        auto_now_add=True)
    updated_at = models.DateTimeField(
        auto_now=True)

    class Meta:
        abstract = True


class AcademicYear(TimeStamped):
    ...
# ১৯টি মডেল এভাবেই শুরু হয়
```

auto_now_add = একবার

তৈরির সময় লেখে, পরে আর বদলায় না — ইতিহাসের সূচনা।

auto_now = প্রতিবার

প্রতিটি `save()`-এ হালনাগাদ — "সর্বশেষ কে ছুঁল?"

abstract কেন

টেবিল না বানিয়ে ১৯ মডেলে কলাম যোগ করে — এক জায়গায় বদলালে সব বদলায়।

যে ১২টি মডেলে এটি নেই, তাদের নিজের সময়-কলাম আছে বা দরকার নেই — যেমন `AuditLog` (`created_at` একটাই), `Plan`, `User` (Django-র নিজস্ব `date_joined`)।

শিক্ষকের নোট: প্রশ্ন করুন: "abstract বাদ দিয়ে কপি-পেস্ট করলে?" উত্তর: ১৯ জায়গায় বদলাতে হবে — আর একদিন কেউ একটি বাদ দেবে।

ধাপ ৬

`__str__` — এক লাইন, বড় পার্থক্য

ড্রপডাউন, অ্যাডমিন, শেল, এরর মেসেজ — সব জায়গায় এটাই দেখায়

```
# Without __str__ — what you see in a dropdown
<AcademicYear: AcademicYear object (3)>
<AcademicYear: AcademicYear object (4)>

# With __str__
2026
2025-2026

# IMS-এর তিনটি ভালো উদাহরণ
def __str__(self):
    return f"{self.number} · {self.workspace.name}
            · {self.amount} {self.currency}"   # Invoice

def __str__(self):
    return f"{self.full_name} - {self.homework.title}"
                                              # Submission

def __str__(self):
    return f"{self.action} by {self.actor_id}"  # AuditLog
```

পরিচয় দেয়

কোন সারি — মানুষ চেনে নামে, ডেটাবেস চেনে id-তে।

ডিবাগ সহজ করে

`print(obj)` করলেই বোঝা যায় কোনটা।

নিয়ম

এক লাইনে, ৮০ অক্ষরের মধ্যে, আর **রহস্যময় তথ্য নয়** (পাসওয়ার্ড, টোকেন কখনো নয়)।

শিক্ষকের নোট: লাইভ দেখান — শেল খুলে একটি মডেল প্রিন্ট করুন যেখানে `__str__` নেই। ওই `<... object (3)>` দেখেই ছাত্ররা বুঝবে কেন এটা দরকার।

ধাপ ৭

clean() বনাম constraint — কে কোথায় থামায়

১৪টি `clean()` (পাইথন) · ২টি `CheckConstraint` (ডেটাবেস)

	clean() · ফর্ম-স্তর	CheckConstraint · ডেটাবেস
কোথায় চলে	`full_clean()`, ModelForm সেভের আগে	প্রতিটি INSERT/UPDATE-এ
ভালো কী করে	বন্ধুসুলভ বার্তা, কোন ফিল্ডে ভুল	কোনো পথেই ভুল ঢুকতে দেয় না
কে এড়াতে পারে	`save()` সরাসরি ডাকলে	কেউ না — এমনকি শেলও না
IMS-এ উদাহরণ	শেষ তারিখ শুরুর পরে · অন্য প্রতিষ্ঠানের ছাত্র নয়	`grade_band_min_le_max` · `coordinator_scope_needs_a_target`

দুটোই রাখুন: clean দেয় ভালো এরর, constraint দেয় শেষ রক্ষা।

শুধু clean যথেষ্ট নয় — `Model.objects.create()` clean ডাকে না, সোজা সেভ করে।

শিক্ষকের নোট: দ্বিতীয় কার্ডটাই আসল ফাঁদ। বলুন: "আপনার ভ্যালিডেশন কাজ করছে — কারণ আপনি ফর্ম দিয়ে টেস্ট করেছেন। শেল দিয়ে টেস্ট করুন।"

ধাপ ৮

@property — ব্যবসার নিয়ম ডেটার ঘরে

১৪টি property; সেরাটি `Subscription.is_writable`

```
@property
def ends_on(self):
    return (self.trial_ends_on
            if self.status == self.Status.TRIAL
            else self.current_period_end)

@property
def days_left(self):
    end = self.ends_on
    return None if end is None \
        else (end - timezone.localdate()).days

@property
def is_writable(self) -> bool:
    """TRIAL আর ACTIVE পারেন। PAST_DUE পড়তে
    পারে, লিখতে পারে না। ডেটা কখনো মোছে না।"""
    if self.status in (self.Status.PAST_DUE,
                       self.Status.SUSPENDED,
                       self.Status.CANCELLED):
        return False
    if self.status == self.Status.TRIAL:
        return self.trial_ends_on is None \
            or self.trial_ends_on >= timezone.localdate()
    return True
```

তিন জায়গায় এক নিয়ম

মিডলওয়্যার, ব্যানার, রিপোর্ট — সবাই `is_writable` পড়ে। কোথাও আলাদা করে লেখা নেই।

নিয়ম বদলালে এক জায়গায়

"ট্রায়াল শেষ = read-only" — এই সিদ্ধান্ত একটাই ফাংশনে।

টেস্ট সহজ

টেস্টে আলাদা রেকর্ড না বানিয়ে শুধু তারিখ বদলে property যাচাই করা যায়।

শিক্ষকের নোট: প্রশ্ন করুন: "এই নিয়মটা তিন জায়গায় কপি করলে?" উত্তর: তিন মাস পরে এক জায়গায় বদলাবেন, দুই জায়গায় ভুল থেকে যাবে — আর ব্যবহারকারী বলবে "কখনো কাজ করে, কখনো করে না।"

ধাপ ৯

save() — মাত্র তিনটি জায়গায়, আর সবগুলোই হিসাব

Enrollment · Submission · Invoice — তিনটিই এমন কিছু গণনা করে যা কেউ ভুলে যেতে পারে

```python
# ১. Invoice — নম্বর কখনো ফাঁকা থাকে না
def save(self, *args, **kwargs):
    if not self.number:
        self.number = self._next_number()
    super().save(*args, **kwargs)

@staticmethod
def _next_number() -> str:
    year = timezone.localdate().year
    prefix = f"{settings.INVOICE_PREFIX}-{year}-"
    last = (Invoice.objects
            .filter(number__startswith=prefix)
            .order_by("-number")
            .values_list("number", flat=True).first())
    n = int(last.split("-")[-1]) + 1 if last else 1
    return f"{prefix}{n:04d}"

# ২. Submission — দেরিতে জমা স্বয়ংক্রিয়
def save(self, *args, **kwargs):
    if self.homework_id and not self.pk:
        self.is_late = timezone.now() > self.homework.due_at
    super().save(*args, **kwargs)
```

নম্বর কেন এখানে

ইনভয়েস কোথা থেকেই তৈরি হোক — নম্বর থাকবেই। `IMS-2026-0001`

কেন `not self.pk`

শুধু প্রথম সেভে — পরে সম্পাদনা করলে `is_late` বদলাবে না।

Enrollment — তৃতীয়টি

`is_active=False` হলে `ended_on` নিজেই বসে যায়।

ফাঁদ: দুই অনুরোধ একসাথে এলে একই নম্বর আসতে পারে — তাই unique + চাপে `Invoice.number`।

শিক্ষকের নোট: সততার সাথে বলুন: "এই নম্বর-বানানোর পদ্ধতি সাধারণ চাপে ঠিক, ভিড়ে দুর্বল। বড় সিস্টেমে ডেটাবেস sequence লাগে।" — সীমাবদ্ধতা জানাই ভালো engineering.

ধাপ ১০

Manager — ব্যবহারকারী তৈরির একমাত্র দরজা

`apps/accounts/managers.py` — কেউ যেন প্লেইন পাসওয়ার্ড লিখতে না পারে

```
class UserManager(BaseUserManager):
    use_in_migrations = True

    def _create_user(self, email, password, **extra):
        if not email:
            raise ValueError("An e-mail address is required.")
        email = self.normalize_email(email).lower()
        user = self.model(email=email, **extra)
        user.set_password(password)   # hashes, never stores plain
        user.save(using=self._db)
        return user

    def create_user(self, email, password=None, **extra):
        extra.setdefault("is_staff", False)
        extra.setdefault("is_superuser", False)
        return self._create_user(email, password, **extra)

    def create_superuser(...):
        ... is_staff=True, is_superuser=True
        return self._create_user(...)
```

ইমেইল নরমালাইজ

সব ছোট হাতে — `Admin@IMS.local` আর `admin@ims.local` একই মানুষ।

set_password

প্লেইন পাসওয়ার্ড কখনো ডেটাবেসে যায় না — হ্যাশ যায়।

কেন ম্যানেজার

`User(email=..., password="abcd")` লিখলে প্লেইন পাসওয়ার্ড সেভ হয়ে যেত — এই ভুলটা কেউ করতেই পারবে না।

শিক্ষকের নোট: বলুন: "ভালো API মানে ভুল কাজটা কঠিন করে তোলা। এখানে ভুলটা প্রায় অসম্ভব — কারণ সব দরজা এক।"

ধাপ ১১

choices — ১১টি জায়গায়, খালি স্ট্রিং নয়

`models.TextChoices`: মান আর লেবেল একসাথে

```
class AttendanceRecord(TimeStamped):
    class Status(models.TextChoices):
        PRESENT = "PRESENT", "Present"
        ABSENT = "ABSENT", "Absent"
        LATE = "LATE", "Late"
        LEAVE = "LEAVE", "Leave"

    status = models.CharField(
        max_length=10, choices=Status.choices,
        default=Status.PRESENT)

# কোডে:
record.status == AttendanceRecord.Status.LATE
record.get_status_display()   # → "Late"
```

কোথায়	কী বাছাই
উপস্থিতি	PRESENT · ABSENT · LATE · LEAVE
ভূমিকা	ORG_ADMIN · COORDINATOR · TEACHER · STUDENT
প্রতিষ্ঠানের ধরন	INSTITUTION · PERSONAL
সাবস্ক্রিপশন	TRIAL · ACTIVE · PAST_DUE · SUSPENDED · CANCELLED
পেমেন্ট	bKash · Nagad · Bank · Cash · Other
হোমওয়ার্ক জমা	SUBMITTED · REVIEWED · RETURNED

লেবেল বদলালে ইংরেজি লেখাটা বদলায়, মানটি থাকে — তাই পুরোনো ডেটা ভাঙে না।

ফাঁদ: `"present"` আর `"PRESENT"` — ছোট-বড় হাতের একটি অক্ষরে রিপোর্ট ভুল দেখাবে, আর এররও আসবে না।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন: "choices না দিলে কী হারাবেন?" উত্তর: ড্রপডাউন, লেবেল, আর ভুল মান আটকানোর ক্ষমতা।

অনুশীলন

২২ মিনিট — নিজের প্রথম তিনটি মডেল

কপি নয়, ক্রম মানুন

১. TimeStamped লিখুন (৩ মিনিট)

abstract base, দুইটি সময়-কলাম। এরপর সব মডেল এটি থেকে শুরু হবে।

২. একটি মূল মডেল (৭ মিনিট)

স্কোপ FK + নাম + ২টি তারিখ/সংখ্যা + `Meta`-তে unique ও ordering + `__str__`।

৩. নিয়ম যোগ করুন (৭ মিনিট)

একটি `clean()` (তারিখের ক্রম) আর একটি `choices`। দুই জায়গায় ভুল ডেটা ঢোকার পথ বন্ধ করুন।

৪. যাচাই (৫ মিনিট)

শেল খুলুন: `full_clean()` দিয়ে ভুল ডেটা দিন — এরর আসছে? তারপর `objects.create()` দিয়ে একই ভুল দিন — কী হলো?

চতুর্থ ধাপেই আজকের সবচেয়ে বড় শিক্ষা: **clean() শুধু ফর্ম পাহারায়, constraint ডেটাবেস পাহারায়**।

শিক্ষকের নোট: প্রত্যেকে যেন চতুর্থ ধাপটা নিজের হাতে করে — ওই দুই আউটপুট পাশাপাশি দেখলেই আর কোনো ব্যাখ্যা লাগবে না।

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

উত্তর এক বাক্যে

- নিয়ম ফর্মে লিখলে কী কী পথ খোলা থাকে?
- `clean()` আর `CheckConstraint` — দুটোই রাখবেন কেন?
- `@property` বনাম ভিউতে হিসাব — কোনটা কোথায় রাখা উচিত?
- `save()` override-এ ব্যবসার নিয়ম লেখা কখন বিপজ্জনক?

শিক্ষকের নোট: শেষ প্রশ্নে বলুন: "bulk_create, update(), loaddata — এগুলো save() ডাকে না। তাই খুব জরুরি নিয়ম save()-এ নয়, constraint-এ রাখা ভালো।"

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
মডেল = কলাম + নিয়ম + আচরণ	৩১ মডেল · ২৩২ ফিল্ড · ১৪ clean · ১৪ property
সাধারণ কলাম abstract base-এ	`TimeStamped` — ১৯টি মডেল
টাকায় Decimal, কখনো Float নয়	DecimalField ৬টি · FloatField ০টি
নিয়ম ডেটাবেসেও রাখুন	২টি CheckConstraint + ৮টি UniqueConstraint
ব্যবসার নিয়ম ডেটার ঘরে	`Subscription.is_writable` — মিডলওয়্যার, ব্যানার, রিপোর্ট সবাই পড়ে
ভুল কাজ কঠিন করে তুলুন	UserManager — প্লেইন পাসওয়ার্ড সেভ করা অসম্ভব

শিক্ষকের নোট: পঞ্চম সারিটা আবার বলুন: "নিয়ম একটা জায়গায়, ব্যবহার বহু জায়গায়।" এটাই ভিউ-এ লজিক না রাখার আসল কারণ।

১৭

পরের ক্লাস — "অনুরোধ কোথায় যায়"

ভিউ ও URL: ৮৫টি রুট · function-based view · decorator · 404/403 · pagination

পর্ব ৪ · কোডিং · ক্লাস ১৭ / ৩৭

অনুরোধ কোথায় যায়

সিলেবাস: প্র্যাকটিক্যাল ৫ · টপিক ৫.২ (ভিউ ও রাউটিং)

৮৫টি রুট · ৫৭টি স্কোপড ভিউ · ৪টি গন্তব্য: render · redirect · 404 · 403

URL → decorator → view → template

শুরু করুন প্রশ্ন দিয়ে

একটি ক্লিকের পরে কতটা সিদ্ধান্ত নেওয়া হয়?

হাত তুলুন: একটি ভিউতে ক'টি সিদ্ধান্ত? ৩? ৫? ১০?

৮৫টি রুট, কিন্তু ৫৭টি ভিউ একই দরজা দিয়ে ঢোকে — @workspace_view, যে চারটি প্রশ্নের উত্তর আগেই দিয়ে দেয়।

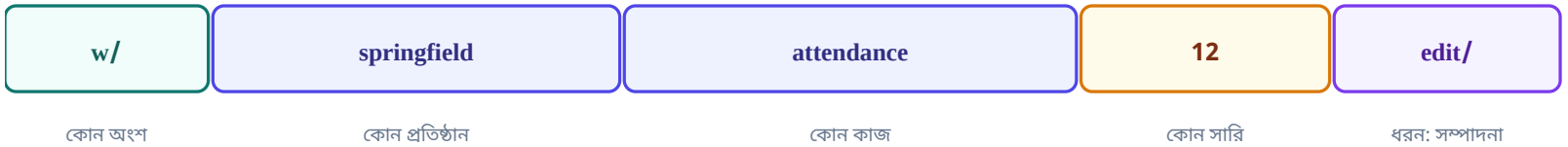
আজকের ক্লাসে প্রতিটি ভিউর ভেতরের ক্রমটা দেখব: কে ঢুকতে পারবে (৪০৪), ভূমিকা কী (৪০৩), কী ডেটা (স্কোপ), তারপর সেভ নাকি দেখানো (POST/GET), শেষে কোথায় ফেরত (redirect)।

শিক্ষকের নোট: বলুন: "ভিউ হলো দরজার প্রহরী — কিন্তু প্রহরীর কাজ বন্ধুকে চেনা নয়, অচেনাকে ঠেকানো।" এই বাক্যেই ক্লাসের সুর।

ধাপ ১

একটি URL পড়তে শেখা

`w/springfield/attendance/12/edit/` — প্রতিটি অংশের মানে আছে



প্যাটার্ন: w/<slug:slug>/attendance/<int:pk>/edit/

৫৭টি ভিউর URL-ই প্রথমে প্রতিষ্ঠানের slug নেয় — তাই টেন্যান্ট ঠিকানা থেকে আসে, সেশনের অনুমানে নয়।

ব্রাউজারে দেখা যায়, শেয়ার করা যায়, বুকমার্কে থাকে — আর ভুল ঠিকানায় গেলে ৪০৪, চুরি করতে গেলে ৪০৩।

বিকল্প: প্রতিষ্ঠান সেশন থেকে নেওয়া (slug ছাড়া URL)। কিন্তু তাহলে দুই ট্যাবে দুই প্রতিষ্ঠান খোলা যায় না — আর শেয়ার করা লিংক অন্য প্রতিষ্ঠানে গিয়ে খোলে। তাই slug URL-এ।

শিক্ষকের নোট: ছাত্রদের বলুন নিজের প্রজেক্টের একটি URL লিখতে, তারপর প্রতিটি অংশে প্রশ্ন করুন "এটা কেন এখানে?" — যে অংশের উত্তর নেই, সেটা বাদ দেওয়ার চেষ্টা করুন।

ধাপ ২

URLconf — ১১টি অ্যাপ, ১১টি include

প্রতিটি অ্যাপের নিজের `urls.py`, নিজের `app_name`: মোট ৮৫টি নাম

```
# config/urls.py
urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("apps.accounts.urls")),
    path("", include("apps.workspaces.urls")),
    path("", include("apps.core.urls")),
    path("", include("apps.academics.urls")),
    path("", include("apps.students.urls")),
    path("", include("apps.attendance.urls")),
    path("", include("apps.lessons.urls")),
    path("", include("apps.assessments.urls")),
    path("", include("apps.reports.urls")),
    path("", include("apps.audit.urls")),
    path("", include("apps.billing.urls")),
]

# apps/attendance/urls.py
app_name = "attendance"
urlpatterns = [
    path("w/<slug:slug>/attendance/", views.take, name="take"),
    path("w/<slug:slug>/attendance/register/", views.register,
name="register"),
    path("w/<slug:slug>/attendance/summary/", views.summary,
name="summary"),
    path("w/<slug:slug>/attendance/<int:pk>/edit/",
         views.record_edit, name="record_edit"),
```

app_name কেন

`"attendance:take"` — একই নাম দুই অ্যাপে থাকতে পারে, সংঘর্ষ নেই।

কোডে নাম, URL নয়

`redirect("attendance:register", slug=slug)` — ঠিকানা বদলালে এক জায়গায় বদলায়।

ক্রম মানে

উপরের প্যাটার্ন আগে মেলে — নির্দিষ্ট পথ সাধারণ পথের আগে রাখুন।

শিক্ষকের নোট: বলুন: "কোথাও যদি `"/w/x/attendance/"` হার্ডকোড করা থাকে, সেটাই লুকানো বাগের জন্ম।" নিজের প্রজেক্টে grep করতে বলুন।

ধাপ ৩

একটি ভিউ, লাইন ধরে ধরে

`apps/attendance/views.py` — `take()`, উপস্থিতি নেওয়ার পর্দা

```python
@workspace_view(*TEACHING)
def take(request, slug):
    ws = request.workspace                      # decorator দিয়েছে
    classes = visible_classes(request.user, ws,
                              request.membership)   # স্কোপ
    class_id = request.GET.get("class") or request.POST.get("class_group")
    form = AttendanceBulkForm(request.POST or None,
                              workspace=ws, user=request.user,
                              class_id=class_id)
    cls = None
    if class_id:
        cls = classes.filter(pk=class_id).first()
        if cls is None:
            raise PermissionDenied("You are not assigned to that class.")
        if request.method == "POST" and form.is_valid():
            day = form.cleaned_data["date"]
            if day > timezone.localdate():
                messages.error(request, "Attendance cannot be taken for a future date.")
            else:
                created, updated = _save_day(request, cls, ...)
                log(request, "attendance.save", cls, summary=...)
                messages.success(request, f"Attendance saved: {created} new, ...")
                return redirect(f"{request.path}?class={cls.pk}&date={day}")
    return render(request, "attendance/take.html", {...})
```

ধাপ	কী হচ্ছে
decorator	লগইন + প্রতিষ্ঠান + ভূমিকা
`visible_classes`	শিক্ষক শুধু নিজের শ্রেণি
`classes.filter(pk=…)`	অন্যের শ্রেণি চাইলে **৪০৩**
`request.POST or None`	GET-এ ফর্ম খালি, POST-এ ভরা
`form.is_valid()`	নিয়ম ফর্মে, ভিউতে নয়
`_save_day()`	এক transaction-এ সব ছাত্র
`log(...)`	অডিট — কে কখন
`redirect` / `render`	সেভের পরে সরাসরি HTML নয়

শিক্ষকের নোট: বলুন: "লক্ষ্য করুন ভিউতে একটি নিয়মও লেখা নেই — কোন ফিল্ড বাধ্যতামূলক, কে অনুমোদিত, সব অন্য জায়গায়। ভিউ শুধু সাজায়।"

ধাপ ৪

@workspace_view — এক decorator, চার সিদ্ধান্ত

`apps/core/permissions.py` · ৫৭টি ভিউতে ব্যবহৃত

```python
def workspace_view(*allowed_roles, allow_platform_admin=True):
    def decorator(view_func):
        @wraps(view_func)
        def _wrapped(request, slug, *args, **kwargs):
            if not request.user.is_authenticated:
                return redirect(f"/accounts/login/?next={request.path}")

            workspace = Workspace.objects.filter(
                slug=slug, is_active=True).first()
            if workspace is None:
                raise Http404("Workspace not found.")

            membership = get_membership(request.user, workspace)
            if membership is None:
                # প্ল্যাটফর্ম অ্যাডমিনও অন্যের ব্যক্তিগত
                # রেকর্ড দেখতে পারে না
                raise Http404("Workspace not found.")

            if allowed_roles and membership.role not in allowed_roles:
                raise PermissionDenied(
                    "Your role in this workspace does not allow this action.")

            request.workspace = workspace
            request.membership = membership
            request.session["active_workspace_id"] = workspace.pk
            return view_func(request, slug, *args, **kwargs)
        return _wrapped
    return decorator
```

১. লগইন নেই → redirect

`?next=` সহ — লগইনের পরে ফিরে আসবে।

২. প্রতিষ্ঠান নেই → ৪০৪

অস্তিত্বই স্বীকার করি না।

৩. সদস্য নয় → ৪০৪

"আছে কিন্তু নিষেধ" বললে প্রতিষ্ঠানের অস্তিত্ব ফাঁস হয়।

৪. ভূমিকা মেলে না → ৪০৩

এখন সে ভেতরে, তাই সত্য বলা যায়।

+ উপহার

`request.workspace` আর `request.membership` — ভিউতে আর খুঁজতে হয় না।

শিক্ষকের নোট: ৩ ও ৪-এর পার্থক্যটাই ক্লাসের প্রাণ। জিজ্ঞেস করুন: "অচেনা মানুষকে কী বলবেন — 'ঘর আছে কিন্তু ঢুকতে দেব না', নাকি 'এমন ঘর নেই'?"

ধাপ ৫

৪০৩ নাকি ৪০৪ — একটি সিদ্ধান্ত, বড় পার্থক্য

কারণ: সদস্যতা জানা মানেই প্রতিষ্ঠানের অস্তিত্ব জানা

পরিস্থিতি	কোড	কেন এটাই
লগইন নেই	৩০২ → login	`?next=` দিয়ে ফেরত — ব্যবহারকারী বিভ্রান্ত নয়
slug নেই / প্রতিষ্ঠান বন্ধ	**৪০৪**	ঠিকানা ভুল — পর্দাই নেই
অন্য প্রতিষ্ঠানের slug	**৪০৪**	৪০৩ বললে "এই প্রতিষ্ঠান আছে" ফাঁস হতো
সদস্য, কিন্তু ভূমিকা নয়	**৪০৩**	সে ভেতরের মানুষ — লুকানোর কিছু নেই
শিক্ষক, অন্য শ্রেণির আইডি	**৪০৩**	স্কোপের বাইরে, কিন্তু প্রতিষ্ঠান তার নিজের
ছাত্র, ক্লাসের ফল এক্সপোর্ট	**৪০৩**	"Students cannot export class results."

ভালো এরর: কী হয়নি + কেন + এখন কী করব (ক্লাস ১২-এর বার্তা-নিয়ম)।

খারাপ: সব জায়গায় ৪০৪ — ব্যবহারকারী ভাবেন সাইট নষ্ট, সাপোর্টে ফোন করেন।

শিক্ষকের নোট: বলুন: "৪০৩ আর ৪০৪-এর পার্থক্য আক্রমণকারীর জন্য ছোট, ব্যবহারকারীর জন্য বিশাল। তাই সদস্যের জন্য ৪০৩, অচেনার জন্য ৪০৪।"

ধাপ ৬

get_object_or_404 — ৩৫ জায়গায়, সবসময় স্কোপসহ

আইডি দিয়ে খোঁজা মানে এখনো টেন্যান্ট যাচাই করা হয়নি

```
# বিপজ্জনক — অন্য প্রতিষ্ঠানের সারিও আসে
rec = AttendanceRecord.objects.get(pk=pk)

# IMS-এর নিয়ম — স্কোপ ভিউতেই
rec = get_object_or_404(
    AttendanceRecord, pk=pk,
    workspace=request.workspace)

# স্কোপের বাইরে গেলে আলাদা ৪০৩
if not teacher_can_access_class(request.user, rec.class_group):
    raise PermissionDenied(
        "You are not assigned to this class.")
```

দুই স্তরের চেক

আগে প্রতিষ্ঠান (৪০৪), পরে শ্রেণি-স্কোপ (৪০৩) — দুই প্রশ্ন, দুই উত্তর।

কেন `.get()` নয়

`.get()` ব্যর্থ হলে ৫০০ — ব্যবহারকারী ভাবেন সার্ভার ভাঙা।

সংখ্যায়

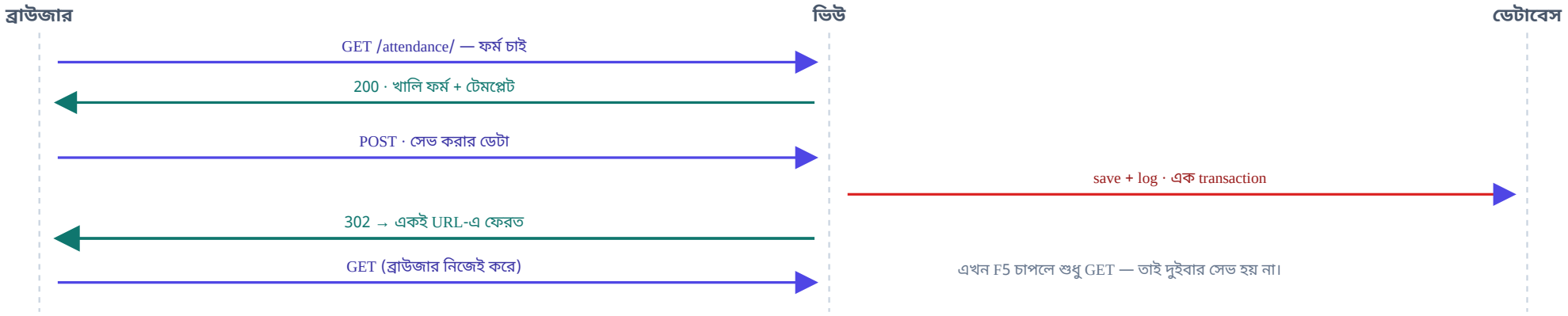
৩৫টি `get_object_or_404` · ২৭টি `PermissionDenied` — কোডবেস জুড়ে একই প্যাটার্ন।

শিক্ষকের নোট: প্রথম লাইনটা বোর্ডে লিখুন, পাশে "কোন প্রতিষ্ঠানের?" প্রশ্ন দিন। উত্তর না থাকলে বলুন: "এই একটা লাইনই পুরো সিস্টেম ফাঁস করতে পারে।"

ধাপ ৭

এক ভিউ, দুই কাজ — GET দেখায়, POST বদলায়

সেভের পরে সবসময় redirect (POST → Redirect → GET) · ৬৬টি redirect



কেন redirect: নাহলে রিফ্রেশে ফর্ম আবার সাবমিট হয় — দুইবার উপস্থিতি।

কেন `request.POST or None`: একটি ভিউতেই দেখানো ও সেভ — দুই ভিউ নয়।

শিক্ষকের নোট: লাইভে F5 চাপুন সেভের পরে — যদি "আবার জমা দিবেন?" বার্তা আসে, বোঝা যায় redirect বাদ পড়েছে। এটা ছাত্রদের সবচেয়ে সাধারণ ভুল।

ধাপ ৮

তালিকা মানেই pagination

দুই জায়গায় ব্যবহৃত: শিক্ষার্থী তালিকা ২৫/পাতা · অডিট লগ ৪০/পাতা

```
qs = (_visible_students(request)
      .select_related("account"))
# সার্চ · ফিল্টার
if q:
    qs = qs.filter(full_name__icontains=q) \
           | qs.filter(admission_no__icontains=q)
if status:
    qs = qs.filter(status=status)
qs = qs.distinct().order_by("full_name")

from django.core.paginator import Paginator

page = Paginator(qs, 25).get_page(request.GET.get("page"))
return render(request, "students/list.html", {
    "page_obj": page,
    "q": q, "status": status,
})
```

get_page, page নয়

`page=999` এলেও ক্র্যাশ করে না — শেষ পাতা দেয়।

ফিল্টার আগে, pagination পরে

ক্রম উল্টালে "২৫টি ফল" ভুল সংখ্যা দেখাবে।

কেন ২৫ আর ৪০

শিক্ষার্থীর সারি লম্বা, অডিটের সারি ছোট — পর্দার মাপ অনুযায়ী।

ফাঁদ: `|` ইউনিয়ন

দুই কোয়েরি জোড়া দিলে `distinct()` লাগে — আর বড় টেবিলে ধীর। `Q(...)` ভালো।

শিক্ষকের নোট: শেষ কার্ডটা সততার সাথে বলুন: "এই কোডটা কাজ করে, কিন্তু এটাই সেরা নয় — `Q(full_name__icontains=q) | Q(admission_no__icontains=q)` লিখলে একটি কোয়েরি, distinct-ও লাগে না।"

ধাপ ৯

কুয়েরি সংখ্যা — ভিউর আসল খরচ

একটি তালিকা পর্দা ৩টি কুয়েরিতে হতে পারে, ৩০৩টিতেও

```
# N+1: প্রতিটি সারির জন্য আলাদা কুয়েরি
for e in cls.enrollments.all():
    print(e.student.full_name)
#   1 + N কুয়েরি

# IMS: join আগেই
for e in cls.enrollments.select_related(
        "student").order_by("student__full_name"): ...
#   1 কুয়েরি

# গণনা ডেটাবেসে, পাইথনে নয়
agg = qs.values("student__full_name").annotate(
    total=Count("id"),
    present=Count("id", filter=Q(status__in=["PRESENT", "LATE"])),
).order_by("student__full_name")
```

কৌশল	কখন
`select_related`	FK/1:1 — JOIN করে আনে
`prefetch_related`	M:N বা উল্টো দিকের তালিকা
`values()`	শুধু দরকারি কলাম — মডেল বানায় না
`annotate(Count)`	পাইথনে লুপ নয়, ডেটাবেসে গণনা
`exists()` / `count()`	পুরো সারি টেনে না এনে প্রশ্নের উত্তর

মাপার উপায়: `connection.queries` বা Django Debug Toolbar — অনুমান নয়।

শিক্ষকের নোট: বলুন: "৫০ জন ছাত্রের ক্লাসে ৫১টি কুয়েরি = ৫১ বার নেটওয়ার্ক। ৫০০ জনে সেটাই সেকেন্ডে পরিণত হয়।" সংখ্যাটা বোর্ডে লিখুন।

সতর্কতা

ভিউ-এ পাঁচটি সাধারণ ভুল

প্রতিটির প্রতিকার একটি লাইনে

ভুল	পরিণতি ও প্রতিকার
১. আইডি দিয়ে সরাসরি `.get(pk=…)`	অন্য প্রতিষ্ঠানের ডেটা দেখা যায় → `workspace=request.workspace` যোগ করুন
২. সেভের পরে `render`	রিফ্রেশে দুইবার সেভ → `redirect` (PRG)
৩. সব ভুলে ৪০৪	ভূমিকা ভুল হলেও ৪০৪ → সদস্যের জন্য ৪০৩
৪. ভিউতে ব্যবসার নিয়ম	দুই ভিউ দুই রকম হিসাব → নিয়ম মডেল/সার্ভিসে
৫. তালিকায় pagination নেই	৫০০০ সারির পর্দা ধীরে মরে → `Paginator`

একটাই প্রশ্ন সবগুলো ধরে: **"এই ভিউটি কোন প্রতিষ্ঠানের?"** — উত্তর কোডে লেখা থাকলে প্রথম ভুলটাও থাকবে না।

শিক্ষকের নোট: প্রতিটি ভুলের জন্য একজন ছাত্রকে নিজের প্রজেক্টে খুঁজতে বলুন — ৫ মিনিটে ৫টি বাস্তব উদাহরণ পাবেন, আর সেগুলোই স্মৃতিতে থাকবে।

অনুশীলন

২০ মিনিট — নিজের তিনটি রুট

তালিকা · তৈরি · সম্পাদনা

১. URL লিখুন (৫ মিনিট)

তিনটি রুট, প্রতিটিতে স্কোপ: `/w/<slug>/items/` · `/new/` · `/<int:pk>/edit/`। `app_name` দিন।

২. decorator লিখুন (৭ মিনিট)

লগইন নেই → redirect · সদস্য নয় → ৪০৪ · ভূমিকা নয় → ৪০৩ · সফল হলে `request.workspace` বসান।

৩. ভিউ তিনটি (৫ মিনিট)

তালিকায় Paginator + search; তৈরি ও সম্পাদনায় `request.POST or None` + সেভের পরে redirect।

৪. যাচাই (৩ মিনিট)

অন্য একটি প্রতিষ্ঠান/অন্য ভূমিকা দিয়ে লগইন করে দেখুন — ৪০৪ আর ৪০৩ ঠিক জায়গায় আসছে?

বাড়তি কাজ: ব্যবহারকারীকে লগআউট করে সরাসরি সম্পাদনার লিংক দিন — লগইনের পরে সে ওই পাতায় ফিরে আসছে? (`?next=`)

শিক্ষকের নোট: চতুর্থ ধাপে সবাই যেন দুটি ভিন্ন ভূমিকা দিয়ে টেস্ট করে — এটাই আসল অভ্যাস, আর পরের পর্বে টেস্ট লেখার ভিত্তি।

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

উত্তর এক বাক্যে

- slug URL-এ না রেখে সেশন থেকে নিলে কী কী ভেঙে যেত?
- সদস্য নয় এমন ব্যবহারকারীকে ৪০৪ কেন, ৪০৩ নয়?
- সেভের পরে `render` করলে ব্যবহারকারী কী ভুল দেখবেন?
- একটি ভিউতে ব্যবসার নিয়ম লিখলে ভবিষ্যতে কী হয়?

শিক্ষকের নোট: তৃতীয় প্রশ্নের উত্তর একজন ছাত্রকে লাইভ দেখাতে দিন (F5 চেপে ডুপ্লিকেট সাবমিট) — মুখে শোনার চেয়ে দেখলে অনেক ভালো মনে থাকে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
প্রতিষ্ঠান URL-এ, সেশনে নয়	৫৭টি রুট `w/<slug>/` দিয়ে শুরু
এক decorator, চার সিদ্ধান্ত	`@workspace_view` ৫৭ ভিউতে
অচেনা → ৪০৪, সদস্য → ৪০৩	`permissions.py`-এর দুই শাখা
আইডি দিয়ে খুঁজলেও স্কোপ লাগে	৩৫টি `get_object_or_404(..., workspace=…)`
সেভের পরে redirect	৬৬টি redirect · ৬৫টি messages
তালিকা মানেই pagination ও select_related	২৫/৪০ প্রতি পাতা · JOIN দিয়ে N+1 এড়ানো

শিক্ষকের নোট: দ্বিতীয় ও চতুর্থ সারি একসাথে বলুন: "প্রহরী দরজায়, আর প্রতিটি দরজার ভেতরেও একবার চেক।" — defense in depth-এর প্রথম পাঠ।

১৮

পরের ক্লাস — "ভুল ইনপুট, ভদ্র উত্তর"

ফর্ম ও ভ্যালিডেশন: ModelForm · field.html · এরর দেখানো · ফাইল আপলোডের নিয়ম

পর্ব ৪ · কোডিং · ক্লাস ১৮ / ৩৭

ভুল ইনপুট, ভদ্র উত্তর

সিলেবাস: প্র্যাকটিক্যাল ৫ · টপিক ৫.৩ (ফর্ম ও ভ্যালিডেশন)

৩৫টি ফর্ম · ১০টি clean · ১২টি widget · ৪টি পাসওয়ার্ড নিয়ম

ModelForm · fields whitelist · add_error

শুরু করুন প্রশ্ন দিয়ে

ভুল ডেটা কোথায় আটকানো উচিত?

হাত তুলুন: ব্রাউজারে? ভিউতে? ফর্মে? ডেটাবেসে?

IMS-এ তিন জায়গায় আটকায় — ফর্ম (ভদ্র বার্তা), মডেল (নিয়ম), ডেটাবেস (শেষ রক্ষা)। আজ প্রথমটির কারিগরি।

ফর্মের কাজ শুধু "ডেটা নেওয়া" নয় — **সেই ডেটাকে বিশ্বাসযোগ্য বানানো**। ৩৫টি ফর্মে ১০টি জায়গায় নিজস্ব নিয়ম লেখা আছে, আর সেগুলোই ব্যবহারকারীর দেখা প্রথম এরর।

শিক্ষকের নোট: উত্তর সাধারণত "ব্রাউজারে" আসে। বলুন: "ব্রাউজারের ভ্যালিডেশন বন্ধ করা যায় একটি বাটনে — তাই সেটা ভদ্রতা, নিরাপত্তা নয়।"

ধাপ ১

ফর্মের চারটি দায়িত্ব

এক ক্লাস, চার কাজ — আলাদা করে ভাবলে কোড পরিষ্কার থাকে

১. সংগ্রহ

POST → cleaned_data

প্রতিটি ফিল্ডের ধরন

"12" → 12 (int)

স্ট্রিং নয়, সত্যিকারের মান

২. যাচাই

required · min_length

clean_<field>()

clean() — দুই ফিল্ডের সম্পর্ক

নিয়ম একবার, সব পথে

৩. ব্যাখ্যা

errors[field]

non_field_errors

help_text

কোন ঘর, কেন, কী করব

৪. সংরক্ষণ

form.save(commit=False)

workspace বসানো

তারপর save()

স্কোপ ভিউ থেকে আসে

চারটি কাজ একই ক্লাসে থাকলে ভিউ ছোট থাকে — আর ব্যবহারকারী সব পর্দায় একই ধরনের বার্তা পান।

শিক্ষকের নোট: বলুন: "ভিউ যদি ফর্ম আর টেমপ্লেটের মাঝের মধ্যস্থতাকারী হয়, তাহলে তার কাজ কম — এটাই ভালো ডিজাইন।"

ধাপ ২

ModelForm — `fields` হলো হোয়াইটলিস্ট

কোন কলাম ফর্মে আসবে, সেটাই নিরাপত্তার প্রথম সিদ্ধান্ত

```
class StudentForm(BootstrapFormMixin, forms.ModelForm):
    class Meta:
        model = StudentProfile
        fields = ["admission_no", "full_name",
                  "guardian_name", "guardian_phone",
                  "date_of_birth", "notes"]
        widgets = {"date_of_birth": forms.DateInput(
            attrs={"type": "date"})}

    def __init__(self, *args, workspace=None, **kwargs):
        super().__init__(*args, **kwargs)
        self.workspace = workspace

# ভিউতে:
obj = form.save(commit=False)
obj.workspace = request.workspace     # কখনো ফর্ম থেকে নয়
obj.save()
```

fields কেন দরকার

`"__all__"` দিলে ব্যবহারকারী নিজের `workspace`, `status` — এমনকি `id` পর্যন্ত পাঠাতে পারে।

workspace ফর্মে নেই

তালিকায় নেই মানে POST-এ পাঠানোও যায় না — mass assignment বন্ধ।

widgets শুধু চেহারা

`type="date"` দিলে ব্রাউজারে তারিখ-নির্বাচক আসে — নিয়ম নয়, সুবিধা।

শিক্ষকের নোট: আক্রমণটা দেখান: ব্রাউজারের ডেভ-টুলে একটি লুকানো ইনপুট যোগ করে `workspace=2` পাঠান — `fields`-এ না থাকলে কিছুই হয় না। এটাই সবচেয়ে বাস্তব নিরাপত্তা-পাঠ।

ধাপ ৩

একটি field-level নিয়ম, তিনটি সূক্ষ্মতা

`clean_admission_no()` — ইউনিক, কিন্তু সম্পাদনার সময় নিজেকে বাদ দিয়ে

```
def clean_admission_no(self):
    value = self.cleaned_data["admission_no"].strip()
    qs = StudentProfile.objects.filter(
        workspace=self.workspace,
        admission_no__iexact=value)
    if self.instance.pk:          # সম্পাদনা করছি
        qs = qs.exclude(pk=self.instance.pk)
    if qs.exists():
        raise forms.ValidationError(
            "Another student in this workspace "
            "already uses this admission number.")
    return value
```

সূক্ষ্মতা	না করলে
`workspace=` ফিল্টার	অন্য প্রতিষ্ঠানের নম্বরও "দখল" দেখাবে
`__iexact`	`A-12` আর `a-12` দুইবার ঢুকবে
`.strip()`	শেষে স্পেস থাকলে ভিন্ন মান
`exclude(pk=…)`	নিজের রেকর্ড সম্পাদনা করতে গেলে "আগেই আছে" এরর
`return value`	পরের ধাপে `None` পাবে — ডেটা হারাবে

শেষ সারিটি সবচেয়ে বেশি ভুল হয়: `clean_<field>` কখনো `None` ফেরাতেই হবে না — **পরিষ্কার করা মান** ফেরাতে হয়।

শিক্ষকের নোট: "সম্পাদনা করুন" আর "নতুন যোগ করুন" — একই ফর্ম। ছাত্ররা exclude(pk) ভুলে যায়, তারপর বলে "আমার ফর্মটা কাজ করে না।" এখানেই সেই বাগের জন্ম।

ধাপ ৪

দুই mixin, একই কাজ — একটি সততার পাঠ

কোডবেসে সবকিছু নিখুঁত নয়; চেনা ও ঠিক করার ক্ষমতাই আসল দক্ষতা

```python
# apps/academics/forms.py
class BootstrapFormMixin:
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        for name, field in self.fields.items():
            w = field.widget
            if isinstance(w, forms.CheckboxInput):
                w.attrs.setdefault("class", "form-check-input")
            elif isinstance(w, forms.Select):
                w.attrs.setdefault("class", "form-select")
            else:
                w.attrs.setdefault("class", "form-control")

# apps/billing/forms.py — একই কাজ, ভিন্ন নাম
class ImsFormMixin:
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        for field in self.fields.values():
            widget = field.widget
            css = widget.attrs.get("class", "")
            if widget.__class__.__name__ == "CheckboxInput":
                widget.attrs["class"] = (css + " form-check-input").strip()
            ... আরও চারটি শাখা
```

সমস্যাটা কী

দুটি ফাইল, দুটি নাম, প্রায় একই কাজ — একটি বদলালে অন্যটি পুরোনো হয়ে থাকবে।

লক্ষণ

কিছু পর্দায় তারিখ-নির্বাচক বা রঙ-নির্বাচক অন্যভাবে দেখায়।

সমাধান

একটি mixin `apps/core/`-এ সরান, অন্যটি ইমপোর্ট করুন — দুই লাইনেই কাজ।

শিক্ষা

প্রতিলিপি কোড নিজে থেকে ভাঙে না — **একদিন একটি কপি আপডেট হয়**, তখন ভাঙে।

শিক্ষকের নোট: এটা স্বীকার করুন: "এই ডেকে আমি IMS-এর ভুলও দেখাই।" ছাত্রদের বলুন নিজের কোডে grep করে দেখতে — `Bootstrap`, `Mixin`, `mixin`।

ধাপ ৫

ড্রপডাউনের ভেতরটা — queryset কে ঠিক করে

`__init__`-এ queryset বদলানো, আর কখনোই `objects.all()` নয়

```
class EnrolForm(forms.Form):
    students = forms.ModelMultipleChoiceField(
        queryset=StudentProfile.objects.none(),   # নিরাপদ শুরু
        widget=forms.CheckboxSelectMultiple(
            attrs={"class": "form-check-input"}),
        help_text="Only active students not already in "
                  "this class are listed.")

    def __init__(self, *args, workspace=None,
                 class_group=None, **kwargs):
        super().__init__(*args, **kwargs)
        qs = StudentProfile.objects.filter(
            workspace=workspace,
            status=StudentProfile.Status.ACTIVE)
        if class_group is not None:
            already = Enrollment.objects.filter(
                class_group=class_group, is_active=True,
            ).values_list("student_id", flat=True)
            qs = qs.exclude(pk__in=already)
        self.fields["students"].queryset = qs.order_by("full_name")
```

`none()` দিয়ে শুরু

কেউ `workspace` দিতে ভুলে গেলে খালি তালিকা — ফাঁস নয়।

আগেই ভর্তি কারা

`exclude(pk__in=already)` — তালিকায় দেখাবে না, আর জমা দিলে ভ্যালিডেশনেও আটকাবে।

দুই স্তরে একই নিয়ম

তালিকা সাজানো = UX · queryset = নিরাপত্তা। দুটোই লাগে।

শিক্ষকের নোট: প্রশ্ন করুন: "ড্রপডাউনে না দেখালে কি কেউ পাঠাতে পারবে?" উত্তর: হ্যাঁ, হাতে POST বানিয়ে — তাই queryset-ই আসল প্রহরী, তালিকাটি নয়।

ধাপ ৬

দুই ফিল্ডের সম্পর্ক — `clean()` আর `add_error()`

`BrandingForm`: ডোমেইন ভেরিফাই করা যাবে, কিন্তু খালি ডোমেইনে নয়

```python
def clean(self):
    data = super().clean()
    if data.get("domain_verified") and not data.get("custom_domain"):
        self.add_error("domain_verified",
                       "Verify a domain only after entering one.")
    return data

# কল্পিত খারাপ সংস্করণ
def clean(self):
    if self.cleaned_data["domain_verified"]:
        raise forms.ValidationError("...")
    return self.cleaned_data
```

add_error(field, ...)

এররটা ঠিক ওই ঘরের নিচে বসে — ব্যবহারকারী এক নজরে দেখেন।

raise ValidationError

উপরের সারাংশে যায় — কোন ঘর ঠিক করবেন বোঝা কঠিন।

super().clean() আগে

নাহলে `cleaned_data`-তে ফিল্ড না-ও থাকতে পারে — KeyError।

একই প্যাটার্ন অন্য জায়গায়

`InstitutionForm.clean_admin_email` — ইমেইলের অ্যাকাউন্ট আগেই আছে কি না।

শিক্ষকের নোট: বলুন: "এরর কোথায় দেখাবে, সেটাও ডিজাইনের অংশ। উপরে সারাংশ দেখালে ব্যবহারকারী ২০টি ঘরের মধ্যে খুঁজতে থাকেন।"

ধাপ ৭

এরর থেকে পর্দা — এক partial, সব ফর্ম

`partials/field.html` — লেবেল, সহায়তা ও এরর একই জায়গায়

```
{% include "partials/field.html"
   with field=form.username %}

{# field.html যা করে #}
<label>{{ field.label }}
  {% if field.field.required %}*{% endif %}</label>
{{ field }}
{% if field.help_text %}
  <div class="ims-hint">{{ field.help_text }}</div>
{% endif %}
{% for error in field.errors %}
  <div class="ims-error">{{ error }}</div>
{% endfor %}
```

কোথায় কী দেখায়	উৎস
ফিল্ডের নিচের এরর	`field.errors`
ফর্মের সার্বিক এরর	`form.non_field_errors`
ধূসর সহায়ক লেখা	`help_text`
সবুজ সেভ বার্তা	`messages.success`
লাল ফ্ল্যাশ	`messages.error`

লগইনে `non_field_errors` উপরে দেখানো হয় — কারণ "ইমেইল" না "পাসওয়ার্ড" ভুল, তা বলা যায় না।

শিক্ষকের নোট: শেষ কথাটা গুরুত্বপূর্ণ: "কোনটা ভুল তা বলা মানে আক্রমণকারীকে বলা 'ইমেইলটা ঠিক আছে'।" তাই লগইনে সবসময় সাধারণ বার্তা।

ধাপ ৮

যেখানে প্রতি সারি একটি ফিল্ড — তবু ফর্ম

`AttendanceBulkForm.statuses_for()` — ৪০ জন ছাত্র, ৪০টি রেডিও

```
class AttendanceBulkForm(forms.Form):
    class_group = forms.ModelChoiceField(...)
    subject = forms.ModelChoiceField(...)
    date = forms.DateField(widget=forms.DateInput(
        attrs={"type": "date"}))

    def statuses_for(self):
        """Read the per-student radio values."""
        data = {}
        for key, value in self.data.items():
            if key.startswith("status_"):
                data[int(key.split("_")[1])] = value
        return data

# ভিউতে যাচাই আবার হয়
if status not in dict(AttendanceRecord.Status.choices):
    continue
```

কেন ডাইনামিক

ছাত্রসংখ্যা প্রতি ক্লাসে বদলায় — ফর্ম ক্লাসে ৪০টি ফিল্ড লিখে রাখা অসম্ভব।

নামকরণই চুক্তি

`status_<student_id>` — টেমপ্লেট ও ফর্ম একই নাম মানে।

ভুলের ঝুঁকি

কোনো ছাত্রের রেডিও বাদ পড়লে সে "অনুপস্থিত" নয় — **রেকর্ডই হয় না**। তাই ভিউ সেভের আগে যাচাই করে।

বিকল্প

ফর্মset বা প্রতি সারির জন্য আলাদা ফর্ম — জটিল, কিন্তু সব ভ্যালিডেশন স্বয়ংক্রিয়।

শিক্ষকের নোট: এখানে সীমাবদ্ধতাটা পরিষ্কার বলুন: "ডাইনামিক ফিল্ড মানে ভ্যালিডেশনের দায়িত্ব আংশিক আপনার — ফর্ম আর সব জানে না।"

ধাপ ৯

পাসওয়ার্ড — একটি ফর্ম নয়, চারটি নিয়ম

`AUTH_PASSWORD_VALIDATORS` — Django নিজেই চারটি যাচাই করে

ভ্যালিডেটর	নিয়ম	কী আটকায়
UserAttributeSimilarity	নাম/ইমেইলের মতো নয়	`admin@ims.local` → `admin123`
MinimumLengthValidator	**কমপক্ষে ১০ অক্ষর**	ছোট পাসওয়ার্ড
CommonPasswordValidator	তালিকাভুক্ত সাধারণ নয়	`password` · `qwerty123`
NumericPasswordValidator	শুধু সংখ্যা নয়	`01711111111`

ফর্মে: `min_length=10` · `PasswordInput` · `help_text`-এ নিয়ম লেখা।

ডেমো পাসওয়ার্ড `DemoPass!2026` — ১১ অক্ষর, তাই চারটি নিয়মই পাশ করে।

ফর্মের `min_length` আর ভ্যালিডেটর — দুটোই দরকার: প্রথমটি ভদ্র বার্তা দেয়, দ্বিতীয়টি **অন্য সব পথেও** কাজ করে (অ্যাডমিন, শেল, রিসেট)।

শিক্ষকের নোট: `python manage.py check --deploy` চালিয়ে দেখান — পাসওয়ার্ড ভ্যালিডেটর বাদ দিলে সেটাও সতর্ক করে।

সততার ফাঁক

যেটা IMS-এ এখনো নেই: আপলোড যাচাই

লোগো ফিল্ড শুধু `help_text`-এ বলে "PNG or SVG" — কোড কিছু যাচাই করে না

```
# যা আছে (মডেলের নির্দেশনা মাত্র)
logo = models.ImageField(
    upload_to="branding/logos/", blank=True, null=True,
    help_text="PNG or SVG, ideally 512x512 or wider.")

# যা যোগ করা উচিত
def clean_logo(self):
    logo = self.cleaned_data.get("logo")
    if logo and hasattr(logo, "size"):
        if logo.size > 2 * 1024 * 1024:
            raise forms.ValidationError(
                "Logo must be under 2 MB.")
        ext = logo.name.rsplit(".", 1)[-1].lower()
        if ext not in ("png", "jpg", "jpeg", "svg", "webp"):
            raise forms.ValidationError(
                "Use PNG, JPG, SVG or WebP.")
    return logo
```

ঝুঁকি ১: আকার

৫০ MB লোগো আপলোড হলে ডিস্ক ও ব্যাকআপ ফুলে যায়।

ঝুঁকি ২: ধরন

HTML/SVG স্ক্রিপ্টসহ আপলোড হলে XSS-এর দরজা খোলে — SVG-তে JS থাকতে পারে।

ঝুঁকি ৩: নাম

ব্যবহারকারীর দেওয়া ফাইলনাম সরাসরি ব্যবহার করবেন না — Django নিজেই নাম বদলায়।

পাঠ

"যা লিখিনি, তা হয়নি।" `help_text` নিয়ম নয়, অনুরোধ।

শিক্ষকের নোট: এই স্লাইডটি ইচ্ছাকৃতভাবে সৎ — বলুন: "বাস্তব প্রজেক্টে এমন ফাঁক থাকে। চেনা, লিখে রাখা, পরে ঠিক করা — এটাই পেশাদারি।"

সতর্কতা

ফর্মের পাঁচটি সাধারণ ভুল

প্রতিটির প্রতিকার এক লাইনে

ভুল	পরিণতি ও প্রতিকার
১. `fields = "__all__"`	ব্যবহারকারী নিজের স্কোপ/স্ট্যাটাস পাঠাতে পারে → নির্দিষ্ট তালিকা
২. `queryset=Model.objects.all()`	অন্য প্রতিষ্ঠানের নাম ড্রপডাউনে → `none()` + `__init__`-এ স্কোপ
৩. `clean_<field>` মান ফেরাত না দেওয়া	পরের ধাপে `None` → `return value`
৪. সব এরর `raise` দিয়ে	ভুল ঘর বোঝা যায় না → `add_error("field", …)`
৫. ফর্মের ভ্যালিডেশনকে নিরাপত্তা ভাবা	অন্য পথ (শেল, ইমপোর্ট) খোলা থাকে → মডেল constraint-ও রাখুন

সবগুলো এক প্রশ্নে ধরা পড়ে: **"এই মানটা কি অন্য কোনো পথে সেভ হতে পারে?"** — হ্যাঁ বললে ফর্ম একা যথেষ্ট নয়।

শিক্ষকের নোট: পঞ্চম ভুলটি ক্লাস ১৬-এর সাথে জোড়া লাগান: "ফর্ম ভদ্র প্রহরী, constraint শেষ দেয়াল।"

অনুশীলন

২০ মিনিট — নিজের একটি ফর্ম শক্ত করুন

একই ফর্মে পাঁচটি স্তর

১. হোয়াইটলিস্ট (৩ মিনিট)

একটি ModelForm-এ `fields` তালিকা লিখুন — `__all__` থাকলে বাদ দিন।

২. স্কোপড queryset (৫ মিনিট)

প্রতিটি ModelChoiceField-এ `none()` + `__init__`-এ workspace ফিল্টার।

৩. একটি field-level নিয়ম (৫ মিনিট)

`clean_<field>` — unique চেক, `iexact`, `exclude(pk)`, `return value`।

৪. একটি cross-field নিয়ম (৪ মিনিট)

`clean()`-এ দুই ফিল্ডের সম্পর্ক, `add_error()` দিয়ে নির্দিষ্ট ঘরে।

৫. যাচাই (৩ মিনিট)

ব্রাউজারের ভ্যালিডেশন বন্ধ করে ফাঁকা ফর্ম পাঠান — এররগুলো ঠিক ঘরে আসছে?

শিক্ষকের নোট: `novalidate`-টা লাইভে দেখান — অনেকের কাছে এটাই সবচেয়ে চোখ খোলা মুহূর্ত: "ব্রাউজার যেটা আটকাচ্ছিল, সেটা আটকাচ্ছিল না আসলে।"

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

উত্তর এক বাক্যে

- একই ইউনিক-চেক কেন সম্পাদনার সময় নিজেকে বাদ দিতে হয়?
- ড্রপডাউনে না দেখিয়ে দিলেও কেন কেউ মান পাঠাতে পারে?
- এরর সব উপরে দেখালে ব্যবহারকারীর কী অসুবিধা?
- ফর্মের নিয়ম থাকলেও মডেলে নিয়ম রাখতে হয় কেন?

শিক্ষকের নোট: দ্বিতীয় প্রশ্নে হাতে POST দেখানোর সুযোগ নিন (curl বা ডেভ-টুল) — বোঝা যায় UI কখনো নিরাপত্তা নয়।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
fields হলো হোয়াইটলিস্ট	`StudentForm`-এ workspace নেই
queryset `none()` দিয়ে শুরু	`EnrolForm` · `AttendanceBulkForm`
clean চেক স্কোপ + iexact + exclude(pk)	`clean_admission_no`
এরর নির্দিষ্ট ঘরে	`add_error("domain_verified", …)`
এক partial, সব ফর্মের চেহারা	`partials/field.html` · ৩৫ ফর্ম
পাসওয়ার্ড নিয়ম ফর্মে + ভ্যালিডেটরে	৪টি validator · `min_length=10`

শিক্ষকের নোট: প্রথম ও তৃতীয় সারি সবচেয়ে বেশি কাজে লাগে — বলুন: "এই দুটো অভ্যাস থাকলে আপনার ৮০% ফর্ম-বাগ উঠে যাবে।"

১৯

পরের ক্লাস — "ডেটা থেকে পর্দা"

টেমপ্লেট: inheritance · context · loop ও empty · custom filter · partial · template-এ লজিকের সীমা

পর্ব ৪ · কোডিং · ক্লাস ১৯ / ৩৭

ডেটা থেকে পর্দা

সিলেবাস: প্র্যাকটিক্যাল ৬ · টপিক ৬.১ (টেমপ্লেট)

৭৫টি টেমপ্লেট · ৪টি শেল · ৬টি partial · ১৫টি ডিজাইন টোকেন

inheritance · context processor · empty state

শুরু করুন প্রশ্ন দিয়ে

৭৫টি ফাইল — তবু ক'টি শেল?

হাত তুলুন: নতুন একটি পাতা বানাতে কতটা HTML লিখতে হয়?

উত্তর: প্রায় শূন্য — শুধু `{% block content %}`-এর ভেতরটা।

sidebar, topbar, মেসেজ, বিলিং ব্যানার — সব একবার লেখা, **৪৪টি পাতা** সেটাই ব্যবহার করে। আজকের ক্লাসে দেখব এই একবার-লেখার কারিগরি, আর তার তিনটি ফাঁদ।

শিক্ষকের নোট: "একটি পাতা বানাতে ২০০ লাইন HTML লেখা মানে ৪৪ বার একই ভুল করার সুযোগ।" এটাই template inheritance-এর পুরো যুক্তি।

ধাপ ১

চারটি খোলস, ৭৫টি পাতা

গোনা হয়েছে `{% extends %}` দিয়ে — ৬৫টি পাতার ঠিকানা

base.html

head · body · scripts

৩টি ব্লক, ০টি পাতা

লগইন ৬ + এরর ৩ পাতা

সরাসরি এর ওপর দাঁড়ায়

app_base.html

sidebar + topbar + content

৫টি include

৪৪টি পাতা

কাজের সব পর্দা এখানে

platform/base.html

৬টি ট্যাব, ভিন্ন সাইডবার

টেন্যান্সির বাইরে

৮টি পাতা

প্ল্যাটফর্ম কনসোল

public/base_public

মার্কেটিং শেল

লগইন ছাড়াই দেখা যায়

২টি পাতা

pricing · signup

templates/partials/ — ৬টি টুকরো: sidebar · topbar · messages · billing_banner · cookie_guard · field

চারটি শেল থাকা মানে চারটি আলাদা জীবনের ধরন: লগইন করা ব্যবহারকারী · প্ল্যাটফর্ম অ্যাডমিন · বাইরের অতিথি · আর শুধুই কাঠামো।

শিক্ষকের নোট: ব্ল্যাকবোর্ডে চারটি নাম লিখুন, তারপর জিজ্ঞেস করুন বিলিং পর্দা কোন শেলে — উত্তর app_base। "নতুন শেল কখন? যখন ব্যবহারকারীর জীবনটাই আলাদা।"

ধাপ ২

৪৪টি পাতার একটাই বাবা — `app_base.html`

১২ লাইন HTML, তার ভেতরেই পুরো অ্যাপের কাঠামো

```
{% extends "base.html" %}
{% block body %}
<div class="ims-shell">
  {% include "partials/sidebar.html" %}
  <div class="ims-main">
    {% include "partials/topbar.html" %}
    <div class="ims-content">
      {% include "partials/messages.html" %}
      {% include "partials/billing_banner.html" %}
      {% block content %}{% endblock %}
    </div>
  </div>
</div>
{% endblock %}
```

পর্দার ক্রমটাই এখানে ঠিক হয়: বার্তা → ব্যানার → আসল কাজ।

ব্লক	কত পাতা ভরে
`title`	**৬৫** — ট্যাবের নাম
`content`	**৫৪** — আসল কাজ
`body`	**১৫** — শেল বদলায়
`scripts`	**২** — চার্ট/এডিটর
`heading`	**১** — topbar-এর শিরোনাম

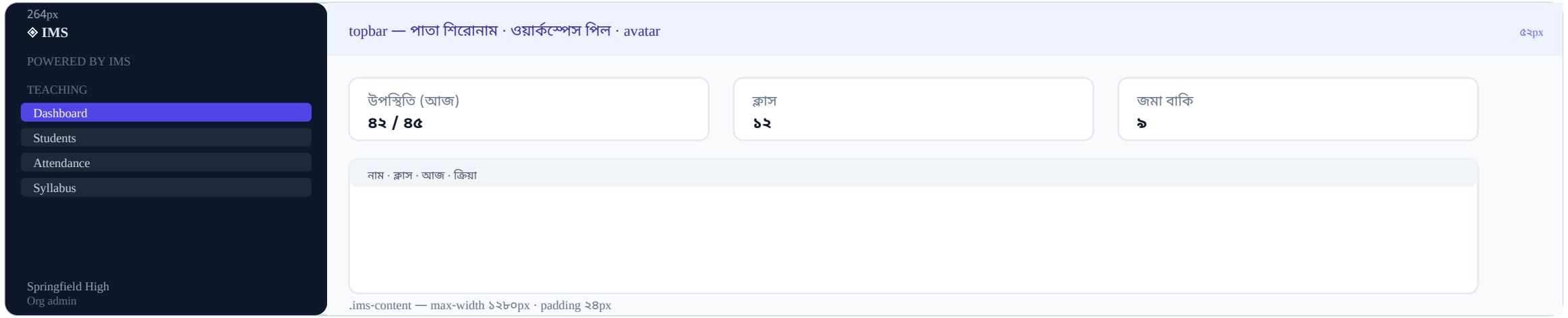
`{% block %}` গুলো সবসময় **একই নামে** থাকতে হবে — নাম বদলালে ৫৪টি পাতায় ওই জায়গাটা ফাঁকা হয়ে যাবে, কোনো এরর ছাড়াই।

শিক্ষকের নোট: শেষ সতর্কতাটা লাইভে দেখান: ব্লকের নাম এক অক্ষর বদলে রেন্ডার করুন — কোনো এরর নেই, শুধু লেখা গায়েব। "নীরব ব্যর্থতা সবচেয়ে খারাপ ব্যর্থতা।"

ধাপ ৩

শেলটা কেমন — মাপসহ

`.ims-shell` = `grid-template-columns: 264px 1fr`



`@media (max-width:820px)` → এক কলাম, সাইডবার লুকিয়ে যায়।

প্রিন্ট/মোবাইলে সাইডবার ছাড়া **ন্যাভিগেশনই থাকে না** — IMS-এ এটি এখনো খোলা প্রশ্ন।

শিক্ষকের নোট: ব্রাউজারে ডিভ-টুল খুলে `264px` বদলিয়ে দেখান — সব ৪৪ পাতায় একসাথে বদলায়। এটাই "একবার ঠিক করুন" নীতির চোখে-দেখা প্রমাণ।

ধাপ ৪

১৫টি টোকেন, পুরো প্রোডাক্টের চেহারা

`static/css/ims.css`-এর `:root` — ৬০০ লাইনের কোড একই ভাষায় কথা বলে

```
:root{
  --ims-primary:#4f46e5; --ims-primary-dark:#4338ca;
  --ims-primary-soft:#eef2ff;
  --ims-ink:#0f172a; --ims-ink-2:#334155;
  --ims-muted:#64748b;
  --ims-line:#e2e8f0; --ims-bg:#f6f7fb;
  --ims-surface:#ffffff;
  --ims-success:#059669; --ims-warning:#d97706;
  --ims-danger:#dc2626;
  --ims-radius:14px; --ims-radius-sm:10px;
  --ims-shadow:0 1px 2px rgba(15,23,42,.06),
               0 8px 24px -12px rgba(15,23,42,.18);
}
```

কেন টোকেন

রঙ এক জায়গায় — ৪৪ পাতায় `#4f46e5` খুঁজতে হয় না।

শৃঙ্খলা

সব সাদা পৃষ্ঠ `--ims-surface`, সব বর্ডার `--ims-line` — নতুন পর্দাও পুরোনোর মতো দেখায়।

হোয়াইট-লেবেলের দরজা

টোকেন থাকায় একটি ভেরিয়েবল বদলেই প্রতিষ্ঠানের রঙ বসানো যায় (পরের দিকে)।

শিক্ষকের নোট: বলুন: "টোকেন হলো ডিজাইনের ভোকাবুলারি। টোকেন না থাকলে প্রতিটি পর্দা নিজের ভাষায় কথা বলে, আর প্রোডাক্ট দেখতে সেলাই-করা কম্বল লাগে।"

ধাপ ৫

একই ড্যাশবোর্ড, তিনটি পর্দা

ভিউ-তে `if role`, টেমপ্লেটে আলাদা ভাগ — এক URL, ভিন্ন অভিজ্ঞতা

ভূমিকা	টেমপ্লেট	pর্দায় কী থাকে
STUDENT	`dashboard_student.html`	উপস্থিতি % · প্রকাশিত পরীক্ষা · খোলা হোমওয়ার্ক (৫) · জমার সংখ্যা
TEACHER	`dashboard_staff.html`	আজকের উপস্থিতি · ক্লাস · অপ্রকাশিত পরীক্ষা · জমা বাকি · সাম্প্রতিক পরীক্ষা (৬)
COORDINATOR / ORG_ADMIN	একই `dashboard_staff.html`	উপরের সব + **সদস্য সংখ্যা · অডিট লগ (৮) · উপস্থিতি নীতি**
PLATFORM_ADMIN	`core/dashboard_platform.html`	প্রতিষ্ঠান · MRR · ট্রায়াল · সাসপেন্ড

`if role in MANAGERS` — একই টেমপ্লেট, বাড়তি কী-তে বাড়তি ব্লক।

`core/`-এ **৪টি ড্যাশবোর্ড টেমপ্লেট** আছে যেগুলো **কোথাও রেন্ডার হয় না** — মৃত ফাইল, শুধু বিভ্রান্তি।

শিক্ষকের নোট: "মৃত টেমপ্লেট খুঁজে বের করা শেখা একটা বাস্তব দক্ষতা" — grep করে দেখান: ভিউয়ের `render()`-এ ওই ফাইলগুলোর নাম নেই।

ধাপ ৬

২৩টি মেনু আইটেম, একটাই সংজ্ঞা

`apps/core/navigation.py` — এক ফাইল, তিন ভোক্তা

```
NAV = [
  {"title": None, "items": [
     _item("Dashboard", "bi-speedometer2",
           "reports:dashboard", STAFF+(STUDENT,))]},
  {"title": "Teaching", "items": [
     _item("Students", "bi-people",
           "students:list", STAFF_ROLES),
     _item("Attendance", "bi-calendar-check",
           "attendance:take", STAFF_ROLES), ...]}]

# রেন্ডারের সময় ফিল্টার
for item in group["items"]:
    if role not in item["roles"]:
        continue
    url = reverse(item["url_name"], kwargs={"slug": slug})

# sidebar-এ সক্রিয় লিংক
class="ims-nav {% if request.path == item.url
  or item.url in request.get_full_path %}is-active{% endif %}"
```

তিন ভোক্তা

সাইডবার দেখায় · ড্যাশবোর্ড হাইলাইট করে · টেস্ট যাচাই করে।

ব্যর্থতার ধরন

নতুন পাতা যোগ করে এই ফাইল ভুলে গেলে — **পাতাটা মেনুতে থাকে না**, কিন্তু URL কাজ করে।

নিরাপত্তার সীমা

মেনু লুকানো **অনুমতি নয়** — সার্ভার প্রতিটি অনুরোধে আলাদা যাচাই করে (ক্লাস ১৭)।

একটি সুক্ষ্ম বাগ

`item.url in request.get_full_path` — সাবস্ট্রিং মিল, তাই কখনো ভুল আইটেমও হাইলাইট হতে পারে।

শিক্ষকের নোট: শেষ কার্ডটি নিয়ে ছাত্রদের ভাবতে দিন: "two" আর "twelve"-এর মতো। সমাধান সাধারণত `startswith` বা সুস্পষ্ট prefix।

ধাপ ৭

খালি তালিকাও একটা পর্দা

১৬টি `{% empty %}` · ৪১টি ফাইলে "empty" — খালি অবস্থা আলাদা করে ভাবা হয়েছে

```
{% for scale in scales %}
  <div class="ims-card">...</div>
{% empty %}
  <div class="ims-card"><div class="ims-empty">
    <i class="bi bi-list-check"></i>
    No grade scale yet.
  </div></div>
{% endfor %}

<tr><td colspan="6" class="ims-hint">
  No invoice matches this filter.
</td></tr>
```

দুটি ভিন্ন বার্তা: **"এখনো কিছু নেই"** আর **"এই ফিল্টারে কিছু নেই"** — ব্যবহারকারীর পরের কাজ আলাদা।

নীতি	কেন
এক আইকন + এক বাক্য	লম্বা ব্যাখ্যা পড়া হয় না
পরের ধাপের লিংক	"নতুন স্কেল" — খালি পর্দা থেকে বেরোনোর পথ
এরর থেকে আলাদা	খালি মানে ভুল নয়, শুধু এখনো কিছু নেই
টেবিলে `colspan`	নাহলে খালি সারি অদ্ভুত দেখায়

সবচেয়ে বড় ভুল: `{% empty %}` না লেখা — ব্যবহারকারী ভাবেন সাইট ভেঙে গেছে।

শিক্ষকের নোট: লাইভে ডেমো: নতুন প্রতিষ্ঠান খুলে খালি ড্যাশবোর্ড দেখান। "প্রথম দিনের ব্যবহারকারী এটাই দেখেন — এটাই আপনার প্রথম ছাপ।"

ধাপ ৮

টেমপ্লেটে লজিক — কতটুকু?

Django ইচ্ছাকৃতভাবে দুর্বল টেমপ্লেট ভাষা দেয়; IMS-এ **০টি** নিজস্ব templatetag

```
{# messages.html — শুধু চেহারা ঠিক করা #}
{% if message.tags == 'success' %}bi-check-circle
{% elif message.tags == 'danger' %}bi-exclamation-octagon
{% elif message.tags == 'warning' %}bi-exclamation-triangle
{% else %}bi-info-circle{% endif %}

{# একই ফাইলে ফিল্টারের কাজ #}
{{ message.tags|default:'info' }}
{{ subscription.days_left }} day{{ subscription.days_left|pluralize }}
{{ request.user.full_name|first|upper }}
{{ brand.color|default:"#4f46e5" }}
```

ফিল্টার	কাজ
`default`	খালি হলে ফলব্যাক রঙ/মান
`pluralize`	"1 day" / "5 days"
`first|upper`	avatar-এর অক্ষর
`date` · `floatformat`	তারিখ ও টাকার অঙ্ক

হিসাব, অনুমতি, কুয়েরি — ভিউতে। টেমপ্লেটে শুধু **দেখানোর সিদ্ধান্ত**।

`templatetags/` ফোল্ডার কোথাও নেই — কারণ যোগ করলে লজিক দুই জায়গায় ছড়িয়ে পড়ে।

শিক্ষকের নোট: প্রশ্ন করুন: "শতাংশের হিসাব টেমপ্লেটে করলে কী ক্ষতি?" উত্তর: টেস্ট লেখা যায় না, আর দুই পর্দায় দুই রকম হিসাব হয়ে যায়।

ধাপ ৯

তিনটি টুকরো, ৩৬টি জায়গায়

`{% include %}` — ৩৬ বার, সবচেয়ে বেশি `field.html`

```
{# partials/field.html #}
<div class="mb-3">
  <label for="{{ field.id_for_label }}">
    {{ field.label }}{% if field.field.required %}
    <span class="text-danger">*</span>{% endif %}
  </label>
  {{ field }}
  {% if field.help_text %}
    <div class="ims-hint">{{ field.help_text }}</div>
  {% endif %}
  {% for error in field.errors %}
    <div class="ims-error">{{ error }}</div>
  {% endfor %}
</div>
```

এক লাইনে ব্যবহার

`{% include "partials/field.html" with field=form.username %}`

লুপেও চলে

গ্রেড স্কেলে: `{% for field in form %}` → প্রতিটি ঘর একই চেহারা।

বাকি দুটি

`messages.html` (ফ্ল্যাশ বার্তা) · `sidebar.html` + `topbar.html` (শেল)।

include বনাম extends

include = ভেতরে বসানো টুকরো (context পায়); extends = উত্তরাধিকার (ব্লক ভরে)। গুলিয়ে ফেলা সাধারণ ভুল।

শিক্ষকের নোট: শেষ কার্ডটা বোর্ডে দুটি ছবি এঁকে বোঝান: একটি বাক্স আরেক বাক্সের ভেতরে (include), বনাম একটি কাঠামো যার ফাঁকা ঘর ভরা হচ্ছে (extends)।

ধাপ ১০

প্রতিষ্ঠানের নিজের রঙ — ৪ লাইনে

`base.html`-এর ইনলাইন `:root` ওভাররাইড — সব ৭৫ পাতায় একসাথে

```
<style>
  :root {
    --ims-brand: {{ brand.color|default:"#4f46e5" }};
  }
  .btn-ims, .ims-nav.is-active,
  .ims-auth__submit { background: var(--ims-brand); }
  .ims-sidebar {
    background: linear-gradient(180deg,
      color-mix(in srgb, var(--ims-brand) 22%, #0b1220),
      #0b1220); }
  a { color: var(--ims-brand); }
</style>

# context processor — brand একটি dict
brand = {"name": row.brand_name, "logo": row.logo.url,
         "color": row.primary_color or "#4f46e5",
         "initial": row.initial, "tagline": row.tagline,
         "login_message": row.login_message,
         "support_email": row.support_email}
```

কী	কোথায় লাগে
`brand.name`	১১ জায়গায় — টাইটেল, সাইডবার, লগইন
`brand.color`	৩ — বাটন, সাইডবার, লিংক
`brand.logo`	৩ — সাইডবার, লগইন, ইনভয়েস
`brand.support_email`	৩ — সহায়তা লিংক
`product.name / short`	৮ — "POWERED BY" লাইন

dict কেন, মডেল কেন নয়? ব্র্যান্ডিং সারি না থাকলে (নতুন ওয়ার্কস্পেস) টেমপ্লেটে **কোনো এররই উঠবে না**।

ঝুঁকি: প্রতিষ্ঠান হালকা রঙ বাছলে সাদা লেখা পড়া যায় না — কনট্রাস্ট ভ্যালিডেশন নেই।

শিক্ষকের নোট: ডেমোতে রঙ বদলে সেভ করুন — পুরো অ্যাপ বদলে যায়। তারপর "লাল রঙ দিলে ডেঞ্জার বাটন আর আলাদা করা যায় না" — এই সীমাটা বলুন।

ধাপ ১১

ব্যানার — সাবস্ক্রিপশনের কণ্ঠস্বর

এক partial, চারটি অবস্থা; কাজের লিংক শুধু `ORG_ADMIN`-এর জন্য

শর্ত	রঙ	ব্যবহারকারী যা দেখেন
`TRIAL` + writable	নীল	"Free trial — N দিন বাকি (plan, ends …)" → Choose a plan
writable নয়, SUSPENDED নয়	হলুদ	"Read-only mode … renew to start editing again" → Renew now
`SUSPENDED`	হলুদ	"Account suspended — data kept and readable" (লিংক নেই, ইচ্ছাকৃত)
সিট ≥ ৯০%	হলুদ	"৪২/৪৫ seats used" → Upgrade

সব ব্যানারের ভেতরে `{% if nav_role == "ORG_ADMIN" %}` — সাধারণ শিক্ষক লিংক দেখেন না, কারণ তিনি কিছু করতে পারেন না।

ব্যানার কী-তে নয়, শেলেই — তাই **৪৪টি পাতার প্রতিটিতে** একই সত্য দেখায়, কেউ ভুলে যায় না।

ব্যানার শুধু জানায়, আটকায় না — লেখা বন্ধ করা হয় মডেল/ভিউ স্তরে (`is_writable`)। UI বার্তা আর নিয়ম আলাদা জিনিস।

শিক্ষকের নোট: `bash run.sh` চালিয়ে ডেটাবেসে ট্রায়াল শেষ করুন — ব্যানার বদলাতে দেখুন। "ব্যবহারকারীকে হঠাৎ অবাক করবেন না" — এটাই এই partial-এর একমাত্র কাজ।

অনুশীলন ও আলোচনা

২০ মিনিট — আর চারটি প্রশ্ন

নিজের প্রজেক্টে তিনটি শেল, তারপর আলোচনা

১. তিনটি শেল (৮ মিনিট)

লগইন · কাজের পর্দা · পাবলিক পাতা — তিনটি base লিখুন, প্রতিটিতে `{% block content %}`।

২. দুটি partial (৭ মিনিট)

`_nav.html` ও `_messages.html` বানিয়ে দুই পাতায় include করুন।

৩. খালি অবস্থা (৫ মিনিট)

একটি লুপে `{% empty %}` যোগ করুন — আইকন + এক বাক্য + পরের ধাপের লিংক।

- একটি ব্লকের নাম বদলালে কেন কোনো এরর আসে না?
- মেনুতে লুকানো থাকলেই কি অনুমতি দেওয়া হয়েছে?
- খালি তালিকা আর এরর — ব্যবহারকারীর কাছে কেন আলাদা?
- ব্র্যান্ড রঙ ব্যবহারকারীর হাতে দিলে কী ঝুঁকি?

শিক্ষকের নোট: তৃতীয় প্রশ্নের সেরা উত্তর: "খালি মানে 'এখনো নয়', এরর মানে 'হয়নি'।" প্রথম প্রশ্নের উত্তর— Django ব্লক না পেলে চুপচাপ খালি রেন্ডার করে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
চারটি শেল, ৬৫টি extends	`app_base` ৪৪ · `base` ১১ · platform ৮ · public ২
এক শেলেই সব সাধারণ জিনিস	`app_base.html` — ১২ লাইন, ৫টি include
চেহারা টোকেনে, যুক্তি ভিউতে	১৫টি CSS ভেরিয়েবল · ০টি templatetag
ভূমিকা বুঝে পর্দা	৩টি জীবিত ড্যাশবোর্ড · `if role in MANAGERS`
খালিও ডিজাইন করা	১৬টি `{% empty %}` · `.ims-empty`
মেনু দেখানো ≠ অনুমতি	`navigation.py` + `@workspace_view`

শিক্ষকের নোট: ষষ্ঠ সারিটা আবার জোর দিন — সাইডবার লুকানো নিরাপত্তা নয়; এটা শুধু ভদ্রতা।

২০

পরের ক্লাস — "প্রমাণ দিয়ে পরিচয়"

অথেন্টিকেশন: লগইন · সেশন · পাসওয়ার্ড রিসেট · ইমেইল বনাম ইউজারনেম · ব্রুট-ফোর্স ও সেশন নিরাপত্তা

পর্ব ৪ · কোডিং · ক্লাস ২০ / ৩৭

প্রমাণ দিয়ে পরিচয়

সিলেবাস: প্র্যাকটিক্যাল ৬ · টপিক ৬.২ (অথেন্টিকেশন)

৯টি অ্যাকাউন্ট রুট · ৯টি টেস্ট · ৪টি পাসওয়ার্ড নিয়ম · ৩টি কাস্টম middleware

AbstractBaseUser · session · নিরাপত্তার স্তর

শুরু করুন প্রশ্ন দিয়ে

লগইনের পর আপনি কোথায় "মনে" থাকেন?

হাত তুলুন: ব্রাউজারে? সার্ভারে? ডেটাবেসে?

ব্রাউজারে থাকে শুধু একটি আইডি — আসল তথ্য সার্ভারের ডেটাবেসে।

তাই কেউ কুকি চুরি করলেও পাসওয়ার্ড পায় না, আর সার্ভার যেকোনো মুহূর্তে সেশন বাতিল করতে পারে। আজকের ক্লাসে এই প্রমাণ-ব্যবস্থার প্রতিটি স্তর — মডেল থেকে কুকি ফ্ল্যাগ পর্যন্ত।

শিক্ষকের নোট: ডিভ-টুলে `sessionid` কুকি দেখান, তারপর ডেটাবেসের `django_session` টেবিল। "এই দুটো মিলেই আপনি লগইন করা" — এটাই সবচেয়ে স্পষ্ট ব্যাখ্যা।

ধাপ ১

প্রথম সিদ্ধান্ত: কাস্টম User মডেল

প্রথম মাইগ্রেশনের **আগে** — নাহলে বদলানো যন্ত্রণার কাজ

```
class User(AbstractBaseUser, PermissionsMixin):
    email = models.EmailField(unique=True, db_index=True)
    full_name = models.CharField(max_length=150)
    phone = models.CharField(max_length=16, blank=True,
                             validators=[phone_validator])
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)
    date_joined = models.DateTimeField(default=timezone.now)

    objects = UserManager()
    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ["full_name"]
```

কেন ডিফল্ট User নয়

ডিফল্টে লগইন আইডি `username` — কিন্তু স্কুলের ব্যবহারকারী মনে রাখে ইমেইল।

ভূমিকা এখানে নেই

একজন মানুষ একাধিক প্রতিষ্ঠানে — তাই ভূমিকা `Membership`-এ, User-এ নয়।

মডেলের ৩টি ক্ষেত্র

Mobile / `full_name` / `is_active` + ডকুমেন্টে `last_login` ও পাসওয়ার্ড হ্যাশ।

দেরি করার খরচ

পরে বদলাতে ৭০টি ForeignKey-এর মাইগ্রেশন — তাই সিদ্ধান্ত দিন এক, দিন শূন্যতে।

শিক্ষকের নোট: IMS-এর মডেল ফাইলের শিরোনামেই লেখা: "never start a Django project with the default User model"। এটা মুখস্থ নিয়ম নয় — ৭০টি FK-এর হিসাব।

ধাপ ২

পাসওয়ার্ড কখনোই সংরক্ষণ নয়

`UserManager` — ইমেইল নরমালাইজ, তারপর `set_password()`

```
def _create_user(self, email, password, **extra):
    if not email:
        raise ValueError("An e-mail address is required.")
    email = self.normalize_email(email).lower()
    user = self.model(email=email, **extra)
    user.set_password(password)   # হ্যাশ করে, প্লেইন নয়
    user.save(using=self._db)
    return user

def create_superuser(self, email, password=None, **extra):
    extra.setdefault("is_staff", True)
    extra.setdefault("is_superuser", True)
    if extra.get("is_staff") is not True:
        raise ValueError("Superuser must have is_staff=True.")
    ...
```

কোন লাইন	কেন
`.lower()`	`Ayesha@Example.COM` → `ayesha@example.com`
`set_password`	PBKDF2 + সল্ট; DB-তে `pbkdf2_sha256$…`
`unique=True`	দুই অ্যাকাউন্ট, এক ইমেইল — অসম্ভব
`is_active=False`	অ্যাকাউন্ট মুছে না, শুধু লগইন বন্ধ

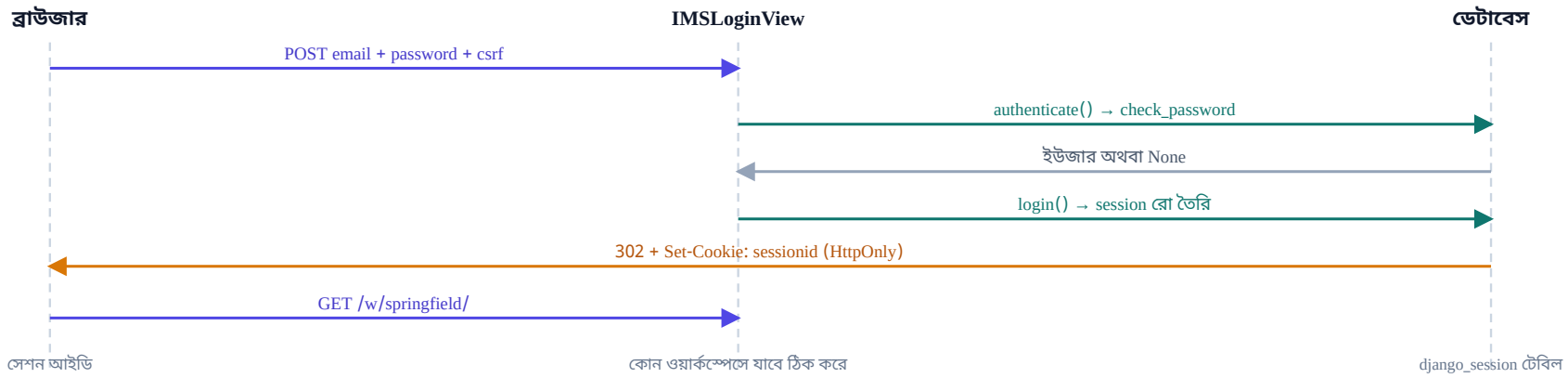
`create_superuser`-এ **স্পষ্ট** `ValueError` — ভুল কনফিগে চুপচাপ এগোয় না, সাথে সাথে থামে।

শিক্ষকের নোট: শেলে `user.password` প্রিন্ট করুন — লম্বা হ্যাশ দেখে ছাত্ররা বোঝে পাসওয়ার্ড আসলে কোথাও নেই। তারপর `check_password`।

ধাপ ৩

লগইনের সাত ধাপ

ফর্ম → `authenticate()` → `login()` → সেশন কুকি → রিডাইরেক্ট



`redirect_authenticated_user = True` — লগইন করা মানুষ লগইন পর্দা আর দেখেন না।

`form_valid`-এ পুরোনো `SESSION_KEY` মুছে দেওয়া হয় — নাহলে আগের প্রতিষ্ঠানেই ঢুকে পড়তেন।

শিক্ষকের নোট: দ্বিতীয় কার্ডটা বাস্তব বাগ থেকে: এক ব্রাউজারে দুই স্কুলের অ্যাডমিন লগইন করলে ভুল স্কুলে ঢুকে যাচ্ছিলেন। "সেশনেও পুরোনো স্মৃতি থাকে।"

ধাপ ৪

একই এরর, দুই কারণে

`IMSAuthenticationForm` — ইচ্ছাকৃতভাবে অস্পষ্ট

```
class IMSAuthenticationForm(AuthenticationForm):
    username = forms.EmailField(label="E-mail",
        widget=forms.EmailInput(attrs={
            "class": "form-control", "autofocus": True,
            "autocomplete": "email"}))
    password = forms.CharField(label="Password",
        widget=forms.PasswordInput(attrs={
            "class": "form-control",
            "autocomplete": "current-password"}))

    error_messages = {
        "invalid_login": "E-mail or password is incorrect.",
        "inactive": "This account is deactivated. "
                    "Contact your administrator.",
    }
```

পরিস্থিতি	ব্যবহারকারী যা দেখেন
ইমেইলই নেই	**"E-mail or password is incorrect."** দুই ক্ষেত্রেই হুবহু এক
ইমেইল আছে, পাসওয়ার্ড ভুল	
অ্যাকাউন্ট নিষ্ক্রিয়	আলাদা বার্তা (মালিককে জানানো দরকার)

কেন? আলাদা বার্তা মানে কেউ একা একা **কোন ইমেইলগুলো নিবন্ধিত** তা বের করে ফেলতে পারে।

`autocomplete`-এ `current-password` — পাসওয়ার্ড ম্যানেজার সঠিকভাবে কাজ করার শর্ত।

শিক্ষকের নোট: প্রশ্ন করুন: "নিষ্ক্রিয় অ্যাকাউন্টের কথা আলাদা করে বলা কি নিরাপদ?" উত্তর: কিছুটা তথ্য ফাঁস হয়, কিন্তু ব্যবহারকারীকে অকারণে দোষারোপ করা আরও খারাপ — এটা সচেতন সমঝোতা।

ধাপ ৫

সেশনের নিয়ম — কোডে লেখা আট লাইন

`config/settings/base.py` ও `prod.py`

সেটিং	মান	মানে
`SESSION_COOKIE_AGE`	৮ ঘণ্টা	এক কর্মদিবস — রাতে নিজেই শেষ
`SESSION_SAVE_EVERY_REQUEST`	True	প্রতিটি অনুরোধে মেয়াদ নবায়ন (কাজ করলে লগআউট নয়)
`EXPIRE_AT_BROWSER_CLOSE`	False	ব্রাউজার বন্ধ করলেই লগআউট হয় না
`SESSION_COOKIE_SECURE`	prod: True	শুধু HTTPS-এ কুকি যায়
`SESSION_COOKIE_HTTPONLY`	prod: True	JavaScript কুকি পড়তে পারে না
`SESSION_COOKIE_SAMESITE`	prod: Lax	অন্য সাইট থেকে পাঠানো POST-এ কুকি যায় না

লগ আউট = সেশন রো মুছে ফেলা + `SESSION_KEY` পপ। বাকি সব কপি তখন অর্থহীন।

এই তিনটি ফ্ল্যাগ **শুধু prod.py-তে** — লোকালে HTTP বলে চালু থাকলে লগইনই হতো না। পার্থক্যের কারণ জানুন।

শিক্ষকের নোট: বলুন: "৮ ঘণ্টা মানে শিক্ষকের এক কর্মদিবস।" সময়সীমা ব্যবসায়িক সিদ্ধান্ত, শুধু কারিগরি নয় — ব্যাংকে ১৫ মিনিট, এখানে ৮ ঘণ্টা।

ধাপ ৬

লগইনের পরে: কোন প্রতিষ্ঠানে?

`CurrentWorkspaceMiddleware` — প্রতিটি অনুরোধে একবার

```
ws_id = request.session.get(SESSION_KEY)   # active_workspace_id
qs = Membership.objects.filter(user=request.user, is_active=True)

if ws_id:
    m = qs.filter(workspace_id=ws_id, workspace__is_active=True).first()
    if m:
        return m

# না থাকলে: সর্বোচ্চ ভূমিকা, তারপর প্রতিষ্ঠান, তারপর নাম
m = (qs.filter(workspace__is_active=True)
       .order_by(ROLE_PRIORITY, KIND_PRIORITY, "workspace__name").first())
if m:
    request.session[SESSION_KEY] = m.workspace_id
return m
```

অগ্রাধিকার	কে আগে
`ROLE_PRIORITY`	ORG_ADMIN → COORDINATOR → TEACHER → STUDENT
`KIND_PRIORITY`	INSTITUTION আগে, PERSONAL পরে
শেষে	নাম অনুযায়ী — ফলাফল নির্ভরযোগ্য

বাস্তব বাগ: এখানে একবার **lazy** অবজেক্ট ফেরানো হয়েছিল। `lazy_obj is None` কখনো True হয় না — ওয়ার্কস্পেসহীন ব্যবহারকারী চেক পাশ করে গিয়েছিলেন।

শিক্ষকের নোট: লাইভে দেখান: এক ব্যবহারকারী দুই প্রতিষ্ঠানে — `bash run.sh` দিয়ে লগইন করে প্রথমে কোনটি খোলে, তারপর switch করে দেখান।

ধাপ ৭

রেজিস্ট্রেশন = তিনটি সারি, একসাথে

`transaction.atomic()` — অর্ধেক অ্যাকাউন্ট কখনো নয়

```
def form_valid(self, form):
    with transaction.atomic():
        user = form.save()
        workspace = Workspace.objects.create(
            kind=WorkspaceKinds.PERSONAL,
            name=form.cleaned_data["workspace_name"],
            owner=user)
        Membership.objects.create(
            user=user, workspace=workspace,
            role=Roles.TEACHER)
    login(self.request, user)
    self.request.session[SESSION_KEY] = workspace.pk
    return redirect("reports:dashboard", slug=workspace.slug)
```

টেস্ট	যা প্রমাণ করে
`test_registration_creates_user_workspace_and_membership`	৩ সারি + সেশন + ৩০২
`test_duplicate_email_is_rejected`	"already exists"
`test_password_mismatch_is_rejected`	"do not match"

টেস্টে ইমেইল `Ayesha@Example.COM` পাঠিয়ে যাচাই করা হয় `ayesha@example.com` সেভ হয়েছে কি না।

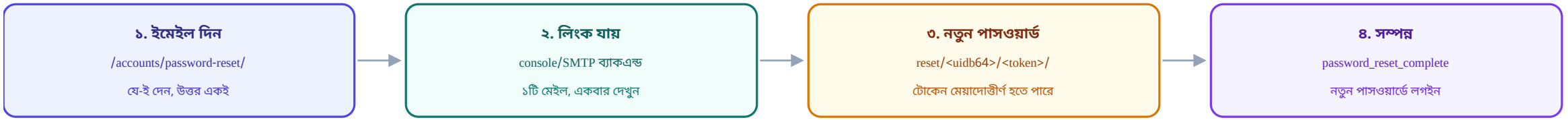
`atomic` না থাকলে দুর্ঘটনায় ব্যবহারকারী তৈরি হয়ে ওয়ার্কস্পেস ছাড়া থেকে যেতে পারে — "লগইন হয়, কিন্তু কোথাও নেই"।

শিক্ষকের নোট: সবুজ টেস্ট দেখান: `pytest apps/accounts` বা `manage.py test apps.accounts`। "দাবি নয়, প্রমাণ" — এই কোর্সের সুর।

ধাপ ৮

পাসওয়ার্ড ভুলে গেলে — চার ধাপ, দুই টোকেন

`uidb64` + `token` — লিংক একবারই কাজ করে, সময়সীমা আছে



টেস্ট: mail.outbox থেকে লিংক তুলে সত্যিকারের পথে হেঁটে নতুন পাসওয়ার্ড সেট করে যাচাই

অজানা ইমেইলেও **একই ৩০২ উত্তর** — কেউ অনুমান করে দেখতে পারে না কে নিবন্ধিত।

ডেমোতে ইমেইল কনসোলে ছাপা হয় (`EMAIL_BACKEND=console`) — প্রোডাকশনে SMTP।

টোকেন কেন দুটি

`uidb64` বলে কে, `token` বলে অনুমতি — শুধু আইডি জানা থাকলে লিংক বানানো যেত।

একবারই

পাসওয়ার্ড সেট হওয়ামাত্র টোকেন অকার্যকর — পুরোনো মেইল আবার ব্যবহার করা যায় না।

মেয়াদ

Django-র ডিফল্ট `PASSWORD_RESET_TIMEOUT` ৩ দিন — দীর্ঘ, কিন্তু সেটাও সচেতন পছন্দ।

শিক্ষকের নোট: `mail.outbox`-এর টেস্টটা দেখান: "মেইল সত্যিই গেছে কি না" — এটাই ওই ফিচারের একমাত্র নির্ভরযোগ্য প্রমাণ, কারণ UI কখনোই জানে না মেইল পৌঁছেছে কি না।

ধাপ ৯

পাসওয়ার্ড বদল — তিনটি ঘর, একটি কঠিন নিয়ম

`PasswordChangeForm` নিজেই পুরোনো পাসওয়ার্ড যাচাই করে

```
class IMSPasswordChangeView(PasswordChangeView):
    template_name = "accounts/password_change.html"
    form_class = IMSPasswordChangeForm
    success_url = reverse_lazy("accounts:profile")

    def form_valid(self, form):
        messages.success(self.request,
                         "Password changed successfully.")
        return super().form_valid(form)

class IMSPasswordChangeForm(PasswordChangeForm):
    old_password = forms.CharField(
        label="Current password",
        widget=forms.PasswordInput(attrs={
            "autocomplete": "current-password"}))
    new_password1 = forms.CharField(
        widget=forms.PasswordInput(attrs={
            "autocomplete": "new-password"}))
```

পুরোনো পাসওয়ার্ড কেন

কেউ চুরি করা সেশন নিয়ে পাসওয়ার্ড বদলে আপনাকে বের করে দিতে পারবে না।

একই নিয়ম আবার

নতুন পাসওয়ার্ডও ৪টি validator পার হতে হয় — ফর্ম বদলালেও নিয়ম বদলায় না।

সেশন টিকে থাকে

পাসওয়ার্ড বদলালে Django সেশন-কি ঘুরিয়ে দেয় — অন্য ডিভাইসের সেশন বাতিল হয় না, কিন্তু কুকি নতুন হয়।

প্রোফাইল আলাদা

`IMSUserProfileForm` — শুধু `full_name` ও `phone`; ইমেইল বদলানো এখানে নেই, ইচ্ছাকৃত।

শিক্ষকের নোট: প্রশ্ন করুন: "প্রোফাইল থেকে ইমেইল বদলানো গেলে কী ঝুঁকি?" উত্তর: লগইন আইডি বদলে যাওয়া + অন্য অ্যাকাউন্টের ইমেইলে দখল — তাই আলাদা, কঠিন প্রবাহ দরকার।

ধাপ ১০

লগইন পর্দাটাই প্রথম ছাপ

দুই কলাম · ব্র্যান্ডের নাম · কুকি গার্ড · ডেমো লগইন

```
{% extends "base.html" %}
<div class="ims-auth">            {# 1.05fr .95fr #}
  <aside class="ims-auth__aside">  {# বাম: ব্র্যান্ড #}
    {{ brand.name|default:"Teacher Management System" }}
    <h1>{{ brand.tagline|default:"Every class, every mark…" }}</h1>
  </aside>
  <main class="ims-auth__main">
    <h2>Sign in{% if brand.name and brand.name != product.short %}
        to {{ brand.name }}{% endif %}</h2>
    {% include "partials/cookie_guard.html" %}
    <form method="post" novalidate>
      {% csrf_token %}
      {% include "partials/field.html" with field=form.username %}
      {% include "partials/field.html" with field=form.password %}</form>
  </main></div>
```

ব্র্যান্ড বদলায় শিরোনাম

প্রতিষ্ঠানের নাম থাকলে "Sign in to Springfield High" — নাহলে শুধু "Sign in"।

কুকি গার্ড

১৫ লাইন JS পরীক্ষা করে কুকি লেখা যায় কি না — না গেলে টাইপ করার আগেই সতর্ক করে।

novalidate কেন

আসল যাচাই সার্ভারে; ব্রাউজারের বাধা শুধুই সুবিধা। এমবেড করা প্রিভিউতেও ফর্ম কাজ করে।

ডেমো লগইন বাক্স

`admin@ · teacher@ · student@ims.local` · `DemoPass!2026` — ক্লাসে সময় বাঁচায়।

শিক্ষকের নোট: কুকি ব্লক করা ব্রাউজারে (বা প্রাইভেট মোড + কঠোর সেটিং) লগইন পর্দা খুলে দেখান — সতর্কবার্তাটা নিজেই বলে দেয় সমস্যা কী।

ধাপ ১১

ছয়টি স্তর, তারপরও একটি ফাঁক

`prod.py`-তে যেগুলো আছে — আর যা এখনো নেই

স্তর	কী আটকায়
`CsrfViewMiddleware` (৩৬ ফর্মে টোকেন)	অন্য সাইট থেকে জাল POST
`SESSION_COOKIE_HTTPONLY`	XSS দিয়ে কুকি চুরি
`SECURE_SSL_REDIRECT` + HSTS ৩০ দিন	HTTP-তে পাসওয়ার্ড যাওয়া
`X_FRAME_OPTIONS = DENY`	ক্লিকজ্যাকিং
পাসওয়ার্ড হ্যাশ + ৪টি validator	দুর্বল পাসওয়ার্ড, ডাম্প/লিক হলে ক্ষতি
prod.py-এর `RuntimeError` গার্ড	ডিফল্ট SECRET_KEY বা `*` হোস্ট দিয়ে চালু হওয়া

যা এখনো নেই: লগইন প্রচেষ্টার সীমা (rate limit) · অ্যাকাউন্ট লকআউট · ইমেইল ভেরিফিকেশন · 2FA। কোডে `ratelimit` / `attempt` শব্দ দুটির খোঁজে **শূন্য** মিল — অর্থাৎ আজ কেউ ১০,০০০ বার পাসওয়ার্ড অনুমান করতে পারবে।

প্রতিকার সাধারণত ক্যাশ-ভিত্তিক কাউন্টার: `(ip, email)` কী-তে ৫ ব্যর্থতায় ১৫ মিনিট ব্লক — মডেল বদলাতে হয় না।

শিক্ষকের নোট: এই স্লাইডটা সৎ থাকুন: "প্রথম সংস্করণে এটাই স্বাভাবিক, কিন্তু প্রকাশের আগে জানা থাকতে হবে।" বাস্তব আক্রমণ: একটি স্কুলের ৪০০ ইমেইলে স্প্রে-হামলা।

অনুশীলন ও আলোচনা

২০ মিনিট — প্রমাণ নিজে পরীক্ষা করুন

তিনটি কাজ, তারপর চারটি প্রশ্ন

১. সেশন কোথায় (৬ মিনিট)

লগইন করে `sessionid` কুকি কপি করুন, তারপর `django_session` টেবিলে ওই কী খুঁজুন।

২. এরর বার্তা (৬ মিনিট)

অস্তিত্বহীন ইমেইল আর ভুল পাসওয়ার্ড — দুইবার লগইন করে দেখুন উত্তর হুবহু এক কি না।

৩. নিষ্ক্রিয় ব্যবহারকারী (৮ মিনিট)

`is_active=False` করে লগইন চেষ্টা করুন; তারপর `apps/accounts/tests.py`-এর টেস্টটি চালান।

- পাসওয়ার্ড কোথায় জমা থাকে — আর কী জমা থাকে না?
- একই এরর বার্তা কেন দুটি ভিন্ন কারণে?
- সেশন কুকি চুরি হলে পাসওয়ার্ডও যায় — সত্যি?
- rate limit না থাকলে প্রথম ২৪ ঘণ্টায় কী হবে?

শিক্ষকের নোট: তৃতীয় প্রশ্নের সঠিক উত্তর: "না — কুকি থাকলে ওই সেশনটাই ব্যবহার করা যায়, পাসওয়ার্ড নয়।" পার্থক্যটা ছাত্ররা প্রায়ই গুলিয়ে ফেলে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
কাস্টম User, প্রথম দিনেই	`USERNAME_FIELD = "email"` · ৭০টি FK
পাসওয়ার্ড কখনো প্লেইন নয়	`set_password()` · `pbkdf2_sha256$…`
এরর বার্তা তথ্য ফাঁস করে না	`invalid_login` এক, দুই কারণে
সেশন = সার্ভারে সত্য, ক্লায়েন্টে আইডি	`django_session` · HttpOnly · ৮ ঘণ্টা
লগইনের পরই ঠিক হয় কোন প্রতিষ্ঠান	`CurrentWorkspaceMiddleware` · ROLE_PRIORITY
যা লেখা হয়নি, তা সুরক্ষাও নয়	**rate limit শূন্য** · লকআউট নেই · 2FA নেই

শিক্ষকের নোট: শেষ সারিটা আবার বলুন: "নিরাপত্তার তালিকা তৈরি করা মানে নিজের ফাঁক জেনে রাখা।" পরের ক্লাসে ফাইল আপলোডেও একই ধরনের ফাঁক দেখবেন।

২১

পরের ক্লাস — "ফাইল ঢোকে, বিপদ ঢোকে না"

ফাইল আপলোড: `MEDIA_ROOT` · `upload_to` · ImageField · SVG/XSS ঝুঁকি · সাইজ ও ধরন যাচাই · ডাউনলোড অনুমতি

পর্ব ৪ · কোডিং · ক্লাস ২১ / ৩৭

ফাইল ঢোকে, বিপদ ঢোকে না

সিলেবাস: প্র্যাকটিক্যাল ৭ · টপিক ৭.২ (ফাইল আপলোড)

২টি ফাইল-ফিল্ড · ৯টি অনুমোদিত এক্সটেনশন · ১০ MB সীমা · ১টি ডাউনলোড ভিউ

MEDIA_ROOT · FileResponse · অনুমতি

শুরু করুন প্রশ্ন দিয়ে

আপলোড মানে তিনটি প্রশ্ন

ডিস্কে কোথায় · কে দেখতে পাবে · কোন ফাইল গ্রহণযোগ্য

প্রথমটি Django করে দেয়। দ্বিতীয়টি আপনার কাজ। তৃতীয়টি কেউ করে দেয় না।

IMS-এ ফাইল আপলোড হয় **এক জায়গায়** — শিক্ষকের লার্নিং ম্যাটেরিয়াল। কিন্তু ওই একটি ফিচারেই স্টোরেজ, অনুমতি, ভাইরাস-সদৃশ ঝুঁকি আর ব্যাকআপ — চারটি ভিন্ন সমস্যা একসাথে আসে।

শিক্ষকের নোট: বলুন: "আপলোড ফিচার মানে শুধু একটা বাটন নয় — এর পেছনে সিদ্ধান্তের সারি।" শিক্ষার্থীর জমা (Submission) শুধু টেক্সট — IMS ইচ্ছাকৃতভাবে ছাত্রদের ফাইল নেয় না।

ধাপ ১

ফাইল কোথায় থাকে — দুই ধরনের ফাইল

`MEDIA_ROOT = BASE_DIR / "media"` · `MEDIA_URL = "media/"`

```
# config/settings/base.py
MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"

# config/urls.py — শুধু ডেভেলপমেন্টে
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
                          document_root=settings.MEDIA_ROOT)

# প্রোডাকশনে nginx এই কাজটি করে, Django নয়
location /media/ { alias /srv/ims/media/; }
```

`DEBUG=False` হলে `static()` কাজ করে না — তাই প্রোডাকশনে "/media/logo.png" ৪০৪।

ফাইল	পথ ও প্রবেশাধিকার
লোগো (`ImageField`)	`media/branding/logos/` — **সর্বজনীন**, লগইন পর্দায় দেখা যায়
ম্যাটেরিয়াল (`FileField`)	`media/materials/2026/11/` — **কখনো সরাসরি নয়**, শুধু ভিউ দিয়ে
স্ট্যাটিক (CSS/JS)	`static/` → `collectstatic` — ডিপ্লয়ের অংশ

লোগো আর পাঠ্য-ম্যাটেরিয়াল এক নয়: একটি ব্র্যান্ডিং, অন্যটি প্রতিষ্ঠানের গোপন সম্পদ। তাই দুই পথ।

শিক্ষকের নোট: ব্রাউজারে `/media/branding/logos/…` খুলে দেখান — লগইন ছাড়াই দেখা যায়। তারপর `/media/materials/…` চেষ্টা করুন: সরাসরি পথ কাজ করে, কিন্তু অ্যাপে সেটি ব্যবহার করাই হয় না।

ধাপ ২

`upload_to` — নামকরণই ভাগ্য নির্ধারণ করে

`materials/%Y/%m/` — তারিখ অনুযায়ী ভাঁজ, সংঘর্ষে Django নিজে নাম বদলায়

```python
class Material(TimeStamped):
    file = models.FileField(upload_to="materials/%Y/%m/")

# ব্যবহারকারী যা আপলোড করল
#   "Chapter 3 (final).pdf"
#
# ডিস্কে যা জমা হলো
#   media/materials/2026/11/Chapter 3 (final).pdf
#
# একই নাম আবার এলে
#   media/materials/2026/11/Chapter 3 (final)_aX9k2.pdf
#
# ডাউনলোডের সময় আসল নাম ফিরিয়ে দেওয়া হয়
return FileResponse(material.file.open("rb"),
                    as_attachment=True,
                    filename=material.file.name.split("/")[-1])
```

তারিখ-ভাঁজ কেন

এক ফোল্ডারে ১০,০০০ ফাইল থাকলে `ls`, ব্যাকআপ আর ডিস্ক-ইনোড দুটোই ভোগে।

সংঘর্ষ নিজেই সমাধান

`get_available_name()` — ৫০ জন একই "notes.pdf" দিলেও কিছু হারায় না।

নাম ব্যবহারকারীর হাতে

Django ফাইলনেম স্যানিটাইজ করে (`get_valid_name`), তাই `../../etc/passwd` লিখেও লাভ নেই।

কিন্তু...

নাম বদলানো মানে **ডাউনলোডে অদ্ভুত নাম** — তাই `filename=` স্পষ্ট দিতে হয় (উপরের কোড)।

শিক্ষকের নোট: লাইভে একই নামে দুইবার আপলোড করুন, তারপর `ls media/materials/` দেখান। ছাত্ররা অবাক হয় — "নাম তো বদলে গেল!" এই জ্ঞানটাই পরে ডাউনলোড বাগ ধরাতে সাহায্য করে।

ধাপ ৩

কোন ফাইল গ্রহণযোগ্য — মডেলেই লেখা

৯টি এক্সটেনশন · ১০ MB · `clean()`-এ যাচাই

```
class Material(TimeStamped):
    ALLOWED_EXTENSIONS = {"pdf", "docx", "xlsx", "pptx", "txt",
                          "jpg", "jpeg", "png", "zip"}
    MAX_BYTES = 10 * 1024 * 1024

    def clean(self):
        ext = (self.file.name.rsplit(".", 1)[-1].lower()
               if self.file else "")
        if ext and ext not in self.ALLOWED_EXTENSIONS:
            raise ValidationError(
                {"file": f"File type .{ext} is not allowed."})
        if self.file and hasattr(self.file, "size") \
                and self.file.size > self.MAX_BYTES:
            raise ValidationError(
                {"file": "File is larger than 10 MB."})
```

নিয়ম কোথায় থাকবে

মডেলে — কারণ সেটাই একমাত্র জায়গা যা **সব পথ** (ফর্ম, শেল, অ্যাডমিন, ম্যানেজমেন্ট কমান্ড) মানে।

`dict` ফেরানো

`ValidationError({"file": …})` — এরর ঠিক ফাইল ঘরে বসে, উপরে নয়।

সেট কেন

খোঁজ O(1); নতুন ধরন যোগ করা এক লাইনের কাজ — তালিকা পড়তেও সহজ।

সীমা

এটি শুধু **নামের শেষ অংশ** দেখে — ভেতরে কী আছে তা নয় (পরের স্লাইড)।

শিক্ষকের নোট: `.exe` বা `.html` আপলোড করে দেখান — ভদ্র এরর, ক্র্যাশ নয়। তারপর বলুন: "ভদ্র এররও একটি ফিচার।"

ধাপ ৪

যাচাই কে, কখন করে?

`MaterialForm.save(commit=False)` → `obj.clean()` → `obj.save()`

```
@workspace_view(*TEACHING)
def material_create(request, slug):
    form = MaterialForm(request.POST or None,
                        request.FILES or None,
                        workspace=request.workspace)
    if request.method == "POST" and form.is_valid():
        obj = form.save(commit=False)
        obj.workspace = request.workspace       # স্কোপ ভিউ থেকে
        obj.uploaded_by = request.user
        obj.clean()                             # আবার যাচাই
        obj.save()
        messages.success(request, "Material uploaded.")
        return redirect("lessons:material_list", slug=slug)
```

স্তর	কে ডাকে
`form.is_valid()`	ফিল্ড-ধরন, required, plus মডেলের `clean()`
`obj.clean()`	ভিউ নিজে — **workspace বসানোর পরে**
`obj.save()`	`clean()` ডাকে না — এটাই ফাঁদ

`request.FILES` না দিলে ফাইল কখনো আসে না — কিন্তু এররও আসে না, শুধু "This field is required"।

দ্বিতীয় `clean()` এখানে বাড়তি সতর্কতা — কারণ `workspace` সেট হওয়ার আগে মডেল-যাচাই চলেছিল।

শিক্ষকের নোট: `save()` নিজে যাচাই করে না — এটা মুখস্থ করার মতো নিয়ম: "save() বিশ্বাস করে, clean() যাচাই করে।" শেলে `Material(...).save()` চালিয়ে দেখান: বিশাল ফাইলও ঢুকে যায়।

ধাপ ৫

ডাউনলোড — IMS-এর সবচেয়ে গুরুত্বপূর্ণ ভিউ

"ফাইল কখনো সর্বজনীন URL থেকে দেওয়া হয় না"

```
@workspace_view(*ALL_ROLES)
def material_download(request, slug, pk):
    """Files are never served from a public URL."""
    material = get_object_or_404(Material, pk=pk,
                                 workspace=request.workspace)
    allowed = request.membership.role in (
        "ORG_ADMIN", "COORDINATOR", "TEACHER") or (
        teacher_can_access_class(request.user, material.class_group))
    student = getattr(request.user, "student_profile", None)
    if student:
        allowed = material.is_published and student.enrollments.filter(
            class_group=material.class_group, is_active=True).exists()
    if not allowed:
        raise PermissionDenied("You cannot download this file.")
    try:
        return FileResponse(material.file.open("rb"),
                            as_attachment=True,
                            filename=material.file.name.split("/")[-1])
    except FileNotFoundError:
        raise Http404("The file is missing on disk.")
```

কে	নিয়ম
শিক্ষক/কোঅর্ডিনেটর/অ্যাডমিন	নিজের স্কোপের ক্লাস হলে হ্যাঁ
ছাত্র	**প্রকাশিত** + নিজের ক্লাসে ভর্তি
অন্য প্রতিষ্ঠানের সদস্য	`get_object_or_404` → 404
লগইনহীন	decorator → লগইনে রিডাইরেক্ট

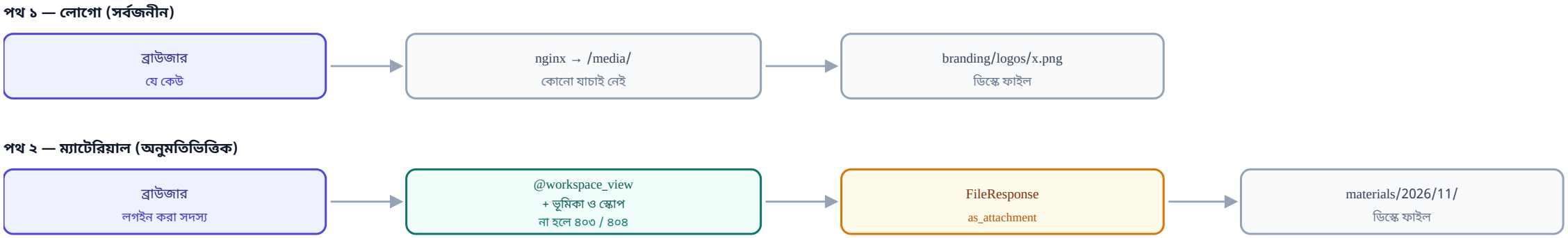
`as_attachment=True` — ব্রাউজারে খোলে না, ডাউনলোড হয়। HTML ফাইল খুলে গেলে XSS-এর দরজা।

শিক্ষকের নোট: ছাত্র হিসেবে লগইন করে অপ্রকাশিত ম্যাটেরিয়ালের URL হাতে চেষ্টা করুন — ৪০৩ আসবে। তারপর বলুন: "নিয়মটি UI-তে নয়, এখানে।"

ধাপ ৬

দুই পথ, এক প্রশ্ন: কে ঢুকতে পারে?

সর্বজনীন লোগো বনাম অনুমতিভিত্তিক ম্যাটেরিয়াল



নিয়ম: ব্যবহারকারীর দেওয়া যেকোনো ফাইল সবসময় পথ ২ দিয়ে যাবে — পথ ১ শুধু ব্র্যান্ডিং ছবির জন্য।

পরীক্ষা: ছাত্রের অ্যাকাউন্টে `/media/materials/…` সরাসরি খুললে ফাইল **দেখা যাবে** (nginx জানে না কে কে)। তাই অ্যাপে ওই URL কখনো তৈরি হয় না — এই সীমাটা জানা থাকা জরুরি।

শিক্ষকের নোট: এই স্লাইডটা সততার: "এখনো পর্যন্ত সুরক্ষা নির্ভর করে URL গোপন থাকার ওপর, যা নিরাপত্তা নয়।" প্রোডাকশনে করণীয়: media ফোল্ডার nginx-এ `internal` করে X-Accel-Redirect ব্যবহার।

ধাপ ৭

এক্সটেনশন যথেষ্ট নয়

তিন স্তরের যাচাই — নাম · ধরন · বিষয়বস্তু

স্তর	যা দেখা হয়	প্রতারণা
এক্সটেনশন (IMS-এ **আছে**)	`.pdf` `.docx` …	`report.pdf.exe` — শেষ অংশ `exe`, আটকাবে
MIME ধরন	`file.content_type`	ক্লায়েন্ট যা বলে, সেটাই — মিথ্যা বলা সহজ
ম্যাজিক বাইট (অনুপস্থিত)	ফাইলের প্রথম কয়েক বাইট	`%PDF` দিয়ে শুরু? — এটাই আসল প্রমাণ

SVG ভেতরে JavaScript ধারণ করতে পারে — IMS-এ SVG অনুমোদিত নয় (`ALLOWED_EXTENSIONS`-এ নেই), লোগোতেও ঝুঁকি (ক্লাস ১৮)।

Zip bomb: ১০ MB zip খুললে ১০ GB হতে পারে — `.zip` অনুমোদিত, কিন্তু সার্ভারে কেউ খোলে না, তাই ঝুঁকি কম।

ম্যাজিক বাইট যোগ করা যায় ছোটকিছু দিয়ে: প্রথম ৪ বাইট পড়ে `%PDF` কি না মিলিয়ে দেখা — `kind="magic"`।

শিক্ষকের নোট: লাইভ ডেমো: একটি `.txt` ফাইলের নাম বদলে `.pdf` করে আপলোড করুন — পাশ করবে। "এক্সটেনশন মিথ্যা বলতে পারে, তাই এর ওপর ভরসা সীমিত।"

ধাপ ৮

পাঁচটি ফাঁদ — আর প্রতিকার

প্রতিটি বাস্তব প্রজেক্টে দেখা যায়

ফাঁদ	যা ঘটে / প্রতিকার
১. `request.FILES` না দেওয়া	ফাইল সেভ হয় না, এররও আসে না → `MaterialForm(request.POST, request.FILES)`
২. ফাইল মুছলে ডিস্কে থেকে যায়	DB সারি গেলে ফাইল অনাথ → `delete()` override বা পরিষ্কার কমান্ড
৩. একই ফাইল দুবার আপলোড	ডিস্কে দুই কপি, ব্যাকআপ ফুলে ওঠে → হ্যাশ মিলিয়ে ডিডুপ্লিকেশন
৪. ওয়েব সার্ভারে `MEDIA_ROOT` নয়	ডেপ্লয়ে ফাইল যায়, কিন্তু ছবি ভাঙা → মিডিয়া ভলিউম/অবজেক্ট স্টোরেজ
৫. `DEBUG=False`-এ `static()` ভরসা	লোকালে চলে, সার্ভারে ৪০৪ → nginx অ্যালিয়াস বা S3

পাঁচটি ফাঁদের চারটিই "লোকালে কাজ করে, সার্ভারে ভাঙে" শ্রেণির। ডেপ্লয়ের আগে একবার সত্যিকারের সার্ভারে আপলোড → ডাউনলোড পরীক্ষা করুন।

শিক্ষকের নোট: দ্বিতীয় ফাঁদটা বোর্ডে আঁকুন: DB সারি মুছে গেল, অথচ ৩০০ MB PDF ডিস্কে পড়ে আছে। "ব্যাকআপ নিলে এই ভূতও ব্যাকআপ হয়।"

ধাপ ৯

ফাইল আপলোডের টেস্ট কীভাবে লিখবেন

`SimpleUploadedFile` — ব্রাউজার ছাড়াই

```
from django.core.files.uploadedfile import SimpleUploadedFile

def test_rejects_exe_and_accepts_pdf(self):
    self.client.force_login(self.teacher)
    url = reverse("lessons:material_create",
                  kwargs={"slug": self.ws.slug})

    bad = SimpleUploadedFile("notes.exe", b"MZ\x90\x00",
                             content_type="application/pdf")
    self.client.post(url, {"title": "X", "class_group": self.cls.pk,
                           "file": bad})
    self.assertFalse(Material.objects.exists())

    good = SimpleUploadedFile("notes.pdf", b"%PDF-1.4 ok",
                              content_type="application/pdf")
    self.client.post(url, {"title": "X", "class_group": self.cls.pk,
                           "file": good})
    self.assertEqual(Material.objects.count(), 1)
```

এখন IMS-এ কয়টি?

শূন্য — `apps/lessons/tests.py`-এ ৪টি টেস্ট, সবই হোমওয়ার্ক ও সিলেবাস নিয়ে; আপলোডের একটিও নেই।

কেন বিপজ্জনক

ডাউনলোডের অনুমতি-নিয়মটাই সবচেয়ে জটিল কোড, অথচ সেটাই অপরীক্ষিত।

যা লেখা উচিত

৩টি: বড় ফাইল প্রত্যাখ্যান · অপ্রকাশিত ফাইলে ছাত্রের ৪০৩ · অন্য প্রতিষ্ঠানের ৪০৪।

মিডিয়া ফোল্ডার

টেস্ট চলাকালে `MEDIA_ROOT` অস্থায়ী ডিরেক্টরিতে দিন — নাহলে টেস্ট ফাইল রিপোতে জমে।

শিক্ষকের নোট: "যেটা টেস্ট করা হয়নি, সেটা কাজ করে — এই বিশ্বাসটাই ভাঙে।" ৫ মিনিটে ওই তিনটি টেস্ট নিজে লিখিয়ে নিন; এটাই সবচেয়ে মূল্যবান অনুশীলন।

ধাপ ১০

৩০০ MB ভিডিও পাঠালে কী হয়?

পাঁচটি সীমা, পাঁচটি ভিন্ন এরর — ব্যবহারকারী শুধু "ব্যর্থ" দেখেন

স্তর	ডিফল্ট	ফলাফল
nginx `client_max_body_size`	১ MB	**৪১৩** — Django পর্যন্ত পৌঁছায়ই না
`DATA_UPLOAD_MAX_MEMORY_SIZE`	২.৫ MB	POST ডেটা বড় হলে `RequestDataTooBig`
`FILE_UPLOAD_MAX_MEMORY_SIZE`	২.৫ MB	এর বেশি হলে RAM-এর বদলে টেম্প ফাইলে
`Material.MAX_BYTES`	১০ MB	ভদ্র এরর: "File is larger than 10 MB."
ডিস্ক	—	জমা হলে ব্যাকআপ সময় ও জায়গা দুটোই বাড়ে

nginx-এর সীমা ডিফল্ট ১ MB — তাই ১০ MB-র নিয়ম থাকলেও ব্যবহারকারী **পাঁচ MB PDF-তে ৪১৩** পাবেন, যদি `client_max_body_size 20m;` না লেখা হয়।

সীমাগুলো **একই সংখ্যায়** মিলিয়ে দিন (nginx ২০ MB ⊃ অ্যাপ ১০ MB) — নাহলে দুটি ভিন্ন বার্তায় ব্যবহারকারী বিভ্রান্ত।

শিক্ষকের নোট: "ভিডিও আপলোড চাইলে উত্তর 'না' — কারণ ৩০০ MB × ৪০০ শিক্ষক = প্রতি মাসে Terra-bytes।" আপলোড নীতি প্রায়ই স্টোরেজ-নীতি, শুধু কোড নয়।

ধাপ ১১

মিডিয়া ব্যাকআপের একটি নীরব ফাঁদ

`ops/backup.sh` — DB ডাম্প + মিডিয়া টার, ১৪ দিনের রিটেনশন

```
#!/usr/bin/env bash
set -euo pipefail
STAMP=$(date +%Y%m%d-%H%M%S)
DEST="${BACKUP_DIR:-./backups}"
mkdir -p "$DEST"
pg_dump -Fc -f "$DEST/ims-$STAMP.dump"
tar -czf "$DEST/media-$STAMP.tar.gz" media/ 2>/dev/null || true
find "$DEST" -name '*.dump' -mtime +14 -delete
find "$DEST" -name '*.tar.gz' -mtime +14 -delete
```

`|| true` — মিডিয়া টার ব্যর্থ হলেও স্ক্রিপ্ট **সফল** দেখায়। ব্যাকআপ আছে ভেবে দিন, অথচ ফাইলগুলো নেই।

DB + ফাইল একসাথে

শুধু `pg_dump` নিলে সারি ফিরে আসে, ফাইল ফেরে না — সব ডাউনলোড ৪০৪।

১৪ দিন কেন

দুই সপ্তাহের বেশি পুরোনো ব্যাকআপ কেউ খোলে না — কিন্তু ভুল ধরা পড়ে ৩ দিনে।

প্রতিকার

`|| true` বাদ → ব্যর্থ হলে স্ক্রিপ্ট **থামুক**; নাহলে মনিটরিং কখনো জানবে না।

ImageField আলাদা

Pillow দরকার (requirements-এ আছে); উচ্চতা/প্রস্থ পড়তে চাইলে `width_field`।

শিক্ষকের নোট: প্রশ্ন করুন: "৫০০ MB মিডিয়া, ১ KB ডেটাবেস — কোনটা বেশি ঝুঁকিতে?" উত্তর: মিডিয়া, কারণ সেটার কোনো ট্রানজ্যাকশন-লগ নেই।

অনুশীলন ও আলোচনা

২০ মিনিট — এক ফিচার, চার পরীক্ষা

নিজের প্রজেক্টে একটি আপলোড যোগ করুন

১. মডেল (৫ মিনিট)

`FileField(upload_to="files/%Y/%m/")` + `ALLOWED_EXTENSIONS` + `clean()`।

২. ভিউ (৭ মিনিট)

ডাউনলোড ভিউ: `get_object_or_404(..., workspace=…)` + ভূমিকা যাচাই + `FileResponse`।

৩. প্রত্যাখ্যান (৫ মিনিট)

১১ MB ফাইল ও `.exe` — দুটোই আটকাচ্ছে কি না দেখুন।

৪. টেস্ট (৩ মিনিট)

`SimpleUploadedFile` দিয়ে গ্রহণ + প্রত্যাখ্যান, দুটোই।

- লোগো ও পাঠ্য-ফাইল — দুই পথ কেন?
- এক্সটেনশন যাচাই কেন অসম্পূর্ণ?
- `save()` কেন `clean()` ডাকে না?
- ফাইল ডিলিট হলে ডিস্কে কী থাকে?

শিক্ষকের নোট: তৃতীয় প্রশ্নের উত্তর: "কারণ `save()` কে ডাকছে তা জানে না — শেল, মাইগ্রেশন, কমান্ডও ডাকে।" তাই যাচাই সবসময় আলাদা স্তরে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
ফাইল ডিস্কে, DB-তে শুধু পথ	`MEDIA_ROOT` · `FileField` নাম রাখে
`upload_to`**-এ তারিখের ভাঁজ**	`materials/%Y/%m/` · `branding/logos/`
নিয়ম মডেলে, বার্তা ফর্মে	`ALLOWED_EXTENSIONS` · `MAX_BYTES` · `clean()`
ডাউনলোড সবসময় অনুমতি-যাচাই করা	`material_download` · `FileResponse`
এক্সটেনশন ≠ প্রমাণ	ম্যাজিক বাইট যাচাই এখনো **নেই**
অপরীক্ষিত পথ = অজানা পথ	আপলোড টেস্ট **০টি** · ডাউনলোড নিয়ম অপরীক্ষিত

শিক্ষকের নোট: শেষ দুই সারিতেই আজকের আসল শিক্ষা: "যা করিনি, তা লিখে রাখুন" — শুধু গর্বের তালিকা নয়, ফাঁকের তালিকাও।

২২

পরের ক্লাস — "সংখ্যা থেকে সিদ্ধান্ত"

রিপোর্ট ও এক্সপোর্ট: `annotate`/`aggregate` · রেজাল্ট শিট · CSV-তে BOM · pagination · বড় ডেটাসেট

পর্ব ৪ · কোডিং · ক্লাস ২২ / ৩৭

সংখ্যা থেকে সিদ্ধান্ত

সিলেবাস: প্র্যাকটিক্যাল ৭ · টপিক ৭.৩ (রিপোর্ট ও এক্সপোর্ট)
৭টি রিপোর্ট রুট · ৩টি CSV · ১টি PDF · ১১টি অ্যাসেসমেন্ট টেস্ট

aggregate · Decimal · BOM · marksheet

শুরু করুন প্রশ্ন দিয়ে

রিপোর্টে সবচেয়ে কঠিন অংশ কোনটি?

হাত তুলুন: কুয়েরি? চার্ট? PDF? নাকি অনুমতি?

উত্তর: হিসাব আর অনুমতি — কুয়েরি Django শিখিয়ে দেয়, কিন্তু "কার ফল দেখা যাবে" সেটা কেউ বলে দেয় না।

আজকের পথ: একটি নম্বর → শতাংশ (Decimal) → গ্রেড (band lookup) → সারসংক্ষেপ → CSV/PDF। প্রতিটি ধাপে একটি করে সিদ্ধান্ত, আর প্রতিটির পাশে টেস্ট।

শিক্ষকের নোট: বলুন: "রিপোর্টে বাগ মানে ভুল নম্বর, আর ভুল নম্বর অভিভাবককে ফোন করেন।" তাই এখানে টেস্ট সবচেয়ে বেশি দরকার — অথচ `apps/reports/tests.py` **নেই**; হিসাব টেস্ট করা আছে assessments-এ।

ধাপ ১

১১টি টেস্ট — হিসাবের প্রমাণ

`apps/assessments/tests.py` — প্রতিটি নিয়মের পাশে একটি টেস্ট

টেস্ট	যা রক্ষা করে
`test_grade_band_lookup`	শতাংশ → গ্রেড
`test_overlapping_bands_are_rejected`	দুই ব্যান্ড একই নম্বরে নয়
`test_result_sheet_handles_absent_zero_and_boundaries`	অনুপস্থিত · শূন্য · সীমানা
`test_marks_entry_rejects_out_of_range_value`	পূর্ণমানের বেশি নয়
`test_results_are_invisible_until_published`	প্রকাশের আগে লুকানো
`test_student_cannot_see_another_students_marksheet`	নিজের ফল ছাড়া নয়
`test_marksheet_pdf_is_rendered`	PDF সত্যিই তৈরি হয়

এই সাতটি দেখলেই বোঝা যায় —**রিপোর্ট মানে শুধু যোগফল নয়**; সীমা, অনুমতি আর প্রকাশের সময়ও হিসাবের অংশ।

`apps/reports/`-এ টেস্ট **০** — CSV-র কলাম ক্রম, ফাইলের নাম, BOM — কিছুই অপরীক্ষিত।

শিক্ষকের নোট: টেস্টের নাম পড়েই ছাত্ররা অনেক সময় ধারণা পেয়ে যায় কোডে কী আছে। বলুন: "ভালো টেস্টের নাম একটা বাক্যের ডকুমেন্টেশন।"

ধাপ ২

নম্বর থেকে গ্রেড — একটাই ফাংশন

`grade_for(percent, scale)` — সব পর্দা, CSV ও PDF একই ফাংশন ডাকে

```
def grade_for(percent, scale):
    """Resolve a percentage to (letter, gpa)."""
    if scale is None or percent is None:
        return ("-", Decimal("0.00"))
    p = Decimal(str(percent))
    for band in scale.bands.all():        # ordering = -min_percent
        if band.min_percent <= p <= band.max_percent:
            return (band.letter, band.gpa)
    return ("-", Decimal("0.00"))

class GradeBand(models.Model):
    letter = models.CharField(max_length=5)
    min_percent = models.DecimalField(max_digits=5, decimal_places=2)
    max_percent = models.DecimalField(max_digits=5, decimal_places=2)
    gpa = models.DecimalField(max_digits=3, decimal_places=2)
```

সিদ্ধান্ত	কারণ
এক ফাংশন, তিন ভোক্তা	স্ক্রিন · CSV · PDF-এ গ্রেড কখনো আলাদা হয় না
`Decimal(str(p))`	`75.0` → `"75.0"` — বাইনারি ভুল এড়ায়
ব্যান্ড না মিললে `"-"`	ক্র্যাশ নয়; ফাঁকা গ্রেড দেখানো
একই workspace-এ ১টি active scale	partial unique constraint (ক্লাস ১৫)
ওভারল্যাপ নিষিদ্ধ	নাহলে কোন ব্যান্ড আগে মিলবে তা অনিশ্চিত

শিক্ষকের নোট: "দুই ব্যান্ড একই নম্বরে মিললে কী হয়?" — প্রথমটা জেতে, মানে ডেটার ক্রম ফলাফল বদলায়। ওভারল্যাপ-টেস্ট এটাই আটকায়।

ধাপ ৩

রেজাল্ট শিট — বিশুদ্ধ ফাংশন

`build_result_sheet(exam)` — কোনো ডেটাবেস লেখা নেই, তাই টেস্ট করা সহজ

```
for mark in exam.marks.select_related("student"):
    obtained = None if mark.is_absent else mark.obtained
    percent = None
    if obtained is not None and exam.full_marks:
        percent = (Decimal(obtained) * 100 / Decimal(exam.full_marks)
                   ).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
    letter, gpa = grade_for(percent, scale)
    rows.append({"student": mark.student, "obtained": obtained,
                 "percent": percent, "letter": letter, "gpa": gpa,
                 "is_pass": obtained is not None
                            and obtained >= exam.pass_marks})

scored = [r for r in rows if r["obtained"] is not None]
summary = {"students": len(rows), "scored": len(scored),
           "absent": ..., "passed": ..., "failed": ...,
           "pass_rate": round(passed * 100 / total_scored, 1),
           "average": ..., "highest": ..., "lowest": ...}
```

অনুপস্থিত আলাদা

`obtained=None` — শূন্য নয়! নাহলে গড় আর পাশের হার দুটোই ভুল হবে।

`scored` তালিকা

গড়-পাশ কেবল যারা পরীক্ষা দিয়েছে তাদের নিয়ে — `total_scored`।

ডেটাবেস লেখে না

পিওর ফাংশন মানে একই ইনপুটে একই আউটপুট — টেস্টে `self.assertEqual` দিয়ে ধরা যায়।

শূন্য দিয়ে ভাগ

কেউ পরীক্ষা দেয়নি? `if total_scored else 0` — নাহলে `ZeroDivisionError`।

শিক্ষকের নোট: বোর্ডে হিসাব করুন: ১০ জনের মধ্যে ২ জন অনুপস্থিত, বাকি ৮ জনের গড় ৬০ — "ক্লাসের গড় ৬০ নাকি ৪৮?" এই আলোচনাই রিপোর্টের রাজনীতি।

ধাপ ৪

`Decimal`, `float` নয়

টাকার হিসাব আর শতাংশে বাইনারি ভুল মেনে নেওয়া যায় না

```
# float — যা ঘটে
>>> 0.1 + 0.2
0.30000000000000004
>>> round(2.675, 2)
2.67          # প্রত্যাশা 2.68

# Decimal — যা চাই
Decimal("2.675").quantize(Decimal("0.01"),
                          rounding=ROUND_HALF_UP)   # 2.68

# IMS-এ একই নিয়ম
percent = (Decimal(obtained) * Decimal("100")
           / Decimal(exam.full_marks)).quantize(
               Decimal("0.01"), rounding=ROUND_HALF_UP)
```

কোথায় Decimal

`GradeBand.min/max/gpa` · ইনভয়েসের `price/amount` · শতাংশ

`ROUND_HALF_UP` কেন

Python-এর ডিফল্ট banker's rounding (`2.5 → 2`) — স্কুলের নম্বরে অপ্রত্যাশিত।

সীমানার ঝুঁকি

৩২.৯৯% vs ৩৩% — পাশ/ফেল ঠিক এই জায়গায় নির্ধারিত হয়, তাই `quantize` আগে, তুলনা পরে।

CSV-তে

`Decimal` সরাসরি লেখা যায় না → `str(percent)` ; নাহলে `csv` মডিউল TypeError দেয়।

শিক্ষকের নোট: টার্মিনালে `0.1+0.2` চালিয়ে দেখান — ক্লাসে সবসময় একই "ওহ!" আসে। তারপর বলুন: "ব্যাংকে ১ পয়সা, এখানে ১ নম্বর।"

ধাপ ৫

CSV-র চারটি লাইন যা সবাই ভুলে যায়

`_csv_response()` — এক ফাংশন, তিনটি এক্সপোর্ট

```
def _csv_response(filename, header, rows):
    resp = HttpResponse(content_type="text/csv; charset=utf-8")
    resp["Content-Disposition"] = (
        f'attachment; filename="{filename}"')
    resp.write("\ufeff")          # BOM so Excel opens UTF-8
    writer = csv.writer(resp)
    writer.writerow(header)
    for row in rows:
        writer.writerow(row)
    return resp
```

লাইন	না থাকলে
`charset=utf-8`	বাংলা নাম মুজিবকে দেখায়
`\ufeff` (BOM)	Excel-এ বাংলা অক্ষর ভাঙে — শুধু Excel-এ, LibreOffice-এ নয়
`attachment`	ব্রাউজারে কাঁচা টেক্সট খুলে যায়
`csv.writer`	নামে কমা থাকলে কলাম সরে যায় (`"Rahman, Jr."`)

ফাইলের নামেও তথ্য: `attendance-Class-8-2026-10-06-to-2026-11-05.csv` — খুলেই বোঝা যায় কী।

শিক্ষকের নোট: BOM-টা চোখে দেখা যায় না, তাই লাইভে Excel-এ খুলে দেখান। "যে কোডটা দেখা যায় না, সেটাই সবচেয়ে বেশি ভুল হয়।"

ধাপ ৬

তিনটি এক্সপোর্ট, তিনটি স্কোপ

একই `_students_for()` / `_classes_for()` — স্ক্রিনে যা, CSV-তে তাই

ফাইল	সোর্স	কলাম ও নিয়ম
`students.csv`	`_students_for()` + `q` / `status` ফিল্টার	ভর্তি নং · নাম · ক্লাস · অভিভাবক · ফোন · স্ট্যাটাস
`attendance-…csv`	`_classes_for()` + ৩০ দিনের ডিফল্ট রেঞ্জ	মোট · উপস্থিত · অনুপস্থিত · শতাংশ
`results-{exam}.csv`	`build_result_sheet(exam)`	নম্বর · পূর্ণমান · % · গ্রেড · পাশ/ফেল

তিনটিই `@workspace_view(*TEACHING)` — ছাত্র CSV নামাতে পারেন না। UI-তে বাটন নেই, সার্ভারেও নিয়ম আছে।

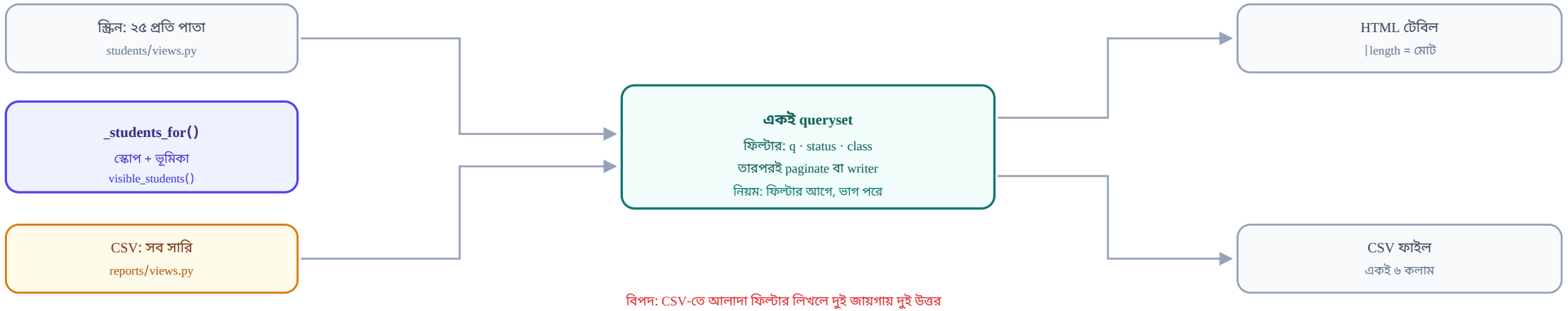
`export_results`-এ আলাদা করে `if role == "STUDENT": raise PermissionDenied` — বাড়তি সতর্কতা, কারণ `TEACHING`-এ STUDENT নেই।

শিক্ষকের নোট: "রিপোর্ট = আরেকটি দরজা" — এই বাক্যটাই ক্লাসের কেন্দ্র। প্রতিটি এক্সপোর্ট নতুন করে অনুমতি লিখলে একদিন কেউ ভুলে যাবে; তাই একই helper।

ধাপ ৭

স্ক্রিন আর CSV — একই পথ

দুই আলাদা কুয়েরি মানে দুই রকম সত্য



স্ক্রিনে ২৫টি দেখালেও CSV-তে **সব** যায় — কিন্তু দুটোই একই কুয়েরি থেকে, তাই "স্ক্রিনে যা দেখিনি CSV-তে তা এল" এমন বিভ্রান্তি হয় না।

শিক্ষকের নোট: প্রশ্ন করুন: "CSV-তে pagination থাকলে কী হয়?" উত্তর: ব্যবহারকারী প্রথম ২৫ জনের রিপোর্ট ভেবে বাকিদের বাদ দিয়ে ফেলেন — বিপজ্জনক নীরব ভুল।

ধাপ ৮

শতাংশের হিসাব ডেটাবেসেই

শর্তসহ `Count` — ৪০ জনের জন্য ৪০টি কুয়েরি নয়

```
agg = (AttendanceRecord.objects
       .filter(class_group=cls, date__range=(start, end))
       .values("student__admission_no", "student__full_name")
       .annotate(
           total=Count("id"),
           present=Count("id", filter=Q(status__in=["PRESENT", "LATE"])),
           absent=Count("id", filter=Q(status="ABSENT")))
       .order_by("student__full_name"))

rows = [(a["student__admission_no"], a["student__full_name"],
         a["total"], a["present"], a["absent"],
         round(a["present"] * 100 / a["total"]) if a["total"] else 0)
        for a in agg]
```

`values()` আগে

গ্রুপিং হয় ছাত্র ধরে; না দিলে পুরো ক্লাসের একটাই সারি আসবে।

`filter=Q(…)`

তিনটি সংখ্যা এক কুয়েরিতে — উপস্থিত, অনুপস্থিত, মোট।

শতাংশ Python-এ

SQL-এ ভাগ করলে `ZeroDivisionError` বা পূর্ণসংখ্যার ভাগ — তাই শেষ ধাপে, `if total else 0` দিয়ে।

বিকল্প

`annotate` ছাড়া প্রতিটি ছাত্রে ৩টি কুয়েরি = ১২০ কুয়েরি (N+1)।

শিক্ষকের নোট: `django.db.connection.queries` চালিয়ে দুই সংস্করণের কুয়েরি সংখ্যা দেখান — ১ বনাম ১২০। সংখ্যাটাই এখানে যুক্তি।

ধাপ ৯

রিপোর্টের ফিল্টার = ব্যবহারকারীর প্রশ্ন

ডিফল্ট মান লিখে রাখা মানে ব্যবহারকারীর হয়ে ভেবে নেওয়া

```
q = (request.GET.get("q") or "").strip()
if q:
    qs = qs.filter(full_name__icontains=q)
if request.GET.get("status"):
    qs = qs.filter(status=request.GET["status"])

today = timezone.localdate()
start = (request.GET.get("from")
         or (today - timedelta(days=30)).isoformat())
end = request.GET.get("to") or today.isoformat()
filter(class_group=cls, date__range=(start, end))

cls = (_classes_for(request)
       .filter(pk=request.GET.get("class")).first()
       or _classes_for(request).first())
```

নীতি	কারণ
ডিফল্ট ৩০ দিন	খালি রিপোর্টের চেয়ে "গত মাস" বেশি কাজে লাগে
`timezone.localdate()`	সার্ভার UTC, স্কুল ঢাকা — তারিখ গড়মিল এড়ায়
`?class=` না মিললে প্রথম ক্লাস	কখনো ভাঙা পর্দা নয়, তবে **ভুল ক্লাসও হতে পারে**
ফিল্টার আগে, ভাগ পরে	নাহলে টোটাল আর সারি মেলে না

`?class=999` (অন্যের ক্লাস) দিলে `_classes_for()`-এ না মিলে **প্রথম ক্লাস** দেখাবে — ৪০৩ নয়। ব্যবহারকারী বিভ্রান্ত হতে পারেন।

শিক্ষকের নোট: শেষ কার্ডটা নিয়ে আলোচনা করুন: নীরব ফলব্যাক ভালো UX, কিন্তু ভুল রিপোর্টের ঝুঁকি। বিকল্প: "এই ক্লাসে আপনার প্রবেশাধিকার নেই" বার্তা।

ধাপ ১০

মার্কশিট — একই ডেটা, দুই চেহারা

HTML পর্দা · PDF (`reportlab`) · অনুমতি একই

```
@workspace_view(*ALL_ROLES)
def marksheet(request, slug, pk):
    exam = get_object_or_404(Exam, pk=pk, workspace=request.workspace)
    student = getattr(request.user, "student_profile", None)
    if student:
        if not exam.is_published:
            raise PermissionDenied("Results are not published yet.")
        mark = exam.marks.filter(student=student).first()
        if mark is None:
            raise PermissionDenied("You have no result for this exam.")
        return _render_sheet(request, exam, [mark], own=True)
    if not teacher_can_access_class(request.user, exam.class_group):
        raise PermissionDenied("You are not assigned to this class.")
    return _render_sheet(request, exam, exam.marks..., own=False)

# ?format=pdf → render_marksheet_pdf(...)
resp["Content-Disposition"] = 'inline; filename="marksheet-3.pdf"'
```

এক ভিউ, দুই ফরম্যাট

`?format=pdf` — একই অনুমতি, একই সারি; শুধু আউটপুট বদলায়।

`inline` বনাম `attachment`

মার্কশিট ব্রাউজারে খোলা দরকার (প্রিন্ট), তাই `inline` ; CSV নামা দরকার, তাই `attachment` ।

শিক্ষকের নিয়ম

শুধু নিজের ক্লাসের — `teacher_can_access_class()` , নাহলে ৪০৩।

ছাত্রের নিয়ম

প্রকাশিত হতে হবে + নিজের সারি। দুই শর্তের একটি বাদ পড়লে ফল ফাঁস।

PDF লাইব্রেরি

`reportlab` — টেবিল নিজে আঁকে (`TableStyle`), হেডার প্রতি পাতায় (`repeatRows=1`)।

শিক্ষকের নোট: HTML বনাম PDF-এর পার্থক্য দেখান: ব্রাউজারের Ctrl+P-ও কাজ করে, কিন্তু প্রতিষ্ঠানের হেডার-লোগোসহ PDF আলাদা পণ্য — অভিভাবককে পাঠানোর জিনিস।

সতর্কতা

রিপোর্টের পাঁচটি ফাঁদ

প্রতিটির একটি করে বাস্তব চিহ্ন

ফাঁদ	চিহ্ন ও প্রতিকার
১. অনুপস্থিতকে শূন্য গোনা	গড় কমে যায় → `obtained=None` রাখুন, আলাদা গোনা
২. `float` দিয়ে শতাংশ	৩২.৯৯ / ৩৩ সীমানায় দোলাচল → `Decimal` + `quantize`
৩. CSV-তে pagination	অর্ধেক রিপোর্ট নীরবে যায় → রপ্তানিতে সব সারি
৪. মৃত কোড রিপোর্টে	`class_summary()`-তে `for … if False else []` — কখনো চলে না, পড়তে বিভ্রান্তিকর
৫. এক্সপোর্টে আলাদা অনুমতি	নতুন রুটে স্কোপ ভুলে যাওয়া → একই `_students_for()`

চতুর্থটি দেখতে ছোট, কিন্তু ভয়ংকর: পরের ডেভেলপার ভাববেন ওই লুপ কিছু একটা করে — আর সেটাকে "ঠিক" করতে গিয়ে নতুন বাগ আনবেন।

শিক্ষকের নোট: চতুর্থটা লাইভে খুলে দেখান — `if False else []` লেখাটা কেউ বিশ্বাসই করবে না। শিক্ষা: "মৃত কোড মুছে ফেলা একটা ফিচার।"

অনুশীলন ও আলোচনা

২০ মিনিট — একটি রিপোর্ট, চার শর্ত

নিজের প্রজেক্টে একটি CSV রিপোর্ট

১. স্কোপ (৫ মিনিট)

একই helper থেকে কুয়েরি নিন — নতুন ফিল্টার লিখবেন না।

২. শর্তসহ গণনা (৮ মিনিট)

`values()` + `annotate(Count(..., filter=Q(...)))` — এক কুয়েরিতে ২টি সংখ্যা।

৩. CSV (৫ মিনিট)

BOM + `charset` + `attachment`; বাংলা নাম দিয়ে Excel-এ খুলে যাচাই।

৪. সীমানা (২ মিনিট)

০ সারি ও ০ দিয়ে ভাগ — দুটোই পরীক্ষা করুন।

- অনুপস্থিত ছাত্র গড়ে ঢুকলে কী ভুল হয়?
- CSV-তে BOM কেন দরকার?
- রিপোর্টে আলাদা অনুমতি লিখলে কী ঝুঁকি?
- PDF আর HTML — কখন কোনটি?

শিক্ষকের নোট: চতুর্থ প্রশ্নের ভালো উত্তর: "PDF হলো যা পাঠানো যায়, HTML হলো যা ব্যবহার করা যায়।" অভিভাবকের কাছে PDF, শিক্ষকের কাছে পর্দা।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
হিসাব একটি বিশুদ্ধ ফাংশনে	`build_result_sheet()` · `grade_for()`
অনুপস্থিত ≠ শূন্য	`obtained=None` · `scored` তালিকা
টাকায় ও নম্বরে `Decimal`	`quantize(…, ROUND_HALF_UP)`
CSV = BOM + charset + attachment	`_csv_response()` · ৩টি এক্সপোর্ট
শর্তসহ গণনা এক কুয়েরিতে	`Count("id", filter=Q(…))`
স্ক্রিন ও রপ্তানি একই অনুমতি	`_students_for()` / `_classes_for()`

শিক্ষকের নোট: প্রথম ও শেষ সারি জোড়া লাগান: "হিসাব ঠিক থাকলেও ভুল মানুষকে দিলে রিপোর্ট ভুল।"

২৩

পরের ক্লাস — "মাস শেষে টাকার হিসাব"

বিলিং ও প্ল্যাটফর্ম: `Subscription` · `Invoice._next_number` · read-only গেট · ট্রায়াল · সাসপেন্ড · প্ল্যাটফর্ম কনসোল

পর্ব ৪ · কোডিং · ক্লাস ২৩ / ৩৭ · পর্বের শেষ ক্লাস

মাস শেষে টাকার হিসাব

সিলেবাস: প্র্যাকটিক্যাল ৮ · টপিক ৮.১ (সাবস্ক্রিপশন ও বিলিং)

১৮টি বিলিং রুট · ৩৬টি টেস্ট · ২টি middleware · ৫টি সাবস্ক্রিপশন অবস্থা

is_writable · Invoice · read-only গেট

শুরু করুন প্রশ্ন দিয়ে

প্রতিষ্ঠান টাকা না দিলে ডেটা মুছে ফেলবেন?

হাত তুলুন: হ্যাঁ / না / ৩০ দিন পর

উত্তর: কিছুই মুছবে না — শুধু লেখা বন্ধ, পড়া খোলা।

স্কুলের ছাত্র-নম্বর-হাজিরা কারও বিল বাকির জিম্মায় নয়। এই একটি নৈতিক সিদ্ধান্ত থেকেই জন্মেছে `is_writable`, read-only গেট, আর "ডেটা নিরাপদ, শুধু নবায়ন করুন" বার্তা।

শিক্ষকের নোট: বলুন: "আপনার প্রোডাক্টের সবচেয়ে মানবিক কোডটি সাধারণত বিলিং ফাইলে থাকে।" এটা ক্লাসের সুর।

ধাপ ১

`Plan` — একটি দাম, একটি সারি

১৩টি ফিল্ড; সিড করা ৩টি প্ল্যান: starter · growth · campus

```
class Plan(models.Model):
    class Interval(models.TextChoices):
        MONTHLY = "MONTHLY", "Monthly"
        YEARLY = "YEARLY", "Yearly"

    code = models.SlugField(max_length=40, unique=True)
    name = models.CharField(max_length=80)
    price = models.DecimalField(max_digits=10,
                                decimal_places=2,
                                validators=[MinValueValidator(Decimal("0"))])
    interval = models.CharField(choices=Interval.choices)
    currency = models.CharField(max_length=3, default="BDT")
    max_students = models.PositiveIntegerField(null=True, blank=True)
    max_staff = models.PositiveIntegerField(null=True, blank=True)
    features = models.TextField(help_text="One feature per line.")
    trial_days = models.PositiveSmallIntegerField(default=14)
    is_public / is_active / sort_order

    def feature_list(self):
        return [l.strip() for l in self.features.splitlines() if l.strip()]
```

`null=True` মানে অসীম

`max_students=None` → "Unlimited students" (স্কুল প্ল্যান)।

টেক্সটে ফিচার

প্রতি লাইনে একটি — নতুন ফিচার যোগে মাইগ্রেশন লাগে না, শুধু লেখা বদলায়।

`is_active` বনাম `is_public`

পুরোনো গ্রাহক `is_active`-এ চলবে; নতুন বিক্রি বন্ধ হয় `is_public=False`-এ।

দাম `Decimal`

৳১৫০০.০০ — float নয়; ইনভয়েস আর রিপোর্টে একই সংখ্যা।

শিক্ষকের নোট: "প্ল্যান বদলালে পুরোনো গ্রাহকের দাম বদলে যাবে" — `Subscription`-এ `plan` FK রাখা হয়েছে, দাম কপি নয়; কিন্তু ইনভয়েসে দাম কপি হয় (পরে দেখা যাবে কেন)।

ধাপ ২

`Subscription` — প্রতিষ্ঠানপ্রতি একটি

`OneToOneField(Workspace)` · ৫টি অবস্থা · `seats_override`

```python
class Subscription(models.Model):
    class Status(models.TextChoices):
        TRIAL = "TRIAL", "Trial"
        ACTIVE = "ACTIVE", "Active"
        PAST_DUE = "PAST_DUE", "Payment overdue (read-only)"
        SUSPENDED = "SUSPENDED", "Suspended by platform"
        CANCELLED = "CANCELLED", "Cancelled"

    workspace = models.OneToOneField(Workspace,
        on_delete=models.CASCADE, related_name="subscription")
    plan = models.ForeignKey(Plan, on_delete=models.PROTECT)
    status = models.CharField(choices=Status.choices,
                              default=Status.TRIAL)
    trial_ends_on / current_period_start / current_period_end
    seats_override = models.PositiveIntegerField(null=True, blank=True)
```

সিদ্ধান্ত	কারণ
`OneToOne`	এক প্রতিষ্ঠানের দুই সাবস্ক্রিপশন মানে দ্বিগুণ বিল — অসম্ভব করে দেয় DB
`plan` -এ `PROTECT`	চালু গ্রাহকের প্ল্যান মুছে ফেলা যাবে না
`seats_override`	"৩০০ সিটের প্ল্যানে ৪০০" — দরকষাকষির জায়গা কোডে
পার্সোনাল ওয়ার্কস্পেস	কোনো সাবস্ক্রিপশনই নেই — `sub is None → writable`

> ব্যক্তিগত শিক্ষকের ওয়ার্কস্পেস কখনো গেটে আটকায় না — এটাও একটি টেস্টে লেখা (`test_personal_workspace_is_never_gated`)।

শিক্ষকের নোট: প্রশ্ন: "সাবস্ক্রিপশন কে বানায় — প্রতিষ্ঠান নাকি প্ল্যাটফর্ম?" উত্তর: `start_trial()` / `provision_institution()` — ব্যবহারকারী নিজে নয়।

ধাপ ৩

একটি property, পাঁচটি অবস্থা

`is_writable` — পুরো প্রোডাক্টের সবচেয়ে দামি ১০ লাইন



নিয়ম (কোডের ক্রমেই):

১. PAST_DUE · SUSPENDED · CANCELLED → False (সবসময়)
২. TRIAL → trial_ends_on খালি বা আজকের সমান/পরে হলে True
৩. ACTIVE → current_period_end একই নিয়ম · ৪. বাকি সব → False

মেয়াদোত্তীর্ণ trial = PAST_DUE-এর মতো:
ডেটা হারায় না, শুধু লেখা বন্ধ।
একটি property · সব ভিউ ও ব্যানার একই সত্য মানে

তারিখ আসে `timezone.localdate()` থেকে — সার্ভার UTC-তে থাকলেও স্কুলের "আজ" ঢাকার তারিখ।

শিক্ষকের নোট: বোর্ডে জিজ্ঞেস করুন: "TRIAL যার trial_ends_on=None — লেখা চালু না বন্ধ?" কোড বলে চালু (নিরাপদ দিক), কারণ `is None` চেক স্পষ্ট লেখা আছে।

ধাপ ৪

একটি তারিখ, চারটি প্রশ্ন

`ends_on` · `days_left` · `state_label` · `seat_percent`

```
@property
def ends_on(self):
    return (self.trial_ends_on if self.status == self.Status.TRIAL
            else self.current_period_end)

@property
def days_left(self):
    end = self.ends_on
    return None if end is None else (end - timezone.localdate()).days

@property
def state_label(self):
    if self.is_writable:
        return "Trial" if self.status == self.Status.TRIAL else "Active"
    return ("Expired" if self.status in (self.Status.TRIAL, self.Status.ACTIVE)
            else self.get_status_display())
```

প্রশ্ন	কোথায় উত্তর
"কবে শেষ?"	ব্যানার · ইনভয়েসের period_end
"কত দিন বাকি?"	`days_left` → "N দিন বাকি"
"কী অবস্থা?"	`state_label` — ব্যবহারকারীর ভাষায়
"সিট কতটা ভরা?"	`seat_percent` → ৯০% হলে সতর্কতা

`state_label`-এ "Expired" কৃত্রিম শব্দ — DB-তে নেই। ব্যবহারকারীকে "PAST_DUE" দেখানো যায় না, তাই অনুবাদ স্তর।

শিক্ষকের নোট: বলুন: "স্টেটাস কোড নয়, বাক্য দেখান।" `TRIAL` বনাম "ট্রায়াল চলছে, ৯ দিন বাকি" — ব্যবহারকারীর ভাষা আলাদা।

ধাপ ৫

গেট — লেখা আটকায়, পড়া নয়

`SubscriptionGateMiddleware` — সব ভিউয়ের আগে একবার

```
ALLOW_PREFIXES = ("/accounts/", "/renew/", "/platform/",
                  "/static/", "/media/", "/admin/", "/healthz",
                  "/workspaces/")
SAFE_METHODS = ("GET", "HEAD", "OPTIONS", "TRACE")

def __call__(self, request):
    if request.method in SAFE_METHODS:
        return self.get_response(request)
    workspace = self._workspace_in_url(request) or request.workspace
    if workspace is None or request.path.startswith(ALLOW_PREFIXES):
        return self.get_response(request)
    if is_writable(workspace):
        return self.get_response(request)
    # ORG_ADMIN → নবায়ন পর্দা; বাকিরা → হোম + বার্তা
    if membership.role == Roles.ORG_ADMIN:
        return redirect(reverse("billing:renew", kwargs={"slug": ...}) + "?blocked=1")
    messages.warning(request, "This institution's subscription has ended … "
                              "Ask your institution administrator to renew it — "
                              "your work is not lost.")
    return redirect(f"/w/{workspace.slug}/")
```

কেন middleware

ভিউ-তে লিখলে ৫৭টি ডেকোরেটরের সাথে আরেকটি জিনিস মনে রাখতে হতো।

GET কেন ছাড়

রিপোর্ট, হোমওয়ার্ক, মার্কশিট — সব পড়া যায়; ডেটা তাদেরই।

দুই ধরনের বার্তা

অ্যাডমিন নবায়ন পর্দায় (তিনিই ঠিক করতে পারেন), শিক্ষক হোমে — বিলিং তথ্য ফাঁস নয়।

বাস্তব বাগ

প্রথমে `request.workspace` (সেশন) দেখা হতো — এক শিক্ষক ব্যক্তিগত ওয়ার্কস্পেসে থাকা অবস্থায় মেয়াদোত্তীর্ণ স্কুলে POST করতে পারতেন। সমাধান: URL-এর slug।

শিক্ষকের নোট: বাগটা বোর্ডে আঁকুন: এক ব্যবহারকারী, দুই প্রতিষ্ঠান, সেশন একটায়, URL অন্যটায়। "নিরাপত্তার নিয়ম সবসময় URL-কে বিশ্বাস করবে, সেশনকে নয়।"

ধাপ ৬

গেটের সীমা — কী ছাড় পায়, কেন

ভুল ছাড় মানে ব্যবহারকারী আটকে যান; ভুল বন্ধ মানে গেট অর্থহীন

পথ	অবস্থা	কারণ
`/accounts/`	সবসময় খোলা	লগইন/লগআউট/পাসওয়ার্ড — নাহলে বেরোনোর পথই নেই
`/renew/`	সবসময় খোলা	সমস্যার সমাধান করতে হবে, বন্ধ করলে চিরকাল আটকে
`/platform/`	সবসময় খোলা	প্ল্যাটফর্ম অ্যাডমিন তিনিই বিল ঠিক করেন
`/media/` · `/static/`	সবসময় খোলা	ছবি/CSS বন্ধ হলে পর্দাই ভাঙে
`/w/<slug>/…`	লেখা বন্ধ	আসল গেট — উপস্থিতি, নম্বর, হোমওয়ার্ক

সুপারইউজার বাইপাস নেই — স্কুলের অ্যাডমিনও যদি প্ল্যাটফর্ম অ্যাডমিন হন, তিনিও মেয়াদ শেষে ওই স্কুলে লিখতে পারবেন না।

কনসোলের কাজ চলে `/platform/` ছাড়ে — তাই গেট নিজের পায়ে কুড়াল মারে না।

শিক্ষকের নোট: প্রশ্ন করুন: "ছাড়ের তালিকায় `/api/` থাকলে কী হবে?" — ভবিষ্যতে API যোগ করলে এই তালিকা আবার ভাবতে হবে; এটাই middleware-এর ঝুঁকি।

ধাপ ৭

`activate()` — টাকা, তারিখ, ইনভয়েস

এক ফাংশন, তিনটি লেখা, সবই `transaction.atomic`-এ

```python
@transaction.atomic
def activate(workspace, plan, *, days=None, amount=None,
             method=Invoice.Method.BKASH, reference="", actor=None):
    sub = ensure_subscription(workspace)
    today = timezone.localdate()
    start = today
    if (sub.current_period_end and sub.current_period_end > today
            and sub.plan_id == plan.pk):
        start = sub.current_period_end      # আগেভাগে নবায়ন: দিন হারায় না
    end = start + timedelta(days=days) if days else period_end(start, plan)

    sub.plan, sub.status = plan, Subscription.Status.ACTIVE
    sub.current_period_start, sub.current_period_end = start, end
    sub.save()

    return Invoice.objects.create(
        workspace=workspace, subscription=sub, plan=plan,
        period_start=start, period_end=end,
        amount=amount if amount is not None else plan.price,
        status=Invoice.Status.PAID, method=method,
        reference=reference[:80], paid_on=today, recorded_by=actor)
```

আগেভাগে নবায়ন

১৫ দিন বাকি থাকলে নতুন মেয়াদ শুরু হয় পুরোনোর শেষ থেকে — `test_early_renewal_extends_instead_of_resetting`।

ইনভয়েস = প্রমাণ

`status=PAID`, `paid_on`, `recorded_by` — কে, কখন, কত, কীভাবে।

দাম কপি হয়

প্ল্যানের দাম পরে বাড়লেও পুরোনো ইনভয়েস বদলায় না — আর্থিক নথি অপরিবর্তনীয়।

atomic কেন

সাবস্ক্রিপশন ACTIVE হলো কিন্তু ইনভয়েস লেখা ব্যর্থ — এমন অর্ধেক অবস্থা কখনো নয়।

শিক্ষকের নোট: "প্রমাণ ছাড়া টাকা নেওয়া বন্ধ করুন" — ইনভয়েস সংখ্যা, তারিখ আর রেফারেন্স তিনটিই থাকতে হবে; নাহলে পরে অভিযোগ মেটানো যায় না।

ধাপ ৮

ইনভয়েস নম্বর — ক্রম, এবং একটুখানি ঝুঁকি

`IMS-2026-0001` · `_next_number()` · `unique=True`

```python
def save(self, *args, **kwargs):
    if not self.number:
        self.number = self._next_number()
    super().save(*args, **kwargs)

@staticmethod
def _next_number() -> str:
    year = timezone.localdate().year
    prefix = f"{settings.INVOICE_PREFIX}-{year}-"
    last = (Invoice.objects.filter(number__startswith=prefix)
            .order_by("-number").values_list("number", flat=True).first())
    n = int(last.split("-")[-1]) + 1 if last else 1
    return f"{prefix}{n:04d}"
```

নীতি	কারণ
বছর prefix-এ	প্রতি বছর ০০০১ থেকে — হিসাবরক্ষকের প্রত্যাশা
`:04d`	সাজালে সংখ্যাগত ক্রম ঠিক থাকে (`-0009` < `-0010`)
`unique=True`	ডুপ্লিকেট নম্বর DB-ই আটকায়
শুধু খালি হলে তৈরি	সম্পাদনায় নম্বর বদলায় না

ঝুঁকি: দুইজন একসাথে ইনভয়েস বানালে দুজনেই একই "পরের নম্বর" পড়তে পারেন → একজন `IntegrityError` পান। সমাধান: `select_for_update()` বা ফাঁদে পড়ে রিট্রাই।

এখন ঝুঁকিটা ছোট কারণ ইনভয়েস বানান শুধু দুটি জায়গায় — তবু জানা থাকা দরকার।

শিক্ষকের নোট: দুটি টার্মিনাল খুলে একসাথে দুটি ইনভয়েস বানানোর চেষ্টা করুন — কখনো একটি ব্যর্থ হবে। "DB constraint হলো শেষ রক্ষা, প্রথম নয়।"

ধাপ ৯

সিট সীমা — সিদ্ধান্ত কোথায় নেওয়া হয়

`seat_block_reason()` → ৪র্থ ছাত্র যোগ করলে সার্ভারই না বলে

```
def seat_block_reason(workspace) -> str | None:
    sub = subscription_for(workspace)
    if sub is None:
        return None                 # ব্যক্তিগত ওয়ার্কস্পেস
    allowed = sub.seats_allowed     # seats_override or plan.max_students
    if allowed is None:
        return None                 # অসীম
    if sub.used_students() >= allowed:
        return (f"Your plan allows {allowed} active students. "
                f"Upgrade to add more.")
    return None

# ভিউতে (ছাত্র যোগের আগেই)
reason = seat_block_reason(request.workspace)
if reason:
    messages.error(request, reason)
    return redirect("students:list", slug=slug)
```

নিয়ম	প্রমাণ / কারণ
গণনা আসল সারি থেকে	`StudentProfile.objects.filter(status=ACTIVE).count()`
নিষ্ক্রিয় ছাত্র গোনায় নয়	পাশ করা ছাত্র সিট ছেড়ে দেয়
`seats_override` আগে	চুক্তি প্ল্যানের চেয়ে বড় হতে পারে
UI নয়, সার্ভার	`test_server_refuses_the_third_student`

> বার্তাটি কর্মের সাথে জোড়া: "Upgrade to add more" — শুধু "সীমা শেষ" নয়।

শিক্ষকের নোট: লাইভে সীমা ছুঁইয়ে দেখান: তৃতীয় ছাত্র যোগ করতে গেলে ভদ্র বার্তা; URL দিয়ে চেষ্টা করলেও একই। "নিয়ম UI-তে নয়, সার্ভারে।"

ধাপ ১০

কাস্টম ডোমেইন — প্রতিষ্ঠানের নিজের ঠিকানা

`CustomDomainMiddleware` · শুধু `domain_verified=True`

```python
class CustomDomainMiddleware:
    def __call__(self, request):
        request.domain_workspace = None
        host = request.get_host().split(":")[0].lower()
        branding = (WorkspaceBranding.objects
                    .select_related("workspace")
                    .filter(custom_domain__iexact=host,
                            domain_verified=True,
                            workspace__is_active=True)
                    .first())
        if branding is not None:
            request.domain_workspace = branding.workspace
            request.branding = branding
            if host not in settings.ALLOWED_HOSTS:
                settings.ALLOWED_HOSTS = list(settings.ALLOWED_HOSTS) + [host]
        response = self.get_response(request)
        patch_vary_headers(response, ["Host"])   # ক্যাশ আলাদা রাখে
        return response
```

ALLOWED_HOSTS কেন বদলায়

প্রোডাকশনে `*` নিষিদ্ধ — তাই যাচাই করা ডোমেইন রানটাইমে যোগ হয়, শুধু সেইগুলো।

চুরি করা যায় না

`test_domain_cannot_be_stolen_by_a_second_institution` — এক ডোমেইন এক প্রতিষ্ঠানের।

যাচাই ছাড়া কিছু নয়

`domain_verified=False` হলে উপেক্ষিত — `test_unverified_domain_is_ignored`।

Vary: Host

নাহলে CDN এক স্কুলের লগইন পর্দা আরেক স্কুলকে দেখাতে পারে।

শিক্ষকের নোট: বাস্তবে ডোমেইন যাচাই DNS TXT রেকর্ড দিয়ে হয় — কোডে শুধু `domain_verified` ফ্ল্যাগ; প্রক্রিয়াটা প্ল্যাটফর্ম অ্যাডমিনের হাতে।

ধাপ ১১

প্ল্যাটফর্ম কনসোল — টেন্যান্সির বাইরে

১৮টি রুট · শুধু সুপারইউজার · URL-এ কোনো `slug` নেই

```
/platform/                          dashboard
/platform/institutions/             তালিকা
/platform/institutions/new/         নতুন প্রতিষ্ঠান
/platform/institutions/<pk>/        বিস্তারিত
/platform/institutions/<pk>/activate/   পেমেন্ট + সক্রিয়
/platform/institutions/<pk>/toggle/     সক্রিয়/নিষ্ক্রিয়
/platform/institutions/<pk>/<action>/   suspend · past_due · …
/platform/plans/                    প্ল্যান তালিকা + নতুন
/platform/invoices/                 ইনভয়েস
/platform/signups/                  সাইনআপ অনুরোধ
```

নিয়ম	কারণ
URL-এ slug নেই	এটি কোনো প্রতিষ্ঠানের ভেতরের পর্দা নয়
`is_platform_admin`	`is_staff or is_superuser` — একটাই চেক
প্রতিষ্ঠান তৈরি	`provision_institution()` — ওয়ার্কস্পেস + অ্যাডমিন + ট্রায়াল, একসাথে
সাইনআপ অনুরোধ	ব্যর্থ হলেও অনুরোধ রাখা হয় (`SignupRequest`)

কনসোলে ভুল ক্লিক মানে ৪০০ স্কুলের বিল — তাই প্রতিটি কাজ `AuditLog`-এ লেখা হয় এবং কোনো ডিলিট বাটন নেই।

শিক্ষকের নোট: ডেমোতে `admin@ims.local` দিয়ে কনসোল দেখান: এক নতুন প্রতিষ্ঠান তৈরি করে ট্রায়াল শুরু করুন — পুরো প্রবাহ ৬০ সেকেন্ডে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
টাকা না দিলে ডেটা মুছবে না	`is_writable` · read-only · ৩৬টি টেস্ট
একটি property, সব পর্দায় একই সত্য	`ends_on` · `days_left` · `state_label`
গেট middleware-এ, ভিউয়ে নয়	`SubscriptionGateMiddleware` · `ALLOW_PREFIXES`
URL-এর স্কোপ, সেশনের নয়	`_workspace_in_url()` — বাস্তব বাগের সমাধান
টাকা আর নথি একসাথে	`activate()` · `transaction.atomic`
নম্বর অপরিবর্তনীয়	`IMS-2026-0001` · `unique` · `PROTECT`

শিক্ষকের নোট: প্রথম সারিটা আবার জোর দিন: "বিলিং কোড লেখার সময় আপনি ঠিক করেন আপনার প্রোডাক্ট কেমন মানুষ।"

পর্ব ৪ সমাপ্তি

আট ক্লাসে যা তৈরি হলো

১৫২ স্লাইড · ৩১টি মডেল · ৮৫টি রুট · ১০৩টি টেস্ট

ক্লাস	মূল ধারণা	দখল
১৬ · মডেল লেখা	২৩২ ফিল্ড · `clean()` · constraint	ডেটার আকৃতি
১৭ · ভিউ ও URL	`@workspace_view` · ৪০৩/৪০৪ · PRG	অনুরোধের পথ
১৮ · ফর্ম	হোয়াইটলিস্ট · স্কোপড queryset · `add_error`	ভুল ইনপুট
১৯ · টেমপ্লেট	৪টি শেল · partial · হোয়াইট-লেবেল	পর্দা
২০ · অথেন্টিকেশন	কাস্টম User · সেশন · রিসেট	পরিচয়
২১ · ফাইল আপলোড	`upload_to` · `clean()` · `FileResponse`	বাইনারি ডেটা
২২ · রিপোর্ট	`Decimal` · CSV BOM · মার্কশিট PDF	সিদ্ধান্ত
২৩ · বিলিং	`is_writable` · গেট · ইনভয়েস	টাকার নিয়ম

একই সুর আটবার: **নিয়ম সার্ভারে, প্রমাণ টেস্টে, বার্তা ব্যবহারকারীর ভাষায়** — এটাই পর্ব ৪-এর সারমর্ম।

শিক্ষকের নোট: এই টেবিলটা ছাত্রদের ডায়েরিতে লিখিয়ে নিন — পর্ব ৫ (টেস্টিং) শুরু হওয়ার আগে নিজেদের কোডে এই আটটি জায়গা খুঁজে দেখতে বলুন।

২৪

পর্ব ৫ শুরু — "যা পরীক্ষা করা হয়নি, তা নেই"

টেস্টিং: `TestCase` · `setUpTestData` · `force_login` · অনুমতির টেস্ট · টেস্ট পিরামিড · ১০৩টি টেস্ট থেকে শেখা

পর্ব ৫ · টেস্টিং · ক্লাস ২৪ / ৩৭

যা পরীক্ষা করা হয়নি, তা নেই

সিলেবাস: প্র্যাকটিক্যাল ৯ · টপিক ৯.১ (ইউনিট টেস্টিং)

১০৩টি টেস্ট · ১,৩৫৩ লাইন · ২২টি টেস্ট ক্লাস · ১৬টি `setUpTestData`

TestCase · setUpTestData · subTest

শুরু করুন প্রশ্ন দিয়ে

এক ঘণ্টা বেশি লাগিয়ে টেস্ট লিখব কেন?

হাত তুলুন: কে কখনো টেস্ট লিখেছেন? কে চালিয়েছেন?

কারণ একটি বাগ দুবার আসে না — যদি একবার লিখে রাখেন "এটা কাজ করে"।

IMS-এ **১০৩টি টেস্ট** আছে — সবচেয়ে বেশি বিলিংয়ে (৩৬), তারপর core-এ (১৯)। আজ শিখব এই টেস্টগুলোর গঠন, আর কেন কিছু ফাইলে **শূন্য** টেস্ট আছে।

শিক্ষকের নোট: বলুন: "টেস্ট লেখা সময় নয়, সময় বাঁচানো।" কিন্তু প্রথম সপ্তাহে ব্যয়টা বেশি লাগে — এটা স্বীকার করুন, নাহলে ছাত্ররা বিশ্বাস করবে না।

ধাপ ১

১০৩টি টেস্ট, ১১টি অ্যাপ

প্রতি অ্যাপে একটি `tests.py` — মোট ১,৩৫৩ লাইন

অ্যাপ	টেস্ট	লাইন
billing	**৩৬**	৪৩৬
core	১৯	২৪৯
assessments	১১	১৫৫
workspaces	১০	১১৮
accounts	৯	১১৪
students	৫	৬৮
attendance	৫	৮৫
academics · lessons	৪ · ৪	৫২ · ৭৬
reports · audit	**০ · ০**	—

বিলিংয়ে সবচেয়ে বেশি টেস্ট কেন? টাকা ভুল হলে ফেরানো কঠিন — আর নিয়মগুলোই জটিল (মেয়াদ · সিট · ইনভয়েস · গেট)।

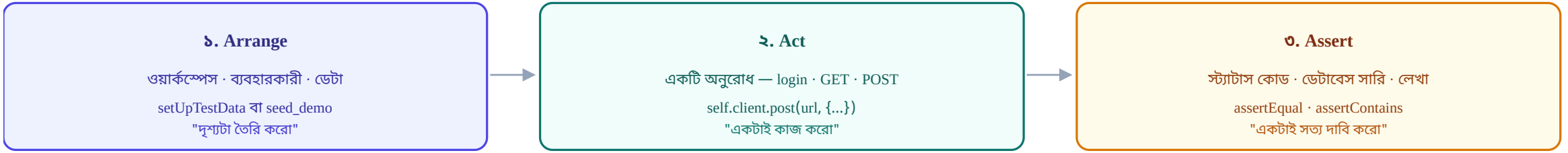
শূন্য টেস্ট = অন্ধ জায়গা: `reports/`-এর CSV আর `audit/`-এর পর্দা। কোড আছে, ভরসা নেই।

শিক্ষকের নোট: `python manage.py test apps` চালিয়ে ১০৩টি টেস্টের আউটপুট দেখান — সংখ্যাটা একবার চোখে দেখলে আর ভুল হয় না।

ধাপ ২

একটি টেস্টের গঠন — তিন ধাপ

Arrange · Act · Assert — নাম মনে রাখলেই যথেষ্ট



এক টেস্টে একটির বেশি Act নয় — নাহলে ব্যর্থ হলে বোঝা যায় না কোনটা ভেঙেছে

ভালো টেস্টের নাম পড়েই বোঝা যায় দৃশ্যটা: `test_expired_trial_goes_read_only_but_keeps_data` — তিনটি দাবি এক নামে।

শিক্ষকের নোট: বোর্ডে একটি খালি টেস্ট লিখুন তিন ভাগে ভাগ করে, তারপর কোডে পূরণ করুন। ছাত্ররা নিজের প্রজেক্টে একই ছক মানবে।

ধাপ ৩

`setUpTestData` বনাম `setUp`

১৬টি ক্লাস-স্তরের প্রস্তুতি · ২০টি মেথড-স্তরের

```
@classmethod
def setUpTestData(cls):
    call_command("seed_demo", verbosity=0)    # একবার, পুরো ক্লাসের জন্য

def setUp(self):
    self.client.force_login(self.teacher)     # প্রতিটি টেস্টের আগে

# পার্থক্য মনে রাখার নিয়ম
# setUpTestData → একবার চলে, তারপর প্রতিটি টেস্টের
#                 শেষে Django নিজেই রোলব্যাক করে
# setUp         → প্রতিটি টেস্টের আগে চলে (কোনো রোলব্যাক নেই)
```

প্রশ্ন	উত্তর
ডেটা কতবার তৈরি হয়?	`setUpTestData`-য় একবার
টেস্ট ডেটা বদলালে?	প্রতিটি টেস্ট শেষে রোলব্যাক — পরের টেস্ট পরিষ্কার
কেন `@classmethod`	ইনস্ট্যান্স নেই, তাই `cls` দিয়ে ডেটা রাখতে হয়
দুর্বলতা	`self` ব্যবহার করা যায় না — শেয়ারড অবজেক্ট বদলাবেন না

> বড় `seed_demo` একবার চালালে ৩৬টি বিলিং টেস্ট দ্রুত চলে — এটাই IMS-এর গতি-কৌশল।

শিক্ষকের নোট: লাইভে একটি টেস্টে ডেটা বদলে পরের টেস্ট চালান — দেখা যাবে বদল টেকেনি। রোলব্যাকের ধারণাটা এখানেই বাস্তব হয়ে ওঠে।

ধাপ ৪

`seed_demo` — ২৮১ লাইনের দৃশ্য

একটি প্রতিষ্ঠান, চার ভূমিকা, ছাত্র-ক্লাস-পরীক্ষা-হাজিরা

```
@classmethod
def setUpTestData(cls):
    call_command("seed_demo", verbosity=0)

SLUG = "sunrise-model-school"
PASSWORD = "DemoPass!2026"

LOGINS = [("admin@ims.local", "ORG_ADMIN"),
          ("coordinator@ims.local", "COORDINATOR"),
          ("teacher@ims.local", "TEACHER"),
          ("student@ims.local", "STUDENT")]

def _login(self, email):
    self.assertTrue(self.client.login(
        email=email, password=PASSWORD))
    self.client.get(reverse("workspaces:switch"),
        {"to": .pk})
```

কেন seed, কেন হাতে তৈরি নয়

হাতে ৩০টি সারি বানালে টেস্ট পড়া কঠিন হয়; seed আসল ডেটার মতোই বাস্তব।

idempotent

দুবার চালালেও ভাঙে না — তাই টেস্ট আর ডেমো দুই জায়গায় একই কমান্ড।

চার লগইন

একই ওয়ার্কস্পেসে চারটি ভূমিকা — তাই অনুমতির টেস্ট লেখা সহজ।

ঝুঁকি

seed বদলালে অনেক টেস্ট একসাথে ভাঙতে পারে — তাই slug ও পাসওয়ার্ড ধ্রুবক।

শিক্ষকের নোট: বলুন: "টেস্টের ডেটাও প্রোডাক্টের অংশ।" seed_demo ভাঙলে ডেমো আর টেস্ট দুটোই ভাঙে — তাই এটি যত্নে রাখা হয়।

ধাপ ৫

লগইনের তিন পদ্ধতি

`login()` ৩০ বার ব্যবহৃত · `force_login()` দ্রুত · `reverse()` ৩৪ বার

পদ্ধতি	কী করে	কখন
`self.client.login(email=…, password=…)`	সত্যিকারের লগইন প্রবাহ	লগইন নিজেই পরীক্ষার বিষয় হলে
`self.client.force_login(user)`	সেশন বানিয়ে দেয়	শুধু অনুমতি/পর্দা দেখতে চাইলে — দ্রুত
`self.client.get(reverse("…"))`	URL নাম থেকে পথ	সবসময় — হার্ডকোড করা URL নয়

`reverse()` ব্যবহার করলে URL বদলালে টেস্ট ভাঙে না — শুধু এক জায়গায় ঠিক করতে হয়।

`force_login` বেশি ব্যবহার করলে লগইন-সংক্রান্ত বাগ ধরা পড়ে না — তাই দুটোই দরকার।

শিক্ষকের নোট: প্রশ্ন: "টেস্টে `/w/sunrise-model-school/students/` হার্ডকোড করলে কী হয়?" উত্তর: slug বদলালে ২০টি টেস্ট ভাঙবে।

ধাপ ৬

কোন assert কখন — সংখ্যায় দেখুন

১২৯টি `assertEqual` · ১৭টি `assertContains` · ১২টি `assertRaises`

assert	কতবার	কী প্রমাণ করে
`assertEqual`	১২৯	মান সমান — স্ট্যাটাস, সংখ্যা, স্ট্রিং
`assertTrue` / `assertFalse`	২৯ / ৮	একটি শর্ত সত্য/মিথ্যা (`check_password`, `is_late`)
`assertContains`	১৭	পর্দায় লেখাটি সত্যিই দেখা যায়
`assertNotContains`	৫	যা দেখা যাবে না, তা সত্যিই দেখা যায় না
`assertRaises`	১২	ভুল ইনপুটে `ValidationError` / `PermissionDenied`
`assertRedirects`	**১**	— খুব কম; বাকিরা শুধু স্ট্যাটাস ৩০২ দেখে

`assertEqual(response.status_code, 302)` বলে না **কোথায়** গেল। `assertRedirects` একটাই ব্যবহার — বাকি জায়গায় গন্তব্য অপরীক্ষিত।

শিক্ষকের নোট: "৩০২ মানে কিছু একটা হয়েছে, কিন্তু কোথায়?" — এই ফাঁকটা দেখিয়ে ছাত্রদের নিজের টেস্টে `assertRedirects` ব্যবহার করতে বলুন।

ধাপ ৭

চার ভূমিকা, একটি টেস্ট — `subTest`

লুপের ভেতরে টেস্ট, কিন্তু ব্যর্থতা আলাদা করে রিপোর্ট হয়

```
def test_each_role_is_offered_only_its_own_links(self):
    for email, rules in EXPECTED_LINKS.items():
        with self.subTest(role=email):
            html = self._sidebar_html(email)
            for label in rules["must_show"]:
                self.assertIn(label, html)
            for label in rules["must_hide"]:
                self.assertNotIn(label, html)

# ৪ ভূমিকা × (দেখতে হবে + দেখাবে না) — এক টেস্টে
# ব্যর্থ হলে: "role='student@ims.local'" লেখা আসে
```

কেন subTest

লুপে সাধারণ assert দিলে প্রথম ব্যর্থতায় বাকি তিনটি ভূমিকা অপরীক্ষিত থেকে যায়।

রিপোর্ট

একই টেস্টে ৪টি আলাদা ফল — কোন ভূমিকা ভেঙেছে স্পষ্ট দেখা যায়।

কোথায় ব্যবহৃত

core-এ ৫ বার (ভূমিকা ও পর্দা), billing-এ ১ বার।

সীমা

এক টেস্টে ২০টি subTest থাকলে ব্যর্থতার তালিকা লম্বা হয় — ৪–৬ এর মধ্যে রাখুন।

শিক্ষকের নোট: ইচ্ছে করে একটি ভূমিকার EXPECTED_LINKS ভেঙে দেখান — আউটপুটে শুধু ওই ভূমিকাটি লাল হবে, বাকিরা সবুজ। এটাই subTest-এর পুরো মূল্য।

ধাপ ৮

৪০৩ বনাম ৪০৪ — টেস্টেই সিদ্ধান্ত

`test_non_member_gets_404_not_403` — কেন এটা জরুরি

```
def test_non_member_gets_404_not_403(self):
    other = User.objects.create_user(
        email="outsider@example.com", password="x", full_name="Out")
    self.client.force_login(other)
    resp = self.client.get(f"/w/{SLUG}/students/")
    self.assertEqual(resp.status_code, 404)

def test_teacher_cannot_open_administration_pages(self):
    self._login("teacher@ims.local")
    for name in ("audit list", "settings", "grading"):
        with self.subTest(page=name):
            resp = self.client.get(reverse(f"…:{name}", kwargs={...}))
            self.assertEqual(resp.status_code, 403)
```

পরিস্থিতি	সঠিক উত্তর
সদস্যই নন	**৪০৪** — প্রতিষ্ঠানটির অস্তিত্বই জানানো হয় না
সদস্য, কিন্তু ভূমিকা নয়	**৪০৩** — "আপনার অনুমতি নেই"
লগইন নেই	লগইনে রিডাইরেক্ট (৩০২)

ভুল হলে তথ্য ফাঁস: ৪০৩ বললে বোঝা যায় "এই ঠিকানায় একটা প্রতিষ্ঠান আছে"।

এই দুটি টেস্টই পর্ব ৪-এর ক্লাস ১৭-এর নিয়মগুলোকে লক করে রেখেছে — নিয়ম বদলালে টেস্ট লাল হবে।

শিক্ষকের নোট: প্রশ্ন করুন: "এক টেস্টে ৪০৩, আরেকটায় ৪০৪ — কেন আলাদা?" উত্তর: কারণ দুটো আলাদা ব্যবসায়িক সিদ্ধান্ত, আর টেস্ট হলো সেই সিদ্ধান্তের দলিল।

ধাপ ৯

টেস্ট চালানো — তিনটি কমান্ড

সব · এক অ্যাপ · এক টেস্ট

```
# সব টেস্ট
python manage.py test apps

# এক অ্যাপ
python manage.py test apps.billing

# একটি ক্লাস
python manage.py test apps.core.tests.RoleInterfaceTests

# একটি টেস্ট
python manage.py test
apps.billing.tests.ScopeTests.test_server_refuses_the_third_student

# দ্রুত পুনরাবৃত্তি (ডেটাবেস ফেলে দেয় না)
python manage.py test apps --keepdb -v 2

# আসল স্কুলের মতো স্মোক টেস্ট (রোল × URL ম্যাট্রিক্স)
python smoke_test.py
```

`--keepdb`

টেস্ট-ডেটাবেস তৈরি করতে ২–৫ সেকেন্ড বাঁচে; বারবার চালালে বড় পার্থক্য।

`-v 2`

প্রতিটি টেস্টের নাম দেখায় — কোনটি ধীর, তা চোখে পড়ে।

smoke_test.py আলাদা কেন

এটি `TestCase` নয় — আসল ডেমো ডেটাবেসে ৪টি লগইন দিয়ে **৬০টি URL** ঘোরে।

প্রতিদিনের অভ্যাস

কোড লেখার আগে নয়, লেখার পরে অন্তত একবার পুরো স্যুট — নাহলে ফাঁক জমে।

শিক্ষকের নোট: `--keepdb` ছাড়া ও সাথে দুইবার চালিয়ে সময়ের পার্থক্য দেখান — গতি নিজেই যুক্তি।

ধাপ ১০

টেস্টের নামই ডকুমেন্টেশন

ভালো নাম পড়লে কোড না দেখেই স্পেসিফিকেশন বোঝা যায়

নাম	যা বলে
`test_expired_trial_goes_read_only_but_keeps_data`	নিয়ম + সীমা + প্রতিশ্রুতি
`test_early_renewal_extends_instead_of_resetting`	ব্যবসায়িক সদিচ্ছা
`test_student_cannot_see_another_students_marksheet`	গোপনীয়তার সীমা
`test_no_role_sees_a_link_it_cannot_open`	দুই পক্ষ (মেনু ও সার্ভার) এক
`test_personal_workspace_is_never_gated`	ব্যতিক্রমটি স্পষ্ট
`test_domain_cannot_be_stolen_by_a_second_institution`	আক্রমণের নাম ধরে টেস্ট

ছক: `test_<অবস্থা>_<প্রত্যাশিত আচরণ>` — শর্ত আগে, ফলাফল পরে।

খারাপ নাম: `test_1`, `test_works`, `test_student` — ব্যর্থ হলে কিছুই বলে না।

শিক্ষকের নোট: `python manage.py test apps -v 2`-এর আউটপুট প্রিন্ট করান — পুরো তালিকাটাই এক পাতার স্পেসিফিকেশন।

সতর্কতা

পাঁচটি টেস্ট-বিরোধী অভ্যাস

প্রতিটির প্রতিকার এক লাইনে

অভ্যাস	পরিণতি ও প্রতিকার
১. এক টেস্টে অনেক assert	প্রথমটাই ফেল করলে বাকি দাবি অজানা → subTest বা ভাগ করুন
২. `setUpTestData`-র ডেটা বদলানো	পরের টেস্ট অপ্রত্যাশিত → শেয়ারড অবজেক্ট অপরিবর্তিত রাখুন
৩. শুধু স্ট্যাটাস ৩০২ দেখা	ভুল জায়গায় রিডাইরেক্টও পাশ → `assertRedirects`
৪. `force_login`-এ ভরসা	লগইনের বাগ চাপা পড়ে → অন্তত একটি সত্যিকারের `login()`
৫. সময়-নির্ভর টেস্ট	`timezone.localdate()` আসল তারিখ দেয় → `freeze_time` বা আপেক্ষিক তারিখ

পঞ্চমটি IMS-এ **এখনো কোথাও ব্যবহৃত হয়নি** — মেয়াদ-সংক্রান্ত টেস্টগুলো আপেক্ষিক তারিখ (`today ± timedelta`) দিয়ে লেখা, তাই আজ-কাল ভাঙে না।

শিক্ষকের নোট: পঞ্চমটার স্বাদ দিন: একটি টেস্টে `trial_ends_on = date(2026,11,5)` লিখে দেখান — আগামীকালই লাল হবে। "টেস্ট ক্যালেন্ডার পড়তে পারে না।"

অনুশীলন ও আলোচনা

২০ মিনিট — নিজের প্রথম তিন টেস্ট

একটি মডেল, একটি ভিউ, একটি অনুমতি

১. মডেলের নিয়ম (৭ মিনিট)

একটি `clean()` বা property — সত্য/মিথ্যা দুই ক্ষেত্রেই assert।

২. ভিউয়ের উত্তর (৭ মিনিট)

`force_login` + `reverse` + স্ট্যাটাস ও `assertRedirects`।

৩. অনুমতি (৬ মিনিট)

ভুল ভূমিকায় ৪০৩, অ-সদস্যে ৪০৪ — দুটোই এক টেস্টে `subTest` দিয়ে।

- টেস্ট লেখা সময় নষ্ট নাকি সঞ্চয়?
- `setUpTestData` কেন `classmethod`?
- ৩০২ দেখলেই যথেষ্ট নয় কেন?
- seed ভাঙলে কী হয়?

শিক্ষকের নোট: চতুর্থ প্রশ্নের উত্তর: "একবারে ৩০টি টেস্ট লাল — আর তখন বোঝা যায় না কোড ভেঙেছে, নাকি seed।" তাই fixture-ও কোডের মতো যত্নের।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
এক টেস্ট, এক দাবি	১০৩ টেস্ট · ১,৩৫৩ লাইন · ২২ ক্লাস
প্রস্তুতি একবার, টেস্ট আলাদা	১৬ `setUpTestData` · `seed_demo` ২৮১ লাইন
URL নাম দিয়ে, হার্ডকোড নয়	`reverse()` ৩৪ বার
লুপে `subTest`	`EXPECTED_LINKS` — ৪ ভূমিকা, এক টেস্ট
৪০৩ বনাম ৪০৪ পরীক্ষিত	`test_non_member_gets_404_not_403`
যা পরীক্ষা হয়নি, তা অজানা	**reports ০ · audit ০**

শিক্ষকের নোট: শেষ সারিটা বারবার ফিরিয়ে আনুন — পরের ক্লাসেই এর বিস্তারিত: কোন ফাঁক কীভাবে বন্ধ করা যায়।

২৫

পরের ক্লাস — "কে কী দেখবে, প্রমাণসহ"

অনুমতি ও স্কোপের টেস্ট: `scoping.py` · ভূমিকা-ভিত্তিক ম্যাট্রিক্স · `smoke_test.py` · ক্রস-ওয়ার্কস্পেস ফাঁস

পর্ব ৫ · টেস্টিং · ক্লাস ২৫ / ৩৭

কে কী দেখবে, প্রমাণসহ

সিলেবাস: প্র্যাকটিক্যাল ৯ · টপিক ৯.২ (অনুমতির টেস্টিং)
৩টি helper · ৮টি স্কোপিং টেস্ট · ৬০টি URL × ৪ ভূমিকা

visible_classes · 403 বনাম 404 · smoke matrix

শুরু করুন প্রশ্ন দিয়ে

"এই পাতা কি ও দেখতে পারবে?"

একটি প্রশ্ন নয়, দুটি — আর দুটির উত্তর দেয় দুটি আলাদা জিনিস

১. মেনুতে দেখায় কি? → `navigation.py`

২. সার্ভার খুলতে দেয় কি? → `scoping.py` + decorator

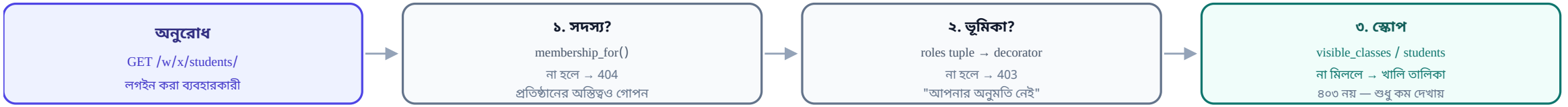
দুটো আলাদা জিনিস, কিন্তু **একই সত্য** বলা চাই। ক্লাস ২৪-এর টেস্ট `test_no_role_sees_a_link_it_cannot_open` ঠিক এই দুটোকে একসাথে বাঁধে — আজ দেখব ভেতরের কারিগরি।

শিক্ষকের নোট: বলুন: "মেনু লুকানো ভদ্রতা, অনুমতি নিরাপত্তা।" এই এক বাক্যেই পুরো ক্লাসের কাঠামো।

ধাপ ১

এক অনুরোধ, চারটি দেয়াল

প্রতিটি দেয়ালে আটকালে ফল আলাদা — ৩০২ · ৪০৪ · ৪০৩ · খালি তালিকা



লগইন নেই → 302 লগইন পর্দা (এটি decorator-এর প্রথম কাজ, তাই ছকে আলাদা দেখানো হয়নি)

ভুল ক্রম মানে ভুল উত্তর: সদস্য নয় বলে ৪০৩ দিলে প্রতিষ্ঠানের অস্তিত্ব ফাঁস হয়

স্কোপ আটকায় "কী দেখব", ৪০৩ আটকায় "কোন পাতায় ঢুকব" — তাই দুই ধরনের টেস্ট

এই চারটি দেয়ালই ক্লাস ১৭-এ লেখা হয়েছিল; আজকের কাজ সেগুলোকে **টেস্টে বেঁধে ফেলা** — যাতে ভবিষ্যতের পরিবর্তন নিয়ম ভাঙতে না পারে।

শিক্ষকের নোট: বোর্ডে চার দেয়াল লিখুন, তারপর প্রতিটির পাশে ফল লিখুন (302/404/403/খালি)। ছাত্ররা এই ছক মুখস্থ নয়, বুঝে নেবে।

ধাপ ২

`scoping.py` — একটি ফাইল, সব তালিকা

"একবার ঠিক করলে নয়টি ভিউয়ে ঠিক" — ফাইলের নিজের মন্তব্য

```python
def visible_classes(user, workspace, membership=None):
    membership = membership or membership_for(user, workspace)
    qs = ClassGroup.objects.filter(workspace=workspace,
                                   is_archived=False)
    if membership is None:
        return qs.none()

    if membership.role == Roles.ORG_ADMIN:
        return qs

    if membership.role == Roles.COORDINATOR:
        scopes = CoordinatorScope.objects.filter(
            workspace=workspace, user=user, is_active=True)
        dept_ids = list(scopes.exclude(department=None)
                        .values_list("department_id", flat=True))
        class_ids = list(scopes.exclude(class_group=None)
                         .values_list("class_group_id", flat=True))
        return qs.filter(models_q(dept_ids, class_ids))

    if membership.role == Roles.TEACHER:
        return qs.filter(assignments__teacher=user,
                         assignments__is_active=True).distinct()

    if membership.role == Roles.STUDENT:
        student = getattr(user, "student_profile", None)
        if student is None:
            return qs.none()
        return qs.filter(enrollments__student=student,
                         enrollments__is_active=True).distinct()
    return qs.none()
```

সদস্য না হলে `none()`

খালি queryset — `None` নয়, কারণ কলকারী সবাই `.count()` ডাকে।

শেষে `none()`

অজানা ভূমিকা = কিছু নয়। ডিফল্ট সব দেখানো বিপজ্জনক।

`membership=None` প্যারামিটার

ভিউ ইতিমধ্যে মেম্বারশিপ জানে — দুবার কুয়েরি হয় না।

`distinct()`

`assignments` ও `enrollments` জয়েনে ডুপ্লিকেট সারি আসে — নাহলে এক ক্লাস দুইবার।

শিক্ষকের নোট: প্রশ্ন: "শেষের `return qs.none()` না লিখলে কী হতো?" উত্তর: ভবিষ্যতের নতুন ভূমিকা সব ডেটা দেখতে পেত — নীরব বিপর্যয়।

ধাপ ৩

আটটি টেস্ট, আটটি সংখ্যা

`core/tests.py → ScopingTests` — পর্দা নয়, queryset পরীক্ষা

টেস্ট	প্রত্যাশা	দেখায়
`test_admin_sees_every_class_and_student`	সব ক্লাস · সব ছাত্র	একমাত্র "সব দেখে" ভূমিকা
`test_teacher_sees_only_assigned_classes`	১টি ক্লাস — "Class 8"	দ্বিতীয় শিক্ষকের "Class 9"
`test_teacher_only_sees_students_of_their_own_classes`	সেট সমান	`set(pk)` তুলনা — ক্রম নয়
`test_teacher_with_no_assignment_sees_no_students`	০ · ০	নতুন সদস্য, শূন্য অনুমতি
`test_coordinator_sees_only_their_scope`	০ → তারপর ১	স্কোপ মুছে/যোগ করে
`test_student_sees_only_own_record`	১ — "Rafid Hasan"	নাম মিলিয়েও যাচাই
`test_inactive_membership_sees_nothing`	০ · ০	`is_active=False`
`test_student_only_gets_published_homework`	`all(is_published)`	প্রকাশ না হলে ফাঁস নয়

সংখ্যা (১, ০, ২) ব্যবহার করা হয়েছে ইচ্ছাকৃতভাবে — "কিছু একটা আছে" নয়, **ঠিক কত** তা দাবি করা হয়েছে।

শিক্ষকের নোট: জোর দিন: `assertEqual(count(), 1)` অনেক শক্তিশালী `assertTrue(count() > 0)`-এর চেয়ে। "সংখ্যা দিয়ে দাবি করুন।"

ধাপ ৪

কোঅর্ডিনেটরের স্কোপ — যোগ-বিয়োগে প্রমাণ

"স্কোপ নেই" অবস্থাটাই সবচেয়ে বিপজ্জনক, তাই সেটাই আগে পরীক্ষা

```
coord = self._user("coordinator@ims.local")
# seed দিয়েছে Science ডিপার্টমেন্ট (Class 8)

CoordinatorScope.objects.filter(workspace=self.ws,
                                user=coord).delete()
self.assertEqual(scoping.visible_classes(coord, self.ws).count(), 0,
                 "a coordinator without a scope must see nothing")

CoordinatorScope.objects.create(
    workspace=self.ws, user=coord,
    department=Department.objects.get(workspace=self.ws,
                                      name="Science"))
self.assertEqual([c.name for c in
                  scoping.visible_classes(coord, self.ws)], ["Class 8"])
```

খালি অবস্থা আগে

ভুল করে "সব দেখানো" বাগ ধরা পড়ে কেবল তখনই, যখন শূন্য প্রত্যাশা করা হয়।

যোগ করার পর

ডিপার্টমেন্ট দিলে ঠিক "Class 8" — নাম দিয়ে যাচাই, সংখ্যা দিয়ে নয়।

`models_q()`

ডিপার্টমেন্ট **অথবা** সরাসরি ক্লাস — দুটোই এক ফিল্টারে (`Q |=`)।

তৃতীয় বার্তা

`assertEqual(..., 0, "a coordinator without a scope…")` — ব্যর্থ হলে কারণ লেখা আসে।

শিক্ষকের নোট: শেষ কার্ডটি দেখান: assert-এ বার্তা দিলে ৬ মাস পর ব্যর্থতা পড়েও বোঝা যায় কী ভাঙল। "টেস্টকে ভবিষ্যতের নিজের কাছে চিঠি লিখুন।"

ধাপ ৫

ক্রস-ওয়ার্কস্পেস — দুই স্কুল, এক টেস্ট

সবচেয়ে বিপজ্জনক ফাঁস: অন্য প্রতিষ্ঠানের সারি দেখা

```
@classmethod
def setUpTestData(cls):
    cls.teacher = User.objects.create_user(email="t@example.com", ...)
    cls.outsider = User.objects.create_user(email="o@example.com", ...)
    cls.ws  = Workspace.objects.create(name="School A", owner=cls.teacher)
    cls.ws2 = Workspace.objects.create(name="School B", owner=cls.outsider)
    Membership.objects.create(user=cls.teacher, workspace=cls.ws, role=Roles.TEACHER)
    Membership.objects.create(user=cls.outsider, workspace=cls.ws2,
role=Roles.ORG_ADMIN)

def test_other_workspace_cannot_open_the_student(self):
    self.client.force_login(self.outsider)     # School B-র অ্যাডমিন
    resp = self.client.get(reverse("students:detail",
                kwargs={"slug": self.ws.slug, "pk": self.student.pk}))
    self.assertEqual(resp.status_code, 404)    # 403 নয়
```

কেন ৪০৪

School B-র অ্যাডমিন School A-র সদস্য নন — তাই School A-র অস্তিত্বও জানানো হয় না।

slug-ই প্রথম প্রাচীর

ভুল slug দিলে `@workspace_view`-এ ৪০৪ — ভেতরের ভিউ পর্যন্ত পৌঁছায়ই না।

আরও একটি

`test_teacher_without_assignment_sees_no_students` — `assertNotContains(resp, "Rafid")`।

ফাঁক

ক্রস-ওয়ার্কস্পেস টেস্ট কেবল students অ্যাপে; attendance · assessments-এ নেই।

শিক্ষকের নোট: প্রশ্ন করুন: "৪০৩ দিলে কী তথ্য ফাঁস হতো?" উত্তর: বোঝা যেত ওই আইডিতে একটি ছাত্র আছে — তাই ৪০৪।

ধাপ ৬

ডেটাবেসকে দিয়েও প্রমাণ

কিছু নিয়ম ভিউয়ে নয়, constraint-এ — সেগুলোও টেস্ট হয়

```
def test_second_active_enrolment_is_blocked_by_the_database(self):
    from django.db import IntegrityError, transaction
    Enrollment.objects.create(workspace=self.ws, student=self.student,
                              class_group=self.cls, is_active=True)
    with self.assertRaises(IntegrityError):
        with transaction.atomic():          # না থাকলে টেস্ট ভেঙে যায়
            Enrollment.objects.create(workspace=self.ws,
                                      student=self.student,
                                      class_group=self.cls,
                                      is_active=True)

def test_ending_an_enrolment_keeps_history(self):
    e.is_active = False; e.end_reason = "Transferred"; e.save()
    e.refresh_from_db()
    self.assertIsNotNone(e.ended_on)          # save() নিজে তারিখ বসায়
    self.assertEqual(Enrollment.objects.filter(student=...).count(), 1)
```

`assertRaises` + `atomic`

IntegrityError-এর পর ট্রানজ্যাকশন ভাঙা থাকে — `atomic` ছাড়া পরের কুয়েরি ব্যর্থ।

ইতিহাস ≠ মুছে ফেলা

ভর্তি বন্ধ মানে সারি থাকবে, `ended_on` বসবে — সারি সংখ্যা ১-ই থাকে।

`refresh_from_db()`

DB-তে কী জমা হলো তা দেখতে বাধ্যতামূলক — মেমরির অবজেক্ট সত্য নয়।

কেন দরকার

ভিউ বাইপাস করে শেল/ইমপোর্টে ডেটা ঢুকলেও constraint আটকাবে — টেস্ট সেই প্রতিশ্রুতির প্রমাণ।

শিক্ষকের নোট: `transaction.atomic()` ছাড়া চালিয়ে দেখান — এরর আসবে "An error occurred in the current transaction"। এটাই বাস্তব ব্যথা।

ধাপ ৭

`smoke_test.py` — ৬০টি URL × ৪ ভূমিকা

আসল ডেমো ডেটাবেসে চলে · `TestCase` নয়, স্ক্রিপ্ট

```
ALL   = {"ORG_ADMIN", "COORDINATOR", "TEACHER", "STUDENT"}
STAFF = {"ORG_ADMIN", "COORDINATOR", "TEACHER"}
OWNER = {"ORG_ADMIN"}

PAGES = [
    ("/",                                  ALL,   "dashboard redirect"),
    (f"/w/{s}/students/",                  STAFF, "student list"),
    (f"/w/{s}/students/{student.pk}/status/", {"ORG_ADMIN", "COORDINATOR"}, "change status"),
    (f"/w/{s}/academics/scopes/",          OWNER, "coordinator scope"),
    (f"/w/{s}/reports/export/results.csv?exam={exam.pk}", STAFF, "results csv"),
    (f"/w/{s}/renew/",                     OWNER, "subscription (owner only)"),
    ...
]

for url, allowed, note in PAGES:
    expect_ok = role in allowed
    acceptable = (200, 302) if expect_ok else (403,)
    response = client.get(url)
    if response.status_code not in acceptable:
        wrong.append((url, response.status_code, acceptable[0]))
```

প্রশ্নটা ভিন্ন

"পাতাটা কাজ করে?" নয় — "এই ভূমিকার জন্য এটি **অনুমোদিত** না **প্রত্যাখ্যাত**?"

তিনটি কোডই ব্যর্থতা

৫০০, অথবা প্রত্যাশিত ৪০৩-এর জায়গায় ২০০ — দুটোই খারাপ।

আউটপুট

`STUDENT 57/60 as expected` — মিসম্যাচ থাকলে URL ও আশা-প্রাপ্তি ছাপে।

সীমা

কেবল স্ট্যাটাস দেখে — পর্দার ভেতরে অন্য স্কুলের নাম আছে কি না দেখে না।

শিক্ষকের নোট: `python smoke_test.py` চালিয়ে আউটপুট দেখান — ৪ ভূমিকার সারি। তারপর একটি সারি ইচ্ছে করে ভেঙে দেখান কীভাবে মিসম্যাচ দেখায়।

ধাপ ৮

ম্যাট্রিক্সের ব্যতিক্রম — দুটি লাইন

প্রতিটি ব্যতিক্রম একদিন বাগের উৎস; তাই স্পষ্ট করে লেখা

```
if url in ("/", f"/w/{s}/results/"):
    acceptable = (200, 302)

# কেন?
#  "/"          → সবার জন্য, শুধু গন্তব্য ভিন্ন
#  "/results/" → staff: exam list · student: নিজের ফল
#                 দুই ভূমিকাতেই ২০০ বা ৩০২ বৈধ

# বাকি সব ক্ষেত্রে:
acceptable = (200, 302) if expect_ok else (403,)
```

ব্যতিক্রম লিখতে হয়

নাহলে ভুল ব্যর্থতা (false alarm) আসে, আর মানুষ টেস্ট উপেক্ষা করতে শেখে।

তবু সীমা

প্রতিটি ব্যতিক্রমের পাশে কারণ লেখা — কোডই ডকুমেন্টেশন।

বিকল্প ডিজাইন

পাতাগুলো আলাদা করলে (`/results/` staff ও student-এ ভিন্ন URL) ব্যতিক্রম লাগত না।

শিক্ষকের নোট: "সব টেস্ট সবুজ রাখতে ব্যতিক্রম বাড়াবেন না" — প্রতিটি ব্যতিক্রমের দাম একদিনের বাগ-সন্ধান। ভালো ডিজাইন ব্যতিক্রম কমায়।

ধাপ ৯

মেনুর দিকটাও পরীক্ষা

`test_each_role_is_offered_only_its_own_links` — HTML পড়ে, স্ট্যাটাস নয়

```
EXPECTED_LINKS = {
  "teacher@ims.local": {
    "must_show": ["Students", "Attendance", "Homework",
                  "Materials", "Exams & results"],
    "must_hide": ["Audit log", "Workspace settings", "Grading",
                  "Coordinator scope", "My oversight area",
                  "My homework", "My results"],
  },
  "student@ims.local": {
    "must_show": ["My homework", "My results", "Materials"],
    "must_hide": ["Students", "Attendance", "Syllabus",
                  "Lesson log", "Exams & results", "Reports", ...],
  },
}

html = self._sidebar_html(email)
self.assertIn(label, html)        # must_show
self.assertNotIn(label, html)     # must_hide
```

দুই তালিকা কেন

`must_show` ছাড়া মেনু ফাঁকা হয়ে গেলেও টেস্ট পাশ করবে।

HTML-এ লেখা খোঁজা

সাইডবারের HTML স্ট্রিং নিয়ে `assertIn` — DOM পার্সিং নয়, তাই দ্রুত।

দুর্বলতা

লেবেল লেখা বদলালে (অনুবাদ!) টেস্ট ভাঙবে — তাই লেবেল স্থির রাখা হয়।

জোড়া

এটি + `smoke_test` = মেনু ও সার্ভার দুই দিকেই প্রমাণ।

শিক্ষকের নোট: "লেবেল যদি বাংলায় অনুবাদ করতেন?" — টেস্ট ভাঙবে। সমাধান: লেবেলের বদলে `url_name` বা HTML-এ `data-nav` অ্যাট্রিবিউট খোঁজা।

ধাপ ১০

ম্যাট্রিক্সটি এক নজরে

✓ = ২০০/৩০২ · ✗ = ৪০৩ — এই ছকটাই `smoke_test`-এর সারমর্ম

পথ	ADMIN	COORD	TEACHER	STUDENT
`/w/x/students/`	✓	✓	✓	✗
`/w/x/students/<pk>/status/`	✓	✓	✗	✗
`/w/x/academics/scopes/`	✓	✗	✗	✗
`/w/x/attendance/`	✓	✓	✓	✗
`/w/x/reports/`	✓	✓	✓	✗
`/w/x/materials/`	✓	✓	✓	✓
`/w/x/renew/`	✓	✗	✗	✗
`/platform/`	✓ (superuser)	✗	✗	✗

কলাম দেখুন, সারি নয় — শিক্ষকের কলামে একটাই ✗ বেশি (status), শিক্ষার্থীর কলামে প্রায় সব।

ম্যাট্রিক্সে ফাঁকা ঘর থাকলে সেটাই প্রশ্ন: "এই ভূমিকা এটি কেন দেখবে/দেখবে না?" উত্তর না থাকলে নকশাই অসম্পূর্ণ।

শিক্ষকের নোট: নতুন ফিচার যোগ করলে এই ছকে একটি সারি যোগ করতে বলুন — তারপর ভাবতে হবে কে দেখবে। "ছকটাই ডিজাইনের যন্ত্র।"

সতর্কতা

অনুমতির টেস্টে পাঁচটি ফাঁক

IMS-এ কোনটি আছে, কোনটি নেই — সৎ হিসাব

ফাঁক	অবস্থা ও প্রতিকার
১. শুধু helper পরীক্ষা, HTTP নয়	`ScopingTests` সরাসরি ফাংশন ডাকে → ভিউ ভুল helper ব্যবহার করলে ধরা পড়বে না
২. পর্দার ভেতরের বিষয়বস্তু অপরীক্ষিত	`assertNotContains` মাত্র ৫ বার → এক স্কুলের নাম অন্য পর্দায় থাকতে পারে
৩. ক্রস-ওয়ার্কস্পেস টেস্ট কম	students-এ আছে; attendance · assessments · lessons-এ নেই
৪. রিপোর্ট ও অডিটে শূন্য টেস্ট	CSV-তে স্কোপ ভুল হলে কেউ ধরবে না
৫. অ্যাডমিন = সব	ORG_ADMIN-এর কোনো সীমা নেই — ভুল ডিজাইন নয়, কিন্তু টেস্টে সীমাটা লেখা থাকা দরকার

প্রথমটি সবচেয়ে ধূর্ত: helper ঠিক, কিন্তু ভিউ অন্য helper ডাকছে — সব টেস্ট সবুজ, তবু ফাঁস।

শিক্ষকের নোট: প্রথম ফাঁকটা লাইভে দেখান: একটি ভিউয়ে helper বদলে দিন — কেউ ধরে না। তারপর বলুন: "তাই HTTP-স্তরের টেস্টও দরকার।"

অনুশীলন ও আলোচনা

২০ মিনিট — নিজের স্কোপ লিখুন ও ভাঙুন

তিনটি কাজ, তারপর চারটি প্রশ্ন

১. helper (৭ মিনিট)

`visible_<entity>(user, workspace)` — চার ভূমিকার শাখা, শেষে `none()`।

২. চারটি টেস্ট (৮ মিনিট)

admin = সব · teacher = ১ · student = নিজে · সদস্য নয় = ০।

৩. ইচ্ছাকৃত ফাঁস (৫ মিনিট)

helper থেকে `none()` সরিয়ে দেখুন কোন টেস্ট লাল হয় — আর কোনগুলো হয় না।

- মেনু লুকানো কেন যথেষ্ট নয়?
- ক্রস-ওয়ার্কস্পেসে ৪০৩ নয়, ৪০৪ কেন?
- `count()==1` বনাম `count()>0` — কোনটা ভালো?
- helper ঠিক, ভিউ ভুল — কীভাবে ধরবেন?

শিক্ষকের নোট: তৃতীয় কাজটাই আসল শিক্ষা — কিছু টেস্ট সবুজ থাকবে, কারণ তারা ওই শাখা ছোঁয়ই না। "কভারেজ মানে সংখ্যা নয়, শাখা।"

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
স্কোপ এক জায়গায়	`scoping.py` — `visible_classes` · `visible_students` · `visible_homework`
চার দেয়াল, চার উত্তর	302 · 404 · 403 · খালি তালিকা
সংখ্যা দিয়ে দাবি	১ · ০ · ২ — "কিছু আছে" নয়
ডেটাবেসের নিয়মও টেস্ট হয়	`assertRaises(IntegrityError)` + `atomic`
ভূমিকা × URL ম্যাট্রিক্স	`smoke_test.py` — ৪ লগইন, ৬০ পথ
helper টেস্ট ≠ HTTP টেস্ট	ফাঁক ১ — ভিউ অন্য helper ডাকলে ধরা পড়ে না

শিক্ষকের নোট: শেষ সারিটা পরের ক্লাসের সূচনা: কভারেজ সংখ্যা যা দেখায়, আর যা লুকায়।

২৬

পরের ক্লাস — "সংখ্যাটা মিথ্যা বলে না, কিন্তু সব বলে না"

কভারেজ ও যা বাকি: `coverage.py` · শাখা বনাম লাইন · ০ টেস্টের অ্যাপ · ফ্লেকি টেস্ট · টেস্ট পিরামিড

পর্ব ৫ · টেস্টিং · ক্লাস ২৬ / ৩৭

সংখ্যাটা মিথ্যা বলে না — সবও বলে না

সিলেবাস: প্র্যাকটিক্যাল ৯ · টপিক ৯.৩ (কভারেজ ও ফাঁক)

১০৩ টেস্ট · ৬৬ HTTP · ৩৭ বিশুদ্ধ · ৫,৪৮০ লাইন অ্যাপ কোড

line vs branch · flaky · CI

শুরু করুন প্রশ্ন দিয়ে

"কভারেজ ৯২%" — এর মানে কী?

হাত তুলুন: কোডের ৯২% কাজ করে? ৯২% বাগ ধরা পড়বে?

কোনোটাই নয় — এর মানে শুধু "৯২% লাইন অন্তত একবার চলেছে"।

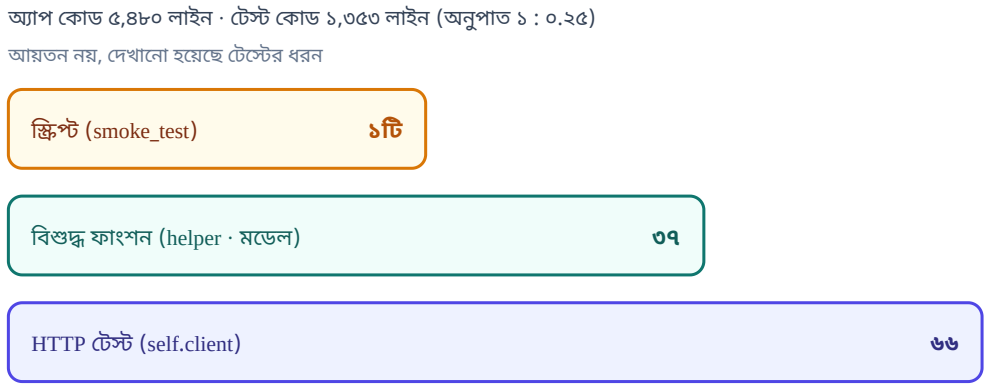
একটি লাইন চলা আর তার **সব শাখা** চলা এক নয়। তাই আজ: কভারেজ কী মাপে, কী মাপে না, আর IMS-এর ১০৩ টেস্ট কোথায় পাহারা দেয় — কোথায় দেয় না।

শিক্ষকের নোট: বলুন: "কভারেজ হলো স্পিডোমিটার, ব্রেক নয়।" বেশি সংখ্যা ভালো, কিন্তু নিরাপত্তার প্রমাণ নয়।

ধাপ ১

আমাদের স্যুটের আসল আকার

১০৩ টেস্ট · ১,৩৫৩ লাইন · অ্যাপ কোড ৫,৪৮০ লাইন



অ্যাপ	টেস্ট	HTTP
billing	৩৬	২৩
core	১৯	১০
assessments	১১	৬
workspaces	১০	৮
accounts	৯	৯
students · attendance	৫ · ৫	৩ · ৩
academics · lessons	৪ · ৪	১ · ৩
reports · audit	**০ · ০**	০

প্রতি ৪ লাইন কোডে ১ লাইন টেস্ট — শিল্পে ১:০.৫–১ সাধারণ; IMS নিচু দিকে, কিন্তু **সঠিক জায়গায়** ঘন।

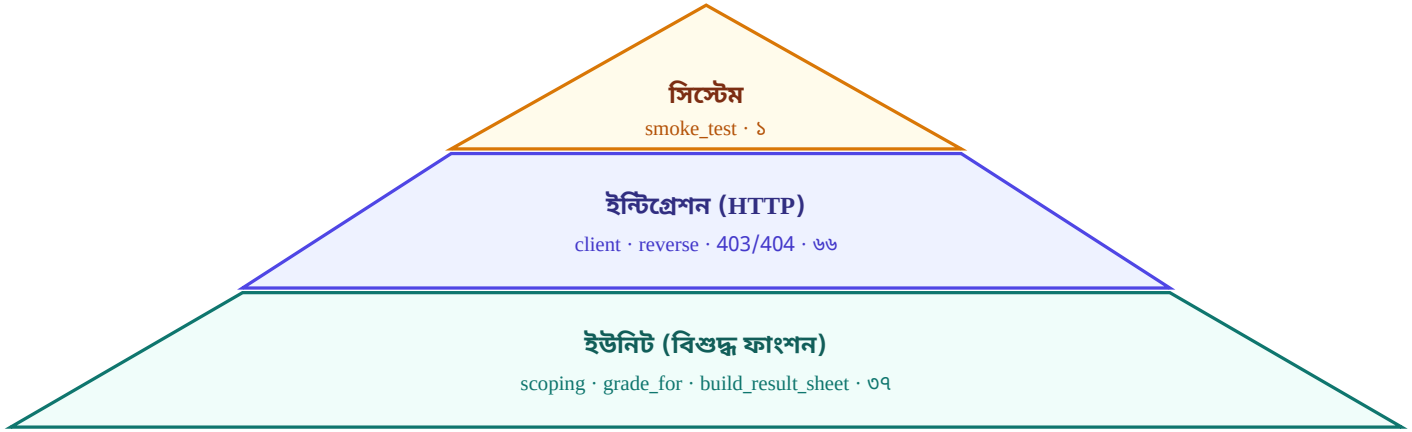
শিক্ষকের নোট: "HTTP ৬৬, বিশুদ্ধ ৩৭" — এই ভাগটাই টেস্ট পিরামিডের বাস্তব রূপ; পরের স্লাইডে কেন এটাই সঠিক।

ধাপ ২

পিরামিড — নিচে বেশি, উপরে কম

বিশুদ্ধ টেস্ট দ্রুত ও স্পষ্ট; HTTP টেস্ট ধীর কিন্তু বাস্তব

উপরের স্তর:
ধীর, ভঙ্গুর
কিন্তু আসল



নিচের স্তর:
দ্রুত, নির্দিষ্ট
কিন্তু "অ্যাপ" দেখে না

IMS-এ ইন্টিগ্রেশন বেশি — কারণ বেশিরভাগ বাগ ছিল **অনুমতি ও URL** নিয়ে, যা শুধু HTTP-তে ধরা পড়ে।

উল্টো পিরামিড (সব এন্ড-টু-এন্ড) ধীর আর ভঙ্গুর — ১০০০ টেস্ট ২০ মিনিট নিলে কেউ চালায় না।

শিক্ষকের নোট: বলুন: "পিরামিড নিয়ম নয়, অনুপাত।" অ্যাপের ধরন অনুযায়ী ভাগ বদলায় — IMS-এ অনুমতি বেশি, তাই ইন্টিগ্রেশন বেশি।

ধাপ ৩

কভারেজ মাপা — তিনটি কমান্ড

`coverage.py` · শাখা-সহ · কোন লাইন বাদ

```
pip install coverage

# ১. টেস্ট চালিয়ে হিসাব নিন
coverage run --branch manage.py test apps
coverage report -m           # কোন লাইন বাদ, দেখায়
coverage html                # htmlcov/index.html

# ২. শাখা মোডেই চালান
# --branch না দিলে if-এর ভেতরে না ঢুকলেও লাইন
# "covering" দেখায় — সবচেয়ে বড় মিথ্যা

# ৩. বাদ দিন যা মাপার দরকার নেই
# .coveragerc
[run]
branch = True
omit = */migrations/*, */tests.py, manage.py

[report]
exclude_lines =
    pragma: no cover
    if TYPE_CHECKING:
```

IMS-এ এখন নেই

`requirements.txt`-এ `coverage` নেই, `.coveragerc` নেই — অর্থাৎ আজ পর্যন্ত কভারেজ **কখনো মাপা হয়নি**।

migrations বাদ কেন

অটো-জেনারেটেড কোড — কভারেজের সংখ্যা বাড়ায়, আত্মবিশ্বাস দেয় না।

লক্ষ্য সংখ্যা নয়

৮০% নিয়ম হিসেবে দিলে লোকেরা অর্থহীন টেস্ট লেখে; বরং **ঝুঁকিপূর্ণ ফাইল** ১০০% চাই।

কোন ফাইল ঝুঁকিপূর্ণ

`scoping.py` · `permissions.py` · `billing/services.py` · `assessments/models.py`

শিক্ষকের নোট: ক্লাসে `coverage run --branch` চালিয়ে দেখান কোন ফাইল লাল — বিশেষ করে `reports/views.py` ও `audit/views.py`।

ধাপ ৪

লাইন কভারেজের মিথ্যা

একটি ফাংশন, চারটি শাখা — একটিমাত্র টেস্টে ৭৫% "কভার"

```
def is_writable(self):
    if self.status in (PAST_DUE, SUSPENDED, CANCELLED):
        return False                          # শাখা ১
    if self.status == TRIAL:
        return self.trial_ends_on is None or self.trial_ends_on >= today   # ২
    if self.status == ACTIVE:
        return self.current_period_end is None or self.current_period_end >= today
# ৩
    return False                              # শাখা ৪

# একটি ACTIVE সাবস্ক্রিপশন নিয়ে একবার কল করলেই
# coverage বলে: "সব লাইন চলেছে" — কিন্তু
# শাখা ১, ২ ও ৪ কখনো ছোঁয়া হয়নি।
```

প্রশ্ন	উত্তর
লাইন কভারেজ	"এই লাইন একবার চলেছে"
শাখা কভারেজ	"`if`-এর দুই দিকই চলেছে"
শর্ত কভারেজ	"`or`-এর দুই অংশ আলাদা চলেছে"
IMS-এ কত শাখা	`is_writable` ৪ · `visible_classes` ৫ · `activate` ২

`or`-এর দ্বিতীয় অংশ কখনো পরীক্ষা না হলে লাইন-কভারেজ ১০০% দেখায়, অথচ একটা সীমা পুরোপুরি অজ্ঞাত।

শিক্ষকের নোট: `is_writable`-এর চারটি রিটার্ন বোর্ডে লিখুন, তারপর জিজ্ঞেস করুন: "billing-এর ৩৬ টেস্ট কি চারটিই ছোঁয়?" উত্তর: প্রায় — মেয়াদ, ACTIVE, PAST_DUE ও SUSPEND সবই আছে।

ধাপ ৫

শূন্য টেস্টের দুই অ্যাপ

`reports/` ৩০০+ লাইন · `audit/` — দুটোই অন্ধ

যা অপরীক্ষিত	ভুল হলে
CSV কলামের ক্রম	Excel-এ ডেটা সরে যায়, কেউ টের পায় না
`_csv_response`-এর BOM	বাংলা নাম ভেঙে যায়
এক্সপোর্টের স্কোপ	শিক্ষক পুরো প্রতিষ্ঠানের ডেটা নামিয়ে ফেলেন
অডিট ফিল্টার (`action` · `actor`)	ভুল রেকর্ড দেখিয়ে তদন্ত বিভ্রান্ত
paginator ৪০/পাতা	শেষ পাতা কেউ দেখেন না

প্রথম তিনটি টেস্ট যা আজই লেখা উচিত: **১.** ছাত্র CSV নামাতে পারে না (৪০৩) · **২.** CSV-র প্রথম বাইট `\ufeff` · **৩.** অন্য প্রতিষ্ঠানের ছাত্র CSV-তে নেই

এগুলো লিখতে ৩০ মিনিট; এগুলো ছাড়া ঝুঁকি **চিরকাল**।

শিক্ষকের নোট: এই স্লাইডটা সরাসরি অনুশীলনে রূপ দিন — কেউ প্রথম টেস্টটা ক্লাসেই লিখে ফেলুক। "আজ ৩০ মিনিট, নাহলে অনন্তকাল।"

ধাপ ৬

ফ্লেকি টেস্ট — মাঝে মাঝে লাল

সবচেয়ে ক্ষতিকর: দল টেস্টকে বিশ্বাস করা বন্ধ করে দেয়

কারণ	লক্ষণ	প্রতিকার
১. সময়	আজ সবুজ, কাল লাল	`today ± timedelta` ব্যবহার — IMS এই পথেই
২. র‍্যান্ডম ডেটা	কখনো ১টি, কখনো ২টি সারি	`random.seed(7)` — **IMS-এ আছে**
৩. টেস্টের ক্রম	একা চালালে পাশ, সব একসাথে ফেল	প্রতিটি টেস্ট নিজের ডেটা বানাবে; শেয়ারড অবজেক্ট নয়
৪. ধীর I/O	মেইল/ফাইল টাইমআউট	`locmem` ইমেইল ব্যাকএন্ড · অস্থায়ী `MEDIA_ROOT`
৫. অনির্দিষ্ট dict ক্রম	সারির ক্রম বদলায়	`order_by()` সবসময়; `set()` দিয়ে তুলনা

`seed_demo`-তে `random.seed(7)` — একই ডেমো প্রতিবার হুবহু একই। এটাই **reproducible fixture**।

কিন্তু `test_result_sheet_handles_absent_zero_and_boundaries` আসল তারিখের ওপর নির্ভর করে না — তাই ২০৩০ সালেও চলবে।

শিক্ষকের নোট: প্রশ্ন: "একটি ফ্লেকি টেস্ট পেলে কী করবেন — আবার চালাবেন?" উত্তর: না, কারণটা খুঁজুন। "এলোমেলো লাল মানে এলোমেলো ভরসা।"

ধাপ ৭

CI নেই — যা এখন হাতের কাজ

প্রতিটি কমিটে কেউ নিজে `manage.py test` চালান

```
# .github/workflows/test.yml — যা এখনো যোগ করা হয়নি
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_DB: ims, POSTGRES_USER: ims, POSTGRES_PASSWORD: ims }
        ports: ["5432:5432"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.13" }
      - run: pip install -r requirements.txt
      - run: python manage.py test apps
      - run: python smoke_test.py
```

এখন কী আছে

শুধু `run.sh`, `build.sh`, `smoke_test.py` — সব হাতে চালানো।

ঝুঁকি

"ভুলে যাওয়া" একটা অদৃশ্য বাগ: যেদিন কেউ টেস্ট চালায়নি, সেদিনই ভাঙা কোড মেইন-এ ঢোকে।

আগে কী যোগ করবেন

১. টেস্ট · ২. `manage.py check --deploy` · ৩. কভারেজ রিপোর্ট আর্টিফ্যাক্ট হিসেবে।

লোকাল গার্ড

CI-র আগে `pre-commit` হুক দিয়ে ফরম্যাট ও টেস্ট — ৩০ সেকেন্ডের বাধা।

শিক্ষকের নোট: "CI মানে আপনার চেয়ে ধৈর্যশীল সহকর্মী।" লোকালে ভুলে যাওয়া স্বাভাবিক — তাই স্বয়ংক্রিয় করা দরকার।

ধাপ ৮

স্যুট দ্রুত রাখা — ৪ সেকেন্ডের রহস্য

ধীর স্যুট = কেউ চালায় না = স্যুট নেই

কৌশল	লাভ
`setUpTestData` (১৬ জায়গায়)	ডেটা একবার, টেস্ট অনেক
`seed_demo` একবার চালানো	৩০ সারির বদলে একটি কল
`--keepdb`	টেস্ট-ডেটাবেস পুনঃতৈরি নয়
SQLite ডেমো সেটিংস	`config/settings/sqlite.py` — পোস্টগ্রেসের চেয়ে দ্রুত
বাইরের কল নেই	ইমেইল `console` / `locmem`, কোনো HTTP ক্লায়েন্ট নয়

১০৩ টেস্ট কয়েক সেকেন্ডে — তাই **প্রতিটি ছোট পরিবর্তনের পরেই** চালানো সম্ভব।

ধীর টেস্টের আসল খরচ সময় নয় — **মনোযোগ**। ৫ মিনিটের অপেক্ষায় কেউ আর চালায় না।

শিক্ষকের নোট: লাইভে টাইম করুন: `time python manage.py test apps`। সংখ্যাটা বোর্ডে লিখুন — সেটাই স্যুটের স্বাস্থ্য।

ধাপ ৯

সীমানার সংখ্যাগুলোই বাগের বাড়ি

৩২.৯৯% · ৩৩ · ০ · পূর্ণমান · একদিন আগে/পরে

ক্ষেত্র	সীমানা	টেস্টে আছে?
পাশ/ফেল	`pass_marks` = ৩৩ → ৩২ · ৩৩ · ৩৪	✔ `test_result_sheet_handles_absent_zero_and_boundaries`
গ্রেড ব্যান্ড	`min_percent` ও `max_percent`-এর প্রান্ত	✔ `test_grade_band_lookup`
পূর্ণমান	১০০-এর বেশি নম্বর	✔ `test_mark_above_full_marks_is_rejected`
অনুপস্থিত	`is_absent=True` + নম্বর দিলে?	✔ `test_absent_student_cannot_have_a_mark`
সিট সীমা	allowed−1 · allowed · allowed+1	✔ `test_server_refuses_the_third_student`
ট্রায়াল মেয়াদ	আজই শেষ · কাল শেষ · গতকাল শেষ	'আজ/গতকাল' পরোক্ষে; মাঝেরটি স্পষ্ট নয়

নিয়ম: প্রতিটি শর্তের **ঠিক আগে, ঠিক উপরে, ঠিক পরে** — তিনটি মান। সমতুল্য ভেতরের মান (৩৩, ৫৫, ৮৮) একটাই যথেষ্ট।

শিক্ষকের নোট: ট্রায়াল মেয়াদের তিনটি তারিখ দিয়ে `is_writable` চালিয়ে দেখান — মাঝেরটি (আজই শেষ) সবচেয়ে বেশি ভুল হয়, কারণ `>=` বনাম `>`।

ধাপ ১০

ঝুঁকি দিয়ে অগ্রাধিকার

সব জায়গায় ১০০% চাইলে কিছুই পাবেন না — তাই ক্রম

অংশ	ভুলের প্রভাব	সম্ভাবনা	অগ্রাধিকার
`scoping.py` · `permissions.py`	ডেটা ফাঁস	মাঝারি	**১ — আজ**
`billing/services.py` · `Invoice`	টাকা ভুল	মাঝারি	**১ — আজ**
`assessments` হিসাব	ভুল ফল	উচ্চ	**২**
`reports` CSV	ভুল রিপোর্ট	মাঝারি	**২** — ০ টেস্ট
`audit`	তদন্ত বিভ্রান্ত	কম	৩
`admin.py` · টেমপ্লেট	সৌন্দর্য	উচ্চ	৪ — চোখেই ধরা পড়ে

দুই মাত্রা: **ক্ষতি কত বড় × কত ঘন ঘন বদলায়**। ঘন বদলানো কোডে টেস্ট বেশি দরকার।

`reports/`-এর অবস্থান ২ নম্বরে, অথচ টেস্ট ০ — এই ফাঁকটাই আজকের ক্লাসের প্রমাণ।

শিক্ষকের নোট: দলের সঙ্গে ছকটা পূরণ করান — সবার অগ্রাধিকার এক নয়, কিন্তু "কোনটা ১ নম্বর" নিয়ে দ্বিমত থাকলে সেটাই আলোচনার শুরু।

সতর্কতা

কভারেজ নিয়ে পাঁচটি ভুল

সংখ্যাকে লক্ষ্য বানালে টেস্ট অর্থহীন হয়ে যায়

ভুল	পরিণতি
১. শতাংশকে লক্ষ্য বানানো	assert ছাড়া টেস্ট লেখা শুরু হয় — লাইন চলে, দাবি নেই
২. `--branch` বাদ দেওয়া	if-এর ভেতরে না ঢুকেও "কভার" দেখায়
৩. মাইগ্রেশন/টেস্ট কোড গোনা	সংখ্যা ফুলে ওঠে, আত্মবিশ্বাস বাড়ে না
৪. ভিউ-কভারেজে সন্তুষ্ট থাকা	helper-এর শাখা অজ্ঞাত থেকে যায় (ক্লাস ২৫-এর ফাঁক ১)
৫. রিপোর্ট না রাখা	প্রতি সপ্তাহে সংখ্যা বাড়ছে না কমছে, জানা যায় না

সবচেয়ে বড় ভুল: **ঝুঁকি ও কভারেজ আলাদা করে ভাবা**। `scoping.py`-এর ১০০% > `admin.py`-এর ১০০%।

শিক্ষকের নোট: শেষ লাইনটা বোর্ডে লিখুন। "কভারেজকে প্রশ্ন করুন — কোন ফাইল, কোন শাখা, কোন ঝুঁকি।"

অনুশীলন ও আলোচনা

২০ মিনিট — ফাঁক বন্ধ করুন

মাপুন, তারপর একটি ফাঁক বন্ধ করুন

১. মাপুন (৬ মিনিট)

`coverage run --branch manage.py test apps` → `coverage report -m`।

২. সবচেয়ে লাল ফাইল (৬ মিনিট)

শাখা-কভারেজ সবচেয়ে কম কোন ফাইলে — সেটি ঝুঁকিপূর্ণ কি না বিচার করুন।

৩. একটি টেস্ট (৮ মিনিট)

`reports`-এর জন্য একটি: ছাত্র CSV-তে ৪০৩ পাওয়া উচিত।

- লাইন কভারেজ ১০০% তবু বাগ — কীভাবে?
- ফ্লেকি টেস্ট কীভাবে চেনেন?
- CI না থাকলে সবচেয়ে বড় ঝুঁকি কী?
- কোন ফাইলে ১০০% চাইবেন?

শিক্ষকের নোট: চতুর্থ প্রশ্নের উত্তর: অনুমতি, স্কোপ, বিলিং হিসাব, মূল্যায়ন — যেখানে ভুল মানে টাকা বা গোপনীয়তা।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর অবস্থা

নীতি	IMS-এ
কভারেজ লক্ষ্য নয়, মাপকাঠি	আজ **কখনো মাপা হয়নি** — coverage ইনস্টল নেই
শাখা কভারেজই আসল	`is_writable` -এর ৪ শাখা · `visible_classes` -এর ৫
পিরামিড মানুন	৩৭ বিশুদ্ধ + ৬৬ HTTP + ১ সিস্টেম
ফ্লেকি টেস্টের উৎস সময় ও র‍্যান্ডম	`random.seed(7)` · আপেক্ষিক তারিখ
দ্রুত স্যুট = ব্যবহৃত স্যুট	`setUpTestData` ×১৬ · `seed_demo` একবার
ফাঁক লিখে রাখুন	**reports · audit ০ টেস্ট · CI নেই**

শিক্ষকের নোট: শেষ সারিটাই এই কোর্সের শিক্ষণপদ্ধতি: গর্বের তালিকার পাশে ফাঁকের তালিকা — দুইটাই সত্য।

২৭

পরের ক্লাস — "টেস্ট আগে, কোড পরে"

টেস্ট ডিজাইন: কী টেস্ট করবেন না · সীমানা ও সমতুল্য শ্রেণি · TDD চক্র · রিফ্যাক্টর নিরাপদে · পর্ব ৫ সমাপ্তি

পর্ব ৫ · টেস্টিং · ক্লাস ২৭ / ৩৭ · পর্বের শেষ ক্লাস

টেস্ট আগে, কোড পরে

সিলেবাস: প্র্যাকটিক্যাল ৯ · টপিক ৯.৪ (টেস্ট ডিজাইন ও TDD)
লাল-সবুজ-রিফ্যাক্টর · সমতুল্য শ্রেণি · ৭টি প্রশ্ন · প্রমাণ-খাতা

red · green · refactor

শুরু করুন প্রশ্ন দিয়ে

টেস্ট আগে লিখলে কী লাভ?

হাত তুলুন: কে কখনো আগে টেস্ট লিখেছেন?

লাভ একটাই, কিন্তু বিশাল: টেস্টটা ব্যর্থ হতে দেখা — তারপর কোড লেখা।

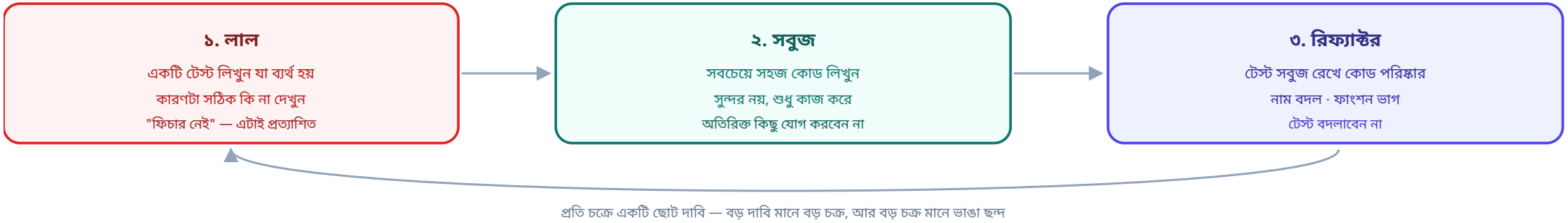
যে টেস্ট কখনো লাল হয়নি, সে টেস্ট আসলে কিছুই পরীক্ষা করছে কি না আপনি জানেন না। আজ: চক্রটি, কখন কোন টেস্ট, আর কী টেস্ট করবেন **না**।

শিক্ষকের নোট: বলুন: "সবুজ টেস্ট দেখে খুশি হবেন না — আগে লাল দেখুন।" এটাই TDD-র একমাত্র অলঙ্ঘনীয় নিয়ম।

ধাপ ১

তিন ধাপের চক্র

প্রতিটি চক্র ৫–১৫ মিনিট — বড় হলে ভেঙে ফেলুন



IMS-এ সব কোড TDD-তে লেখা হয়নি — কিন্তু যেখানে বাগ হয়েছিল (সেশন স্কোপ, গেটের URL), সেখানেই আগে টেস্ট লেখা হয়েছে।

শিক্ষকের নোট: সততার সাথে বলুন: "প্রথম থেকেই TDD কঠিন।" বাস্তব পথ: বাগ পেলেই আগে টেস্ট, তারপর ঠিক। এটাই সবচেয়ে বেশি কাজে দেয়।

ধাপ ২

একটি নতুন নিয়মে চক্রটি

"মেয়াদ শেষ হলে প্রতিষ্ঠান পড়তে পারবে, লিখতে পারবে না"

```
# ---- ১. লাল: আগে টেস্ট (is_writable এখনো নেই)
def test_expired_trial_goes_read_only_but_keeps_data(self):
    sub = self.sub
    sub.status = Subscription.Status.TRIAL
    sub.trial_ends_on = timezone.localdate() - timedelta(days=1)
    sub.save()
    self.assertFalse(sub.is_writable)                       # AttributeError = লাল
    self.assertEqual(AttendanceRecord.objects.filter(
        workspace=self.ws).count(), 42)                     # ডেটা অটুট

# ---- ২. সবুজ: সবচেয়ে ছোট কোড
@property
def is_writable(self):
    if self.status != self.Status.TRIAL:
        return False
    end = self.trial_ends_on
    return end is None or end >= timezone.localdate()

# ---- ৩. রিফ্যাক্টর: বাকি চার অবস্থা যোগ (নতুন টেস্টে, একই ফাইলে)
```

প্রথমে `AttributeError`

সেটাও এক ধরনের লাল — প্রমাণ করে টেস্টটি সত্যিই নতুন কোড ডাকছে।

"ডেটা অটুট" একই টেস্টে

নিয়মের সঙ্গে প্রতিশ্রুতিও পরীক্ষিত — নাহলে "read-only" মানে "মুছে ফেলা"ও হতে পারত।

সহজ কোড আগে

শুধু TRIAL হ্যান্ডল করে সবুজ করা — তারপর ACTIVE · PAST_DUE যোগ।

আসল কোডে

চূড়ান্ত সংস্করণে ৪টি শাখা — শেষে লেখা হয়েছিল, শুরুতে নয়।

শিক্ষকের নোট: ব্ল্যাকবোর্ডে তিন ধাপ আলাদা রঙে লিখুন, প্রতিবার টেস্ট চালানোর ফল লিখুন। ছোট চক্রের শক্তি তখনই বোঝা যায়।

ধাপ ৩

সমতুল্য শ্রেণি — সব মান নয়, প্রতিটি দল

২,৫০০টি নম্বর পরীক্ষা নয়; প্রতিটি "ধরনের" একটি

ইনপুট	শ্রেণি	প্রতিনিধি মান
নম্বর (০–১০০)	নেগেটিভ · শূন্য · ভেতরে · পূর্ণমান · বেশি	−১ · ০ · ৫৫ · ১০০ · ১০১
সিট (allowed = ৩০০)	কম · ঠিক · বেশি	২৯৯ · ৩০০ · ৩০১
পাস নম্বর ৩৩	নিচে · সমান · উপরে	৩২ · ৩৩ · ৩৪
মেয়াদ	গতকাল · আজ · কাল	শেষ হয়েছে · আজই শেষ · আগামীকাল
টেক্সট	খালি · স্পেস · খুব লম্বা	`""` · `" "` · ৪০০ অক্ষর

IMS-এ এই ছক মেনে লেখা: `test_marks_entry_rejects_out_of_range_value` · `test_server_refuses_the_third_student`।

শূন্য ও খালি সবচেয়ে বেশি বাদ পড়ে — অথচ `ZeroDivisionError` ঠিক সেখানেই লুকিয়ে থাকে।

শিক্ষকের নোট: ছকটি বোর্ডে এঁকে ছাত্রদের নিজের ফিচারের জন্য পূরণ করতে বলুন — প্রতিটি দলের একটি মান। "শ্রেণি ভাবুন, সংখ্যা নয়।"

ধাপ ৪

নেগেটিভ টেস্ট আগে

"যা হবে না" প্রমাণ করা "যা হবে" প্রমাণের চেয়ে শক্ত

```
# BUILD_STATUS.md-এর ভাষায়
| Workspaces, memberships, roles, switching,
  isolation | negative tests: 404 on foreign slug |
| Result publication gate | student gets 403 before
  publish, 200 after |
| Teacher assignments | teacher without assignment
  sees nothing |

# কোডে সেই তিনটি
self.assertEqual(resp.status_code, 404)          # অন্য স্কুল
self.assertNotContains(resp, "Rafid")            # ছাত্র তালিকা ফাঁকা
self.assertTrue(all(h.is_published for h in mine))  # হোমওয়ার্ক
self.assertEqual(scoping.visible_classes(idle, ws).count(), 0)
```

কেন শক্ত

"কাজ করে" টেস্ট পাশ করে কম কোডেও; "কাজ করে না" টেস্ট ব্যর্থ হয় বাড়তি কোডে।

নিরাপত্তার একমাত্র ভাষা

ফাঁস প্রমাণ করতে হয় `assertNotContains` বা `count()==0` দিয়ে।

পুরোনো বাগ থেকে

প্রতিটি নেগেটিভ টেস্ট একদিনের বাস্তব ফাঁসের স্মৃতি — `test_no_role_sees_a_link_it_cannot_open`।

অনুপাত

১০৩ টেস্টে `assertRaises` ১২ · `assertNotContains` ৫ · `count()==0` বহুবার।

শিক্ষকের নোট: "নতুন ফিচারের জন্য প্রথম টেস্টটি লিখুন — 'কে এটা করতে পারবে না?'" এই এক লাইনেই বেশিরভাগ নিরাপত্তা-বাগ আটকায়।

ধাপ ৫

যা টেস্ট করবেন না

ভুল জায়গায় টেস্ট লিখলে ভঙ্গুর স্যুট, আর কম ভরসা

যা নয়	কারণ ও বিকল্প
Django-র নিজের কাজ	`ForeignKey` cascade, `reverse()` — ফ্রেমওয়ার্কের টেস্ট আছে
HTML-এর হুবহু গঠন	`<div class="…">` বদলালেই ভাঙে → লেখা বা স্ট্যাটাস দেখুন
প্রাইভেট মেথড	`_next_number` সরাসরি নয় — `Invoice.save()` দিয়ে প্রমাণ
তৃতীয় পক্ষের লাইব্রেরি	`reportlab`-এর ফন্ট, `Pillow`-এর রিসাইজ — শুধু "PDF তৈরি হয়" দেখুন
লগের হুবহু বাক্য	বানান বদলালেই ২০টি টেস্ট লাল → `action` কোড দেখুন
নিজের লেখা টেস্ট নকল	একই জিনিস তিন জায়গায় — একটি টেস্ট, একটি দাবি

নিয়ম: টেস্ট করুন **আপনার সিদ্ধান্ত**, ফ্রেমওয়ার্কের নয়। "কে অনুমতি দেবে" আপনার সিদ্ধান্ত; "`reverse()` কী ফেরে" নয়।

IMS-এ `test_each_role_is_offered_only_its_own_links` লেবেল লেখার ওপর নির্ভরশীল — ভঙ্গুরতার স্বীকারোক্তি।

শিক্ষকের নোট: প্রশ্ন করুন: "কেন লগের বাক্য পরীক্ষা করা খারাপ?" উত্তর: বাক্য নয়, ঘটনাটাই চুক্তি — বাক্য বদলানো উচিত কাজ, টেস্ট ভাঙা নয়।

ধাপ ৬

লাইভ ওয়ার্কথ্রু — `reports`-এর প্রথম টেস্ট

৬ ধাপ, ৮ মিনিট — ক্লাস ২৬-এর ফাঁক বন্ধ

```
# ১. ফাইল ও ক্লাস
# apps/reports/tests.py
class ExportScopeTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_demo", verbosity=0)
        cls.ws = Workspace.objects.get(slug="sunrise-model-school")
        slug = cls.ws.slug

    # ২. প্রথম টেস্ট: ছাত্র CSV নামাতে পারবে না
    def test_student_cannot_export_csv(self):
        self.assertTrue(self.client.login(
            email="student@ims.local", password="DemoPass!2026"))
        url = reverse("reports:export_students",
                      kwargs={"slug": slug})
        self.assertEqual(self.client.get(url).status_code, 403)

    # ৩. চালান → লাল? ৪. কোড ঠিক করুন → সবুজ
    # ৫. দ্বিতীয় টেস্ট: CSV-র প্রথম বাইট BOM
    # ৬. তৃতীয়: অন্য প্রতিষ্ঠানের নাম CSV-তে নেই
```

ধাপ ১–২ কেন আগে

`setUpTestData` + seed = ১০ সেকেন্ডে বাস্তব প্রতিষ্ঠান।

ফলাফল এখনই সবুজ

ভিউয়ে `*TEACHING` আছে — তাই ৪০৩ আসবে। প্রথম টেস্ট সবুজ হওয়া মানে নিয়ম আগেই ঠিক ছিল; এখন সেটি **লক** হলো।

BOM টেস্ট

`resp.content.startswith(b"\xef\xbb\xbf")` — এটাই আসল প্রমাণ।

সময়

তিনটি টেস্ট ~৩০ মিনিটের কাজ; ক্লাসে প্রথমটি একসাথে লিখুন।

শিক্ষকের নোট: সবসময় লাল দেখতে পাবেন না — পুরোনো নিয়ম লক করাও টেস্টের কাজ। "সবুজ মানেও ভালো, যদি দাবিটা অর্থবহ হয়।"

ধাপ ৭

টেস্ট-ডেটা: seed বনাম factory

IMS বেছে নিয়েছে seed — দুই দিকেরই দাম আছে

দিক	`seed_demo`	factory (যেমন factory_boy)
প্রস্তুতির গতি	১ লাইন	প্রতি টেস্টে কয়েক লাইন
পড়ার সহজতা	ডেটা লুকানো — `seed_demo` পড়তে হয়	টেস্টেই দেখা যায় কী বানানো হচ্ছে
সূক্ষ্ম নিয়ন্ত্রণ	কঠিন (সব কিছু আগেই আছে)	সহজ (ঠিক যা দরকার)
বাস্তবতা	উচ্চ — আসল অ্যাপের মতো	কম — শুধু প্রয়োজনীয় সারি
ভাঙার ঝুঁকি	seed বদলালে অনেক টেস্ট	ছোট, আলাদা

সেরা মিশ্রণ: **seed** দিয়ে বাস্তব জগৎ, **হাতে তৈরি** ছোট সারি দিয়ে সূক্ষ্ম নিয়ম (IMS-এ `EnrolmentTests` ঠিক এটাই করে)।

`seed_demo`-এর ভেতরের সংখ্যা (৪২টি অনুপস্থিতি, "Class 8") টেস্টে হার্ডকোড করা আছে — seed বদলালে ভাঙবে। বিকল্প: টেস্টে `seed` থেকে সংখ্যা পড়ে নেওয়া।

শিক্ষকের নোট: প্রশ্ন: "কেন seed-এর সংখ্যা হার্ডকোড করলে ঝুঁকি?" উত্তর: fixture আর টেস্টের মধ্যে অদৃশ্য চুক্তি তৈরি হয় — এক পাশ বদলালে অন্যপাশ নীরবে ভাঙে।

ধাপ ৮

রিফ্যাক্টর — টেস্ট হলো জাল

ক্লাস ১৮-এর দুই mixin একসাথে করার নিরাপদ পথ

```
# আগে: দুটি mixin, একই কাজ
# apps/academics/forms.py  → BootstrapFormMixin
# apps/billing/forms.py    → ImsFormMixin

# রিফ্যাক্টরের ক্রম (টেস্ট আগে!)
def test_forms_all_get_design_classes(self):     # ১. জাল পাতুন
    f = StudentForm(workspace=self.ws).fields
    self.assertIn("form-control", f["full_name"].widget.attrs["class"])
    self.assertIn("form-select", f["class_group"].widget.attrs["class"])

# ২. সবুজ দেখে নিন (এখনো দুই কপি)
# ৩. একটিকে apps/core/forms.py-তে সরান, অন্যটি ইমপোর্ট করে
# ৪. আবার পুরো স্যুট — ১০৩ টেস্ট সবুজ মানে আচরণ বদলায়নি
```

জাল আগে

রিফ্যাক্টরের আগে টেস্ট না থাকলে "কাজ করছে" প্রমাণ করার উপায় নেই।

আচরণ অপরিবর্তিত

রিফ্যাক্টরের একমাত্র অনুমোদন: **কোনো টেস্ট বদলানো যাবে না**।

এক ধাপে এক কাজ

নাম বদল · সরানো · ভাগ করা — আলাদা কমিট, আলাদা স্যুট রান।

সীমা

টেস্ট নেই এমন জায়গায় রিফ্যাক্টর = অনুমান। আগে টেস্ট, পরে সাহস।

শিক্ষকের নোট: "টেস্ট ছাড়া রিফ্যাক্টর মানে চোখ বন্ধ করে সিঁড়ি ভাঙা।" এটাই সবচেয়ে কাজে লাগা রূপক।

ধাপ ৯

প্রমাণ-খাতা — `BUILD_STATUS.md`

দাবির পাশে প্রমাণ; আর যা হয়নি, তাও লেখা

```
| Area | Evidence |
|---|---|
| Custom user model, email login | 43 tests + screenshots |
| Workspaces, roles, isolation | negative tests: 404 on foreign slug |
| Result publication gate | 403 before publish, 200 after |
| Marksheet + PDF | `%PDF` header asserted |

## Verified numbers
python manage.py test        ->  90 tests, 90 OK
python smoke_test.py         ->  55 pages x 4 roles
manage.py check --deploy     ->  no issues

## Remaining work (not faked, not skipped)
1. Invitation workflow
2. Platform console
4. Backup & restore console
5. Security attack suite
```

"Evidence" কলাম

প্রতিটি দাবির পাশে কী প্রমাণ করে — টেস্টের সংখ্যা, নেগেটিভ টেস্ট, স্ক্রিনশট।

সংখ্যাগুলো চালিয়ে পাওয়া

"৯০ টেস্ট OK" — কমান্ডের আউটপুট, অনুমান নয়। আজ ১০৩ (স্লাইস বাড়ায়)।

"Remaining work"

যা বাকি তা **লেখা** — লুকানো নয়। এটাই পেশাদারির চিহ্ন।

কেন দরকার

নতুন ডেভেলপার (বা ভবিষ্যতের আপনি) ৫ মিনিটে অবস্থা বুঝে নিতে পারেন।

শিক্ষকের নোট: README-তে এখনো "13 tests" লেখা আছে (স্লাইস ১-এর স্ন্যাপশট) — ডকুমেন্টেশন পুরোনো হয়ে যাওয়ার বাস্তব উদাহরণ। "খাতা রাখলে তাজা রাখতে হবে।"

ধাপ ১০

নতুন ফিচারের সাতটি প্রশ্ন

প্রতিটি প্রশ্ন = অন্তত একটি টেস্ট

প্রশ্ন	টেস্টের ধরন
১. কে এটি খুলতে পারবে?	ভূমিকা × URL ম্যাট্রিক্স (৪০৩)
২. অন্য প্রতিষ্ঠান কী দেখবে?	ক্রস-ওয়ার্কস্পেস ৪০৪
৩. ভুল ইনপুটে কী হয়?	`assertRaises` বা `assertContains(..., "error")`
৪. খালি অবস্থায় কী দেখায়?	`{% empty %}` / `count()==0`
৫. মেয়াদ শেষে কী হয়?	`is_writable` · গেট
৬. দুইবার একই কাজ করলে?	ডুপ্লিকেট · `IntegrityError` · idempotent
৭. কে, কখন, কী করল?	`AuditLog` সারি আছে কি না

সাতটিই এক টেস্টে ঠেসে দেবেন না — সাতটি আলাদা টেস্ট, প্রতিটির নামে প্রশ্নটির উত্তর।

শিক্ষকের নোট: ছকটা প্রিন্ট করে দেওয়ার মতো — নতুন ফিচারের PR-এ এই সাতটি চেকবক্স রাখলে রিভিউ সহজ হয়।

ধাপ ১১

পুরো স্যুট একসাথে — এক আদেশে প্রমাণ

৫টি কমান্ড · ১০৩ টেস্ট · ৫১.৫ সেকেন্ড (SQLite)

```
# ১. পূর্ণ স্যুট (লোকাল, SQLite)
python manage.py test --settings=config.settings.sqlite
#   Ran 103 tests in 51.469s
#   OK

# ২. দ্রুত পুনরাবৃত্তি: ডিবি নতুন করে বানাবে না
python manage.py test --settings=config.settings.sqlite --keepdb

# ৩. সমান্তরাল (কোর অনুযায়ী প্রসেস)
python manage.py test --settings=config.settings.sqlite --parallel 4

# ৪. পাতা-স্তরের চোখ: ৫৫ পাতা × ৪ ভূমিকা
python smoke_test.py

# ৫. ডিপ্লয়-প্রস্তুতি: গোপনীয়তা, HSTS, ডিবাগ
python manage.py check --deploy
```

৫১.৫ সেকেন্ড মানে

প্রতি টেস্ট গড়ে ০.৫ সেকেন্ড — কফি খেতে যাওয়ার দরকার নেই, তাই কেউ ফাঁকি দেবে না।

--keepdb

টেস্ট-ডিবি মুছে নতুন করে মাইগ্রেট করে না — বারবার চালানোর জন্য।

--parallel

SQLite-এ লাভ কম, PostgreSQL-এ বেশি। প্রতিটি শ্রমিক আলাদা ডিবি পায়।

smoke_test আলাদা কেন

seeded ডেটায় **প্রতিটি URL × প্রতিটি ভূমিকা**; ৫০০ হলেই ব্যর্থ।

শিক্ষকের নোট: ক্লাসে ১ নম্বর কমান্ডটি চালিয়ে "Ran 103 tests ... OK" লাইনটি দেখান — এটাই কোর্সের সবচেয়ে বড় প্রমাণ।

ধাপ ১২

মিউটেশন মহড়া — নিজের স্যুটকে আক্রমণ

ইচ্ছাকৃত বাগ ঢুকিয়ে দেখুন টেস্ট ধরে কি না · আজ সত্যিই চালানো হয়েছে

#	ইচ্ছাকৃত পরিবর্তন	ফলাফল
A	`visible_homework` থেকে ছাত্রের `is_published=True` সরানো	**০ টেস্ট লাল** — স্যুট টের পায়নি
B	`visible_exams` থেকে একই ফিল্টার সরানো	**০ টেস্ট লাল**
C	শিক্ষকের `visible_classes` স্কোপ সরিয়ে সব ক্লাস দেওয়া	**৩ টেস্ট লাল** — `test_teacher_sees_only_assigned_classes` · `test_teacher_with_no_assignment_sees_no_students` · `EnrolmentTests.test_teacher_without_assignment_sees_no_students`

০ লাল মানে "বাগ নেই" নয় — "টেস্ট নেই"

A-এর কারণ: seed-এ একটিও অপ্রকাশিত হোমওয়ার্ক নেই, তাই `all(...)` খালি তালিকায় সত্য।

C কেন ধরা পড়ল

কারণ প্রতিটি স্কোপ নিয়মের জন্য **প্রতিপক্ষ** (অ্যাসাইনমেন্ট নেই এমন শিক্ষক) fixture-এ আছেন।

হাতের এই মহড়াই "মিউটেশন টেস্টিং"-এর হালকা রূপ; স্বয়ংক্রিয় করতে `mutmut` বা `cosmic-ray`, কিন্তু ৩টি মিউটেশন হাতে করলেই বোঝা যায় স্যুট কতটা শক্ত।

শিক্ষকের নোট: A আর B একসাথে দেখান — শিক্ষার্থীরা অবাক হবে যে ১০৩টি "OK" থাকা সত্ত্বেও দুটি নিরাপত্তা নিয়ম অরক্ষিত। এটাই এই ক্লাসের সবচেয়ে বড় শিক্ষা।

ধাপ ১৩

শূন্য-প্রমাণ টেস্ট — চার রূপ

টেস্ট সবুজ, কিন্তু কিছুই প্রমাণ করছে না

রূপ	উদাহরণ
১. প্রতিপক্ষই নেই	seed-এ অপ্রকাশিত হোমওয়ার্ক নেই → A/B টেস্ট টের পায় না
২. সব মান একই	সব নম্বর ৭০–৯৮ → সীমানা (−১, ০, ১০১) কেউ ছোঁয় না
৩. ব্যতিক্রম গিলে ফেলা	`try: … except: pass` — `assertRaises` কখনো ব্যর্থ হয় না
৪. মান যাচাই নেই	শুধু `status_code == 200`; পর্দায় কী আছে দেখা হয়নি

লক্ষণ: মিউটেশন চালালেই ধরা পড়ে — কোড ভাঙার পরও টেস্ট সবুজ।

```
# প্রতিকার: প্রতিপক্ষ নিজে বানান
def test_student_never_sees_unpublished_homework(self):
    hw = Homework.objects.filter(workspace=self.ws).first()
    hw.is_published = False          # শত্রু সারি
    hw.save()

    seen = {h.title for h in
            scoping.visible_homework(self.student, self.ws)}
    self.assertNotIn(hw.title, seen)   # এখন লাল হবে

# তারপর কোড ঠিক করুন → সবুজ
# এই একটি টেস্টই A-মিউটেশন ধরে ফেলবে
```

নিয়ম: **প্রতিটি নেগেটিভ টেস্টে একটি শত্রু সারি** নিজের হাতে তৈরি করুন — seed-এর ওপর ভরসা নয়।

শিক্ষকের নোট: বোর্ডে লিখুন: "সবুজ টেস্ট ≠ প্রমাণ। প্রমাণ = লাল হওয়ার ক্ষমতা।" এরপর ধাপ ১২-এর A মিউটেশনটি আবার চালিয়ে দেখান নতুন টেস্টটি লাল হয় কি না।

ধাপ ১৪

স্যুটের স্বাস্থ্য — ছয়টি সংকেত

সংখ্যা নয়, লক্ষণ দেখুন

সংকেত	খারাপ লক্ষণ	IMS-এ আজ
গতি	এক রানে ৫ মিনিটের বেশি → কেউ চালায় না	১০৩ টেস্ট · ৫১.৫ সেকেন্ড
নির্ধারিততা	প্রতিবার আলাদা ফল	`random.seed(7)` + `setUpTestData`
skip	“পরে ঠিক করব” জমে থাকা	০ skip
নাম	একই নাম দুই অ্যাপে → রিপোর্টে গুলিয়ে যায়	`test_teacher_without_assignment_sees_no_students` দুই অ্যাপে আছে
ব্যর্থতার বার্তা	শুধু `False is not true`	`"unpublished homework leaked to a student"`
নিরাপত্তা-আবরণ	এক স্তরে গার্ড, টেস্ট অন্য স্তরে	ভিউ-স্তরে পরীক্ষিত (৪০৩→২০০), হেল্পার-স্তরে নয়

শেষ সারিটি সবচেয়ে সূক্ষ্ম: **প্রতিরক্ষা দুই স্তরে, প্রমাণ এক স্তরে** — তাই দ্বিতীয় স্তর নীরবে সরে গেলেও স্যুট খুশি থাকে।

শিক্ষকের নোট: ছাত্রদের বলুন নিজের স্যুটে এই ছয়টি সংকেত মিলিয়ে একটি স্কোর লিখতে — কোনটিতে সবচেয়ে দুর্বলতা, সেটাই পরের কাজ।

ধাপ ১৫

ক্লাসে ডেমো স্ক্রিপ্ট — ৮ মিনিট

টার্মিনালে যা চালাবেন, পর্দায় যা দেখাবেন

মিনিট	কমান্ড	পর্দায় যা দেখাবেন
০–১	`manage.py test … --settings=config.settings.sqlite`	শেষ দুই লাইন: `Ran 103 tests in 51.469s` · `OK`
১–৩	`--verbose 2` সহ একই কমান্ড, একটি অ্যাপ	টেস্টের নামগুলো — নামেই দাবি পড়া যায়
৩–৫	একটি টেস্ট ইচ্ছাকৃত ভাঙুন (`assertEqual` → `assertNotEqual`)	লাল আউটপুট, ব্যর্থতার বার্তা, কোন লাইনে
৫–৬	সেই টেস্টে `print()` বা `--pdb`	ডিবাগার থেমে যাওয়া — `n` , `p variable`
৬–৮	`python smoke_test.py`	৫৫ পাতা × ৪ ভূমিকা — সব ২০০/৪০৩, ৫০০ নেই

প্রস্তুতি (ক্লাসের আগে): `bash run.sh --reset` একবার চালিয়ে seeded ডেটাবেস তৈরি রাখুন।

সতর্কতা: `smoke_test.py` dev সেটিংসে চলে — PostgreSQL চালু না থাকলে প্রথমেই সংযোগ-ত্রুটি দেখাবে।

শিক্ষকের নোট: লাল আউটপুট দেখানোই সবচেয়ে জরুরি — শিক্ষার্থীরা প্রথমবার দেখে ভয় পায়, তাই ভাঙা-ঠিক করা একসাথে করুন।

ধাপ ১৬

নিজের প্রকল্পের টেস্ট-অডিট — ৮টি প্রশ্ন

প্রতিটির উত্তর "না" মানে একটি কাজ

প্রশ্ন	যদি উত্তর "না" হয়
১. পুরো স্যুট ২ মিনিটে চলে?	ধীর টেস্ট ভাগ করুন, `setUpTestData` ব্যবহার করুন
২. একটি টেস্ট ইচ্ছাকৃত ভেঙে লাল দেখেছেন?	প্রথমে সেটাই করুন — না হলে সবুজের ওপর ভরসা নেই
৩. প্রতিটি ভূমিকার জন্য একটি ৪০৩ টেস্ট আছে?	ভূমিকা × URL ম্যাট্রিক্স লিখুন
৪. ক্রস-ওয়ার্কস্পেস ৪০৪ টেস্ট আছে?	দ্বিতীয় প্রতিষ্ঠান বানিয়ে একটি টেস্ট
৫. নেগেটিভ টেস্টে প্রতিপক্ষ সারি নিজে বানানো?	seed-এর ওপর ভরসা বাদ দিন
৬. কোনো skip জমে আছে?	ঠিক করুন বা মুছুন — skip মানে অদৃশ্য ঋণ
৭. নাম দেখেই দাবি বোঝা যায়?	`test_<what>_<when>_<expected>`
৮. সবুজ স্যুটের পরেও `check --deploy` চালিয়েছেন?	টেস্ট আর ডিপ্লয়-চেক আলাদা কাজ করে

শিক্ষকের নোট: এই ছকটি হোমওয়ার্ক হিসেবে দিন — নিজের প্রজেক্টের আটটি উত্তর লিখে আনলে পরের ক্লাসে সরাসরি আলোচনা করা যাবে।

পর্ব ৫ সমাপ্তি

চার ক্লাসে যা শেখা হলো

৭০ স্লাইড · ১০৩ টেস্ট · ৭টি ফাঁক · ৩টি মিউটেশন মহড়া

ক্লাস	মূল ধারণা	দখল
২৪ · টেস্টের ভিত্তি	Arrange–Act–Assert · `setUpTestData` · `subTest`	একটি টেস্ট লেখা
২৫ · অনুমতি ও স্কোপ	`scoping.py` · ৪০৩/৪০৪ · `smoke_test`	কে কী দেখবে
২৬ · কভারেজ ও ফাঁক	শাখা কভারেজ · পিরামিড · ফ্লেকি · CI	কতটা আস্থা রাখা যায়
২৭ · ডিজাইন ও TDD	লাল-সবুজ-রিফ্যাক্টর · সমতুল্য শ্রেণি	কী টেস্ট করবেন, কী নয়

চিহ্নিত ফাঁক: reports ০ · audit ০ · আপলোড ০ · ক্রস-ওয়ার্কস্পেস কেবল students-এ · CI নেই · কভারেজ মাপা হয়নি · helper টেস্ট ≠ HTTP টেস্ট

পরের পর্বে (ডিপ্লয়) এই ফাঁকগুলোই **ঝুঁকি** হয়ে ফিরে আসে — কারণ সার্ভারে ভুল হলে টেস্ট নয়, ব্যবহারকারী প্রথম জানেন।

শিক্ষকের নোট: ছাত্রদের বলুন নিজের প্রজেক্টে এই সাতটি ফাঁকের কোনটি আছে তা লিখতে — পরের ক্লাসে সেটাই আলোচনা।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

এই পর্বের পুরো সারাংশ

নীতি	প্রমাণ
লাল না দেখলে সবুজের দাম নেই	TDD চক্র · ৩ ধাপ
শ্রেণি ভাবুন, সংখ্যা নয়	৩২ · ৩৩ · ৩৪ · ২৯৯ · ৩০০ · ৩০১
নেগেটিভ টেস্ট আগে	`404 foreign slug` · `403 before publish`
আপনার সিদ্ধান্ত টেস্ট করুন	লেবেল/HTML নয় — আচরণ
রিফ্যাক্টরের একমাত্র অনুমোদন: সবুজ	১০৩ টেস্ট · কোনো টেস্ট বদল নয়
ফাঁক লিখে রাখুন	`BUILD_STATUS.md` — "not faked, not skipped"

শিক্ষকের নোট: শেষ সারিটি আবার বলুন: "সৎ খাতা রাখা নিজের প্রতি সম্মান" — যা হয়নি তা লেখাও পেশাদারির অংশ।

অনুশীলন ও আলোচনা

২০ মিনিট — এক চক্র সম্পূর্ণ করুন

লাল থেকে সবুজ পর্যন্ত, এক বসায়

১. দাবি লিখুন (৩ মিনিট)

একটি বাক্যে: "...হলে ...হবে না" — সমতুল্য শ্রেণির ৩টি মান লিখুন।

২. লাল (৫ মিনিট)

টেস্ট লিখে চালান — ব্যর্থতার কারণটি কি প্রত্যাশিত?

৩. সবুজ (৭ মিনিট)

সবচেয়ে সহজ কোড; বাড়তি কিছু নয়।

৪. রিফ্যাক্টর (৫ মিনিট)

নাম পরিষ্কার করুন — টেস্ট বদলানো চলবে না।

- TDD ধীর করে না — কেন?
- কোন টেস্টটি আপনি লিখবেন না?
- রিফ্যাক্টরে টেস্ট বদলাতে হলে?
- প্রমাণ-খাতা কে পড়ে?

শিক্ষকের নোট: তৃতীয় প্রশ্নের সঠিক উত্তর: "টেস্ট বদলাতে হলে আচরণ বদলেছে — তাই সেটি রিফ্যাক্টর নয়, নতুন ফিচার।" আলাদা কমিট করুন।

২৮

পর্ব ৬ শুরু — "ল্যাপটপ থেকে সার্ভার পর্যন্ত"

ডিপ্লয়মেন্ট: `prod.py` · gunicorn · nginx · `collectstatic` · মাইগ্রেশন · `check --deploy` · ব্যাকআপ

পর্ব ৬ · ক্লাস ২৮

ল্যাপটপ থেকে সার্ভার পর্যন্ত

ডিপ্লয়মেন্টের মৌলিক: **prod.py** · পরিবেশ চলক · Gunicorn · WhiteNoise · PostgreSQL · রিহার্সাল

IMS · ক্লাস ২৮ · স্লাইড ১–১৯

শুরু

সার্ভারে `runserver` কেন চলবে না?

কাজ করে, কিন্তু দায়িত্ব নেয় না

১. এক থ্রেড, এক অনুরোধ

ডেভ সার্ভার অনুরোধের পর অনুরোধ লম্বা লাইনে সার্ভ করে। ৩০ জন শিক্ষক একসাথে মার্কস এন্ট্রি দিলে সবাই অপেক্ষা করবে।

২. নিজেই বলে "প্রোডাকশনে চালাবেন না"

`runserver` চালু হলেই সতর্কবার্তা দেয়; নিরাপত্তা হার্ডেনিং, স্ট্যাটিক ফাইল, লগ — কোনোটাই এর কাজ নয়।

৩. DEBUG=True মানে সব গোপন তথ্য পর্দায়

একটি এরর হলেই সেটিংস, SQL, ফাইল-পথ — যা কিছু দরকার। শিক্ষকের ব্রাউজারে সেটি দেখানো মানে সেটি ফাঁস করা।

তাই ডিপ্লয়মেন্ট মানে কেবল "অন্য কম্পিউটারে চালানো" নয় — **অন্য একটি ভঙ্গি**: কঠোর সেটিংস, সত্যিকারের অ্যাপ সার্ভার, আসল ডেটাবেস, আর পেছনে ওয়েব সার্ভার।

শিক্ষকের নোট: প্রথম প্রশ্ন করুন: "আপনার প্রজেক্ট সার্ভারে দিলে কী হবে?" বেশিরভাগ উত্তর "চলবে" — আসল উত্তর "চলবে, কিন্তু ভাঙবে"।

মানচিত্র

তিনটি পরিবেশ, তিনটি উদ্দেশ্য

IMS-এ তিনটি সেটিংস ফাইল আছে — একটি নয়

দিক	dev	`sqlite` (ডেমো)	prod
DEBUG	True	True	**False**
ডেটাবেস	PostgreSQL	SQLite ফাইল	PostgreSQL
স্ট্যাটিক	ডেভ সার্ভার	ডেভ সার্ভার	WhiteNoise (সংকুচিত)
ইমেইল	কনসোলে ছাপে	কনসোলে ছাপে	SMTP
ALLOWED_HOSTS	`*`	`*`	আসল ডোমেইন, `*` নিষিদ্ধ
HTTPS	নেই	নেই	রিডাইরেক্ট + HSTS
কমান্ড	`runserver`	`run.sh`	`gunicorn`

তিনটি ফাইলই `base.py` থেকে উত্তরাধিকার নেয় — শুধু পার্থক্যটুকু ওভাররাইড করে। `from .base import *`

একটি ফাইলে `if DEBUG:` লিখে সব সামলানো যায় না — একটি ভুল মানেই প্রোডাকশনে ডিবাগ চালু।

শিক্ষকের নোট: বলুন: "ডেমো আর প্রোডাকশনের মধ্যে তফাত সেটিংসে, কোডে নয়।" একই কোড তিনটি ভঙ্গিতে চলে — এটাই সেটিংস-স্প্লিটের পুরো লাভ।

প্রথম নিয়ম

`prod.py` নিজেই থামে, ভুল মান নিলে

দুইটি রক্ষী — সত্যি চালিয়ে দেখা আউটপুট

```
# config/settings/prod.py
DEBUG = False

if SECRET_KEY == DEV_INSECURE_KEY:
    raise RuntimeError(
        "Refusing to start: SECRET_KEY is the "
        "development default. Generate one with: "
        "python -c \"import secrets;"
        " print(secrets.token_urlsafe(50))\""
    )

if not ALLOWED_HOSTS or "*" in ALLOWED_HOSTS:
    raise RuntimeError(
        "ALLOWED_HOSTS must list real hostnames "
        "in production; '*' is not allowed."
    )
```

চালিয়ে পাওয়া ফল

`RuntimeError: Refusing to start: SECRET_KEY …` ·
`RuntimeError: ALLOWED_HOSTS must list real hostnames…`

ডিজাইনের সিদ্ধান্ত

সার্ভার চালু হওয়ার আগেই ব্যর্থ — ভাঙা সাইট চালু থাকার চেয়ে বন্ধ থাকা ভালো।

একই কৌশল

`DEBUG` ডিফল্ট `False`; ভুলে চালু রাখা প্রায় অসম্ভব।

দর্শন

"ডিপ্লয় কাজ করাতে সেটিং শিথিল করবেন না — কারণ ঠিক করুন, সেটিং নয়।"

শিক্ষকের নোট: ক্লাসে ইচ্ছাকৃতভাবে পুরোনো ডেভ কী দিয়ে চালান — লম্বা এররটি পর্দায় দেখান। "সেটিং নিজে নিজেকে রক্ষা করছে।"

প্রমাণ

`check --deploy` — এক লাইনের নিরীক্ষা

আজ সত্যি চালানো: কোনো সমস্যা নেই

```
# ১. আসল কী বানান
python -c "import secrets;
print(secrets.token_urlsafe(50))"

# ২. প্রোডাকশন সেটিংসে চালান
SECRET_KEY="…" \
ALLOWED_HOSTS="ims.example.com" \
DJANGO_SETTINGS_MODULE=config.settings.prod \
python manage.py check --deploy

# আউটপুট:
System check identified no issues (0 silenced).
```

এটি কী দেখে

HSTS · SSL রিডাইরেক্ট · সিকিউর কুকি · `X-Frame-Options` · `DEBUG` · সিক্রেট · হোস্ট

কেন নিজে না করে

এটি ডিপ্লয়ের **সর্বনিম্ন** শর্ত — পাস করলেই নিরাপদ নয়, কিন্তু ফেল করলে অবশ্যই অনিরাপদ।

কোথায় চালাবেন

লোকালে rehearsel-এ **এবং** সার্ভারে ডিপ্লয়ের পরপরই — দুই জায়গায়।

শিক্ষকের নোট: "0 silenced" কথাটি ব্যাখ্যা করুন — কোনো চেক নীরবে চাপা দেওয়া হয়নি; সব চেক চলে, সব পাস করেছে।

কনফিগারেশন

প্রতিটি পরিবর্তনশীল মান = পরিবেশ চলক

কোডে কোনো পাসওয়ার্ড নয়, কোনো ডোমেইন নয়

```
# config/settings/base.py
def env(key, default=None):
    value = os.environ.get(key, default)
    if value is None:
        raise RuntimeError(
            f"Missing required environment "
            f"variable: {key}. Copy .env.example "
            f"to .env and fill it in.")
    return value

def env_bool(key, default=False):
    return env(key, str(default)).strip().lower() \
        in {"1", "true", "yes", "on"}

def env_list(key, default=""):
    raw = env(key, default)
    return [x.strip() for x in raw.split(",") if x.strip()]
```

চলক	থাকে কোথায়
`SECRET_KEY`	হোস্টের সিক্রেট স্টোর
`ALLOWED_HOSTS`	প্রতি পরিবেশে আলাদা
`CSRF_TRUSTED_ORIGINS`	আসল https ডোমেইন
`DB_NAME / DB_USER / DB_PASSWORD / DB_HOST`	ডেটাবেস প্রদানকারী
`EMAIL_HOST / USER / PASSWORD`	মেইল সেবা
`LOG_LEVEL / SECURE_SSL_REDIRECT`	অপারেটরের সিদ্ধান্ত

`.env` কখনো Git-এ নয় — `.env.example` যায়, আসলটি যায় না। ফাঁস হওয়া কী = ফাঁস হওয়া অ্যাপ।

শিক্ষকের নোট: `env()`-এর কাজ শুধু পড়া নয় — **না থাকলে স্পষ্ট বার্তা দিয়ে থামা**। নীরব ডিফল্ট সবচেয়ে বিপজ্জনক।

গোপনীয়তা

`SECRET_KEY` ফাঁস হলে কী ভাঙে

এটি পাসওয়ার্ড নয় — এটি অ্যাপের হস্তাক্ষর

সেশন কুকি

কী দিয়ে সেশন স্বাক্ষরিত — কী জানলে যে কেউ বৈধ সেশন কুকি বানাতে পারে।

পাসওয়ার্ড রিসেট টোকেন

টোকেনের স্বাক্ষরে এই কী — কী জানলে যেকোনো অ্যাকাউন্টের রিসেট লিংক বানানো যায়।

CSRF টোকেন

ফর্মের গোপন মান — কী জানলে ক্রস-সাইট অনুরোধে অনুকরণ সহজ।

বানানো: `python -c "import secrets; print(secrets.token_urlsafe(50))"` — ৫০+ অক্ষর, প্রতিটি পরিবেশে আলাদা।

ফাঁস হলে: কী পাল্টান → সব সেশন বাতিল হবে (সবাই আবার লগইন করবে), রিসেট লিংকের পুরোনো টোকেনও অচল।

শিক্ষকের নোট: জোর দিন: এই কী রোটেট করলে ব্যবহারকারীদের লগআউট হয়ে যায় — সেটি দোষ নয়, নিরাপত্তার দাম। রাতে করুন, ব্যবহার কম থাকলে।

ঠিকানা

`ALLOWED_HOSTS` ও `CSRF_TRUSTED_ORIGINS`

দুটি আলাদা দরজা, দুটি আলাদা আক্রমণ

সেটিং	কোন আক্রমণ থামায়	ভুল হলে লক্ষণ
`ALLOWED_HOSTS`	Host হেডারের জালিয়াতি — অন্য ডোমেইনের নামে অনুরোধ পাঠিয়ে পাসওয়ার্ড-রিসেট লিংক বিষাক্ত করা	অচেনা হোস্টে **400 Bad Request**
`CSRF_TRUSTED_ORIGINS`	অন্য সাইট থেকে ফর্ম সাবমিট — লগইন থাকা ব্রাউজার দিয়ে	https ডোমেইনে সাবমিট করলে **403 CSRF**
`SECURE_PROXY_SSL_HEADER`	প্রক্সির পেছনে থাকা অ্যাপকে "এটি https" বোঝানো	অসীম **রিডাইরেক্ট লুপ**

IMS-এ: `ALLOWED_HOSTS="*"` হলে `prod.py` চালুই হয় না — এই ফাঁদটা কাঠামোগতভাবে বন্ধ।

বাস্তব ফল: সাবডোমেইন বা কাস্টম ডোমেইন যোগ করলে এই দুটি তালিকা **দুই জায়গায়** আপডেট করতে হয় — নইলে 400/403।

শিক্ষকের নোট: প্রশ্ন: "আইপি দিয়ে টেস্ট করছি, ডোমেইন নেই — তখন কী?" উত্তর: হোস্টের দেওয়া ডোমেইনটাই লিখুন; `*` দিয়ে "পরে ঠিক করব" সবচেয়ে সাধারণ ও সবচেয়ে বিপজ্জনক অভ্যাস।

HTTPS

সাতটি সুইচ একসাথে

`prod.py`-তে যা ইতিমধ্যেই চালু

সেটিং	মান	কার কাজ
`SECURE_SSL_REDIRECT`	True	http → https
`SECURE_HSTS_SECONDS`	2592000 (৩০ দিন)	ব্রাউজারকে বলে: এই সাইটে আর কখনো http নয়
`SECURE_HSTS_PRELOAD`	True	ব্রাউজার-তালিকায় স্থায়ী এন্ট্রি
`SESSION_COOKIE_SECURE`	True	সেশন কুকি কেবল https-এ যাবে
`CSRF_COOKIE_SECURE`	True	একই নিয়ম CSRF কুকিতে
`SESSION_COOKIE_HTTPONLY`	True	জাভাস্ক্রিপ্ট কুকি পড়তে পারবে না
`X_FRAME_OPTIONS`	DENY	অন্য সাইটে iframe-এ বসানো যাবে না

HSTS-এর ফাঁদ: একবার ব্রাউজার এই হেডার পেলে ৩০ দিন https-ই দাবি করবে। সার্টিফিকেট না থাকলে সাইট **খুলবেই না** — তাই TLS আগে, HSTS পরে।

শিক্ষকের নোট: প্রথম দিনেই `preload` জমা দেবেন না — ৩০ দিন চালু রেখে সমস্যা নেই দেখে তারপর। ভুল হলে ফেরানো কঠিন, কারণ ব্রাউজারে হেডার জমে থাকে।

স্ট্যাটিক

`collectstatic` ও WhiteNoise

৫টি ফাইল, ৫২৮ কিলোবাইট — প্রোডাকশনে কে পরিবেশন করবে?

```
python manage.py collectstatic --noinput
# static/ -> staticfiles/
#   ims.css
#   vendor/bootstrap.min.css
#   vendor/bootstrap-icons.css
#   vendor/bootstrap.bundle.min.js
#   vendor/fonts/bootstrap-icons.woff2

# prod.py
MIDDLEWARE = ["whitenoise.middleware."
              "WhiteNoiseMiddleware"] + MIDDLEWARE
STORAGES = {
  "staticfiles": {"BACKEND": "whitenoise.storage."
                  "CompressedManifestStaticFilesStorage"},
}
WHITENOISE_MAX_AGE = 60 * 60 * 24 * 30
```

CompressedManifest কী করে

ফাইলের নামে হ্যাশ যোগ করে (`ims.7f3c1a.css`) — ৩০ দিন ক্যাশ করা নিরাপদ।

Manifest-এর ফাঁদ

টেমপ্লেটে `{% static "x.css" %}` না লিখে হার্ডকোড করলে `ValueError: Missing staticfiles manifest entry`।

WhiteNoise কেবল static

লোগো ও আপলোড করা `media/` এখান দিয়ে যায় না — আলাদা ভলিউম বা nginx দরকার।

কেন গুনছি ৫টি

CDN নেই — নিজের ৫টি ফাইলই যথেষ্ট। কমানো ফাইল-সংখ্যা = কম ভাঙার জায়গা।

শিক্ষকের নোট: `media/` ভুলে যাওয়া সবচেয়ে সাধারণ ডিপ্লয়-বাগ: রিডিপ্লয়ে লোগো মুছে যায়। ক্লাসে বলুন: "কোড আর ডেটা আলাদা — আপলোড করা ফাইল কোডের সঙ্গে যায় না।"

অ্যাপ সার্ভার

Gunicorn — `runserver`-এর জায়গা নেয় কে

Procfile-এ এক লাইনেই পুরো সিদ্ধান্ত

```
# Procfile
release: python manage.py migrate --noinput
web: gunicorn config.wsgi:application \
       --bind 0.0.0.0:$PORT \
       --workers 3 \
       --timeout 60 \
       --access-logfile - \
       --error-logfile -
```

প্যারামিটার	কারণ
`0.0.0.0:$PORT`	হোস্ট যে পোর্ট দেয়, সেটিতেই শুনতে হবে
`--workers 3`	আনুমানিক ২×CPU+১; ছোট ভিড়ে ৩টিই যথেষ্ট
`--timeout 60`	৬০ সেকেন্ডে উত্তর না দিলে শ্রমিক মারা যাবে (আটকে থাকা অনুরোধ নয়)
`--access-logfile -`	লগ stdout-এ — হোস্টের লগ ভিউয়ারে দেখা যাবে
`release:`	নতুন কোড চালু হওয়ার **আগে** মাইগ্রেশন

শ্রমিক বাড়ালে মেমরি বাড়ে, আর `CONN_MAX_AGE=60` মানে প্রতি শ্রমিক ডেটাবেসে সংযোগ ধরে রাখে — ৩ শ্রমিক × সংযোগ = হিসাব করে বাড়ান, ইচ্ছেমতো নয়।

শিক্ষকের নোট: "Gunicorn = অনেক শ্রমিক, প্রত্যেকে একটি অনুরোধ সামলায়; Django কোড অপরিবর্তিত থাকে।" WSGI কী — সংক্ষেপে: পাইথন অ্যাপ আর ওয়েব সার্ভারের মধ্যে চুক্তি।

বিল্ড

চার লাইনের `build.sh`

ক্রমটাই আসল — উল্টালে ডিপ্লয় ভাঙে

```
#!/usr/bin/env bash
set -euo pipefail          # যেকোনো ধাপ ব্যর্থ = থামো
python -m pip install --upgrade pip
pip install -r requirements-prod.txt
python manage.py collectstatic --noinput
python manage.py migrate --noinput
```

কেন `set -euo pipefail`

ব্যর্থ ধাপ চুপচাপ পেরিয়ে গেলে আধা-বিল্ড চালু হয় — সবচেয়ে বিভ্রান্তিকর ব্যর্থতা।

requirements-prod.txt

`-r requirements.txt` + `gunicorn` + `whitenoise` + `dj-database-url`

collectstatic আগে, migrate পরে

হ্যাশ করা ফাইল নাম দরকার; মাইগ্রেশন ব্যর্থ হলে নতুন কোড চালু হবে না, পুরোনো চলবে।

--noinput কেন

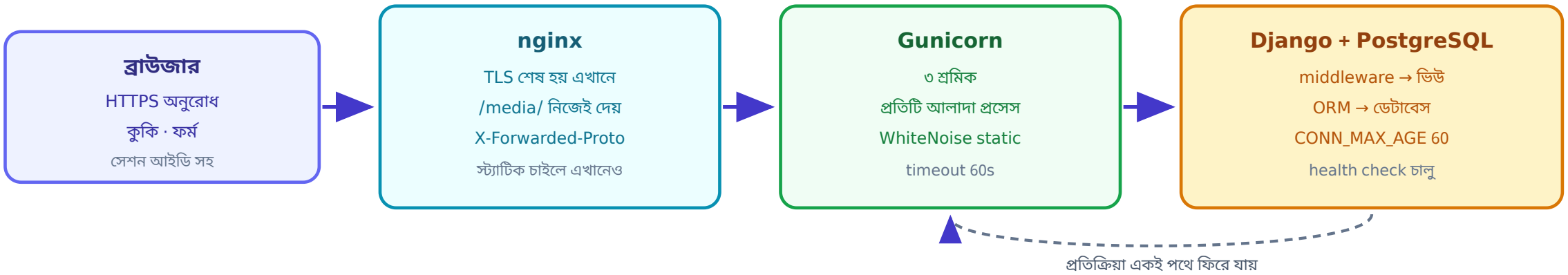
স্বয়ংক্রিয় বিল্ডে কেউ "yes" চাপবে না — প্রশ্ন করলেই বিল্ড ঝুলে থাকবে।

শিক্ষকের নোট: বলুন: "বিল্ড স্ক্রিপ্ট হলো রেসিপি — টার্মিনালে হাতে যা করেন, সেটাই লিখে রাখা।" হাতে করা ধাপ ভুলে যাওয়াই সবচেয়ে সাধারণ ডিপ্লয়-বাগ।

পথ

একটি অনুরোধের যাত্রা

ব্রাউজার → nginx → Gunicorn → Django



কেন দুটি সার্ভার: nginx ধীর ক্লায়েন্ট, TLS, ফাইল সামলায়; Gunicorn শুধু পাইথন কোড।

ভুল হলে: `SECURE_PROXY_SSL_HEADER` না থাকলে Django-র কাছে অনুরোধ http — রিডাইরেক্ট লুপ।

শিক্ষকের নোট: বোর্ডে চারটি বাক্স আঁকুন, তারপর জিজ্ঞেস করুন: "কোন ধাপে ডেটা পড়া যায়?" উত্তরে বোঝান — TLS nginx-এ শেষ, তাই nginx আর Gunicorn-এর মধ্যে প্লেইন HTTP শুধু ভেতরের নেটওয়ার্কে।

ডেটাবেস

প্রোডাকশনে PostgreSQL

তিনটি সিদ্ধান্ত: সংযোগ, মাইগ্রেশন, ডেটা

```
# prod.py — সংযোগ পুনরায় ব্যবহার
DATABASES["default"]["CONN_MAX_AGE"] = 60
DATABASES["default"]["CONN_HEALTH_CHECKS"] = True

# সার্ভারে (আসল কাজ)
createdb ims_prod
python manage.py migrate --settings=config.settings.prod
python manage.py createsuperuser --settings=config.settings.prod

# NEVER on a real deployment:
python manage.py seed_demo     # admin@ims.local + DemoPass!2026
```

CONN_MAX_AGE = 60

প্রতি অনুরোধে নতুন সংযোগ নয় — ৬০ সেকেন্ড সংযোগ ধরে রাখা; শিক্ষার্থীর তালিকা খুললে পার্থক্য বোঝা যায়।

HEALTH_CHECKS

পুরোনো সংযোগ মৃত হলে ব্যবহারের আগেই ধরা পড়ে — "server closed the connection" এরর কমে।

seed_demo নিষিদ্ধ

সবার জানা পাসওয়ার্ডের সুপারইউজার বানায় — ডেমোর জন্য ভালো, প্রোডাকশনে বিপর্যয়।

SQLite কেন নয়

এক লেখক, দুর্বল কনকারেন্সি — ৩০ শিক্ষকের মার্কস এন্ট্রিতে লক।

শিক্ষকের নোট: ডেমো ও প্রোডাকশনের ডেটাবেস আলাদা — এই সতর্কতাটি বারবার বলুন, কারণ `seed_demo` চালানো খুব সহজ।

রিহার্সাল

`deploy_check.sh` — ডিপ্লয়ের আগেই ডিপ্লয়

Gunicorn :8010, DEBUG=False, ৫টি দাবি

```
gunicorn config.wsgi:application --bind 127.0.0.1:8010 \
         --workers 2 --daemon --pid /tmp/ims-prod.pid

check "login page"               /accounts/login/     # 200
check "static css"               /static/css/ims.css  # 200
check "unknown page (404 page)"  /does-not-exist      # 404
check "workspace page anon"      /w/sunrise-model-school/  # 302

curl -sI /accounts/login/ | grep -iE \
  "x-content-type|x-frame|referrer|cross-origin"
```

৫টি প্রশ্নের উত্তর

লগইন খোলে? CSS আসে? ৪০৪ পাতা কাজ করে? অচেনা মানুষ ভেতরে ঢোকে না? হেডার আছে?

কেন লোকালে

সার্ভারে গিয়ে ভাঙার চেয়ে নিজের মেশিনে ভাঙা অনেক সস্তা।

কমDEBUG-এ যা দেখা যায়

স্ট্যাটিক ৪০৪, ৫০০ পাতার রূপ, রিডাইরেক্ট — এগুলো DEBUG=True-তে কখনো ধরা পড়ে না।

শিক্ষকের নোট: ক্লাসে চালাতে চাইলে PostgreSQL চালু থাকতে হবে — নইলে ৫০০ দেখে বিভ্রান্ত হবেন। বিকল্প: `check --deploy` আর হেডারগুলো দেখান।

চেকলিস্ট

Go-live-এর আটটি দরজা

DEPLOYMENT.md §8 থেকে বাছাই

যাচাই	না হলে
`SECRET_KEY` ৫০+ অক্ষর, হোস্টের সিক্রেট স্টোরে	কোডে লেখা কী = ফাঁস হওয়া কী
`ALLOWED_HOSTS` ও `CSRF_TRUSTED_ORIGINS`-এ আসল ডোমেইন	400 / 403, ফর্ম সাবমিট ব্যর্থ
http → https রিডাইরেক্ট কাজ করে	পাসওয়ার্ড প্লেইনে যায়
"Forgot password" একবার শুরু থেকে শেষ পর্যন্ত	SMTP ভুল — ব্যবহারকারী আটকে থাকে
`check --deploy` সার্ভারে "no issues"	অজানা ফাঁক রয়ে যায়
ব্যাকআপ চলেছে **এবং** restore একবার পরীক্ষা হয়েছে	ব্যাকআপ আছে বলেই ভরসা — ভুল ভরসা
ডেমো ডেটা মুছে (`DemoPass!2026` নেই)	যে কেউ সুপারইউজার
হোস্ট `gunicorn` চালায়, `runserver` নয়	একজন ব্যবহারকারীতেই সাইট আটকে যায়

DEPLOYMENT.md-এ লেখা আছে: "It has **not** yet been deployed to a public host" — অর্থাৎ এই ৮টি দরজা এখনো কেউ সম্পূর্ণ পেরোয়নি। সৎ খাতা রাখাই এখানেও নিয়ম।

শিক্ষকের নোট: ছাত্রদের বলুন এই ৮টি নিজের প্রজেক্টে মিলিয়ে দেখতে; যেটি "না", সেটাই পরের ক্লাসের আলোচনা।

অনুশীলন ও আলোচনা

২০ মিনিট — নিজের অ্যাপকে সার্ভারের ভঙ্গিতে চালান

প্রথম ১০ মিনিট হাতে, শেষ ১০ মিনিট প্রশ্ন

১. ডেভ কী বানান

```
python -c "import secrets; print(secrets.token_urlsafe(50))"
```

২. ভুল মান দিয়ে চালান

প্রথমে একই ডেভ কী, পরে `ALLOWED_HOSTS="*"` — দুই রকম থামা দেখুন।

৩. সঠিক মান দিয়ে `check --deploy`

"no issues" না আসা পর্যন্ত সেটিং ঠিক করুন — সেটিং নয়, কারণ ঠিক করুন।

৪. হেডার যাচাই

`curl -sI` দিয়ে HSTS, X-Frame-Options, Referrer-Policy খুঁজুন।

- DEBUG=True রাখলে কী কী ফাঁস হয়?
- HSTS আগে চালু করলে কী বিপদ?
- মিডিয়া ফাইল কোথায় থাকবে?
- শ্রমিক সংখ্যা বাড়ালে কী বাড়ে?
- `seed_demo` প্রোডাকশনে চালালে?

শিক্ষকের নোট: দ্বিতীয় ধাপে সবাই নিজের চোখে থামতে দেখুক — এটি মুখস্থ নিয়মের চেয়ে অনেক বেশি কার্যকর।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

ডিপ্লয়মেন্টের পুরো সারাংশ

নীতি	প্রমাণ
runserver সার্ভারের জন্য নয়	`Procfile`: gunicorn, ৩ শ্রমিক
সেটিংস তিন ভাগে	`base` / `dev` / `prod`
গোপন মান পরিবেশে, কোডে নয়	`env()` · `.env.example`
ভুল মানে অ্যাপ চালুই হবে না	দুটি `RuntimeError`
ডিপ্লয়ের আগে লোকালে রিহার্সাল	`deploy_check.sh` · ৫টি চেক
ব্যাকআপ যাচাই না করলে ব্যাকআপ নয়	`ops/restore.sh` — scratch ডিবি

পরের ক্লাসে: এই সার্ভারকে **দিনের পর দিন** সচল রাখা — লগ, মনিটরিং, ব্যাকআপের সময়সূচি, আপডেট, এবং রোলব্যাক।

শিক্ষকের নোট: শেষ বাক্যটি দিয়ে পরের ক্লাসের টান তৈরি করুন — "ডিপ্লয় একবারের কাজ, চালু রাখা প্রতিদিনের কাজ।"

২৯

পরিচালনা — সার্ভার চালু রাখা

লগ পড়া · মনিটরিং · ব্যাকআপ সময়সূচি · আপডেট · রোলব্যাক · ৫০০ এরর তদন্ত

পর্ব ৬ · ক্লাস ২৯

সার্ভার চালু রাখা

লগ · অডিট · ব্যাকআপ ও রিস্টোর · মাইগ্রেশন · রোলব্যাক · মনিটরিং

IMS · ক্লাস ২৯ · স্লাইড ২০–৩৮

শুরু

ডিপ্লয় একদিনের কাজ, পরিচালনা প্রতিদিনের

কেন "চালু হয়েছে" মানে "শেষ" নয়

প্রথম দিন

কেউ ভুল করে, তবু কাজ চলে। তৃতীয় সপ্তাহে: ডেটা বাড়ে, ধীর হয়, একবার ৫০০ আসে, কেউ জিজ্ঞেস করে "গত মাসে কে মার্কস বদলেছিল?"

যে তিনটি প্রশ্ন আসবেই

১. কী ভেঙেছে? (লগ) ২. কে কী করেছিল? (অডিট) ৩. কিছু হারিয়েছে কি? (ব্যাকআপ)

আর একটি

৪. এখন ঠিক করব কীভাবে, আর ফেরাব কীভাবে? (মাইগ্রেশন ও রোলব্যাক)

ডিপ্লয়মেন্ট ক্লাসে শিখেছেন "কীভাবে চালু করতে হয়"; এই ক্লাসে শিখবেন **কীভাবে টিকে থাকে** — এবং প্রথম নিয়ম: যা মাপা হয় না, তা রক্ষা করা যায় না।

শিক্ষকের নোট: জিজ্ঞেস করুন: "আপনার অ্যাপে এই চারটি প্রশ্নের উত্তর কোথায় আছে?" বেশিরভাগের উত্তর "কোথাও নেই" — এটাই এই ক্লাসের কারণ।

ছন্দ

দৈনিক · সাপ্তাহিক · মাসিক

পরিচালনার কাজ ভাগ করে নিন — সব একদিনে নয়

কখন	কাজ	IMS-এ কী দিয়ে
প্রতিদিন	ত্রুটি দেখুন · ব্যাকআপ চলেছে কি না দেখুন	হোস্টের লগ · `ops/backup.sh` (ক্রন)
প্রতিদিন	বাণিজ্যিক অবস্থা	`manage.py billing_report`
সাপ্তাহিক	ডিস্ক, ধীর কোয়েরি, বাড়তে থাকা টেবিল	ডেটাবেস প্রদানকারীর ড্যাশবোর্ড
সাপ্তাহিক	একটি রিস্টোর পরীক্ষা (scratch ডিবি)	`ops/restore.sh`
মাসিক	নির্ভরশীল প্যাকেজ আপডেট ও `check`	`pip list --outdated` · `test` · `check --deploy`
প্রতি রিলিজে	ব্যাকআপ → মাইগ্রেশন → ধোঁয়া পরীক্ষা	`build.sh` · `tests` · `smoke_test`

নিয়ম: **প্রতিটি কাজের একটি কমান্ড থাকতে হবে** — মনে রাখার মতো কাজ নয়, চালানোর মতো কাজ।

শিক্ষকের নোট: ছোট প্রতিষ্ঠানে এই ছকটি একজনেরই কাজ — তাই যত ছোট হোক, লিখিত থাকতে হবে; ছুটিতে অন্য কেউ চালাবে।

লগ

লগ কোথায় যায়

IMS-এ কেবল কনসোল — সেটাই যথেষ্ট, যদি আপনি দেখেন

```
# base.py
LOGGING = {
  "handlers": {"console": {"class":
      "logging.StreamHandler", ...}},
  "root": {"handlers": ["console"],
           "level": env("LOG_LEVEL", "INFO")},
  "loggers": {
    "django.request":  {"level": "WARNING", ...},
    "django.security": {"level": "WARNING", ...},
  },
}

# prod.py — একই কাঠামো, তবে ফরম্যাটে
# [{asctime}] {levelname} {name} {process:d}: {message}
# # প্রসেস আইডি থাকে => কোন শ্রমিক ব্যর্থ
# # হয়েছে তা বোঝা যায়
```

stdout কেন

কনটেইনার যুগে হোস্ট stdout সংগ্রহ করে — ফাইলে লিখলে রিডিপ্লয়ে হারায়।

দুই বিশেষ লগার

`django.request` = ৫০০ ও 4xx; `django.security` = CSRF, হোস্ট, সন্দেহজনক অনুরোধ।

যা এখানে নেই

ফাইল হ্যান্ডলার নেই, Sentry নেই, APM নেই — `requirements-prod.txt`-এ কোনো মনিটরিং প্যাকেজ নেই।

প্রথম সংযোজন

`LOG_LEVEL=INFO` রাখুন; এরর খুঁজতে হোস্টের লগ ভিউয়ারে `ERROR` ফিল্টার করুন।

শিক্ষকের নোট: সৎ কথা বলুন: পেশাদার সেটআপে Sentry-জাতীয় সেবা থাকে; IMS-এ নেই। শিক্ষার্থীদের প্রশ্ন করুন — "কী হারাচ্ছি এতে?" উত্তর: পাওয়া যায় না কোন ব্যবহারকারী কোন পৃষ্ঠায় ব্যর্থ হয়েছেন।

পড়া

লগে কী খুঁজবেন

চারটি প্যাটার্ন, চারটি প্রশ্ন

যা খুঁজছেন	লগে দেখতে হবে	তারপর
৫০০ এরর	`ERROR django.request Internal Server Error: /w/…`	ট্রেসব্যাক, সময়, প্রসেস আইডি
হঠাৎ ৪০৩ ঝাঁপ	`WARNING django.request Forbidden` বারবার	ভূমিকা/স্কোপ বদল হয়েছে কি না
CSRF ব্যর্থতা	`WARNING django.security.csrf`	ডোমেইন বা প্রক্সি হেডার
ধীর অনুরোধ	দেখা যায় না — সময় মাপা হয় না	এখানেই ফাঁক: পরে যোগ করতে হবে

সোনার নিয়ম: লগে ব্যক্তিগত তথ্য নয় — `AuditLog` মডেলের ডকুমেন্টেশনেই লেখা: "never store passwords, reset tokens or secrets"।

ফাঁদ: শিক্ষকের ইমেইল বা ছাত্রের নম্বর লগে ছাপালে সেটি সবার দেখার জায়গায় চলে যায় — সম্মতি ছাড়াই।

শিক্ষকের নোট: চতুর্থ সারিটি জোর দিয়ে বলুন — "যা মাপি না, তা জানি না।" পারফরম্যান্স সমস্যা ধরার একমাত্র উপায় সময় মাপা।

অডিট

অ্যাপ্লিকেশন লগ ≠ অডিট ট্রেইল

একটি ডেভেলপারের জন্য, অন্যটি প্রতিষ্ঠানের জন্য

দিক	লগ (stdout)	AuditLog (ডেটাবেস)
প্রশ্ন	"কী ভেঙেছে?"	"কে, কখন, কী বদলাল?"
থাকে কোথায়	হোস্টের লগ সংরক্ষণে (দিন-সপ্তাহ)	ডেটাবেসে — ব্যাকআপের সঙ্গে
কে পড়ে	ডেভেলপার/অপারেটর	প্রতিষ্ঠানের অ্যাডমিন (`OWNERS`)
রেকর্ড	অগঠিত লেখা	গঠিত: `action` , `object_type` , `before` , `after` , `reason` , `ip_address`
না থাকলে	অন্ধ তদন্ত	বিশ্বাস হারানো — "আমার নম্বর কে বদলাল?"

IMS-এ ২৩ ধরনের অ্যাকশন লগ হয়: `students.create` · `attendance.correct` · `assessments.publish` · `homework.review` …

নকশার সিদ্ধান্ত: `log()` কখনো এরর তোলে না — অডিট ব্যর্থ হয়ে কাজ আটকাবে না, তবে ফাঁক থেকে যেতে পারে।

শিক্ষকের নোট: প্রশ্ন: "অডিট লেখা ব্যর্থ হলে কাজ থামানো উচিত?" ভালো আলোচনা: থামালে ব্যবহারকারী ক্ষতিগ্রস্ত, না থামালে প্রমাণ হারায় — IMS দ্বিতীয়টি বেছেছে, সচেতনভাবে।

খোঁজা

অডিট ট্রেইল পড়া

পর্দায় ফিল্টার, কোডে সাধারণ ORM

```
@workspace_view(*OWNERS)
def audit_list(request, slug):
    qs = AuditLog.objects.filter(
        workspace=request.workspace).select_related("actor")
    if request.GET.get("action"):
        qs = qs.filter(action__startswith=
                       request.GET["action"])
    if request.GET.get("actor"):
        qs = qs.filter(actor_id=request.GET["actor"])
    page = Paginator(qs, 40).get_page(
        request.GET.get("page"))
    actions = (AuditLog.objects
               .filter(workspace=request.workspace)
               .values_list("action", flat=True)
               .distinct().order_by("action"))
```

কেবল OWNERS

শিক্ষক বা ছাত্র নিজের বিরুদ্ধে লেখা রেকর্ড পড়তে পারে না — `@workspace_view(*OWNERS)`।

ইনডেক্স আগেই

`(workspace, -created_at)` আর `action`-এ ইনডেক্স — তালিকা বাড়লেও ধীর হবে না।

আজকের মাপা অবস্থা

`AuditLog.objects.count()` = ০ — কারণ `seed_demo` কোনো কাজ করে না। বাস্তবে শিক্ষক যা করবেন, তা-ই জমবে।

তদন্তের ছন্দ

অ্যাকশন দিয়ে ফিল্টার → সময় দেখুন → `before` / `after` মিলিয়ে দেখুন।

শিক্ষকের নোট: ডেমোতে ০ সারি দেখে হতাশ হবেন না — ক্লাসেই একটি মার্কস সংশোধন করুন, তারপর আবার তালিকা খুলুন: নতুন সারি হাজির।

ব্যাকআপ

ব্যাকআপ = দুইটি জিনিস

ডেটাবেস + আপলোড করা ফাইল

```
#!/usr/bin/env bash
set -euo pipefail
STAMP=$(date +%Y%m%d-%H%M%S)
DEST="${BACKUP_DIR:-./backups}"
mkdir -p "$DEST"

pg_dump -Fc -f "$DEST/ims-$STAMP.dump"          # ডেটাবেস
tar -czf "$DEST/media-$STAMP.tar.gz" media/ \
    2>/dev/null || true                          # লোগো/ফাইল

find "$DEST" -name '*.dump'   -mtime +14 -delete
find "$DEST" -name '*.tar.gz' -mtime +14 -delete

# cron: রাত ২টায়
# 0 2 * * * BACKUP_DIR=/var/backups/ims \
#   bash /srv/ims/ops/backup.sh
```

কেন `-Fc`

কাস্টম ফরম্যাট = সংকুচিত + টেবিল বাছাই করে রিস্টোর করা যায়।

media আলাদা কেন

PostgreSQL-এ লোগো নেই — সেটি ফাইলে; ব্যাকআপ না করলে রিস্টোরে ফাঁকা লোগো।

১৪ দিন কেন

"গত সপ্তাহে ভুল হয়েছিল" ধরার জন্য যথেষ্ট; তার বেশি রাখলে ডিস্ক ভরে যায়।

ব্যাকআপ কোথায় রাখবেন না

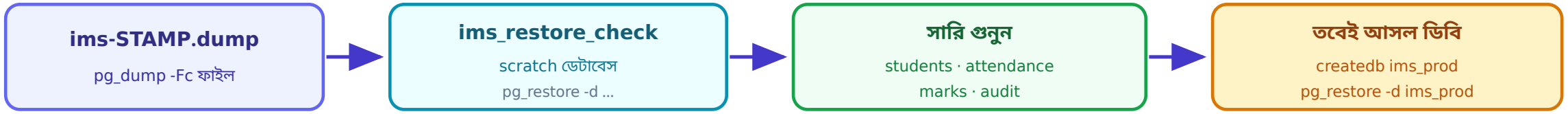
একই সার্ভারে — সার্ভার গেলে ব্যাকআপও যায়। আলাদা জায়গা বা অবজেক্ট স্টোরেজ।

শিক্ষকের নোট: প্রশ্ন করুন: "ডেটাবেসের ব্যাকআপ আছে কিন্তু media নেই — কী হারাবেন?" উত্তর: সব লোগো, আপলোড করা পড়ার উপকরণ।

রিস্টোর

আগে scratch-এ, তারপর আসল

জীবন্ত ডেটাবেসের ওপর কখনো রিস্টোর নয়



সংখ্যা মিলছে না? এখানেই থামুন — আসল ডেটাবেসে হাত দেবেন না।

> **"A backup you have never restored is not a backup."** — DEPLOYMENT.md §6। প্রতিটি রিস্টোর একই সঙ্গে ব্যাকআপের **পরীক্ষা**।

scratch ডেটাবেস রাখা যায় না বলে মুছে ফেলুন, তবে **রিস্টোরের সময় ও সারি-সংখ্যা লিখে রাখুন** — পরের বার তুলনা করার জন্য।

শিক্ষকের নোট: বলুন: "রিস্টোর শিখতে সঙ্কটের জন্য অপেক্ষা করবেন না।" মাসে একবার scratch-এ রিস্টোর করুন — ৫ মিনিটের কাজ।

মাইগ্রেশন

স্কিমা বদলানো — সবচেয়ে ঝুঁকির কাজ

ক্রম, ব্যাকআপ, আর অপরিবর্তনীয়তার নিয়ম

#	ধাপ	কেন
১	`ops/backup.sh` হাতে একবার	মাইগ্রেশন ব্যর্থ হলে ফেরার পথ থাকতে হবে
২	`makemigrations` → ফাইল পড়া	ORM যা বানিয়েছে, তা-ই তো চেয়েছিলেন?
৩	প্রথমে যোগ করুন, পরে সরান	পুরোনো কোড চলাকালীন নতুন কলাম দরকার হয়
৪	`migrate --noinput` (release step)	নতুন কোড চালু হওয়ার আগে স্কিমা তৈরি
৫	`python manage.py test` + ধোঁয়া পরীক্ষা	মাইগ্রেশনই সবচেয়ে বড় অদৃশ্য ভাঙার জায়গা

দুই ধাপের পদ্ধতি: প্রথম রিলিজে নতুন কলাম (nullable), দ্বিতীয় রিলিজে পুরোনো কলাম বাদ — এতে রোলব্যাক নিরাপদ থাকে।

ফাঁদ: `migrate` স্বয়ংক্রিয়ভাবে ডেটা ফেরায় না — কলাম মুছে ফেললে সেটি **চিরতরে** যায়।

শিক্ষকের নোট: ৩ নম্বর ধাপটি পেশাদার অভ্যাস — "expand then contract"। শিক্ষার্থীদের বলুন বড় স্কিমা বদল সবসময় দুই রিলিজে ভাগ করতে।

রোলব্যাক

"ফিরিয়ে নিন" — দুই রকম, দুই সমস্যা

কোড ফেরানো সহজ, ডেটা ফেরানো কঠিন

কোড রোলব্যাক

আগের রিলিজে ফেরানো — হোস্টের "Rollback to previous deploy"। দ্রুত, নিরাপদ, কারণ পুরোনো কোড আগেও চলেছে।

স্কিমা রোলব্যাক

`migrate app 0007` — টেবিল/কলাম ফেরে, কিন্তু নতুন রিলিজে লেখা **ডেটা ফেরে না**।

ডেটা রোলব্যাক

শুধু ব্যাকআপ থেকে। তাই রিলিজের আগে ব্যাকআপ নেওয়া = রোলব্যাকের একমাত্র পথ।

পরিস্থিতি	প্রতিক্রিয়া
নতুন পাতা ৫০০ দিচ্ছে	কোড রোলব্যাক
মার্কস ভুল দেখাচ্ছে	কোড ঠিক + অডিট দেখুন
মাইগ্রেশন অর্ধেক চলেছে	ব্যাকআপ রিস্টোর (scratch পরীক্ষা করে)
ভুল ডেটা লেখা হয়েছে অনেক আগে	অডিট `before` / `after` দিয়ে হাতে সংশোধন

নিয়ম: রোলব্যাক করার আগে **বর্তমান অবস্থার ব্যাকআপ** নিন — নইলে ভুল ঠিক করতে গিয়ে আরও ডেটা হারাবেন।

শিক্ষকের নোট: এই স্লাইডের মূল কথা: "কোডের সংস্করণ আছে, ডেটার নেই।" তাই রিলিজ-পূর্ব ব্যাকআপকে অভ্যাসে পরিণত করুন।

মনিটরিং

কী মাপবেন — ছয়টি সংখ্যা

আর IMS-এ এখন কী নেই, সেটাও স্পষ্ট

মাপ	কেন	সস্তা উপায়
uptime	সাইট খোলা আছে কি না	বাইরের পিং সেবা (৫ মিনিট পরপর)
৫০০ রেট	ব্যবহারকারী ভাঙা পর্দা দেখছে কি না	লগে `ERROR` সারি গোনা
উত্তর-সময় (p95)	ধীর হচ্ছে কি না	হোস্টের ড্যাশবোর্ড
ডেটাবেস সংযোগ	শ্রমিক × `CONN_MAX_AGE` সীমা ছাড়াচ্ছে?	ডেটাবেস প্রদানকারীর গ্রাফ
ডিস্ক	ব্যাকআপ ও `media/` ভরছে	হোস্টের অ্যালার্ট
ব্যাকআপ সফলতা	নীরবে ব্যর্থ হওয়া সবচেয়ে বিপজ্জনক	স্ক্রিপ্ট শেষে ইমেইল/cron mail

সৎ হিসাব: IMS-এ Sentry/APM/uptime বোর্ড নেই — ৬টির মধ্যে কেবল হোস্টের ডিফল্ট মেট্রিক আছে। সবচেয়ে সস্তা তিনটি আগে যোগ করুন: বাইরের পিং, লগ-অ্যালার্ট, ব্যাকআপ-ইমেইল।

শিক্ষকের নোট: "নীরব ব্যর্থতা" শব্দটি ব্যাখ্যা করুন: cron কাজ করেছে কিন্তু pg_dump ব্যর্থ — কোনো কেউ জানে না, তিন মাস পর জানা যায়।

বাণিজ্যিক

সাবস্ক্রিপশন চালানো — একটি কমান্ড

আজ সত্যি চালানো আউটপুট

```
$ python manage.py billing_report
Commercial report 2026-11-05
  active institutions : 1
  on trial            : 1
  MRR                 : ৳6,500   ARR ৳78,000
  student seats       : 10

No unpaid invoices.
```

স্ক্রিপ্ট লেখার উদ্দেশ্যই ছিল: **সকালের স্ট্যান্ড-আপে পেস্ট করা** অথবা cron-এ চালিয়ে ইমেইল করা।

কী দেখায়

সক্রিয় প্রতিষ্ঠান · ট্রায়ালে কত · MRR/ARR · মোট সিট · ৭ দিনে শেষ হওয়া ট্রায়াল · unpaid ইনভয়েস

মেয়াদ শেষে কী হয়

`SubscriptionGateMiddleware` লেখা আটকায়, পড়া চালু রাখে — ডেটা কখনো মুছে যায় না।

কে কী দেখে

অ্যাডমিন → রিনিউ পর্দায় রিডাইরেক্ট;
বাকিরা → "ask your administrator" বার্তা, কোনো বিলিং তথ্য নয়।

এখনো হাতে

অনলাইন পেমেন্ট নেই — bKash/ব্যাংক ট্রানজেকশন আইডি হাতে এন্ট্রি, ইচ্ছাকৃতভাবে।

শিক্ষকের নোট: প্রথম দিনেই `billing_report` চালান — বাস্তব সংখ্যা দেখলে "SaaS চালানো" বিমূর্ত থাকে না।

ইন্সিডেন্ট

কিছু ভেঙেছে — পাঁচ ধাপ

শৃঙ্খলা, তারপর দক্ষতা

#	ধাপ	কাজ
১	থামান ও জানান	প্রভাব বোঝা: কে আটকে, কী হারাচ্ছে। ব্যবহারকারীকে সৎ বার্তা দিন।
২	পুনরুৎপাদন	কোন URL, কোন ভূমিকা, কোন সময় — লগে সেই মুহূর্ত খুঁজুন।
৩	সীমিত করুন	দরকার হলে ফিচার বন্ধ/রোলব্যাক — ডেটা বাঁচানো আগে, কারণ পরে।
৪	ঠিক করুন + টেস্ট	প্রথমে ব্যর্থ টেস্ট (ক্লাস ২৭-এর লাল-সবুজ), তারপর সমাধান।
৫	লিখে রাখুন	কী ঘটল, কেন, কী বদলাল — পরের জন যেন একই ফাঁদে না পড়ে।

দোষ নয়, কারণ: "কে ভুল করল" নয় — "কোন নিয়ম অনুপস্থিত ছিল" এই প্রশ্নেই আসল উত্তর মেলে।

সময় লিখুন: টাইমস্ট্যাম্প ছাড়া লগ অর্থহীন — সার্ভারের সময় অঞ্চল `Asia/Dhaka` কিনা নিশ্চিত করুন।

শিক্ষকের নোট: ক্লাসে একটি কাল্পনিক ইন্সিডেন্ট বানিয়ে পাঁচ ধাপ হাঁটুন: "শিক্ষক মার্কস সংরক্ষণ করতে পারছেন না, ৫০৩ আসছে।"

৫০০ পাতা

ব্যবহারকারী যা দেখে, আর ডেটা যা ভোগে

ভদ্র বার্তা, লেনদেন অটুট — তবে একটি প্রতিশ্রুতি এখনো অপূর্ণ

```
Something went wrong on our side
The error has been written to the application
log with a reference number. No data was
lost and nothing you entered was half-saved
— every write is wrapped in a database
transaction.

If it keeps happening, contact the
administrator with the time it occurred.

[Try the dashboard again]
```

যা সত্যি

DEBUG=False-এ ডেটা ফাঁস হয় না; লেখা `transaction.atomic`-এ মোড়া, তাই অর্ধেক লেখা থাকে না।

যা এখনো সত্যি নয়

"reference number" লেখা আছে, কিন্তু কোডে এমন কোনো আইডি তৈরি হয় **না** — ভবিষ্যতের কাজ।

কেন গুরুত্বপূর্ণ

ব্যবহারকারী সময় বললে আপনি লগে খুঁজতে পারেন — আইডি থাকলে খোঁজা সেকেন্ডের কাজ হতো।

শিক্ষা

বার্তা যত সুন্দর, প্রতিশ্রুতি তত যাচাইযোগ্য হতে হবে — নইলে ভরসা ভাঙে।

শিক্ষকের নোট: এই স্লাইডটি সততার উদাহরণ হিসেবে ব্যবহার করুন — কোড আর লেখা মিলছে না, সেটি লিখে রাখাই পেশাদারির প্রথম ধাপ।

পরিবর্তন

পরিবর্তন ব্যবস্থাপনা — ছোট অভ্যাস, বড় রক্ষা

প্রতিটি রিলিজ নিজের গল্প বলুক

অভ্যাস	কেন
এক কমিট = এক পরিবর্তন	রোলব্যাক করতে হলে ছোট টুকরো ফেরানো সহজ
কমিট বার্তায় "কী + কেন"	"fix" লিখলে ছয় মাস পর কিছুই বোঝা যায় না
রিলিজ নোট: কী বদলাল, কী মাইগ্রেশন	কেউ জিজ্ঞেস করলে এক জায়গায় উত্তর
বড় বদলের আগে `--dry-run` / `sqlmigrate`	চালানোর আগে SQL দেখে নেওয়া
রিলিজের আগে ব্যাকআপ, পরে ধোঁয়া পরীক্ষা	ফেরার পথ আর প্রমাণ — দুটোই দরকার

IMS থেকে: ১৯টি কমিট, প্রতিটি একটি শ্রেণি-কাজের সীমানা; `BUILD_STATUS.md`-এ প্রমাণের খাতা।

যা এখনো নেই: CI পাইপলাইন — রিলিজের আগে স্বয়ংক্রিয় টেস্ট এখনো হাতে চালাতে হয়।

শিক্ষকের নোট: বলুন: "ভবিষ্যতের আপনি অচেনা মানুষ — তাকে চিঠি লিখে রাখুন।" কমিট বার্তা সেই চিঠি।

অনুশীলন ও আলোচনা

২০ মিনিট — পরিচালনার চার হাতে-কলমে কাজ

প্রতিটি কাজ ৫ মিনিট

১. অডিট জন্ম দিন

একটি নম্বর সংশোধন করুন → `audit_list` খুলে নতুন সারি দেখুন।

২. ব্যাকআপ নিন

`BACKUP_DIR=/tmp/bk bash ops/backup.sh` → ফাইলের আকার দেখুন।

৩. রিস্টোর পরীক্ষা করুন

scratch ডিবিতে রিস্টোর → চারটি টেবিলের সারি গুনে মিলিয়ে দেখুন।

৪. বাণিজ্যিক ছবি তুলুন

`billing_report` চালিয়ে সংখ্যাগুলো নিজের ভাষায় ব্যাখ্যা করুন।

- লগ আর অডিট — কোনটি কতদিন রাখবেন?
- ব্যাকআপ একই সার্ভারে রাখলে?
- migrate ফেরানো কি সবসময় সম্ভব?
- ব্যাকআপ নীরবে ব্যর্থ হলে বুঝবেন কীভাবে?
- ৫০০ পাতার প্রতিশ্রুতি রক্ষা করবেন কীভাবে?

শিক্ষকের নোট: তৃতীয় প্রশ্নের উত্তর: "নতুন রিলিজে লেখা ডেটা ফেরে না — তাই ব্যাকআপই আসল রোলব্যাক।"

সারমর্ম

ছয়টি কথা সঙ্গে নিন

পরিচালনার পুরো সারাংশ

নীতি	প্রমাণ / কমান্ড
যা মাপা হয় না, রক্ষা করা যায় না	লগে ৫০০ · p95 · ব্যাকআপ সফলতা
লগ ডেভেলপারের, অডিট প্রতিষ্ঠানের	২৩ অ্যাকশন · `AuditLog` · `OWNERS`
ব্যাকআপ = ডেটাবেস + `media/`	`pg_dump -Fc` · `tar -czf` · ১৪ দিন
রিস্টোর না করা ব্যাকআপ ব্যাকআপ নয়	`ops/restore.sh` → scratch ডিবি
কোডের সংস্করণ আছে, ডেটার নেই	রিলিজ-পূর্ব ব্যাকআপ
ইন্সিডেন্টে শৃঙ্খলা, দোষ নয়	৫ ধাপ · লাল টেস্ট আগে

পরের ক্লাস (৩০): ডিপ্লয়মেন্ট পর্বের শেষ ধাপ — পূর্ণাঙ্গ রিহার্সাল, সম্পূর্ণ go-live, আর পর্ব ৭-এর সূচনা।

শিক্ষকের নোট: ক্লাস শেষে প্রত্যেককে একটি কাজ দিন: "আপনার প্রজেক্টে একটি ব্যাকআপ স্ক্রিপ্ট লিখুন এবং একবার রিস্টোর করে দেখান।"

৩০

পূর্ণাঙ্গ রিহার্সাল ও go-live

শেষ তিনটি দরজা · DNS · প্রথম ব্যবহারকারী · ফোনে পাওয়া প্রথম কল

পর্ব ৬ · ক্লাস ৩০

Go-live — চালু করার দিন

DNS · TLS · অনবোর্ডিং · ডেমো ডেটা সরানো · প্রথম ৪৮ ঘণ্টা · রোলব্যাক সিদ্ধান্ত

IMS · ক্লাস ৩০ · স্লাইড ৩৯–৫৬

শুরু

চালু করার দিন সবচেয়ে বেশি ভুল হয়

পাঁচটি বাস্তব ঝুঁকি — প্রতিটির প্রতিকার আগেই জানা

ঝুঁকি	লক্ষণ	প্রতিকার
DNS এখনো ছড়ায়নি	"সাইট খুলছে না", অথচ সার্ভার ঠিক	চালু করার আগেই রেকর্ড দিন (TTL ছোট করুন)
`ALLOWED_HOSTS` পুরোনো	হঠাৎ **400 Bad Request**	ডোমেইন + `CSRF_TRUSTED_ORIGINS` আগেই বসান
ডেমো ডেটা রয়ে গেছে	কেউ `admin@ims.local` দিয়ে ঢুকে পড়ে	যাওয়ার আগে মুছুন — চেকলিস্টে আছে
ইমেইল কাজ করছে না	পাসওয়ার্ড রিসেট "পাঠানো হয়েছে" কিন্তু আসে না	একবার নিজে "Forgot password" চালান
TLS নেই, HSTS চালু	সাইট **খুলবেই না**	TLS আগে, HSTS পরে

নিয়ম: **প্রতিটি ঝুঁকির একটি যাচাইয়ের কমান্ড থাকবে** — "দেখে নিয়েছি" নয়, "আউটপুট নিয়েছি"।

শিক্ষকের নোট: পাঁচটি ঝুঁকির মধ্যে চারটিই কনফিগারেশন, কোড নয়। শিক্ষার্থীদের বলুন: "Go-live-এর ৮০% কাজ আগের সপ্তাহের প্রস্তুতি।"

কাউন্টডাউন

D-14 থেকে D-0

আগেই করুন, যাতে শেষ দিনে কেবল দেখাশোনা করতে হয়

দিন	কাজ	প্রমাণ
D-14	আসল আয়োজক বেছে নিন · ডোমেইন কেনা · TLS	https খোলে
D-10	প্রোডাকশন সেটিংস ও `.env` ভরা	`check --deploy` → no issues
D-7	প্রথম প্রতিষ্ঠানের ডেটা তৈরি (year→term→class→subject)	স্ক্রিনশট
D-5	ডেমো ডেটা সরানো · আসল অ্যাডমিন অ্যাকাউন্ট	পুরোনো লগইন ব্যর্থ
D-3	রিস্টোর ড্রিল — scratch ডিবিতে	সারি-সংখ্যা মিলেছে
D-1	ব্যাকআপ · ধোঁয়া পরীক্ষা · ব্যবহারকারীদের জানানো	চেকলিস্টে টিক
D-0	শুধু মনিটরিং — নতুন কিছু বদলাবেন না	লগ ও ৫০০-রেট

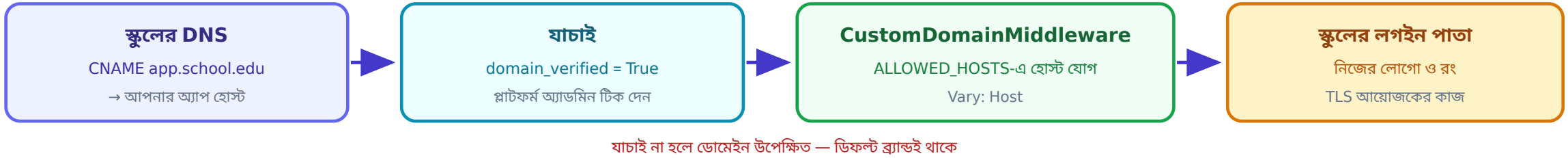
D-0-তে নতুন কিছু বদলানো **নিষিদ্ধ** — পরিবর্তন আর পর্যবেক্ষণ একসাথে করলে কোনটি সমস্যা বানাল, বোঝা যায় না।

শিক্ষকের নোট: ক্লাসে কাউন্টডাউনটি বোর্ডে লিখুন এবং প্রত্যেককে নিজের প্রজেক্টের জন্য তারিখ বসিয়ে নিতে বলুন।

DNS

কাস্টম ডোমেইন — তিন স্তর

DNS · যাচাই · রানটাইমে হোস্ট গ্রহণ



যাচাই না হলে ডোমেইন উপেক্ষিত — ডিফল্ট ব্র্যান্ডই থাকে

তিনটি টেস্টই প্রমাণ

`test_verified_domain_shows_that_institution_on_the_login_page` ·
`test_unverified_domain_is_ignored` ·
`test_domain_cannot_be_stolen_by_a_second_institution`

এক ডোমেইন, এক স্কুল

ডেটাবেসে `UniqueConstraint` — দ্বিতীয়বার লেখা যায় না।

```
if branding is not None:
    request.domain_workspace = branding.workspace
    if host not in settings.ALLOWED_HOSTS:
        settings.ALLOWED_HOSTS = (
            list(settings.ALLOWED_HOSTS) + [host])
```

শিক্ষকের নোট: প্রশ্ন: "TLS কে দেয়?" উত্তর: আয়োজক — এই অ্যাপ সার্টিফিকেট বানায় না; অ্যাপ শুধু হোস্ট গ্রহণ করে। "ডোমেইন দখল" টেস্টটির কথা বলুন: একটি ভুল রেকর্ড অন্য স্কুলের ব্র্যান্ড দেখাতে পারবে না।

কমান্ড

`sync_domains` — অনুমান বন্ধ করে

আজ সত্যি চালানো: এখনো কোনো কাস্টম ডোমেইন নেই

```
$ python manage.py sync_domains
No institution has a custom domain yet.

# ডোমেইন যোগ হলে যা ছাপে:
  app.school.edu    Sunrise Model School   verified
  old.school.edu    Sunrise Model School   PENDING

DNS records the school must create:
  CNAME  app.school.edu  ->

Environment on the server:
  ALLOWED_HOSTS=,app.school.edu
  CSRF_TRUSTED_ORIGINS=https://,https://app.school.edu
```

কেন একটি কমান্ড

DNS রেকর্ড আর দুইটি তালিকা হাতে লিখতে গিয়ে ভুল হয় — কমান্ড নকল করার মতো আউটপুট দেয়।

PENDING মানে কী

যাচাই হয়নি — এই ডোমেইন আপাতত ব্র্যান্ড পাবে না, `ALLOWED_HOSTS`-এও যাবে না।

একটি ফাঁক

`ALLOW_PREFIXES`-এ `/healthz` আছে, কিন্তু এমন কোনো URL এখনো নেই — uptime মনিটরের জন্য দরকার।

শিক্ষকের নোট: ফাঁকটি সৎভাবে দেখান: "নীতির তালিকায় আছে, বাস্তবে নেই — এভাবেই ডকুমেন্টেশন আর কোডের দূরত্ব তৈরি হয়।"

TLS

সার্টিফিকেট কার কাজ

অ্যাপ নয় — আয়োজক বা প্রক্সি

স্তর	দায়িত্ব	IMS-এ কী আছে
DNS	CNAME/A রেকর্ড	স্কুল করে · `sync_domains` যাচাইয়ে সাহায্য করে
TLS সার্টিফিকেট	আয়োজক (Render/Fly/Caddy) বা nginx + certbot	অ্যাপে নেই — ইচ্ছাকৃত
রিডাইরেক্ট ও HSTS	Django	`SECURE_SSL_REDIRECT` · `SECURE_HSTS_SECONDS`
প্রক্সি হেডার	nginx → অ্যাপ	`SECURE_PROXY_SSL_HEADER`
কুকি সুরক্ষা	Django	`Secure` · `HttpOnly` · `SameSite=Lax`

ক্রম: DNS → TLS কাজ করছে → তারপর HSTS। উল্টো করলে সাইট বন্ধ হয়ে যাবে, ফেরানো কঠিন।

যাচাই: `curl -sI https://…/accounts/login/` — `strict-transport-security` আর `x-frame-options` দেখা যায় কি না।

শিক্ষকের নোট: "সার্টিফিকেট মেয়াদ শেষ" সবচেয়ে সাধারণ আউটেজ — আয়োজক স্বয়ংক্রিয় নবায়ন করে কি না, প্রথম দিনেই দেখে রাখুন।

অনবোর্ডিং

একটি স্কুল, ২০ মিনিটে

LAUNCH_PLAYBOOK §4 — ছয় ধাপ

#	ধাপ	কোথায় / কীভাবে
১	অ্যাকাউন্ট	নিজে সাইন আপ, বা প্লাটফর্ম কনসোল → নতুন প্রতিষ্ঠান (প্ল্যান, অ্যাডমিন ইমেইল, অস্থায়ী পাসওয়ার্ড)
২	ব্র্যান্ডিং	লোগো · রং · tagline · support ইমেইল · লগইন বার্তা
৩	কাঠামো	শিক্ষাবর্ষ → টার্ম → বিভাগ → ক্লাস → বিষয় → শিক্ষক-অ্যাসাইনমেন্ট
৪	মানুষ	শিক্ষক (Teacher), কোঅর্ডিনেটর (স্কোপ সারি) — তার বেশি কিছু নয়
৫	ছাত্র	UI দিয়ে যোগ করুন — CSV আমদানি এখনো তৈরি হয়নি
৬	রিভার্সাল	একটি রিপোর্ট কার্ড একসাথে দেখুন — **এটাই তারা কিনেছে**

৬ নম্বর ধাপ বাদ দিলে ইনভয়েসের পরে "এটা তো আমরা চাইনি" শোনার ঝুঁকি — রিপোর্ট কার্ডই আসল পণ্য।

শিক্ষকের নোট: ক্লাসে ২০ মিনিটের অনবোর্ডিংটি লাইভ করুন — শিক্ষার্থীরা দেখবে কাঠামো আগে, মানুষ পরে; উল্টো করলে স্কোপ ভাঙে।

সেটআপ

ব্র্যান্ডিং, স্কোপ ও সিট সীমা

তিনটি জিনিস যা প্রথম দিনেই ঠিক হতে হবে

ব্র্যান্ডিং

`display_name` · `tagline` · `logo` · `primary_color` · `login_message` · `support_email` · `support_phone`

কোঅর্ডিনেটর স্কোপ

প্রতি বিভাগ বা ক্লাসে একটি সারি — "`CoordinatorScope`"। না দিলে সে কিছুই দেখে না, যা ভুল নয় — নিরাপদ।

সিট সীমা

ইনভয়েস নয় — **আসল ছাত্র সারি** গুনে প্ল্যান-সীমা প্রয়োগ হয়, সার্ভারে।

কে কী বদলাতে পারে

ডোমেইন ও প্ল্যান: কেবল প্লাটফর্ম অ্যাডমিন। নিজের ব্র্যান্ডিং ও লোক: প্রতিষ্ঠান অ্যাডমিন।

কেন ডোমেইন owner-only: একটি ভুল ডোমেইন অন্য প্রতিষ্ঠানের লগইন পাতায় ভুল ব্র্যান্ড দেখাবে — লিক, যদিও ডেটা নয়।

শিক্ষকের নোট: সিট সীমা প্রসঙ্গে বলুন: "বিলিং সত্য বলছে কি না, তা ডেটাবেস গুনেই ঠিক হয় — কাগজ নয়।"

পরিচ্ছন্নতা

ডেমো ডেটা সরানো — বিপদের এক নম্বর জায়গা

`seed_demo` সবার জানা পাসওয়ার্ড বানায়

```
# যা সত্যিই বিপজ্জনক
admin@ims.local         # সুপারইউজার + প্রতিষ্ঠান অ্যাডমিন
DemoPass!2026           # প্রতিটি ডেমো অ্যাকাউন্টের পাসওয়ার্ড
sunrise-model-school    # ডেমোর প্রতিষ্ঠান

# সরানোর ধাপ (নিজের দায়িত্বে, আগে ব্যাকআপ)
python manage.py shell
>>> from apps.accounts.models import User
>>> User.objects.filter(email__endswith="@ims.local").delete()
>>> User.objects.filter(is_superuser=True,
...     email="admin@ims.local").delete()

# আসল অ্যাডমিন
python manage.py createsuperuser --settings=config.settings.prod
```

যাচাই

পুরোনো লগইন চেষ্টা করুন — ব্যর্থ হওয়া **উত্তর**, অনুমান নয়।

ডেমো ডেটা মুছলে কী ভাঙে

আসল কাঠামো (বর্ষ, ক্লাস, বিষয়) নিজে বানাতে হবে — তাই আগে দেখে নিন কোনটি ডেমো, কোনটি আসল।

প্রথম দিনের সবচেয়ে বড় ভুল

"পরে সরাব" — পরে আর কেউ মনে রাখে না। চেকলিস্টে এটি থাকার কারণই এই।

শিক্ষকের নোট: `delete()` চালানোর আগে অবশ্যই ব্যাকআপ — ক্লাসেই এই শৃঙ্খলা অভ্যাস করান। আরও নিরাপদ: purging না করে নতুন প্রোডাকশন ডেটাবেসে শুধু আসল প্রতিষ্ঠানটি রাখা।

ইমেইল

পাসওয়ার্ড রিসেট — একবার নিজে চালান

সবচেয়ে বেশি আসা সাপোর্ট টিকেট

```
# dev / sqlite: কনসোলে ছাপে
EMAIL_BACKEND=django.core.mail.backends.console.EmailBackend
# prod: SMTP (Brevo / Mailgun / Gmail app password)
EMAIL_HOST=smtp-relay.brevo.com
EMAIL_PORT=587
EMAIL_USE_TLS=True
EMAIL_HOST_USER=…
EMAIL_HOST_PASSWORD=…
DEFAULT_FROM_EMAIL=IMS

# যাচাই: /accounts/password-reset/ খুলে নিজের
# ইমেইল দিন, লিংকটি মেইলে আসা চাই
```

কনসোল ব্যাকএন্ডে প্রোডাকশনে

মেইল কোথাও যায় না — শুধু সার্ভারের লগে ছাপে। ব্যবহারকারী চিরকাল অপেক্ষা করবে।

SPF/DKIM

সেবা প্রদানকারীর ডোমেইন যাচাই না করলে মেইল স্প্যামে যাবে — "লিংক আসেনি" এররটি এখানেই।

নীরবতা নিয়ম

অ্যাকাউন্ট আছে কি নেই — উত্তর একই। তবু ব্যবহারকারী ভাবেন "মেইল এল না"; তাই সাপোর্ট স্ক্রিপ্টে এটি আগেই বলুন।

টোকেনের গোপনীয়তা

`SECRET_KEY` বদলালে পুরোনো রিসেট লিংক অচল — নিরাপত্তার দাম, জানিয়ে রাখুন।

শিক্ষকের নোট: ক্লাসে কনসোল ব্যাকএন্ডে রিসেট চালিয়ে টার্মিনালে লিংক দেখান — তারপর বলুন, প্রোডাকশনে ঠিক এই লিংকটাই মেইলে যাবে।

D-0

Go-live দিনের রানবুক

H-2 থেকে H+2 — যা করবেন, যে ক্রমে

সময়	কাজ	প্রমাণ
H-2	ব্যাকআপ নিন · `migrate` চলেছে কি না দেখুন	dump ফাইলের আকার
H-1	`tests` + `smoke_test` + `check --deploy`	তিনটি আউটপুট
H-0	রিলিজ · লগইন · একটি রিপোর্ট কার্ড · একটি CSV	স্ক্রিনশট
H+0.5	লগে `ERROR` আছে কি না	গোনা সংখ্যা
H+1	প্রথম ব্যবহারকারীকে ফোন/সরাসরি সাহায্য	একটি সমাধান হওয়া সমস্যা
H+2	ব্যাকআপ আবার · বাণিজ্যিক রিপোর্ট	`billing_report`

রানবুকে **"নতুন কিছু বদলাবেন না"** লাইনটি রাখুন — আজকের কাজ পর্যবেক্ষণ, উন্নয়ন নয়।

শিক্ষকের নোট: এই ছকটি প্রিন্ট করে দেওয়ার মতো — শিক্ষার্থীরা নিজের প্রকল্পের go-live-এ ব্যবহার করবে।

৪৮ ঘণ্টা

প্রথম দুই দিনে কী দেখবেন

সংখ্যা, গুজব নয়

সংকেত	স্বাভাবিক	উদ্বেগের
৫০০ এরর	০ (এক-দুই হলে দেখে নিন)	প্রতি ঘণ্টায় একাধিক
৪০৩	প্রথম দিনে কিছু — নতুন ভূমিকা শিখছে	একই ব্যবহারকারীর জন্য বারবার
লগইন ব্যর্থতা	প্রথম ঘণ্টায় কয়েকটি (পাসওয়ার্ড ভুল)	একটি অ্যাকাউন্টে কয়েকবার — লকআউট নেই
ধীর পাতা	বড় রিপোর্ট ধীর	ড্যাশবোর্ডও ধীর
সাপোর্ট কল	"কীভাবে..." ধরনের	"আমার ডেটা ভুল" ধরনের

যেকোনো "ডেটা ভুল" কল = সাথে সাথে **অডিট ট্রেইল** খুলুন — কার, কখন, `before` / `after`। এটি IMS-এর সবচেয়ে দামি সরঞ্জাম।

শিক্ষকের নোট: বলুন: "সাপোর্ট কলের ধরনই বলে দেয় সাইট স্বাস্থ্যকর কি না — 'কীভাবে' ভালো, 'ভুল দেখাচ্ছে' খারাপ।"

সিদ্ধান্ত

কখন রোলব্যাক করবেন

তিনটি ট্রিগার — আগেই ঠিক করে রাখুন

১. ডেটা লিখতে বাধা

শিক্ষক মার্কস/উপস্থিতি সংরক্ষণ করতে পারছেন না — এটি ব্যবসার ক্ষতি, এক ঘণ্টাও চলবে না।

২. নিয়ম ভাঙছে

এক ভূমিকা অন্যের ডেটা দেখছে — নিরাপত্তা ঘটনা; ফেরানোই সঠিক উত্তর, "পরে ঠিক করব" নয়।

৩. ৫০০ ছড়িয়ে পড়ছে

একই এরর অনেক ব্যবহারকারীর কাছে — আগের রিলিজে ফিরে লগ পড়া শুরু করুন।

নয়: "কেউ অভিযোগ করেছে" বা "পর্দায় রং অন্য রকম" — এগুলো রিলিজ বন্ধ করার কারণ নয়, তালিকা করার।

আগে: রোলব্যাকের আগেও ব্যাকআপ — নতুন রিলিজে লেখা ডেটা ফেরার একমাত্র পথ।

শিক্ষকের নোট: সিদ্ধান্ত আগে নেওয়ার কারণ: সংকটের মুহূর্তে সবাই চাপে থাকে, তখন নতুন নীতি ঠিক করা যায় না।

সাপোর্ট

ব্যবহারকারীকে কী বলবেন

সৎ, সংক্ষিপ্ত, নির্দিষ্ট — তিনটি নিয়ম

যা বলবেন না	যা বলবেন
"সার্ভারে সমস্যা হচ্ছে"	"মার্কস সংরক্ষণ এখন কাজ করছে না — ২০ মিনিটে জানাচ্ছি"
"আপনার ব্রাউজারে সমস্যা"	"আমরা দেখছি; আপনি কোন পর্দায় ছিলেন?"
"ডেটা হারাবে না" (না জেনে)	"ডেটা অটুট আছে কি না ৫ মিনিটে নিশ্চিত করছি"
"শীঘ্রই ঠিক হবে"	"আগামী ৩০ মিনিটে আপডেট দেব"

ভরসা একবারে ভাঙে, একবারে তৈরি হয় না — প্রতিটি সৎ আপডেট জমা হয়।

```
সাপোর্ট টিকেটের ছক

১. কে / কোন প্রতিষ্ঠান
২. কোন পর্দা + URL
৩. কখন (ঘড়ির সময়)
৪. কী আশা করেছিলেন
৫. কী ঘটেছে
৬. স্ক্রিনশট
৭. জরুরি? (ডেটা বন্ধ / নিয়ম ভাঙা)

→ ২ ও ৩ নম্বর দিয়েই লগে খোঁজা শুরু
```

শিক্ষকের নোট: ক্লাসে দুটি উদাহরণ লিখুন — একটি ভালো টিকেট, একটি খারাপ ("সাইট কাজ করছে না") — পার্থক্য নিজেরাই বলবে।

এরপর

প্রথম সপ্তাহের পরের তালিকা

যা বাকি, তা লেখা আছে — লুকোনো নয়

কাজ	কেন এখন
uptime মনিটর + `/healthz`	নীরব ডাউনটাইম ধরার একমাত্র উপায়
লগ-অ্যালার্ট (৫০০ হলে ইমেইল)	ব্যবহারকারীর আগে জানুন
CI: প্রতিটি পুশে `test`	হাতে চালানো ভুলে যায়
রেট লিমিট (লগইনে)	এখন কোনো থ্রটল নেই
CSV আমদানি	অনবোর্ডিংয়ের সবচেয়ে ধীর ধাপ
ব্যাকআপ কনসোল	এখন শুধু `ops/` স্ক্রিপ্ট

BUILD_STATUS-এ লেখা বাকি কাজ

invitation workflow · platform console · backup & restore console · security attack suite

নিয়ম

প্রতিটি নতুন কাজের সঙ্গে একটি টেস্ট — নইলে তালিকা আরও লম্বা হবে, আস্থা বাড়বে না।

শিক্ষকের নোট: "সফটওয়্যার কখনো শেষ হয় না — শুধু দায়িত্ব বদলায়" এই বাক্য দিয়ে পর্ব ৭-এর দিকে নিয়ে যান।

অনুশীলন ও আলোচনা

২০ মিনিট — নিজের go-live রানবুক লিখুন

দলবদ্ধ কাজ, তারপর ভাগাভাগি

১. D-14 → D-0 ছক

নিজের প্রকল্পের জন্য তারিখ বসান — কোনটি আজই করা যায়?

২. রোলব্যাক ট্রিগার

তিনটি ট্রিগার নিজের ভাষায় লিখুন — সংখ্যা দিয়ে।

৩. ডেমো ডেটা তালিকা

কোন অ্যাকাউন্ট, কোন প্রতিষ্ঠান, কোন ফাইল — লিখে ফেলুন।

৪. সাপোর্ট ছক

৭ প্রশ্নের টিকেট ছকটি নিজের টুলে বসান (ইশিখন/ইমেইল/শীট)।

- D-0-তে কেন কিছু বদলাবেন না?
- TLS আগে না HSTS আগে?
- ডেমো ডেটা রয়ে গেলে কী হয়?
- "আমার ডেটা ভুল" — প্রথম কী দেখবেন?
- রোলব্যাকের আগে কী নেবেন?

শিক্ষকের নোট: চতুর্থ প্রশ্নের উত্তর: অডিট ট্রেইল — কার, কখন, before/after। এটি মুখস্থ করিয়ে দিন।

পর্ব ৬ সমাপ্তি

তিন ক্লাসে যা শেখা হলো

৫৬ স্লাইড · ৩ ক্লাস · ৯টি যাচাইযোগ্য প্রমাণ

ক্লাস	মূল ধারণা	প্রমাণ
২৮ · সার্ভারে তোলা	prod সেটিংস · চলক · Gunicorn · WhiteNoise	`check --deploy` → no issues · দুই `RuntimeError`
২৯ · চালু রাখা	লগ · অডিট · ব্যাকআপ/রিস্টোর	`billing_report` (MRR ৳6,500) · `AuditLog` ০ সারি
৩০ · Go-live	DNS · TLS · অনবোর্ডিং · ৪৮ ঘণ্টা	`sync_domains` · ৩টি ডোমেইন টেস্ট · ডেমো-মুক্তি

সৎ ফাঁক: `/healthz` নেই · uptime/Sentry নেই · CI নেই · রেট লিমিট নেই · ৫০০ পাতার "reference number" অপূর্ণ

পর্ব ৭-এ এই ফাঁকগুলোই **রক্ষণাবেক্ষণের দায়িত্ব** হয়ে ফিরবে — কোড বাড়লে ঝুঁকি বাড়ে।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন তিনটি ফাঁক বেছে নিয়ে লেখার জন্য — পরের পর্বের আলোচনা সেখান থেকেই শুরু হবে।

৩১

পর্ব ৭ — রক্ষণাবেক্ষণ

কোড বাড়ানো · রিফ্যাক্টর · ডেপেন্ডেন্সি আপডেট · ডেটা বৃদ্ধি · টিম ও হাতবদল

পর্ব ৭ · ক্লাস ৩১

কোড বড় হয়, দায়িত্বও

কোডের মানচিত্র · ঋণ কোথায় লেখা · নিরাপদ রিফ্যাক্টর · ডেপেন্ডেন্সি · ডেটা বৃদ্ধি

IMS · ক্লাস ৩১ · স্লাইড ১–১৭

শুরু

এখন কোডটি কত বড়?

আজ গোনা সংখ্যা — অনুমান নয়

অ্যাপ কোড ৫,৪৮০ লাইন

১১টি অ্যাপে, মাইগ্রেশন ও টেস্ট বাদ দিয়ে। সবচেয়ে বড়: `billing` (১,৩৩০)।

টেস্ট কোড ১,৩৫৩ লাইন

অনুপাত ১ : ০.২৫ — অ্যাপ কোডের এক-চতুর্থাংশ। বেশি নয়, কমও নয়।

মোট ৬,৮৩৩ লাইন

৩১ মডেল · ১২ মাইগ্রেশন · ১০৩ টেস্ট · ১০ middleware · ১৯ কমিট

সংখ্যা নিজে থেকে খারাপ নয় — কিন্তু **বড় ফাইল আর নকল যুক্তি** একদিনে তৈরি হয় না, ধীরে জমে। এই ক্লাসের কাজ সেই জমা চেনা।

শিক্ষকের নোট: ক্লাসের শুরুতেই একটি সত্যি কথা: "৬,৮৩৩ লাইন আপনার হাতে লেখা"। এরপর জিজ্ঞেস করুন — "এতে নতুন কেউ ঢুকলে কোথা থেকে শুরু করবে?"

মানচিত্র

১১টি অ্যাপ, ১১টি দায়িত্ব

কোড আর টেস্টের অনুপাতই বলে দেয় কোথায় ঝুঁকি

অ্যাপ	কোড	টেস্ট	অনুপাত	মন্তব্য
billing	১,৩৩০	৪৩৬	১:০.৩৩	সবচেয়ে বড় ও সবচেয়ে পরীক্ষিত
core	৬৫৮	২৪৯	১:০.৩৮	স্কোপিং, পারমিশন, সিড
academics	৫৮৯	৫২	১:০.০৯	**সবচেয়ে কম** — ফাঁক
assessments	৫৬৯	১৫৫	১:০.২৭	নম্বর ও পাবলিশ গেট
lessons	৫২৭	৭৬	১:০.১৪	হোমওয়ার্ক, সিলেবাস
workspaces	৩৯৫	১১৮	১:০.৩০	ব্র্যান্ডিং, ভূমিকা
accounts	৩৬২	১১৪	১:০.৩১	লগইন, রিসেট
students	৩৫৫	৬৮	১:০.১৯	ভর্তি ও স্থানান্তর
attendance	৩৪৩	৮৫	১:০.২৫	নীতি ও সংশোধন
reports · audit	৩৫২	০	—	**শূন্য টেস্ট** — ক্লাস ২৬-এ চিহ্নিত

শিক্ষকের নোট: ছকে দুইটি লাল দাগ দেখান — academics-এর কম অনুপাত আর reports/audit-এর শূন্য। "কোড বড় হওয়ার সঙ্গে সঙ্গে টেস্টও বাড়ে না — সেটাই ঋণ।"

বড় ফাইল

পাঁচটি ফাইল, পাঁচটি সংকেত

ফাইলের আকার নিজে দোষ নয় — কিন্তু সীমা দরকার

ফাইল	লাইন	সংকেত
`apps/billing/tests.py`	৪৩৬	টেস্ট বেশি = ভালো; তবে এক ফাইলে সব = খুঁজতে সময়
`apps/billing/views_platform.py`	৩১৭	এক ফাইলে বহু ভিউ — ভাগ করার প্রার্থী
`core/management/commands/seed_demo.py`	২৮১	ডেমো ডেটার বর্ণনা — লম্বা হওয়াই স্বাভাবিক
`apps/billing/models.py`	২৪৯	Plan + Subscription + Invoice + Payment — তিন মডেল এক ফাইলে
`apps/lessons/views.py`	২৪৮	৯ ভিউ — স্কোপিং নিয়ম একই থাকলে ভাগ করা সহজ

নিয়ম: **৩০০ লাইনের বেশি = থামুন ও ভাবুন**। কিন্তু ভাগ করবেন তখনই, যখন টেস্ট আগে থেকে আছে — নইলে ভাগ করা মানে ঝুঁকি নেওয়া।

শিক্ষকের নোট: জিজ্ঞেস করুন: "views_platform.py ভাগ করলে কী লাভ, কী ক্ষতি?" লাভ: পড়া সহজ; ক্ষতি: ইমপোর্ট আর URL নাম বদলাতে হবে।

ঋণের খাতা

ঝুঁকি কোথায় লেখা থাকে

IMS-এ কোডে একটিও `TODO` নেই

কোডে মন্তব্য-ঋণ

`# TODO: পরে ঠিক করব` — গোনা হয়েছে **০**। ভালো লক্ষণ, তবে মানে এই নয় ঋণ নেই।

যেখানে আসলে লেখা

`BUILD_STATUS.md` — "Remaining work (not faked, not skipped)" · `DEPLOYMENT.md` — "not yet deployed"

কেন এভাবে

একটি তালিকা সবার চোখে থাকে; ছড়িয়ে থাকা `TODO` কেউ পড়ে না।

ঝুঁকি

খাতা হালনাগাদ না হলে ঋণ আবার লুকিয়ে যায় — তাই রিলিজের সঙ্গে খাতা বদলাতে হবে।

```
## Remaining work
1. Invitation workflow
2. Platform console
4. Backup & restore console
5. Security attack suite

## Verified numbers
test            ->  90 tests, 90 OK
smoke_test.py   ->  55 pages x 4 roles
check --deploy  ->  no issues
```

শিক্ষকের নোট: কৌশলটি শেখান: ঋণ লিখুন **এক জায়গায়**, আর প্রতিটি সংখ্যার সঙ্গে প্রমাণের কমান্ড। এটাই ক্লাস ২৭-এর প্রমাণ-খাতার ধারাবাহিকতা।

সিদ্ধান্ত

কখন রিফ্যাক্টর করবেন, কখন নয়

"তিনবার নিয়ম" — শেখার সবচেয়ে কাজের নিয়ম

তিনবার নিয়ম

একই জিনিস প্রথমবার লিখলে — শুধু লিখুন।
দ্বিতীয়বার — মেনে নিন।
তৃতীয়বার — তখনই সাধারণ রূপ বের করুন।

করুন	করবেন না
তিনবার একই যুক্তি	ভবিষ্যতে "হতে পারে" বলে
বাগের পরে, টেস্ট লেখার পর	টেস্ট ছাড়া
একটি ছোট ধাপে	পুরো ফাইল একসাথে
কাজের অংশ হিসেবে	"শুধু রিফ্যাক্টরের" আলাদা সপ্তাহে

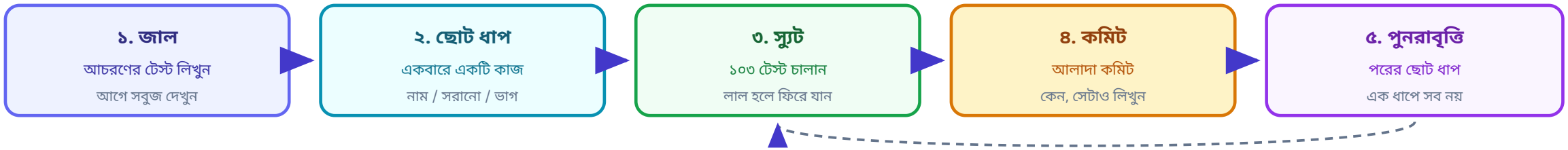
IMS-এ বাস্তব: `BootstrapFormMixin` আর `ImsFormMixin` — দুটি mixin, একই কাজ। তৃতীয়বার দরকার হলেই এক করা উচিত।

শিক্ষকের নোট: "Rule of three" মুখস্থ করান। শিক্ষার্থীরা প্রায়ই আগেভাগে abstraction বানায় — সেটিই সবচেয়ে বেশি ক্ষতি করে।

প্রোটোকল

নিরাপদ রিফ্যাক্টরের পাঁচ ধাপ

প্রতিটি ধাপের শেষে স্যুট সবুজ — নইলে পিছিয়ে যান



একমাত্র অনুমোদন: রিফ্যাক্টরে কোনো টেস্ট **বদলানো যাবে না**। টেস্ট বদলাতে হলে আচরণ বদলেছে — সেটি নতুন ফিচার, রিফ্যাক্টর নয়।

বাস্তব সময়: ১০০ লাইনের রিফ্যাক্টর ২০ মিনিট; ১,০০০ লাইনের "রিফ্যাক্টর" দুই দিন — আর সেটি প্রায়ই ব্যর্থ হয়।

শিক্ষকের নোট: বোর্ডে ৫ ধাপ লিখে রাখুন; ক্লাসের বাকি অংশে প্রতিটি উদাহরণে এই ধাপগুলোই দেখাবেন।

নকল

দুই mixin, এক কাজ

কীভাবে এক করবেন — টেস্ট আগে

```
# জাল: আচরণ লিখে ফেলুন (কোড বদলানোর আগে)
def test_every_form_field_gets_design_classes(self):
    f = StudentForm(workspace=self.ws).fields
    self.assertIn("form-control", f["full_name"].widget.attrs["class"])
    self.assertIn("form-select", f["class_group"].widget.attrs["class"])

# ধাপ ১: সবুজ দেখুন (দুই কপি এখনো আছে)
# ধাপ ২: একটিকে apps/core/forms.py-তে নিন
#   class DesignClassesMixin:  ...  (এক জায়গায়)
# ধাপ ৩: academics আর billing দুটোই
#         from apps.core.forms import DesignClassesMixin
# ধাপ ৪: পুরো স্যুট — ১০৩ টেস্ট সবুজ = আচরণ অপরিবর্তিত
```

কেন দুই কপি বিপজ্জনক

এক কপিতে বাগ ঠিক করলে অন্য কপিতে থাকে — "কিছু ফর্মে ত্রুটি দেখায় না" ধরনের অদ্ভুত বাগ।

ধাপ ৩-এর ফাঁদ

একসাথে দুই ফাইল বদলালে কোনটি ভাঙল বোঝা যায় না — ফাইল ধরে ধরে এগোন, প্রতি ধাপে স্যুট।

নতুন জায়গা

`apps/core/forms.py` — শেয়ারড ফর্ম-আচরণের ঘর; অ্যাপগুলো এখান থেকে নেয়।

শিক্ষকের নোট: জোর দিন: নকল সরানোর আগে **টেস্ট** — নইলে "সব ঠিক আছে" বলার প্রমাণ থাকে না।

নির্ভরতা

৯টি প্যাকেজ, সবগুলোর নির্দিষ্ট সংস্করণ

ওপেন রেঞ্জ নয় — পিন করা

```
# requirements.txt (dev + app)
Django==6.1.2
psycopg[binary]==3.3.6
python-dotenv==1.2.4
Pillow==12.3.0
openpyxl==3.1.5      # এক্সেল এক্সপোর্ট
reportlab==5.1.0     # মার্কশিট PDF

# requirements-prod.txt
-r requirements.txt
gunicorn==23.0.0
whitenoise==6.11.0
dj-database-url==3.0.1
```

কেন `==`

আজ যা পরীক্ষা করেছেন, কাল সার্ভারে ঠিক সেটাই বসবে — নীরব আপডেট নেই।

কীভাবে আপডেট

একটি প্যাকেজ → শাখা → `pip install -U` → টেস্ট → স্টেজিং → ডিপ্লয়।

নিরাপত্তা আপডেট

Django-র নিরাপত্তা রিলিজ বাদ দেওয়া সবচেয়ে ব্যয়বহুল অবহেলা — মাসে একবার দেখার অভ্যাস।

ফাঁক

অটোমেটেড নির্ভরতা-স্ক্যান (dependabot/audit) নেই — হাতে দেখতে হয়।

শিক্ষকের নোট: প্রশ্ন: "উপকারী প্যাকেজ যোগ করার আগে কী ভাববেন?" উত্তর: রক্ষণাবেক্ষণ কে করে, লাইসেন্স কী, আর সরানোর খরচ কত।

আপগ্রেড

ফ্রেমওয়ার্ক আপডেট — ভয় নয়, ক্রম

Django 6.1.2-এ দাঁড়িয়ে আছে; পরের লাফের প্রস্তুতি

#	ধাপ	কেন
১	রিলিজ নোট পড়ুন (deprecations)	কী সরছে, তা আগেই জানা
২	ওয়ার্নিং চালু করে টেস্ট	`python -W error::DeprecationWarning manage.py test`
৩	এক মাইনর সংস্করণ করে এগোন	৬.১ → ৬.২, একসাথে ৬.১ → ৭.০ নয়
৪	পূর্ণ স্যুট + `smoke_test` + `check --deploy`	তিনটি ভিন্ন দৃষ্টি
৫	প্রথমে স্টেজিং/রিহার্সাল, পরে আসল	`deploy_check.sh` এখানেই কাজে লাগে

IMS-এর সুবিধা: `override_settings` আর `django.conf.settings` ব্যবহার — হার্ডকোড কম, তাই আপগ্রেডে কম ভাঙে।

ফাঁদ: আপগ্রেড আর নতুন ফিচার একই রিলিজে নয় — ভাঙলে কারণ দুটির কোনটি, বোঝা যায় না।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন: "আপগ্রেড একটি কাজ, ফিচার আরেকটি কাজ" — একসাথে করলেই বিভ্রান্তি।

বৃদ্ধি

ডেটা বাড়লে কী বাড়ে

ডেমোর সংখ্যা বনাম এক বছরের বাস্তবতা

টেবিল	ডেমোতে আজ	৮০০ ছাত্র · ১ বছর
ছাত্র প্রোফাইল	১০	৮০০
উপস্থিতি সারি	২০০	৮০০ × ২০০ দিন ≈ **১,৬০,০০০**
নম্বর	৩০	৮০০ × ৮ বিষয় × ৩ পরীক্ষা ≈ **১৯,২০০**
হোমওয়ার্ক জমা	১৫	৮০০ × ৩০ ≈ ২৪,০০০
অডিট সারি	০	প্রতিদিন ১০০ কর্ম ≈ **৩৬,৫০০**

মূল কথা: ২ লাখ সারিতে ইনডেক্স ছাড়া তালিকা খুললে সেকেন্ডের হিসাব বদলে যায়।

IMS-এ আছে: ১৪টি `db_index` / `indexes` ঘোষণা আর ৫০টি unique সীমা — তবে সব কোয়েরি মাপা হয়নি।

শিক্ষকের নোট: বোর্ডে হিসাবটি করুন: ৮০০ × ২০০ = ১,৬০,০০০। শিক্ষার্থীরা নিজে গুণ করলে সংখ্যাটি মনে থাকে।

ধীর কোয়েরি

N+1 — সবচেয়ে সাধারণ ধীরতা

লুপের ভেতরে কোয়েরি = বিপদ

```
# খারাপ: প্রতি সারিতে একটি কোয়েরি (N+1)
for st in students:            # ৮০০ কোয়েরি
    print(st.class_group.name) # +৮০০

# ভালো: এক কোয়েরিতে সব
students = StudentProfile.objects.filter(
    workspace=ws).select_related("class_group")   # ১ কোয়েরি

# অনেক-এর-অনেক হলে
    .prefetch_related("enrollments__class_group")

# মাপার উপায় (dev-এ)
from django.test.utils import CaptureQueriesContext
from django.db import connection
with CaptureQueriesContext(connection) as ctx:
    list(students)
print(len(ctx.captured_queries))     # সংখ্যাটাই প্রমাণ
```

কেন চোখে ধরা পড়ে না

ডেমোতে ১০ ছাত্র — ৫ মিলিসেকেন্ড। বাস্তবে ৮০০ — ৪ সেকেন্ড।

মাপার অভ্যাস

নতুন তালিকা-ভিউ লিখলে একবার কোয়েরি গুনে নিন; সংখ্যা জানলে উন্নতি বোঝা যায়।

ক্যাশ আগে নয়

ভুল কোয়েরি ক্যাশ করলে ভুল উত্তর দ্রুত আসে — আগে কোয়েরি ঠিক করুন।

শিক্ষকের নোট: `CaptureQueriesContext` ক্লাসে চালিয়ে দেখান — সংখ্যা মুদ্রিত হলে "অনুভূতি" বদলে যায়।

মাইগ্রেশন

১২টি মাইগ্রেশন — কখন জমানো থামাবেন

ইতিহাস মুছবেন না, তবে গুছিয়ে রাখুন

১১ অ্যাপে ১২টি মাইগ্রেশন

প্রতিটি অ্যাপের প্রথম মাইগ্রেশনেই বেশিরভাগ মডেল — ছোট, পড়ার মতো ইতিহাস।

squash কখন

একই অ্যাপে ২০+ মাইগ্রেশন হলে, বা পুরোনো রিলিজ আর দরকার না হলে — `squashmigrations`।

সবচেয়ে বিপজ্জনক

ইতিমধ্যে ডিপ্লয় করা মাইগ্রেশন সম্পাদনা করা — অন্য পরিবেশে ইতিহাস অমিল হয়ে যায়।

যাচাই

`makemigrations --check --dry-run` — মডেল আর মাইগ্রেশন অমিল হলে ধরা পড়ে।

```
# নিয়মিত তিনটি কমান্ড
python manage.py makemigrations --check --dry-run
python manage.py showmigrations
python manage.py migrate --plan

# শাখা মিলিয়ে নেওয়ার পর
python manage.py migrate --merge
```

শিক্ষকের নোট: বলুন: "মাইগ্রেশন ফাইল হলো ডেটাবেসের ইতিহাস — ইতিহাস নতুন করে লেখা যায় না, শুধু সামনে এগোয়।"

হাতবদল

নতুন ডেভেলপারের প্রথম দিন

দস্তাবেজ যদি কাজ করে, তবে কোড পড়া কম লাগে

#	কাজ
১	`START_HERE.md` — দুই পথ: SQLite দ্রুত, PostgreSQL আসল
২	`bash run.sh` → চারটি লগইন দিয়ে ঢুকে দেখুন
৩	`BUILD_STATUS.md` — কী তৈরি, কী বাকি, কোন প্রমাণে
৪	`python manage.py test` → ১০৩ সবুজ দেখুন
৫	একটি ছোট বাগ ঠিক করে টেস্টসহ কমিট

প্রতিটি ধাপের একটি **যাচাইযোগ্য ফল** আছে — "পড়ে নিন" নয়, "চালিয়ে দেখুন"।

দস্তাবেজ যা আছে

START_HERE · README · BUILD_STATUS · DEPLOYMENT · LAUNCH_PLAYBOOK · USER_MANUAL (বাংলা) · ৫৩ স্ক্রিনশট

যা নেই

আর্কিটেকচার সিদ্ধান্তের আলাদা নথি (ADR) — কেন এভাবে, তা কোড-মন্তব্যেই ছড়িয়ে আছে।

শিক্ষকের নোট: ছাত্রদের বলুন: "আপনার প্রজেক্টে নতুন কেউ এলে প্রথম দিনের পাঁচ ধাপ লিখে দিন" — এটাই হাতবদলের প্রস্তুতি।

অনুশীলন ও আলোচনা

২০ মিনিট — নিজের কোডের ঋণ মাপুন

চারটি কাজ, সবই মাপা যায়

১. বড় ফাইল খুঁজুন

```
find . -name "*.py" | xargs wc -l | sort -rn | head
```

২. কোয়েরি গুনুন

একটি তালিকা-ভিউতে `CaptureQueriesContext` দিয়ে সংখ্যা লিখে রাখুন।

৩. নকল খুঁজুন

একই ৫ লাইনের বেশি দুই জায়গায়? তালিকায় লিখুন।

৪. ঋণ-খাতা লিখুন

তিনটি কাজ, প্রতিটির সঙ্গে কারণ ও ঝুঁকি।

- বড় ফাইল ভাঙলে কী লাভ?
- N+1 ধরা পড়ে কীভাবে?
- Django আপগ্রেড কেন ধাপে ধাপে?
- প্যাকেজ পিন না করলে কী হয়?
- ৮০০ ছাত্রে কোন টেবিল সবচেয়ে বড়?

শিক্ষকের নোট: শেষ প্রশ্নের উত্তর: উপস্থিতি (attendance) — ১,৬০,০০০ সারি। এটাই প্রথম জায়গা যেখানে ইনডেক্স দরকার হবে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

রক্ষণাবেক্ষণের প্রথম ক্লাসের সারাংশ

নীতি	প্রমাণ
কোড বাড়লে দায়িত্ব বাড়ে	৫,৪৮০ লাইন · ৬,৮৩৩ মোট
টেস্টের অনুপাতই ঝুঁকির মানচিত্র	academics ১:০.০৯ · reports/audit ০
ঋণ এক জায়গায় লিখুন	`BUILD_STATUS.md` · ০ `TODO`
তিনবার নিয়ম	দুই mixin — তৃতীয়বারে এক করুন
রিফ্যাক্টরের অনুমোদন = সবুজ স্যুট	১০৩ টেস্ট অপরিবর্তিত
বৃদ্ধি মাপুন, অনুমান নয়	১,৬০,০০০ উপস্থিতি সারি/বছর

পরের ক্লাসে: এই কোড দল বদলালে, দায়িত্ব হাতবদল হলে, আর এক বছর পরের IMS কেমন হবে।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন নিজের প্রজেক্টের জন্য এই ছয়টি সারির একটি নিজের ভাষায় লিখতে।

০২

এক বছর পরের IMS

হাতবদল ও দল · সিদ্ধান্তের নথি · রোডম্যাপ · ফিচার বনাম ঋণ · কখন থামবেন

পর্ব ৭ · ক্লাস ৩২

এক বছর পরের IMS

রোডম্যাপ · সিদ্ধান্তের নথি · দল ও রিলিজ · ফিচার বনাম ঋণ · কখন নতুন করে লিখবেন না

IMS · ক্লাস ৩২ · স্লাইড ১৮–৩৪

শুরু

৩৬৫ দিনে চারটি জিনিস বদলায়

আর প্রতিটির জন্য আলাদা প্রস্তুতি দরকার

কী বদলায়	কেমন দেখায়	প্রস্তুতি
ব্যবহারকারী	৪ জন → ৪০ জন; কেউ প্রশিক্ষণ নিয়ে আসে, কেউ নয়	ব্যবহারবিধি ও সহায়তা (USER_MANUAL)
ডেটা	২০০ উপস্থিতি → ১,৬০,০০০ সারি	ইনডেক্স, ব্যাকআপ, আর্কাইভ
কোড	৫,৪৮০ → সম্ভবত ৮,০০০ লাইন	টেস্টের অনুপাত ধরে রাখা
মানুষ	আপনি একা → একজন সহকারী, বা কেউ নেই	দস্তাবেজ ও সিদ্ধান্তের নথি

চারটির মধ্যে **শুধু কোড** নিজে থেকে ঠিক থাকে না বাকি তিনটি পরিকল্পনা ছাড়া এলোমেলো হয় — আর সেটাই এক বছর পরের আসল ঝুঁকি

শিক্ষকের নোট: জিজ্ঞেস করুন: "এক বছর পর আপনার প্রজেক্টে সবচেয়ে বড় সমস্যা কী হবে?" উত্তর প্রায় সবসময় ডেটা বা মানুষ, কোড নয়

রোডম্যাপ

১২ মাসের তিনটি ধাপ

প্রতিটি ধাপের সঙ্গে একটি যাচাইযোগ্য লক্ষ্য

সময়	কাজ	লক্ষ্য / প্রমাণ
মাস ১–৩	মূল ব্যবহার শক্ত করা: মার্কস এন্ট্রি, উপস্থিতি, রিপোর্ট কার্ড	প্রতিদিন ব্যবহার, ৫০০ = ০
মাস ১–৩	ইনভাইট ও CSV আমদানি (অনবোর্ডিংয়ের ধীর ধাপ)	একটি স্কুল ২০ মিনিটে ওঠে
মাস ৪–৬	মনিটরিং, CI, রেট লিমিট, ব্যাকআপ কনসোল	অ্যালার্ট আসে, রিস্টোর ড্রিল পাস
মাস ৪–৬	academics-এর টেস্ট (১:০.০৯ → ১:০.৩)	টেস্ট সংখ্যা বাড়ে, বাগ কমে
মাস ৭–১২	নতুন ফিচার (শিক্ষক-উপস্থিতি, এসএমএস, পেমেন্ট গেটওয়ে)	গ্রাহক-অনুরোধের তালিকা থেকে
সারা বছর	প্রতি রিলিজে দস্তাবেজ হালনাগাদ	BUILD_STATUS সংখ্যা সত্যি

নিয়ম: রোডম্যাপে **তারিখের চেয়ে ক্রম বেশি গুরুত্বপূর্ণ** — প্রথম তিন মাসের কাজ শেষ না করে চতুর্থ মাসের ফিচারে যাবেন না

শিক্ষকের নোট: BUILD_STATUS-এর "Remaining work" তালিকাটি এই রোডম্যাপের বীজ — ক্লাসে দুটিকে পাশাপাশি রেখে দেখান

ADR

সিদ্ধান্তের নথি — কেন এভাবে

কোড বলে "কী", ADR বলে "কেন"

```
# ADR-004: সাবস্ক্রিপশন গেট middleware-এ

## প্রসঙ্গ
মেয়াদ শেষ হলে কিছুই লেখা যাবে না, কিন্তু পড়া
আর এক্সপোর্ট চালু থাকবে ২০+ ভিউ

## সিদ্ধান্ত
প্রতি ভিউয়ে নয় — এক middleware-এ
SAFE_METHODS বাদ, ALLOW_PREFIXES ছাড়

## বিকল্প যা বাদ গেল
ভিউ ডেকোরেটর: ২০ জায়গায় একই কোড, একটায়
ভুল হলে ফাঁক

## পরিণতি
নতুন ভিউ স্বয়ংক্রিয়ভাবে সুরক্ষিত; কিন্তু
বুক হলে ডিবাগ করা কঠিনতর
```

IMS-এর চারটি আসল সিদ্ধান্ত

modular monolith (মাইক্রোসার্ভিস নয়) · কাস্টম User (email লগইন) · বিলিং অ্যাপের ভেতরে · AI/বাহ্যিক সেবা নেই

গঠন

প্রসঙ্গ → সিদ্ধান্ত → বাদ পড়া বিকল্প → পরিণতি চারটি শিরোনাম, এক পাতা

এখনো নেই

IMS-এ আলাদা `docs/adr/` ফোল্ডার নেই — চার নম্বর তথ্যটি কোড-মন্তব্য ও ক্লাসে ছড়িয়ে আছে

শিক্ষকের নোট: বলুন: "ছয় মাস পর কেউ জিজ্ঞেস করবে 'কেন middleware, ভিউ ডেকোরেটর নয়?' — ADR সেই উত্তর" ছাত্রদের একটি ADR লিখিয়ে নিন

কোথায় কী লেখে

চার জায়গা, চার দায়িত্ব

একই তথ্য দুই জায়গায় লিখলে একদিন অমিল হবে

জায়গা	কী থাকে	কে পড়ে
কমিট বার্তা	এই পরিবর্তনটি কী ও কেন	কোড-ইতিহাস খুঁড়তে যাওয়া কেউ
ADR	বড় সিদ্ধান্ত ও বাদ পড়া বিকল্প	নতুন টিম-সদস্য
`BUILD_STATUS.md`	কী তৈরি, কী বাকি, কোন প্রমাণে	গ্রাহক, আপনি নিজে, নতুন ডেভেলপার
ইনলাইন মন্তব্য	অদ্ভুত কোডের **কারণ** (কী নয়)	পাশের লাইন পড়তে যাওয়া কেউ

ভালো মন্তব্য: "সেশন ওয়ার্কস্পেস আর URL আলাদা হতে পারে — তাই slug থেকে পড়ি" (কেন)

খারাপ মন্তব্য: "ওয়ার্কস্পেস নিন" — কোডেই তো দেখা যাচ্ছে মন্তব্য যা বলে তা নয়, যা বলে না তা লিখুন

শিক্ষকের নোট: এই স্লাইডটি প্রায়ই শিক্ষার্থীদের ধারণা বদলায়: "কোড নিজেই দস্তাবেজ" — আধা-সত্য; কোড "কী" বলে, "কেন" নয়

দল

একজন থেকে তিনজন

ভূমিকা আলাদা করুন, নিয়ম লিখিত রাখুন

ভূমিকা	দায়িত্ব
রক্ষক (maintainer)	মার্জ, রিলিজ, মাইগ্রেশন — একজনের হাতে
নির্মাতা	ফিচার + টেস্ট একসাথে
সহায়তা (support)	টিকেট, ব্যবহারকারীর প্রশ্ন, ডেটা সংশোধন
গ্রাহক-সফলতা	অনবোর্ডিং, বিলিং, প্রশিক্ষণ

তিনটি অলঙ্ঘনীয় নিয়ম

১. টেস্ট ছাড়া মার্জ নয়
২. মাইগ্রেশন রিভিউ ছাড়া ডিপ্লয় নয়
৩. রিলিজের আগে ব্যাকআপ

বাস্তব ঝুঁকি

সব সিদ্ধান্ত একজনের মাথায় = **bus factor ১**; ছুটি বা অসুস্থতাতেই প্রকল্প থেমে যায়

দল ছোট হলে নিয়ম আরও লিখিত হতে হয় — মুখে বলা নিয়ম কেবল উপস্থিত থাকা মানুষদের কাজে লাগে

শিক্ষকের নোট: ক্লাসে "bus factor" ধারণাটি ব্যাখ্যা করুন: "আপনি অসুস্থ হলে কত দিন প্রকল্প চলবে?"

রিলিজ

সংস্করণ, ট্যাগ ও পরিবর্তন-তালিকা

যাতে "কোন সংস্করণে কী ছিল" প্রশ্নের উত্তর থাকে

```
# সংস্করণ: MAJOR.MINOR.PATCH
1.4.0   নতুন ফিচার (ইনভাইট)
1.4.1   বাগ ঠিক
2.0.0   ভাঙা পরিবর্তন (URL বদল)

# ট্যাগ ও নোট
git tag -a v1.4.0 -m "Invitations + CSV import"
git push --tags

CHANGELOG.md
## 1.4.0 — 2026-12-01
+ invitation workflow (F-10..12)
* academics tests 52 -> 90
! migration: students 0004
```

কেন MAJOR লাগে

গ্রাহক বলবেন "আপনাদের ২.০-তে URL বদলেছে" — তখন উত্তর প্রস্তুত

মাইগ্রেশন উল্লেখ করুন

রিলিজ নোটে মাইগ্রেশন না লিখলে কেউ ব্যাকআপ নেয় না

আজকের বাস্তব

IMS-এ ১৯টি কমিট আছে, কিন্তু কোনো git tag বা CHANGELOG নেই — যোগ করা প্রথম কাজগুলোর একটি

শিক্ষকের নোট: শিক্ষার্থীদের বলুন প্রতিটি রিলিজে তিনটি জিনিস লিখতে: তারিখ, কী বদলাল, কোন মাইগ্রেশন

বাজেট

ফিচার বনাম ঋণ — ৭০ / ২০ / ১০

প্রতি রিলিজের কাজ ভাগ করে নিন

৭০% — নতুন ফিচার (গ্রাহক যা চায়) | **২০% ঋণ** | **১০%**

ফিচার — ব্যবহারকারীর কাজ সহজ করে

ঋণ — টেস্ট, রিফ্যাক্টর, ইনডেক্স, দস্তাবেজ

১০% — অনিশ্চিত কাজ / পরীক্ষা (নতুন ফিচার বা ঝুঁকি)

ঋণ শূন্য হলে ঋণ ৩০% করুন

কখনো ০% নয়

IMS-এর ঋণ-তালিকা বাস্তবে: academics টেস্ট · reports/audit টেস্ট · CI · healthz · রেট লিমিট · ৫০০ পাতার reference id

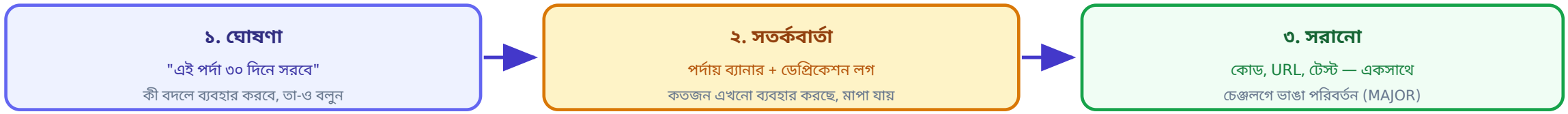
ফাঁদ: ঋণ ০% রাখলে কয়েক মাস পর ফিচারের গতি অর্ধেক — প্রতিটি ফিচার আগের ফিচারের কাঁটায় আটকে যায়

শিক্ষকের নোট: "ঋণ শোধ করা মানে নতুন কাজ না করা" — এই ভুল ধারণাটি ভাঙুন; ২০% নিয়মিত রাখলে বড় সংকট আসে না

ডেপ্রিকেশন

ফিচার বন্ধ করা — তিন ধাপ

হঠাৎ বন্ধ করলেই ব্যবহারকারী হারাবেন



উদাহরণ যেভাবে হবে: ধরা যাক ম্যানুয়াল bKash এন্ট্রি বদলে গেটওয়ে আসছে — পুরোনো পর্দা ৩০ দিন ব্যানারসহ থাকবে, তারপর সরবে ডেটা (`Invoice.reference`) থাকবে

কখনো করবেন না: ডেটা সরানো পর্দা সরানো যায়, ইতিহাস নয় — অডিট আর ইনভয়েস চিরকাল থাকা দরকার

শিক্ষকের নোট: "কোড মুছে ফেলা সহজ, ব্যবহারকারীর অভ্যাস মুছে ফেলা কঠিন" — তিন ধাপের কারণ এটাই

গোপনীয়তা

কোন ডেটা কত দিন রাখবেন

অডিট, ছাত্র, ব্যাকআপ — তিনটি আলাদা নিয়ম

ডেটা	কত দিন	কারণ
নম্বর, উপস্থিতি	যত দিন প্রতিষ্ঠান গ্রাহক	রিপোর্ট কার্ড ও বছরের তুলনা দরকার
`AuditLog`	অন্তত ১ বছর	"কে বদলাল" প্রশ্ন বছরের শেষেও আসে
ব্যাকআপ	১৪ দিন (বর্তমানে)	ভুল ধরা ও ফেরানোর জন্য যথেষ্ট
লগ	আয়োজক যত রাখে	ডিবাগিংয়ের জন্য; ব্যক্তিগত তথ্য নয়
মুছে ফেলা প্রতিষ্ঠান	প্রথমে ব্যাকআপ, পরে মুছে ফেলা	UI-তে ডিলিট নেই — ইচ্ছাকৃত

অনুরোধ এলে: "আমার ডেটা দিন" → রিপোর্ট/এক্সপোর্ট (CSV, XLSX, PDF) আছে; "মুছুন" → ব্যাকআপ নিয়ে, মালিকের অনুমোদনে

যা লগে রাখবেন না: পাসওয়ার্ড, রিসেট টোকেন, পুরো ছাত্র-রেকর্ড — `AuditLog`-এর নিয়মেই লেখা

শিক্ষকের নোট: বাংলাদেশে নাগরিক তথ্য সুরক্ষা নিয়ে আলোচনা চলছে; নীতি এখনই লিখে রাখলে ভবিষ্যতে সুবিধা — "কম রাখুন, দরকারে রাখুন"

খরচ

এক বছরে খরচ কত

৮০০ ছাত্র, একটি প্রতিষ্ঠান — মোটামুটি হিসাব

খাত	মাসিক (আনুমানিক)	নোট
অ্যাপ হোস্টিং (ছোট ইনস্ট্যান্স)	মার্কিন $৭–২৫	Render/Railway/Fly ফ্রি টিয়ারও চলে
PostgreSQL (ম্যানেজড)	$০–১৫	ফ্রি টিয়ারে সীমা; ব্যাকআপ আলাদা
ডোমেইন	বছরে ~$১২	কাস্টম ডোমেইন ক্লায়েন্টের
ইমেইল (৩০০০/মাস)	$০–১০	Brevo ফ্রি টিয়ার যথেষ্ট শুরুতে
স্টোরেজ (`media/`)	কয়েক ডলার	লোগো ছোট, পড়ার উপকরণ বড় হতে পারে

আয়: স্টার্টার ৳১,৫০০/মাস — এক প্রতিষ্ঠানই হোস্টিংয়ের কয়েকগুণ কভার করে

লুকানো খরচ: আপনার সময় সাপোর্ট, ব্যাকআপ যাচাই, আপডেট — এগুলোই আসল মাসিক ব্যয়

শিক্ষকের নোট: সংখ্যাগুলো আজকের বাজারদর নয়, মাত্রা বোঝানোর জন্য — শিক্ষার্থীদের নিজে যাচাই করতে বলুন

ফাঁদ

"নতুন করে লিখে ফেলি" — প্রায় সবসময় ভুল

পুরোনো কোডে যা আছে, তা দেখা যায় না — কিন্তু থাকে

পুরোনো কোডে লুকানো আছে	নতুন কোডে
এজ-কেস: অনুপস্থিত ছাত্র, মেয়াদোত্তীর্ণ ট্রায়াল	প্রথম থেকে শিখতে হবে
স্কোপিংয়ের সূক্ষ্ম নিয়ম	আবার বাগ হবে
১০৩ টেস্টের জ্ঞান	শূন্য থেকে শুরু
গ্রাহকের অভ্যস্ত পর্দা	আবার প্রশিক্ষণ
মাইগ্রেশন-ইতিহাস	ডেটা সরানোর ঝুঁকি

বিকল্প: ধীরে বদল (strangler)

একটি অংশ নতুন করে লিখুন, পুরোনোটা পাশে রেখে — ট্রাফিক ধীরে সরান, তারপর পুরোনোটা সরান

IMS-এ কোথায় প্রযোজ্য

রিপোর্ট বা রফতানি ধীর হলে সেটিকে আলাদা সার্ভিস করা যায় — পুরো সিস্টেম নয়

প্রথম প্রশ্ন

"কোন সমস্যার সমাধান করছি?" উত্তর না থাকলে রিওয়াইট নয়, রিফ্যাক্টর করুন

শিক্ষকের নোট: জোর দিন: "রিওয়াইট মানে পুরোনো জ্ঞান ফেলে নতুন করে শেখা" — াসে এটি সবচেয়ে দরকারি সতর্কতা

প্রযুক্তি

ফ্রেমওয়ার্ক বা ভাষা বদলাবেন কখন

প্রায় কখনোই কারিগরি কারণে নয়

যুক্তিসঙ্গত কারণ

ফ্রেমওয়ার্ক আর রক্ষণাবেক্ষণ পাচ্ছে না (নিরাপত্তা আপডেট নেই) · স্কেলিংয়ে প্রকৃত দেয়াল · পুরো দলের দক্ষতা অন্য স্ট্যাকে

অজুহাত

"নতুন প্রযুক্তি বেশি আধুনিক" · "অন্যরা করছে" · "আমার পছন্দের নয়" — এগুলো দিয়ে শুরু হলে বছর শেষে কিছুই তৈরি হয় না

লক-ইন হিসাব

Django, PostgreSQL, WhiteNoise — সবগুলো ওপেন ও প্রতিস্থাপনযোগ্য সবচেয়ে বড় লক-ইন আপনার নিজের কোড, ফ্রেমওয়ার্ক নয়

IMS-এ Django 6.1.2 · Python 3.13 · PostgreSQL — দশ বছরের বেশি রক্ষণাবেক্ষণের ইতিহাস আছে এমন স্ট্যাক; নিরাপত্তা আপডেটই আসল কারণ

শিক্ষকের নোট: প্রশ্ন: "নতুন ফ্রেমওয়ার্কে যেতে কত মাস লাগবে, আর কী লাভ?" সংখ্যা দুইটা পাশাপাশি লিখলেই উত্তর স্পষ্ট হয়

বীমা

আপনি ছাড়া প্রজেক্ট চলবে?

দস্তাবেজ = bus factor-এর একমাত্র প্রতিকার

৫০০+ লাইনের দস্তাবেজ

START_HERE ২১১ · USER_MANUAL ২০৩ · BUILD_STATUS ১৭০ · LAUNCH_PLAYBOOK ১৩৬ · DEPLOYMENT ১২২ · README ৫৮

দস্তাবেজও কোড

`docs/gen_feature_catalogue.py` · `gen_proof_pdf.py` · `gen_roadmap.py` · `build_status.py` · `shoot.py` (৫৩ স্ক্রিনশট)

কেন স্ক্রিপ্ট

হাতে লেখা দস্তাবেজ পুরোনো হয়; স্ক্রিপ্ট চালালেই ছবি ও সংখ্যা নতুন — BUILD_STATUS-এর "৯০ টেস্ট" এভাবেই বাসি হয়েছে

তবু ফাঁক

স্ক্রিপ্ট চালানোর কথা কেউ মনে রাখে না — রিলিজ চেকলিস্টে "দস্তাবেজ পুনরুত্পাদন" ধাপ যোগ করুন

```
$ python docs/build_status.py
$ python docs/gen_feature_catalogue.py
$ python docs/gen_proof_pdf.py
$ python docs/gen_roadmap.py
$ python docs/shoot.py     # ৫৩ স্ক্রিনশট

# এক কমান্ডে সব (এখনো নেই — বানাতে হবে)
$ bash docs/rebuild_docs.sh
```

শিক্ষকের নোট: শেষ লাইনটি সত্যি নয় — এমন `rebuild_docs.sh` ফাইল এখনো নেই; শিক্ষার্থীদের বলুন সেটি বানানোই প্রথম কাজ (সৎ উদাহরণ: দস্তাবেজও দাবি করে, যাচাই করতে হয়)

অনুশীলন ও আলোচনা

২০ মিনিট — এক বছরের পরিকল্পনা লিখুন

চারটি ছোট ফলাফল

১. একটি ADR
নিজের প্রজেক্টের একটি সিদ্ধান্ত — প্রসঙ্গ, সিদ্ধান্ত, বাদ পড়া বিকল্প, পরিণতি

২. ৭০/২০/১০ বাজেট
পরের তিন মাসের কাজ তিন ভাগে ভাগ করুন

৩. ডেটা-নীতি
কোন টেবিল কত দিন রাখবেন — পাঁচটি সারি লিখুন

৪. bus factor পরীক্ষা
একজন সহপাঠীকে দিন — সে কি আপনার প্রজেক্ট চালু করতে পারল?

- কেন ADR আলাদা নথি?
- ঋণ ০% রাখলে কী হয়?
- রিওয়াইটের আগে প্রথম প্রশ্ন?
- দস্তাবেজ পুরোনো হলে কীভাবে বুঝবেন?
- কোন ডেটা কখনো মুছবেন না?

শিক্ষকের নোট: চতুর্থ কাজটি সত্যিই করান — এক সহপাঠীকে ২০ মিনিট দিন; যেখানে আটকে যায়, সেটাই দস্তাবেজের ফাঁক

পর্ব ৭ সমাপ্তি

দুই ক্লাসে যা শেখা হলো

৩৪ স্লাইড · রক্ষণাবেক্ষণের পুরো ছবি

ক্লাস	মূল ধারণা	প্রমাণ
৩১ · কোড বড় হয়	মানচিত্র · ঋণ · রিফ্যাক্টর · নির্ভরতা · বৃদ্ধি	৬,৮৩৩ লাইন · ১,৬০,০০০ উপস্থিতি সারি
৩২ · এক বছর পর	ADR · রিলিজ · ফিচার বনাম ঋণ · রিওয়াইট নয়	১৯ কমিট · ৯০০ লাইন দস্তাবেজ · ৫৩ স্ক্রিনশট

মনে রাখুন: কোড বাড়লে **নিয়ম** বাড়াতে হয়, আর নিয়ম লিখে রাখতে হয় — নইলে তা কেবল আপনার মাথায় থাকে

সৎ ফাঁক: ADR ফোল্ডার নেই · git tag/CHANGELOG নেই · দস্তাবেজ পুনরুত্পাদনের এক-কমান্ড স্ক্রিপ্ট নেই

পরের পর্বে (৮): দস্তাবেজ — এই ফাঁকগুলো পূরণ করার ক্লাস ব্যবহারকারীর জন্য লেখা, ডেভেলপারের জন্য লেখা, আর প্রমাণের দস্তাবেজ

শিক্ষকের নোট: পর্ব ৮-এর সূচনা এই ফাঁকগুলো থেকেই — শিক্ষার্থীরা যেন দেখে দস্তাবেজ কেবল সাজসজ্জা নয়, প্রকৌশলের অংশ

৩৩

পর্ব ৮ — দস্তাবেজ

ব্যবহারবিধি · ডেভেলপার গাইড · প্রমাণের দস্তাবেজ · স্ক্রিনশট · বাংলায় লেখা

পর্ব ৮ · ক্লাস ৩৩

দস্তাবেজ — কে পড়বে, কী কাজে

ব্যবহারবিধি · নতুনদের পথ · প্রমাণের দস্তাবেজ · স্ক্রিনশট · বাংলায় লেখা

IMS · ক্লাস ৩৩ · স্লাইড ১–১৫

শুরু

আপনার README এখনো ভুল বলছে

আসল উদাহরণ, কল্পনা নয়

```
# README.md — আজ যা লেখা আছে

## What exists right now — Slice 1

| Students, academics, attendance,
| lessons, assessments, reports, audit
|                    | **not built yet** |

# বাস্তবে: ১১টি অ্যাপ, ১০৩ টেস্ট,
# ৫৫ পাতা, ৭৭ স্ক্রিনশট — সব চলছে
```

এটাই ডকুমেন্ট-ড্রিফট

কোড এগিয়েছে, দস্তাবেজ দাঁড়িয়ে আছে। মিথ্যা বলা নয় — **ভুলে যাওয়া**।

কেন ভয়ঙ্কর

নতুন ডেভেলপার প্রথমেই README পড়ে — ভুল তথ্য পেলে হয় অপ্রয়োজনীয় কাজ করে, নয়তো ভরসা হারায়।

প্রতিকার

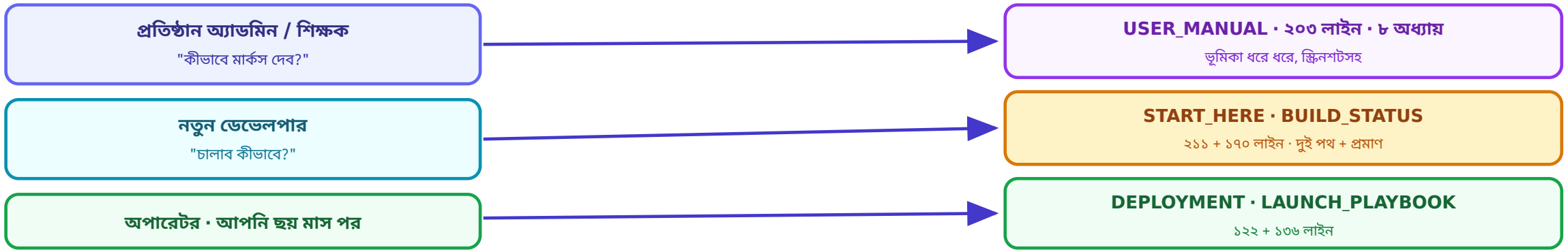
সংখ্যা হাতে না লিখে **স্ক্রিপ্ট** তৈরি করা — এই ক্লাসের শেষ অংশে সেটাই দেখব।

শিক্ষকের নোট: ক্লাস শুরুর এই আঘাতটাই দরকারি: "আপনার নিজের প্রজেক্টেও এমন লাইন আছে।" README-র এই অংশটি প্রজেক্টে সত্যিই আছে — শিক্ষার্থীদের দেখিয়ে দিন।

মানচিত্র

চার পাঠক, চার রকম দস্তাবেজ

একটি দস্তাবেজ সবাইয়ের জন্য নয়



একটি দস্তাবেজে সব লেখার চেষ্টা করলে প্রত্যেকে অপ্রয়োজনীয় অংশ পড়ে থেমে যায় — আর কেউ পুরোটা পড়ে না।

শিক্ষকের নোট: জিজ্ঞেস করুন: "আপনার প্রজেক্টের দস্তাবেজ কে পড়ে?" উত্তর না মিললে দস্তাবেজটি সাজসজ্জা, প্রয়োজন নয়।

তালিকা

ছয়টি ফাইল, ৯০০ লাইন

প্রতিটির একটি নির্দিষ্ট কাজ

ফাইল	লাইন	পাঠক	প্রশ্ন যার উত্তর দেয়
`README.md`	৫৮	সবাই	এটি কী, কী দিয়ে বানানো — **এখন বাসি**
`START_HERE.md`	২১১	নতুন ডেভেলপার	নিজের মেশিনে চালাব কীভাবে
`USER_MANUAL.md`	২০৩	প্রতিষ্ঠান / শিক্ষক	কোন পর্দায় কী করব
`BUILD_STATUS.md`	১৭০	আপনি ও গ্রাহক	কী তৈরি, কী বাকি, কোন প্রমাণে
`LAUNCH_PLAYBOOK.md`	১৩৬	আপনি (অনবোর্ডিং)	স্কুল আনব কীভাবে, টাকা কীভাবে নেব
`DEPLOYMENT.md`	১২২	অপারেটর	সার্ভারে তুলব কীভাবে

ছয়টি ফাইলই মূল (Markdown) — একই লেখা HTML/PDF-এও যায় (`docs/manual/user-manual-bn.html` , `docs/IMS_User_Manual_BN.pdf`)। এক উৎস, অনেক রূপ।

শিক্ষকের নোট: টেবিলের শেষ কলামটাই মূল — প্রতিটি ফাইলের **একটি প্রশ্ন** থাকা দরকার। প্রশ্ন না থাকলে ফাইলটি বাড়ে, কাজে লাগে না।

ব্যবহারবিধি

ভূমিকা ধরে সাজানো ম্যানুয়াল

আটটি অংশ — শুরুতে "কে কী পাবে"

অংশ	বিষয়
১–২	শুরু করার আগে · লগইন, পাসওয়ার্ড, ওয়ার্কস্পেস
অধ্যায় ১	প্রতিষ্ঠান অ্যাডমিন
অধ্যায় ২	কোঅর্ডিনেটর (স্কোপ)
অধ্যায় ৩	শিক্ষক
অধ্যায় ৪	শিক্ষার্থী
অধ্যায় ৫	প্লাটফর্ম অ্যাডমিন
অধ্যায় ৬	সাবস্ক্রিপশন, ট্রায়াল, পেমেন্ট
৭–৮	সমস্যা ও সমাধান · পরিশিষ্ট

কেন ভূমিকা দিয়ে

শিক্ষক "প্রতিষ্ঠান তৈরি" অংশ পড়তে চান না। ভূমিকা-ভিত্তিক ভাগ = কম পড়া, দ্রুত উত্তর।

শেষে সমস্যা-সমাধান

"লগইন হচ্ছে না", "মার্কস সংরক্ষণ হচ্ছে না" — বাস্তব প্রশ্ন দিয়ে শুরু, ব্যাখ্যা পরে।

ফাঁক

ম্যানুয়ালের সঙ্গে স্ক্রিনশটের মিল হাতে রাখা হয় — UI বদলালে পুরোনো ছবি থেকে যায়।

শিক্ষকের নোট: ক্লাসে ম্যানুয়ালের একটি অধ্যায় খুলে দেখান — তারপর জিজ্ঞেস করুন: "এটি পড়ে একজন শিক্ষক সত্যিই কাজ করতে পারবেন?"

নিয়ম

ব্যবহারবিধি লেখার পাঁচ নিয়ম

লেখা ভালো না হলে ব্যবহারকারী ফোন করবে — সেটাই আসল খরচ

নিয়ম	খারাপ	ভালো
এক ধাপে একটি কাজ	"মার্কস দিন এবং প্রকাশ করুন এবং রিপোর্ট দেখুন"	তিনটি আলাদা ধাপ, তিনটি ছবি
পর্দার নাম হুবহু	"সেটিংসে যান"	"Classes & subjects → New class"
ফলাফল আগে বলুন	"ফর্ম পূরণ করুন"	"ফর্ম পূরণ করলে ক্লাস তালিকায় দেখা যাবে"
ভুল হলে কী হবে	কিছুই না	"অনুপস্থিত ছাত্রে নম্বর দিলে সংরক্ষণ হবে না"
স্ক্রিনশট সত্যি হতে হবে	পুরোনো ছবি	চলমান অ্যাপ থেকে তোলা (`shoot.py`)

পরীক্ষা: কাউকে শুধু ম্যানুয়াল দিয়ে কাজটি করতে দিন — আটকে গেলে সেই জায়গাটাই আবার লিখতে হবে। (এটাই bus-factor পরীক্ষা, ক্লাস ৩২-এর।)

শিক্ষকের নোট: শিক্ষার্থীদের জোড়ায় ভাগ করে একটি অধ্যায় লিখিয়ে অন্য জনকে দিয়ে চালিয়ে দেখান — সবচেয়ে দ্রুত শেখার উপায়।

স্ক্রিনশট

৭৭টি ছবি, একটি স্ক্রিপ্ট

হাতে তোলা ছবি পুরোনো হয় — স্ক্রিপ্টের ছবি নয়

```
# docs/shoot.py — চলমান অ্যাপ থেকে ছবি
BASE = "http://127.0.0.1:8000"
DESKTOP = {"width": 1440, "height": 980}
MOBILE  = {"width": 390,  "height": 844}

def login(page, email):        # আসল লগইন
    page.fill("#id_username", email)
    page.fill("#id_password", PASSWORD)
    page.click("button[type=submit]")

def shot(page, name):          # নেটওয়ার্ক শান্ত হলে ছবি
    page.wait_for_load_state("networkidle")
    page.screenshot(path=f"{OUT}/{name}")
```

ফোল্ডার	সংখ্যা
`screenshots/` (মূল)	৫৩
`screenshots/saas/`	১৩
`screenshots/roles/`	৯
`screenshots/manual/`	৫
মোট	**৭৭ PNG**

নামকরণ: `30_attendance_take.png` — ক্রম + পর্দার নাম; খোঁজা সহজ, অধ্যায়ের ক্রমও মেলে।

শিক্ষকের নোট: `shoot.py` চালাতে সার্ভার ও ডেমো ডেটা দরকার — ক্লাসের আগে `bash run.sh --reset` করে রাখুন।

প্রমাণ

পরিকল্পনা বনাম প্রমাণ

৫৭ ফিচার · ৫৪টির ছবি-প্রমাণ · দুই আলাদা ফাইল

```
# docs/build_status.py — প্রমাণের মানচিত্র
BUILT = {
    "F-01": "01_login.png",
    "F-02": "02_register.png",
    "F-05": "07_profile.png",
    "F-08": "05_switch_workspace.png",
    "F-09": "08_create_institution.png",
    "F-13": "10_org_admin_dashboard.png",
    "F-16": "04_teacher_dashboard.png",
    ...
}

# ডকুমেন্টেশনে লেখা আছে:
# "Kept separate from catalogue_data.py on purpose:
#  the catalogue is the plan, this file is the evidence."
```

দুই ফাইল, দুই প্রশ্ন

`catalogue_data.py` (৩১ KB) — কী হওয়ার কথা (৫৭ ফিচার)।

`build_status.py` — কোনটির ছবি-প্রমাণ আছে (৫৪টি)।

কেন আলাদা

"ছবি তালিকা আবার তৈরি করলে ফিচার-স্পেসিফিকেশন নষ্ট হবে না" — ডিজাইনের সিদ্ধান্ত, কোডেই লেখা।

ফল

দাবি করা আর প্রমাণ করা **একই জায়গায়** থাকে — তাই ফাঁক লুকায় না, গোনা যায়।

শিক্ষকের নোট: এটি এই কোর্সের মূল দর্শন: "দাবি + প্রমাণ"। শিক্ষার্থীকে বলুন নিজের প্রজেক্টে F-নম্বরের বদলে নিজের ফিচার-নাম দিয়ে এমন একটি মানচিত্র বানাতে।

দস্তাবেজও কোড

পাঁচটি জেনারেটর স্ক্রিপ্ট

হাতে লেখা সংখ্যা বাসি হয় — চালানো সংখ্যা নয়

স্ক্রিপ্ট	আকার	কী তৈরি করে
`docs/catalogue_data.py`	৩১ KB	ফিচার-ক্যাটালগের উপাদান (৫৭টি)
`docs/gen_feature_catalogue.py`	৩৪ KB	ক্যাটালগ PDF (স্ক্রিনশটসহ)
`docs/gen_proof_pdf.py`	১৯ KB	`Build_Proof.pdf` — প্রমাণের নথি
`docs/gen_roadmap.py`	২৮ KB	ক্লাস-রোডম্যাপ নথি
`docs/build_status.py`	৩.৩ KB	ফিচার → স্ক্রিনশট মানচিত্র
`docs/shoot.py`	৬.২ KB	৭৭টি স্ক্রিনশট (ডেস্কটপ + মোবাইল)

সঠিক ক্রম: `run.sh` → `shoot.py` (ছবি) → `gen_*` (নথি) — ছবি ছাড়া নথি তৈরি হলে ফাঁকা পাতা।

ফাঁক: এই ছয়টি চালানোর এক-কমান্ড স্ক্রিপ্ট (`rebuild_docs.sh`) এখনো নেই — কেউ একটা ভুলে যায়।

শিক্ষকের নোট: ক্লাসে `python docs/build_status.py` চালিয়ে দেখান — সংখ্যা নিজে থেকে হালনাগাদ হচ্ছে কি না। এই একটি মুহূর্ত অনেকের ধারণা বদলায়।

নতুনদের পথ

`START_HERE` — দুই পথ, এক গন্তব্য

২১১ লাইন, যাতে প্রথম দিনে কেউ আটকে না থাকে

পথ A — SQLite (প্রস্তাবিত প্রথমবার)

শুধু Python লাগে · `bash run.sh` · একটি ফাইল ডেটাবেস · ৩ মিনিটে চলা শুরু

পথ B — PostgreSQL (আসল ভঙ্গি)

ডেটাবেস সার্ভার + `.env` · প্রোডাকশনের মতো · ডেভেলপার যেটা শিখবে সেটাই সার্ভারে

যা দুই পথেই আছে

ভার্চুয়াল এনভায়রনমেন্ট · `pip install -r requirements.txt` · `seed_demo` · চারটি লগইন

```
# পথ A — তিন কমান্ড
bash run.sh
#  http://127.0.0.1:8000/

# পথ B
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env      # মান বসান
python manage.py migrate
python manage.py seed_demo
python manage.py runserver
```

শিক্ষকের নোট: দুই পথ রাখার কারণ: প্রথমবার যেটা সহজ, সেটাই শেখার পথ; পরে আসল ভঙ্গি। একটাই পথ রাখলে নতুনদের অর্ধেক আটকে যাবে।

সংখ্যার খাতা

`BUILD_STATUS` — সংখ্যা ও প্রমাণ

প্রতিটি সংখ্যার পাশে কমান্ড, প্রতিটি দাবির পাশে টেস্ট

```
## Verified numbers
python manage.py test     ->  90 tests, 90 OK
python smoke_test.py      ->  55 pages x 4 roles
manage.py check --deploy  ->  no issues

## Remaining work (not faked, not skipped)
1. Invitation workflow
2. Platform console
4. Backup & restore console
5. Security attack suite
```

এটি বাসি হয়ে গেছে

এখন ৯০ নয়, **১০৩ টেস্ট** — সংখ্যাটি লেখার সময়ের সত্যি, আজকের নয়।

হালনাগাদের নিয়ম

প্রতিটি রিলিজে তিনটি সংখ্যা আবার চালান: `test` · `smoke_test` · `check --deploy`।

দর্শন

"not faked, not skipped" — যা হয়নি, তাও লেখা। এটাই দস্তাবেজকে বিশ্বাসযোগ্য করে।

শিক্ষকের নোট: এই স্লাইডটি ভালো ও খারাপ দুই দিকই দেখায়: প্রমাণ-খাতার ধারণা সঠিক, কিন্তু হালনাগাদ না হলে সেটিও মিথ্যা বলতে শুরু করে।

সততা

সৎ ভাষায় লেখা

যা নেই তা লেখাই দস্তাবেজের সবচেয়ে দামি অংশ

প্রজেক্টে যা লেখা আছে	মানে
"Remaining work (**not faked, not skipped**)"	ফাঁক লুকানো হয়নি, বাদও দেওয়া হয়নি
"It has **not** yet been deployed to a public host"	রিহার্সাল হয়েছে, ডিপ্লয় হয়নি — পার্থক্য স্পষ্ট
"**Demo data removed** — DemoPass!2026 gone"	চেকলিস্টে দায়িত্ব, অনুমান নয়
"A backup you have never restored is not a backup"	নীতি এক বাক্যে, মনে রাখার মতো

যা লিখবেন না: "সব কাজ করছে" · "শীঘ্রই আসছে" · যাচাই না করা সংখ্যা

যা লিখবেন: "কাজ করছে (টেস্ট ১০৩, আজ ২০২৬-১১-০৫)" · "আসছে: কবে, কীভাবে বোঝা যাবে"

শিক্ষকের নোট: সৎ দস্তাবেজের সবচেয়ে বড় লাভ: গ্রাহক একবার ভুল তথ্য পেলে আর কিছুতেই ভরসা করে না। "না" বলা বিশ্বাসযোগ্যতা বাড়ায়।

ভাষা

বাংলায় টেকনিক্যাল লেখা

শব্দ বাংলা, পরিভাষা ইংরেজি — মিশ্রণ নিয়ম মেনে

যা অনুবাদ করবেন না	কারণ
`migrate` · `commit` · `deploy`	টার্মিনালে যেটা লেখে, দস্তাবেজেও সেটাই
`middleware` · `queryset`	বাংলা করলে খুঁজে পাওয়া যায় না
পর্দার নাম (ইংরেজি)	ব্যবহারকারী পর্দায় ইংরেজিই দেখেন

যা বাংলায় লিখবেন

ব্যাখ্যা, কারণ, সতর্কতা, ধাপের ক্রিয়া — "সংরক্ষণ করুন", "যাচাই করুন", "ফেরান"। এগুলো পড়ার গতি বাড়ায়।

```
# ভালো মিশ্রণ
"মার্কস সংরক্ষণ করতে Save চাপুন।
অনুপস্থিত ছাত্রে নম্বর দিলে ফর্ম
গ্রহণ করবে না।"

# খারাপ
"মার্কসংরক্ষণকার্যসম্পাদনসূচক বোতাম"

# আরও খারাপ
"Click the save button to persist
the marks record in the database"
```

শিক্ষকের নোট: শিক্ষার্থীরা দুই চরমে যায় — পুরো ইংরেজি, বা কঠিন বাংলা। মাঝের পথটাই ব্যবহারিক; একটি প্যারাগ্রাফ একসাথে লিখে দেখান।

অনুশীলন ও আলোচনা

২০ মিনিট — একটি পাতা লিখুন

চারটি কাজ, সবই ব্যবহারযোগ্য

১. একটি কাজের পাতা

"উপস্থিতি নেওয়া" — ৫ ধাপ, পর্দার নাম হুবহু, স্ক্রিনশটের জায়গা চিহ্নিত।

২. প্রশ্ন-উত্তর পাঁচটি

"লগইন হচ্ছে না" ধরনের বাস্তব সমস্যা, সংক্ষিপ্ত সমাধান।

৩. সত্যি সংখ্যা

নিজের প্রজেক্টে `test` চালিয়ে সংখ্যা লিখুন — সাথে তারিখ।

৪. প্রমাণ-মানচিত্র

পাঁচটি ফিচার → কোন স্ক্রিনশট প্রমাণ করে (ছোট ডিকশনারি)।

- দস্তাবেজ কে পড়বে?
- সংখ্যা হাতে লিখলে কী হয়?
- কেন পরিকল্পনা ও প্রমাণ আলাদা ফাইল?
- টেকনিক্যাল শব্দ অনুবাদ করলে?
- যা নেই তা লেখা কেন দরকার?

শিক্ষকের নোট: চারটি কাজের ফল জমা নিন — পরের ক্লাসে সেগুলোই আরও পরিষ্কার করা হবে (পর্ব ৮-এর দ্বিতীয় ক্লাস)।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

দস্তাবেজের প্রথম ক্লাসের সারাংশ

নীতি	প্রমাণ
প্রতি পাঠকের জন্য আলাদা দস্তাবেজ	২১১ · ২০৩ · ১৭০ · ১৩৬ · ১২২ · ৫৮ লাইন
এক ধাপে একটি কাজ, পর্দার নাম হুবহু	USER_MANUAL-এর ৮ অধ্যায়
ছবি স্ক্রিপ্টে, হাতে নয়	`shoot.py` · ৭৭ PNG
পরিকল্পনা ও প্রমাণ আলাদা রাখুন	৫৭ ফিচার · ৫৪ ছবি-প্রমাণ
সংখ্যা জেনারেট করুন, লিখবেন না	`docs/gen_*.py` পাঁচটি
যা নেই তা লিখুন	"not faked, not skipped"

পরের ক্লাসে: ডেভেলপার-দস্তাবেজ — কোডের ভেতরের ব্যাখ্যা, চেঞ্জলগ, এবং "কেন এভাবে" লেখার কৌশল।

শিক্ষকের নোট: শেষে একটি কাজ দিন: "আপনার README-এর বাসি লাইনটি আজই ঠিক করুন" — ছোট, কিন্তু বদল দেখা যায়।

পর্ব ৮ · ক্লাস ৩৪

কোডের ভেতরের দস্তাবেজ

docstring · "কেন" মন্তব্য · URL মানচিত্র · CHANGELOG · ADR · CONTRIBUTING

IMS · ক্লাস ৩৪ · স্লাইড ১৬–৩০

শুরু

কোডে দস্তাবেজ কতটা আছে?

আজ গোনা সংখ্যা — ৮৭ ফাইল, ৫৭৬ def/class

৭৫টি docstring — ১৩.০%

৫৭৬টি ফাংশন/ক্লাসের মধ্যে মাত্র ৭৫টির ব্যাখ্যা আছে। বাকিগুলো নামের ওপরই ভরসা করছে।

মডিউল-স্তরে ভালো

প্রায় প্রতিটি অ্যাপের `models.py` / `views.py` -এর শুরুতে উদ্দেশ্য লেখা — যেমন "Exams, marks, grading and results (C112–C119)".

ফাংশন-স্তরে দুর্বল

জটিল helper-গুলোর ব্যাখ্যা নেই — পঠনযোগ্যতা নামের ওপর নির্ভরশীল। এই ফাঁকটাই আজকের কাজ।

সংখ্যা বাড়ানোই লক্ষ্য নয় — **যেখানে নাম বলতে পারে না, সেখানেই** ব্যাখ্যা দরকার। সব ফাংশনে docstring দিলে তা আবার পড়া হয় না।

শিক্ষকের নোট: পরিমাপটি দেখান: `ast` দিয়ে ২০ লাইনের স্ক্রিপ্টেই এই সংখ্যা বের হয় — শিক্ষার্থীরা নিজের প্রজেক্টে চালাতে পারবে।

মডিউল-ডকস্ট্রিং

ফাইলের শুরুতে তিনটি প্রশ্নের উত্তর

এটি কী · নিয়ম কী · কেন এখানে

```
"""One place that answers: 'what may this person
see?' (production hardening, C126).

Every list view asks these helpers instead of
writing its own filter. That is how a role bug is
fixed once instead of in nine views.

    admin       -> everything in the workspace
    coordinator -> only the departments / classes
                   assigned to them
    teacher     -> only classes assigned to teach
    student     -> only their own record
"""
# apps/core/scoping.py
```

প্রথম লাইনে উদ্দেশ্য

"এক জায়গা যেখানে উত্তর আছে: এই মানুষটি কী দেখতে পারে?"

তারপর নিয়ম

চার ভূমিকার চারটি স্কোপ — টেবিলের মতো, চোখে পড়ে।

শেষে "কেন"

"নয়টি ভিউয়ে আলাদা করে লিখলে বাগ নয় জায়গায় ঠিক করতে হতো।"

যা লিখবেন না

"এই ফাইলে স্কোপিং ফাংশন আছে" — সেটা ফাইলের নামেই আছে।

শিক্ষকের নোট: এই docstring-টি প্রজেক্টে সত্যিই আছে — পড়ে দেখান। তারপর জিজ্ঞেস করুন: "এটি না থাকলে নতুন ডেভেলপার কত সময় নষ্ট করত?"

"কেন" মন্তব্য

যেখানে কোড নিজেই বিভ্রান্তিকর

অদ্ভুত দেখায় এমন লাইনের পাশে কারণ লিখুন

```
# apps/billing/middleware.py
# The membership that matters is for the workspace in
# the URL -- the session one can belong to a different
# institution entirely (admin of one, coordinator of
# another), which is exactly how this branch was wrong
# the first time.

# apps/audit/models.py
def log(...):
    """Write one audit row. Never raises: logging must
    not break a workflow."""
```

মূল্য

এই মন্তব্য না থাকলে পরের কেউ "অপ্রয়োজনীয়" ভেবে সরিয়ে দিত — আর বাগটি ফিরে আসত।

সূত্র

"this is exactly how this branch was wrong the first time" — সত্যি ঘটে যাওয়া বাগের স্মৃতি।

নিয়ম

মন্তব্যে "কী" নয়, "কেন" — এবং সম্ভব হলে ঘটে যাওয়া সমস্যার কথা।

শিক্ষকের নোট: বলুন: "সবচেয়ে দামি মন্তব্য সেই যেটি ভবিষ্যতের কাউকে একই বাগ করতে বাধা দেয়।" নিজের প্রজেক্টে এমন একটি খুঁজতে বলুন।

ডকস্ট্রিং কখন

কোথায় ব্যাখ্যা দরকার, কোথায় নয়

নামই যদি বলে দেয়, docstring বাড়তি

দরকার নেই

`def can_manage_workspace(membership):`
নামই বলছে — প্রতিষ্ঠান অ্যাডমিন কি না।

`def visible_classes(user, workspace):`
স্কোপিং ফাইলে আছে, উদ্দেশ্য স্পষ্ট।

দরকার আছে

`def is_writable(workspace):` — কোন পাঁচ অবস্থায় সত্য, তা দ্রুত দেখা দরকার।

`def _next_number(prefix):` — ফরম্যাট, প্রতিযোগিতা ও ব্যর্থতার নিয়ম ব্যাখ্যা দরকার।

যেখানে **নিয়ম** আছে (নিরাপত্তা, টাকা, তারিখ) — সেখানে ব্যাখ্যা।

নিয়ম: **যে ফাংশনে ভুল হলে টাকা বা নিরাপত্তা ক্ষতিগ্রস্ত হয়** — সেখানে সবসময় ব্যাখ্যা। বাকিতে নাম ও টেস্টই যথেষ্ট।

শিক্ষকের নোট: এটি সময় বাঁচানোর নিয়ম: সব ফাংশনে docstring লিখতে গেলে কেউ কিছুই লিখবে না। অগ্রাধিকার দিন গুরুত্বপূর্ণ জায়গায়।

মানচিত্র

URL মানচিত্র — ৮৫টি রুট, ১১ অ্যাপ

নতুন ডেভেলপারের প্রথম প্রশ্ন: কোন পর্দা কোথায়?

অ্যাপ	রুট	প্রধান পর্দা
`billing`	১৮	প্ল্যান · পাবলিক প্রাইসিং · সাবস্ক্রিপশন · ইনভয়েস · প্লাটফর্ম কনসোল
`academics`	১৩	শিক্ষাবর্ষ · টার্ম · বিভাগ · ক্লাস · বিষয় · অ্যাসাইনমেন্ট
`lessons`	১৩	সিলেবাস · টপিক · হোমওয়ার্ক · জমা
`accounts`	৯	লগইন · রেজিস্ট্রেশন · প্রোফাইল · পাসওয়ার্ড
`assessments`	৯	পরীক্ষা · নম্বর এন্ট্রি · প্রকাশ · মার্কশিট
`reports` · `students`	৭ + ৭	রিপোর্ট · CSV/XLSX · ছাত্র তালিকা ও প্রোফাইল
`attendance` · `audit` · `workspaces` · `core`	৪ · ২ · ২ · ১	উপস্থিতি · অডিট · সুইচ/তৈরি · ড্যাশবোর্ড

এই ছকটি কোথাও লেখা নেই — `grep 'path('` চালিয়ে তৈরি হয়েছে। `docs/url-map.md` বানানো পরের কাজ; নতুনদের জন্য এটি সবচেয়ে দ্রুত মানচিত্র।

শিক্ষকের নোট: ক্লাসেই ছকটি তৈরি করে দেখান: `for f in apps/*/urls.py; do echo $(dirname $f) $(grep -c 'path(' $f); done`

নামকরণ

নামই যখন দস্তাবেজ

ভালো নাম লাইন কমায়, খারাপ নাম মন্তব্য বাড়ায়

IMS-এর নাম	কী বলে
`test_student_only_gets_published_homework`	দাবিটাই নামে লেখা — ব্যর্থ হলে বোঝা যায়
`test_server_refuses_the_third_student`	সিট সীমার নিয়ম, ঠিক কোথায় ভাঙে
`visible_homework` · `is_writable`	ক্রিয়া + উদ্দেশ্য; বাড়তি ব্যাখ্যা লাগে না
`views_platform.py`	ফাইলটি প্লাটফর্ম-কনসোলের — ভাগ করা সহজ
`seed_demo` · `billing_report` · `sync_domains`	ম্যানেজমেন্ট কমান্ডের কাজ নামেই

পরীক্ষা: নাম শুনে কাজ বোঝা যায়? না হলে নাম বদলান — মন্তব্য লেখার আগে।

দুই অ্যাপে একই টেস্ট-নাম: `test_teacher_without_assignment_sees_no_students` — রিপোর্টে গুলিয়ে যায় (ক্লাস ২৬-এ চিহ্নিত)।

শিক্ষকের নোট: শিক্ষার্থীদের সবচেয়ে খারাপ তিনটি নাম লিখতে বলুন, তারপর নতুন নাম প্রস্তাব করতে — দ্রুত উন্নতি দেখা যায়।

CHANGELOG

রিলিজ নোট — এখনো নেই, তবে দরকার

গ্রাহকের প্রথম প্রশ্ন: "নতুন কী এল?"

```
# CHANGELOG.md (প্রস্তাবিত)

## 1.4.0 — 2026-12-01
### যোগ হয়েছে
+ invitation workflow (F-10..12)
+ CSV student import

### বদলেছে
* academics tests 52 -> 90

### মাইগ্রেশন
! students 0004_add_invitation

### ঠিক হয়েছে
# marks entry rejected 0 as absent

## 1.3.2 — 2026-11-20
### ঠিক হয়েছে
# CSS cache busting on redeploy
```

আজকের অবস্থা

১৯টি কমিট আছে, `CHANGELOG.md` বা git tag নেই — "কোন সংস্করণে কী ছিল" প্রশ্নের উত্তর নেই।

কেন মাইগ্রেশন আলাদা লাইন

রিলিজ নোট পড়ে অপারেটর ব্যাকআপ নেয় — মাইগ্রেশন উল্লেখ না থাকলে নেয় না।

কে লেখে

যে রিলিজ করে — কারণ সে-ই জানে কী বদলেছে। পরে লিখতে গেলে ভুলে যাবে।

শিক্ষকের নোট: বলুন: "চেঞ্জলগ গ্রাহকের জন্য নয় শুধু, নিজের জন্যও — ছয় মাস পর নিজেই ভুলে যাবেন।" প্রথম এন্ট্রিটি ক্লাসেই লিখিয়ে নিন।

ADR

`docs/adr/` — সিদ্ধান্তের স্থায়ী ঠিকানা

এক সিদ্ধান্ত, এক ফাইল, কখনো সম্পাদনা নয়

```
docs/adr/
  0001-modular-monolith.md
  0002-custom-user-email-login.md
  0003-billing-inside-the-app.md
  0004-subscription-gate-middleware.md
  0005-custom-domain-verified-only.md

# প্রতিটির গঠন
## প্রসঙ্গ
## সিদ্ধান্ত
## বাদ পড়া বিকল্প
## পরিণতি
## তারিখ ও অবস্থা (গৃহীত / বাতিল)
```

৫টি আসল সিদ্ধান্ত, ৫টি ফাইল

প্রতিটির পক্ষে-বিপক্ষে যুক্তি কোডে ছড়িয়ে আছে — এক জায়গায় আনলেই নতুনদের সময় বাঁচে।

সম্পাদনা করবেন না

সিদ্ধান্ত বদলালে নতুন ADR লিখুন, পুরোনোটির অবস্থা "বাতিল" করুন — ইতিহাস রাখুন।

সংখ্যাই ক্রম

`0004` থেকে বোঝা যায় কোনটি আগে — তারিখ মুখস্থ করতে হয় না।

শিক্ষকের নোট: ক্লাসে `docs/adr/0001-modular-monolith.md` তৈরি করে দেখান — ২০ মিনিটের কাজ, কিন্তু বছরের জন্য।

CONTRIBUTING

দলে যোগ দেওয়ার এক পাতা

নিয়ম লিখিত না হলে নিয়ম নেই

```
# CONTRIBUTING.md

## প্রথম দিন
1. START_HERE.md পড়ে অ্যাপ চালান
2. manage.py test -> ১০৩ সবুজ
3. BUILD_STATUS.md পড়ুন

## কাজ শুরু
* শাখা: fix/marks-boundary
* ছোট কমিট, বার্তায় "কী + কেন"
* টেস্ট ছাড়া PR নয়

## মাইগ্রেশন থাকলে
* রিলিজ নোটে উল্লেখ করুন
* রিভিউয়ারকে আলাদা করে বলুন

## মার্জের শর্ত
টেস্ট সবুজ + একজন রিভিউ + দস্তাবেজ হালনাগাদ
```

কেন দরকার

"কেউ বলে দেবে" ধরে নিলে নতুন সদস্য ভুল করে, আর আপনি বিরক্ত হন — দুটোই এড়ানো যায়।

এক পাতা যথেষ্ট

৫০ লাইনের বেশি লিখলে কেউ পড়ে না — আজকের তিনটি নিয়মই যথেষ্ট।

আজ নেই

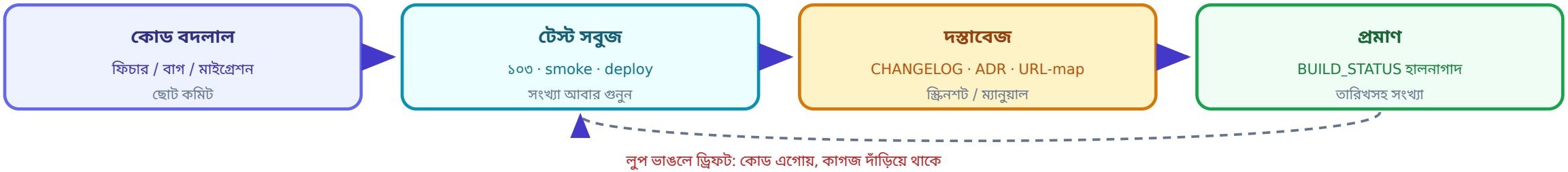
প্রজেক্টে `CONTRIBUTING.md` বা `CHANGELOG.md` নেই — দুটিই প্রথম সপ্তাহের কাজ।

শিক্ষকের নোট: শিক্ষার্থীদের নিজের দলে "মার্জের শর্ত" তিনটি লিখতে বলুন — এটাই সবচেয়ে কাজে লাগা অংশ।

ড্রিফট

কোড আর দস্তাবেজ মিলিয়ে রাখার লুপ

প্রতি রিলিজে একই চার ধাপ



যাচাইয়ের কমান্ড: `makemigrations --check --dry-run` · `test` · `check --deploy` · `build_status.py`

ফাঁক: দস্তাবেজ হালনাগাদ এখনো হাতে — CI বা প্রি-কমিট হুক থাকলে ভুলে যাওয়া বন্ধ হতো।

শিক্ষকের নোট: এই চার ধাপ রিলিজ-চেকলিস্টে বসিয়ে দিন — কাগজে থাকলে মনে থাকবে, মুখে থাকলে থাকবে না।

কমান্ডের খাতা

নয়টি স্ক্রিপ্ট, একটি ছক

যে কেউ যেন চালাতে পারে

কমান্ড	কী করে	কখন চালাবেন
`bash run.sh`	SQLite ডেমো চালু (৮০০০)	প্রথমবার / ক্লাসে
`bash run.sh --reset`	ডেটাবেস নতুন করে বানায়	ডেমো ডেটা নষ্ট হলে
`bash build.sh`	install + collectstatic + migrate	হোস্টে রিলিজ
`bash docs/deploy_check.sh`	Gunicorn :8010 + ৫টি চেক	ডিপ্লয়ের আগে
`ops/backup.sh` · `ops/restore.sh`	ডাম্প · scratch-এ রিস্টোর	রাত ২টা · মাসে একবার
`manage.py seed_demo` · `seed_plans`	ডেমো ডেটা · প্ল্যান	কখনো প্রোডাকশনে নয় (ডেমো)
`manage.py billing_report` · `sync_domains`	বাণিজ্যিক রিপোর্ট · DNS ছক	প্রতিদিন সকালে · ডোমেইন যোগে

প্রজেক্টে চারটি কাস্টম ম্যানেজমেন্ট কমান্ড: `seed_demo` · `seed_plans` · `billing_report` · `sync_domains` — প্রতিটির `--help` লেখা আছে, কে কখন চালাবে তা এই ছকে।

শিক্ষকের নোট: ক্লাসে `python manage.py billing_report --help` চালিয়ে দেখান — স্ব-দস্তাবেজিত কমান্ডের সুবিধা।

গঠন

`docs/` ফোল্ডার কেমন হবে

আজ কী আছে, কাল কী যোগ হবে

```
docs/
  adr/                        # নতুন
    0001-modular-monolith.md
  manual/
    user-manual-bn.html
  screenshots/      (৭৭ PNG)
  catalogue_data.py
  gen_feature_catalogue.py
  gen_proof_pdf.py
  gen_roadmap.py
  build_status.py
  deploy_check.sh
  shoot.py
  url-map.md                  # নতুন
  README.md                   # নতুন: কোন ফাইল কীসের
```

docs/README.md কেন

দশটি ফাইল থাকলে কেউ জানে না কোনটি পড়বে — একটি সূচি সব সমস্যা মেটায়।

স্ক্রিপ্ট আর দস্তাবেজ আলাদা

যা চালাতে হয় আর যা পড়তে হয় — দুই ধরনের ফাইল একই ফোল্ডারে থাকলে নামে পার্থক্য রাখুন।

যা ফেলে রাখবেন না

`old/`, `backup/`, `final_v2/` — পুরোনো দস্তাবেজ মুছুন বা আর্কাইভে সরান।

শিক্ষকের নোট: এই গঠনটি ক্লাসেই তৈরি করুন: `docs/adr/`, `docs/url-map.md`, `docs/README.md` — তিনটি ফাইল, বিশ মিনিট।

অনুশীলন ও আলোচনা

২০ মিনিট — কোডে দস্তাবেজ যোগ করুন

চারটি কাজ, সবই কমিট করার মতো

১. একটি মডিউল-ডকস্ট্রিং

নিজের সবচেয়ে জটিল ফাইলে: উদ্দেশ্য · নিয়ম · কেন এখানে।

২. একটি "কেন" মন্তব্য

যে লাইনটি অন্য কেউ "অপ্রয়োজনীয়" ভেবে মুছে দিতে পারে।

৩. CHANGELOG-এর প্রথম এন্ট্রি

আজকের কাজ, মাইগ্রেশনসহ।

৪. একটি ADR

নিজের প্রজেক্টের এক সিদ্ধান্ত, চার শিরোনামে।

- সব ফাংশনে docstring দরকার?
- ADR কেন সম্পাদনা করবেন না?
- CHANGELOG-এ মাইগ্রেশন কেন আলাদা?
- URL মানচিত্র কে ব্যবহার করবে?
- দস্তাবেজ ড্রিফট ধরবেন কীভাবে?

শিক্ষকের নোট: চারটি কাজই ছোট; ক্লাসেই করে ফেলা যায়। যেটি করা যায়নি, সেটি পরের ক্লাসের শুরুতে দেখে নিন।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

ডেভেলপার-দস্তাবেজের সারাংশ

নীতি	প্রমাণ
মডিউল-ডকস্ট্রিং: উদ্দেশ্য · নিয়ম · কেন	`scoping.py` · `assessments/models.py`
মন্তব্যে "কেন", "কী" নয়	`middleware.py`-র সেশন-স্কোপ মন্তব্য
docstring যেখানে টাকা/নিরাপত্তা জড়িত	৭৫ / ৫৭৬ = ১৩% — ফাঁক চিহ্নিত
নামই প্রথম দস্তাবেজ	`test_server_refuses_the_third_student`
CHANGELOG + ADR + CONTRIBUTING	তিনটিই এখনো নেই — এই সপ্তাহের কাজ
প্রতি রিলিজে চার ধাপের লুপ	কোড → টেস্ট → দস্তাবেজ → প্রমাণ

পরের ক্লাস (৩৫): প্রমাণের দস্তাবেজ ও উপস্থাপন — স্ক্রিনশট, প্রমাণ-প্যাক, এবং পর্ব ৮-এর সমাপ্তি।

শিক্ষকের নোট: শেষে একটি কাজ দিন: "আজ একটি ADR ফাইল তৈরি করুন" — ছোট কাজ, কিন্তু অভ্যাস তৈরি হয়।

পর্ব ৮ · ক্লাস ৩৫

প্রমাণ দেখানো

প্রমাণ-প্যাক · তিনটি PDF · ডেমো স্ক্রিপ্ট · এক-পাতার সারসংক্ষেপ · রিপোজিটরি স্বাস্থ্য

IMS · ক্লাস ৩৫ · স্লাইড ৩১–৪৪

শুরু

"আমরা বানিয়েছি" — প্রমাণ কোথায়?

তিনটি PDF, ১৩৩ পাতা, ১১.৫ মেগাবাইট

Build_Proof.pdf

৩২ পাতা · ৩.৮৭ MB — কোন ফিচার, কোন টেস্ট, কোন কমান্ডের আউটপুট।

IMS_Feature_Catalogue.pdf

৭০ পাতা · ৪.৮৭ MB — ৫৭ ফিচার, প্রতিটির স্ক্রিনশট ও অবস্থা।

IMS_User_Manual_BN.pdf

৩১ পাতা · ২.৮৪ MB — বাংলায় ব্যবহারবিধি, ব্যবহারকারীর হাতে দেওয়ার মতো।

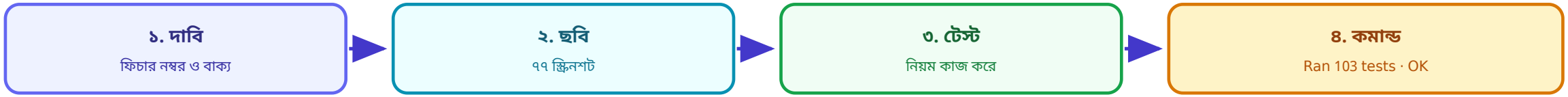
এই তিনটি কেউ হাতে লেখেনি — `docs/gen_proof_pdf.py`, `gen_feature_catalogue.py` আর `manual/print_manual.py` তৈরি করে। তাই সংখ্যা বদলালে প্রমাণও বদলায়।

শিক্ষকের নোট: ক্লাসে তিনটি PDF খুলে দেখান — বিশেষ করে স্ক্রিনশটের পাতা, যেখানে ফিচারের নামের নিচে আসল পর্দা। "এটাই প্রমাণ, কথা নয়।"

প্রমাণ-প্যাক

চার স্তরের প্রমাণ

দাবি → ছবি → টেস্ট → কমান্ড



স্তর	কী দেখায়	IMS-এ কোথায়
১. দাবি	"মার্কস প্রকাশের আগে শিক্ষার্থী দেখতে পায় না"	`catalogue_data.py` F-নম্বর
২. ছবি	পর্দাটি সত্যিই আছে	`docs/screenshots/` — ৭৭ PNG
৩. টেস্ট	নিয়মটি সত্যিই কাজ করে	`test_results_are_invisible_until_published` (৪০৩ → ২০০)
৪. কমান্ড	পুরো সিস্টেম একসাথে চলে	`Ran 103 tests … OK` · `55 pages × 4 roles`

একটি দাবি, তিনটি প্রমাণ — ছবি ছাড়া বিশ্বাস হয় না, টেস্ট ছাড়া ভরসা হয় না, কমান্ড ছাড়া পুরোটা দেখা যায় না।

যেটি নেই: পারফরম্যান্স ও লোড-টেস্টের প্রমাণ — "৮০০ ছাত্রে কেমন চলে" এখনো মাপা হয়নি।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন নিজের প্রজেক্টে এই চার স্তরের একটি ছক বানাতে — কোথায় প্রমাণ নেই, সাথে সাথে দেখা যাবে।

Build_Proof

৩২ পাতার প্রমাণ-নথি

কীভাবে সাজানো, কীভাবে পড়া

কাঠামো

ভূমিকা → ফিচার-অবস্থা সারণি → স্ক্রিনশট প্রমাণ → টেস্ট ও কমান্ড আউটপুট → সীমা ও বাকি কাজ

তৈরি হয় কীভাবে

`docs/gen_proof_pdf.py` (১৯ KB) — reportlab দিয়ে, ছবি ও সংখ্যা একসাথে

কে পড়ে

গ্রাহক, শিক্ষক, মূল্যায়নকারী — যে কেউ "সত্যিই হয়েছে?" জিজ্ঞেস করে

কেন PDF

পাঠানোর জন্য সহজ, প্রিন্ট করা যায়, রূপ বদলায় না — দস্তাবেজের "ছবি"

```
Build_Proof.pdf — ৩২ পাতা

১.  শিরোনাম + তারিখ
২.  প্রযুক্তি ও সংখ্যা
৩.  ফিচার-অবস্থা সারণি
৪–২০. ফিচার + স্ক্রিনশট
২১. টেস্ট ফলাফল
২২. smoke_test ফলাফল
২৩. check --deploy
২৪. সীমা (যা নেই)
২৫–৩২. পরিশিষ্ট
```

শিক্ষকের নোট: "কমান্ডের আউটপুট কপি-পেস্ট নয়, স্ক্রিপ্ট থেকে" — এটাই নথিটিকে বিশ্বাসযোগ্য করে। ক্লাসে `gen_proof_pdf.py` চালিয়ে দেখান।

ক্যাটালগ

৭০ পাতার ফিচার ক্যাটালগ

৫৭টি ফিচার — কোনটি তৈরি, কোনটি বাকি

অংশ	বিষয়	প্রমাণ
F-01 … F-09	অ্যাকাউন্ট ও প্রতিষ্ঠান	লগইন, রেজিস্ট্রেশন, ওয়ার্কস্পেস সুইচ
F-10 … F-15	ভূমিকা ও আমন্ত্রণ (আংশিক)	ভূমিকা আছে, ইনভাইট নেই
F-16 … F-30	একাডেমিক কাঠামো	ক্লাস, বিষয়, শিক্ষাবর্ষ, টার্ম
F-31 … F-45	উপস্থিতি, পাঠ, হোমওয়ার্ক	নেওয়া, সংশোধন, জমা, রিভিউ
F-46 … F-57	মূল্যায়ন, রিপোর্ট, বিলিং	নম্বর, প্রকাশ-গেট, CSV/XLSX/PDF, প্ল্যান

দুই ফাইলের ভূমিকা: `catalogue_data.py` (৩১ KB) = পরিকল্পনা · `build_status.py` = প্রমাণের মানচিত্র (৫৪টি ফিচারের ছবি)।

সৎ অবস্থা: ৫৭-এর ৫৪টির ছবি-প্রমাণ; বাকি তিনটি লিখিত আছে, তৈরি নয় — ক্যাটালগেই সেভাবেই আছে।

শিক্ষকের নোট: ক্যাটালগের একটি পাতা খুলে দেখান — ফিচারের বর্ণনা, অবস্থা আর স্ক্রিনশট একই পাতায়। এতে "কতটা হয়েছে" প্রশ্নের উত্তর এক নজরে।

ব্যবহারবিধি

৩১ পাতার বাংলা ম্যানুয়াল

HTML থেকে PDF — এক উৎস, দুই রূপ

```
docs/manual/
  user-manual-bn.html     # মূল (৮ অধ্যায়)
  print_manual.py         # PDF বানায়

# চালানোর নিয়ম
python docs/manual/print_manual.py
#  -> docs/IMS_User_Manual_BN.pdf  (৩১ পাতা)

# কেন HTML মূল
#  ব্রাউজারে পড়া যায়, PDF-এ ছাপা যায়,
#  ভবিষ্যতে ওয়েব-সহায়তাও একই ফাইল থেকে
```

ভূমিকা ধরে অধ্যায়

অ্যাডমিন · কোঅর্ডিনেটর · শিক্ষক · শিক্ষার্থী · প্লাটফর্ম — প্রত্যেকে নিজের অংশ পড়ে।

প্রশ্ন-উত্তর অংশ আছে

"লগইন হচ্ছে না", "মার্কস সংরক্ষণ হচ্ছে না" — সাপোর্ট কল কমানোর সবচেয়ে সস্তা উপায়।

ফাঁক

UI বদলালে HTML-এর ছবি ও লেখা হাতে বদলাতে হয় — ম্যানুয়ালের কোনো স্বয়ংক্রিয় পরীক্ষা নেই।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন ম্যানুয়ালের একটি অধ্যায় পড়ে দেখতে — তারপর জিজ্ঞেস করুন: "কোথায় আটকে গেলেন?" সেই জায়গাটাই লেখার কাজ।

নিয়ম

স্ক্রিনশট-প্রমাণের পাঁচ শর্ত

ছবি সহজে মিথ্যা বলে — তাই নিয়ম

শর্ত	কারণ
আসল অ্যাপ থেকে, আসল ডেটায়	হাতে বানানো মকআপ একদিনে ধরা পড়ে
তারিখ লেখা থাকুক	"এখনো আছে?" প্রশ্নের উত্তর পাওয়া যায়
এক ছবি, এক দাবি	এক ছবিতে পাঁচ ফিচার দেখালে কিছুই প্রমাণ হয় না
ঝাপসা / কাটা নয়	পড়া না গেলে প্রমাণ নয়, সাজসজ্জা
গোপন তথ্য থাকবে না	ডেমো পাসওয়ার্ড, আসল ছাত্রের নাম-ফোন নয়

IMS-এ ডেটা: `seed_demo` নাম (Rafid Hasan, Sunrise Model School) — কাল্পনিক, তাই ছবি দেখানো নিরাপদ।

সতর্কতা: আসল প্রতিষ্ঠানের ডেটা দিয়ে ডেমো করলে আগে অনুমতি নিন, নইলে স্ক্রিনশটেই গোপনীয়তা ভাঙে।

শিক্ষকের নোট: পঞ্চম শর্তটি বাস্তবে সবচেয়ে বেশি ভাঙা হয় — ক্লাসে একটি পুরোনো স্ক্রিনশটে ডেমো পাসওয়ার্ড দেখা যায় কি না খুঁজতে বলুন।

ডেমো

১০ মিনিটের উপস্থাপনা স্ক্রিপ্ট

পাঁচ ধাপ — প্রতিটির একটি প্রমাণ

মিনিট	কী করবেন	যা প্রমাণ হয়
০–১	লগইন, চার ভূমিকার ড্যাশবোর্ড	এক অ্যাপ, চার রকম পর্দা
১–৩	শিক্ষক: উপস্থিতি নিন → সংশোধন করুন	নীতি + অডিটে জমা
৩–৬	নম্বর এন্ট্রি → প্রকাশ → শিক্ষার্থীর পর্দায়	প্রকাশ-গেট (৪০৩ → ২০০)
৬–৮	রিপোর্ট কার্ড PDF + CSV রফতানি	আসল নথি, আসল ফরম্যাট
৮–১০	অডিট লগ + সাবস্ক্রিপশন ব্যানার	জবাবদিহি + বাণিজ্যিক নিয়ম

নিয়ম: ডেমোতে **নতুন কিছু বানাবেন না** — আগেই তৈরি ডেটা ব্যবহার করুন। আর প্রতিটি ধাপে এক বাক্যে বলুন "এতে কী প্রমাণ হচ্ছে"।

শিক্ষকের নোট: ক্লাসে দুই জনকে ১০ মিনিটের ডেমো করতে দিন — সময় কমিয়ে ৫ মিনিটে নামিয়ে আনতে বলুন; এতে অপ্রয়োজনীয় অংশ বাদ পড়ে।

এক পাতা

গ্রাহকের জন্য এক-পাতার সারসংক্ষেপ

কেউ ২০ পাতার নথি পড়ে না

১. এটি কী

একটি প্রতিষ্ঠানের উপস্থিতি, নম্বর, রিপোর্ট কার্ড ও সিলেবাস — এক জায়গায়।

২. কে কী পাবে

চার ভূমিকা — অ্যাডমিন, কোঅর্ডিনেটর, শিক্ষক, শিক্ষার্থী।

৩. প্রমাণ

১০৩ টেস্ট · ৫৫ পাতা × ৪ ভূমিকা · ৭৭ স্ক্রিনশট · ৩২ পাতার প্রমাণ-নথি।

৪. দাম ও শর্ত

৳১,৫০০ / ৳৩,৫০০ / ৳৬,৫০০ — ১৪ দিনের ট্রায়াল, সিট-সীমা, মেয়াদ শেষে read-only।

৫. যা এখনো নেই

ইনভাইট · CSV আমদানি · অনলাইন পেমেন্ট — সৎভাবে লেখা।

কেন ৫ নম্বর থাকবে: যা নেই তা আগেই বলা থাকলে পরে অভিযোগ আসে না — আর ভরসা বাড়ে।

দৈর্ঘ্য: এক A4 পাতা; তার বেশি হলে কেউ শেষ পর্যন্ত পড়ে না।

শিক্ষকের নোট: শিক্ষার্থীদের এক-পাতার সারসংক্ষেপ লিখিয়ে নিন — সবচেয়ে কঠিন কাজ, কারণ সব বাদ দিয়ে মূল কথাটি রাখতে হয়।

রিপোজিটরি

প্রথম নজরে যা দেখা যায়

কেউ রিপো খুললে ৩০ সেকেন্ডে কী বোঝে

ফাইল	আছে?	কেন দরকার
`README.md`	আছে (বাসি)	প্রথম পরিচয় — হালনাগাদ করা জরুরি
`START_HERE.md`	আছে	চালানোর পথ, দুই বিকল্প
`.gitignore`	আছে	`.env`, `media/`, `backups/`, `__pycache__` বাদ
`.env.example` · `.env.production.example`	আছে	কী কী মান লাগবে, গোপন মান ছাড়া
`Procfile` · `build.sh`	আছে	হোস্ট কীভাবে চালাবে
`docs/`	আছে	প্রমাণ, ম্যানুয়াল, স্ক্রিনশট, জেনারেটর
`CHANGELOG.md` · `CONTRIBUTING.md` · `LICENSE`	**নেই**	সংস্করণ-ইতিহাস, দলের নিয়ম, ব্যবহারের শর্ত

প্রথম চারটি সারি ঠিক থাকলে রিপো "পেশাদার" দেখায়; শেষ সারি বাকি থাকলে দীর্ঘমেয়াদে সমস্যা — বিশেষ করে লাইসেন্স, কারণ ব্যবহারের শর্ত ছাড়া কোড share করা ঝুঁকি।

শিক্ষকের নোট: লাইসেন্স নিয়ে সতর্ক করুন: প্রতিষ্ঠানের কোড কখনো ওপেন রিপোতে দেবেন না; ব্যক্তিগত প্রজেক্টে MIT/Apache-2.0 সাধারণ পছন্দ।

যাচাই

কেউ প্রশ্ন করলে কোন প্রমাণ দেখাবেন

ছয়টি প্রশ্ন, ছয়টি উত্তর — তৈরি রাখুন

প্রশ্ন	প্রমাণ
"কোড কাজ করে?"	`Ran 103 tests in 51.469s · OK`
"পাতাগুলো ভাঙে না?"	`smoke_test.py` — ৫৫ পাতা × ৪ ভূমিকা
"সার্ভারে চলবে?"	`check --deploy` → no issues · `deploy_check.sh`
"ডেটা নিরাপদ?"	স্কোপিং টেস্ট · ৪০৪ ক্রস-ওয়ার্কস্পেস · `AuditLog`
"ভুল হলে?"	রোলব্যাক + `ops/backup.sh` + restore ড্রিল
"আমাদের স্কুলে চলবে?"	২০ মিনিটের অনবোর্ডিং (LAUNCH_PLAYBOOK §4) + রিপোর্ট কার্ড

প্রস্তুতি: এই ছয়টি কমান্ডের আউটপুট এক ফোল্ডারে রেখে দিন (`docs/proof/`) — প্রশ্নের মুহূর্তে খুঁজতে হবে না।

শিক্ষকের নোট: ক্লাসে প্রতিটি কমান্ড একবার চালিয়ে আউটপুট সংরক্ষণ করুন — এটাই "প্রমাণ-ফোল্ডার" বানানোর সবচেয়ে সহজ উপায়।

পর্ব ৮ সমাপ্তি

তিন ক্লাসে যা শেখা হলো

৪৪ স্লাইড · ছয়টি দস্তাবেজ · তিনটি প্রমাণ-নথি

ক্লাস	মূল ধারণা	প্রমাণ
৩৩ · কে পড়বে	চার পাঠক · ব্যবহারবিধি · স্ক্রিনশট	৯০০ লাইন · ৭৭ PNG · ৫৭ ফিচার
৩৪ · কোডের ভেতরে	docstring · "কেন" মন্তব্য · CHANGELOG · ADR	৭৫/৫৭৬ docstring · ৮৫ রুট
৩৫ · প্রমাণ দেখানো	চার স্তরের প্রমাণ · ডেমো · এক-পাতা	৩২+৭০+৩১ পাতা · ১১.৫ MB

সৎ ফাঁক: ADR ফোল্ডার · CHANGELOG · CONTRIBUTING · LICENSE · `docs/url-map.md` · `rebuild_docs.sh` · লোড-টেস্টের প্রমাণ

পর্ব ৯-এ: ভবিষ্যৎ — কোন ফিচার আগে, কীভাবে অনুমান করবেন, আর কোথায় থামবেন।

শিক্ষকের নোট: ফাঁকের তালিকা থেকে প্রত্যেককে দুটি বেছে নিয়ে পরের সপ্তাহে করতে বলুন — ছোট, তবু শেষ হয়।

অনুশীলন ও আলোচনা

২০ মিনিট — প্রমাণ-প্যাক বানান

নিজের প্রজেক্টের জন্য চারটি জিনিস

১. প্রমাণ ফোল্ডার

চারটি কমান্ডের আউটপুট → `docs/proof/`

২. দাবি-ছবি-টেস্ট ছক

পাঁচটি ফিচার → ছবি → টেস্টের নাম

৩. ৫ মিনিটের ডেমো

লিখিত স্ক্রিপ্ট, প্রতিটি ধাপে "কী প্রমাণ হচ্ছে"

৪. এক-পাতার সারসংক্ষেপ

পাঁচটি শিরোনাম, এক A4

- দাবির সঙ্গে কয়টি প্রমাণ?
- স্ক্রিনশটে কী থাকবে না?
- এক-পাতায় কী বাদ দেবেন?
- "চলবে?" প্রশ্নের প্রথম উত্তর?
- লাইসেন্স না থাকলে কী ঝুঁকি?

শিক্ষকের নোট: চতুর্থ প্রশ্নের উত্তর: টেস্টের সংখ্যা ও `smoke_test` — তারপর অনবোর্ডিং। "কথা নয়, আউটপুট।"

৩৬

পর্ব ৯ — ভবিষ্যৎ

কোন ফিচার আগে · অনুমান বনাম প্রমাণ · ব্যবহারকারীর কাছ থেকে শেখা · কখন থামবেন

পর্ব ৯ · ক্লাস ৩৬

পরের ফিচার কোনটি?

ব্যবহারকারীর কাছ থেকে শেখা · অনুমান বনাম প্রমাণ · অগ্রাধিকার · কখন থামবেন

IMS · ক্লাস ৩৬ · স্লাইড ১–১৬

শুরু

আটটি প্রার্থী, একটি দল

সবার ইচ্ছা আলাদা — সিদ্ধান্ত কীসের ভিত্তিতে?

প্রার্থী ফিচার	কে চাইছে	যে প্রশ্নটি আগে
CSV দিয়ে ছাত্র আমদানি	আপনি (অনবোর্ডিং ধীর)	কত স্কুল এতে আটকে আছে?
ইনভাইট ওয়ার্কফ্লো	প্রতিষ্ঠান অ্যাডমিন	এখন পাসওয়ার্ড কীভাবে দেন?
মনিটরিং + `/healthz`	আপনি (নিজের ঘুম)	কোন ব্যবহারকারী সমস্যা জানিয়েছে?
অনলাইন পেমেন্ট (bKash)	আপনি (আয় সহজ হবে)	ম্যানুয়ালে কত সময় যাচ্ছে?
SMS নোটিফিকেশন	শিক্ষক/অভিভাবক	মূল্য ও সম্মতি কে সামলাবে?
মোবাইল অ্যাপ	সবার "মনে হয়"	মোবাইল ওয়েব কি যথেষ্ট নয়?

নিয়ম: **"কে চাইছে" নয়, "কতজন আটকে আছে"** — সবচেয়ে জোরে বলার লোক সবচেয়ে বেশি প্রতিনিধি নয়।

শিক্ষকের নোট: ক্লাসে ভোট নিন: "এই ছয়টির মধ্যে একটি বেছে নিন"। তারপর বলুন — ভোটই ভালো সিদ্ধান্ত নয়; প্রমাণ দরকার।

শেখা

ব্যবহারকারীর কাছ থেকে শেখার তিন উপায়

জিজ্ঞেস করা সবচেয়ে দুর্বল পদ্ধতি

১. কাজ দেখুন (shadowing)

একজন শিক্ষকের পাশে বসে মার্কস এন্ট্রি দেখুন — কোথায় থামেন, কোথায় দ্বিতীয়বার ক্লিক করেন। কথায় যা বলেন না, হাতে দেখা যায়।

২. সাপোর্ট টিকেট গুনুন

একই ধরনের সমস্যা তিনবার এলে সেটি ফিচারের অভাবে নয়, নকশার ত্রুটি। টিকেটে ট্যাগ দিন।

৩. আচরণের সংখ্যা দেখুন

কতজন অনবোর্ডিং শেষ করে, কতজন দ্বিতীয় সপ্তাহেও ঢোকে — অনুরোধ নয়, আচরণই সত্যি।

আরেকটি নিয়ম: **"আপনি এটি ব্যবহার করবেন?"** প্রশ্নের উত্তর সবসময় "হ্যাঁ" — তাই ভবিষ্যৎ নয়, **অতীত** জিজ্ঞেস করুন: "শেষবার এই কাজটি কীভাবে করলেন?"

শিক্ষকের নোট: এই স্লাইডটি কোর্সের অন্যতম ব্যবহারিক অংশ — "ভবিষ্যৎ নয়, অতীত জিজ্ঞেস করুন" বাক্যটি বোর্ডে লিখুন।

প্রমাণ

অনুমান থেকে প্রমাণে

প্রতিটি অনুমানের একটি সস্তা পরীক্ষা আছে

অনুমান	সস্তা পরীক্ষা	প্রমাণ
"CSV আমদানি ছাড়া অনবোর্ডিং হয় না"	এক স্কুলের ছাত্র তালিকা হাতে নিন, সময় মাপুন	৪০ জন ছাত্র = কত মিনিট
"শিক্ষকেরা মোবাইলে মার্কস দিতে চান"	এক সপ্তাহ মোবাইল ব্রাউজার ব্যবহারের সংখ্যা	আসল ব্যবহারের লগ
"পেমেন্ট গেটওয়ে দরকার"	গত মাসে ম্যানুয়াল এন্ট্রিতে কত সময় গেছে	মিনিট × সংখ্যা
"ইনভাইট ছাড়া স্কুল যোগ দিতে পারে না"	এখন কীভাবে পাসওয়ার্ড দেওয়া হয়, তার ধাপ গুনুন	ধাপ-সংখ্যা + টিকেট

নিয়ম: পরীক্ষার খরচ কাজের চেয়ে কম হতে হবে — নইলে সেটি পরীক্ষা নয়, প্রকল্প।

সময়সীমা: প্রতিটি পরীক্ষার আগে ঠিক করুন — কত দিনে, কী দেখলে "হ্যাঁ" ধরবেন।

শিক্ষকের নোট: শিক্ষার্থীদের নিজের প্রজেক্টের একটি অনুমান নিয়ে সস্তা পরীক্ষা ডিজাইন করতে বলুন — ১০ মিনিটেই হয়।

অগ্রাধিকার

তিনটি সংখ্যা দিয়ে স্কোর

প্রভাব × আত্মবিশ্বাস × সহজতা (ICE)

প্রার্থী	প্রভাব	আত্মবিশ্বাস	সহজতা	স্কোর	মন্তব্য
CSV ছাত্র আমদানি	৫	৫	৪	**১০০**	অনবোর্ডিংয়ের আসল বাধা
মনিটরিং + `/healthz`	৪	৫	৫	**১০০**	সস্তা, ঝুঁকি কমায়
ইনভাইট ওয়ার্কফ্লো	৪	৪	৩	**৪৮**	আগে ধাপ গুনে দেখুন
CI পাইপলাইন	৪	৫	৪	**৮০**	প্রতি রিলিজে সময় বাঁচায়
অনলাইন পেমেন্ট	৩	২	২	**১২**	কাগজপত্র ও VAT দরকার
SMS নোটিফিকেশন	৩	২	২	**১২**	খরচ ও সম্মতির প্রশ্ন
মোবাইল অ্যাপ	২	২	১	**৪**	মোবাইল ওয়েব আগে মাপুন

স্কোর = প্রভাব × আত্মবিশ্বাস × সহজতা (প্রতিটি ১–৫)। **এগুলো আমাদের অনুমান**, প্রমাণ নয় — আত্মবিশ্বাসের ঘরটি পূরণ হয় ব্যবহারকারীর কথা শুনে।

শিক্ষকের নোট: সংখ্যাগুলো নিয়ে বিতর্ক করুন — এটাই শেখার অংশ; "কেন আত্মবিশ্বাস ২?" প্রশ্ন করলে অনুমান ধরা পড়ে।

সিদ্ধান্ত

পরের তিনটি কাজ

স্কোর থেকে বাস্তব পরিকল্পনা

ক্রম	কাজ	কেন এই ক্রমে	শেষ হলে প্রমাণ
১	CSV ছাত্র আমদানি	অনবোর্ডিংয়ের সবচেয়ে ধীর ধাপ সরাসরি হ্রাস পায়	CSV ফাইল দিয়ে টেস্ট + স্ক্রিনশট
২	মনিটরিং: `/healthz` + অ্যালার্ট + CI	এটি ছাড়া ১ নম্বরের কাজও নীরবে ভাঙতে পারে	অ্যালার্ট এসেছে, CI সবুজ
৩	ইনভাইট ওয়ার্কফ্লো	আগে ধাপ গুনুন; সম্ভবত ১ ও ২-এর পর সহজ লাগবে	একজন অ্যাডমিন নিজে শিক্ষক যোগ করেন

তিন মাসের ছক: মাস ১ = CSV · মাস ২ = মনিটরিং ও CI · মাস ৩ = ইনভাইট + academics টেস্ট

যা এখনই নয়: পেমেন্ট গেটওয়ে, SMS, মোবাইল অ্যাপ — অনুমান এখনো প্রমাণে বদলায়নি।

শিক্ষকের নোট: বলুন: "একসাথে তিনটি কাজ নয়" — একটির পর একটি, প্রতিটির পরে ব্যবহারকারীর সঙ্গে যাচাই।

কম করুন

প্রতিটি ফিচারের একটি বার্ষিক দাম

যোগ করা সহজ, রাখা কঠিন

ফিচারের খরচ তিন রকম

বানানোর খরচ — একবার। **রক্ষণাবেক্ষণের খরচ** — প্রতি রিলিজে। **ব্যাখ্যার খরচ** — প্রতিটি নতুন ব্যবহারকারী ও নতুন ডেভেলপারের জন্য।

IMS-এর উদাহরণ

৫৭ ফিচারের ক্যাটালগ, ৮৫ রুট, ৭৭ স্ক্রিনশট — প্রতিটি নতুন ফিচার এই সংখ্যাগুলো বাড়ায়, আর দস্তাবেজও বাড়াতে হয়।

প্রশ্ন যা সবসময় করতে হবে

"এই ফিচারটি সরালে কতজন কষ্ট পাবে?" উত্তর যদি "কেউ না" হয় — তবে সেটি যোগও করবেন না।

অভ্যাস: প্রতি ছয় মাসে তালিকা দেখুন — কোন ফিচার ব্যবহারই হচ্ছে না। সেটি সরানোর প্রস্তাব লিখুন।

শিক্ষকের নোট: শিক্ষার্থীরা প্রায়ই ভাবে বেশি ফিচার = ভালো প্রোডাক্ট। এখানেই সেই ধারণা ভাঙুন — রক্ষণাবেক্ষণের হিসাব দেখান।

যা বানাবেন না

পাঁচটি ফাঁদ — এখনই নয়

প্রতিটির পেছনে একটি করে যুক্তি

ফিচার	কেন এখন নয়
নেটিভ মোবাইল অ্যাপ	দুই প্ল্যাটফর্ম, দুই রিলিজ, দুই ডিবাগ — মোবাইল ওয়েবের ব্যবহার মাপার আগে নয়
SMS নোটিফিকেশন	প্রতি এসএমএসে খরচ, সম্মতি ও অপব্যবহারের ঝুঁকি; আগে দেখুন কতজন চায়
AI দিয়ে নম্বর / খাতা মূল্যায়ন	ভুল হলে অভিভাবককে ব্যাখ্যা দিতে হবে; শিক্ষকের সিদ্ধান্তই এখানে আসল
রিয়েল-টাইম চ্যাট	সম্পূর্ণ আলাদা কাঠামো (websocket, উপস্থিতি) — বর্তমান সমস্যার সমাধান নয়
মাল্টি-কান্ট্রি / মাল্টি-কারেন্সি	এক দেশের প্রয়োজন এখনো পূরণ হয়নি; পরে যোগ করলে ভাঙে না

এই তালিকা মানে "কখনো নয়" নয় — "**প্রমাণ ছাড়া নয়**"। প্রতিটির জন্য একটি শর্ত লেখা থাকা উচিত: "কতজন চাইলে, কোন প্রমাণ পেলে"।

শিক্ষকের নোট: শিক্ষার্থীদের নিজের প্রজেক্টের "লোভের তালিকা" লিখতে বলুন — তারপর প্রতিটির শর্ত ঠিক করতে বলুন।

প্রযুক্তি

প্রযুক্তিগত বাজি — কখন যোগ করবেন

প্রতিটির একটি ট্রিগার থাকা দরকার

প্রযুক্তি	কখন দরকার	আজকের অবস্থা
অবজেক্ট স্টোরেজ (S3-জাতীয়)	একাধিক সার্ভার বা বড় ফাইল	লোকাল `media/` — এক সার্ভারে যথেষ্ট
ব্যাকগ্রাউন্ড কাজ (Celery/queue)	ধীর কাজ: বড় PDF, ইমেইল ব্যাচ	সব অনুরোধেই — এখনো ধীর হয়নি
Redis / ক্যাশ	একই কোয়েরি বারবার, বা রেট লিমিট	নেই — তবে রেট লিমিটের জন্য লাগবে
আলাদা ডেটাবেস রেপ্লিকা	পড়ার চাপ লেখার চাপ ছাড়ালে	`CONN_MAX_AGE 60` এখন যথেষ্ট
CDN	ভিন্ন শহর/দেশে ব্যবহারকারী	WhiteNoise + ৫টি স্ট্যাটিক ফাইল

নিয়ম: **প্রযুক্তি আগে নয়, সমস্যা আগে** — "হয়তো লাগবে" ভেবে যোগ করলে জটিলতা বাড়ে, গতি কমে।

শিক্ষকের নোট: প্রশ্ন করুন: "আজকের সেটআপে সবচেয়ে আগে কী ভাঙবে?" উত্তর সম্ভবত ডেটা-বৃদ্ধি বা ব্যাকআপ — সেই অনুযায়ী বিনিয়োগ করুন।

মাপা

পাঁচটি সংখ্যা যা বলে দেয় সফল কি না

আজ IMS-এ কোনটি মাপা যায়, কোনটি যায় না



সংখ্যা	আজ মাপা যায়?	কীভাবে
সাইনআপ → অনবোর্ডিং	হ্যাঁ, হাতে	প্রতিটি প্রতিষ্ঠান দেখে নোট রাখুন
সাপ্তাহিক সক্রিয় প্রতিষ্ঠান	আংশিক	`AuditLog` বা `Mark.created_at` গুনে
ত্রুটির হার (৫০০)	না	হোস্টের লগ থেকে গুনতে হবে
পাঠানো/বাতিল (churn)	হ্যাঁ	`billing_report`
সাপোর্ট টিকেটের ধরন	না	ট্যাগিং চালু করতে হবে

শিক্ষকের নোট: বলুন: "মাপা যায় না — মানে জানা যায় না।" প্রথম কাজ: যে তিনটি সংখ্যা এখনই হাতে মাপা যায়, সেগুলো প্রতি সপ্তাহে লিখে রাখা।

ফিডব্যাক

ব্যবহারকারীর কথা ফেরানোর লুপ

অ্যাপে ফিডব্যাক বোতাম নেই — সস্তা তিনটি পথ

১. মাসিক ১০ মিনিটের কল

প্রতি মাসে একজন শিক্ষক ও একজন অ্যাডমিনের সঙ্গে — তিনটি প্রশ্ন, উত্তর লিখে রাখুন।

২. টিকেটে ট্যাগ

`bug` · `how-to` · `missing-feature` · `slow` — মাস শেষে গুনুন, ধরনই রোডম্যাপ।

৩. রিলিজের পরে এক প্রশ্ন

"নতুন যা এল, সেটি কাজে লেগেছে?" — উত্তর না এলে ফিচারটি যোগ করা বৃথা।

```
মাসিক ফিডব্যাক ছক

১. এই মাসে কোন কাজটি
   সবচেয়ে বেশি সময় নিয়েছে?
২. কোন পর্দায় আটকে গেছেন?
৩. একটি জিনিস বদলাতে পারলে
   কী বদলাতেন?

→ উত্তর তিনটি লাইনে লিখুন
→ তিন মাস পর আবার একই প্রশ্ন
```

ফাঁদ: যিনি সবচেয়ে বেশি ফোন করেন, তার চাহিদাই রোডম্যাপ হয়ে যায়। সংখ্যা দিয়ে যাচাই করুন — কতজন একই কথা বলছে।

শিক্ষকের নোট: ক্লাসে দুই জন শিক্ষার্থীকে ভূমিকা দিন — একজন শিক্ষক, একজন ডেভেলপার; তিন প্রশ্নের রোলপ্লে করান।

থামা

কখন থামবেন, কখন ফিচার বন্ধ করবেন

শেষ করাও প্রকৌশলের অংশ

পরিস্থিতি	সিদ্ধান্ত
একটি ফিচার ৬ মাসে কেউ ব্যবহার করেনি	বন্ধ করার প্রস্তাব লিখুন (তিন ধাপ: ঘোষণা → ব্যানার → সরানো)
একটি প্রতিষ্ঠান ৩ মাস ট্রায়ালেই আছে, অনবোর্ডিং শেষ হয়নি	ফিচার নয়, অনবোর্ডিং ঠিক করুন — তার সঙ্গে কথা বলুন
সাপোর্টের ৬০% একটি পর্দার সমস্যা	নতুন ফিচার বন্ধ, ওই পর্দা নতুন করে ভাবুন
ব্যক্তিগত প্রজেক্ট: ৩ মাস কিছুই হয়নি	ছোট করে শেষ করুন — README + প্রমাণ-নথি + আর্কাইভ
রক্ষণাবেক্ষণের সময় নেই	বন্ধ করুন বা হাতবদল করুন — নিরাপত্তা আপডেট ছাড়া চালু রাখা বিপজ্জনক

শেষ করার নিয়ম: সবসময় ব্যাকআপ → ব্যবহারকারীকে জানানো → ডেটা রপ্তানির পথ খোলা রাখা। ভরসা ভাঙা সবচেয়ে বড় ক্ষতি।

শিক্ষকের নোট: শিক্ষার্থীদের জন্য কঠিন সত্য: "শেষ না করা প্রজেক্ট শেখায় না" — শেষ করার অভ্যাসই পোর্টফোলিও বানায়।

দীর্ঘমেয়াদ

দূরের তিনটি দিক

আজ না করলেও, নকশায় জায়গা রাখুন

বাংলা ইন্টারফেস

এখন পর্দা ইংরেজি, ম্যানুয়াল বাংলা। Django-র `gettext` দিয়ে পরে অনুবাদযোগ্য — তবে **শব্দগুলো** এখন থেকেই স্পষ্ট রাখুন, পরে অনুবাদ সহজ হবে।

অফলাইন উপস্থিতি

শ্রেণিকক্ষে নেট দুর্বল। PWA/সার্ভিস-ওয়ার্কার দিয়ে "নেট এলে জমা হবে" — কাঠামো বদলাবে, তাই আগে থেকেই ভাবা ভালো।

একাধিক প্রতিষ্ঠান, এক ব্যবহারকারী

এক শিক্ষক দুই স্কুলে পড়ান — IMS-এ ওয়ার্কস্পেস সুইচ আছে, কিন্তু স্কোপিং নিয়ম প্রতিটি নতুন ফিচারে আবার যাচাই করতে হবে।

নকশার নিয়ম: **ভাষা, সময় অঞ্চল ও নেট-অনুপস্থিতি** — এই তিনটি শুরুতে ভাবলে পরে বদলাতে হয় না। IMS-এ `TIME_ZONE = "Asia/Dhaka"` আর `USE_TZ = True` আগেই ঠিক করা।

শিক্ষকের নোট: বলুন: "যে সিদ্ধান্ত পরে বদলানো কঠিন, সেটি শুরুতে ঠিক করুন।" — সময় অঞ্চল, মুদ্রা, ভাষা, আইডি-কাঠামো।

অনুশীলন ও আলোচনা

২০ মিনিট — নিজের রোডম্যাপ লিখুন

চারটি কাজ, এক পাতা

১. ছয়টি প্রার্থী
নিজের প্রজেক্টের ছয়টি ইচ্ছা লিখুন, "কে চাইছে" কলামসহ।

২. ICE স্কোর
প্রভাব · আত্মবিশ্বাস · সহজতা (১–৫) — গুণফল লিখুন।

৩. তিনটি সস্তা পরীক্ষা
শীর্ষ তিনটি অনুমানের জন্য: কী দেখলে "হ্যাঁ" ধরবেন, কত দিনে।

৪. "যা বানাব না" তালিকা
তিনটি, প্রতিটির শর্তসহ ("কতজন চাইলে")।

- ভবিষ্যৎ নয়, অতীত কেন জিজ্ঞেস করবেন?
- ICE-এর কোন সংখ্যাটি সবচেয়ে অনুমাননির্ভর?
- ফিচারের রক্ষণাবেক্ষণ-খরচ কী?
- কখন একটি ফিচার বন্ধ করবেন?
- কোন সিদ্ধান্ত পরে বদলানো কঠিন?

শিক্ষকের নোট: দ্বিতীয় প্রশ্নের উত্তর: "আত্মবিশ্বাস" — কারণ সেটি প্রমাণ ছাড়া সংখ্যা। তাই সেটি বাড়ানোর কাজ আগে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

ভবিষ্যৎ পরিকল্পনার সারাংশ

নীতি	প্রমাণ / উদাহরণ
"কে চাইছে" নয়, "কতজন আটকে আছে"	CSV আমদানি vs মোবাইল অ্যাপ
ভবিষ্যৎ নয়, অতীত জিজ্ঞেস করুন	"শেষবার কীভাবে করলেন?"
প্রতিটি অনুমানের সস্তা পরীক্ষা আছে	৪০ জন ছাত্র = কত মিনিট
প্রভাব × আত্মবিশ্বাস × সহজতা	ICE স্কোর: ১০০ · ৮০ · ৪৮ · ১২ · ৪
প্রতিটি ফিচারের বার্ষিক দাম	৫৭ ফিচার · ৮৫ রুট · ৭৭ স্ক্রিনশট
শেষ করাও প্রকৌশল	বন্ধ করার তিন ধাপ + ব্যাকআপ

পরের পর্ব (১০): পুরো যাত্রার উপস্থাপন — ৩৭ ক্লাসে যা শেখা হলো, তা এক সেশনে কীভাবে দেখাবেন।

শিক্ষকের নোট: এই ছয়টি সারি শিক্ষার্থীদের নিজের ভাষায় লিখিয়ে নিন — ভবিষ্যতে সিদ্ধান্ত নেওয়ার সময় এগুলোই কাজে লাগবে।

৩৭

পর্ব ১০ — উপস্থাপন

পুরো যাত্রার সারসংক্ষেপ · ডেমো · প্রশ্নের উত্তর · পোর্টফোলিও

পর্ব ১০ · ক্লাস ৩৭

৩৭ ক্লাসের গল্প

উপস্থাপন · ১৫ মিনিটের ডেমো · প্রশ্নোত্তর · পোর্টফোলিও · যাচাই

IMS · ক্লাস ৩৭ · স্লাইড ১–১৬ · শেষ ক্লাস

শুরু

১৫ মিনিটে তিনটি প্রশ্নের উত্তর

শ্রোতা এই তিনটির উত্তর খুঁজছেন

১. এটি কী, কাকে কাজে লাগবে?

এক বাক্যে: "একটি প্রতিষ্ঠানের উপস্থিতি, নম্বর, রিপোর্ট কার্ড ও সিলেবাস — চার ভূমিকায়, এক জায়গায়।"

২. সত্যিই তৈরি হয়েছে?

দাবি নয়, প্রমাণ: ১০৩ টেস্ট · ৫৫ পাতা × ৪ ভূমিকা · ৭৭ স্ক্রিনশট · ৩২ পাতার প্রমাণ-নথি।

৩. আপনি কী শিখেছেন?

সিদ্ধান্তের কারণ বলতে পারা — "কেন middleware", "কেন ৪০৪ নয় ৪০৩", "কেন মাইগ্রেশন দুই ধাপে"।

তৃতীয় প্রশ্নটিই আসল পার্থক্য তৈরি করে — প্রজেক্ট সবাই দেখাতে পারে; **সিদ্ধান্তের ব্যাখ্যা** কেবল সে-ই দিতে পারে যে নিজে ভেবেছে।

শিক্ষকের নোট: ক্লাসে প্রতি শিক্ষার্থীকে এক বাক্যে প্রথম প্রশ্নের উত্তর লিখতে বলুন — ৩০ সেকেন্ডে। যা লিখবে, সেটাই তার উপস্থাপনার প্রথম বাক্য।

গল্প

দশ পর্বের গল্প-রেখা

প্রতিটি পর্ব আগেরটির একটি প্রশ্নের উত্তর

১. কনসেপ্ট
কী বানাচ্ছি, কেন
৪ ক্লাস · ৫৮ স্লাইড

২. প্ল্যান
কোন ক্রমে, কত সময়ে
৬ ক্লাস · ১০৬ স্লাইড

৩. ডিজাইন
ডেটা, স্ক্রিন, নিরাপত্তা
৫ ক্লাস · ৮৩ স্লাইড

৪. কোড
৩১ মডেল, ৮৫ রুট
৮ ক্লাস · ১৩০ স্লাইড

৫. টেস্টিং
প্রমাণ কোথায়
৪ ক্লাস · ৭০ স্লাইড

৬. ডিপ্লয়
সার্ভারে তোলা ও চালু রাখা
৩ ক্লাস · ৫৬ স্লাইড

৭. রক্ষণাবেক্ষণ
কোড বড় হলে
২ ক্লাস · ৩৪ স্লাইড

৮. দস্তাবেজ
কে পড়বে, কী প্রমাণ
৩ ক্লাস · ৪৪ স্লাইড

৯. ভবিষ্যৎ
পরের ফিচার কোনটি
১ ক্লাস · ১৬ স্লাইড

১০. উপস্থাপন
এই ক্লাস
১ ক্লাস · ১৬ স্লাইড

শিক্ষকের নোট: এই ছবিটি উপস্থাপনার দ্বিতীয় স্লাইড হিসেবে ব্যবহার করুন — শ্রোতা পুরো যাত্রার মানচিত্র এক নজরে পায়।

সংখ্যা

যা তৈরি হয়েছে, সংখ্যায়

প্রতিটি সংখ্যার পাশে কোথায় যাচাই করবেন

সংখ্যা	মান	যাচাই
ক্লাস	৩৭	১০ পর্ব · এই কিট
স্লাইড (মূল কিট)	৬২৯	`part1` … `part10` PDF
কোড	৫,৪৮০ লাইন	`find apps -name "*.py" | xargs wc -l`
মডেল · রুট · টেমপ্লেট	৩১ · ৮৫ · ৭৫	`models.py` · `urls.py` · `templates/`
টেস্ট	১০৩	`manage.py test` → OK
স্ক্রিনশট	৭৭	`docs/screenshots/`

উপস্থাপনায় তিনটি সংখ্যাই যথেষ্ট: **৩৭ ক্লাস · ১০৩ টেস্ট · ৮৫ রুট**। বাকিগুলো প্রশ্নের উত্তরে।

শিক্ষকের নোট: বেশি সংখ্যা বললে শ্রোতা সন্দেহ করে; কম বললে ছোট দেখায়। তাই তিনটি বেছে নিয়ে মুখস্থ করুন।

ডেমো

১৫ মিনিটের পাঁচ ব্লক

প্রতিটি ব্লকে একটি করে প্রমাণ

মিনিট	কী দেখাবেন	যা বলবেন
০–২	লগইন → চার ভূমিকার ড্যাশবোর্ড	"এক অ্যাপ, চার ভিন্ন পর্দা — একই ডেটা, ভিন্ন স্কোপ"
২–৬	শিক্ষক: উপস্থিতি নিন → সংশোধন করুন	"সংশোধন অডিটে জমা হয় — কে, কখন, আগে-পরে"
৬–১০	নম্বর এন্ট্রি → প্রকাশ → শিক্ষার্থীর পর্দা	"প্রকাশের আগে ৪০৩, পরে ২০০ — টেস্টেই লেখা"
১০–১৩	রিপোর্ট কার্ড PDF + CSV রফতানি	"আসল নথি, আসল ফরম্যাট — স্ক্রিনশট নয়"
১৩–১৫	অডিট লগ → সাবস্ক্রিপশন ব্যান্ড	"জবাবদিহি আর বাণিজ্যিক নিয়ম — দুটোই কোডে"

প্রস্তুতি: `bash run.sh --reset` — পরিষ্কার ডেমো ডেটা; ব্রাউজারে শুধু প্রাসঙ্গিক ট্যাব খোলা।

নিয়ম: লাইভ কোড লিখবেন না — ১৫ মিনিটে কিছু ভাঙলে পুরো উপস্থাপনা ভাঙে।

শিক্ষকের নোট: প্রতিটি ব্লক দুইবার প্র্যাকটিস করান — সময় ধরে। বেশিরভাগ শিক্ষার্থী প্রথমবার সময়ের চেয়ে ৪০% বেশি খরচ করে।

ঝুঁকি

লাইভ ডেমো ভাঙলে কী করবেন

প্রতিটি ঝুঁকির একটি বিকল্প তৈরি রাখুন

ঝুঁকি	লক্ষণ	বিকল্প
ইন্টারনেট নেই	পাতা খোলে না (ক্লাউডে)	লোকাল `run.sh` — SQLite, নেট ছাড়াই চলে
ডেটাবেস নষ্ট	৫০০ / খালি তালিকা	আগের ব্যাকআপ + `run.sh --reset`
প্রজেক্টর/রং সমস্যা	লেখা দেখা যায় না	স্ক্রিনশট-প্যাক (৭৭ PNG) দিয়ে বলা
সময় শেষ	১০ মিনিটে ৩ ব্লক বাকি	১৩–১৫ নম্বর ব্লক বাদ দিন, মূল তিনটি রাখুন
প্রশ্ন এসে থামিয়ে দিল	প্রবাহ ভাঙে	"শেষে ফিরে আসছি" — লিখে রাখুন

সবচেয়ে ভালো বীমা: **৩ মিনিটের রেকর্ড করা ভিডিও** — ইন্টারনেট বা সার্ভার যাই হোক, দেখানোর মতো কিছু থাকবে।

শিক্ষকের নোট: ক্লাসেই একবার রেকর্ড করে রাখতে বলুন (ফোনে স্ক্রিন রেকর্ডিংই যথেষ্ট) — এটি পোর্টফোলিওতেও কাজে লাগে।

প্রশ্নোত্তর

আটটি কঠিন প্রশ্ন — তৈরি উত্তর

সৎ উত্তরই সবচেয়ে শক্ত উত্তর

প্রশ্ন	উত্তর
"এটা কি স্কুলে ব্যবহার হচ্ছে?"	"না — রিহার্সাল হয়েছে, পাবলিক হোস্টে ডিপ্লয় হয়নি; DEPLOYMENT.md-এ সেটাই লেখা"
"ডেটা নিরাপদ?"	"স্কোপিং এক জায়গায় (`scoping.py`), ক্রস-ওয়ার্কস্পেসে ৪০৪, ভুল ভূমিকায় ৪০৩ — টেস্টে প্রমাণ আছে"
"কেন Django, Laravel/Node নয়?"	"টিমের দক্ষতা ও রক্ষণাবেক্ষণের ইতিহাস; সিদ্ধান্তটি ADR-এ লেখা"
"একসাথে ১০০০ ব্যবহারকারী চালাবে?"	"মাপা হয়নি — লোড-টেস্ট বাকি; এটাই এখন সবচেয়ে বড় অজানা"
"পেমেন্ট আছে?"	"প্ল্যান, ট্রায়াল ও ইনভয়েস আছে; অনলাইন গেটওয়ে নেই — ম্যানুয়াল এন্ট্রি, ইচ্ছাকৃতভাবে"
"টেস্ট কতটা?"	"১০৩ টেস্ট, ৫১.৫ সেকেন্ড; নেগেটিভ টেস্ট কম, সেটি জানি"
"কোড কে লিখেছে?"	"আমি, ১০ পর্বে — প্রতিটি সিদ্ধান্তের ব্যাখ্যা দিতে পারব"
"কী বদলাবেন?"	"CSV আমদানি, মনিটরিং ও CI — ICE স্কোরে শীর্ষে"

শিক্ষকের নোট: প্রতিটি প্রশ্নের উত্তর **এক বাক্যে** — বেশি বললে দুর্বলতা ঢাকার চেষ্টা মনে হয়।

সততা

"যা নেই" কীভাবে বলবেন

তিন বাক্যের ছক

```
প্রথম বাক্য — কী নেই
"ইনভাইট, CSV আমদানি আর অনলাইন পেমেন্ট
এখনো নেই।"

দ্বিতীয় বাক্য — কেন
"অগ্রাধিকার তালিকায় ছিল, কিন্তু আগে
ডেটা-নিরাপত্তা ও ব্যাকআপ সেরেছি।"

তৃতীয় বাক্য — কখন/কীভাবে
"CSV প্রথম, তারপর মনিটরিং ও CI —
তিন মাসের ছকে লেখা আছে।"
```

কেন কাজ করে

"নেই" বললে ভরসা কমে না; "নেই এবং জানি না কেন" বললে কমে।

যা বলবেন না

"শীঘ্রই আসছে" · "প্রায় শেষ" · "ওটা তো দরকারই নেই" — তিনটিই শ্রোতা বুঝে ফেলে।

লিখিত প্রমাণ

`BUILD_STATUS.md`-এর "Remaining work" — পর্দায় দেখালে আর প্রশ্ন থাকে না।

শিক্ষকের নোট: ক্লাসে প্রত্যেককে নিজের প্রজেক্টের তিন বাক্য লিখতে বলুন — এটাই সবচেয়ে বেশি কাজে লাগা অনুশীলন।

পোর্টফোলিও

যা জমা রাখবেন

পাঁচটি জিনিস — এক ফোল্ডারে

জিনিস	IMS-এ কোথায়	কেন রাখবেন
কোড রিপোজিটরি	১৯ কমিট · ১১ অ্যাপ	কেউ নিজে চালিয়ে দেখতে পারে
প্রমাণ-নথি	`docs/Build_Proof.pdf` (৩২ পাতা)	দাবির প্রমাণ এক ফাইলে
ক্যাটালগ	`IMS_Feature_Catalogue.pdf` (৭০ পাতা)	কী আছে, কী নেই — এক নজরে
ম্যানুয়াল	`IMS_User_Manual_BN.pdf` (৩১ পাতা)	ব্যবহারকারীর কথা ভেবেছেন বোঝায়
ডেমো ভিডিও	৩ মিনিটের স্ক্রিন রেকর্ডিং	কেউ ৩ মিনিটে বুঝে নিতে পারে

যোগ করুন: `README.md` (হালনাগাদ), `screenshots/` (৭৭), এবং এক-পাতার সারসংক্ষেপ — এই আটটি মিলে একটি সম্পূর্ণ প্যাকেজ।

শিক্ষকের নোট: বলুন: "পোর্টফোলিও মানে ফাইল-তালিকা নয় — কেউ যেন ১০ মিনিটে আপনার কাজ বুঝতে পারে।"

লেখা

রিপো ও সিভিতে এক-পাতার বর্ণনা

প্রতিটি বুলেটে একটি সংখ্যা

```
IMS — Institute Management System
Django 6.1 · PostgreSQL · ১১ অ্যাপ · ৩৭ ক্লাস

• ৪ ভূমিকার স্কোপিং এক জায়গায়
  (scoping.py): ক্রস-ওয়ার্কস্পেস ৪০৪,
  ভুল ভূমিকায় ৪০৩ — ১০৩ টেস্টে প্রমাণিত
• মার্কস প্রকাশের গেট: প্রকাশের আগে
  শিক্ষার্থী ৪০৩, পরে ২০০
• রিপোর্ট কার্ড PDF, CSV ও XLSX রপ্তানি
• সাবস্ক্রিপশন: প্ল্যান, ট্রায়াল, ইনভয়েস;
  মেয়াদ শেষে read-only, ডেটা অটুট
• ডেমো: bash run.sh → ৪ লগইন, ৫৫ পাতা
```

নিয়ম ১

বিশেষণ নয়, সংখ্যা — "শক্তিশালী নিরাপত্তা" নয়, "৪০৪ + ৪০৩ টেস্ট"।

নিয়ম ২

ছয়টি বুলেটের বেশি নয় — recruiter ২০ সেকেন্ড দেয়।

নিয়ম ৩

প্রতিটি বুলেটের পাশে প্রমাণ কোথায় আছে জানুন — প্রশ্ন এলে সাথে সাথে দেখাতে পারবেন।

শিক্ষকের নোট: শিক্ষার্থীদের নিজের প্রজেক্টের ছয়টি বুলেট লিখিয়ে নিন, তারপর জোড়ায় বদলে "এটি সত্যি?" প্রশ্ন করান।

যাচাই

শেষ পরীক্ষা — কাউকে শেখান

যা ব্যাখ্যা করতে পারেন, তা-ই জানেন

১০ মিনিটে একটি ধারণা

সহপাঠীকে "কেন সাবস্ক্রিপশন গেট middleware-এ" বোঝান — প্রশ্ন করলে আটকে যাবেন, সেটাই শেখার জায়গা।

সে চালাতে পারে কি না

START_HERE দেখে সে কি নিজের মেশিনে চালাতে পারল? যেখানে আটকাল, সেখানেই দস্তাবেজের ফাঁক।

একটি বাগ ধরতে দাও

তাকে একটি ছোট বাগ ঠিক করতে দিন — টেস্ট লিখে, কমিট করে। পারলে আপনার কোড সত্যিই বোঝা যাচ্ছে।

উপস্থাপনার শেষেও এই পরীক্ষাটি করুন: **"আপনি যা বললেন, তা অন্য একজন আরেকজনকে বলতে পারবে?"** — না পারলে ব্যাখ্যাটি অসম্পূর্ণ।

শিক্ষকের নোট: এই স্লাইডটি কোর্সের মূল্যায়নের সবচেয়ে ভালো রূপ — নিজে জানা নয়, অন্যকে জানানো।

দল

দলে উপস্থাপনা — কে কী বলবে

তিন জন, তিন ভূমিকা, এক গল্প

ভূমিকা	কী বলবে	কী প্রস্তুত রাখবে
পরিচিতি ও সমস্যা	"কী বানাচ্ছি, কাকে কাজে লাগবে" — ৩ মিনিট	প্রথম স্লাইড + এক বাক্যের বর্ণনা
ডেমো	"এটি কীভাবে চলে" — ৭ মিনিট	পরিষ্কার ডেটা, খোলা ট্যাব
প্রমাণ ও ভবিষ্যৎ	"সত্যিই তৈরি, আর এরপর কী" — ৫ মিনিট	PROOF PDF + তিন মাসের ছক

হাতবদলের নিয়ম: একজনের শেষ বাক্যেই পরের জন শুরু করবে — "আর এই ডেটাই পরের অংশ" ধরনের সেতু।

প্রশ্ন কার কাছে: আগেই ভাগ করুন — ডেটা/নিরাপত্তা একজনের, ফিচার/ভবিষ্যৎ অন্যজনের।

শিক্ষকের নোট: দল হিসেবে উপস্থাপনা হলে একবার পুরো রিহার্সাল করান — সেতু বাক্যগুলোই সবচেয়ে বেশি ভাঙে।

এরপর

পাঁচটি ডিপ ডেক — আরও গভীরে

এই কিটের পরের অংশ

ডেক	স্লাইড	বিষয়
D1 ডেটাবেস	৩০	৩১ মডেল · কী ইনডেক্স · কনকারেন্সি · ব্যাকআপ কৌশল
D2 স্ট্যাক ও সংযোগ	২৮	অনুরোধের পথ · middleware ক্রম · WSGI · টেমপ্লেট ইনহেরিটেন্স
D3 নিরাপত্তা	২২	স্কোপিং · CSRF · সেশন · স্পেসিফিক আক্রমণ ও প্রতিরোধ
D4 লাইফসাইকেল ও অপস	২০	ট্রায়াল → গ্রাহক → সাসপেন্ড → ডেটা নীতি
D5 ডিজাইন সিস্টেম	১৮	রং · টাইপ · স্পেসিং · কম্পোনেন্ট · অ্যাক্সেসিবিলিটি

ডিপ ডেক পড়ার নিয়ম: **প্রশ্ন নিয়ে পড়ুন** — "কেন এই ইনডেক্স?" "কেন এই middleware ক্রম?" উত্তর খুঁজতে খুঁজতে পড়লে মনে থাকে।

শিক্ষকের নোট: এখানেই মূল ৩৭ ক্লাস শেষ; ডিপ ডেকগুলো ঐচ্ছিক গভীরতা — যারা আরও এগোতে চায় তাদের জন্য।

অনুশীলন ও আলোচনা

২০ মিনিট — নিজের উপস্থাপনা তৈরি করুন

চারটি ছোট ফলাফল

১. এক বাক্যের পরিচয়

"...একটি ... যা ... করে" — ৩০ সেকেন্ডে বলা যায় এমন।

২. পাঁচ মিনিটের ডেমো ছক

পাঁচ ব্লক, প্রতি ব্লকে "কী প্রমাণ হচ্ছে"।

৩. তিন বাক্যে "যা নেই"

নেই · কেন · কখন।

৪. ছয়টি বুলেট

প্রতিটিতে একটি সংখ্যা, প্রতিটির প্রমাণ জানা।

- প্রথম ৩০ সেকেন্ডে কী বলবেন?
- কোন সংখ্যা তিনটি মুখস্থ?
- ডেমো ভাঙলে বিকল্প কী?
- কোন প্রশ্নে "জানি না" বলবেন?
- সবচেয়ে কঠিন প্রশ্ন কোনটি?

শিক্ষকের নোট: শেষ প্রশ্নের ভালো উত্তর: "লোড-টেস্ট হয়নি" — কারণ সেটি সত্য, এবং জানা সীমার কথা বলা সাহসের পরিচয়।

সমাপ্তি

৩৭ ক্লাসে যা হয়ে গেল

একটি ধারণা থেকে একটি চালু-যোগ্য সিস্টেম

যা শিখলেন	কোথায় দেখা যায়	মনে রাখার কথা
কাঠামো ভাবা	৩১ মডেল · ১১ অ্যাপ	"এক জায়গায় এক নিয়ম"
নিরাপত্তা নকশায়	`scoping.py` · middleware গেট	"ভুল উত্তর নয়, উত্তর নেই" (৪০৪)
প্রমাণ দিয়ে বলা	১০৩ টেস্ট · ৭৭ স্ক্রিনশট	"সবুজ নয়, প্রমাণ"
সার্ভারে তোলা	prod সেটিংস · gunicorn · ব্যাকআপ	"যা মাপা হয় না, রক্ষা করা যায় না"
টিকিয়ে রাখা	দস্তাবেজ · ADR · রোডম্যাপ	"ভবিষ্যতের আপনি অচেনা মানুষ"

শেষ কথা: সবচেয়ে বড় অর্জন স্লাইড বা কোড নয় — এখন আপনি **সিদ্ধান্তের কারণ** বলতে পারেন। সেটাই একজন প্রকৌশলী আর একজন কপি-পেস্টার-এর পার্থক্য।

শিক্ষকের নোট: ক্লাস শেষে শিক্ষার্থীদের বলুন: "এখন আপনার প্রজেক্টে ফিরে যান, একটি ADR লিখুন, আর একজনকে চালিয়ে দেখান।" এটাই কোর্সের আসল শেষ।

D1

গভীর ডেক — ডেটাবেস ডিজাইন

৩১ মডেল · ফিল্ড-নির্বাচন · ইনডেক্স · কনকারেন্সি · ব্যাকআপ কৌশল

গভীর ডেক · D1

ডেটাবেস ডিজাইন

৩১ টেবিল · ৩০১ কলাম · ৭৪ সম্পর্ক · ২৩ মাইগ্রেশন-এ না, ১১টিতে — স্কুলের কাগজ কীভাবে টেবিল হলো

মানচিত্র

এই ৩০ স্লাইডে কী আছে

চারটি ব্লক — প্রতিটি ব্লকের শেষে একটি সংখ্যা প্রমাণ হিসেবে

১. আকার ও নাম

স্লাইড ৩–৫ · ১১ অ্যাপ · ৩১ মডেল · ৩০১ কলাম · নামকরণের ৪ নিয়ম

প্রমাণ: ইন্ট্রোস্পেকশন আউটপুট

২. সীমানা ও সম্পর্ক

স্লাইড ৬–১১ · workspace দেয়াল · ইউজার মডেল · academ কশিরা

প্রমাণ: ২৩ টেবিলে workspace কলাম

৩. নিয়ম ডেটাবেজে

স্লাইড ১২–১৯ · ১৩ unique_together · ৮ শর্তসাপেক্ষ unique · ২ check · ১৩ ইনডেক্স

প্রমাণ: কনস্ট্রেইন্টের নাম

৪. আসল ব্যবহার

স্লাইড ২০–৩০ · হাজিরা · নম্বর · টাকা · সময় · কোয়েরি · মাইগ্রেশন · ব্যাকআপ · ফাঁক

প্রমাণ: ১০৩ টেস্ট · ৫৭ select_related

শেষ তিন স্লাইড ইচ্ছাকৃতভাবে **অস্বস্তিকর**: যা মাপা হয়নি, যা EXPLAIN করা হয়নি, যা এখনো টেবিলে নেই।

শিক্ষকের নোট: এই স্লাইডটি ৩০ সেকেন্ডেই ছাড়ুন — উদ্দেশ্য কেবল শ্রোতাকে জানানো যে কোথায় কী পাবেন। শিক্ষার্থীদের বলুন কোন ব্লকটি তাদের সবচেয়ে দরকার, তার উপর জোর দেবেন।

উৎস

৩১ টেবিল কোথা থেকে এল

টেবিল বানানো হয় না — বিদ্যালয়ের কাগজ থেকেই টেবিল বেরোয়

বিদ্যালয়ে যা আছে	যে প্রশ্নটা জিজ্ঞেস করলাম	যে টেবিল হলো	কেন এটাই সঠিক
ভর্তি খাতা	"এই ছেলেটি কে, আলাদা করে চেনার উপায় কী?"	students.StudentProfile	নাম বদলাতে পারে, `admission_no` নয়
শ্রেণি খাতা	"ছাত্রটি কোন শ্রেণিতে, কোন সেশনে?"	students.Enrollment	শ্রেণি বদল ইতিহাস, মুছে ফেলা নয়
হাজিরা রেজিস্টার	"একই দিনে একই ক্লাসে দুবার লেখা যাবে?"	attendance.AttendanceRecord	প্রতিটি ঘর = এক সারি, unique দিয়ে আটকানো
নম্বরের তালিকা	"অনুপস্থিত আর শূন্য কি এক?"	assessments.Mark	তাই `is_absent` আলাদা কলাম
সিলেবাস ও ডায়েরি	"অধ্যায় → টপিক, কে শেষ করল?"	lessons.Chapter · Topic	দুই ধাপের গাছ, এক টেবিলে গুঁজে দিইনি
হিসাবের খাতা	"কোন প্ল্যানে কত টাকা, কত দিন?"	billing.Plan · Subscription	প্ল্যান বদলালেও পুরনো ইনভয়েস অটুট

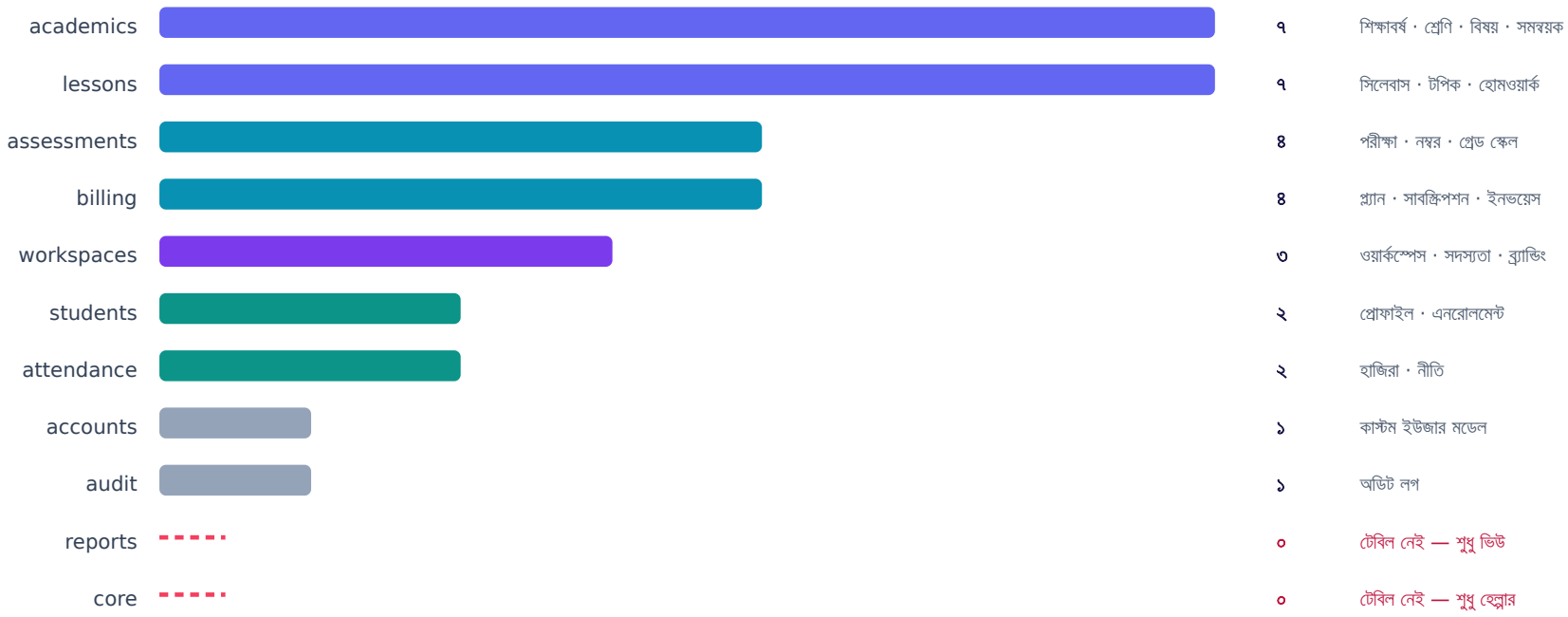
নিয়ম: **কাগজে যে ঘরটি আলাদা করে লেখা হয়, ডেটাবেজে সেটি আলাদা কলাম।** যে ঘর আলাদা লেখা হয় না, সেটি নতুন টেবিল।

শিক্ষকের নোট: বোর্ডে একটি হাজিরা রেজিস্টারের পাতা এঁকে দেখান — তারিখ, শ্রেণি, ৪০টি নাম। তারপর জিজ্ঞেস করুন: "এই পাতায় কতটি আলাদা তথ্য?" উত্তর ৪০ নয়, চারটি: তারিখ, শ্রেণি, বিষয় আর প্রতিটি ছাত্রের একটি অবস্থা।

আকার

১১ অ্যাপে ৩১ মডেল — কে কতটা ভারী

নামের পাশে সংখ্যা = মডেল সংখ্যা; মোট ৩০১ কলাম, ৭৪ সম্পর্ক



reports ও **core**-এ শূন্য মডেল — ইচ্ছাকৃত। রিপোর্ট অন্য অ্যাপের টেবিল পড়ে, নিজের টেবিল বানালে ডেটা দুই জায়গায় থাকত।

শিক্ষকের নোট: যে দুটি অ্যাপে মডেল নেই, সেখানেই থামুন। প্রশ্ন করুন: "রিপোর্টের নিজের টেবিল থাকলে কী সমস্যা হতো?" উত্তর — একই হাজিরা দুই জায়গায়, একটি বদলালে অন্যটি পুরনো।

নামকরণ

চারটি নামকরণ-নিয়ম, ৩১ টেবিলে অক্ষরে অক্ষরে

নাম ভুল হলে পরের ডেভেলপার প্রতিটি কোয়েরিতে ভুল করবে

নিয়ম	উদাহরণ	কেন এভাবে
মডেল একবচন, টেবিল বহুবচন	`ClassGroup` → `academics_classgroup`	কোডে একটি সারি বোঝায়, ডেটাবেজে অনেক সারি — দুই জায়গায় দুই বচন স্বাভাবিক
কলামে `_id` থাকে, মডেলে থাকে না	`class_group` লিখি, DB-তে `class_group_id`	`obj.class_group` = object, `obj.class_group_id` = সংখ্যা — এক নামে দুই জিনিস নয়
`related_name` সবসময় দেওয়া	`related_name="enrollments"`	উল্টো দিকের নাম না দিলে Django `enrollment_set` বানায় — পড়তে খারাপ, দুই অ্যাপে সংঘর্ষও হয়
`related_name="+"` = উল্টো দিক দরকার নেই	`recorded_by`, `entered_by`, `published_by`	ইউজার থেকে "সে কী কী হাজিরা লিখেছে" তালিকা কেউ চায় না; অকারণ রিভার্স-কোয়েরি বন্ধ
শর্ত বোঝায় এমন নাম	`is_archived` · `is_published` · `is_absent`	`status` নামের বুলিয়ানে অর্থ হারায়; `is_` উপসর্গে সত্য/মিথ্যা স্পষ্ট

বাস্তব ফল: `related_name` থাকার কারণে এক কোয়েরিতেই `class_group.enrollments.filter(is_active=True)` লেখা যায় — তিনটি ভিন্ন ভিউতে একই লাইন, তিনবার ভিন্ন কোয়েরি নয়।

শিক্ষকের নোট: নামকরণের নিয়ম মুখস্থ করাবেন না — নিজের প্রজেক্টের একটি মডেল দেখিয়ে প্রতিটি নিয়ম কোথায় লঙ্ঘিত হয়েছে খুঁজতে বলুন। এরপর শিক্ষার্থীদের বলুন নিজের প্রজেক্টে একটি মডেলের সব `related_name` লিখে ফেলতে।

সীমানা

`workspace` কলাম — ২৩ টেবিলে আছে, ৮-এ নেই, কারণ লেখা আছে

একটি ডেটাবেজে বহু বিদ্যালয়; দেয়ালটা কোথায়

শ্রেণি	সংখ্যা	কোনগুলো	কেন
নিজের `workspace` কলাম আছে	২৩	ClassGroup · Exam · AttendanceRecord · Homework · Invoice …	প্রতিটি লিস্ট-কোয়েরি এক লাইনেই সীমা টানে
মূল/গ্লোবাল টেবিল	৩	`Workspace` · `User` · `Plan`	এরা নিজেরাই সীমানার বাইরে — এক ইউজার বহু বিদ্যালয়ে
প্যারেন্ট দিয়ে সীমানা	৫	`Chapter` → Syllabus · `Topic` → Chapter · `Submission` → Homework · `Mark` → Exam · `GradeBand` → GradeScale	কলাম যোগ করলে দুই জায়গায় সত্য থাকত, একদিন মিলত না

দাম: এই পাঁচটি টেবিলে `filter(workspace=…)` লেখা যায় না — `filter(exam__workspace=…)` লিখতে হয়। ভুল করলে কোয়েরি চলে, কিন্তু অন্য বিদ্যালয়ের ডেটাও আসতে পারে।

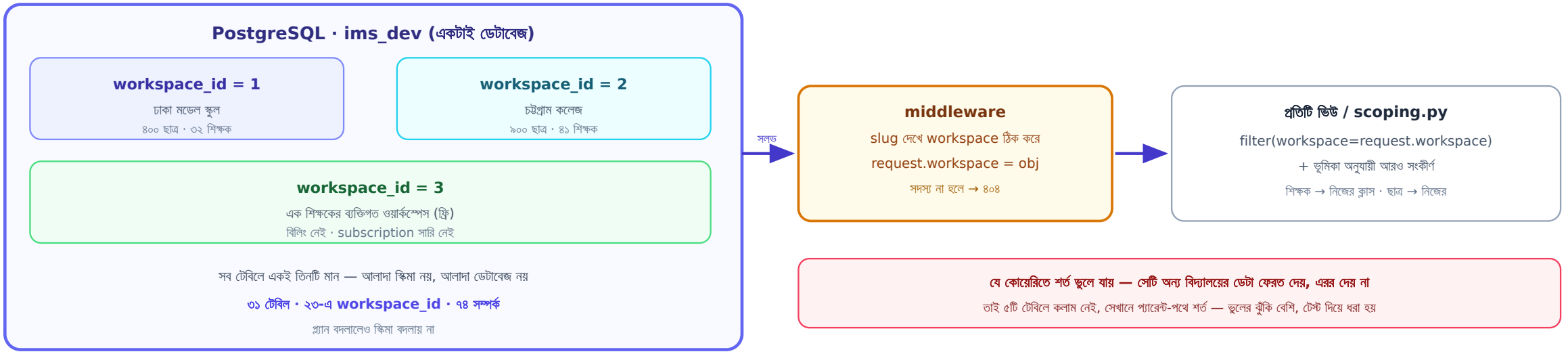
প্রতিরোধ: ভিউ কখনো সরাসরি ফিল্টার লেখে না — `apps/core/scoping.py`-র হেল্পার ডাকে, যেখানে সীমাটা একবার লেখা।

শিক্ষকের নোট: এখানেই বলুন — "মাল্টি-টেন্যান্ট" শব্দটা জাদু নয়। এটি কেবল একটি কলাম আর একটি অভ্যাস: প্রতিটি কোয়েরিতে ওই কলামের শর্ত। অভ্যাস ভাঙলে ডেটা মিশে যায়, আর মিশে গেলে ফেরানো প্রায় অসম্ভব।

দেয়াল

এক ডেটাবেজ, তিন বিদ্যালয় — দেয়াল কীভাবে টানে

মিডলওয়্যার একবার ঠিক করে, প্রতিটি কোয়েরি সেটাই ব্যবহার করে



শিক্ষকের নোট: ডান দিকের লাল বাক্সটি পড়ে জিজ্ঞেস করুন: "এরর না দিয়ে ভুল ডেটা দেওয়া আর এরর দেওয়া — কোনটা বিপজ্জনক?" উত্তর: ভুল ডেটা, কারণ কেউ টের পায় না। এটাই পরের স্লাইডের ভিত্তি।

সিদ্ধান্ত ১

কলাম যোগ করা, না প্যারেন্ট দিয়ে খোঁজা?

একই প্রশ্ন পাঁচটি টেবিলে — উত্তর দুই রকম

যখন কলাম যোগ করলাম

```
# attendance/models.py  (workspace আছে)
qs = AttendanceRecord.objects.filter(
    workspace=request.workspace, date=today)

# lessons/models.py  (Homework-এ workspace আছে)
qs = Homework.objects.filter(
    workspace=ws, class_group=cg, due_at__gte=now)
```

এক JOIN কম, কোয়েরি ছোট, ভুলের সুযোগ কম।

যখন কলাম দিইনি

```
# Mark: workspace কলাম নেই, exam-এ আছে
Mark.objects.filter(
    exam__workspace=ws, exam__is_published=True)

# Chapter: syllabus-এ workspace আছে
Chapter.objects.filter(syllabus__workspace=ws)
```

এক JOIN বেশি, কিন্তু সত্য এক জায়গায়।

প্রশ্ন	কলাম যোগ	প্যারেন্ট-পথ
কখন বেছে নিলাম	টেবিলটা নিজে ঘন ঘন লিস্ট হয়, এবং প্যারেন্ট ছাড়াও কাজে লাগে (`Homework`, `Exam`, `AttendanceRecord`)	টেবিলটা প্যারেন্ট ছাড়া কখনো পড়া হয় না (`Mark`, `Chapter`, `Topic`, `Submission`, `GradeBand`)
ভুলের ধরন	দুই জায়গায় `workspace` — একদিন একটি সারিতে ভুল মান ঢুকে যায়	ভিউতে `workspace` ফিল্টার ভুলে যাওয়া — ডেটা মিশে যায়, এরর নেই
এই ভুল ধরার উপায়	`clean()`-এ প্যারেন্টের workspace মিলিয়ে দেখা	`scoping.py`-র একক হেল্পার + ১০৩ টেস্টের আইসোলেশন টেস্ট

শিক্ষকের নোট: শিক্ষার্থীরা প্রায় সবসময়ই সব টেবিলে `workspace` যোগ করতে চায় — সহজ বলে। তাদের জিজ্ঞেস করুন: "তাহলে Mark-এ workspace আর Exam-এ workspace আলাদা হলে কোনটা সত্য?" দ্বিতীয় সত্য সবসময় হারায়।

সিদ্ধান্ত ২

প্রথম মাইগ্রেশনের আগেই নিজের ইউজার মডেল

একবার ভুল করলে সারাজীবন বয়ে বেড়াতে হয় — তাই একবারেই ঠিক

```
# apps/accounts/models.py
class User(AbstractBaseUser, PermissionsMixin):
    email = models.EmailField(unique=True, db_index=True)
    full_name = models.CharField(max_length=150)
    phone = models.CharField(max_length=16, blank=True,
                             validators=[phone_validator])
    is_active = models.BooleanField(default=True)
    USERNAME_FIELD = "email"
```

`USERNAME_FIELD = "email"` — লগইন আইডি ইমেইল। নাম বদলালে লগইন বদলায় না; `@ims.local` দিয়ে ডেমো চলে, বাস্তবে আসল ইমেইল।

যদি ডিফল্ট `User` দিয়ে শুরু করতাম	পরিণতি
পরে নিজের মডেল দরকার হলে	সব FK ভাঙে — `AUTH_USER_MODEL` বদলানো মানে ৭০টি সম্পর্ক আবার
ইউজারনেম রাখতে হতো	স্কুলে সবাই ইমেইলেই চেনে; ইউজারনেম বানানো বাড়তি কাজ, ভুলের উৎস
ফোন নম্বর আলাদা প্রোফাইলে	আরও একটি টেবিল, আরও একটি JOIN — প্রতিটি SMS-এ

প্রমাণ: `apps/accounts/migrations/0001_initial.py` — প্রজেক্টের প্রথম মাইগ্রেশন, আর সেটিই নিজের মডেল।

শিক্ষকের নোট: এই সিদ্ধান্তটি ডেটাবেজের নয়, শৃঙ্খলার। বলুন: "শেষে ঠিক করা যায়" — ডেটাবেজে এই বাক্যটি প্রায়ই মিথ্যা। যা পরে বদলাতে অসাধ্য, তা প্রথম দিনেই ঠিক করতে হয়।

সিদ্ধান্ত ৩

এক ইউজার, বহু বিদ্যালয়, প্রতি বিদ্যালয়ে ভিন্ন ভূমিকা

ভূমিকা ইউজারের কলামে নয় — `Membership` সারিতে

টেবিল	কী বলে	কেন এখানে
`accounts.User`	"এই ব্যক্তিটি কে" — ইমেইল, নাম, ফোন	পরিচয় গ্লোবাল; স্কুল বদলালেও পরিচয় এক
`workspaces.Membership`	"এই বিদ্যালয়ে তার ভূমিকা কী"	একই মানুষ এক স্কুলে শিক্ষক, আরেক স্কুলে সমন্বয়ক — একটি `role` কলাম দিয়ে সম্ভব নয়
`students.StudentProfile`	"ছাত্র হিসেবে তার নম্বর, ভর্তি নম্বর"	প্রতিটি ছাত্রের অ্যাকাউন্ট থাকে না; থাকলেও প্রোফাইল আলাদা তথ্য
`academics.CoordinatorScope`	"সমন্বয়ক কোন বিভাগ/ক্লাস দেখবে"	অনুমতি ডেটা হিসেবে রাখা — রোল-স্ট্রিং-এ লুকানো নয়

প্রমাণ ১: `Membership`-এ `UniqueConstraint(["user","workspace"])` — এক স্কুলে এক ভূমিকা। দুই ভূমিকা দরকার হলে সেটি ডিজাইনের সংকেত, দুটি সারি নয়।

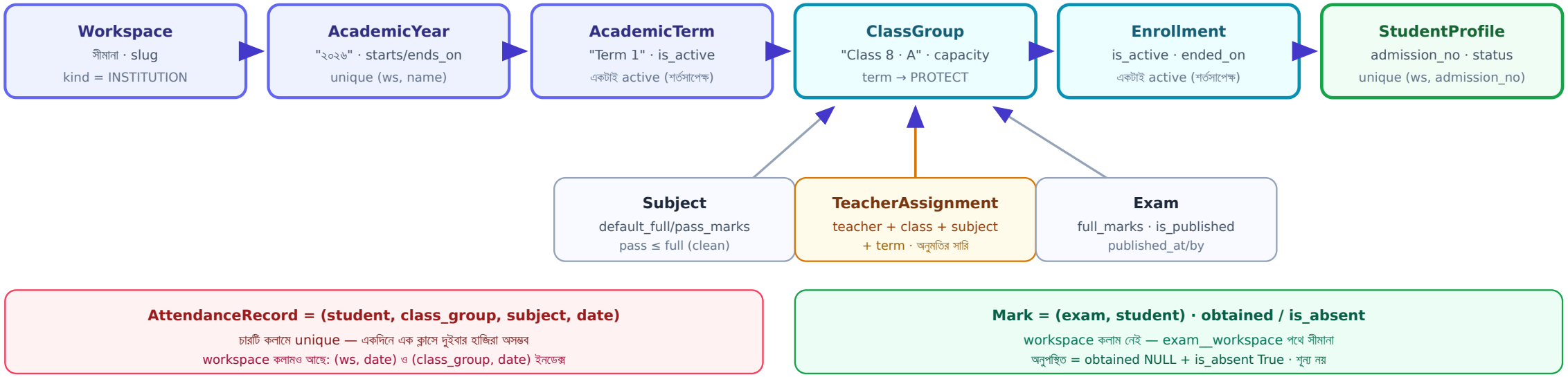
প্রমাণ ২: `StudentProfile.account` ← `OneToOneField(..., on_delete=SET_NULL, null=True)`। অ্যাকাউন্ট মুছলেও ছাত্রের নম্বর-হাজিরা টিকে থাকে।

শিক্ষকের নোট: বাস্তব উদাহরণ দিন: একজন শিক্ষক দুটি স্কুলে পড়ান। তিনি দ্বিতীয় স্কুলে যোগ দিলে নতুন অ্যাকাউন্ট বানাতে হয় না — কেবল একটি `Membership` সারি। এটাই "এক ইউজার, বহু প্রতিষ্ঠান"।

কশিরা

একাডেমিক মেরুদণ্ড — ছয় ধাপের শৃঙ্খল

প্রতিটি রেকর্ড এই শৃঙ্খলের একটি সংযোগ-বিন্দুতে ঝোলে



শিক্ষকের নোট: আঙুল দিয়ে শৃঙ্খল ধরে ধরে বলুন: "উপরের ছয়টি টেবিল না থাকলে নিচের দুটির কোনো অর্থ নেই।" তারপর জিজ্ঞেস করুন — ছাত্র ক্লাস বদলালে কী হয়? উত্তর: নতুন `Enrollment` সারি, পুরনোটি `ended_on` নিয়ে নিষ্ক্রিয়।

নিয়ম ১

`unique_together` — ১৩টি, প্রতিটি একটি বাস্তব ভুল আটকায়

"একই জিনিস দুবার" সমস্যার ডেটাবেজ-স্তরের উত্তর

টেবিল	unique	কোন ভুল আটকায়
StudentProfile	workspace · admission_no	এক স্কুলে একই ভর্তি নম্বর দুজন ছাত্রের নয়
AcademicYear	workspace · name	"২০২৬" দুইবার তৈরি
AcademicTerm	workspace · year · name	এক বছরেই দুইবার "Term 1"
ClassGroup	workspace · term · name · section	"Class 8 · A" দুবার, একই সেশনে
TeacherAssignment	teacher · class_group · subject · term	একই বিষয়ে একজনকে দুইবার যোগ
AttendanceRecord	student · class_group · subject · date	একদিনে এক ছাত্রের দুই হাজিরা
Exam	class_group · subject · term · name	"মিডটার্ম" পরীক্ষা দুইবার
LessonLog · Syllabus · Chapter · Topic · GradeScale · Department · Subject	৩–২ কলাম	পাঠ-ডায়েরি, সিলেবাস ও গ্রেড স্কেলে একই পুনরাবৃত্তি

সবগুলোতেই `workspace` প্রথমে — কারণ uniqueness-ও স্কুলের ভেতরে, স্কুল জুড়ে নয়। দুটি স্কুলের "Class 8 · A" এক হতে পারে, এবং হওয়া উচিত।

শিক্ষকের নোট: শেষ বাক্যটি বোর্ডে লিখুন। শিক্ষার্থীরা প্রথমে `unique=True` দিতে চায় — তখন দ্বিতীয় স্কুল "Class 8 · A" বানাতে পারে না। বাগটা ডেমোতে চোখে দেখা যায় না, দ্বিতীয় স্কুল যোগ করলেই দেখা যায়।

নিয়ম ২

শর্তসাপেক্ষ unique — "একটাই active", ডেটাবেজেই আটকানো

৮টি `UniqueConstraint(condition=…)`; পাইথন নয়, ডেটাবেজ সত্য বলে

```
class AcademicTerm(TimeStamped):
    is_active = models.BooleanField(default=False)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["workspace"],
                condition=Q(is_active=True),
                name="uniq_active_term_per_workspace",
            )
        ]
```

অর্থ: যেসব সারিতে `is_active=True`, তাদের মধ্যে `workspace` অনন্য। বাকি সারি যত খুশি।

কনস্ট্রেইন্টের নাম	যা অনন্য
`uniq_active_term_per_workspace`	সক্রিয় সেশন
`uniq_active_grade_scale_per_workspace`	সক্রিয় গ্রেড স্কেল
`uniq_active_enrollment_student_class`	ছাত্রের সক্রিয় এনরোলমেন্ট
`uniq_membership_user_workspace`	স্কুলে ইউজারের ভূমিকা
`unique_custom_domain`	কাস্টম ডোমেইন

কেন `clean()` নয়? দুই ইউজার একসাথে সেভ করলে দুটোই `clean()` পাশ করে যেতে পারে — কেবল ডেটাবেজ-স্তরের নিয়মই সত্যিকারের দেয়াল।

শিক্ষকের নোট: "রেস কন্ডিশন" শব্দটি এখানেই প্রথম ব্যবহার করুন — দুই শিক্ষক একই মুহূর্তে সেশন বদলালে কী হয়? পাইথনে চেক করে নয়, ইনডেক্সই শেষ কথা বলে।

নিয়ম ৩

`CheckConstraint` — মিথ্যা তথ্য ঢোকার পথ বন্ধ

২টি check; বাকি সব নিয়ম `clean()`-এ — কেন এই ভাগ

ডেটাবেজে (২টি)

```
# assessments · নাম: grade_band_min_le_max
Q(min_percent__lte=F("max_percent"))

# academics · নাম: coordinator_scope_needs_a_target
~(Q(department__isnull=True) &
  Q(class_group__isnull=True))
```

একটি সারির **নিজের** দুটি কলাম তুলনা — JOIN বা অন্য টেবিল লাগে না।

অ্যাপ্লিকেশনে (`clean()`)

`ends_on > starts_on`	AcademicYear · Term
`pass_marks ≤ full_marks`	Subject · Exam
মার্ক ≤ full_marks · ঋণাত্মক নয়	Mark
অনুপস্থিত ছাত্রের নম্বর নেই	Mark
প্যারেন্ট অন্য workspace-এর নয়	ClassGroup · Term
ভবিষ্যতের তারিখে হাজিরা নয়	AttendanceRecord

সৎ ফাঁক: উপরের ছয়টি নিয়ম কেবল ফর্ম/ভিউ দিয়ে গেলে চলে; সরাসরি `.objects.create()` বা শেল দিয়ে গেলে `clean()` ডাকা হয় না। এই ছয়টির একটিও ডেটাবেজ-স্তরে নেই।

শিক্ষকের নোট: এই ফাঁকটি লুকাবেন না — জিজ্ঞেস করুন "কোন নিয়মটা ডেটাবেজে রাখা উচিত ছিল?" উত্তর: `pass_marks ≤ full_marks` (নিজের দুই কলাম, তাই সহজে সম্ভব ছিল) আর প্যারেন্ট-ওয়ার্কস্পেস মিল (অন্য টেবিল, তাই কঠিন)।

নিয়ম ৪

এনরোলমেন্ট: ইতিহাস মুছি না, নিষ্ক্রিয় করি

শর্তসাপেক্ষ unique-এর সবচেয়ে দরকারি ব্যবহার

```
class Enrollment(TimeStamped):
    student = FK(StudentProfile, CASCADE)
    class_group = FK(ClassGroup, CASCADE)
    is_active = models.BooleanField(default=True)
    ended_on = models.DateField(null=True, blank=True)
    end_reason = CharField(max_length=200)

    class Meta:
        constraints = [UniqueConstraint(
            fields=["student", "class_group"],
            condition=Q(is_active=True),
            name="uniq_active_enrollment_student_class")]
```

ঘটনা	ডেটায় যা ঘটে
ক্লাস বদল	পুরনো সারি: `is_active=False` + `ended_on` + `end_reason`; নতুন সারি তৈরি
একই ক্লাসে ফেরা	নতুন সারি — পুরনোটি ইতিহাসে থেকে যায়
ভুল করে দুইবার active	ডেটাবেজ নিজেই আটকায়, ভিউ নয়
হাজিরা নেওয়া	`clean()` দেখে ছাত্র সত্যিই এই ক্লাসে active কি না

একই প্যাটার্ন তিন জায়গায়: সক্রিয় **সেশন**, সক্রিয় **গ্রেড স্কেল**, সক্রিয় **ভূমিকা**।

দাম: এক ছাত্র পরপর দুই বার একই ক্লাসে ভর্তি হলে ডেটা ঠিকই থাকে, কিন্তু `ended_on` ছাড়া পুরনো সারি নিষ্ক্রিয় করার সুযোগ নেই — ভিউতে অবশ্যই `ended_on` সেট করতে হয়, নাহলে ইতিহাসে ফাঁক থেকে যায়।

শিক্ষকের নোট: বাস্তব প্রশ্ন করুন: "ছাত্র ক্লাস বদলাল, রিপোর্ট কার্ডে পুরনো ক্লাসের নম্বর কোথায় থাকবে?" উত্তর — পুরনো এনরোলমেন্ট সারিতে, কারণ নম্বর কখনো সরানো হয় না।

গতি

১৩টি হাতে লেখা ইনডেক্স — কোন কোয়েরির জন্য

মোট ইনডেক্স ≈ ১৩৯; তার ১২৬টি Django নিজেই বানায়

ইনডেক্স	যে কোয়েরি দ্রুত হয়
`(workspace, date)` · Attendance	আজকের হাজিরা, স্কুলের সব ক্লাস
`(class_group, date)`	এক ক্লাসের দৈনিক হাজিরা
`(workspace, class_group, is_active)`	ক্লাসের সক্রিয় ছাত্র-তালিকা
`(workspace, status)` · Student	সক্রিয় ছাত্রের তালিকা
`(workspace, term, is_archived)`	সেশনের ক্লাস-তালিকা
`(workspace, term, is_published)`	প্রকাশিত ফলাফল
`(workspace, class_group, due_at)`	আসন্ন হোমওয়ার্ক
`(workspace, -created_at)` · Audit	সাম্প্রতিক অডিট লগ
`(user, is_active)` · Membership	ইউজারের স্কুল-তালিকা (প্রতি রিকোয়েস্টে)

৩১ · প্রাইমারি কী

স্বয়ংক্রিয়

৭৪ · ফরেন কী

Django প্রতিটি FK-তে

২১ · unique নিয়ম

১৩ unique_together + ৮ শর্তসাপেক্ষ

১৩ · হাতে লেখা

কেবল আসল কোয়েরি দেখে

এগুলোর কোনোটিই `EXPLAIN` দেখে নেওয়া নয় — লিস্ট-ভিউয়ের `filter()` পড়ে অনুমান করা। প্রকৃত পরিমাপ এখনো হয়নি।

শিক্ষকের নোট: সততাটুকু জোর দিয়ে বলুন: ইনডেক্স "সঠিক" প্রমাণ করতে `EXPLAIN ANALYZE` লাগে, অনুমান নয়। ৪০০ ছাত্রের ডেমোতে পার্থক্যই দেখা যায় না; পার্থক্য দেখা যায় ৫০ হাজার সারিতে।

সিদ্ধান্ত ৪

কিছু মুছলে কী হয় — ৭০টি FK-র তিন রকম উত্তর

CASCADE ৪৮ · SET_NULL ১৬ · PROTECT ৬ — আর ৪টি OneToOne

CASCADE · ৪৮

প্যারেন্ট গেলে সন্তানও যায়। **কেবল তখনই**, যখন সন্তানের স্বাধীন অস্তিত্ব নেই।

ClassGroup মুছলে তার Exam, AttendanceRecord, Homework — সব। বিদ্যালয় মুছলে পুরো স্কুলের ডেটা।

SET_NULL · ১৬

রেফারেন্স মুছে যায়, তথ্য থাকে — "কে করেছিল" ঘরটি ফাঁকা হয়।

১৬টির মধ্যে **১২টিই ইউজার**: `recorded_by`, `entered_by`, `published_by`, `reviewed_by` ...

PROTECT · ৬

মুছতেই দেওয়া হয় না — `ProtectedError` ওঠে। রক্ষা করতে চাইলে **আগে** সিদ্ধান্ত নিতে হয়।

টার্ম, প্ল্যান, ইনভয়েস, স্কুলের মালিক — এই চার ধরনের জিনিস।

নিয়ম: **যে সম্পর্কে অর্থ হারায়, সেখানে CASCADE; যে সম্পর্কে কেবল দায়িত্ব হারায়, সেখানে SET_NULL; যে সম্পর্কে হিসাব ভাঙে, সেখানে PROTECT।**

শিক্ষকের নোট: শিক্ষার্থীদের সবসময় CASCADE লিখতে অভ্যাস। তিনটি উদাহরণ জিজ্ঞেস করুন: "শিক্ষক চাকরি ছাড়লে তার নেওয়া হাজিরা মুছবে?" — না, তাই SET_NULL। "টার্ম মুছলে তার ক্লাস?" — না, PROTECT।

PROTECT

যে ছয়টি জিনিস ভুল করে মুছে ফেলা যাবে না

প্রতিটির পেছনে একটি বাস্তব দুঃস্বপ্ন

FK	মুছতে গেলে	কেন এটাই সঠিক
`Workspace.owner` → User	`ProtectedError`	মালিকহীন স্কুল = কেউ ঢুকতে পারে না, কেউ মেটাতে পারে না
`ClassGroup.term` → AcademicTerm	আটকে যায়	সেশন মুছলে তার ক্লাস, হাজিরা, পরীক্ষা — সব অনাথ
`Exam.term` → AcademicTerm	আটকে যায়	ফলাফল কোন সেশনের, তার প্রমাণ হারায়
`Subscription.plan` → Plan	আটকে যায়	বিলিং ইতিহাসে দামের ভিত্তি অদৃশ্য হয়ে যায়
`Invoice.workspace` → Workspace	আটকে যায়	টাকা নেওয়ার রেকর্ড কখনো "মুছে ফেলা" যায় না
`Invoice.plan` → Plan	আটকে যায়	কোন প্ল্যানে কত টাকা — ইনভয়েস নিজেই সত্য বলে

দাম: প্ল্যানের দাম বদলাতে চাইলে পুরনো প্ল্যান মুছবেন না — নতুন প্ল্যান বানিয়ে `is_active=False` করুন।

টেস্ট: এই ছয়টির একটিও আলাদা টেস্টে লেখা নেই — `PROTECT` Django-র নিজের আচরণ, তাই বাড়তি টেস্ট লেখা হয়নি।

শিক্ষকের নোট: "কেন মালিক PROTECT, অথচ হাজিরা-লেখক SET_NULL?" — এই প্রশ্নটিই পাঠের মূল। উত্তর: মালিক ছাড়া স্কুল চলে না; হাজিরা-লেখক ছাড়া হাজিরা চলে।

SET_NULL

"কে করেছিল" ঘরটি ফাঁকা হয়, তথ্য থাকে

১৬টি SET_NULL; ১২টি ইউজারের দিকে — একটি ইচ্ছাকৃত প্যাটার্ন

```
recorded_by = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.SET_NULL, null=True,
    related_name="+")

updated_by = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.SET_NULL, null=True,
    blank=True, related_name="+")
```

হাজিরা সারি থেকে যায়; কে লিখেছিল, তা আর দেখা যায় না।

SET_NULL (১২টি ইউজার)	কেন
`AttendanceRecord.recorded_by` · `updated_by`	হাজিরা ছাত্রের তথ্য, শিক্ষকের নয়
`Mark.entered_by` · `Exam.published_by`	নম্বর থাকা চাই, লেখক বদলাতে পারে
`AuditLog.actor`	লগ থাকে, ইউজার না-ও থাকতে পারে
`Enrollment.enrolled_by` · `Homework.created_by`	ভর্তি ইতিহাস মুছবে না

ফাঁক: ইউজার মুছে গেলে "কে করেছিল" কেবল `AuditLog`-এ থাকে — আর সেটি সব কাজে লেখা হয় না।

শিক্ষকের নোট: প্রশ্ন করুন: "শিক্ষক অবসরে গেলে তার ৬ বছরের হাজিরা মুছে ফেলা কি ঠিক?" উত্তর স্পষ্ট না হলেও আলোচনাটাই শেখায় — ডেটাবেজ ডিজাইনে সবসময় "একটি সারি" নয়, "একটি মানুষের কাজের ইতিহাস" জড়িত।

কেস ১

একটি হাজিরা সারির ১৩টি ঘর — কেন প্রতিটি দরকার

attendance.AttendanceRecord · ১৩ কলাম · ১ unique · ২ ইনডেক্স

কলাম	কেন আছে
`workspace`	লিস্ট-কোয়েরির একক শর্ত
`student` · `class_group`	"কোন ছাত্র, কোন ক্লাসে"
`subject`	বিষয়ভিত্তিক হাজিরা; একই দিনে ৬ বিষয়
`date`	দিন; ভবিষ্যতের তারিখ নিষিদ্ধ (`clean()`)
`status`	PRESENT · ABSENT · LATE · LEAVE
`note`	২০০ অক্ষরের টীকা

কলাম	কেন আছে
`recorded_by`	কে প্রথম লিখল (SET_NULL)
`updated_by`	কে সংশোধন করল — না থাকলে সংশোধন নয়
`correction_reason`	সংশোধনের কারণ, ২০০ অক্ষরে
`created_at` · `updated_at`	কখন লেখা, কখন বদলানো
`id`	সারির পরিচয়

সংশোধনের অনুমতি কার, কত দিন পর্যন্ত — `AttendancePolicy.correction_window_days = 7` ; ন্যূনতম উপস্থিতি ৭৫%। নীতি ডেটায়, কোডে নয়।

খেয়াল করুন: সংশোধন মানে পুরনো সারি বদলানো, নতুন সারি নয়। তাই ইতিহাস এক সারিতেই — কে, কখন, কেন বদলাল, তিনটিই ওই সারির ভেতরে।

শিক্ষকের নোট: প্রশ্ন করুন: "শিক্ষক ভুল করে সবাইকে অনুপস্থিত দিয়ে ফেললেন — ডেটাবেজে কী ঘটবে?" দুটি উত্তর শোনান: বদলে ফেলা (ইতিহাস হারায়) আর নতুন সারি (ডুপ্লিকেট)। প্রকল্পে তৃতীয় পথ: একই সারি + `updated_by` + `correction_reason` ।

কেস ২

অনুপস্থিত আর শূন্য নম্বর — ডেটায় আলাদা, তাই আলাদা ফল

assessments.Mark · ৮ কলাম · (exam, student) unique

```
obtained = DecimalField(max_digits=6, decimal_places=2,
                        null=True, blank=True)
is_absent = models.BooleanField(default=False)

def clean(self):
    if self.is_absent and self.obtained is not None:
        raise ValidationError(...)     # দুটোই সত্য নয়
    if self.obtained > self.exam.full_marks:
        raise ValidationError(...)
```

অবস্থা	ডেটা	রিপোর্টে
উপস্থিত, ০ পেয়েছে	`obtained=0`, `is_absent=False`	গড়ে ০ ধরা হয়, ফেল
অনুপস্থিত	`obtained=NULL`, `is_absent=True`	গড় থেকে বাদ, আলাদা গণনা
নম্বর এখনো তোলা হয়নি	`NULL`, `is_absent=False`	গড় থেকে বাদ

যদি অনুপস্থিতকে ০ লিখতাম, ক্লাসের গড় আর পাশের হার দুটোই মিথ্যা হতো — আর কেউ ধরতে পারত না।

গণনার নিয়ম এক জায়গায়: `build_result_sheet(exam)` — শুধু পড়ে, লেখে না। তাই হাজিরা-রিপোর্ট আর রেজাল্ট-শিট একই ফাংশন ডাকে, দুটি আলাদা হিসাব হয় না।

শিক্ষকের নোট: এই স্লাইডটিই "NULL-এর অর্থ" শেখানোর সেরা জায়গা। তিন ধরনের অনিশ্চয়তা দেখান: অজানা (NULL), শূন্য (০), প্রযোজ্য নয় (অনুপস্থিত) — তিনটিই আলাদা।

কেস ৩

ফলাফল প্রকাশ — এক বুলিয়ান নয়, একটি তারিখও

is_published · published_at · published_by · তিনটিই দরকার

কলাম	উত্তর দেয়
`is_published`	ছাত্র এখন দেখতে পাবে কি না
`published_at`	কখন প্রকাশ হলো — অভিযোগের সময় কাজে লাগে
`published_by`	কে প্রকাশ করল (SET_NULL, `related_name="+"`)

```
exam.is_published = True
exam.published_at = timezone.now()
exam.published_by = request.user
exam.save(update_fields=[
    "is_published", "published_at",
    "published_by", "updated_at"])
```

প্রশ্ন: শুধু `is_published` রাখলে কী হারাতাম? উত্তর: "কবে, কে" — এবং প্রতারণার অভিযোগ এলে কোনো প্রমাণ থাকত না।

প্রবেশ-নিয়ন্ত্রণ দুই জায়গায়: ভিউতে ভূমিকা (`TEACHING`) + `teacher_can_access_class()`; আর ছাত্রের পথে অপ্রকাশিত ফল হলে `PermissionDenied` → ৪০৩।

সৎ ফাঁক: প্রকাশের সময় কোনো `transaction.atomic` নেই — একটি সারি বদলানোয় দরকারও নেই; কিন্তু `log()` আর `save()` আলাদা লেনদেন, তাই লগ লেখা ব্যর্থ হলে (নীরবে) থেকেই যায়।

শিক্ষকের নোট: বাস্তব ঘটনা বলুন: অভিভাবক এসে বললেন "গতকাল রাতে নম্বর বদলেছে"। `published_at` + অডিট লগ থাকলে উত্তর দেওয়া যায়; না থাকলে কেবল অনুমান।

কেস ৪

টাকা কখনো `float` নয় — ৬টি `DecimalField`

আর ইনভয়েস নম্বরের একটি বাস্তব দুর্বলতা

কলাম	সেটিং
`Plan.price`	max_digits=10 · decimal_places=2 · ≥ ০
`Invoice.amount`	max_digits=10 · decimal_places=2
`Mark.obtained`	max_digits=6 · decimal_places=2 · NULL অনুমোদিত
`GradeBand.min/max_percent`	max_digits=5 · decimal_places=2
`GradeBand.gpa`	max_digits=3 · default 0.00

`float`-এ ০.১ + ০.২ = ০.৩০০০০০০০০০০০০০০৪। এক বছরে হাজার ইনভয়েসে সেটি টাকায় পরিণত হয়।

```
@staticmethod
def _next_number():
    year = timezone.localdate().year
    prefix = f"{settings.INVOICE_PREFIX}-{year}-"
    last = (Invoice.objects
            .filter(number__startswith=prefix)
            .order_by("-number")
            .values_list("number", flat=True).first())
    n = int(last.split("-")[-1]) + 1 if last else 1
    return f"{prefix}{n:04d}"
```

দুর্বলতা: "শেষটি পড়ে এক যোগ" — একই মুহূর্তে দুই ইনভয়েস হলে দুটোই একই নম্বর পায়। `number`-এ `unique=True` থাকায় ডেটাবেজ আটকায়, কিন্তু ব্যবহারকারী ৫০০ এরর দেখে।

সঠিক পথ হতো ডেটাবেজ সিকোয়েন্স, অথবা `select_for_update()` দিয়ে সারি লক করে নম্বর বাড়ানো। এখনো করা হয়নি — এটি প্রকৃত, চিহ্নিত ঋণ।

শিক্ষকের নোট: এই স্লাইডে সততাই মূল্য: কোড কাজ করে, কিন্তু একই সময়ে দুই ইনভয়েস হলে ভাঙে। শিক্ষার্থীদের জিজ্ঞেস করুন — "লোড কম, তাই ধরা পড়েনি" এটা কি যথেষ্ট কারণ?

কেস ৫

সময়ের ৫৩টি ঘর — কে লেখে, কে বদলায়

১৯টি মডেল `TimeStamped` থেকে উত্তরাধিকার পায় (১৯ × ২ = ৩৮ ঘর)

```
class TimeStamped(models.Model):
    created_at = DateTimeField(auto_now_add=True)
    updated_at = DateTimeField(auto_now=True)

    class Meta:
        abstract = True
```

ফিল্ড	কে লেখে
`auto_now_add`	প্রথম সেভে একবার; পরে কেউ বদলাতে পারে না
`auto_now`	প্রতিটি `save()`-এ আবার
`default=timezone.now`	মডেল-স্তরে নয়, মাইগ্রেশনে মান লেখা হয়

যেখানে সময় গুরুত্বপূর্ণ	কেন
`AttendanceRecord.date` (`DateField`)	হাজিরার দিন — সময় নয়, তারিখ
`Homework.due_at` (datetime)	"আজ রাত ১১:৫৯" — সময় দরকার
`AuditLog.created_at`	সাম্প্রতিক লগ ইনডেক্সের ভিত্তি
`Invoice.issued_on`	`timezone.localdate()` — সার্ভার UTC নয়, স্থানীয় দিন

`USE_TZ=True` · ডেটাবেজে UTC · পর্দায় `Asia/Dhaka`। ভুল করলে "আজকের হাজিরা" রাত ১২টার পর একদিন পিছিয়ে যায়।

নিয়ম: **যে তথ্য ব্যবহারকারী বেছে নেয়, তার জন্য `default`; যে তথ্য সিস্টেম লেখে, তার জন্য `auto_now*`।** দুটো মিশিয়ে ফেললেই পরীক্ষায় সময় নিয়ে বিভ্রান্তি।

শিক্ষকের নোট: প্রশ্ন করুন: "মাইগ্রেশনে `auto_now_add` কেন সমস্যা করে?" — কারণ পুরনো সারিতে তারিখ লাগে, আর `auto_now_add` মাইগ্রেশনকে সেটি লিখতে দেয় না; তখন `default` দিয়ে দুই ধাপে করতে হয়।

প্রমাণ

কোন টেস্ট কোন ডেটাবেজ-নিয়ম প্রমাণ করে

১০৩টি টেস্ট · এর মধ্যে ডেটা-নিয়মের প্রমাণ ১৪টি

টেস্টের নাম (আসল)	যে নিয়ম প্রমাণ করে	স্তর
`test_only_one_term_can_be_active`	`uniq_active_term_per_workspace`	ডেটাবেজ
`test_saving_twice_does_not_duplicate`	`uniq_attendance_student_class_subject_date`	ডেটাবেজ
`test_domain_cannot_be_stolen_by_a_second_institution`	`unique_custom_domain`	ডেটাবেজ
`test_absent_student_cannot_have_a_mark`	Mark — অনুপস্থিত + নম্বর একসাথে নয়	`clean()`
`test_mark_above_full_marks_is_rejected`	Mark — পূর্ণমানের সীমা	`clean()`
`test_overlapping_bands_are_rejected`	GradeBand — ওভারল্যাপ নয়	`clean()`
`test_unenrolled_student_cannot_be_marked`	AttendanceRecord ↔ Enrollment	`clean()` + কোয়েরি
`test_future_date_is_rejected`	ভবিষ্যতের হাজিরা নয়	`clean()`
`test_invoice_numbers_are_sequential`	ইনভয়েস নম্বরের ধারা	এক-থ্রেডে মাত্র
`test_used_seats_come_from_real_rows`	আসন গোনা হয় সারি থেকে, কাউন্টার থেকে নয়	কোয়েরি

শেষ সারিটি মনে রাখুন: `test_invoice_numbers_are_sequential` ধারাবাহিকতা প্রমাণ করে, কিন্তু **সমান্তরালতায়** ভাঙে কি না তা পরীক্ষা করে না।

শিক্ষকের নোট: টেস্টের নাম পড়ে বলুন — ভালো টেস্টের নামই ডকুমেন্টেশন। "কোন নিয়মটি সত্যিই প্রমাণিত, আর কোনটি নিছক অনুমান" — এই প্রশ্নটি শিক্ষার্থীদের নিজের প্রজেক্টে করতে দিন।

কোয়েরি

N+1 সমস্যা, আর ৫৭টি `select_related`

৩১ টেবিল থাকলেই ধীর হয় না — JOIN হারিয়ে গেলেই ধীর হয়

৪০ ছাত্রের হাজিরা, খারাপ পথ

```
for r in AttendanceRecord.objects.filter(
        workspace=ws, date=today):
    print(r.student.full_name)    # প্রতি সারিতে ১ কোয়েরি
```

১ + ৪০ = ৪১ কোয়েরি

ভালো পথ

```
qs = (AttendanceRecord.objects
      .filter(workspace=ws, date=today)
      .select_related("student", "subject"))
```

২ কোয়েরি — ৪১ নয়

৫৭ · `select_related`
সরাসরি FK — একই SQL-এ JOIN

২ · `prefetch_related`
উল্টো দিক / many-to-many — আলাদা কোয়েরি, তারপর জোড়া

৭ · `Count()`
গোনা ডেটাবেজে, পাইথনে নয়

৩ · `aggregate()` · ২ · `annotate()`
রিপোর্টের গড়/সর্বোচ্চ

সৎ ফাঁক: কোনো কোয়েরি-সংখ্যা কখনো **মাপা হয়নি** — django-debug-toolbar নেই, `connection.queries` কোথাও ব্যবহৃত নয়। ৫৭টি `select_related` লেখা হয়েছে কোড পড়ে, প্রোফাইল করে নয়।

শিক্ষকের নোট: বোর্ডে ১ + ৪০ লিখুন, তারপর প্রশ্ন করুন: "শিক্ষার্থী ৪০০ হলে?" — ৪০১ কোয়েরি। এটাই N+1; নামটাই বলে দেয় কেন: N সারি, প্রতিটির জন্য আরও ১।

ইতিহাস

১২টি মাইগ্রেশন — ডেটাবেজের ডায়েরি

৯ অ্যাপে `0001_initial` · ৩ অ্যাপে `0002` · core ও reports-এ শূন্য

ফাইল	কী যোগ করল
accounts/0001 · academics/0001 · students/0001 · attendance/0001 · lessons/0001 · assessments/0001 · audit/0001 · workspaces/0001	প্রথম স্কিমা (৮টি অ্যাপের মূল টেবিল)
billing/0001	প্ল্যান · সাবস্ক্রিপশন · ইনভয়েস
workspaces/0002_workspacebranding	লোগো · রঙ · কাস্টম ডোমেইন
academics/0002_coordinatorscope	সমন্বয়কের পরিধি
billing/0002_seed_plans_and_grandfather	**ডেটা** মাইগ্রেশন — ৩ প্ল্যান + পুরনো স্কুলের সাবস্ক্রিপশন

```
def forwards(apps, schema_editor):
    Plan = apps.get_model("billing", "Plan")
    ...
    # নতুন স্কুল "চিরকাল ফ্রি" হয়ে যাওয়া
    # ঠেকাতে প্রতিটি workspace-এ
    # একটি Subscription সারি বানানো হয়
```

নিয়ম: স্কিমা মাইগ্রেশন আর ডেটা মাইগ্রেশন আলাদা ফাইলে। একসাথে করলে রোলব্যাকের সময় ডেটা হারায়।

সৎ ফাঁক: `makemigrations --check` বা মাইগ্রেশন-লিন্টার কোথাও স্বয়ংক্রিয় নয়; সিআই নেই। ভুলে মডেল বদলে মাইগ্রেশন না বানালে ধরা পড়বে কেবল ডিপ্লয়ে করে।

শিক্ষকের নোট: জিজ্ঞেস করুন: "মাইগ্রেশন ফাইল হাতে লেখা যায়?" — যায়, এবং কখনো দরকার হয় (ডেটা রূপান্তর)। কিন্তু স্কিমার জন্য কখনো হাতে লিখবেন না; `makemigrations`-এর আউটপুট পড়ুন, তারপর বিশ্বাস করুন।

নিরাপত্তা

ব্যাকআপ — স্কিমা যত ভালো, হারানো ডেটা তত বেদনাদায়ক

`ops/backup.sh` · `ops/restore.sh` · দুটোই ২০ লাইনের কম

```
STAMP=$(date +%Y%m%d-%H%M%S)
pg_dump -Fc -f "$DEST/ims-$STAMP.dump"
tar -czf "$DEST/media-$STAMP.tar.gz" media/
find "$DEST" -name '*.dump'  -mtime +14 -delete
find "$DEST" -name '*.tar.gz' -mtime +14 -delete
```

দুটি জিনিস আলাদা: **ডেটাবেজ** (pg_dump) আর **ফাইল** (media/)। একটি ডাম্প দুইয়ের কোনোটিকেই ঢাকে না।

সিদ্ধান্ত	কারণ
`-Fc` ফরম্যাট	কাস্টম ফরম্যাট — `pg_restore` দিয়ে টেবিল ধরে ধরে ফেরানো যায়
১৪ দিন রাখা	দুই সপ্তাহের মধ্যে ভুল ধরা পড়ে; তার বেশি রাখলে জায়গা নষ্ট
`set -euo pipefail`	মাঝপথে ব্যর্থ হলে স্ক্রিপ্ট থামে, আধা-ব্যাকআপ নয়

সৎ ফাঁক: ব্যাকআপ **একই সার্ভারে** থাকে (`./backups`) — সার্ভার হারালে ব্যাকআপও হারায়। অফসাইট কপি নেই, রিস্টোর কখনো স্বয়ংক্রিয়ভাবে পরীক্ষা করা হয়নি, WAL আর্কাইভ/PITR নেই।

নিয়ম: **যে ব্যাকআপ কখনো ফেরানো হয়নি, সেটি ব্যাকআপ কি না জানা নেই।** তাই পরের কাজ — মাসে একবার রিস্টোর-অনুশীলন।

শিক্ষকের নোট: প্রশ্ন করুন: "১৪ দিনের পুরনো ফাইল মুছে যায় — কোন অবস্থায় সেটি বিপদ?" উত্তর: কেউ দুই সপ্তাহ ধরে ভুল ডেটা লিখলে, আর সেটি তখনই ধরা পড়লে — ভালো ব্যাকআপও ইতিমধ্যে মুছে গেছে।

সততা

এই ডেটাবেজে যা নেই — জেনে রাখুন

আটটি ফাঁক, প্রতিটির সঙ্গে "কীভাবে ধরা পড়বে"

যা নেই	ফল
`EXPLAIN ANALYZE`	ইনডেক্সগুলো অনুমানে বাছা; ধীর কোয়েরি অজানা
সারি-সংখ্যার পরিমাপ	স্কেল কত বড়, তার কোনো হিসাব নেই
Row Level Security	সীমানা অ্যাপে, ডেটাবেজে নয়
রেপ্লিকা / শার্ডিং	একটাই ডেটাবেজ, একটাই সার্ভার

যা নেই	ফল
৮ টেবিলে `workspace` কলাম	প্যারেন্ট-পথে শর্ত; ভুলে গেলে ডেটা মিশে
ইনভয়েস নম্বরে লক	সমান্তরাল সেভে `IntegrityError`
`clean()`-এর DB সংস্করণ	শেল/স্ক্রিপ্ট নিয়ম এড়াতে পারে
মাইগ্রেশন সিআই	মাইগ্রেশন ভুলে যাওয়া ধরা পড়ে ডিপ্লয়ে গিয়ে

৫টি সিদ্ধান্ত যা ঠিক আছে: workspace কলাম · শর্তসাপেক্ষ unique · SET_NULL বনাম PROTECT · ডেটা মাইগ্রেশন · অনুপস্থিত ≠ ০

৫টি কাজ যা এখনো বাকি: EXPLAIN · ইনভয়েস লক · RLS নয়তো টেস্ট-জাল · অফসাইট ব্যাকআপ · মাইগ্রেশন চেক

শিক্ষকের নোট: এই স্লাইডটি মুখস্থ করাতে হবে না — শুধু দেখান যে সৎ তালিকা দিলে বিশ্বাস বাড়ে, কমে না। শেষে জিজ্ঞেস করুন: "এর মধ্যে একটি ফাঁক আজ ঠিক করতে বললে কোনটি বাছতেন?" — উত্তর সাধারণত ইনভয়েস লক।

২

D2 — স্ট্যাক ও সংযোগ

ডেটাবেজ আছে, এখন সেটি কোন তার দিয়ে পর্দায় পৌঁছায় · ২৮ স্লাইড

গভীর ডেক · D2

স্ট্যাক ও সংযোগ

Python 3.13 · Django 6.1.2 · PostgreSQL · ৪টি settings মডিউল · ১০টি মিডলওয়্যার · ৮৫টি অ্যাপ রুট — একটি রিকোয়েস্ট কোথা দিয়ে যায়

মানচিত্র

এই ২৮ স্লাইডে কী আছে

চারটি ব্লক — স্ট্যাক থেকে শুরু, রিকোয়েস্টের পথ ধরে শেষ

১. স্ট্যাক

স্লাইড ৩–৬ · ৯টি প্যাকেজ · দুই requirements · কেন কোনো React/Celery/Redis নেই

প্রমাণ: pinned ভার্সন

২. কনফিগারেশন

স্লাইড ৭–১০ · ৪টি settings মডিউল · env() · prod-এর দুই গার্ড

প্রমাণ: RuntimeError আউটপুট

৩. রিকোয়েস্টের পথ

স্লাইড ১১–১৮ · ১০ মিডলওয়্যার · ৮৫ রুট · ৫২টি সাজানো ভিউ

প্রমাণ: রুট-গণনা

৪. পর্দা ও পরিবহন

স্লাইড ১৯–২৮ · কনটেক্সট · ৭৫ টেমপ্লেট · ৫ স্ট্যাটিক ফাইল · সেশন · কমান্ড · ফাঁক

প্রমাণ: ফাইল-গণনা

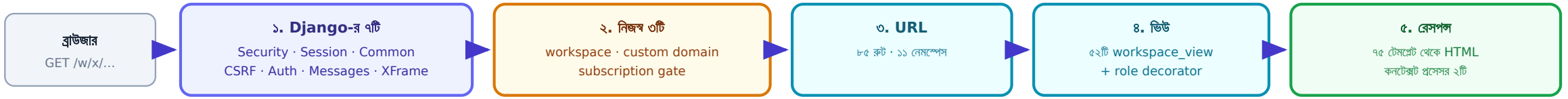
এই ডেকের প্রতিটি সংখ্যা `pip list`, `get_resolver()` আর `find` দিয়ে গোনা — কোনো স্লাইডে অনুমান নেই।

শিক্ষকের নোট: বলুন এই ডেকটি "চার্ট" নয়, "নকশা" — প্রতিটি সেটিং কেন সেখানে, সেটাই মূল প্রশ্ন। শিক্ষার্থীদের নিজের প্রজেক্টের `settings.py` খুলে একই প্রশ্ন করতে বলুন।

পথ

একটি GET রিকোয়েস্টের পুরো যাত্রা

ব্রাউজার থেকে পর্দা পর্যন্ত — ৫টি ধাপ, ১০টি দরজা



মিডলওয়্যারে যা তৈরি হয়

request.workspace · request.membership
request.branding · request.domain_workspace
ভিউ শরীরে আর কোনো লুকআপ নেই

ভিউ যা করে

scoping.py → দৃশ্যমান জিনিসের কোয়েরি
forms.validation → লিখবে কি না
৫২টি ভিউতে একই দুই প্রশ্ন

টেমপ্লেট যা পায়

nav_groups · brand · product · subscription
কনটেক্সট প্রসেসর থেকেই, প্রতি পাতায়
কোনো ভিউ আলাদা করে পাঠায় না

ক্রমটাই নিরাপত্তা: Auth মিডলওয়্যার আগে থাকায় আমাদের workspace মিডলওয়্যার request.user পায়

আর SubscriptionGate সবার পরে — কারণ সে request.workspace-এর উপর নির্ভর করে, যা তার আগের মিডলওয়্যার বানিয়েছে

ক্রম বদলালে ভেঙে যায়, কিন্তু এরর আসে না — শুধু কিছু রিকোয়েস্ট চুপচাপ ভুল পথে যায়

শিক্ষকের নোট: বোর্ডে শুধু "10 → 85 → 52" লিখুন। তারপর জিজ্ঞেস করুন: "মিডলওয়্যার আর ভিউ — কে কী জানে?" মিডলওয়্যার জানে "কে, কোন স্কুলে"; ভিউ জানে "কী দেখাবে"।

স্ট্যাক

৯টি প্যাকেজ — প্রতিটির একটি কাজ

সবগুলো ভার্সন পিন করা · `requirements.txt` + `requirements-prod.txt`

প্যাকেজ	কী কাজে
`Django==6.1.2`	ওয়েব ফ্রেমওয়ার্ক · ORM · টেমপ্লেট · admin
`psycopg[binary]==3.3.6`	PostgreSQL ড্রাইভার (v3, binary wheel)
`python-dotenv==1.2.4`	`.env` পড়া — কোডে সিক্রেট নয়
`Pillow==12.3.0`	লোগো/ছবি যাচাই (`ImageField`)
`openpyxl==3.1.5`	CSV/Excel এক্সপোর্ট
`reportlab==5.1.0`	রেজাল্ট-শিট PDF

শুধু প্রোডাকশনে	কী কাজে
`gunicorn==23.0.0`	WSGI সার্ভার — `runserver` নয়
`whitenoise==6.11.0`	স্ট্যাটিক ফাইল অ্যাপ থেকেই সার্ভ
`dj-database-url==3.0.1`	`DATABASE_URL` পার্সার — **এখনো ব্যবহৃত হয়নি**

সৎ ফাঁক: `dj-database-url` ইনস্টল করা আছে আর `.env.example`-এ `DATABASE_URL` লেখা আছে, কিন্তু কোডে কোথাও তা পড়া হয় না — `base.py` আলাদা `DB_NAME/DB_USER/…` ব্যবহার করে।

সব ভার্সন **ঠিক** লেখা (`==`), `>=` নয়। কারণ: "কাল অন্য ভার্সনে চলে গেল" — এই ব্যাখ্যা কোনো ডেমোতে কেউ মানে না।

শিক্ষকের নোট: শেষ বাক্যটি জোর দিন। শিক্ষার্থীদের অভ্যাস থাকে `pip install django` আর `pip freeze` না করা — ছয় মাস পরে নিজের প্রজেক্টই আর চলে না।

সিদ্ধান্ত

যা ইচ্ছাকৃতভাবে রাখা হয়নি

প্রতিটি বাদ দেওয়া একটা সিদ্ধান্ত — আর একটা সীমা

যা নেই	কেন বাদ	কখন দরকার হবে
React / SPA	২ ভূমিকা, ৮৫ রুট, ফর্ম-ভারী পর্দা — server-rendered HTML যথেষ্ট; একটি বিল্ড-ধাপ কম মানে একটি ভাঙার জায়গা কম	যখন অফলাইন বা রিয়েল-টাইম হাজিরা দরকার হবে
Celery / Redis	এখন কোনো কাজই ব্যাকগ্রাউন্ডে চলে না; PDF ১ সেকেন্ডে হয়	ইমেইল-বাল্ক, রিপোর্ট-জেনারেশন, রিমাইন্ডার
Docker	ডেমোটি `bash run.sh` — শিক্ষার্থীর মেশিনে Docker মানে আরও একটি ইনস্টল	দল বড় হলে, বা সার্ভার একাধিক হলে
GraphQL / REST API	এখন কনজিউমার কেবল ব্রাউজার; API বানালে সেটি রক্ষণাবেক্ষণের আরও একটি চুক্তি	মোবাইল অ্যাপ এলে (D2-এর বাইরে, ভবিষ্যৎ ডেকে)
মাইক্রোসার্ভিস	একই ডেটাবেজ, একই ট্রানজেকশন — ভাগ করলে JOIN-ই HTTP কল হয়ে যেত	দল ২০+ হলে, আর ভিন্ন স্কেলিং দরকার হলে
ক্যাশ (Memcached)	ডেটা ছোট, কোয়েরি indexed; ক্যাশ যোগ করলে "পুরনো ডেটা" বাগ আসে	ড্যাশবোর্ড ধীর হলে — মেপে দেখে

নিয়ম: **প্রতিটি বাদ দেওয়া জিনিসের জন্য একটি লিখিত শর্ত** — "কখন যোগ করব"। শর্ত ছাড়া বাদ দেওয়া মানে ভুলে যাওয়া।

শিক্ষকের নোট: শিক্ষার্থীরা প্রায়ই জিজ্ঞেস করে "React ছাড়া চলে?" — উত্তর: চলে, এবং এখানে সেটাই সঠিক সিদ্ধান্ত, কারণ সমস্যাটি UI-এর জটিলতা নয়, ডেটার নিয়মের জটিলতা। প্রতিটি সারিতে "কখন দরকার" কলামটি তুলে ধরুন।

দুই ফাইল

`requirements.txt` বনাম `requirements-prod.txt`

৬ + ৪ = ১০টি লাইন, কিন্তু সিদ্ধান্ত বড়

```
# requirements.txt  (দুই জায়গায়)
Django==6.1.2
psycopg[binary]==3.3.6
python-dotenv==1.2.4
Pillow==12.3.0
openpyxl==3.1.5
reportlab==5.1.0
```

```
# requirements-prod.txt
-r requirements.txt
gunicorn==23.0.0
whitenoise==6.11.0
dj-database-url==3.0.1
```

ডেমো, টেস্ট, ডেভেলপমেন্ট — সবখানে একই।

প্রথম লাইনটাই মূল: `-r` — base তালিকা আবার লিখতে হয় না, তাই দুটো কখনো আলাদা হয়ে যায় না।

প্রশ্ন	উত্তর
কেন gunicorn বেসে নেই?	শিক্ষার্থীর মেশিনে `runserver` চলে; gunicorn ইনস্টল থাকলে কেউ ভুল করে সেটি ডেমোতে চালাত
কেন whitenoise আলাদা?	ডেভে `runserver` নিজেই স্ট্যাটিক দেয়; প্রোডে সেই সুবিধা নেই
`pip freeze` চলে?	না — হাতে লেখা, তাই কোনটি সরাসরি দরকার আর কোনটি টানল, তা পরিষ্কার

শিক্ষকের নোট: ছোট প্রশ্ন: "prod ফাইলে `-r` না লিখে ৬টি লাইন কপি করলে কী হয়?" উত্তর: একদিন Django ভার্সন এক জায়গায় বাড়বে, অন্যটায় থাকবে — আর প্রোডে অচেনা বাগ।

কনফিগ

একটি `settings.py` নয় — চারটি মডিউল, ৩০০ লাইন

`config/settings/` · base ১৮৯ · dev ১৪ · prod ৭৪ · sqlite ২৩

মডিউল	লাইন	কী থাকে	কী বদলায়
base.py	১৮৯	সব সাধারণ সেটিং: ১১টি অ্যাপ, ১০টি মিডলওয়্যার, টেমপ্লেট, DB, i18n, লগিং	কিছুই না — এখানে কোনো গোপন মান নেই
dev.py	১৪	`DEBUG=True`, `ALLOWED_HOSTS=["*"]`	কনসোল ইমেইল, লোকাল CSRF
sqlite.py	২৩	শুধু ডেটাবেজ বদল — PostgreSQL নয়, ফাইল	শূন্য-সেটআপ ডেমোর জন্য
prod.py	৭৪	HTTPS, HSTS, cookie-র flags, WhiteNoise, SMTP, লগিং	নিরাপত্তার সব কঠোর মান

প্রতিটি ফাইলের প্রথম লাইন এক: `from .base import *`। তাই base-এ কিছু যোগ করলে চার জায়গায় পাওয়া যায় — কপি করে রাখতে হয় না।

শৃঙ্খলা: `dev.py` কখনো prod-এ ব্যবহার হয় না, `sqlite.py` কখনো ডিপ্লয়ে হয় না। কোনটি চলছে, তা `DJANGO_SETTINGS_MODULE` বলে।

শিক্ষকের নোট: প্রশ্ন করুন: "সব এক ফাইলে রাখলে কী হারাতাম?" উত্তর: নিরাপত্তা-মানগুলো আলাদা করে দেখার সুযোগ। prod.py খুললেই বোঝা যায় প্রোডাকশনে কী কী কঠোর নিয়ম আছে।

গোপন

এক ফাংশন `env()`, আর সিক্রেট কোথাও নেই

base.py-তে একটি পাসওয়ার্ডও লেখা নেই — সব `.env` থেকে

```
def env(key, default=None):
    value = os.environ.get(key, default)
    if value is None:
        raise RuntimeError(
            f"Missing required environment variable: {key}. "
            f"Copy .env.example to .env and fill it in.")
    return value

SECRET_KEY = env("SECRET_KEY", DEV_INSECURE_KEY)
DEBUG = env_bool("DEBUG", False)
ALLOWED_HOSTS = env_list("ALLOWED_HOSTS", "127.0.0.1,localhost")
```

ফাংশন	ব্যবহার
`env()`	স্ট্রিং; না পেলে স্পষ্ট এরর, নীরব None নয়
`env_bool()`	`"1/true/yes/on"` → True
`env_list()`	কমা দিয়ে ভাগ করা তালিকা

কেন স্পষ্ট এরর? `os.environ.get()` চুপচাপ `None` দিলে অ্যাপ চালু হয়, তারপর অর্ধেক কাজ করে — ভুলটা তখন অনেক দূরে খুঁজতে হয়।

.env.example-এ ৫টি কী — কিন্তু কোডে পড়া হয় ৪টির বেশি: `DB_NAME/DB_USER/DB_PASSWORD/DB_HOST/DB_PORT` ডকুমেন্টেড নয়। নতুন কেউ সেটআপ করতে গেলে অনুমান করতে হয়।

শিক্ষকের নোট: `.env` কখনো Git-এ যায় না — `.gitignore`-এ আছে। আর `.env.example` যায়, কারণ তাতে কোনো আসল মান নেই। এই দুই ফাইলের পার্থক্যটাই নতুন ডেভেলপারের প্রথম পাঠ।

গার্ড

prod.py চালুই হবে না — দুটি ভুল সেটিং পেলে

সেটিং "ঢিলা করে" ডিপ্লয় ঠিক করার সংস্কৃতি বন্ধ

```
$ SECRET_KEY=dev-insecure-key-... \
  DJANGO_SETTINGS_MODULE=config.settings.prod \
  python -c "import django; django.setup()"

RuntimeError: Refusing to start: SECRET_KEY is the
development default. Generate one with:
python -c "import secrets; print(
    secrets.token_urlsafe(50))"
```

```
$ ALLOWED_HOSTS='*' ... django.setup()

RuntimeError: ALLOWED_HOSTS must list real
hostnames in production; '*' is not allowed.
```

সঠিক সেটিং দিয়ে Django-র নিজের পরীক্ষা

```
$ SECRET_KEY=$(python -c "import secrets;
    print(secrets.token_urlsafe(50))")
$ ALLOWED_HOSTS=ims.example.com \
  python manage.py check --deploy

System check identified no issues
(0 silenced).
```

ফল: `check --deploy`-এর কোনো নিরাপত্তা-সতর্কতা নেই — HSTS, cookie flags, SSL redirect সব সেট।

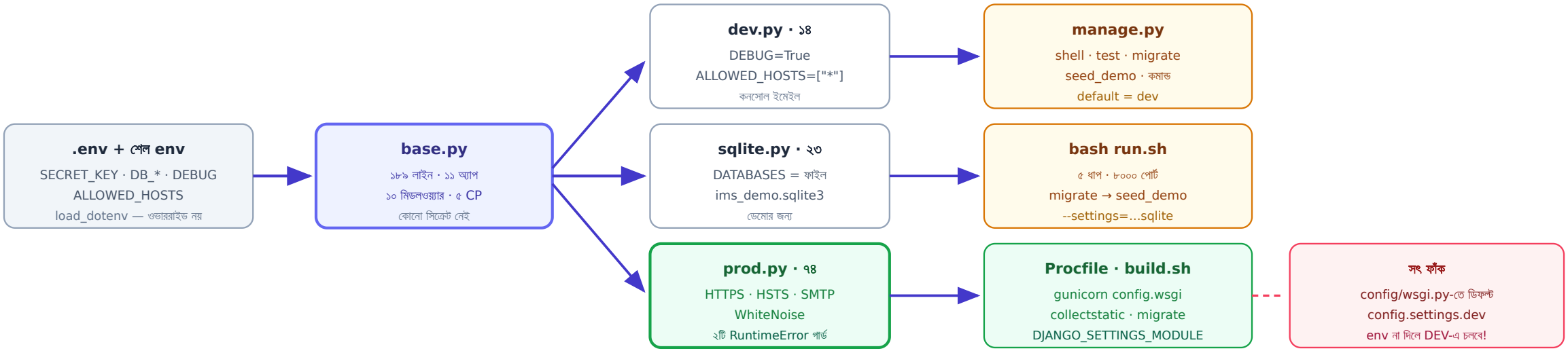
সৎ নোট: এই দুটি গার্ড কোডে লেখা, কিন্তু কোনো টেস্টে নেই — কেউ ভুলে গার্ড সরিয়ে দিলেকেবল CI নয়, কেউ ধরবে না।

শিক্ষকের নোট: দুই গার্ডের যুক্তি এক: "ডিপ্লয় কাজ করছে না" — এর সবচেয়ে সহজ সমাধান সেটিং ঢিলা করা, আর সেটাই সবচেয়ে বিপজ্জনক। এখানে সেই পথ বন্ধ।

এন্ট্রি

কোন দরজা দিয়ে ঢুকলে কোন সেটিং আরম্ভ হয়

একই কোড, চারটি প্রবেশপথ — ভুল প্রবেশপথ মানে ভুল নিরাপত্তা



শিক্ষকের নোট: লাল বাক্সটি পড়ুন শেষে: `wsgi.py`-তে ডিফল্ট dev। অর্থাৎ `DJANGO_SETTINGS_MODULE` সেট না করে gunicorn চালালে ডেমো-সেটিংসে প্রোডাকশন চলবে — এটি প্রকৃত ঝুঁকি, ঠিক করা বাকি।

দরজা

১০টি মিডলওয়্যার, ক্রম অনুযায়ী

উপরে থেকে নিচে — ভেতরে যাওয়ার সময় এই ক্রমে, ফেরার সময় উল্টো

#	মিডলওয়্যার	এক লাইনে কাজ
১–৭	`Security` · `Session` · `Common` · `CSRF` · `Auth` · `Messages` · `XFrame`	Django-র নিজের সাতটি — নিরাপত্তা, সেশন, `request.user`, মেসেজ, ক্লিকজ্যাকিং
৮	`billing.CustomDomainMiddleware`	হোস্ট দেখে প্রতিষ্ঠান চেনে; `Vary: Host` বসায়
৯	`workspaces.CurrentWorkspaceMiddleware`	সেশন/সদস্যতা দেখে `request.workspace` ঠিক করে
১০	`billing.SubscriptionGateMiddleware`	সাবস্ক্রিপশন শেষ হলে লিখতে দেয় না, পড়তে দেয়

প্রোডাকশনে ১১: `prod.py` তালিকার **শুরুতে** `WhiteNoiseMiddleware` যোগ করে — নাহলে স্ট্যাটিক ফাইল পাওয়া যেত না।

ক্রম ভাঙলে: Auth-এর আগে workspace মিডলওয়্যার বসলে `request.user` নেই → সবাই অ্যানোনিমাস, কিন্তু এরর নেই — শুধু ৪০৪।

শিক্ষকের নোট: বোর্ডে তালিকাটি নম্বর দিয়ে লিখুন এবং তীর দিন নিচের দিকে। তারপর জিজ্ঞেস করুন: "উল্টো পথে ফেরার সময় কোনটি আগে চলে?" — সেটাই response-এ হেডার যোগ করার জায়গা।

কেন

workspace ঠিক করা ডেকোরেটরের কাজ নয় — মিডলওয়্যারের

একবার ঠিক করলে ৫২টি ভিউ আর ভাবতে হয় না

প্রশ্ন	ডেকোরেটরে করলে	মিডলওয়্যারে করলে (যা করা হয়েছে)
কত জায়গায় লিখতে হয়	প্রতিটি ভিউতে, অথবা প্রতিটি ভিউতে ডেকোরেটর লাগাতে হয়	একবার, `MIDDLEWARE`-এ
ভুলে যাওয়ার সম্ভাবনা	নতুন ভিউ যোগ করলে ডেকোরেটর বাদ পড়তে পারে	শূন্য — সব রিকোয়েস্ট একই পথে যায়
টেমপ্লেটে পাওয়া যায়?	না, কনটেক্সট আলাদা করে পাঠাতে হয়	হ্যাঁ, `request.workspace` কনটেক্সট প্রসেসরে যায়
প্রতি রিকোয়েস্টে খরচ	একই কোয়েরি, তবু প্রতিটি ভিউতে আলাদা	একটি indexed কোয়েরি, একবার

তবে **ভূমিকা-যাচাই এখনো ডেকোরেটরে** (`@workspace_view(*TEACHING)`) — কারণ "কে কী করতে পারবে" ভিউ-ভিত্তিক সিদ্ধান্ত, আর "কে কোন স্কুলে" রিকোয়েস্ট-ভিত্তিক।

এই ভাগটাই মূল শিক্ষা: **প্রেক্ষাপট (context) মিডলওয়্যারে, অনুমতি (permission) ডেকোরেটরে।**

শিক্ষকের নোট: শিক্ষার্থীদের বলুন নিজের প্রজেক্টে একটি ভিউ বানিয়ে দেখতে — ডেকোরেটর ছাড়া। তারপর জিজ্ঞেস করুন "এটি কোন স্কুলের ডেটা দেখছে?" উত্তর না থাকলেই বুঝবে মিডলওয়্যার কেন দরকার।

কেস ১

CurrentWorkspaceMiddleware — এবং একটি আসল বাগ

সেশন → সদস্যতা → না মিললে ভূমিকা-অনুযায়ী ডিফল্ট

```
ws_id = request.session.get(SESSION_KEY)
qs = Membership.objects.filter(user=request.user,
                               is_active=True)
if ws_id:
    membership = qs.filter(workspace_id=ws_id).first()
    if membership:
        return membership

# ডিফল্ট: সর্বোচ্চ ভূমিকা, তারপর প্রতিষ্ঠান
membership = (qs.order_by(ROLE_PRIORITY,
                          KIND_PRIORITY,
                          "workspace__name").first())
```

অবস্থা	ফল
স্কুলের অ্যাডমিন + অন্যত্র সমন্বয়ক	স্কুলেই খোলে — ভূমিকা-ক্রম দেখে
স্কুলের শিক্ষক + নিজের ব্যক্তিগত স্পেস	স্কুলই আগে — ব্যক্তিগতটি "পিছনের ঘর"
কোনো সদস্যতা নেই	`request.workspace = None` → ভিউ ৪০৪

আসল বাগ: প্রথমে রেজলভিং ছিল "lazy"। কিন্তু `lazy_obj is None` সবসময় False — তাই workspace-হীন ইউজার None-চেক পাশ করে ভেতরে ঢুকে যেত।

সমাধান: রিকোয়েস্টের শুরুতে **সরাসরি** রেজলভ (eager) — প্রতি রিকোয়েস্টে একটি ছোট indexed কোয়েরি।

শিক্ষকের নোট: এই বাগটি শিক্ষার্থীদের সবচেয়ে বেশি কাজে লাগবে — "lazy মানে ফ্রি নয়"। lazy অবজেক্টে `is None` কাজ করে না; শুধু `if obj` ডাকলে সত্য-মিথ্যা বোঝা যায়। এটাই ছিল আসল ফাঁদ।

কেস ২

CustomDomainMiddleware — হোস্টই প্রতিষ্ঠানের পরিচয়

school.example.com → সেই স্কুলের লোগো ও নাম, লগইন পাতাতেই

```
host = request.get_host().split(":")[0].lower()
branding = (WorkspaceBranding.objects
    .select_related("workspace")
    .filter(custom_domain__iexact=host,
            domain_verified=True,
            workspace__is_active=True)
    .first())

if branding:
    request.domain_workspace = branding.workspace
    if host not in settings.ALLOWED_HOSTS:
        settings.ALLOWED_HOSTS += [host]
```

বিবরণ	কেন এভাবে
`domain_verified=True`	যাচাই না হলে ডোমেইন ডাকনাম মাত্র — টাইপ করা হোস্ট বিশ্বাস নয়
`iexact`	DNS নাম case-insensitive; `Example.com` = `example.com`
`ALLOWED_HOSTS`-এ যোগ	ডোমেইনটি ব্যবহারকারীর নিজের, সার্ভার-তালিকায় আগে থেকে নেই
`patch_vary_headers(["Host"])`	একই URL, ভিন্ন স্কুল — ক্যাশে গুলিয়ে ফেলতে পারে না
`OperationalError` ধরা	মাইগ্রেশনের আগে টেবিল না থাকলেও সাইট চলে

প্রমাণ: `test_domain_cannot_be_stolen_by_a_second_institution` · `test_unverified_domain_is_ignored` · `test_verified_domain_shows_that_institution_on_the_login_page` — তিনটি টেস্ট এই মিডলওয়্যারের তিনটি নিয়ম।

শিক্ষকের নোট: `Vary: Host` বাদ দিলে কী হয় — জিজ্ঞেস করুন। উত্তর: এক স্কুলের লগইন পাতা ক্যাশে হয়ে আরেক স্কুলের ব্যবহারকারী সেটাই দেখতে পারে। ক্যাশের বাগ চোখে দেখা যায় না, তাই নিয়মটা লিখে রাখা।

কেস ৩

SubscriptionGateMiddleware — লিখা বন্ধ, ডেটা অটুট

পড়া কখনো বন্ধ হয় না; মুছে ফেলা কখনোই হয় না

নিয়ম	বাস্তবে
`SAFE_METHODS`	GET/HEAD/OPTIONS/TRACE কখনো ব্লক হয় না
`ALLOW_PREFIXES`	`/accounts/` `/renew/` `/platform/` `/static/` `/media/` `/admin/` `/healthz` `/workspaces/`
ORG_ADMIN	`/renew/?blocked=1`-এ পাঠানো হয়
অন্য সবাই	ড্যাশবোর্ডে ফেরত + বার্তা: "আপনার কাজ হারায়নি"

প্রথমে ভুল ছিল: মিডলওয়্যার সেশনের workspace দেখত, URL-এর নয়। এক শিক্ষক নিজের ব্যক্তিগত স্পেসে থাকা অবস্থায় মেয়াদোত্তীর্ণ স্কুলে POST করলে গেট পাশ করে যেত। সমাধান: `_workspace_in_url()` — পথ থেকে slug পড়া।

```
if request.method in SAFE_METHODS:
    return self.get_response(request)

workspace = (self._workspace_in_url(request)
             or getattr(request, "workspace", None))
if workspace is None or any(
        path.startswith(p) for p in ALLOW_PREFIXES):
    return self.get_response(request)

if is_writable(workspace):
    return self.get_response(request)
# ... রিনিউ পাতায় পাঠানো
```

সুপারইউজার ছাড় নেই: প্ল্যাটফর্ম-অ্যাডমিনও যদি ওই স্কুলের অ্যাডমিন হন, তবু লিখতে পারবেন না — কেবল `/platform/` খোলা।

শিক্ষকের নোট: এখানে দুটি নীতি একসাথে: (১) বিলিং মানে ডেটা জিম্মি নয়, (২) অনুমতি URL দেখে, সেশন দেখে নয়। দ্বিতীয়টি না থাকলে এক স্কুল অন্য স্কুলের মেয়াদে ব্লক হয়ে যেত।

ঠিকানা

২৫৩টি URL — কে কোথায় থাকে

`get_resolver()` দিয়ে গোনা · admin ১৬৭ · অ্যাপ ৮৫ · media ১

নেমস্পেস	রুট	উদাহরণ
`billing`	১৮	pricing · renew · console
`academics`	১৩	class_list · term_create
`lessons`	১৩	syllabus · homework_detail
`accounts`	৯	login · signup · password_reset
`assessments`	৯	exam_detail · marksheet
`students` · `reports`	৭ · ৭	student_list · export_students
`attendance`	৪	take · register · summary
`workspaces` · `audit` · `dashboard`	২ · ২ · ১	switch · log_list · home

১১টি `include()`, সবই `""`-এ: `config/urls.py`-তে প্রতিটি অ্যাপের নিজের ফাইল, কিন্তু prefix নেই — পথের প্রথম অংশ প্রতিটি অ্যাপ নিজেই ঠিক করে।

ফল: কে কোন পথ দখল করছে, তা এক নজরে দেখতে `config/urls.py` যথেষ্ট নয় — ১১টি ফাইল খুলতে হয়। এটি জেনে রাখা দুর্বলতা (`docs/url-map.md` নেই)।

সৎ ফাঁক: ১৬৭টি admin URL কোনো ডিজাইন নয় — Django-র admin নিজেই বানায়। তাই "৮৫টি রুট" বলাটাই সৎ।

শিক্ষকের নোট: জিজ্ঞেস করুন: "একই পথ দুই অ্যাপে লিখলে কী হয়?" উত্তর: আগে `include` করা অ্যাপটি জেতে — তাই ক্রমও একটা সিদ্ধান্ত।

নাম

প্রতিটি ঠিকানার একটি নাম — `reverse()`

টেমপ্লেটে `{% url %}`, কোডে `reverse()`, কখনো হাতে লেখা পথ নয়

```
# apps/attendance/urls.py
app_name = "attendance"
urlpatterns = [
    path("w//attendance/",
         views.take, name="take"),
    path("w//attendance/summary/",
         views.summary, name="summary"),
]
```

প্রতিটি ভেতরের পথেই `w/<slug>/` — অর্থাৎ প্রতিটি রুট নিজেই বলে "কোন স্কুল"।

যেখানে	কীভাবে
টেমপ্লেট	`{% url 'attendance:take' slug=ws.slug %}`
ভিউ	`redirect("attendance:summary", slug=…)`
রিডাইরেক্ট	`reverse("billing:renew", kwargs=…)`
মিডলওয়্যার	`reverse(...)` — হার্ডকোড নয়, কারণ গেট সঠিক ঠিকানায় পাঠায়

লাভ: পথ বদলালে এক জায়গায় বদলায় — ভাঙা লিংকের সন্ধানে ৭৫ টেমপ্লেট ঘুরতে হয় না।

কিন্তু: `w/<slug>/` হাতে লেখা প্রতিটি ভিউতে — অর্থাৎ স্কুল-সহ পথ আর স্কুল-ছাড়া পথ একসাথে আছে (`/platform/`, `/accounts/`, `/pricing/`)। কেন, সেটি পরের স্লাইড।

শিক্ষকের নোট: একটি ভাঙা লিংকের গল্প বলুন: URL হাতে লেখা থাকলে এক অক্ষর বদলালেই ৪০৪, আর টেস্ট না থাকলে ধরা পড়ে ব্যবহারকারীর কাছে। `reverse()` সেই ভুলটাই অসম্ভব করে।

সাজ

৫২টি ভিউ একই সাজে — `@workspace_view`

প্রতিটি workspace-ভিউতে তিনটি প্রশ্ন, একটাই উত্তরদাতা

```
@workspace_view(*TEACHING)
def take(request, slug):
    ...

# decorator যা করে:
# ১. লগইন নেই → /accounts/login/?next=...
# ২. ওই স্কুলে সদস্য নয় → 404
# ৩. ভূমিকা অনুমোদিত নয় → 403
# ৪. পাশ হলে request.workspace + membership
```

ভূমিকা-সেট	কারা
`TEACHING`	ORG_ADMIN · COORDINATOR · TEACHER (৩২টি ভিউ)
`ALL_ROLES`	সবাই + STUDENT (৮টি ভিউ)
`OWNERS`	ORG_ADMIN (১১টি ভিউ)
`MANAGERS`	ORG_ADMIN · COORDINATOR (১টি)

তিনটি স্তর: সদস্যতা (404) → ভূমিকা (403) → বস্তু-স্তরের অনুমতি, যেমন `teacher_can_access_class()`। তৃতীয়টি ছাড়া একজন শিক্ষক অন্য ক্লাসের নম্বর দেখতে পারতেন।

কেন ৪০৪ আর ৪০৩ আলাদা: সদস্য না হলে "এই স্কুল আছে কি না" জানানোই তথ্য ফাঁস; তাই ৪০৪। সদস্য কিন্তু ভূমিকা ভুল হলে ৪০৩ — কেন না, সেটা বলা নিরাপদ।

শিক্ষকের নোট: জিজ্ঞেস করুন: "একজন ছাত্র `attendance:take` খুললে কী হবে?" — ৪০৩, কারণ সে সদস্য (৪০৪ নয়) কিন্তু ভূমিকা TEACHING নয়।

কনটেক্সট

প্রতি পাতায় যে ৫টি জিনিস নিজে থেকেই আসে

৩টি Django-র, ২টি নিজস্ব — কোন ভিউ আলাদা করে পাঠায় না

প্রসেসর	যা দেয়	কোথায় কাজে লাগে
`request`	`request`	টেমপ্লেটে পথ, হোস্ট, মিডলওয়্যারের হাতের তথ্য
`auth`	`user` · `perms`	`{% if user.is_authenticated %}`
`messages`	`messages`	ফ্ল্যাশ বার্তা — ৫টি শেল-টেমপ্লেটে একই ব্লক
`workspace_context`	`nav_groups` · `nav_role` · `active_workspace` · `my_memberships`	সাইডবারের মেনু — ভূমিকা অনুযায়ী আলাদা
`branding_context`	`brand` · `product` · `subscription` · `platform_mode`	লোগো, রঙ, পণ্যের নাম, বিলিং ব্যানার

brand একটি dict, মডেল নয়: নতুন স্কুলের ব্র্যান্ডিং সারি না থাকলেও টেমপ্লেট কখনো ক্র্যাশ করবে না — ডিফল্ট পণ্য-নাম দেখাবে।

দাম: এই দুই প্রসেসর প্রতি রিকোয়েস্টে একাধিক কোয়েরি চালায় (সদস্যতা, ব্র্যান্ডিং, সাবস্ক্রিপশন) — লগইন-পাতায়ও চলে, যদিও সেখানে দরকার নেই।

শিক্ষকের নোট: শিক্ষার্থীদের জিজ্ঞেস করুন: "সাইডবারের মেনু কোন ভিউ পাঠায়?" উত্তর — কেউ না। এটাই কনটেক্সট প্রসেসরের লাভ: ৫২টি ভিউতে একই কোড লিখতে হয় না।

পর্দা

৭৫টি টেমপ্লেট — সব এক জায়গায়

`templates/` · অ্যাপের ভেতরে শূন্য টেমপ্লেট · `APP_DIRS=True` তবু ব্যবহার হয়নি

ফোল্ডার	কী
`templates/app_base.html`	অ্যাপের শেল (সাইডবার + টপবার)
`templates/<app>/…`	প্রতি অ্যাপের পাতা
`templates/partials/`	পুনর্ব্যবহারযোগ্য টুকরো
`templates/platform/`	প্ল্যাটফর্ম কনসোল
`templates/public/`	মার্কেটিং পাতা (pricing, signup)
`templates/registration/`	লগইন, পাসওয়ার্ড রিসেট

কেন কেন্দ্রীভূত: ১১টি অ্যাপে টেমপ্লেট ছড়িয়ে দিলে `{% extends "app_base.html" %}` খুঁজতে ১১টি ফোল্ডার লাগত; আর একই নামের দুই টেমপ্লেট সংঘর্ষ করত।

দাম: অ্যাপ আলাদা করে সরানো এখন কঠিন — টেমপ্লেট তার বাইরে থাকে। মডুলারিটি এখানে ফোল্ডার-স্তরে, প্যাকেজ-স্তরে নয়।

প্রমাণ: `find apps -name "*.html"` → **০** · `find templates -name "*.html"` → **৭৫**

শিক্ষকের নোট: দুই পদ্ধতিরই দাম আছে — অ্যাপ-ভেতরে টেমপ্লেট মানে অ্যাপ সরানো সহজ, কেন্দ্রীভূত মানে ভাগ করা শেল সহজ। এখানে দ্বিতীয়টি বেছেছেন, কারণ শেল-ই বড় কাজ।

স্ট্যাটিক

৫টি ফাইল, ৫৩৪ KB — এবং একটি CDN-ও নেই

`static/vendor/` — Bootstrap আর আইকন নিজের ভেতরে রাখা

ফাইল	আকার
`vendor/bootstrap.min.css`	২২৭ KB
`vendor/bootstrap-icons.woff2`	১২৭ KB
`vendor/bootstrap-icons.css`	৮৪ KB
`vendor/bootstrap.bundle.min.js`	৭৯ KB
`css/ims.css`	৮ KB

কেন CDN নয়: স্কুলের নেটওয়ার্ক প্রায়ই বন্ধ-নিয়ন্ত্রিত; CDN ব্লক হলে পর্দা ভেঙে পড়ে। নিজের ফাইল মানে নিজের নিয়ন্ত্রণ।

দাম: আপডেট নিজে টানতে হয় — Bootstrap-এর নিরাপত্তা-প্যাচ স্বয়ংক্রিয়ভাবে আসে না। ভার্সনও কোথাও লেখা নেই।

প্রোডাকশনে: `collectstatic` → `staticfiles/` ; WhiteNoise manifest + ৩০ দিন ক্যাশ; ফাইলের নামে হ্যাশ, তাই পুরনো ক্যাশ কখনো ভুল ফাইল দেয় না।

সৎ ফাঁক: `staticfiles/` ফোল্ডার `.gitignore`-এ আছে, কিন্তু কোন Bootstrap ভার্সন, তা কোথাও লেখা নেই — আপডেট করতে গেলে প্রথমেই অনুমান করতে হবে।

শিক্ষকের নোট: "CDN মানে দ্রুত" — এই ধারণাটি প্রশ্ন করুন। বাংলাদেশের স্কুল-নেটওয়ার্কে CDN ব্লক হওয়ার ঘটনা বাস্তব; ৫৩৪ KB নিজের সার্ভার থেকে দিলে ৫০ms বেশি লাগে, কিন্তু পর্দা কখনো ভাঙে না।

সেশন

সেশন ও কুকি — ৮ ঘণ্টা, আর ৩টি পতাকা

"কে লগইন করা আছে" আর "কোন স্কুলে আছে" — দুটোই সেশনে

সেটিং	মান
`SESSION_COOKIE_AGE`	৮ ঘণ্টা (এক কর্মদিবস)
`SESSION_SAVE_EVERY_REQUEST`	`True` — ব্যবহারে মেয়াদ বাড়ে
`SESSION_EXPIRE_AT_BROWSER_CLOSE`	`False`
প্রোডাকশনে	`Secure` · `HttpOnly` · `SameSite=Lax`

সেশন-কী: `active_workspace_id` — কোন স্কুলে আছেন, সেটাই সেশনে থাকে।

একটি আসল ফাঁদ: সেশনের workspace আর URL-এর workspace আলাদা হতে পারে। এই পার্থক্যই একবার SubscriptionGate-এ বাগ তৈরি করেছিল (আগের স্লাইড)।

কেন ৮ ঘণ্টা: স্কুলে একটি শিফট ৬–৭ ঘণ্টা; মাঝপথে লগআউট হলে শিক্ষক হাজিরার মাঝখানে আটকে যাবেন। ৩০ দিন রাখলে শেয়ার্ড কম্পিউটারে ঝুঁকি।

CSRF: আলাদা কুকি, আলাদা টোকেন; ব্যর্থ হলে নিজের ভিউ `apps.core.views.csrf_failure` — কুৎসিত ডিফল্ট পাতা নয়।

শিক্ষকের নোট: শেয়ার্ড কম্পিউটারের কথা বলুন — স্কুলের ল্যাবে একজন লগআউট না করে চলে গেলে পরেরজন তার নামে হাজিরা দিতে পারে। এটাই সেশন-মেয়াদের আসল প্রশ্ন, প্রযুক্তিগত নয়।

ইমেইল

একই কোড, দুই ব্যাকএন্ড — কনসোল নাকি SMTP

ডেমোতে ইমেইল টার্মিনালে, প্রোডাকশনে আসল সার্ভারে

পরিবেশ	ব্যাকএন্ড ও ফল
dev · sqlite	`console.EmailBackend` — চিঠি টার্মিনালে ছাপা হয়, কেউ পায় না
prod	`smtp.EmailBackend` · পোর্ট ৫৮৭ · STARTTLS
প্রেরক	`DEFAULT_FROM_EMAIL` — পণ্যের নাম থেকে, হার্ডকোড নয়

কোন পথে ইমেইল যায়: পাসওয়ার্ড রিসেট · স্কুল-অনুরোধের স্বীকৃতি · আমন্ত্রণ।

প্রমাণ: `test_email_is_sent_and_link_sets_a_new_password` — মেইল বেরোয়, লিংকের টোকেন কাজ করে, পাসওয়ার্ড বদলায়। তিন ধাপ এক টেস্টে।

সৎ ফাঁক: SMTP পাঠানোর কোনো রিট্রাই নেই — পাঠানো ব্যর্থ হলে ইমেইল হারায়, ব্যবহারকারী শুধু "লিংক আসেনি" বলেন। Celery/কিউ ছাড়া এটাই সীমা।

টেস্টে আসল মেইল যায় না: Django-র `mail.outbox` ব্যবহার হয় — ইন্টারনেট ছাড়াই টেস্ট চলে।

শিক্ষকের নোট: কনসোল ব্যাকএন্ড শিক্ষার্থীদের প্রিয় — কারণ লিংকটা সরাসরি দেখতে পান। বলুন: "ডেমোতে ইমেইল মানে টার্মিনাল; প্রোডাকশনে ইমেইল মানে দায়িত্ব।"

কমান্ড

৪টি ম্যানেজমেন্ট কমান্ড, ৪০৩ লাইন

যে কাজ একবার করতে হয়, তার জন্য ভিউ নয় — কমান্ড

কমান্ড	লাইন	কী করে
`seed_demo`	২৮১	demo ডেটা: স্কুল, ৪০ ছাত্র, শ্রেণি, হাজিরা, পরীক্ষা — `run.sh` নিজে ডাকে
`billing_report`	৫১	কোন স্কুল কত seats ব্যবহার করছে, কত টাকা বাকি — টার্মিনালে ছাপা
`sync_domains`	৩৭	যাচাই করা কাস্টম ডোমেইন নিয়ে `ALLOWED_HOSTS`-এর তালিকা ছাপে/মেলায়
`seed_plans`	৩৪	তিনটি প্ল্যান তৈরি/আপডেট — মাইগ্রেশন ছাড়াই পুনরায় চালানো যায়

কেন ভিউ নয়: এই কাজের জন্য লগইন, সেশন বা CSRF লাগে না — আর ভুল করে কোনো ব্রাউজার থেকে চালানোও উচিত নয়।

সবচেয়ে বড়িটি `seed_demo` : ২৮১ লাইন — ডেমোর পুরো গল্প এখানেই লেখা; তাই ডেমো রিসেট করা (`--reset`) মানে এক কমান্ডে নতুন গল্প।

সৎ ফাঁক: `seed_demo`-র কোনো টেস্ট নেই। কেউ ভেঙে দিলে টের পাবেন কেবল ডেমো চালিয়ে।

শিক্ষকের নোট: প্রশ্ন করুন: "ডেটা বানানোর কাজটা ভিউতে রাখলে কী সমস্যা?" উত্তর: লগইন লাগত, আর ভুল করে প্রোডাকশনে কেউ ক্লিক করলে ডেমো-ডেটা ঢুকে যেত।

সংযোগ

ডেটাবেজ-সংযোগ: একবার খুলে রাখা, ৬০ সেকেন্ড

`CONN_MAX_AGE=60` · `CONN_HEALTH_CHECKS=True` · psycopg 3

```
"ENGINE": "django.db.backends.postgresql",
"NAME": env("DB_NAME", "ims_dev"),
"USER": env("DB_USER", "ims_user"),
"HOST": env("DB_HOST", "127.0.0.1"),
"PORT": env("DB_PORT", "5432"),
"CONN_MAX_AGE": 60,
"CONN_HEALTH_CHECKS": True,   # prod-এ
```

সেটিং	কেন
CONN_MAX_AGE ৬০	প্রতি রিকোয়েস্টে নতুন সংযোগ নয় — হ্যান্ডশেক বাঁচে
HEALTH_CHECKS	পুরনো সংযোগ মৃত হলে ব্যবহারের আগেই ধরা পড়ে
gunicorn workers ৩	worker প্রতি আলাদা সংযোগ-পুল; ৪টি worker মানে ৪০+ সংযোগ

সীমা: পুলিং লাইব্রেরি (pgbouncer) নেই — worker বাড়ালে PostgreSQL-এর `max_connections` -এ আটকে যাবে।

তিনটি পরিবেশ, একই কোড: PostgreSQL (`dev` / `prod`) · SQLite (`sqlite` — শুধু ডেমো)। SQLite-তে `CONN_MAX_AGE` -এর অর্থ আলাদা, তাই ডেমোর গতি আর প্রোডের গতি কখনো এক নয়।

শিক্ষকের নোট: শিক্ষার্থীদের জিজ্ঞেস করুন: "worker ৩ থেকে ৩০ করলে কী ভাঙবে?" উত্তর: সংযোগের সংখ্যা — আর সেটি ডেটাবেজ-সার্ভারেই ধরা পড়ে, অ্যাপে নয়।

তার

তিনটি স্ক্রিপ্ট যা সব জোড়া লাগায়

`run.sh` (ডেমো) · `build.sh` (বিল্ড) · `Procfile` (চালু)

run.sh — ৫ ধাপ

```
১. python3 আছে? সংস্করণ ≥ 3.12?
২. .venv না থাকলে বানাও
৩. pip install -r requirements.txt
৪. migrate + seed_demo (sqlite)
৫. runserver 127.0.0.1:8000
```

প্রতিটি ধাপে স্পষ্ট বার্তা — "কেন ব্যর্থ হলো" না বলে থামে না।

build.sh

```
pip install -r requirements-prod.txt
python manage.py collectstatic --noinput
python manage.py migrate --noinput
```

Procfile

```
release: migrate --noinput
web: gunicorn config.wsgi --workers 3 --timeout 60
```

প্রশ্ন	উত্তর
কেন migrate দুই জায়গায়?	`build.sh`-এ বিল্ডের সময়, `release`-এ চালুর সময় — কোনও PaaS একটিকে ডাকে, অন্যরা অন্যটি
কেন `--noinput`?	স্বয়ংক্রিয় পরিবেশে প্রশ্ন করার কেউ নেই; প্রশ্ন এলে ডিপ্লয় ঝুলে থাকে
কেন ৩ worker?	ডেমোর লোডে যথেষ্ট; বাড়ানো হিসাব করে, অনুমানে নয়

শিক্ষকের নোট: "একই কাজ দুই জায়গায়" দেখে শিক্ষার্থীরা ভুল ভাবে — এখানে ইচ্ছাকৃত, কারণ ভিন্ন প্ল্যাটফর্ম ভিন্ন ধাপ ডাকে। বলুন: পুনরাবৃত্তি সবসময় দোষ নয়, অজানা ক্রম থেকেও খারাপ।

সততা

সংযোগ-স্তরের ৮টি ফাঁক — প্রতিটির একটি সংশোধন

লুকানো নেই, কারণ এগুলোই পরের কাজের তালিকা

ফাঁক	আজকের ঝুঁকি	সংশোধন
`/healthz` এন্ডপয়েন্ট নেই	গেটে অনুমতি আছে, ঠিকানা নেই — মনিটরিং কী দেখবে?	একটি ভিউ যোগ, DB-চেক সহ
`DATABASE_URL` অপ্রয়োগ	PaaS-এর ডিফল্ট চলবে না; `dj-database-url` অচল প্যাকেজ	`prod.py`-তে URL পার্স করা
`wsgi.py` ডিফল্ট `dev`	env ভুলে গেলে প্রোডাকশন ডেমো-সেটিংসে	ডিফল্ট `prod`, ওভাররাইড env দিয়ে
`.env.example`-এ `DB_*` নেই	নতুন কেউ সেটআপে অনুমান করে	৫টি কী যোগ
ক্যাশ স্তর নেই	প্রতি রিকোয়েস্টে কনটেক্সট-কোয়েরি	মেপে দরকার হলে Redis
ASGI ঘোষিত, WSGI ব্যবহৃত	দুই এন্ট্রি পয়েন্ট, একটি অব্যবহৃত	ASGI সরানো, বা ব্যবহার শুরু
prod-এর ২টি গার্ড টেস্টহীন	গার্ড সরিয়ে দিলে কেউ ধরবে না	দুটি ছোট টেস্ট
vendor ভার্সন অনির্দিষ্ট	Bootstrap আপডেটে ভাঙার ঝুঁকি অজানা	ভার্সন ফাইলে লিখে রাখা

প্রথম চারটি একদিনের কাজ — তবু ডেমো ঠিক চলছে, তাই কেউ করে না। **এটাই প্রযুক্তিগত ঋণের আসল চেহারা:** ভাঙা নয়, শুধু ঝুঁকিপূর্ণ।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন এই তালিকা থেকে একটি বেছে নিয়ে ঠিক করতে — ৩০ মিনিটে সম্ভব। যে কাজ নিজে করেছে, সেটাই সত্যিকারের শেখা।

৩

D3 — নিরাপত্তা

রিকোয়েস্ট ভেতরে ঢুকল, এখন কে কী দেখবে আর কী করতে পারবে · ২২ স্লাইড

গভীর ডেক · D3

নিরাপত্তা

কে? কোন স্কুল? কী করতে পারে? — ৩২টি নিরাপত্তা-টেস্ট · ৩৬টি `PermissionDenied` · ৪টি ভূমিকা

মানচিত্র

নিরাপত্তা মানে তিনটি প্রশ্ন

প্রতিটি রিকোয়েস্টে তিনটি, আর প্রতিটির উত্তর দেওয়ার আলাদা জায়গা

১. কে?

লগইন হয়েছে? অ্যাকাউন্ট সক্রিয়? পাসওয়ার্ড সঠিক?

উত্তরদাতা: Django-র Auth মিডলওয়্যার + নিজের User মডেল

ব্যর্থ হলে: লগইন পাতায় রিডাইরেক্ট

২. কোন স্কুল?

এই ইউজার এই workspace-এর সদস্য? স্কুলটি সক্রিয়?

উত্তরদাতা: `workspace_view` ডেকোরেটর

ব্যর্থ হলে: 404 — স্কুলটির অস্তিত্বই স্বীকার নয়

৩. কী করতে পারে?

ভূমিকা অনুমোদিত? ক্লাস/ছাত্র তার পরিধিতে?

উত্তরদাতা: role set + `scoping.py`

ব্যর্থ হলে: 403 বা খালি তালিকা

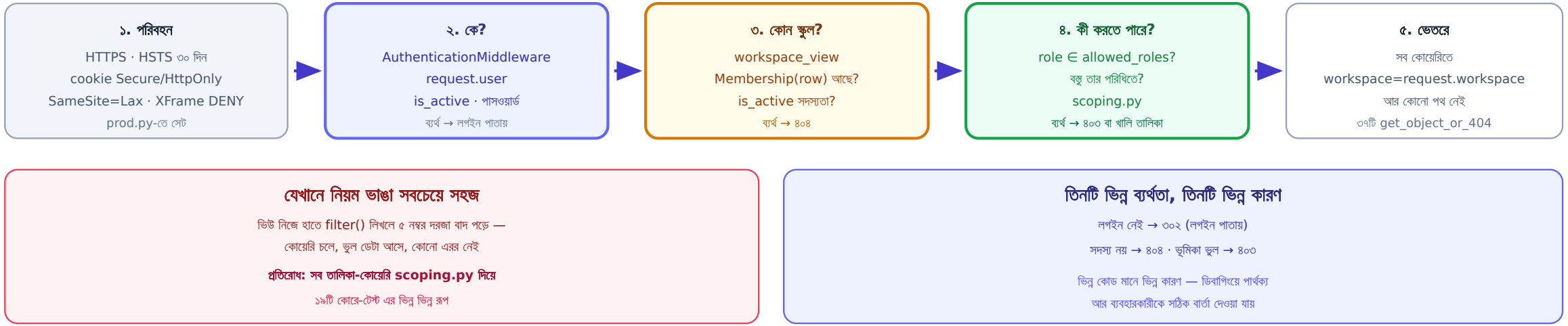
তিনটি প্রশ্নের উত্তর **তিন জায়গায়** — এটাই নকশা। এক জায়গায় গুঁজে দিলে একটি ফাঁক সব কিছু ফাঁকা করে দিত।

শিক্ষকের নোট: এই তিনটি প্রশ্ন বোর্ডে লিখুন এবং সারাক্ষণ ফিরে আসুন। শিক্ষার্থীরা প্রায়ই দ্বিতীয়টি ভুলে যায় — আর মাল্টি-টেন্যান্ট অ্যাপের সবচেয়ে বিপজ্জনক বাগ ঠিক সেখানেই হয়।

স্তর

একটি অননুমোদিত রিকোয়েস্ট চারটি দরজা পেরোয়

প্রতিটি দরজা আলাদা প্রশ্ন, আলাদা উত্তর, আলাদা ব্যর্থতা



শিক্ষকের নোট: পঞ্চম দরজাটি ব্যাখ্যা করুন — প্রতিটি কোয়েরিতে workspace শর্ত। এটি লেখা থাকে ভিউতে, তাই এখানেই ভুল হয়। বাঁচার উপায় একটাই: হাতে ফিল্টার না লেখা।

স্তর ২-৩

`workspace_view` — চারটি ধাপ, ২৮ লাইন

৫২টি ভিউ এই একটি ডেকোরেটরের উপর দাঁড়িয়ে

```
if not request.user.is_authenticated:
    return redirect(f"/accounts/login/?next={request.path}")

workspace = Workspace.objects.filter(
    slug=slug, is_active=True).first()
if workspace is None:
    raise Http404("Workspace not found.")

membership = get_membership(request.user, workspace)
if membership is None:
    raise Http404("Workspace not found.")   # ইচ্ছাকৃত

if allowed_roles and membership.role not in allowed_roles:
    raise PermissionDenied("Your role ... does not allow this action.")

request.workspace = workspace
request.membership = membership
request.session["active_workspace_id"] = workspace.pk
```

ধাপ	ফল
১. লগইন?	না হলে ৩০২ + `?next=` — কাজ হারায় না
২. স্কুল আছে?	না হলে ৪০৪
৩. সদস্য?	না হলে **৪০৪**, ৪০৩ নয়
৪. ভূমিকা?	ভুল হলে ৪০৩ — বার্তা সহ
সফল হলে	`request.workspace` + সেশন আপডেট

প্ল্যাটফর্ম-অ্যাডমিনও ছাড় পায় না: ডেকোরেটরের মন্তব্যে লেখা — "platform admin may look at platform-level pages, never at private records of a workspace they do not belong to."

শিক্ষকের নোট: ৩য় ধাপে থামুন। জিজ্ঞেস করুন: "সদস্য না হলে ৪০৩ দিলে কী ক্ষতি?" উত্তর: ৪০৩ মানে "এই স্কুল আছে, কিন্তু তোমার অধিকার নেই" — সেটাই তথ্য ফাঁস।

প্রমাণ

কেন ৪০৪ বনাম ৪০৩ — এবং টেস্ট যা তা প্রমাণ করে

প্রতিটি কোডের পেছনে একটি টেস্টের নাম আছে

টেস্ট (আসল নাম)	প্রত্যাশা	কেন এটাই সঠিক
`test_non_member_gets_404_not_403`	৪০৪	সদস্যপদ নেই মানে স্কুলটির অস্তিত্ব জানানোর অধিকারও নেই
`test_anonymous_visitor_is_sent_to_login`	৩০২	লগইন নেই — লগইনই সমাধান, তাই সেখানেই পাঠানো
`test_student_cannot_open_teaching_pages`	৪০৩	সদস্য, কিন্তু ভূমিকা ভুল — "কেন নয়" বলা নিরাপদ
`test_coordinator_cannot_open_owner_only_pages`	৪০৩	সমন্বয়ক দেখে, কিন্তু সেটিং বদলায় না
`test_teacher_cannot_open_administration_pages_by_typing_the_url`	৪০৩	মেনুতে লিংক নেই — তবু URL টাইপ করে ঢোকা যায় না
`test_inactive_membership_sees_nothing`	৪০৪ / খালি	সদস্যতা বন্ধ করা মানে অপসারণ, কিন্তু ডেটা রাখা

নিয়ম: প্রতিটি অনুমতি-সিদ্ধান্তের একটি টেস্ট থাকা চাই — নাহলে ভবিষ্যতের কেউ ডেকোরেটর বদলে দিলে কেউ টের পাবে না।

শিক্ষকের নোট: "মেনুতে লিংক নেই" আর "ঢোকা যায় না" — এই দুইয়ের পার্থক্য শিক্ষার্থীদের প্রায়ই বুঝতে হয় না। UI লুকানো নিরাপত্তা নয়; সার্ভার-চেকই নিরাপত্তা।

পরিধি

`scoping.py` — এক জায়গায় "কে কী দেখবে"

তালিকা-কোয়েরি আর বস্তু-যাচাই একই ফাংশন ডাকে, তাই কখনো মতভেদ হয় না

```
def visible_students(user, workspace, membership=None):
    qs = StudentProfile.objects.filter(workspace=workspace)
    if membership is None:
        return qs.none()

    if role == ORG_ADMIN:
        return qs                          # পুরো স্কুল

    if role in (COORDINATOR, TEACHER):
        return qs.filter(
            enrollments__is_active=True,
            enrollments__class_group__in=
                visible_classes(user, workspace)).distinct()

    if role == STUDENT:
        return qs.filter(pk=student.pk)  # কেবল নিজের

    return qs.none()
```

ফাংশন	কী ফেরত দেয়
`visible_classes`	ভূমিকা অনুযায়ী ক্লাসের queryset
`visible_students`	ওই ক্লাসগুলোর সক্রিয় ছাত্র
`visible_exams`	ছাত্রের জন্য কেবল `is_published` + নিজের নম্বর
`visible_homework`	ছাত্রের জন্য কেবল প্রকাশিত
`teacher_can_access_class()`	একই ফাংশনের `.filter(pk=…).exists()`

কারণ: **তালিকা ও বস্তু যাচাই আলাদা লজিক হলে** একদিন একটি পথ খোলা থেকে যায় — তালিকায় দেখা যায় না, কিন্তু URL জানলে ঢোকা যায়।

শিক্ষকের নোট: এই ফাইলটিই ডেকের হৃদয়। প্রশ্ন করুন: "নতুন ভূমিকা যোগ করলে কত জায়গায় বদলাতে হবে?" উত্তর: এক জায়গায় — `scoping.py`. এটাই ভূমিকা-বাগ একবার ঠিক করার নকশা।

ম্যাট্রিক্স

চারটি ভূমিকা, আটটি কাজ

প্রতিটি ঘর কোডে লেখা — ডকুমেন্টে নয়, তাই পুরনো হওয়ার সুযোগ নেই

কাজ	ORG_ADMIN	COORDINATOR	TEACHER	STUDENT
হাজিরা নেওয়া	✓	✓	✓	×
নম্বর এন্ট্রি	✓	✓	✓	×
ফলাফল প্রকাশ	✓	✓	✓ (নিজের ক্লাস)	×
শ্রেণি/বিষয় তৈরি	✓	×	×	×
শিক্ষক-অ্যাসাইনমেন্ট	✓	×	×	×
অডিট লগ দেখা	✓	×	×	×
ব্র্যান্ডিং / সেটিং	✓	×	×	×
নিজের রেজাল্ট দেখা	✓	✓	✓	✓ (কেবল নিজের)

কোডে: `OWNERS` (১টি) · `MANAGERS` (২টি) · `TEACHING` (৩২টি ভিউ) · `ALL_ROLES` (৮টি ভিউ)

সূক্ষ্মতা: TEACHER "ফল প্রকাশ" পারে, কিন্তু কেবল `teacher_can_access_class()` পাশ করলে — ভূমিকা ঠিক, বস্তু ভুল, তখন ৪০৩।

শিক্ষকের নোট: ম্যাট্রিক্সটি বোর্ডে আঁকুন এবং শিক্ষার্থীদের জিজ্ঞেস করুন: "সমন্বয়ক নম্বর দিতে পারে, কিন্তু শ্রেণি বানাতে পারে না — কেন?" উত্তর: নম্বর তার দৈনন্দিন কাজ, শ্রেণি-কাঠামো প্রতিষ্ঠানের সিদ্ধান্ত।

পরিধি-২

সমন্বয়কের পরিধি — অনুমতি ডেটা হিসেবে

`CoordinatorScope` · "এই দুই বিভাগ আর এই এক ক্লাস"

```
class CoordinatorScope(models.Model):
    workspace = FK(Workspace)
    user = FK(User)
    department = FK(Department, null=True)
    class_group = FK(ClassGroup, null=True)
    is_active = Bool(default=True)

    class Meta:
        constraints = [CheckConstraint(
            condition=~(Q(department__isnull=True) &
                        Q(class_group__isnull=True)),
            name="coordinator_scope_needs_a_target")]
        indexes = [Index(fields=["workspace", "user", "is_active"])]
```

সিদ্ধান্ত	কারণ
রোল-স্ট্রিং নয়, সারি	"সমন্বয়ক-২" বানালে প্রতিটি নতুন স্কুলে নতুন কোড লাগত
`department` বা `class_group`	কেউ বিভাগভিত্তিক, কেউ এক ক্লাসের — দুটোই সম্ভব
`CheckConstraint`	দুটোই null মানে "কিছুই দেখবে না" — অর্থহীন সারি, তাই ডেটাবেজেই বন্ধ
`is_active` + ইনডেক্স	পরিধি বাতিল মানে মুছে ফেলা নয়; কোয়েরি indexed

প্রমাণ: `test_coordinator_sees_only_their_scope` — দুই সমন্বয়ক, আলাদা পরিধি, একে অন্যের ক্লাস দেখে না।

সৎ সীমা: পরিধি একই স্কুলের ভেতরে সীমিত করে, কিন্তু ডেটাবেজ-স্তরে (RLS) নয়। কেউ ভিউতে `scoping.py` বাদ দিলে ডেটাবেজ তাকে আটকাবে না।

শিক্ষকের নোট: "অনুমতি ডেটা হিসেবে রাখা" — এই বাক্যটি মনে রাখতে বলুন। এর অর্থ: নতুন পরিধি যোগ করতে মাইগ্রেশন লাগে না, কেবল একটি সারি।

স্তর ৪

ভূমিকা ঠিক, বস্তু ভুল — সবচেয়ে সূক্ষ্ম ফাঁক

শিক্ষক একজন, কিন্তু ক্লাস ৪০টি; ডেকোরেটর কেবল প্রথমটি জানে

```
@workspace_view(*TEACHING)
def exam_detail(request, slug, pk):
    exam = get_object_or_404(Exam, pk=pk,
                             workspace=request.workspace)
    if not teacher_can_access_class(request.user,
                                    exam.class_group):
        raise PermissionDenied(
            "You are not assigned to this class.")
```

একই তিন লাইন ভিউতে যেখানে ক্লাস/ছাত্র/পরীক্ষা জড়িত — নাহলে পরিধি বাইরে যাওয়া যায়।

স্তর	যা জানে
১. সদস্যতা	ইউজার এই স্কুলের সদস্য
২. ভূমিকা	তার ভূমিকা এই কাজটি করতে পারে
৩. বস্তু	এই নির্দিষ্ট ক্লাস/ছাত্র তার পরিধিতে

ফাঁদ: ২ নম্বর পাশ করলেই সব ক্লাস খুলে যায় — যদি ৩ নম্বর না থাকে। শিক্ষক ৭ম শ্রেণির, কিন্তু URL-এ ৯ম শ্রেণির `pk` বসালে দেখতে পাবেন।

প্রমাণ: `test_teacher_sees_only_assigned_classes` · `test_teacher_only_sees_students_of_their_own_classes` · `test_teacher_with_no_assignment_sees_no_students`

শিক্ষকের নোট: এটি IDOR (insecure direct object reference) — নামটি বলুন, কারণ ইন্টারভিউতে জিজ্ঞেস করা হয়। উদাহরণ: নিজের রেজাল্ট পাতার URL-এ অন্যের `pk` বসানো।

৩২ টেস্ট

প্রতিটি স্তরের প্রমাণ — নাম ধরে

১০৩টি টেস্টের মধ্যে ৩২টি সরাসরি নিরাপত্তার (৩১%)

স্তর ১–২ (১০টি)	প্রমাণ করে
`anonymous_visitor_is_sent_to_login`	লগইন ছাড়া কিছু নয়
`dashboard_requires_login`	টাইপ করা URL-ও নয়
`deactivated_user_cannot_log_in`	অ্যাকাউন্ট বন্ধ = ঢোকা বন্ধ
`wrong_password_fails`	ভুল পাসওয়ার্ড পরিষ্কার ব্যর্থ
`non_member_gets_404_not_403`	স্কুলের অস্তিত্ব ফাঁস নয়
`inactive_membership_sees_nothing`	সদস্যতা বাতিল = দেখা বন্ধ
`verified_domain_shows_that_institution…`	ডোমেইন → সঠিক স্কুল
`unverified_domain_is_ignored`	যাচাই না হলে উপেক্ষা
`domain_cannot_be_stolen_by_second…`	ডোমেইন দখল অসম্ভব
`only_the_platform_admin_can_open_console`	প্ল্যাটফর্ম আলাদা

স্তর ৩–৪ (২২টি)	প্রমাণ করে
`teacher_cannot_open_administration…`	URL টাইপেও নয়
`student_cannot_open_teaching_pages`	ভূমিকা = দেয়াল
`coordinator_cannot_open_owner_only_pages`	দেখা ≠ বদলানো
`each_role_is_offered_only_its_own_links`	মেনুও মিথ্যা বলে না
`teacher_sees_only_assigned_classes`	পরিধি কার্যকর
`coordinator_sees_only_their_scope`	ডেটা-ভিত্তিক পরিধি
`student_sees_only_own_record`	নিজের বাইরে নয়
`student_cannot_see_another_students_marksheet`	IDOR বন্ধ
`results_are_invisible_until_published`	প্রকাশ-নিয়ন্ত্রণ
`publishing_is_audited` · `correction_requires_a_reason…`	প্রতিটি বদল লিপিবদ্ধ

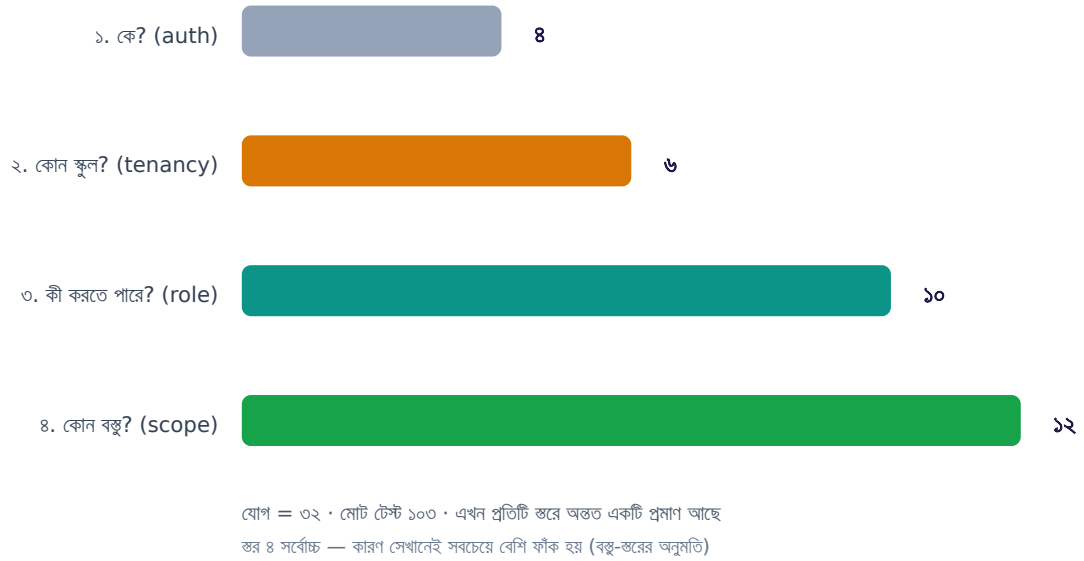
বাকি ৭১টি টেস্ট কার্যকারিতা যাচাই করে (হাজিরা সেভ হয়, নম্বর যোগ হয়) — নিরাপত্তার প্রমাণ নয়। দুই ধরনই দরকার।

শিক্ষকের নোট: এই দুই কলাম একসাথে পড়ুন — এটাই "নিরাপত্তা ডকুমেন্টে লেখা থাকে না, টেস্টে থাকে" এর প্রমাণ। শিক্ষার্থীদের নিজের প্রজেক্টে এমন একটি তালিকা বানাতে দিন।

বণ্টন

৩২টি টেস্ট কোন স্তরে কত

কম সংখ্যা মানে দুর্বলতা নয় — কিন্তু দেখতে হবে কোথায়



যোগ = ৩২ · মোট টেস্ট ১০৩ · এখন প্রতিটি স্তরে অন্তত একটি প্রমাণ আছে

স্তর ৪ সর্বোচ্চ — কারণ সেখানেই সবচেয়ে বেশি ফাঁক হয় (বস্তু-স্তরের অনুমতি)

যা এখনো প্রমাণহীন

রেট-লিমিট নেই → টেস্টও নেই

CSRF ব্যর্থতার পাতা

আপলোডের আকার/ধরন যাচাই

prod-এর নিরাপত্তা-হেডার

সেশন-মেয়াদ শেষ হওয়া

স্তর ৪ কেন সবচেয়ে বড়

একটি ভূমিকা-চেক সহজ, কিন্তু

৪০টি ক্লাসের প্রতিটির অনুমতি

আলাদা করে ভাবতে হয়

তাই সেখানে ১২টি টেস্ট

শিক্ষকের নোট: ডান পাশের লাল বাক্সটি আগে পড়ুন — সততাই এখানে মূল শিক্ষা। "কত টেস্ট" নয়, "কোন প্রশ্নের উত্তর নেই" — সেটাই আসল মাপ।

পাসওয়ার্ড

চারটি ভ্যালিডেটর — আর একটি ইচ্ছাকৃত সীমা

`AUTH_PASSWORD_VALIDATORS` · ন্যূনতম ১০ অক্ষর

ভ্যালিডেটর	কী আটকায়
UserAttributeSimilarity	নাম/ইমেইলের কাছাকাছি পাসওয়ার্ড
MinimumLength **min_length=10**	ছোট পাসওয়ার্ড
CommonPassword	"password123" জাতীয় তালিকা
NumericPassword	কেবল সংখ্যা — "01712345678" নয়

ডেমো পাসওয়ার্ড `DemoPass!2026` — ১০ অক্ষরের বেশি, তালিকায় নেই, সংখ্যা-একক নয়।

সৎ সীমা: "কতবার ভুল হয়েছে" গণনা কোথাও নেই — ১০০ বার ভুল পাসওয়ার্ড দিলেও কিছু হয় না। brute-force ঠেকানোর কোনো স্তর নেই।

ভালো দিক: লগইনে ভুল হলে বার্তা একটাই — "E-mail or password is incorrect." ফলে কোন ইমেইল আছে, তা বোঝা যায় না (user enumeration বন্ধ)।

কিন্তু: নিষ্ক্রিয় অ্যাকাউন্টে আলাদা বার্তা — "This account is deactivated." এতে বোঝা যায় অ্যাকাউন্টটি আছে। ব্যবহারকারীর সুবিধার জন্য নেওয়া ছাড়, তথ্য ফাঁসের সামান্য দাম।

শিক্ষকের নোট: শেষ পয়েন্টটি সূক্ষ্ম — নিরাপত্তা আর সহায়তার মধ্যে এই ধরনের ছাড় প্রতিটি অ্যাপে থাকে। শিক্ষার্থীদের জিজ্ঞেস করুন: "কোনটি বেছে নিতেন?" — সঠিক উত্তর নেই, কিন্তু সিদ্ধান্তটা জানা দরকার।

লগইন

লগইনে তিনটি ভালো সিদ্ধান্ত, একটি বড় ফাঁক

`IMSLoginView` · ৯ লাইন কোড, বড় প্রভাব

```
def form_valid(self, form):
    # আগের সেশনের workspace মুছে ফেলো:
    # এই ব্রাউজারে অন্য কেউ লগইন করেছিল
    self.request.session.pop(SESSION_KEY, None)
    response = super().form_valid(form)
    messages.success(self.request, "Welcome back!")
    return response
```

সিদ্ধান্ত	কেন
সেশন-কী মুছে ফেলা	শেয়ার্ড কম্পিউটার: আগের ব্যবহারকারীর স্কুলে নতুন ব্যবহারকারী ঢুকে যেত না
`redirect_authenticated_user`	লগইন থাকা অবস্থায় লগইন পাতায় গেলে ড্যাশবোর্ডে ফেরত
Django-র `login()`	সেশন-কী ঘোরায় (session fixation ঠেকায়) — নিজে লিখতে হয় না

বড় ফাঁক: সফল বা ব্যর্থ লগইন — কোনোটিই `AuditLog`-এ যায় না। "কে কখন ঢুকল" প্রশ্নের উত্তর ডেটাবেজে নেই, কেবল সার্ভার লগে (যা ঘুরিয়ে ফেলা যায়)।

শিক্ষকের নোট: সেশন-কী মুছে ফেলার লাইনটি সবচেয়ে বড় শিক্ষা — ছোট কোড, বড় ঝুঁকি বন্ধ। জিজ্ঞেস করুন: "এটি না থাকলে কী হতো?" উত্তর: ল্যাবে একজন লগআউট না করে গেলে পরেরজন তার স্কুলে ঢুকে যেত।

সেশন

সেশন ও কুকি — ছয়টি পতাকা, একটি কারণ

প্রতিটি পতাকা একটি নির্দিষ্ট আক্রমণ ঠেকায়

সেটিং (prod)	মান	যা ঠেকায়
`SESSION_COOKIE_SECURE`	True	HTTP-তে কুকি যাওয়া — নেটওয়ার্কে আটকানো
`SESSION_COOKIE_HTTPONLY`	True	JavaScript দিয়ে কুকি পড়া (XSS-এর ক্ষতি সীমিত)
`SESSION_COOKIE_SAMESITE` · `CSRF_COOKIE_SAMESITE`	Lax	অন্য সাইট থেকে আসা POST-এ কুকি পাঠানো
`SECURE_SSL_REDIRECT`	True	সব HTTP → HTTPS
`SECURE_HSTS_SECONDS`	৩০ দিন	ব্রাউজারকে "এই সাইট কেবল HTTPS" মনে রাখানো
`X_FRAME_OPTIONS`	DENY	অন্য সাইটে iframe-এ বসিয়ে ক্লিকজ্যাকিং

মেয়াদ ৮ ঘণ্টা + `SESSION_SAVE_EVERY_REQUEST` — কাজ করলে মেয়াদ বাড়ে, ফেলে রাখলে শেষ হয়।

যা নেই: CSP হেডার নেই (`django-csp` ইনস্টল নয়) — তাই XSS ঘটলে ব্রাউজার-স্তরে বাধা নেই; ভরসা কেবল template autoescape।

শিক্ষকের নোট: শিক্ষার্থীরা HSTS-এর মানে জানে না। ব্যাখ্যা: একবার HSTS পাঠালে ব্রাউজার ৩০ দিন HTTP-তেই সংযোগ করবে না — অর্থাৎ ভুল সেট করলে ফিরিয়ে আনা কঠিন।

CSRF

CSRF — টোকেন, উৎস, আর একটি ত্রুটি-পাতা

ডিফল্ট `CsrfViewMiddleware` + নিজের বার্তা

```
CSRF_FAILURE_VIEW = "apps.core.views.csrf_failure"
CSRF_TRUSTED_ORIGINS = env_list(
    "CSRF_TRUSTED_ORIGINS",
    "http://127.0.0.1:8000,http://localhost:8000,"
    "https://*.e2b.app")

def csrf_failure(request, reason=""):
    return render(request, "403_csrf.html",
        {"reason": reason,
         "origin": request.META.get("HTTP_ORIGIN","")},
        status=403)
```

প্রশ্ন	উত্তর
আক্রমণ কেমন?	অন্য সাইটে লুকানো ফর্ম → আপনার ব্রাউজারের কুকি দিয়ে POST
প্রতিরোধ কী?	ফর্মে টোকেন, যা অন্য সাইট জানে না
ব্যর্থ হলে?	ডিফল্ট ৪০৩-এর বদলে বাংলা/ইংরেজি ব্যাখ্যা-পাতা, উৎস দেখা যায়
কেন `TRUSTED_ORIGINS`?	HTTPS প্রক্সি/স্যান্ডবক্স ডোমেইন থেকে আসা ফর্ম বৈধ

সৎ ফাঁক: `csrf_failure` পাতার কোনো টেস্ট নেই — কেউ ভাঙলে টের পাওয়া যাবে ব্যবহারকারীর অভিযোগে।

মাপা হিসাব: ৩৫টি টেমপ্লেটে ফর্ম আছে · ২৭টিতে `{% csrf_token %}` · বাকি ৮টি `method="get"` ফিল্টার — সেখানে টোকেন লাগেই না। অর্থাৎ প্রতিটি POST ফর্মে টোকেন আছে।

শিক্ষকের নোট: বোর্ডে তিন লিখুন: কুকি (কে), টোকেন (এই ফর্ম), Origin (কোন সাইট)। তিনটি না মিললে ৪০৩ — এটাই CSRF-এর পুরো গল্প।

হোস্ট

কাস্টম ডোমেইন দখল করা কেন কঠিন

তিনটি বাধা: যাচাই, অনন্যতা, আর "নিজে সেট করা যায় না"

বাধা	কোথায়
যাচাই করা ছাড়া কাজ করে না	`filter(domain_verified=True)` — মিডলওয়্যারে
এক ডোমেইন একবারই	`UniqueConstraint(name="unique_custom_domain")`
মালিক নিজে বসাতে পারেন না	প্ল্যাটফর্ম-অ্যাডমিনই সেট করেন — কারণ ডোমেইন যাচাই প্রমাণ দরকার

প্রমাণ: `test_owner_cannot_set_a_custom_domain_themselves` · `test_domain_cannot_be_stolen_by_a_second_institution`

যে ঝুঁকিটা লেখা হয়েছে: মিডলওয়্যার রানটাইমে `settings.ALLOWED_HOSTS`-এ ডোমেইন যোগ করে। এটি কাজ করে, কিন্তু অ্যাপের গ্লোবাল অবস্থা বদলায় — একই প্রসেসে অন্য রিকোয়েস্টও এই তালিকা দেখে।

ভালো দিক: `patch_vary_headers(response, ["Host"])` — হোস্টভেদে আলাদা ক্যাশ, নাহলে এক স্কুলের লগইন পাতা আরেক স্কুলে দেখাত।

সংশোধনের পথ: তালিকা বদলে রিকোয়েস্টে পাস করা, বা `sync_domains` কমান্ড দিয়ে আগেই তালিকা তৈরি রাখা।

শিক্ষকের নোট: "গ্লোবাল সেটিং রানটাইমে বদলানো" — এটি প্রায়ই বাগের উৎস। প্রশ্ন করুন: "দুই স্কুলের অনুরোধ একসাথে এলে কী হয়?" উত্তর: সাধারণত কাজ করে, কিন্তু নিশ্চয়তা নেই।

ফাইল

আপলোড করা ফাইল — সবচেয়ে দুর্বল জায়গা

লোগো `ImageField`, ম্যাটেরিয়াল `FileField` — দুটির সুরক্ষা সমান নয়

জিনিস	যা যাচাই হয়
`WorkspaceBranding.logo`	`ImageField` → Pillow ছবি কি না যাচাই করে (অ-ছবি আপলোড ব্যর্থ)
`Material.file`	**কিছুই না** — শুধু ফাইল-নাম আর পথ (`materials/%Y/%m/`)
আকার-সীমা	কোনো `FILE_UPLOAD_MAX_MEMORY_SIZE` কাস্টম নেই
ধরন-সীমা	PDF/DOCX নয় এমন ফাইলও গ্রহণযোগ্য

ঝুঁকি: কেউ `.html` বা `.svg` আপলোড করে লিংক শেয়ার করলে ব্রাউজারে স্ক্রিপ্ট চলতে পারে — যদি মিডিয়া ফাইল পাবলিক পথে সার্ভ হয়।

এখন যা হয়: `/media/` কেবল `DEBUG=True`-তে সার্ভ হয় (`config/urls.py`)। প্রোডাকশনে মিডিয়া ওয়েব-সার্ভারকে দিতে হয় — এবং সেই নিয়ম কোথাও লেখা নেই।

সংশোধন: অনুমোদিত এক্সটেনশন-তালিকা + আকার-সীমা + `Content-Disposition: attachment`। এখনো বাকি।

শিক্ষকের নোট: "ফাইল আপলোড মানে ব্যবহারকারীর দেওয়া কোড আপনার সার্ভারে" — এই বাক্যটি শেখান। Student-এর দেওয়া হোমওয়ার্ক ফাইলও একই ঝুঁকি বহন করে।

অডিট

২৩ ধরনের কাজ লিপিবদ্ধ — আর একটি নীরব ফাঁদ

`audit.log()` কখনো রিকোয়েস্ট ভাঙে না — কিন্তু তাই কিছু চুপচাপ হারায়ও যায়

```
def log(request, action, obj=None, ...):
    try:
        return AuditLog.objects.create(
            workspace=..., actor=request.user,
            action=action, before=..., after=...,
            ip_address=request.META["REMOTE_ADDR"])
    except Exception:      # noqa: BLE001
        return None        # অডিট কখনো ভাঙবে না
```

দাম: লগ লেখা ব্যর্থ হলে কেউ জানে না — ডেটাবেজ ভরে গেলে, বা কলাম লম্বা হলে, ইতিহাস নীরবে ফাঁকা থেকে যায়।

লিপিবদ্ধ কাজ (নমুনা)	কোন অ্যাপ
`attendance.save` · `attendance.correct`	হাজিরা
`assessments.publish` · `.unpublish` · `.marks.save`	মূল্যায়ন
`students.enrol` · `.status` · `.enrolment.end`	ছাত্র
`homework.submit` · `.review` · `.create`	পাঠ
`settings.attendance.update`	নীতি

মোট ২৩টি action নাম · `before` / `after` ৩০০ অক্ষরে কাটা · দেখার অধিকার কেবল `OWNERS`।

যা লিপিবদ্ধ হয় না: লগইন/লগআউট · ব্যর্থ লগইন চেষ্টা · পাসওয়ার্ড বদল · সেটিং ছাড়া অন্যান্য পঠন। অর্থাৎ "কে দেখল" প্রশ্নের উত্তর নেই — কেবল "কে বদলাল"।

শিক্ষকের নোট: দুটি বাক্য তুলনা করুন: "লগ লেখা কখনো কাজ থামাবে না" বনাম "লগ লেখা ব্যর্থ হলে কেউ জানবে না"। দুটোই সত্য — এবং একটির দাম অন্যটি।

প্ল্যাটফর্ম

প্ল্যাটফর্ম কনসোল — একটাই নিয়ম: `is_superuser`

১৮টি ভিউ, ১৮টি ডেকোরেটর, শূন্য ব্যতিক্রম

```
def platform_admin_required(view):
    @wraps(view)
    def _wrapped(request, *args, **kwargs):
        if not request.user.is_superuser:
            raise PermissionDenied(
                "The platform console is for the "
                "system administrator only.")
        return view(request, *args, **kwargs)
    return _wrapped
```

সিদ্ধান্ত	কারণ
workspace-র ভূমিকা নয়	প্ল্যাটফর্ম মালিক কোনো স্কুলের সদস্য নন — সদস্যতা দিয়ে ঢোকা যায় না
কনসোল স্কুলের বাইরে	`/platform/` — No workspace pill, no billing banner
সুপারইউজারও স্কুলের ডেটা পড়েন না	`workspace_view`-তে ব্যতিক্রম নেই — প্ল্যাটফর্ম অ্যাডমিন অন্য স্কুলের ছাত্র-তালিকা খুলতে পারেন না

দুই ক্ষমতা আলাদা: প্ল্যাটফর্ম মালিক স্কুল তৈরি/বন্ধ করতে পারেন, প্ল্যান বদলাতে পারেন — কিন্তু সেই স্কুলের হাজিরা বা নম্বর দেখার ক্ষমতা তাঁর নেই। এটি ইচ্ছাকৃত নকশা, দুর্ঘটনা নয়।

প্রমাণ: `test_only_the_platform_admin_can_open_the_console` — সাধারণ অ্যাডমিন ৪০৩ পান।

শিক্ষকের নোট: "সুপারইউজার সব পারেন" — এই ধারণাটি এখানে ভাঙুন। প্রশ্ন করুন: "সাপোর্ট দিতে গিয়ে ছাত্রের নম্বর দেখা কি ঠিক?" উত্তর: না — প্রয়োজনে সম্মতি নিয়ে, লিপিবদ্ধভাবে, আলাদা প্রক্রিয়ায়।

যাচাই

ছয়টি হাতে-কলমে পরীক্ষা — ১৫ মিনিট

ডেমো চালিয়ে নিজেই যাচাই করুন; প্রতিটির প্রত্যাশিত ফল দেওয়া

#	কী করবেন	যা দেখা উচিত
১	লগআউট করে `/w/springfield/students/` খুলুন	লগইন পাতায় যাবে, `?next=` সহ
২	`teacher@ims.local` দিয়ে ঢুকে অ্যাডমিন-পাতার URL টাইপ করুন	৪০৩ — পর্দা ভাঙে না, বার্তা দেখায়
৩	শিক্ষক দিয়ে অন্য ক্লাসের নম্বর পাতায় যান (pk বদলে)	৪০৩ — "not assigned to this class"
৪	`student@ims.local` দিয়ে অন্য ছাত্রের রেজাল্ট খুলুন	৪০৩ — নিজের রেকর্ডেই সীমা
৫	একটি স্কুলে সাবস্ক্রিপশন মেয়াদোত্তীর্ণ করুন, তারপর সেভ করুন	সেভ হয় না, বার্তা: "your work is not lost"
৬	অচেনা `slug` দিয়ে URL খুলুন	৪০৪ — দুই ক্ষেত্রেই একই বার্তা

নিয়ম: প্রতিটি পরীক্ষা দুবার করুন — একজন অধিকারপ্রাপ্ত ব্যবহারকারী দিয়ে, একজন বঞ্চিত দিয়ে। কেবল প্রথমটি করলে "কাজ করে" প্রমাণ হয়, "বন্ধ" প্রমাণ হয় না।

শিক্ষকের নোট: এই ছয়টি শিক্ষার্থীদের নিজে করতে দিন — ১৫ মিনিটেই বোঝে। ২ ও ৩ নম্বরেই বেশিরভাগের ধারণা বদলায়।

সততা

নিরাপত্তায় যা নেই — ১০টি ফাঁক

প্রতিটির ঝুঁকি এবং প্রতিটির সম্ভাব্য সংশোধন

যা নেই	ঝুঁকি
রেট-লিমিট / লকআউট	পাসওয়ার্ড অনুমান অনন্তকাল চালানো যায়
লগইন অডিট	"কে কখন ঢুকল" বলা যায় না
2FA	পাসওয়ার্ড ফাঁস হলে সব ফাঁস
CSP হেডার	XSS ঘটলে ব্রাউজার-বাধা নেই
ফাইল আকার/ধরন যাচাই	বিপজ্জনক আপলোড সম্ভব

যা নেই	ঝুঁকি
RLS (ডেটাবেজ-স্তরে সীমানা)	ভিউতে ভুল হলে ডেটাবেজ থামায় না
নির্ভরতা-স্ক্যান / Dependabot	প্যাকেজের নিরাপত্তা-প্যাচ অজানা থাকে
এরর-ট্র্যাকিং (Sentry)	ব্যর্থতা চোখে পড়ে ব্যবহারকারীর অভিযোগে
মিডিয়া-ফাইল সুরক্ষা	লিংক জানলে ফাইল দেখা যায়
পাসওয়ার্ড-রিসেটের মেয়াদ কাস্টম	Django ডিফল্ট (৩ দিন) — জেনে রাখা হয়নি

প্রথম তিনটি এক সপ্তাহের কাজ, কিন্তু ডেমো ঠিক চলছে বলে কেউ করে না। **প্রযুক্তিগত ঋণ ভাঙা কোডে নয়, অনুপস্থিত কোডে থাকে** — এই তালিকাই তার প্রমাণ।

শিক্ষকের নোট: শেষ বাক্যটি বোর্ডে লিখুন। শিক্ষার্থীদের জিজ্ঞেস করুন: "এই দশটির মধ্যে কোনটি আজ রাতে ঠিক করা যায়?" — রেট-লিমিট ও লগইন অডিট, দুটোই এক ঘণ্টার কাজ।

8

D4 — লাইফসাইকেল ও অপস

কোড লেখা শেষ, এখন তাকে বাঁচিয়ে রাখা — মাইগ্রেশন, ব্যাকআপ, মনিটরিং, রিলিজ · ২০ স্লাইড

গভীর ডেক · D4

লাইফসাইকেল ও অপস

১২ মাইগ্রেশন · ২০ commit · ১টি পুরনো ট্যাগ · ব্যাকআপ ১৪ দিন · কোনো মনিটরিং নেই — সফটওয়্যার চালু হওয়ার পর কী হয়

মানচিত্র

তিনটি ব্লক: প্রস্তুতি → চালু → বাঁচিয়ে রাখা

কোড লেখা শেষ হওয়া মানে জীবন শুরু, শেষ নয়

১. প্রস্তুতি

স্লাইড ৩–৬ · রিহার্সাল স্ক্রিপ্ট · ১৩ দফা চেকলিস্ট · prod সেটিংস প্রমাণ

প্রমাণ: `check --deploy` → ০ সমস্যা

২. চালু

স্লাইড ৭–১০ · build → release → gunicorn · মাইগ্রেশনের ঝুঁকি · রোলব্যাক

প্রমাণ: Procfile ও build.sh

৩. বাঁচিয়ে রাখা

স্লাইড ১১–১৯ · ব্যাকআপ/রিস্টোর · মনিটরিং · লগ · ডেটা-জীবনচক্র · বিলিং রুটিন

প্রমাণ: `ops/` স্ক্রিপ্ট ও ৪টি row-count

এই ডেকের সবচেয়ে গুরুত্বপূর্ণ স্লাইডটি শেষের দিকের — **যা কখনো চালানো হয়নি** তার তালিকা। প্ল্যান আর প্রমাণের পার্থক্য সেখানেই।

শিক্ষকের নোট: শুরুতে বলুন: "অ্যাপ চালু করা একটি দিনের কাজ; চালু রাখা প্রতিদিনের কাজ।" শিক্ষার্থীদের অভিজ্ঞতা প্রায়ই প্রথমটিতে সীমিত — এই ডেক দ্বিতীয়টির কথা।

পথ

চারটি পরিবেশ, একটি কোড

প্রতিটির আলাদা ঝুঁকি, আলাদা দায়িত্ব

১. ডেমো

bash run.sh

SQLite · runserver

seed_demo ডেটা

কেউ ইমেইল পায় না

ঝুঁকি: শূন্য

২. রিহার্সাল

docs/deploy_check.sh

gunicorn :8010 · ২ worker

DEBUG=False · PostgreSQL

৫টি চেক + হেডার পরীক্ষা

ঝুঁকি: নিজের মেশিন

৩. প্রোডাকশন

build.sh → release → web

gunicorn · ৩ worker

WhiteNoise · আসল ডোমেইন

আসল স্কুল, আসল ডেটা

ঝুঁকি: সুনাম

৪. দৈনিক অপস

backup.sh · restore.sh (drill)

billing_report (cron 09:00)

লগ পড়া · ট্রায়াল শেষ হওয়া

প্রতিদিন ১০ মিনিট

ঝুঁকি: ভুলে যাওয়া

সৎ অবস্থান: ৩ নম্বর ধাপটি এখনো ঘটেনি — DEPLOYMENT.md নিজেই লিখেছে "not yet deployed to a public host"

অর্থাৎ ১, ২ ও ৪ (আংশিক) প্রমাণিত; ৩ কেবল প্রস্তুত — কোনো স্কুলের আসল ডেটা কখনো এই কোডে আসেনি

এই পার্থক্যটা লুকানো হয় না, কারণ লুকালে পরের মানুষ ভুল পরিকল্পনা করে

শিক্ষকের নোট: চতুর্থ বাক্সটি প্রথমে দেখান — শিক্ষার্থীরা ভাবে "ডিপ্লয়" মানে একটি কমান্ড। আসলে চতুর্থ ধাপটাই সবচেয়ে দীর্ঘ: প্রতি দিনের রুটিন।

রিহার্সাল

`deploy_check.sh` — ৩০ লাইনে প্রোডাকশন মহড়া

আসল gunicorn, আসল `prod` সেটিংস, তারপর ৫টি অনুরোধ

```
export DJANGO_SETTINGS_MODULE=config.settings.prod
python manage.py collectstatic --noinput
gunicorn config.wsgi:application \
    --bind 127.0.0.1:8010 --workers 2 --daemon
sleep 3

check "login page"         /accounts/login/      # 200
check "static css"         /static/css/ims.css   # 200
check "unknown page"       /does-not-exist       # 404
check "workspace anon"     /w/sunrise…/          # 302
```

যা যাচাই হয়	কেন এটিই
লগইন পাতা ২০০	অ্যাপ উঠেছে কি না
স্ট্যাটিক CSS ২০০	WhiteNoise ঠিক কাজ করছে
অচেনা URL → ৪০৪	নিজের এরর-পাতা, ডিফল্ট নয়
অ্যানোনিমাস workspace → ৩০২	টেন্যান্সি-রিডাইরেক্ট জীবিত
নিরাপত্তা-হেডার grep	prod.py-র পতাকাগুলো সত্যিই বেরোচ্ছে

ফাঁক: স্ক্রিপ্টটি `.env.production.local` ফাইল দাবি করে, যা রিপোতে নেই — তাই নতুন কেউ চালাতে গেলে নিজে বানাতে হয়। আর স্ক্রিপ্ট নিজে কোনো টেস্টে চলে না।

ধারণাটাই মূল: "ডিপ্লয়ের আগে ডিপ্লয়ের মহড়া"। `runserver`-এ কাজ করা মানে প্রোডাকশনে কাজ করা নয়।

শিক্ষকের নোট: জিজ্ঞেস করুন: "runserver আর gunicorn-এ কী পার্থক্য?" — runserver ডেভেলপমেন্টের জন্য, স্ট্যাটিক নিজে দেয়, DEBUG-নির্ভর আচরণ আছে; gunicorn প্রোডাকশনের, WhiteNoise ছাড়া স্ট্যাটিক নেই।

চেকলিস্ট

গো-লাইভের ১৩টি দফা

`DEPLOYMENT.md`-তে লেখা — প্রতিটির পেছনে একটি বাস্তব দুঃস্বপ্ন

দফা	না করলে
SECRET_KEY ৫০+ র‍্যান্ডম, সিক্রেট স্টোরে	সেশন/টোকেন জাল করা যায়
ALLOWED_HOSTS ও CSRF origins আসল ডোমেইন	prod.py শুরুই হবে না (ভালো)
HTTPS কাজ করে, http রিডাইরেক্ট	কুকি খোলা পথে যায়
"Forgot password" একবার পুরো চালানো	SMTP ভুল, ব্যবহারকারী আটকে
`check --deploy` → ০ সমস্যা	কোন সেটিং বাদ, অজানা
ব্যাকআপ চালেছে + রিস্টোর ড্রিল হয়েছে	ব্যাকআপ অস্তিত্ব প্রমাণহীন
ডেমো ডেটা মুছে, `DemoPass!2026` নেই	সবাই জানে পাসওয়ার্ড

দফা	না করলে
host-এ `runserver` নয়, gunicorn	ডেভেলপমেন্ট সার্ভার সর্বজনীন
অন্য কেউ শিক্ষক ও ছাত্র হয়ে ঢুকে বলেছে "শুধু নিজেরটাই দেখি"	অনুমতি কোডে ঠিক, চোখে দেখা হয়নি
`seed_plans` চালানো, দাম মিলছে	প্রাইসিং পাতায় মিথ্যা দাম
প্ল্যাটফর্ম অ্যাডমিন আসল মানুষের	`admin@ims.local` চিরকাল অ্যাডমিন
`media/` স্থায়ী ভলিউমে	রিডিপ্লয়ে লোগো মুছে যায়
`billing_report` দৈনিক	প্রথম ট্রায়াল শেষ হওয়া কেউ দেখে না

নবম দফাটি সবচেয়ে কম ব্যবহৃত, সবচেয়ে দরকারি: **অন্য একজন মানুষ ঢুকে বলবে "আমি শুধু নিজেরটাই দেখি"** — এটাই অনুমতির আসল পরীক্ষা।

শিক্ষকের নোট: চেকলিস্ট মুখস্থ করাবেন না — শিক্ষার্থীদের নিজের প্রজেক্টের জন্য ১০টি দফার তালিকা লিখতে বলুন। তালিকা লেখার অভ্যাসই আসল শেখা।

রিলিজ

একটি রিলিজের তিনটি ধাপ

build → release → web; তিনটির দায়িত্ব আলাদা

```
# build.sh
pip install -r requirements-prod.txt
python manage.py collectstatic --noinput
python manage.py migrate --noinput

# Procfile
release: python manage.py migrate --noinput
web: gunicorn config.wsgi:application \
     --bind 0.0.0.0:$PORT --workers 3 \
     --timeout 60 --access-logfile - \
     --error-logfile -
```

ধাপ	কেন আলাদা
build	নির্ভরতা + স্ট্যাটিক — একবার, ইমেজ তৈরি হওয়ার সময়
release	স্কিমা বদল — নতুন কোড চালু হওয়ার ঠিক আগে, একবার
web	অনুরোধ সামলানো — একাধিক worker, যেকোনো সময় বন্ধ হতে পারে

ঝুঁকি: ৩ worker একসাথে চালু হলে migrate একবারই হয় (`release` ধাপে) — কিন্তু worker মাঝপথে নতুন স্কিমা পেলে পুরনো কোড ভাঙতে পারে। ব্লু-গ্রিন ডিপ্লয় নেই।

--timeout 60: ৬০ সেকেন্ডে উত্তর না দিলে worker মারা যায় — PDF/রিপোর্ট বড় হলে এই সীমা ছুঁতে পারে।

শিক্ষকের নোট: "worker মাঝপথে নতুন স্কিমা" — এটি বোঝানো কঠিন কিন্তু জরুরি। উদাহরণ: কলামের নাম বদলালে পুরনো worker ভাঙবে; তাই দুই ধাপে নাম বদলানো হয় (যোগ → ব্যবহার → পুরনো মোছা)।

মাইগ্রেশন

১২টি মাইগ্রেশন — কেবল সামনের দিকে

৯ অ্যাপে `0001`, ৩ অ্যাপে `0002`; কোনো reverse migration লেখা নেই

ধাপ	কেন
১. নতুন কলাম যোগ, `null=True`	পুরনো সারি ভাঙে না
২. ডেটা মাইগ্রেশন দিয়ে পূরণ	কোড আর ডেটা আলাদা ফাইলে
৩. পরের রিলিজে `null=False`	তখন সব সারিতে মান আছে
৪. পুরনো কলাম মোছা (আরেক রিলিজ)	আগে কোড বদলাতে হয়

সৎ ফাঁক: `RunPython.noop` ছাড়া কোনো reverse লেখা নেই — অর্থাৎ ডেটা মাইগ্রেশন (`0002_seed_plans`) হুবহু ফিরিয়ে আনা যায় না।

```
# ডেটা মাইগ্রেশন (billing/0002)
def forwards(apps, schema_editor):
    Plan = apps.get_model("billing", "Plan")
    Subscription = apps.get_model(
        "billing", "Subscription")
    ...
    # পুরনো প্রতিটি workspace-এ
    # একটি Subscription সারি

operations = [migrations.RunPython(forwards)]
```

`apps.get_model()` — ভবিষ্যতের মডেল নয়, ওই মুহূর্তের মডেল। এটাই ডেটা মাইগ্রেশনের একমাত্র নিরাপদ নিয়ম।

মাইগ্রেশন = ডেটাবেজের লাইফসাইকেল: প্রতিটি ফাইল ইতিহাসের একটি পৃষ্ঠা। হাতে বদলাবেন না, ফাইল মুছবেন না — কেবল সামনে এগোবেন।

শিক্ষকের নোট: প্রশ্ন করুন: "মাইগ্রেশন ফাইল মুছে দিলে কী হয়?" উত্তর: অন্য সব পরিবেশের ইতিহাস আলাদা হয়ে যায় — আর কেউ আর জানে না কোনটা সত্য।

রোলব্যাক

ভাঙলে কী করবেন — রোলব্যাকের তিনটি পথ

আর একটি অপ্রিয় সত্য: ডেটা বদলালে রোলব্যাক সহজ নয়

পথ	কখন	কী খেয়াল
সামনে এগিয়ে ঠিক করা	বাগ ছোট, ডেটা অটুট — সবচেয়ে নিরাপদ	দ্রুত ঠিক করা + লগ দেখা; ইতিহাস সৎ থাকে
আগের commit-এ ফেরা	নতুন কোডই সমস্যা, স্কিমা বদলায়নি	`git revert` + রিডিপ্লয়; migrate ফেরাতে হয় না
ব্যাকআপ থেকে রিস্টোর	ডেটা নষ্ট হয়েছে (ভুল ডিলিট, ভুল আপডেট)	সব লেখা হারাবে — শেষ ব্যাকআপের পরের সব কাজ

এখানে সবচেয়ে সহজ পথ নেই: কোনো ভার্সন-ট্যাগ নেই (একটিমাত্র ট্যাগ `c073-slice1`, যা slice ১-এর)। অর্থাৎ "গত সপ্তাহের কোডে ফিরুন" — এই নির্দেশ দেওয়ার মতো কোনো চিহ্ন নেই।

২০টি commit: প্রতিটির বার্তা পড়ে বোঝা যায় কী বদলেছে — ভালো অভ্যাস। কিন্তু রিলিজ-চিহ্ন ছাড়া ২০টি commit মানে ২০টি সম্ভাব্য ফেরার বিন্দু, একটিও চিহ্নিত নয়।

সংশোধন (এক লাইনের কাজ): প্রতি রিলিজে `git tag -a v1.4 -m "…"` — তখন "গত ট্যাগে ফেরো" একটি সম্পূর্ণ নির্দেশ হয়ে যায়।

শিক্ষকের নোট: শিক্ষার্থীদের জিজ্ঞেস করুন: "ডেটা বদলে গেলে রোলব্যাক করা যায়?" উত্তর: কোড ফেরানো যায়, ডেটা নয় — কেবল ব্যাকআপ থেকে, আর তাতে নতুন কাজ হারায়। তাই ডেটা-বদলানো রিলিজ আলাদা করে, ছোট করে।

নজর

মনিটরিং নেই — কিন্তু কী দেখতে হতো

পাঁচটি সংকেত, যার একটিও আজ দেখা যায় না

সংকেত	কী বলত	আজ কী আছে
জীবিত আছে?	প্রতি ৩০ সেকেন্ডে ২০০	`/healthz` নেই — গেটে অনুমতি আছে, ঠিকানা নেই
এরর হার	৫০০-এর সংখ্যা হঠাৎ বাড়লে সতর্কতা	Sentry নেই; `gunicorn` error log কেউ পড়ে না
ধীরতা	p95 রেসপন্স সময়	কোনো মাপ নেই — "ধীর লাগছে" কেবল অনুভব
ডেটাবেজ	সংযোগ সংখ্যা, বড় কোয়েরি	`CONN_MAX_AGE=60` সেট, কিন্তু নজর নেই
ব্যবসায়িক	দৈনিক কত স্কুল সক্রিয়	`billing_report` হাতে চালাতে হয়

পরিণতি: সমস্যা জানা যায় ব্যবহারকারীর ফোনে। ডেমো-প্রজেক্টে গ্রহণযোগ্য; আসল স্কুলে নয় — কারণ প্রথম জানার অধিকার গ্রাহকের নয়, অপারেটরের।

শিক্ষকের নোট: প্রশ্ন করুন: "গ্রাহক আগে জানবে, না আপনি?" — এই একটি প্রশ্নই মনিটরিংয়ের পুরো যুক্তি। প্রথম সংকেতটি (healthz) এক ঘণ্টার কাজ।

লগ

লগ কোথায় যায়, আর কী কখনো যাবে না

সব লগ `stdout`-এ — ফাইলে নয়, তাই ঘুরিয়ে ফেলা সহজ

```
LOGGING = {
  "handlers": {"console": {"class":
      "logging.StreamHandler"}},
  "root": {"handlers": ["console"],
           "level": env("LOG_LEVEL", "INFO")},
  "loggers": {
    "django.request": {"level": "WARNING"},
    "django.security": {"level": "WARNING"},
  },
}
```

gunicorn-এর `--access-logfile -` আর `--error-logfile -` — দুটোই একই স্রোতে।

নিয়ম	কারণ
ফাইলে লেখা হয় না	host/stdout সংগ্রহ করে; অ্যাপ নিজে ফাইল ঘোরাতে হয় না
`django.request` WARNING	প্রতিটি 4xx/5xx একটি এন্ট্রি — কিন্তু কেউ পড়ে না
মেয়াদ host-এর হাতে	অ্যাপের জানা নেই লগ কত দিন থাকে

অডিট মডেলের নিয়ম: "পাসওয়ার্ড, রিসেট টোকেন বা সিক্রেট কখনো সংরক্ষণ করবেন না" — `AuditLog`-এ কেবল শনাক্তকারী আর সংক্ষিপ্ত সারাংশ।

তিন ধরনের লগ, তিন জায়গা: অ্যাপ-এরর → stdout · ব্যবসায়িক কাজ → `AuditLog` টেবিল · অ্যাক্সেস → gunicorn। কোনটা কোথায়, তা জানা থাকলে সমস্যার সময় খোঁজা সহজ।

শিক্ষকের নোট: প্রশ্ন করুন: "ব্যবহারকারী বলল 'আমার নম্বর বদলে গেছে' — কোথায় খুঁজবেন?" উত্তর: `AuditLog` (কে বদলাল), তারপর gunicorn access log (কখন), শেষে অ্যাপ লগ (এরর ছিল কি না)।

ব্যাকআপ

`backup.sh` — ৬টি কার্যকর লাইন

`pg_dump -Fc` · media tar · ১৪ দিন · `set -euo pipefail`

```
set -euo pipefail
STAMP=$(date +%Y%m%d-%H%M%S)
DEST="${BACKUP_DIR:-./backups}"
mkdir -p "$DEST"
pg_dump -Fc -f "$DEST/ims-$STAMP.dump"
tar -czf "$DEST/media-$STAMP.tar.gz" media/ || true
find "$DEST" -name '*.dump' -mtime +14 -delete
```

সিদ্ধান্ত	কারণ
ডেটাবেজ + media আলাদা	লোগো/ম্যাটেরিয়াল ডাম্পে থাকে না
`-Fc` ফরম্যাট	টেবিল ধরে ধরে ফেরানো যায়
১৪ দিন	দুই সপ্তাহের মধ্যে ভুল ধরা পড়ে
`set -euo pipefail`	মাঝপথে ব্যর্থ = ব্যাকআপ নেই, আধা-ব্যাকআপ নয়

সীমা: ডিফল্ট গন্তব্য `./backups` — অর্থাৎ সার্ভারের ভেতরেই। সার্ভার হারালে ব্যাকআপও যায়। অফসাইট কপি নেই, এনক্রিপশন নেই, WAL/PITR নেই।

আরও একটি: `pg_dump` চালানোর অনুমতি ও ডাটাবেজ-নাম কোথাও লেখা নেই — স্ক্রিপ্ট ধরে নেয় আপনি ওই মেশিনে PostgreSQL-এর সুপারইউজার। বাস্তবে এটি প্রথমেই আটকে যায়।

শিক্ষকের নোট: "ব্যাকআপ আছে" বাক্যটি কখনো প্রমাণ নয়। যা প্রমাণ: একটি ফাইল, একটি তারিখ, একটি আকার — আর একটি সফল রিস্টোর।

ড্রিল

`restore.sh` — ব্যাকআপ ফেরানোর নিরাপদ পথ

আসল ডেটাবেজে কখনো সরাসরি রিস্টোর নয়

```
dropdb --if-exists "$SCRATCH"
createdb "$SCRATCH"
pg_restore -d "$SCRATCH" "$DUMP"

psql -d "$SCRATCH" -c "select 'students', count(*)
   from students_studentprofile
 union all select 'attendance', count(*)
   from attendance_attendancerecord
 union all select 'marks', count(*)
   from assessments_mark
 union all select 'audit', count(*)
   from audit_auditlog;"
```

নীতি	কারণ
স্ক্র্যাচ ডেটাবেজে প্রথমে	ভাঙা ডাম্প চালালে আসল ডেটা যায় না
৪টি টেবিলের সংখ্যা ছাপা	"ফেরানো হয়েছে" নয়, "কত সারি ফিরল" — যাচাইযোগ্য
swap কমান্ড মানুষ চালাবে	শেষ সিদ্ধান্ত স্বয়ংক্রিয় নয় — কারণ সেটি অপূরণীয়

সৎ ফাঁক: ড্রিলের ফল কোথাও লিপিবদ্ধ হয় না — কে, কবে, কত সারি ফিরে পেল, তার কোনো রেকর্ড নেই। মাসিক ড্রিল এক লাইনের কাজ, তবু হয় না।

মূলমন্ত্র: যে ব্যাকআপ কখনো ফেরানো হয়নি, সেটি ব্যাকআপ কি না জানা নেই — এমনকি ফাইলটি থাকলেও।

শিক্ষকের নোট: শিক্ষার্থীদের ছোট কাজ দিন: নিজের ডেমো-ডেটাবেজের একটি ডাম্প নিয়ে স্ক্র্যাচে ফিরিয়ে ৪টি সংখ্যা মেলাতে। যারা একবার করেছে, তারাই আসলে জানে।

জীবনচক্র

স্কুল চালু থেকে বন্ধ — চারটি অবস্থা

কোনো অবস্থাতেই ডেটা মোছা হয় না

অবস্থা	স্কুল যা দেখে	কী চলে	কী বন্ধ
ট্রায়াল	নীল ব্যানার + দিন-গণনা	সব	—
মেয়াদোত্তীর্ণ / past due	"Read-only mode"	পড়া + এক্সপোর্ট	সব লেখা
স্থগিত (প্ল্যাটফর্ম থেকে)	"Account suspended" + কারণ	পড়া	সব লেখা
বাতিল	পড়া	এক্সপোর্ট	সব লেখা

কারিগরি রূপ: `SubscriptionGateMiddleware` কেবল `SAFE_METHODS` ছাড়া বাকি সব থামায়; `ALLOW_PREFIXES` লগইন, রিনিউ, প্ল্যাটফর্ম ও স্ট্যাটিক খোলা রাখে।

যা এখনো নেই: UI-তে "স্কুল মুছুন" বাটন নেই। কেউ চাইলে `backup.sh` চালিয়ে Django admin থেকে workspace মুছতে হয় — অর্থাৎ মুছে ফেলা একটি **হাতে করা** অপারেশন, নকশা নয়।

নীতি: **অর্থ না দিলে ডেটা জিম্মি হয় না।** স্কুল তার রেকর্ড এক্সপোর্ট করে নিয়ে যেতে পারে — এটি চুক্তির নয়, কোডের নিয়ম।

শিক্ষকের নোট: এই স্লাইডটি নৈতিক সিদ্ধান্তের — "না দেওয়া মানে ডেটা আটকে রাখা নয়"। শিক্ষার্থীদের জিজ্ঞেস করুন: "এক স্কুল যদি সিস্টেম বদলাতে চায়, তার ডেটা নেওয়ার অধিকার আছে কি?"

ডেটা

কী জমা থাকে, কত দিন, আর কে দেখতে পারে

শিশুর ডেটা রাখা মানে বাড়তি দায়িত্ব

ডেটা	কোথায়	নিয়ম
ছাত্রের নাম, অভিভাবকের ফোন, জন্মতারিখ	`StudentProfile`	স্কুল নিজের ডেটা; অপারেটর কাজে ব্যবহার করে না
হাজিরা ও নম্বর	`AttendanceRecord` · `Mark`	কখনো মুছে ফেলা হয় না — সংশোধন হয়, লিপিবদ্ধভাবে
অডিট লগ	`AuditLog`	কে বদলাল, কখন, কেন — ৩০০ অক্ষরে
ইমেইল ও পাসওয়ার্ড	`User`	পাসওয়ার্ড হ্যাশ; কখনো লগে নয়, কখনো অডিটে নয়
ব্যাকআপ	সার্ভারের ডিস্ক	**১৪ দিন** — তারপর স্বয়ংক্রিয় মুছে যায়

সৎ ফাঁক: কোনো গোপনীয়তা-নীতি (privacy policy) বা ডেটা-মুছে-ফেলার নীতি লেখা নেই — `LAUNCH_PLAYBOOK` নিজেই বলেছে "Terms + privacy page — not in this repo yet"।

ব্যবহারিক নিয়ম: ব্যাকআপ ফাইলে পুরো স্কুলের ডেটা থাকে — সেটি হাতে পাওয়া মানে সব ডেটা হাতে পাওয়া। তাই ব্যাকআপ ডিস্কের অনুমতিও সমান গুরুত্বপূর্ণ।

শিক্ষকের নোট: "শিশুর ডেটা" — এই শব্দ দুটি জোর দিন। প্রাপ্তবয়স্ক ব্যবহারকারীর ডেটার চেয়ে ভিন্ন দায়িত্ব, ভিন্ন আইন, ভিন্ন নৈতিকতার প্রশ্ন।

রুটিন

প্রতিদিনের ১০ মিনিট — চারটি কাজ

অপস মানে বীরত্ব নয়, অভ্যাস

```
0 9 * * * cd /srv/ims && .venv/bin/python \
  manage.py billing_report | mail -s "IMS" you@x.com

0 2 * * * cd /srv/ims && BACKUP_DIR=/var/backups/ims \
  bash ops/backup.sh
```

প্রথম লাইনটি `LAUNCH_PLAYBOOK.md`-তে লেখা আছে; দ্বিতীয়টি লেখা নেই — কিন্তু দরকার।

কাজ	কী দেখবেন
`billing_report`	সক্রিয় স্কুল, MRR, ৭ দিনে শেষ হওয়া ট্রায়াল, unpaid ইনভয়েস
ব্যাকআপের ফাইল	আজকের ডাম্প আছে? আকার গতকালের মতো?
এরর-লগ	গত ২৪ ঘণ্টায় নতুন ৫০০?
ট্রায়াল-শেষ রিপোর্ট	স্কুলকে মনে করানো — রিনিউয়ের আসল কাজ

প্রথম ট্রায়াল-শেষ হাতে করতে হবে: স্বয়ংক্রিয় রিমাইন্ডার নেই — ১৪ দিন পর স্কুল চুপচাপ read-only হয়ে যাবে, আর কেউ টের পাবে না। `LAUNCH_PLAYBOOK` এটাই স্বীকার করেছে।

শিক্ষকের নোট: ক্রন-লাইনটি বোর্ডে লিখুন। "প্রতিদিন একটি ইমেইল" — এটাই সবচেয়ে সস্তা মনিটরিং, যা আজও কাজ করছে।

সংকট

কিছু ভাঙলে — পাঁচটি ধাপ

ক্রমটাই গুরুত্বপূর্ণ; প্রথম ধাপ কখনো "ঠিক করা" নয়

#	ধাপ	IMS-এ বাস্তবে
১	**থামাও**	ভুল মাইগ্রেশন চললে ডিপ্লয় থামান; সাইট নামানো দরকার হলে নামান
২	**মাপো**	কত স্কুল, কত সারি, কোন সময় থেকে — `AuditLog` + gunicorn log
৩	**আগে ব্যাকআপ**	**যেকোনো সংশোধনের আগে** `ops/backup.sh` — নাহলে ফেরার পথ নেই
৪	**ছোট করে ঠিক করো**	একটি commit, একটি কারণ; বড় রিফ্যাক্টর সংকটের সময় নয়
৫	**লিখে রাখো**	কী ঘটল, কেন, কী বদলাল — নাহলে দ্বিতীয়বার একই ভুল

উদাহরণ (কাল্পনিক নয়): কোনো স্কুলের সাবস্ক্রিপশন ভুলে মেয়াদোত্তীর্ণ দেখালে — গেট সব লেখা বন্ধ করবে, কিন্তু ডেটা অটুট। সমাধান বিলিং ডেটা ঠিক করা, রিস্টোর নয়।

সবচেয়ে খারাপ প্রতিক্রিয়া: আতঙ্কে সেটিং ঢিলা করা (`DEBUG=True` , গেট বন্ধ)। এটি সমস্যা থামায় না, লুকায়।

শিক্ষকের নোট: ৩ নম্বর ধাপে থামুন: "সংশোধনের আগে ব্যাকআপ"। শিক্ষার্থীরা উল্টো করে — আগে ঠিক করে, তারপর ভাবে। ভুল ডেটা বদলালে ফেরার পথ আর থাকে না।

ধারণক্ষমতা

একটি স্কুল এক বছরে কত সারি বানায়

হিসাবটা সরল; মাপাটা হয়নি — দুটোই সত্য

হিসাব (৮০০ ছাত্রের স্কুল)	সারি/বছর
হাজিরা: ৮০০ × ৮ বিষয় × ২০০ দিন	১২,৮০,০০০
নম্বর: ৮০০ × ৮ × ৩ পরীক্ষা	১৯,২০০
অডিট: প্রতিদিন ~২০০ কাজ	৭৩,০০০
এনরোলমেন্ট + হোমওয়ার্ক + সাবমিশন	~৩০,০০০

মূল চাপ হাজিরায় — একাই বাকি সব মিলিয়ে যা হয় তার প্রায় ১০ গুণ।

সম্পদ	আজকের অবস্থা
gunicorn worker	৩টি · timeout ৬০ সে
DB সংযোগ	`CONN_MAX_AGE=60` · worker-প্রতি একটি
ডিস্ক	`media/` স্থায়ী ভলিউমে থাকতে হবে
লোড-টেস্ট	**কখনো হয়নি**

তাই "১০০০ ব্যবহারকারী চলবে?" প্রশ্নের সৎ উত্তর: "জানি না — মাপা হয়নি।" ইনডেক্স আছে, কিন্তু `EXPLAIN` করে দেখা হয়নি।

পরের ধাপ স্পষ্ট: বড় স্কুলের ডেমো-ডেটা বানিয়ে (৫০ হাজার সারি) আগে `EXPLAIN ANALYZE`, তারপর দরকার হলে লোড-টেস্ট। অনুমান দিয়ে স্কেলিং নয়।

শিক্ষকের নোট: হিসাবটি বোর্ডে করুন — শিক্ষার্থীরা অবাক হয় দেখে যে হাজিরাই ৯০% ডেটা। এখান থেকেই বোঝা যায় কেন `(class_group, date)` ইনডেক্স সবচেয়ে জরুরি।

সততা

যা লেখা আছে, কিন্তু কখনো চালানো হয়নি

প্ল্যান আর প্রমাণের পার্থক্য — এই স্লাইডটাই সেই পার্থক্য

জিনিস	অবস্থা	প্রথমবার চালালে যা ভাঙতে পারে
`ops/backup.sh`	লেখা, চালানো হয়নি	`pg_dump`-এর অনুমতি, `media/` না থাকলে ব্যর্থ
`ops/restore.sh`	লেখা, ড্রিল হয়নি	স্ক্র্যাচ DB-র অনুমতি; swap ধাপ হাতে
`docs/deploy_check.sh`	লেখা, দরকার `.env.production.local`	ফাইলটি রিপোতে নেই — নতুনদের প্রথম বাধা
প্রোডাকশন ডিপ্লয়	**কখনো হয়নি**	DEPLOYMENT.md নিজেই স্বীকার করেছে
লোড-টেস্ট / EXPLAIN	হয়নি	ধীরতার প্রশ্নের উত্তর নেই
ট্যাগ-ভিত্তিক রিলিজ	শুধু একটি পুরনো ট্যাগ	"গত রিলিজে ফেরো" বলা যায় না

এগুলো ব্যর্থতা নয় — **অসমাপ্ত কাজ**। পার্থক্য একটাই: অসমাপ্ত কাজের তালিকা থাকে, ব্যর্থতা লুকানো থাকে।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন নিজের প্রজেক্টে এমন একটি তালিকা লিখতে — "যা লিখেছি কিন্তু চালাইনি"। বেশিরভাগের তালিকায় ডিপ্লয়, ব্যাকআপ আর টেস্ট থাকবে।

সারমর্ম

ছয়টি নীতি, আর চারটি কাজ

অপস শেখা যায় তিনভাবে: পড়ে, করে, আর ভুল করে

- **যে ব্যাকআপ ফেরানো হয়নি, সেটি ব্যাকআপ নয়** — ফাইল থাকলেই প্রমাণ হয় না
- **মাইগ্রেশন কেবল সামনে** — ডেটা বদলালে রোলব্যাক মানে ব্যাকআপ, কোড নয়
- **সংশোধনের আগে ব্যাকআপ** — উল্টো ক্রমে করলে ফেরার পথ হারায়
- **গ্রাহকের আগে আপনার জানা উচিত** — মনিটরিং এজন্যই, সৌন্দর্য নয়
- **অর্থ না দিলে ডেটা জিম্মি হয় না** — read-only, কখনো মুছে ফেলা নয়
- **মাপা হয়নি মানে জানা নেই** — ধারণক্ষমতার প্রশ্নে "জানি না" বলাই সৎ উত্তর

চারটি কাজ (আজ): ১) প্রতি রিলিজে tag · ২) `/healthz` · ৩) ক্রনে backup + মাসিক restore drill · ৪) `.env.example`-এ `DB_*` যোগ করা।

শিক্ষকের নোট: চারটি কাজের প্রতিটি ১০–৩০ মিনিটের। শিক্ষার্থীদের একটি বেছে আজই করতে বলুন — শেখার সবচেয়ে কার্যকর রূপ।

D5 — ডিজাইন সিস্টেম

সিস্টেম চলছে; এখন তাকে দেখতে যেমন লাগে, তেমন বানানো · ১৮ স্লাইড

গভীর ডেক · D5

ডিজাইন সিস্টেম

১৫টি টোকেন · ৫৯টি ক্লাস · ৮ KB নিজের CSS · ৫৩৪ KB vendor · ৭৫ টেমপ্লেট — একটি পর্দা কেন এমন দেখতে

শুরু

ডিজাইন সিস্টেম মানে CSS নয় — সিদ্ধান্তের তালিকা

প্রতিটি নতুন পর্দা পুরনো পর্দার মতো দেখতে হয়, কারণ সিদ্ধান্ত আগেই নেওয়া

টোকেন

১৫টি CSS ভেরিয়েবল — রঙ, ব্যাসার্ধ, ছায়া। এক জায়গায় বদলালে পুরো অ্যাপ বদলায়।

`static/css/ims.css` · ৮ KB

কম্পোনেন্ট

৫৯টি ক্লাস — card, kpi, badge, empty, banner, nav, form, alert। একই নাম সব পর্দায়।

১,০০৭টি ব্যবহার, ৫৪টি ভিন্ন ক্লাস

ভিত্তি

Bootstrap **৫** vendor-এ — গ্রিড আর ফর্মের ভিত্তি, নিজে চাকা বানানো হয়নি।

CDN নেই — নিজের ফাইলে

সাদা-লেবেল

স্কুল নিজের রঙ দিলে **একটি ভেরিয়েবল** বদলায় — বাকি সব নিজে থেকে বদলে যায়।

`brand.color` → `--ims-brand`

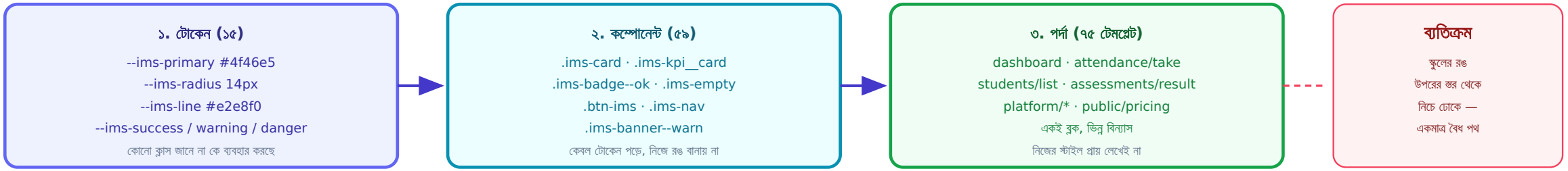
এই ডেকের সৎ অবস্থান: সিস্টেমটি ছোট ও কাজ করে, কিন্তু এর একটি নথিও নেই — কোন টোকেন কী অর্থে, তা কেবল CSS পড়ে জানা যায়।

শিক্ষকের নোট: শিক্ষার্থীদের জিজ্ঞেস করুন: "নতুন পর্দা বানাতে কতটা CSS লিখতে হবে?" উত্তর: প্রায়শই **শূন্য** — শুধু পুরনো ক্লাস ব্যবহার। এটাই ডিজাইন সিস্টেমের লাভ।

স্তর

তিন স্তর: টোকেন → কম্পোনেন্ট → পর্দা

নিচের স্তর বদলালে উপরের সব বদলায়; উপরের স্তর কখনো নিচকে বদলায় না



নিয়ম: পর্দায় কখনো color:#dc2626 লেখা হয় না — লেখা হয় class="ims-badge--warn" জাতীয় অর্থবাহী নাম।

রঙের নাম নয়, অর্থ ব্যবহার করুন — কারণ স্কুল নিজের রঙ বদলালে অর্থ একই থাকে, নাম বদলায়।

শিক্ষকের নোট: নিয়মটি বোর্ডে লিখুন: "সিদ্ধান্ত একবার, ব্যবহার বহুবার"। শিক্ষার্থীদের নিজের প্রজেক্টে খুঁজতে বলুন — কোথায় কোথায় সরাসরি রঙ লেখা আছে, যা বদলাতে সব জায়গায় হাত দিতে হয়।

টোকেন

১৫টি টোকেন — পুরো সিস্টেমের অভিধান

`:root`-এ একবার; ৫৯টি ক্লাস এগুলোই পড়ে

রঙের টোকেন	মান
`--ims-primary`	#4f46e5 (ইন্ডিগো)
`--ims-primary-dark`	#4338ca (hover)
`--ims-primary-soft`	#eef2ff (pill/badge)
`--ims-ink` · `--ims-ink-2`	#0f172a · #334155
`--ims-muted`	#64748b (সহায়ক লেখা)
`--ims-line` · `--ims-bg` · `--ims-surface`	#e2e8f0 · #f6f7fb · #ffffff
`--ims-success` · `--warning` · `--danger`	#059669 · #d97706 · #dc2626

আকৃতির টোকেন	মান
`--ims-radius`	14px (কার্ড)
`--ims-radius-sm`	10px (বাটন/ইনপুট)
`--ims-shadow`	দুই স্তরের নরম ছায়া

কেন ১৪px আর ১০px: বড় পৃষ্ঠ (কার্ড) বেশি বাঁকা, ছোট উপাদান (বাটন) কম — একই ব্যাসার্ধ দুটোতে দিলে একটি ভারী দেখায়।

সৎ ফাঁক: টাইপ-স্কেল, স্পেসিং-স্কেল (৪/৮/১৬px), z-index — এগুলোর কোনো টোকেন নেই; মানগুলো ক্লাসের ভেতরে ছড়ানো।

শিক্ষকের নোট: জিজ্ঞেস করুন: "স্কুলের রঙ বদলালে কোন কোন জিনিস বদলায়?" উত্তর: লিংক, বাটন, সক্রিয় মেনু, sidebar-এর গ্রেডিয়েন্ট — কারণ সবাই `--ims-brand` পড়ে।

রঙ

রঙের অর্থ — কোন অবস্থায় কোনটা

রঙ সাজসজ্জা নয়, সংকেত; একই রঙ দুই অর্থে ব্যবহার করলে সংকেত ভাঙে

টোকেন	কোথায় ব্যবহার	অর্থ
primary #4f46e5	বাটন, সক্রিয় মেনু, লিংক, KPI আইকন	"এটি করো" — এক পর্দায় একটাই প্রাথমিক কাজ
success #059669	`ims-badge--ok` , `alert-success`	সম্পন্ন · উপস্থিত · সক্রিয়
warning #d97706	`ims-badge--warn` , `ims-banner--warn` , `ims-slice-note`	খেয়াল করুন · ট্রায়াল শেষ হচ্ছে · read-only
danger #dc2626	`ims-error` , `alert-danger`	ভুল হয়েছে · হবে না
muted #64748b	`ims-hint` (১৬৫ বার — সবচেয়ে বেশি)	সাহায্যকারী লেখা, পড়া বাধ্যতামূলক নয়

মাপা কনট্রাস্ট: sidebar-এ #cbd5e1 / #0f172a = **12.0:1** (উত্তম) · primary-তে সাদা = **6.3:1** (AA পাশ)

মাপা ব্যর্থতা: sidebar-এর `--ims-muted` #64748b / #0f172a = **3.75:1** — AA-এর ন্যূনতম 4.5:1 ছোঁয় না। ছোট লেবেলগুলো দুর্বল।

আরেকটি: সবুজ #059669-এ সাদা লেখা = **3.77:1** — তাই success ব্যাজে সবুজ লেখা হালকা পটভূমিতে ব্যবহৃত হয়েছে, সাদা সবুজে নয়।

শিক্ষকের নোট: কনট্রাস্টের হিসাবটি বোর্ডে করুন — এটাই "ডিজাইন যাচাইযোগ্য" হওয়ার প্রমাণ। রুচি বিতর্কের বিষয়, 3.75:1 গণনার বিষয়।

কম্পোনেন্ট

৫টি ব্লক যা ৭৫টি পর্দা বহন করে

সংখ্যাগুলো টেমপ্লেটে ব্যবহারের গণনা — অনুমান নয়

ক্লাস	বার	কী কাজ করে / নিয়ম
`ims-card` + `__head` + `__body`	৯৬ + ৫৪ + ১০৩	পর্দার মূল ব্লক — শিরোনাম, তারপর বিষয়বস্তু; কার্ড ছাড়া কনটেন্ট বসানো হয় না
`ims-hint`	১৬৫	ফিল্ডের নিচে সাহায্য-text — সবচেয়ে বেশি ব্যবহৃত; কারণ প্রতিটি ইনপুট ব্যাখ্যা চায়
`ims-badge` (+ `--ok` ২৭)	৭১	অবস্থার সংকেত — ছাত্র সক্রিয়, ট্রায়াল চলছে, unpaid
`btn-ims` / `btn-ims-outline`	৪৯ / ৬৭	প্রাথমিক কাজ ভরাট বাটন, দ্বিতীয় কাজ রূপরেখা — এক পর্দায় একটাই ভরাট
`ims-empty`	৪১	খালি তালিকার ব্যাখ্যা — "কিছু নেই" নয়, "কী করলে আসবে"
`ims-kpi__card` + `__label` + `__value`	৩৭ + ৩৭ + ৩৫	ড্যাশবোর্ডের সংখ্যা-কার্ড — লেবেল ছোট বড়হাতের, মান বড়

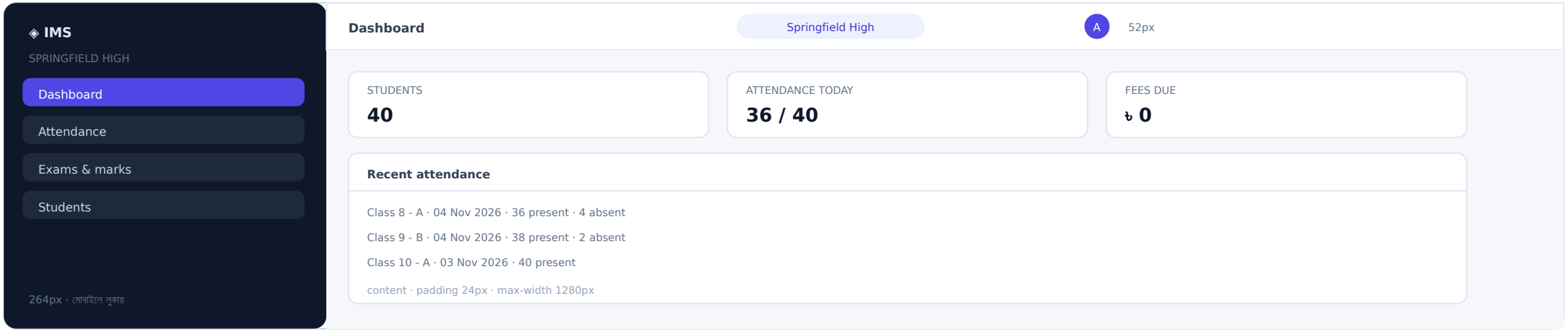
যা দেখা যায়: `ims-hint` ও `ims-card__body`-র সংখ্যা সবচেয়ে বেশি — অর্থাৎ এই অ্যাপের আসল সমস্যা "দেখানো" নয়, "বোঝানো": প্রতিটি স্ক্রিনে ব্যাখ্যা দরকার।

শিক্ষকের নোট: এই টেবিলটি ব্যবহারের পরিসংখ্যান, রুচি নয়। শিক্ষার্থীদের নিজের প্রকল্পে একই গণনা করতে বলুন — কোন ক্লাস সবচেয়ে বেশি, সেটাই আসল ডিজাইন-সমস্যা বলে দেয়।

খাঁচা

অ্যাপ-শেল: ২৬৪px সাইডবার, ৫২px টপবার, ১২৮০px কনটেন্ট

তিনটি সংখ্যা পুরো অ্যাপের বিন্যাস ঠিক করে



একই খাঁচা সব ভূমিকায়: শিক্ষক, সমন্বয়ক, ছাত্র — সাইডবারের মেনু বদলায় (`navigation_for()`), খাঁচা বদলায় না। তাই একজন শিক্ষক অন্য ভূমিকায় গেলে পর্দা চেনা থাকে।

শিক্ষকের নোট: জিজ্ঞেস করুন: "সাইডবার ২৬৪px কেন?" উত্তর: দীর্ঘ মেনু-নাম ("Exams & marks") কাটা না পড়ে ঢোকার মতো জায়গা — সংখ্যাটি রুচি নয়, পরিমাপ।

খালি

`ims-empty` — ৪১ বার, আর প্রতিটি একটি সিদ্ধান্ত

"কিছু নেই" লেখা সহজ, কিন্তু ব্যবহারকারী তখন কী করবে?

অবস্থা	ভালো বার্তা কী বলে
কোনো ছাত্র নেই	"এখনো কোনো ছাত্র যোগ করা হয়নি" + **Add student** বাটন
আজ হাজিরা নেওয়া হয়নি	"আজকের রেজিস্টার খালি" + **Take attendance**
ফিল্টারে কিছু মেলেনি	"এই শর্তে কাউকে পাওয়া যায়নি" + **Clear filters**
ভূমিকার জন্য কিছু নেই	"আপনার দেখার মতো কিছু নেই" — বাটন নেই, কারণ করার কিছু নেই

পার্থক্য: প্রথম তিনটি **খালি**, চতুর্থটি **শূন্য**। একই পর্দা, ভিন্ন অর্থ — ভিন্ন বার্তা দরকার।

```
{% else %}


    No students match these filters.


{% endif %}

{% else %}


    No attendance recorded this month
    for this class.


{% endif %}
```

আসল কোড: বাটন নেই, কেবল ব্যাখ্যা। **প্রস্তাব:** যেখানে ব্যবহারকারীর অধিকার আছে, সেখানে একটি বাটন যোগ করা — এখনো করা হয়নি।

সৎ ফাঁক: ৪১টি `ims-empty` ব্লক আছে, কিন্তু তাদের বার্তাগুলো একরকম নয় — কেউ স্পষ্ট নির্দেশ, কেউ শুধু বিবরণ — আর কোনোটিতেই পরবর্তী কাজের বাটন নেই।

শিক্ষকের নোট: খালি পর্দা নিয়ে শিক্ষার্থীরা ভাবে না — অথচ প্রথম দিনেই ব্যবহারকারী ঠিক সেখানেই দাঁড়ায়। জিজ্ঞেস করুন: "নতুন স্কুল প্রথম লগইনে কী দেখবে?"

সাদা-লেবেল

স্কুলের রঙ ঢোকে একটি ভেরিয়েবল দিয়ে

`base.html`-এ ৯ লাইন — আর ৭৫টি পর্দার রঙ বদলে যায়

```
<style>
  :root {
    --ims-indigo: {{ brand.color|default:"#4f46e5" }};
    --ims-brand:  {{ brand.color|default:"#4f46e5" }};
  }
  .btn-ims, .ims-nav.is-active,
  .ims-auth__submit { background: var(--ims-brand); }
  .ims-sidebar { background: linear-gradient(180deg,
      color-mix(in srgb, var(--ims-brand) 22%, #0b1220), #0b1220); }
  a { color: var(--ims-brand); }
</style>
```

কৌশল	কেন
CSS ভেরিয়েবল ইনজেক্ট	৭৫ টেমপ্লেটে একটি রঙের নাম লিখতে হয় না
`color-mix()`	স্কুলের রঙ যাই হোক, sidebar-এর গাঢ় সংস্করণ নিজে বানায়
`default:"#4f46e5"`	রঙ না দেওয়া স্কুলও ভাঙে না
শুধু ভরাট রঙ, লেখার রঙ নয়	লেখার রঙ বদলালে কনট্রাস্ট ভাঙতে পারত

সীমা: স্কুল যদি খুব হালকা রঙ দেয় (যেমন #fde047), সাদা লেখা পড়া যায় না — কনট্রাস্ট যাচাই করা হয় না, কেবল ফরম্যাট যাচাই হয়।

একই পদ্ধতি, দুই স্তর: পণ্যের নামও একইভাবে আসে (`product.name` সেটিংস থেকে), তাই "IMS" শব্দটি কোথাও হার্ডকোড নয় — একবার `PRODUCT_NAME` বদলালে পুরো অ্যাপ বদলায়।

শিক্ষকের নোট: জিজ্ঞেস করুন: "স্কুলকে পুরো রঙ-তালিকা দেওয়া যায় না কেন?" উত্তর: ১২টি রঙের সমন্বয় ঠিক রাখা অসম্ভব; একটি রঙ দিয়ে বাকিটা নিজে বানানোই নিরাপদ।

ঝুঁকি

রঙ একটি ইনপুট — তাই এটি নিরাপত্তার প্রশ্নও

ব্যবহারকারীর দেওয়া মান `<style>`-এর ভেতরে বসছে

```
def clean(self):
    color = self.primary_color
    if color and not (color.startswith("#")
                      and len(color) == 7):
        raise ValidationError({
            "primary_color":
            "Use a 6-digit hex colour like #0f766e."})
```

দুটি শর্ত: `#` দিয়ে শুরু, ঠিক ৭ অক্ষর।

না থাকলে কী হতো: কেউ মান দিত `red; } body{display:none} .x{` — ৭ অক্ষরের সীমা ও `#` শর্ত দুটিই এই ধরনের ইনজেকশন ভেঙে দেয়।

তবু যা বাকি: `#` দিয়ে শুরু আর ৭ অক্ষর মানেই বৈধ hex নয় — `#zzzzzz`-ও পাশ করে; শুধু রঙ কাজ করে না। কঠোর নিয়ম হতো `RegexValidator(r"^#[0-9a-fA-F]{6}$")`।

প্রমাণ: রঙের ভ্যালিডেশন নিয়ে কোনো টেস্ট নেই — `#zzzzzz` ঢুকে গেলে কেউ ধরবে না।

সাধারণ নিয়ম: ব্যবহারকারীর দেওয়া যেকোনো স্ট্রিং যদি কোডে/স্টাইলে বসে, তবে সেটি ভ্যালিডেট না করে বসানো মানে কোড-ইনজেকশনের দরজা খোলা।

শিক্ষকের নোট: এই স্লাইডটি ডিজাইন আর নিরাপত্তার সেতু — দেখান যে "রঙের ইনপুট" আর "HTML ইনপুট" একই পরিবারের সমস্যা। `#zzzzzz`-এর উদাহরণটি বোর্ডে লিখুন।

পর্দার আকার

দুটি breakpoint — আর একটি অসম্পূর্ণ সিদ্ধান্ত

৯০০px · ৮২০px — মোবাইলে কী লুকায়, কী থাকে

breakpoint	কী বদলায়
`max-width:900px`	লগইন পাতার দুই কলাম এক কলাম হয়; বাঁ দিকের পিচ-প্যানেল **লুকায়**
`max-width:820px`	অ্যাপ-শেল এক কলাম; **সাইডবার লুকায়**, কনটেন্ট পুরো প্রস্থ
Bootstrap গ্রিড	KPI কার্ড `auto-fit minmax(190px,1fr)` — নিজে থেকে ভাঁজ হয়

অসম্পূর্ণ: মোবাইলে সাইডবার লুকালে **নেভিগেশনের কোনো বিকল্প নেই** — হ্যামবার্গার মেনু নেই। শিক্ষক ফোনে হাজিরা নিতে গেলে অন্য পাতায় যাওয়ার পথ নেই।

কেন এটা গুরুত্বপূর্ণ: বাংলাদেশে অনেক শিক্ষকের একমাত্র ডিভাইস ফোন। ডেস্কটপে-কেবল ডিজাইন মানে অর্ধেক ব্যবহারকারী বাদ।

যা ভালো কাজ করে: ১০টি টেমপ্লেটে টেবিল `table-responsive`-এ মোড়া — ফোনে টেবিল ভাঙে না, কেবল ডানে-বাঁয়ে সরাতে হয়।

সংশোধনের পথ: একই মিডিয়া-কোয়েরিতে একটি বাটন + অফ-ক্যানভাস সাইডবার — CSS-এ ২০ লাইনের কাজ।

শিক্ষকের নোট: ফোনে ডেমো খুলে দেখান — সাইডবার নেই। শিক্ষার্থীদের জিজ্ঞেস করুন: "এখন অন্য পাতায় যাবেন কীভাবে?" হাতে-কলমে দেখা ফাঁক কখনো ভোলা যায় না।

ফর্ম

প্রতিটি ইনপুটে তিনটি প্রশ্নের উত্তর

লেবেল · সাহায্য · ভুলের কারণ — তিনটি আলাদা ক্লাস

```
<label class="form-label"
       for="id_minimum_percent">
  Minimum attendance %
</label>
<input class="form-control" id="id_minimum_percent"
       name="minimum_percent" value="75">
<p class="ims-hint">
  Students below this are flagged in reports.</p>
<p class="ims-error">{{ field.errors }}</p>
```

ক্লাস	কী বলে
`form-label`	"এই ঘরে কী বসবে" — ০.৮৬rem, গাঢ়
`ims-hint`	"কেন, কীভাবে" — muted, ছোট (১৬৫ বার)
`ims-error`	"কী ভুল হয়েছে" — danger রঙ
`alert-danger`	পুরো ফর্ম ব্যর্থ হলে উপরে সারাংশ

নিয়ম: ভুল বার্তা কখনো "Invalid input" নয় — "Pass marks cannot exceed full marks" (আসল কোডে ঠিক এই বাক্য)।

সৎ ফাঁক: ভুলগুলো ফিল্ডের নিচে দেখানো হয়, কিন্তু ফোকাস স্বয়ংক্রিয়ভাবে প্রথম ভুল ফিল্ডে যায় না — বড় ফর্মে (মার্কস এন্ট্রি, ৪০ সারি) ব্যবহারকারীকে খুঁজতে হয়।

শিক্ষকের নোট: জিজ্ঞেস করুন: "কেন hint সবচেয়ে বেশি ব্যবহৃত ক্লাস?" উত্তর: কারণ এই ডোমেইনে ব্যবহারকারী ক্লার্ক নন — "minimum percent" কী, তা ব্যাখ্যা করতে হয়। ভালো UI লেখার কাজ, সাজানোর নয়।

আইকন

আইকন ও ছবি — পাঁচটি নিয়ম

Bootstrap Icons vendor-এ; কোনো ছবি CDN থেকে নয়

নিয়ম	কারণ
আইকন কেবল শোভা নয়, সংকেত	`bi-people` খালি ছাত্র-তালিকায়, `bi-grid-3x3` হাজিরায়
ইমোজি নয়	ফন্ট-নির্ভর, প্ল্যাটফর্মে বদলায়, রঙ নিয়ন্ত্রণহীন
লোগো ৩৪/৩৮px	সাইডবার ৩৪px, লগইন প্যানেল ৩৮px — দুই জায়গায় দুই আকার, একটাই অনুপাত
লোগো না থাকলে অক্ষর	`brand.initial` — প্রথম অক্ষর বৃত্তে; নতুন স্কুলও পূর্ণ দেখায়
`object-fit: contain`	স্কুলের লোগো লম্বা/চওড়া যা-ই হোক, কাটা পড়ে না

ছবি কোথা থেকে: শুধু ব্যবহারকারীর আপলোড (`media/`) — কোনো স্টক ছবি বা আইকন-ফন্ট CDN-এ নেই। ফলে নেট বন্ধ থাকলেও পর্দা সম্পূর্ণ।

দাম: vendor ফোল্ডারে ৫৩৪ KB — নিজের সার্ভার থেকেই যায়; প্রথম লোডে CDN-এর চেয়ে ধীর, কিন্তু নির্ভরযোগ্য।

সৎ ফাঁক: আইকন-ফন্টের ভার্সন কোথাও লেখা নেই; আপডেট করতে গেলে জানতে হবে কোন সংস্করণ।

শিক্ষকের নোট: "ইমোজি নয়" নিয়মটি ব্যাখ্যা করুন: একই ইমোজি Windows-এ রঙিন, পুরনো ফোনে সাদা-কালো — ব্র্যান্ডের অংশ হিসেবে ব্যবহার করা যায় না।

প্রিন্ট

পর্দা আর কাগজ — দুই রকম ডিজাইন

রেজাল্ট-শিট PDF reportlab দিয়ে, HTML থেকে নয়

জিনিস	কীভাবে
রেজাল্ট-শিট	`reportlab` দিয়ে সরাসরি PDF — A4, স্কুলের নাম ও লোগো
স্ক্রিনশট	`docs/shoot.py` · Playwright · ৭৭টি PNG
এক্সপোর্ট	`openpyxl` → XLSX · CSV
প্রিন্ট CSS	**নেই** — `@media print` একটিও লেখা হয়নি

পরিণতি: ব্রাউজারে Ctrl+P চাপলে সাইডবার, টপবার ও বাটনগুলোও ছাপা হবে — কারণ লুকানোর নিয়ম নেই।

কেন PDF আলাদা করে: কাগজের ডিজাইন আলাদা — মার্জিন, পৃষ্ঠা-বিভাজন, স্কুলের হেডার, স্বাক্ষরের জায়গা। HTML প্রিন্ট করে সেটি পাওয়া যায় না, তাই PDF নিজে আঁকা।

স্ক্রিনশটের নিয়ম: একই আকার (১৪৪০×৯০০), একই জুম, একটাই ডেমো-ডেটা — তাই ৭৭টি ছবি পাশাপাশি রাখলে একই অ্যাপ মনে হয়।

সংশোধনের পথ: ২০ লাইনের `@media print` — সাইডবার/টপবার `display:none`, টেবিলের সীমানা কালো, লিংকের রঙ কালো।

শিক্ষকের নোট: স্কুলের আসল কাজ কাগজে শেষ হয় — রেজাল্ট-শিট ছাপা হয়, স্বাক্ষর হয়, বন্ধ করা হয়। তাই "প্রিন্ট" ডিজাইন সিস্টেমের বাইরের জিনিস নয়, তার একটি অংশ।

স্তর

তিন স্তরের বার্তা — ব্যাজ, ব্যানার, অ্যালার্ট

কতটা জরুরি, তার ভিত্তিতে কোনটা কোথায়

স্তর	কোথায়	ব্যবহার (মাপা)
`ims-badge`	টেবিলের সেলের ভিতরে	৭১ বার — ok ২৭ · warn ১১ · info ১০
`ims-banner`	পৃষ্ঠার একদম উপরে	৭ বার — warn ৬ · info ১
`alert-*`	ফর্মের পরে ফলাফল	danger ৫ · warning ৫ · info ৩

```
{% if subscription.is_grace %}
<div class="ims-banner ims-banner--warn">
  <i class="bi bi-exclamation-triangle"></i>
  <div>Grace period ends in {{ days_left }} days.</div>
</div>
{% endif %}
```

নিয়ম: ব্যাজ = তথ্য ("Paid"), ব্যানার = সময়ভিত্তিক সতর্কতা ("৫ দিন বাকি"), অ্যালার্ট = কাজের ফল ("সেভ হয়েছে")। একটি দিয়ে অন্যটা করলে ব্যবহারকারী সতর্কতা চিনতে ভুল করে।

সৎ ফাঁক ১: `ims-badge--danger` নেই — লাল ব্যাজ দরকার হলে Bootstrap-এর `text-danger` ব্যবহার করা হয়। অর্থ একই, ক্লাস আলাদা।

সৎ ফাঁক ২: `alert-success` ও `alert-secondary` CSS-এ সংজ্ঞায়িত, কিন্তু টেমপ্লেটে একবারও ব্যবহৃত নয় — মৃত কোড।

কোথায় বসে: ব্যাজ → টেবিলের সেল · ব্যানার → পৃষ্ঠার উপরে, পুরো প্রস্থ · অ্যালার্ট → ফর্ম/কাজের পরে · ব্যাজ কখনো পৃষ্ঠার উপরে নয় — সেটি ব্যানারের কাজ

শিক্ষকের নোট: তিন স্তরের পার্থক্য একটা বাক্যে বলুন: "ব্যাজ জিনিসটা কী, ব্যানার সময় কত বাকি, অ্যালার্ট এইমাত্র কী হলো।" ছাত্ররা প্রায়ই সবকিছু alert দিয়ে করে — কোর্সের সবচেয়ে সাধারণ UI ভুল।

ড্যাশবোর্ড

KPI কার্ড — এক নজরে সংখ্যা, তারপর পথ

`auto-fit minmax(190px,1fr)` — কোনো মিডিয়া কোয়েরি ছাড়াই

```
.ims-kpi{display:grid;
  grid-template-columns:repeat(auto-fit,minmax(190px,1fr));
  gap:1rem}
.ims-kpi__card{background:var(--ims-surface);
  border:1px solid var(--ims-line);border-radius:14px;
  padding:1rem 1.1rem}
.ims-kpi__value{font-size:1.75rem;font-weight:700}
.ims-kpi__label{font-size:.78rem;color:var(--ims-muted);
  text-transform:uppercase;letter-spacing:.04em}
```

কেন `auto-fit`: ১৯০px-এ এক কলাম ভরে না মানে দুই কলাম — ব্রাউজার নিজেই ঠিক করে। ৩ কার্ড থাকলে ৩ কলাম, ফোনে ১ কলাম। কোডে একটি মাত্র লাইন।

ক্লাস	ব্যবহার
`ims-kpi__card`	৩৭
`ims-kpi__label`	৩৭
`ims-kpi__value`	৩৫
`ims-kpi__icon`	১৩
`ims-kpi` (গ্রিড)	৯

ছন্দ: ১০টি পর্দায় ৩৭টি কার্ড — গড়ে ৩.৭। প্রতিটি বাক্সে ৩–৪টি কার্ড, তাই গ্রিড কখনো অর্ধেক খালি দেখায় না। সংখ্যাটা কাকতালীয় নয়, ইচ্ছাকৃত ছন্দ।

শিক্ষকের নোট: বোর্ডে ১৯০px × ৩ = ৫৭০px লিখুন, তারপর প্রশ্ন করুন — ৬৪০px পর্দায় কয়টি কার্ড বসবে? (৩টি, কারণ padding বাদ দিয়ে জায়গা কমে)। এটাই CSS Grid বোঝানোর সবচেয়ে দ্রুত পথ।

নির্ভরতা

নিজের ৮ KB বনাম Bootstrap-এর ৫৩৪ KB

কী কিনছি, কী ফেলে দিচ্ছি — এবং দুটোর সংঘর্ষ কোথায়

ফাইল	আকার	কী কাজে
`ims.css`	৭,৯৯০ B	নিজের ৫৯ ক্লাস
`bootstrap.min.css`	২৩২,৭৫৮ B	গ্রিড, ইউটিলিটি, `alert`, `table-responsive`
`bootstrap.bundle.min.js`	৮০,৬৭৩ B	ড্রপডাউন, ট্যাব, মোডাল
`bootstrap-icons.css` + ফন্ট	৮৫,৭৬০ + ১,৩০,৩৯৬ B	আইকন

অনুপাত: নিজের কোড মোটের ১.৫%। কিন্তু পর্দায় যা দেখা যায় তার বেশিরভাগই নিজের কোড — কারণ Bootstrap-এর ব্যবহার মূলত কাঠামোয়, চেহারায় নয়।

সংঘর্ষ: Bootstrap নিজেও `.alert`, `.badge`, `.table` সংজ্ঞায়িত করে। `ims.css` পরে লোড হয়, তাই নিজের নিয়ম জেতে — কিন্তু লোডের ক্রম বদলালে চেহারা বদলে যাবে।

বিকল্প যা ভাবা হয়নি: Bootstrap-এর শুধু গ্রিড+ইউটিলিটি বেছে নিয়ে (কাস্টম বিল্ড) ২৩২ KB থেকে ~৪০ KB-তে নামা যেত। করা হয়নি, তাই এখন পুরোটা যায়।

সৎ ফাঁক: কোনো মিনিফিকেশন বা ফিঙ্গারপ্রিন্টিং নেই — ফাইল বদলালেও ব্রাউজারের ক্যাশ একই থাকে, ব্যবহারকারী পুরনো CSS দেখতে পারে।

শিক্ষকের নোট: এখানেই "ফ্রেমওয়ার্ক বনাম নিজের CSS" বিতর্কের আসল উত্তর: দুইটাই ঠিক, কিন্তু লোডের ক্রম ও ভার্সন লিখে রাখা বাধ্যতামূলক। না লিখলে ছয় মাস পর কেউ জানবে না কোনটা কার উপরে।

প্রবেশগম্যতা

কীবোর্ড, ফোকাস ও স্ক্রিন-রিডার

মাপা অবস্থা: যেখানে ভালো, যেখানে ফাঁকা

যা দরকার	মাপা	ফল
`aria-*` অ্যাট্রিবিউট	**০**	স্ক্রিন-রিডারকে কিছুই বলা হয় না
`<th scope="col">`	**০ / ৩৫ টেবিল**	টেবিল পড়ার সময় মাথা মেলে না
skip-link	**০**	Tab চেপে ১২+ মেনু পার হতে হয়
`:focus` নিয়ম	**১**	কেবল ফর্ম ঘরে; বাটনে নেই
`alt`	৫	২টিতে স্কুলের নাম, ৩টিতে খালি (পাশে লেখা আছে)
`<label for>`	৭ টেমপ্লেট	লেবেলে ক্লিক করলেই ঘর সক্রিয়

যা ভালো: সিমান্টিক HTML ব্যবহার হয়েছে — `<table>`, `<label>`, `<nav>`, আসল `<button>`। ফলে ভিত্তিটা ঠিক; শুধু উপরের স্তরটুকু বাকি।

```
/* সবচেয়ে ছোট উন্নতি — ৩ লাইন */
.btn-ims:focus-visible,
.ims-nav a:focus-visible{
  outline:2px solid var(--ims-primary);
  outline-offset:2px;
}
```

সৎ ফাঁক: কোনো অ্যাক্সেসিবিলিটি টেস্ট নেই, কোনো অডিট চালানো হয়নি — তাই এই সংখ্যাগুলোই একমাত্র প্রমাণ।

শিক্ষকের নোট: ল্যাপটপ বন্ধ করুন, শুধু Tab চেপে লগইন করুন — ছাত্ররা নিজেই দেখবে ফোকাস কোথায় আছে বোঝা যায় না। এই একটি অভিজ্ঞতাই পুরো স্লাইডের চেয়ে বেশি কাজ করে।

সততা

ডিজাইন সিস্টেমের পাঁচটি ফাঁক

ডকুমেন্টেশন নেই — জ্ঞান শুধু কোডে ও মানুষের মাথায়

ফাঁক	বাস্তব পরিণতি
ডকুমেন্টেশন নেই	নতুন ডেভেলপার `ims-*` তালিকা কোথায় পাবে জানে না — ৫৯ ক্লাস মুখস্থ করতে হয়
টাইপ/স্পেসিং টোকেন নেই	ফন্ট-আকার ৬টি জায়গায় আলাদা হার্ডকোডেড — "একই রকম" দেখতে হলেও মাপ আলাদা
z-index পরিকল্পনা নেই	স্টিকি টপবার ও মোডাল একই স্তরে — ভবিষ্যতে মোডাল টপবারের নিচে চলে যেতে পারে
ডার্ক মোড নেই	রঙ সবই হার্ডকোডেড হেক্স; `prefers-color-scheme` কোথাও নেই
কনট্রাস্ট লঙ্ঘন	সাইডবারের muted লেখা ৩.৭৫:১ — AA-এর নিচে (আগের স্লাইডে মাপা)

কেন এটি ব্যর্থতা নয়: ৭৫ টেমপ্লেট, ৫৯ ক্লাস, ঠিক ১৫টি টোকেন — এই একরূপতাটাই মূল অর্জন। ডকুমেন্টেশন থাকলে আরও ভালো হতো, কিন্তু না থাকলেও পর্দাগুলো একই রকম দেখায়।

সবচেয়ে সস্তা সমাধান: একটি মাত্র `DESIGN.md` — টোকেনের তালিকা, প্রতিটি ক্লাসের এক লাইন ব্যবহার, আর ছয়টি ফন্ট-আকারের অনুমোদিত মান। ৩০ মিনিটের কাজ।

সবচেয়ে ব্যয়বহুল ফাঁক: কনট্রাস্ট — এটি ব্যবহারকারীর চোখে সরাসরি বাজে, আর স্কুলের শিক্ষকরা দিনে ৬–৮ ঘণ্টা এই পর্দায় তাকিয়ে থাকেন।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন — "কোড লেখা শেষ মানে কাজ শেষ নয়; পরের মানুষ যেন বোঝে, সেটাও কাজের অংশ।" ডিজাইন সিস্টেমে ডকুমেন্টেশন বিলাসিতা নয়, তালা-চাবি।

সমাপ্তি

D5 শেষ — এবং পুরো কোর্স শেষ

১০ পর্ব · ৩৭ ক্লাস · ৫টি ডিপ ডেক · ১৭টি PDF

ডেক	স্লাইড	মূল শিক্ষা
D1	৩০	ডেটাবেস সিদ্ধান্ত — ৩০১ ফিল্ড, ৭০ FK
D2	২৮	স্ট্যাক সংযোগ — request থেকে DB
D3	২২	নিরাপত্তা — ৩৬ PermissionDenied, ০ raw SQL
D4	২০	জীবনচক্র — রিলিজ, ব্যাকআপ, হিসাব
D5	২০	ডিজাইন সিস্টেম — ১৫ টোকেন, ৫৯ ক্লাস

মোট: মূল কিট ৬২৯ স্লাইড + ডিপ ১২০ = **৭৪৯ স্লাইড**, ১৭টি PDF।

শেষ বাক্য যা সব ডেকে ফিরে আসে: প্রমাণ ছাড়া কোনো দাবি নয়। ১,০০৭ ব্যবহার, ৩.৭৫ কনট্রাস্ট, ০ aria — এগুলো মাপা, অনুমান নয়।

পরের ধাপ (শিক্ষার্থীদের জন্য): ডার্ক মোড নয় — আগে ডকুমেন্টেশন, তারপর কনট্রাস্ট ঠিক, তারপর মোবাইল নেভিগেশন।

প্রকল্প চালাতে: `bash run.sh` · লগইন `@ims.local` / `DemoPass!2026` · `bash run.sh --reset` ডেটাবেস নতুন করে।

শিক্ষার্থীকে দেওয়ার তিনটি কাজ: ① `DESIGN.md` লিখুন (টোকেন + ৫৯ ক্লাসের তালিকা) · ② সাইডবারের muted রঙ ৪.৫:১-এ তুলুন · ③ ৮২০px-এ হ্যামবার্গার মেনু যোগ করুন। তিনটি মিলিয়ে একদিনের কাজ।

শিক্ষকের নোট: কোর্স শেষে একটি প্রশ্ন করুন: "আপনার নিজের প্রজেক্টে কোন একটি ফাঁক আপনি প্রথমে ঠিক করবেন, কেন?" — উত্তরেই বোঝা যাবে কে সত্যিই শিখেছে।